

Lab #1 - Geometry (part 1)

Informática Gráfica

Adolfo Muñoz - Julio Marco

Pablo Luesia - J. Daniel Subías – Óscar Pueyo



Before we begin...

- **Five** groups:
 - Wednesday 15-17h (L0.04)
 - Wednesday 18-20h (L0.02)
 - Thursday 15-17h (L0.03)
 - Friday 18-20h (L0.03)
- Everything should be done in **pairs**.
 - Individual → OK, but be careful with the workload.
 - Three or more → Nope.

Before we begin...

- Practical sessions:
 - Intermediate assignments: no submission required

Before we begin...

- Practical sessions:
 - Intermediate assignments: no submission required
 - **Highly recommended** to be completed at certain tentative deadlines
 - For the first and second sessions: **September 25th**

Before we begin...

- Practical sessions:
 - Intermediate assignments: no submission required
 - **Highly recommended** to be completed at certain tentative deadlines
 - For the first and second sessions: **September 25th**
 - Your final work will build upon the stuff you'll do here!
 - **80%** of the final grade (including written report)

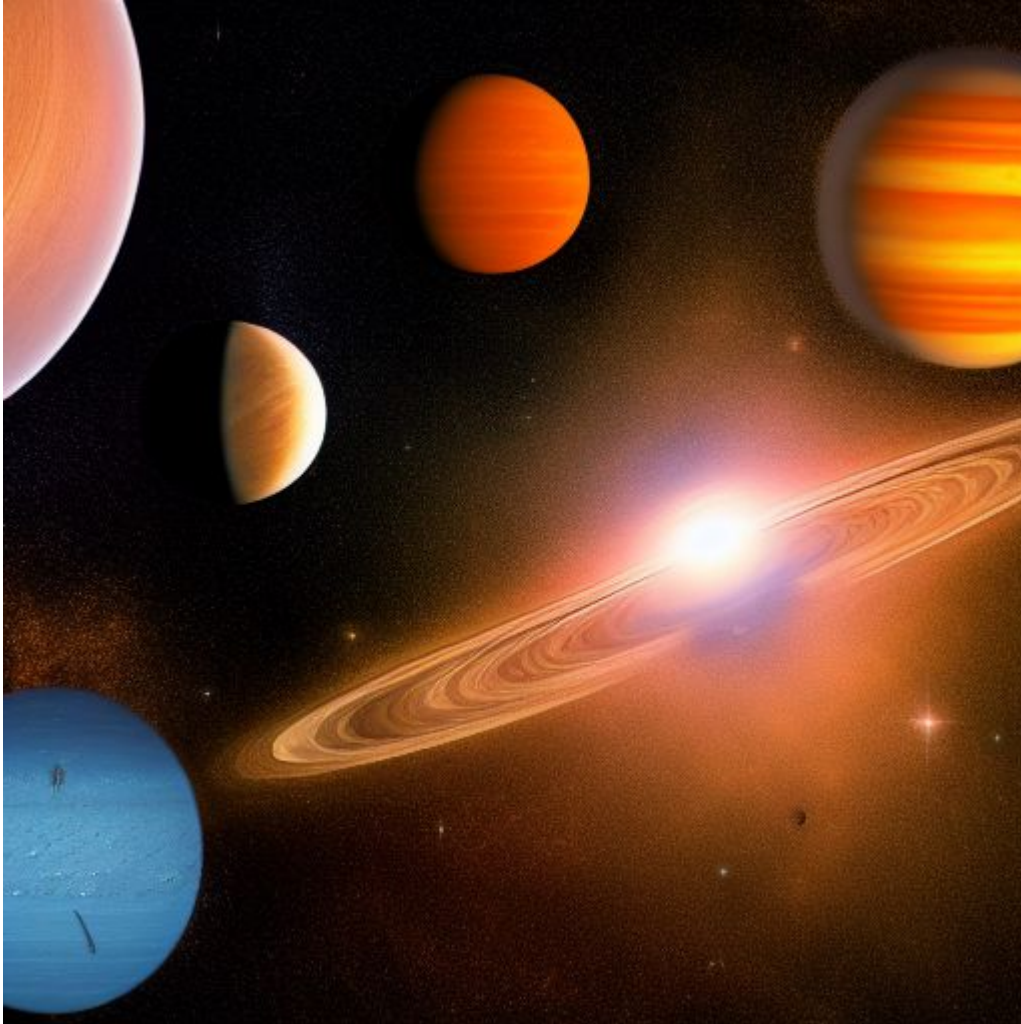
Lab #1 - Geometry (part 1)

Informática Gráfica

Adolfo Muñoz - Julio Marco - Pablo Luesia - Daniel Subías – Óscar Pueyo

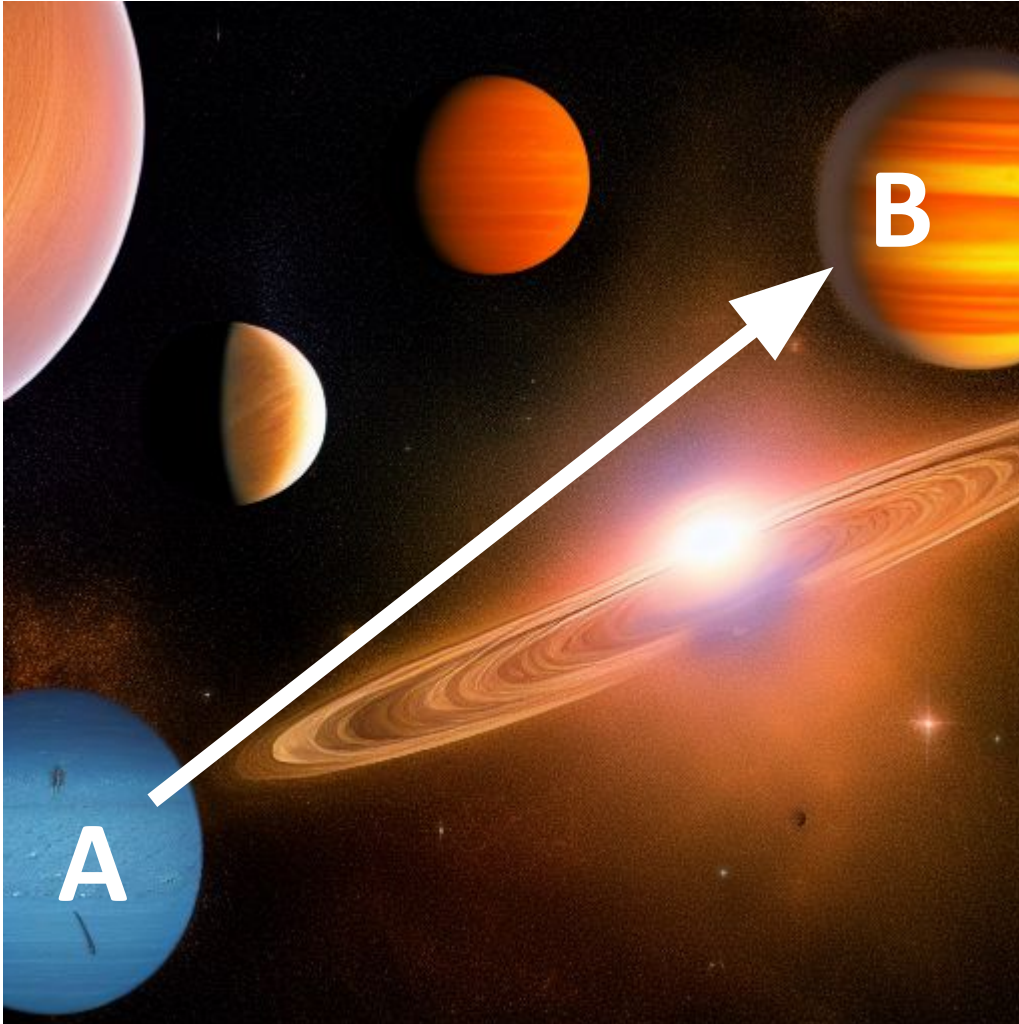


Your new job at FTL dynamics

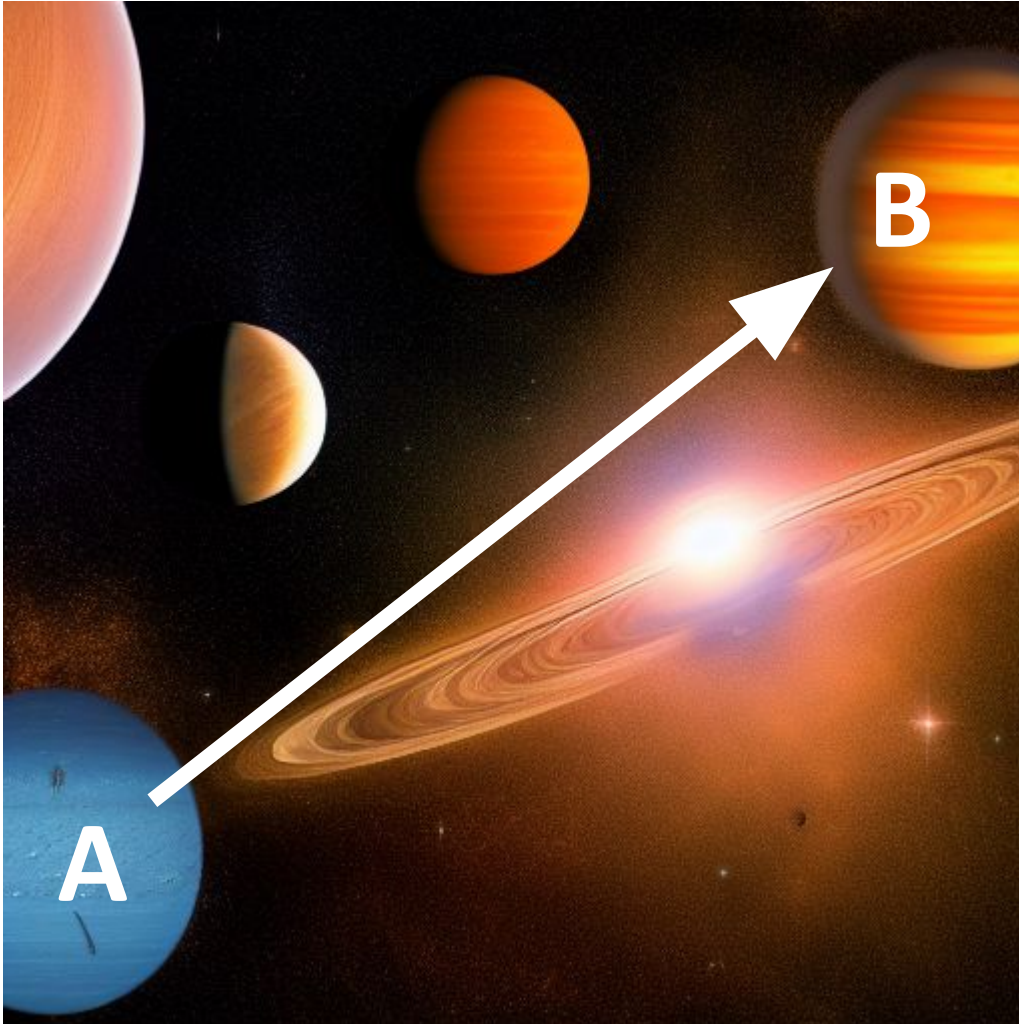


Images generated using Stable Diffusion

Your new job at FTL dynamics

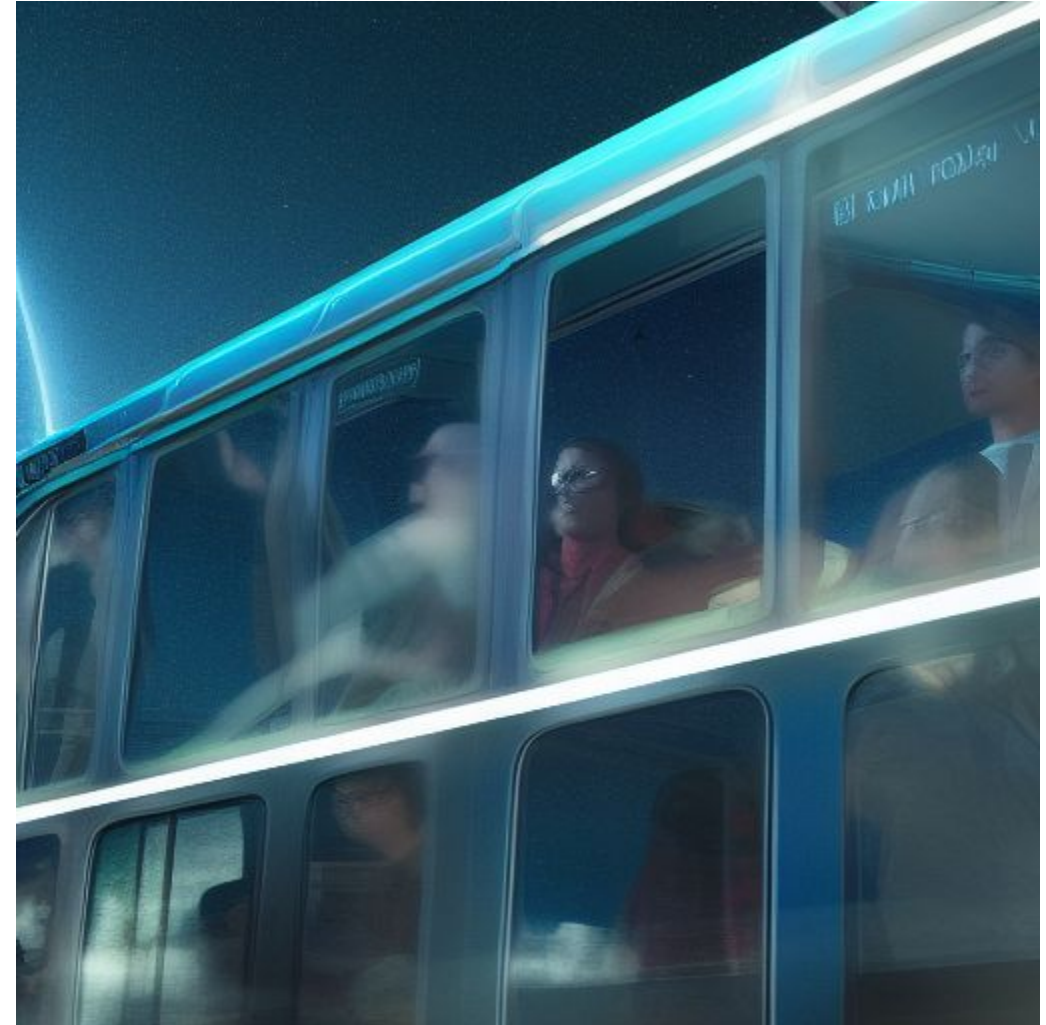
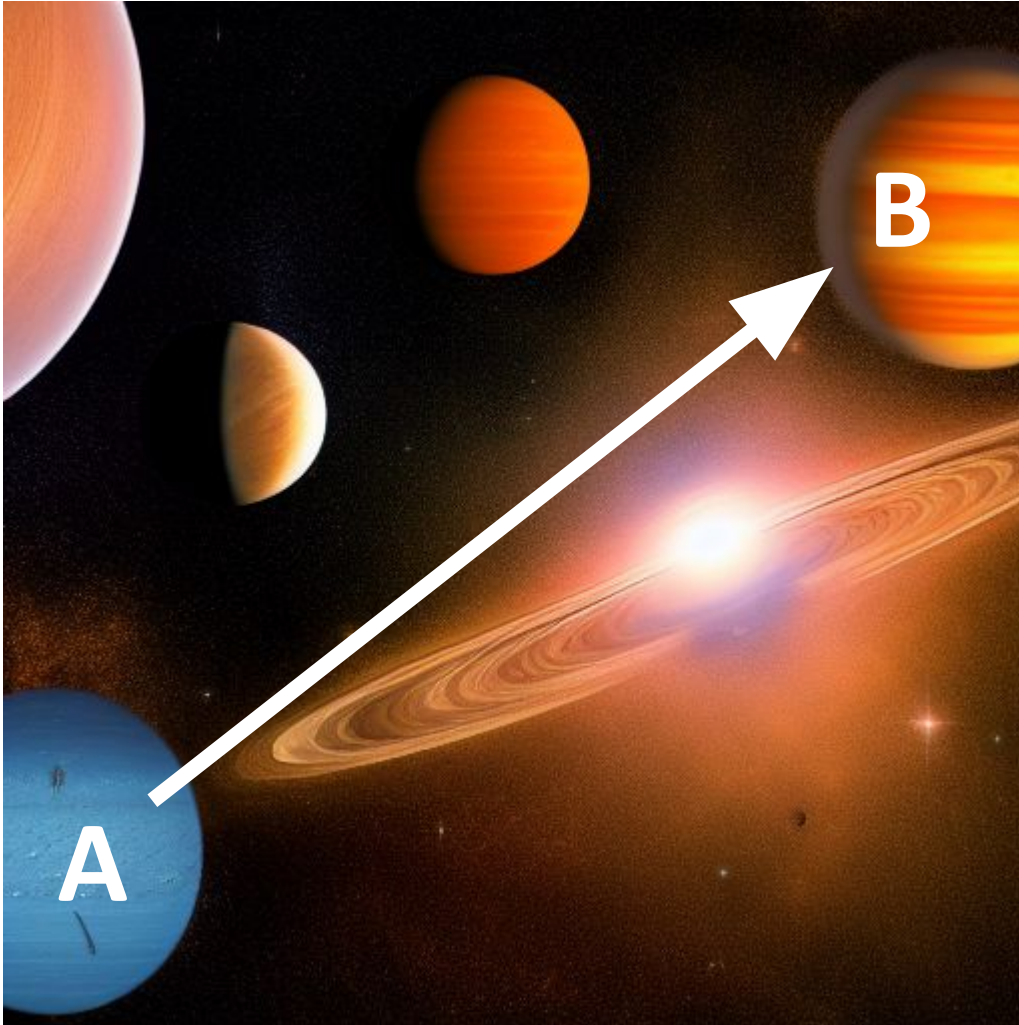


Your new job at FTL dynamics



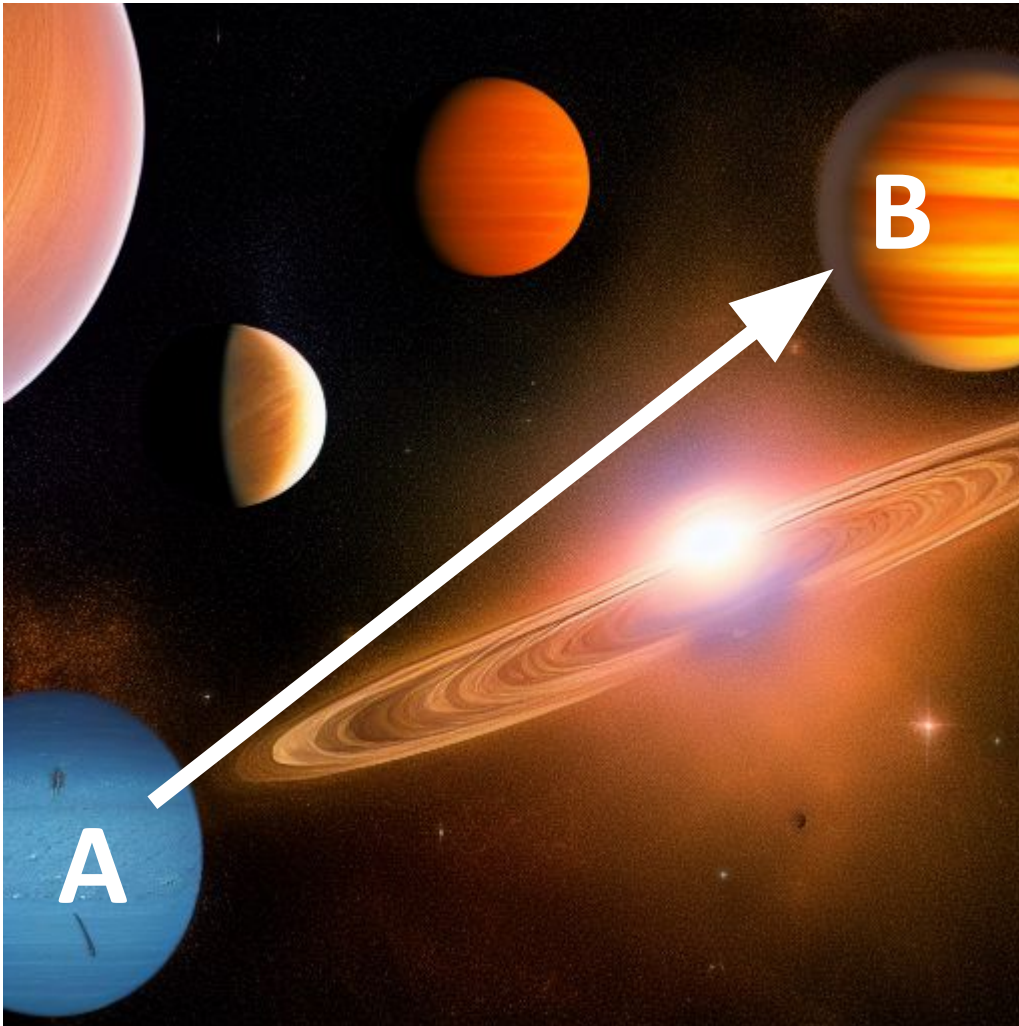
Images generated using Stable Diffusion

Your new job at FTL dynamics

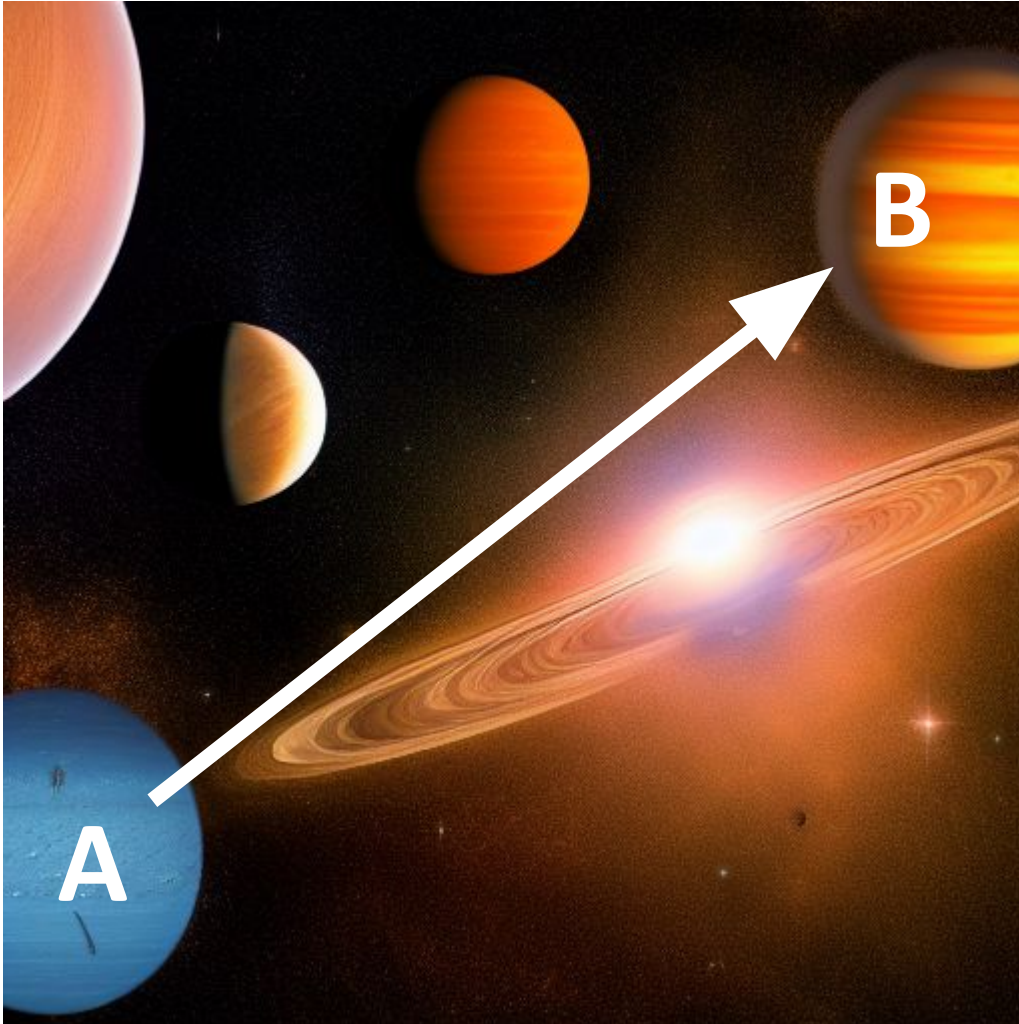


Images generated using Stable Diffusion

Your new job at FTL dynamics

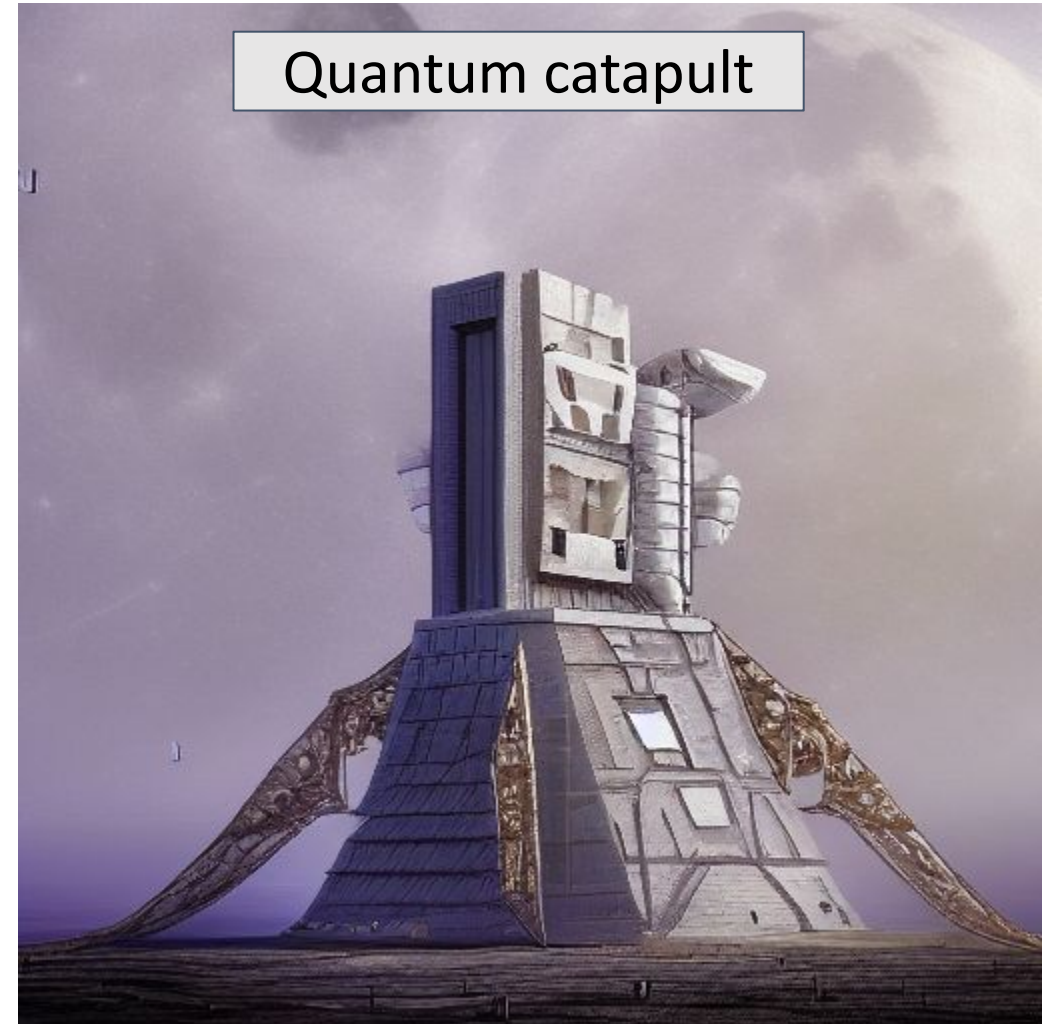
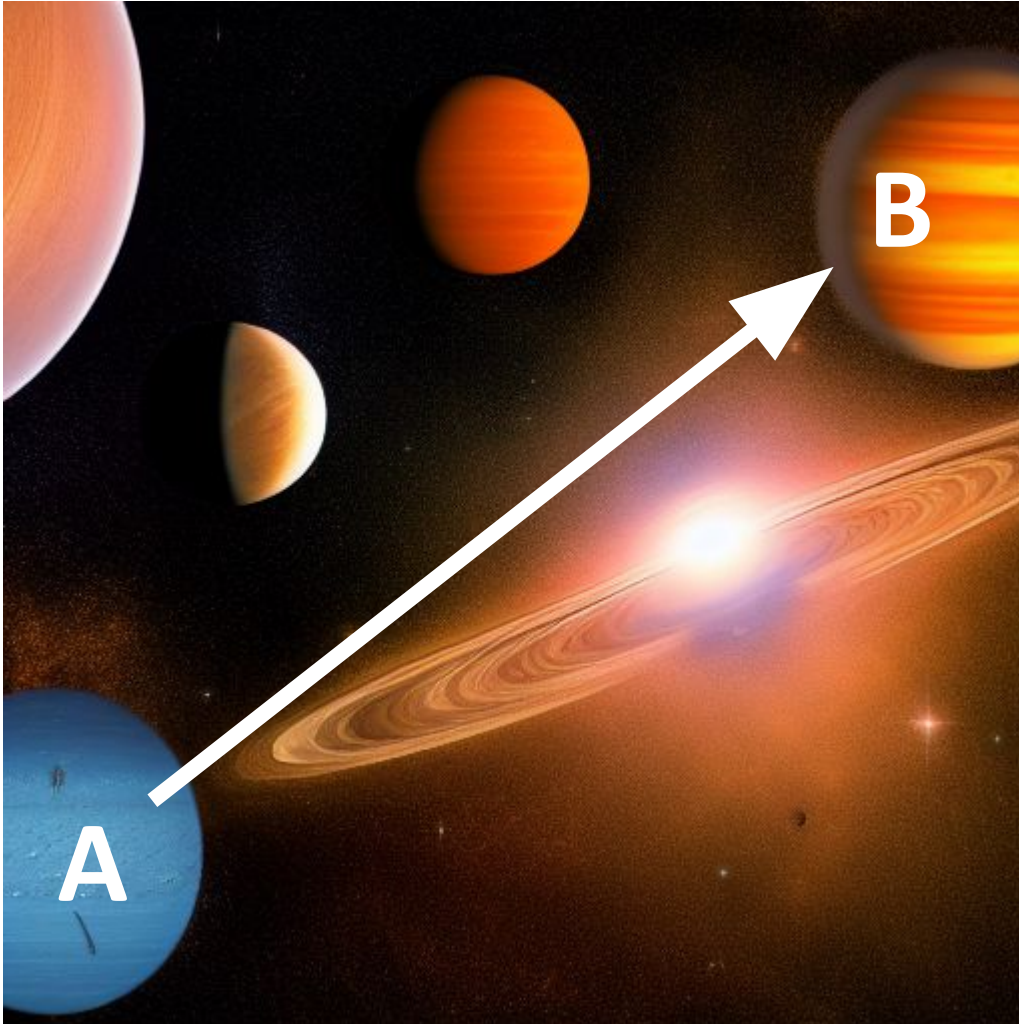


Your new job at FTL dynamics

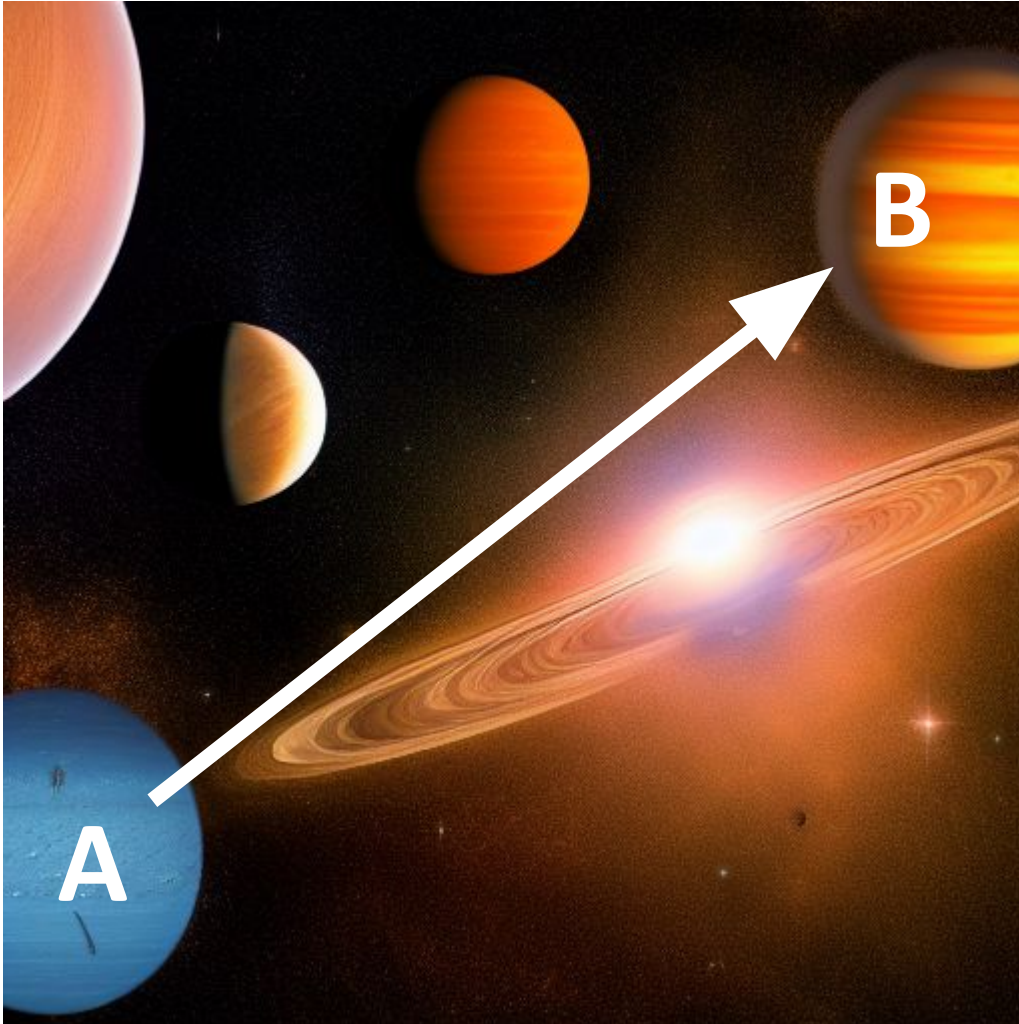


Images generated using Stable Diffusion

Your new job at FTL dynamics

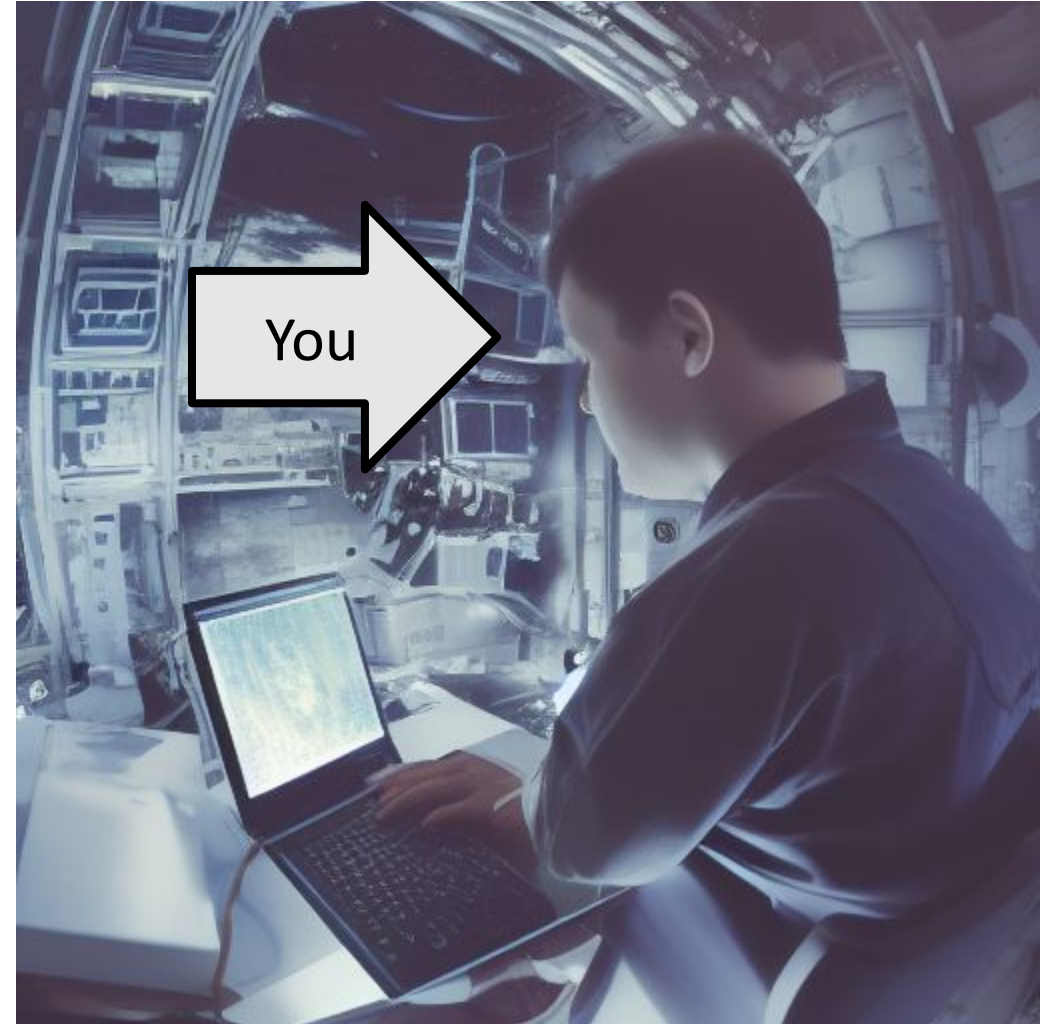
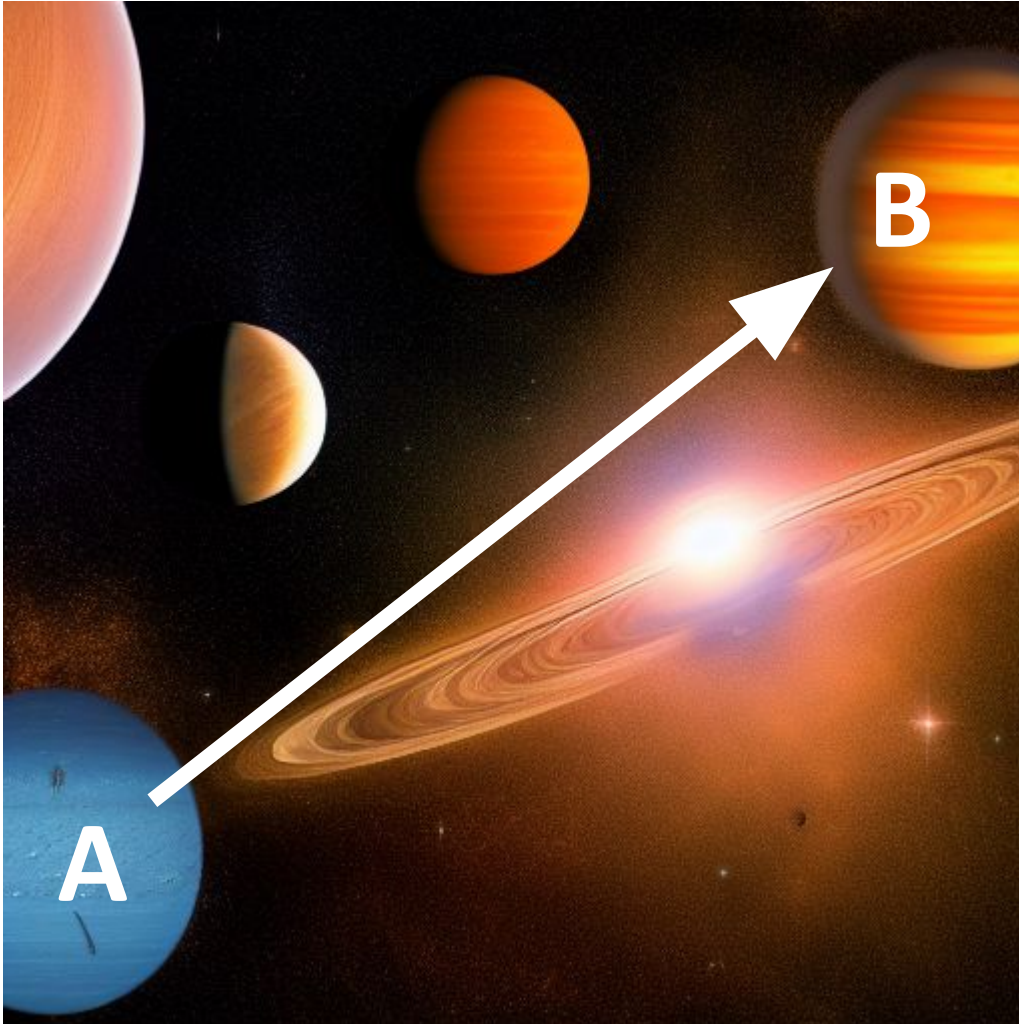


Your new job at FTL dynamics



Images generated using Stable Diffusion

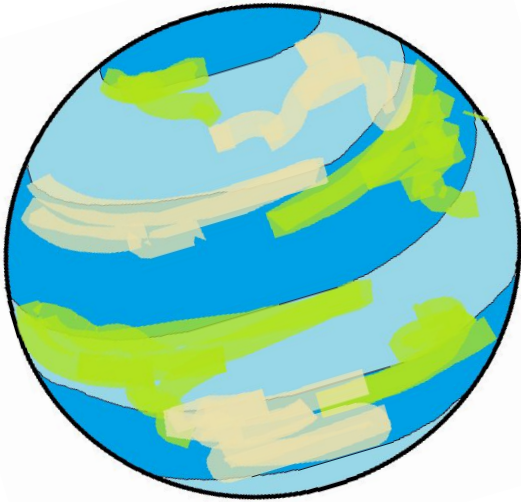
Your new job at FTL dynamics



Images generated using Stable Diffusion

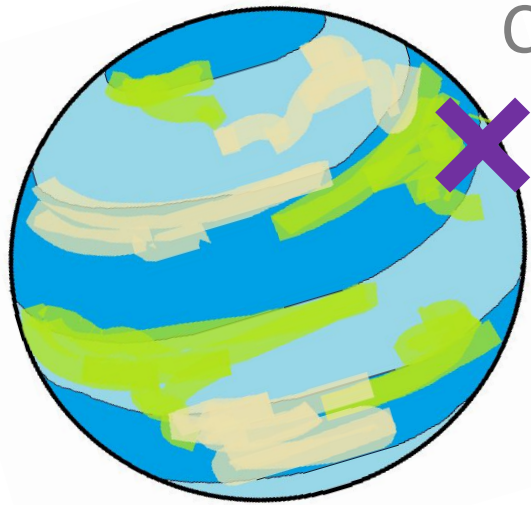
Problem statement

Ideal scenario



Problem statement

Ideal scenario

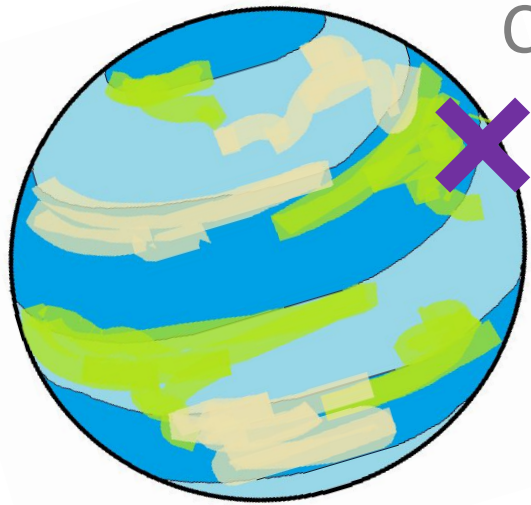


City launcher



Problem statement

Ideal scenario



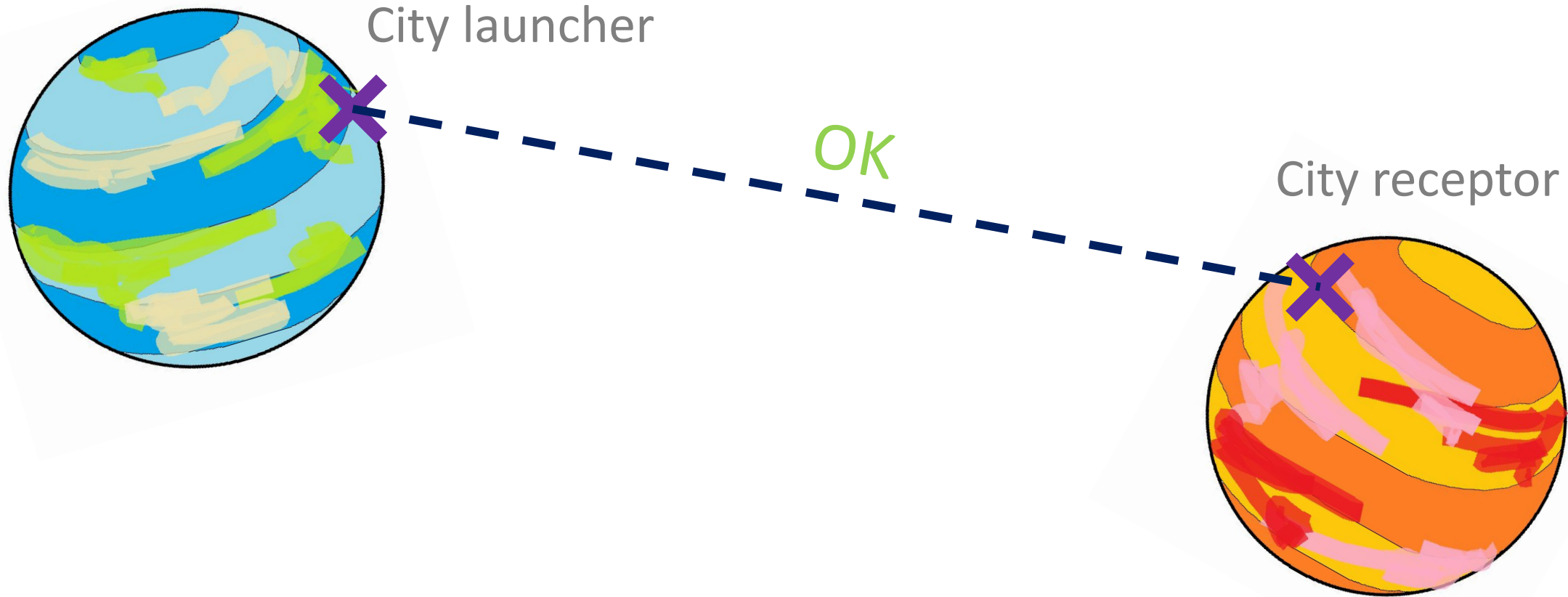
City launcher

City receptor



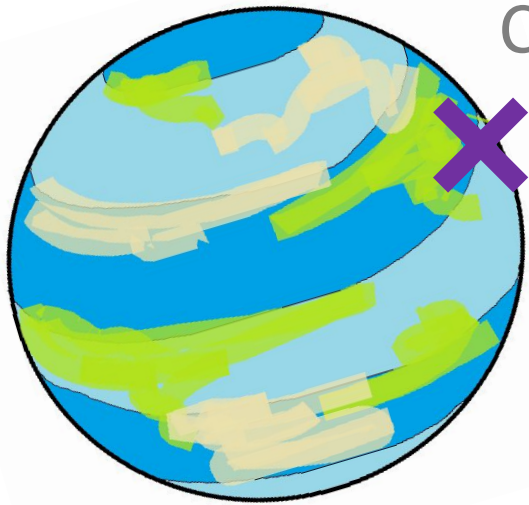
Problem statement

Ideal scenario



Problem statement

Fatal scenario (1)



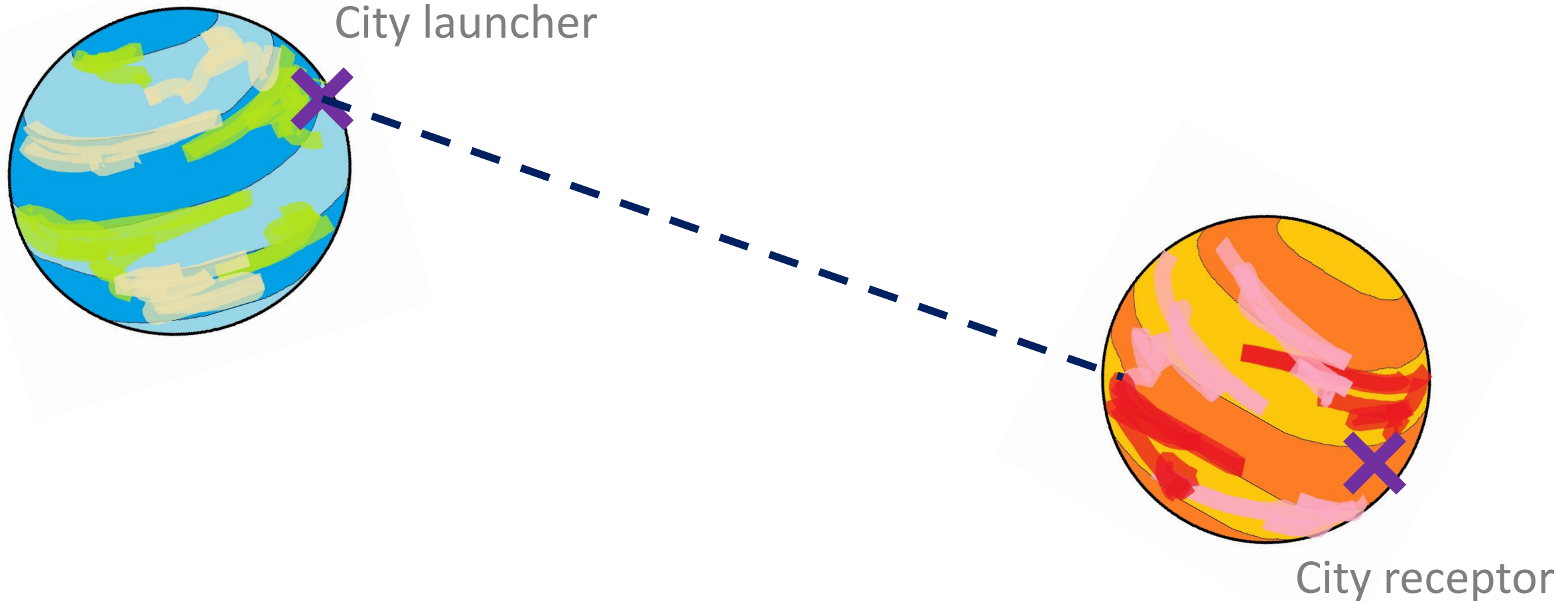
City launcher



City receptor

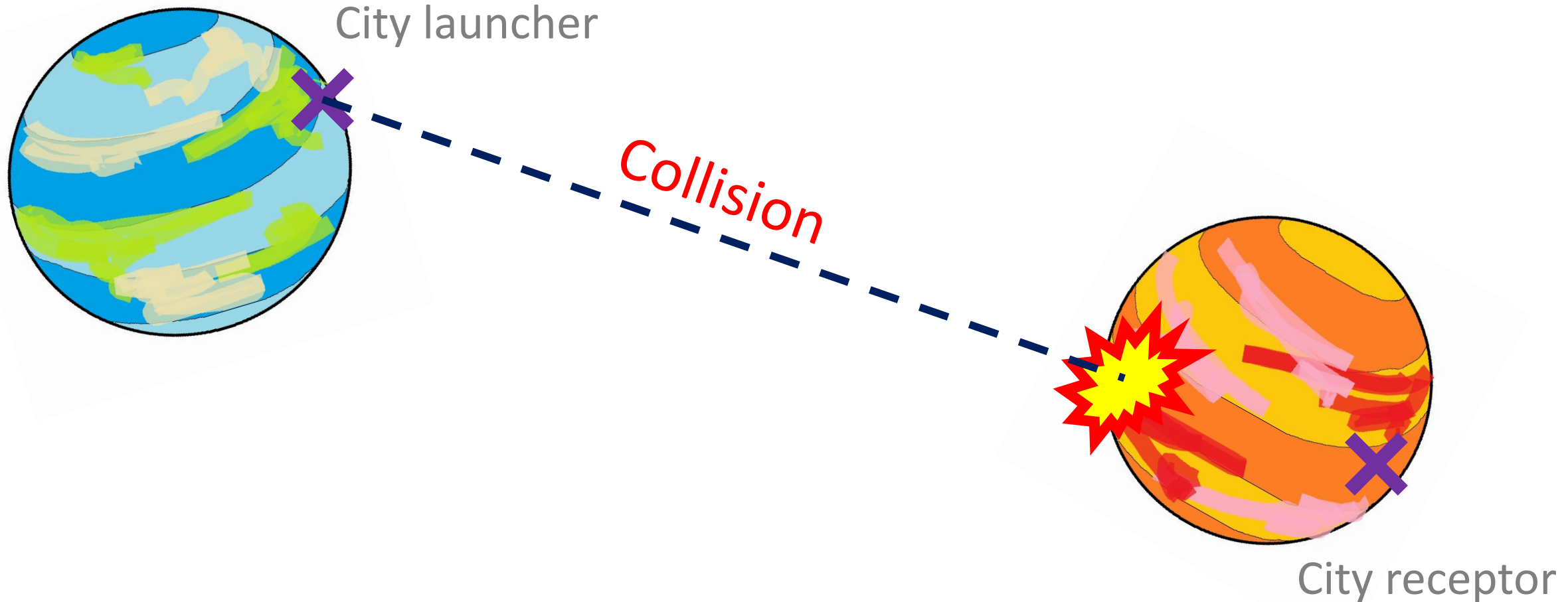
Problem statement

Fatal scenario (1)



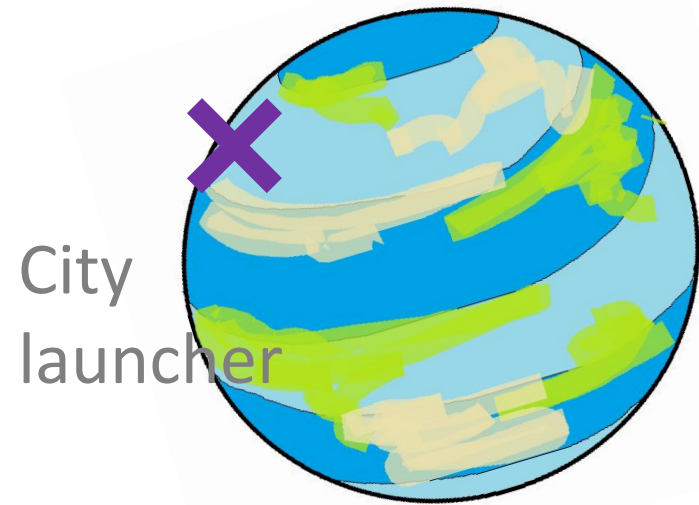
Problem statement

Fatal scenario (1)



Problem statement

Fatal scenario (2)



City receptor



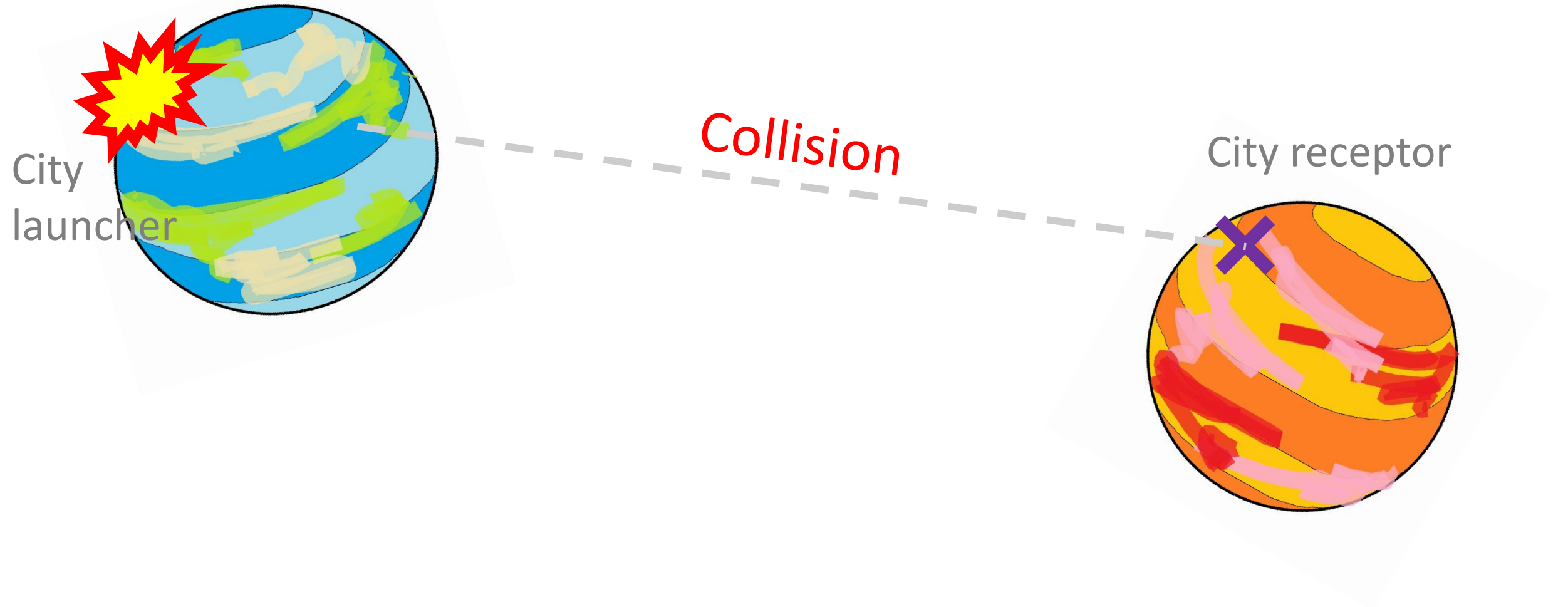
Problem statement

Fatal scenario (2)



Problem statement

Fatal scenario (2)



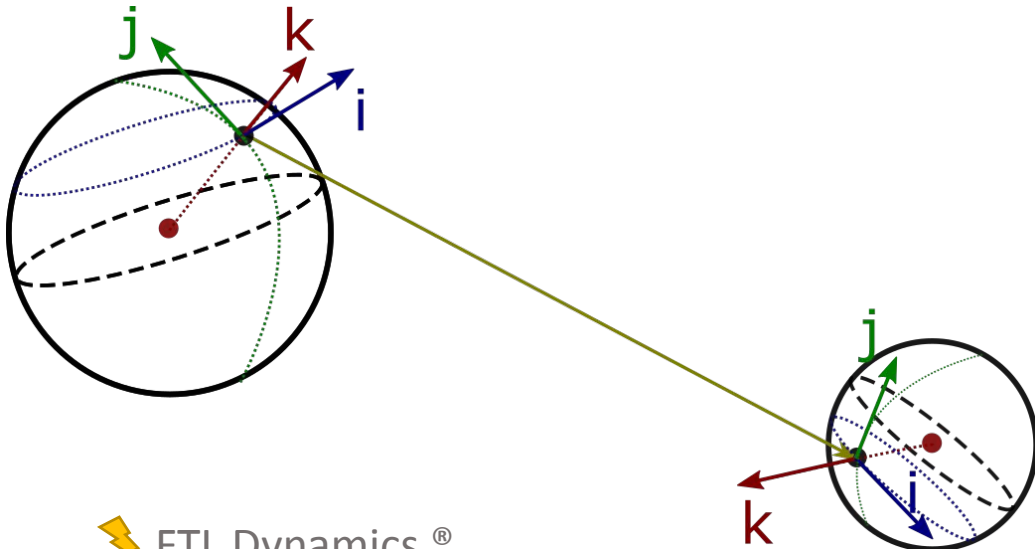
Why do all this?

To apply knowledge about geometry: points, directions, vectorial operations...

Why do all this?

To apply knowledge about geometry: points, directions, vectorial operations...

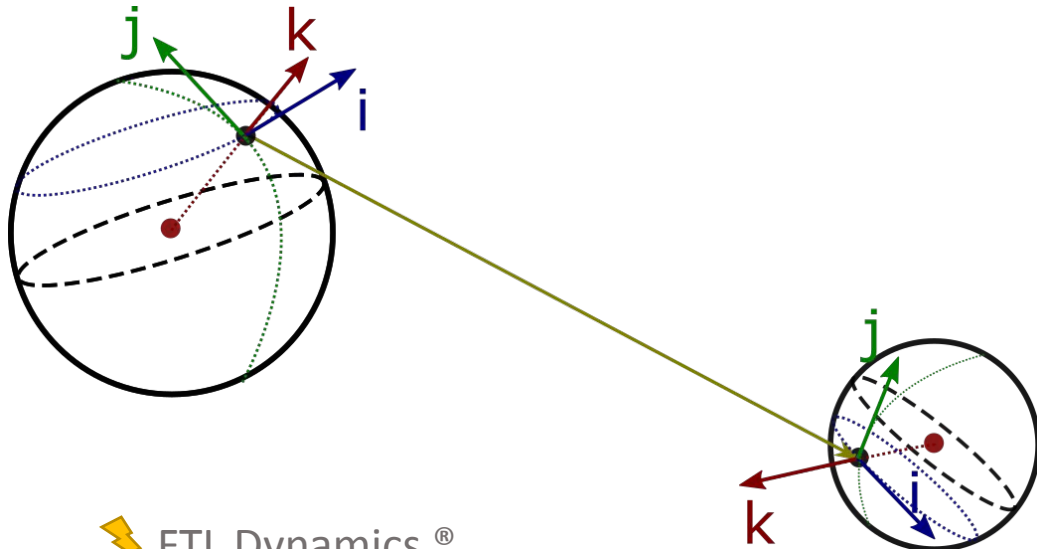
- In the short term (sesiones 1+2)



Why do all this?

To apply knowledge about geometry: points, directions, vectorial operations...

- In the short term (sesiones 1+2)

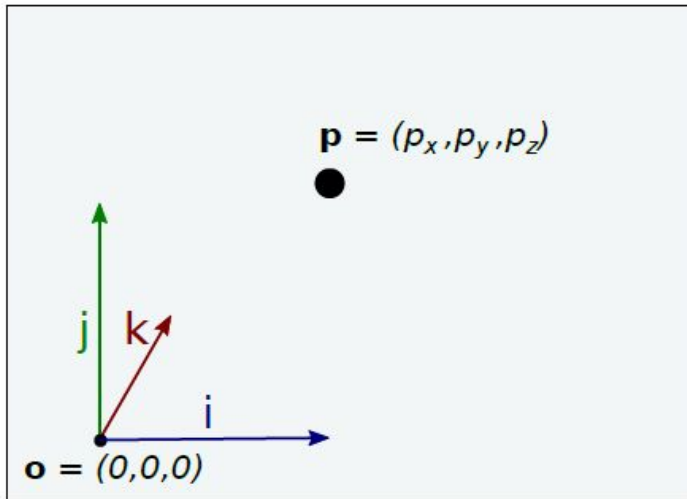


- In the long term



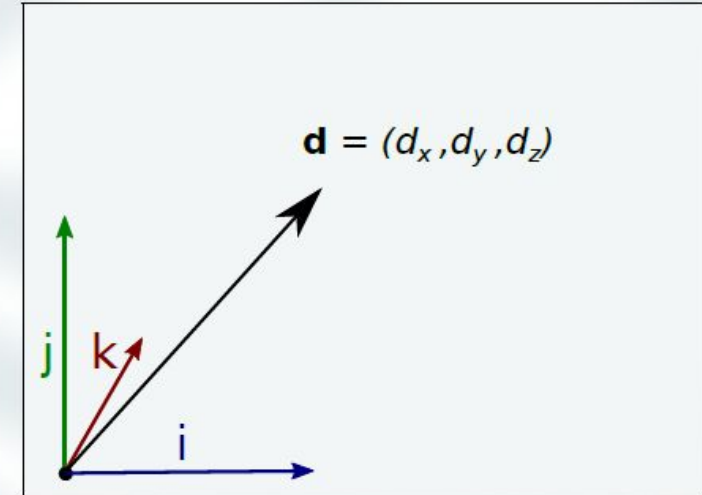
- Data basics

Point



$$\mathbf{p} = \mathbf{o} + p_x \mathbf{i} + p_y \mathbf{j} + p_z \mathbf{k}$$

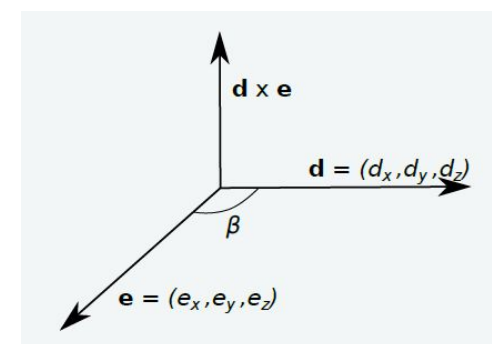
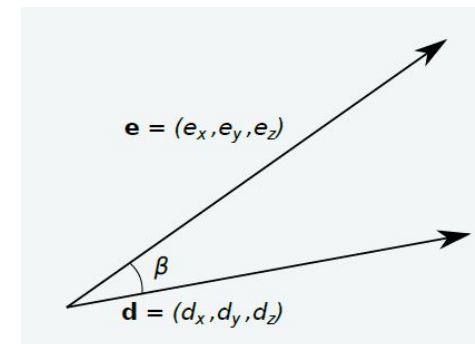
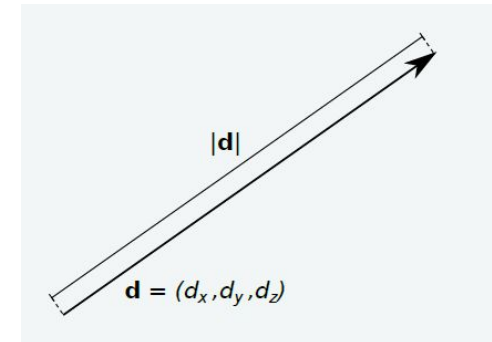
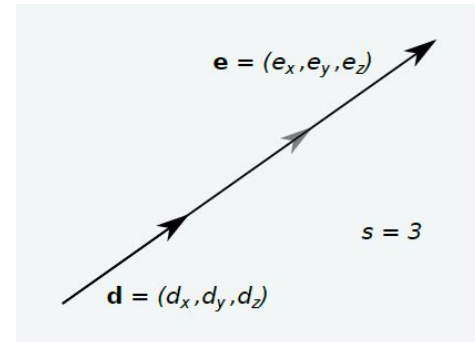
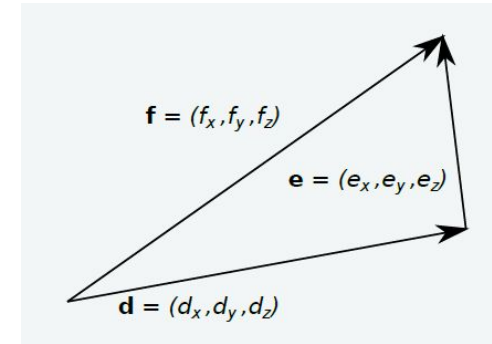
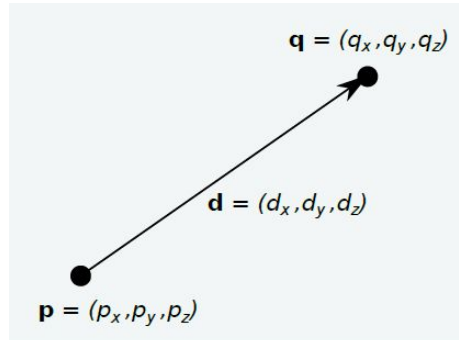
Direction



$$\mathbf{d} = d_x \mathbf{i} + d_y \mathbf{j} + d_z \mathbf{k}$$

Basics

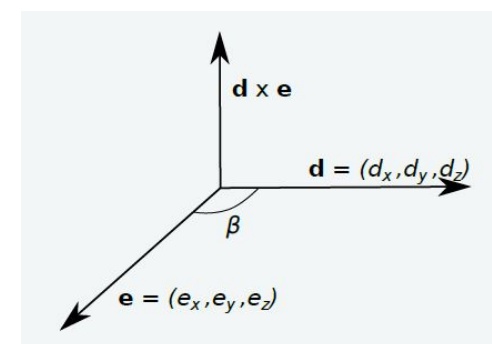
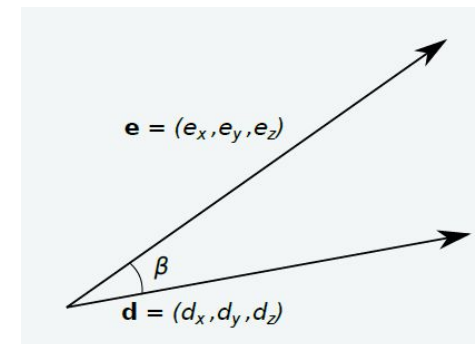
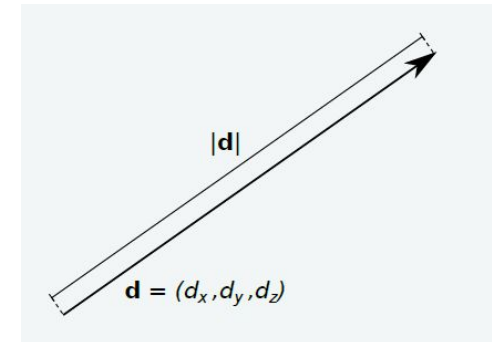
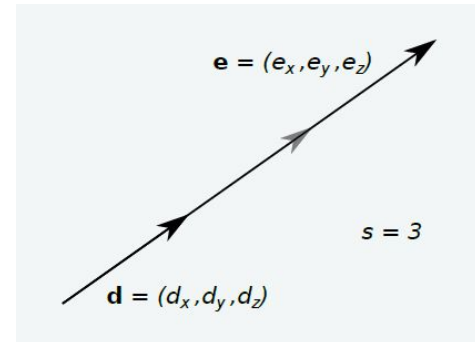
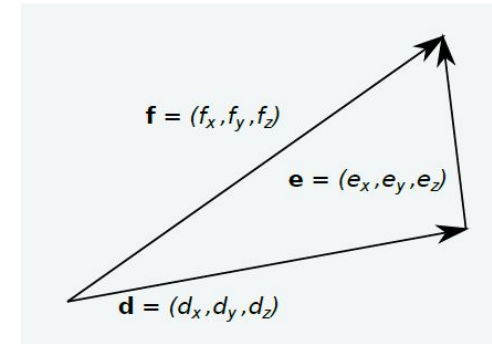
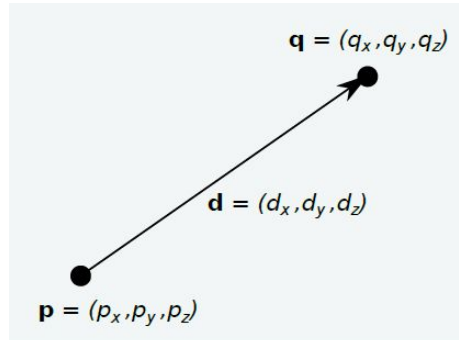
- Data basics
- Operations
 - Addition, subtraction
 - Scalar multiplication/division
 - Modulus, normalization
 - Dot and cross products



Basics

- Data basics
- Operations
 - Addition, subtraction
 - Scalar multiplication/division
 - Modulus, normalization
 - Dot and cross products
 - Pretty stdout operator

```
std::cout << point << std::endl;  
[5.0, 2.3, 1.2]
```



What to expect from this session

In the programming language of your choice, implement:

- **Data basics:** 3D points and directions
- **Operations:**
 - Addition, subtraction, scalar multiplication and scalar division
 - Modulus, normalization
 - Dot and cross products, pretty stdout operator
- Test your implementation with several examples
- Do you have **extra time**?
 - Matrices
 - Homogeneous coordinates. 3x3 matrices or 4x4?
 - Translation, rotation, change of scale, inverse transform. Combinations.

Remember: Programming advice

1. Be effective, **do not overdesign**.

Question:

Do you need separate data types for points and directions (and RGB tuples)?

Remember: Programming advice

1. Be effective, **do not overdesign**.

Question:

Do you need separate data types for points and directions (and RGB tuples)?

Answer:

Entirely up to you.

Pros:

- Specific behavior
- Compile time checks

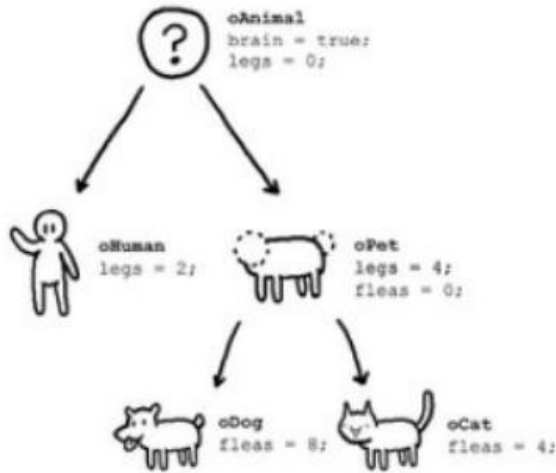
Cons:

- Lots of common behavior
- Extra code

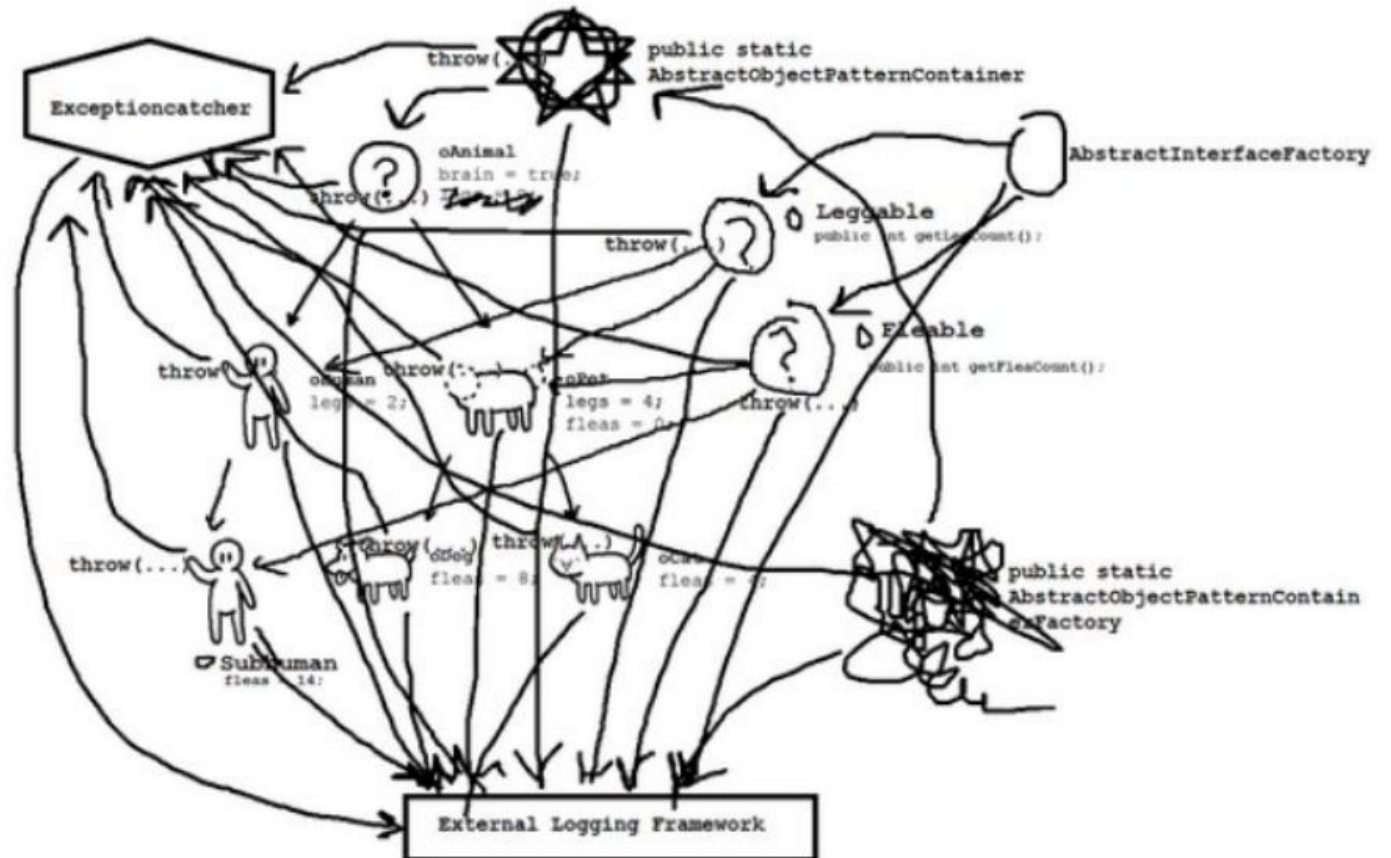
Remember: Programming advice

1. Be effective, do not overdesign.

What OOP users claim



What actually happens



Remember: Programming advice

2. Prefer **functional** over state machine behavior

```
1  //Very wrong  
2  Vec3 v3 = v1;  
3  v3.multiplyBy(v2);
```


Remember: Programming advice

2. Prefer **functional** over state machine behavior

```
1  //Very wrong  
2  Vec3 v3 = v1;  
3  v3.multiplyBy(v2);  
4  //Better  
5  Vec3 v3 = v1.multiply(v2);  
6  //or  
7  Vec3 v3 = multiply(v1, v2);
```

Remember: Programming advice

3. Prefer **operators** over long function / method names.

1 *//Wrong*

2 **Vec3 v3 = multiply(v1, v2);**

Remember: Programming advice

3. Prefer **operators** over long function / method names.

1 *//Wrong*

2 **Vec3 v3 = multiply(v1, v2);**

3 *//Better*

4 **Vec3 v3 = v1*v2;**

5 *//In the long run*

6 **Vec w = 2*(cross(u, v) - n)*a;**

Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

Java

Right or wrong?

```
1 class Point{  
2     float [] c;  
3     public Point(float x, float y, float z){  
4         c = new float [3];  
5         c[0] = x; c[1] = y; c[2] = z;  
6     }  
7 };
```

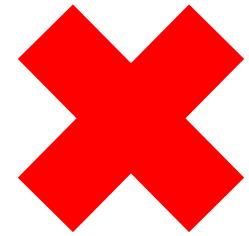
Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

Java

Right or wrong?

```
1 class Point{  
2     float [] c;  
3     public Point(float x, float y, float z){  
4         c = new float [3];  
5         c[0] = x; c[1] = y; c[2] = z;  
6     }  
7 };
```



Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

Java

Right or wrong?

```
1 class Point{  
2     float cx, cy, cz;  
3     public Point(float x, float y, float z){  
4         cx = x; cy = y; cz = z;  
5     }  
6 };
```

Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

Java

Right or wrong?

```
1 class Point{  
2     float cx, cy, cz;  
3     public Point(float x, float y, float z){  
4         cx = x; cy = y; cz = z;  
5     }  
6 };
```

OK

Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

C++

Right or wrong?

```
1 class Point{  
2     std::vector<float> c;  
3     public Point(float x, float y, float z){  
4         c[0] = x; c[1] = y; c[2] = z;  
5     }  
6 };
```

Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

C++

Right or wrong?

```
1 class Point{  
2     std::vector<float> c;  
3     public Point(float x, float y, float z){  
4         c[0] = x; c[1] = y; c[2] = z;  
5     }  
6 };
```



Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

C++

Right or wrong?

```
1 class Point{  
2     float c[3]; // or std::array<float,3> c  
3     public Point(float x, float y, float z){  
4         c[0] = x; c[1] = y; c[2] = z;  
5     }  
6 };
```


Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

C++

Right or wrong?

OK

```
1 class Point{
2     float c[3]; // or std::array<float,3> c
3     public Point(float x, float y, float z){
4         c[0] = x; c[1] = y; c[2] = z;
5     }
6 };
```

Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

Question:

Which data types represent the three coordinates?

Remember: Programming advice

4. Avoid memory management. It is not your battle (this time)

Question:

Which data types represent the three coordinates?

Answer:

Anything that avoids memory creation / destruction.

Even if you need C++ pointers:

```
1    //This deletes itself
2    std::shared_ptr<Object> o =
3        std::make_shared<Sphere>(center, radius);
4    //This doesn't
5    Object* o = new Sphere(center, radius);
```

Remember: Programming advice

1. Be effective, **do not overdesign**.
2. Prefer **functional** over state machine behavior.
3. Prefer **operators** over long function / method names.
4. **Avoid memory management**. It is not your battle (this time).
5. Premature optimization is the root of all devil (Donald Knuth).
6. Choose the **right programming** language for you
7. **Enjoy** visualizing your results.

They're better than terminal output or a boring interface.
Funnier when they're wrong, beautiful when they're right.

Remember: Programming advice

1. Be effective, **do not overdesign**.
2. Prefer **functional** over state machine behavior.
3. Prefer **operators** over long function / method names.
4. **Avoid memory management**. It is not your battle (this time).
5. Premature optimization is the root of all devil (Donald Knuth).
6. Choose the **right programming** language for you
7. **Enjoy** visualizing your results.

They're better than terminal output or a boring interface.
Funnier when they're wrong, beautiful when they're right.

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice, implement:

- **Data basics:** 3D points and directions
- **Operations:**
 - Addition, subtraction, scalar multiplication and scalar division
 - Modulus, normalization
 - Dot and cross products, pretty stdout operator
- Test your implementation with several examples
- Do you have **extra time**?
 - Matrices
 - Homogeneous coordinates. 3x3 matrices or 4x4?
 - Translation, rotation, change of scale, inverse transform. Combinations.
- **Recommended deadline (sesiones 1 + 2): October 25th**

Lab #1 - Geometry (part 2)

Informática Gráfica

Adolfo Muñoz - Julio Marco

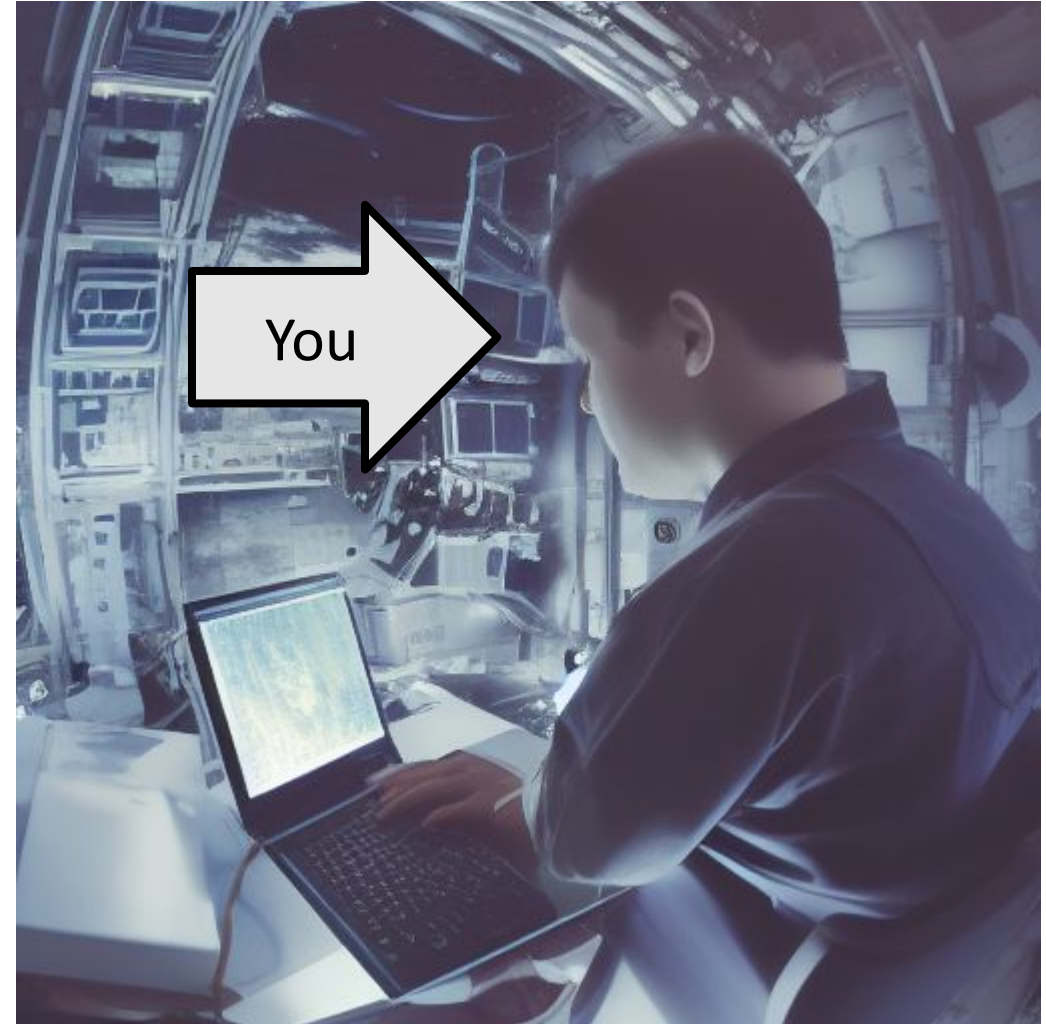
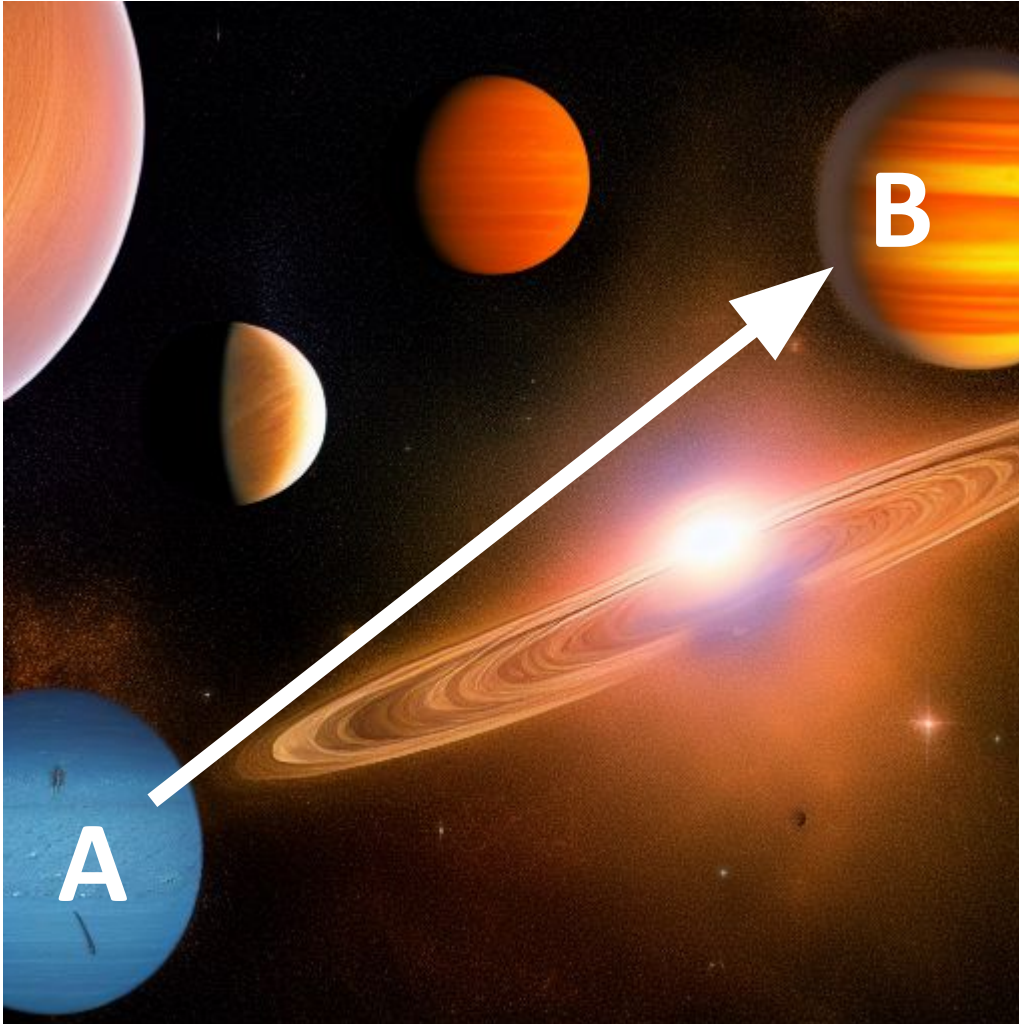
Pablo Luesia - J. Daniel Subías – Óscar Pueyo



Before we begin...

- Practical sessions:
 - Intermediate assignments: no submission required
 - **Highly recommended** to be completed at certain tentative deadlines
 - For the first and second sessions: **September 25th**
 - Your final work will build upon the stuff you'll do here!
 - **80%** of the final grade (including written report)
- If you need to find a partner for lab sessions, message us

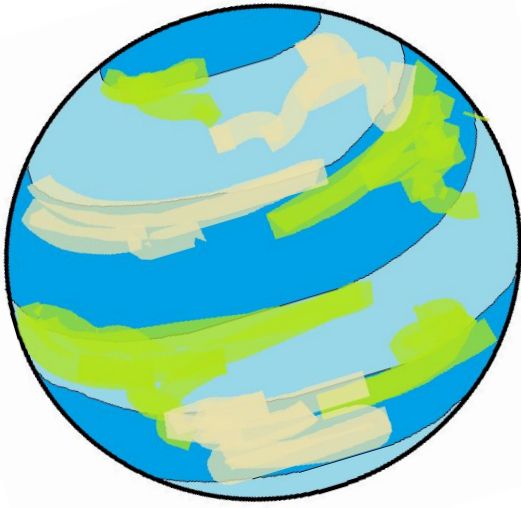
Your new job at FTL dynamics



Images generated using Stable Diffusion

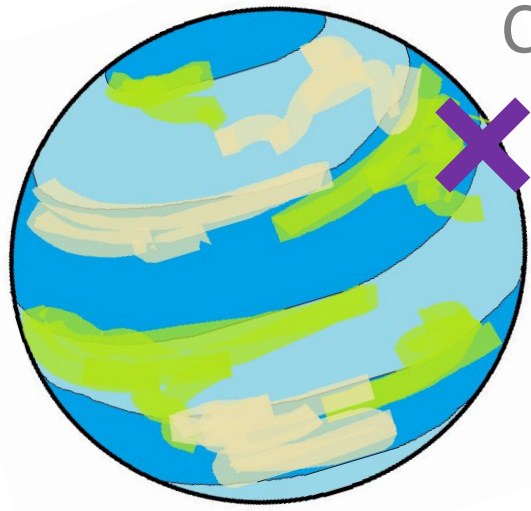
Problem statement

Ideal scenario



Problem statement

Ideal scenario

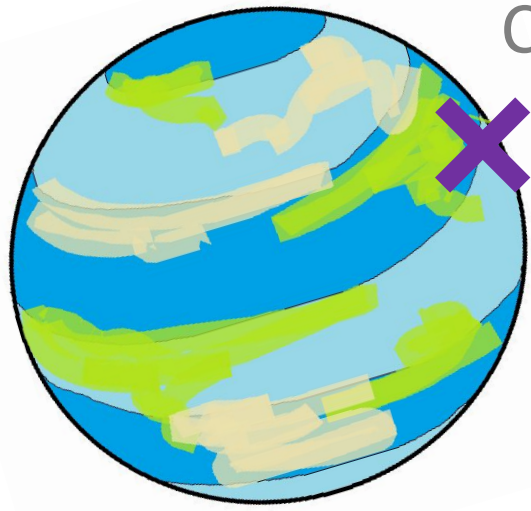


City launcher



Problem statement

Ideal scenario



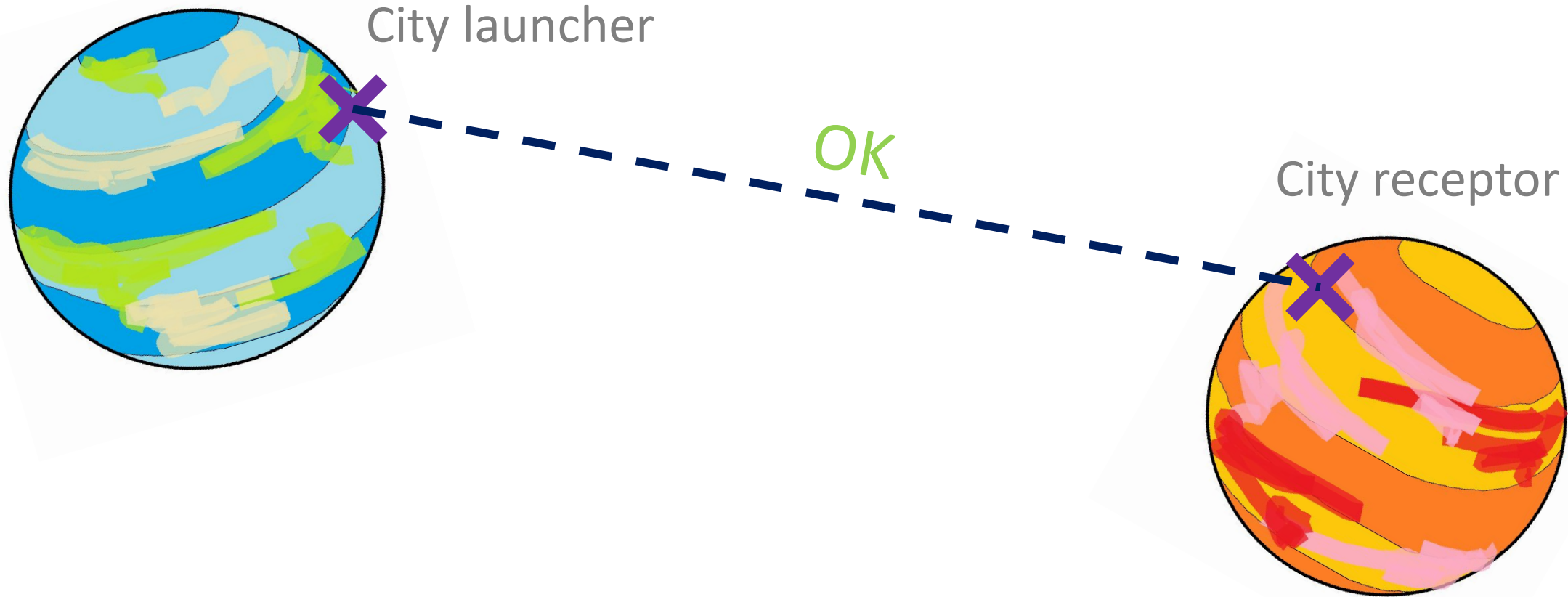
City launcher

City receptor



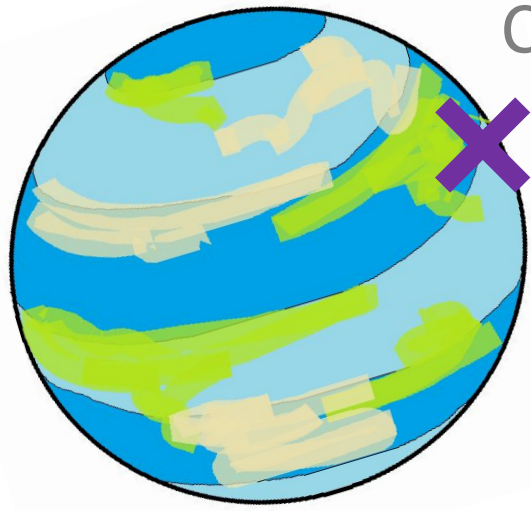
Problem statement

Ideal scenario



Problem statement

Fatal scenario (1)



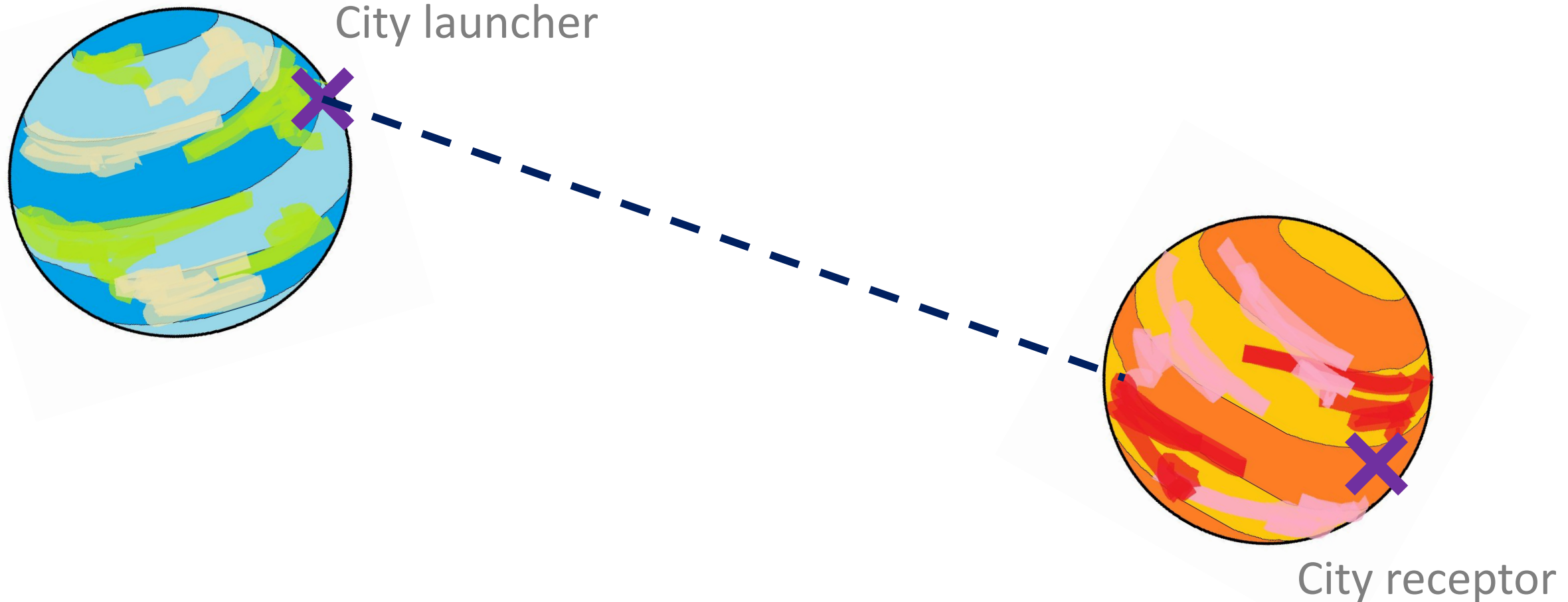
City launcher



City receptor

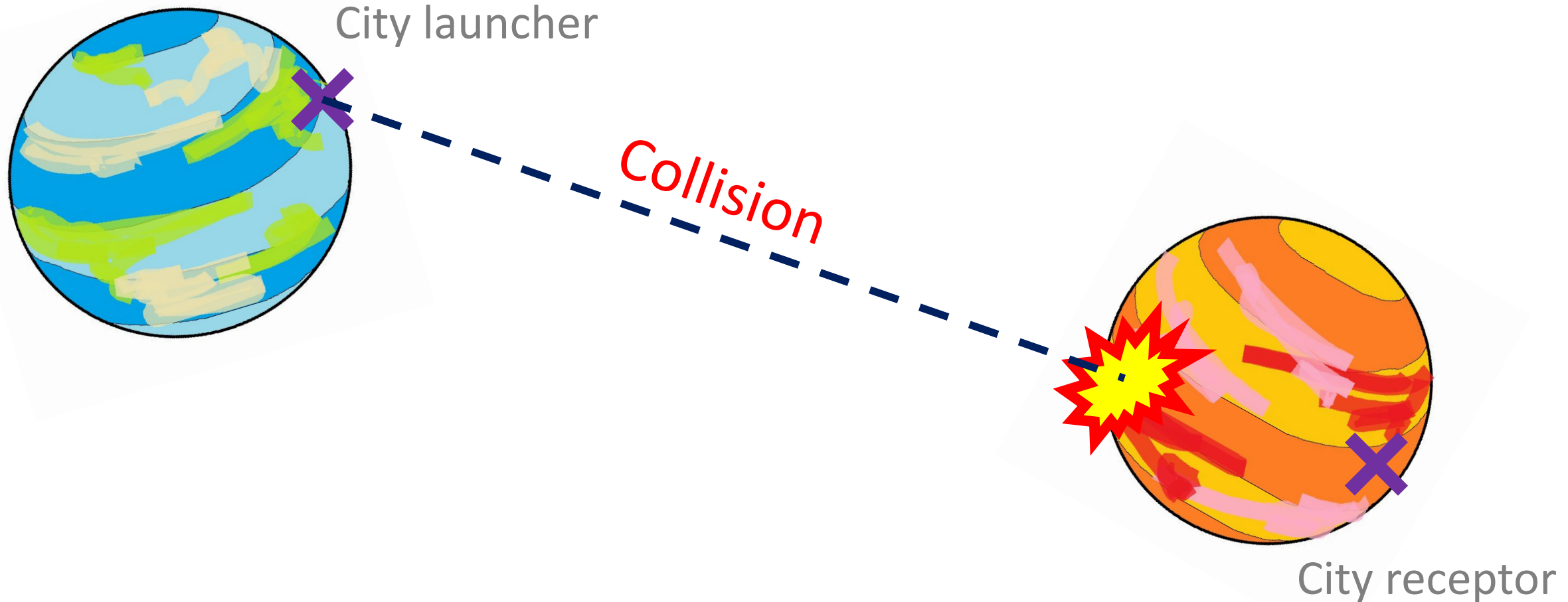
Problem statement

Fatal scenario (1)



Problem statement

Fatal scenario (1)



Problem statement

Fatal scenario (2)



City receptor



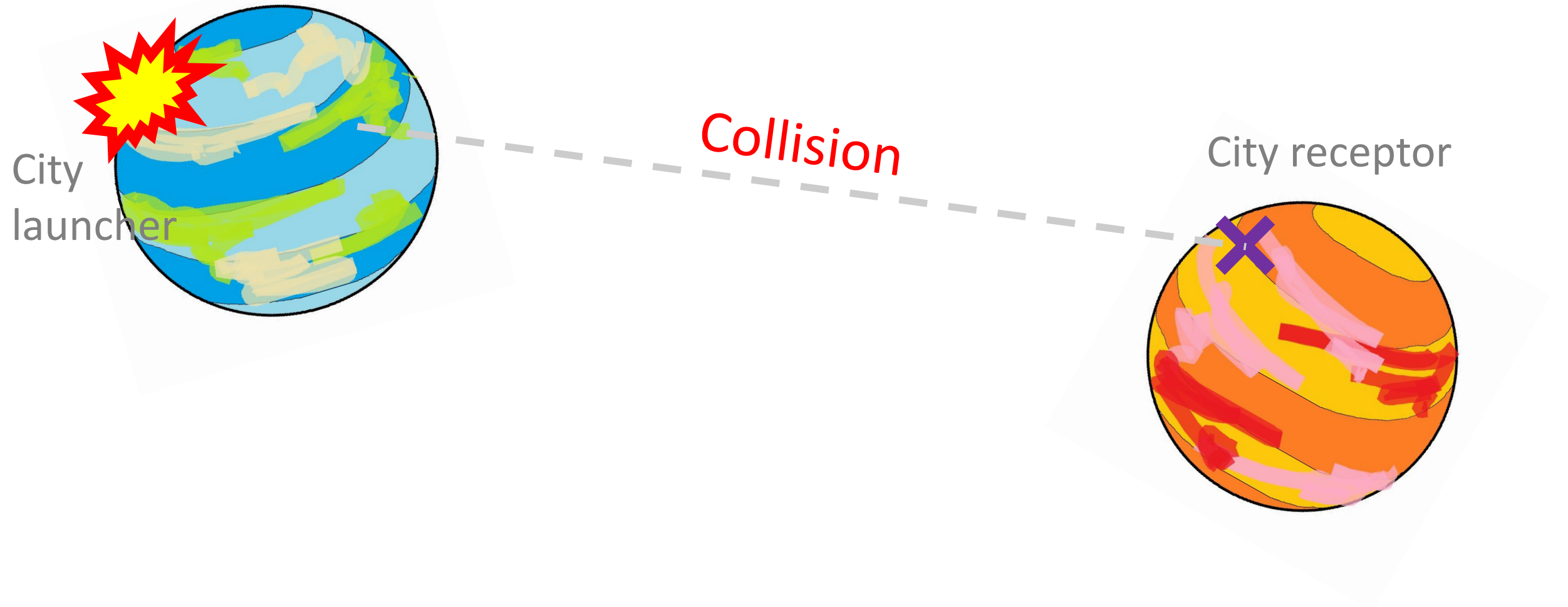
Problem statement

Fatal scenario (2)



Problem statement

Fatal scenario (2)



- Homogeneous coordinates

Point

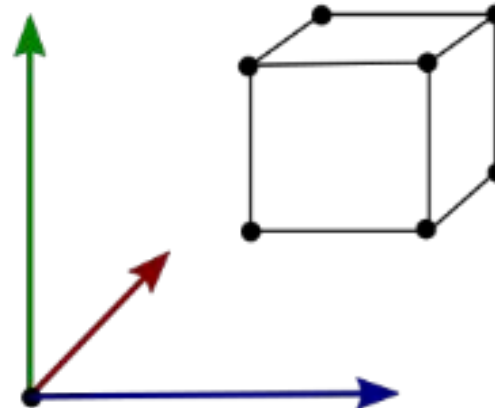
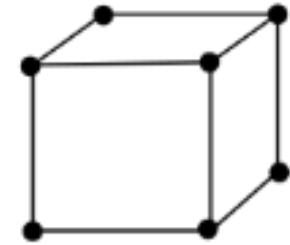
$$\mathbf{p} = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix} \Rightarrow \mathbf{v} = \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

Direction

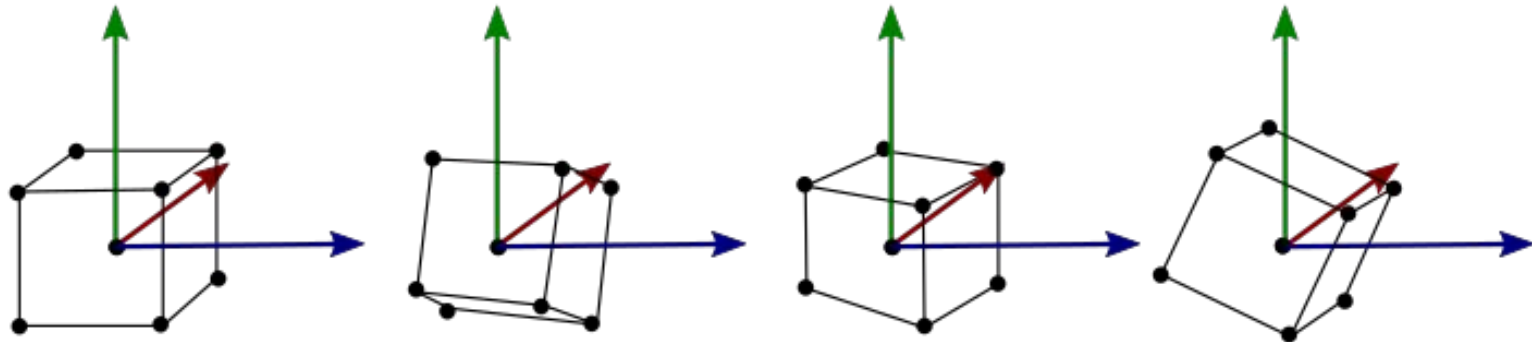
$$\mathbf{d} = \begin{pmatrix} d_x \\ d_y \\ d_z \end{pmatrix} \Rightarrow \mathbf{v} = \begin{pmatrix} d_x \\ d_y \\ d_z \\ 0 \end{pmatrix}$$

Basics

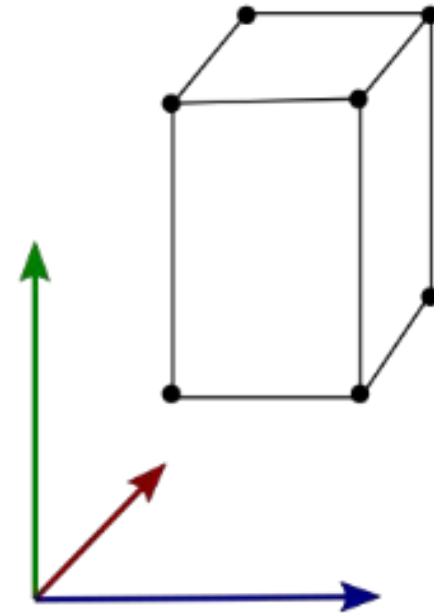
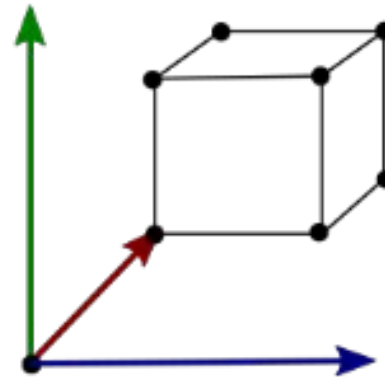
- **Homogeneous coordinates**
- **Transformation matrices**
 - Translation



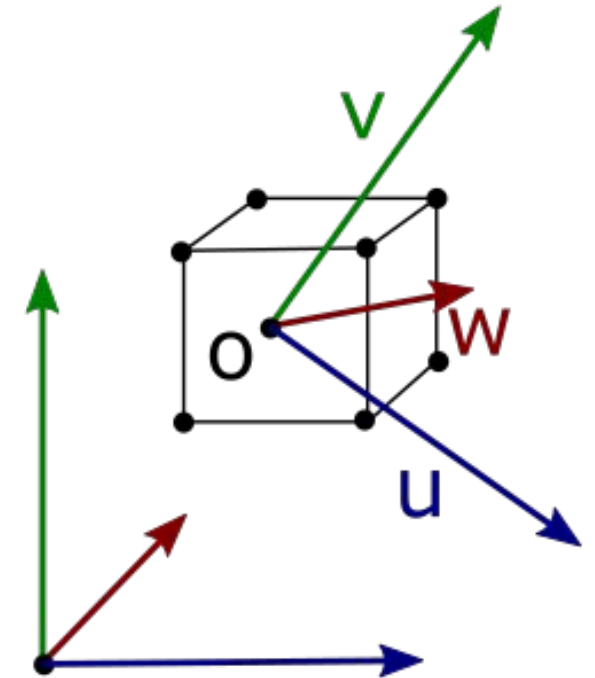
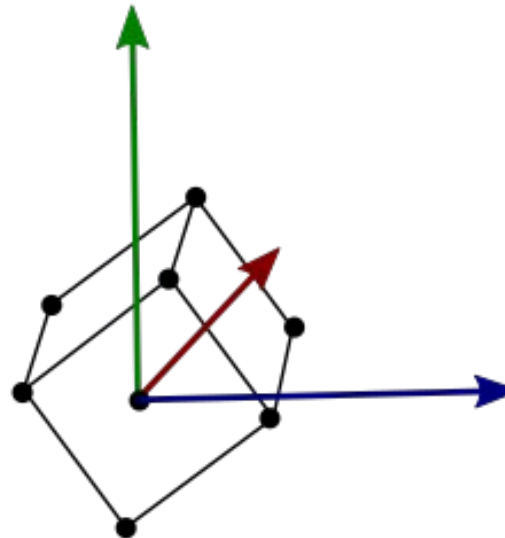
- **Homogeneous coordinates**
- **Transformation matrices**
 - Translation
 - Rotation



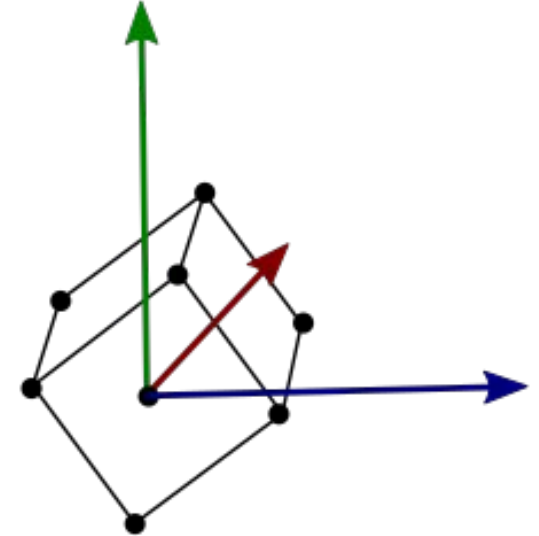
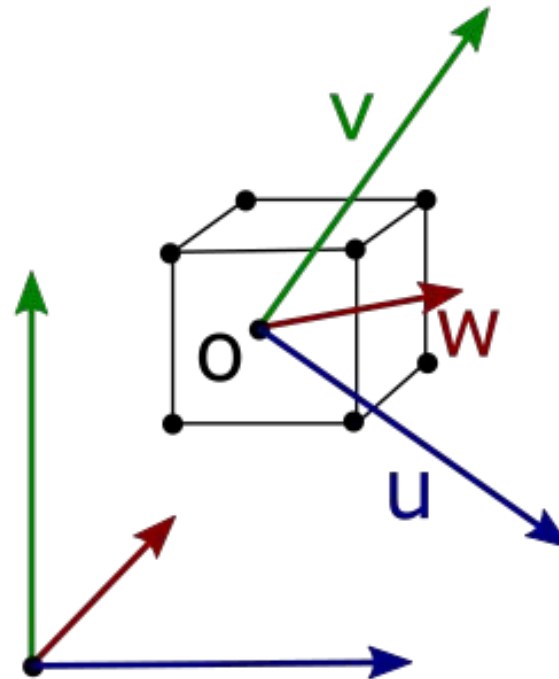
- **Homogeneous coordinates**
- **Transformation matrices**
 - Translation
 - Rotation
 - Scale



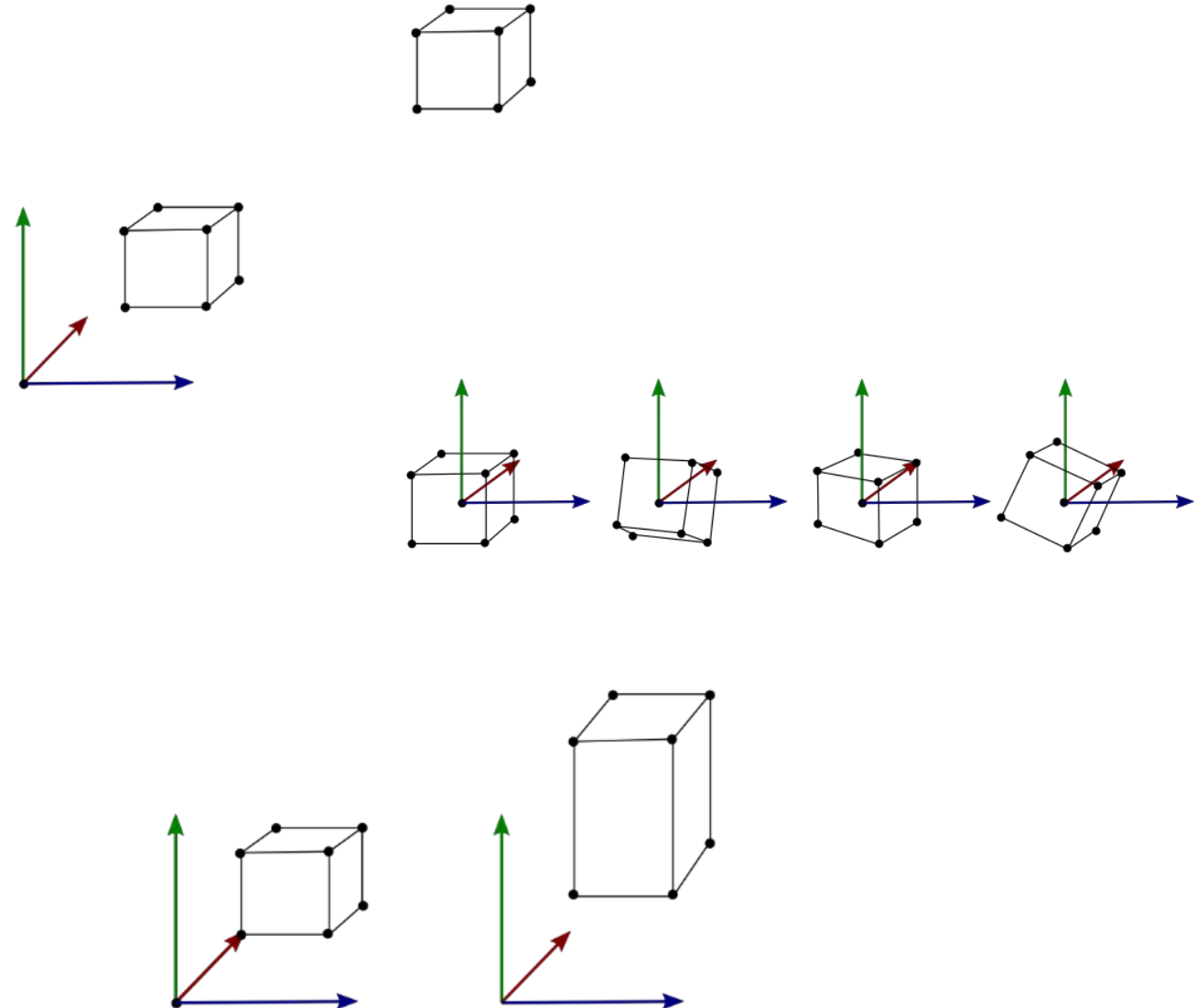
- Homogeneous coordinates
- Transformation matrices
 - Translation
 - Rotation
 - Scale
 - Change of base



- Homogeneous coordinates
- Transformation matrices
 - Translation
 - Rotation
 - Scale
 - Change of base
 - Inverse transform

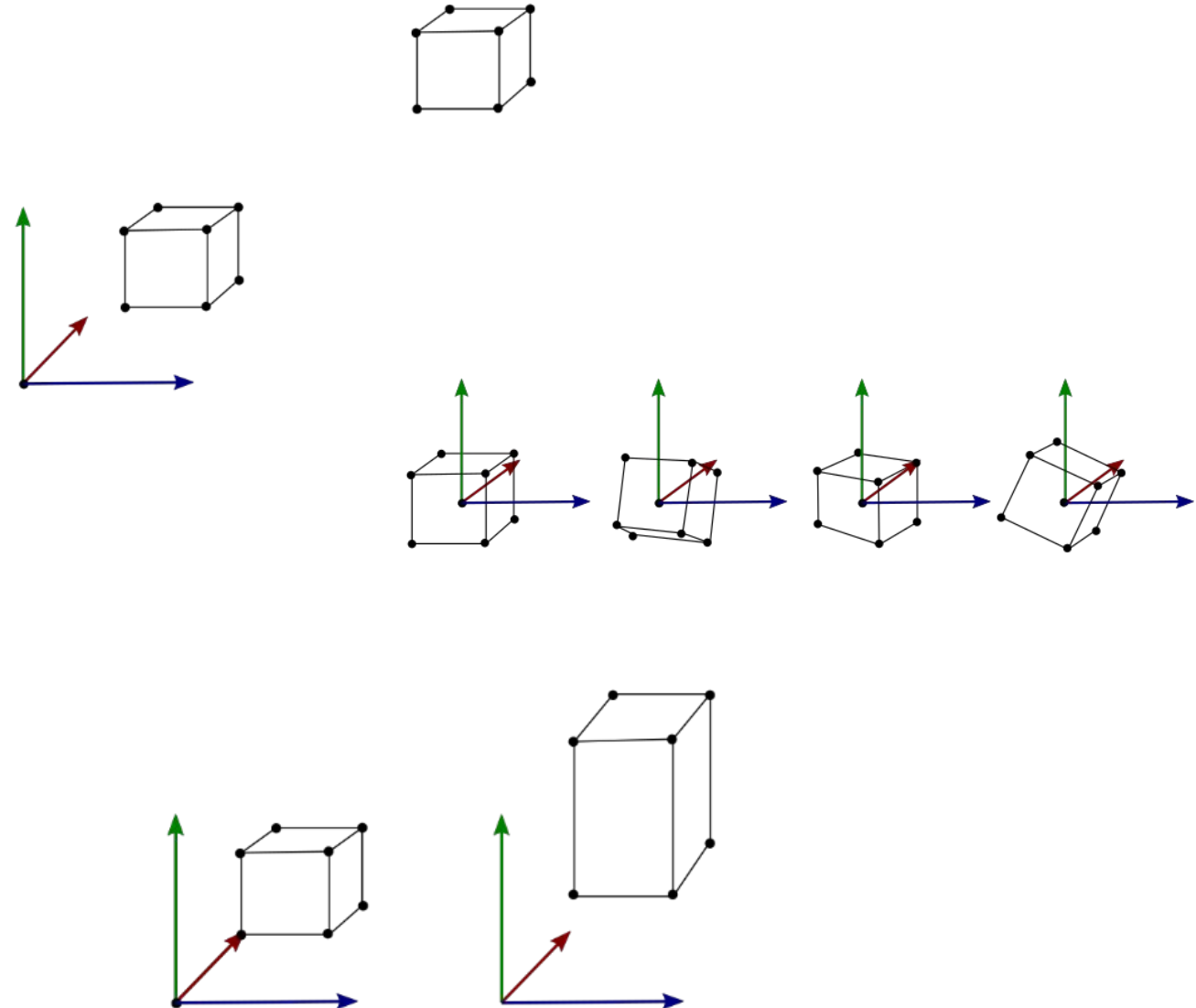


- **Homogeneous coordinates**
- **Transformation matrices**
 - Translation
 - Rotation
 - Scale
 - Change of base
 - Inverse transform
- **Combine transform matrices**



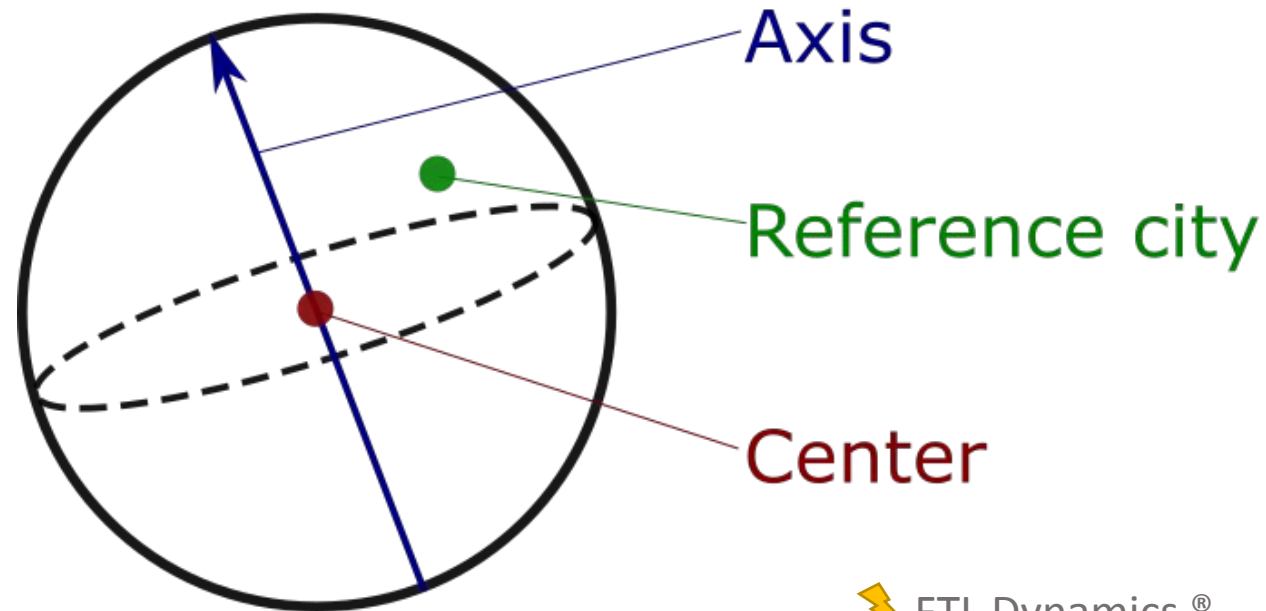
- **Homogeneous coordinates**
- **Transformation matrices**
 - Translation
 - Rotation
 - Scale
 - Change of base
 - Inverse transform
- **Combine transform matrices**

```
std::cout << matrix << std::endl;  
/ 1.0  0.0  0.0  0.0 \  
| 0.0  1.0  0.0  0.0 |  
| 0.0  0.0  1.0  0.0 |  
\ 0.0  0.0  0.0  1.0 /
```



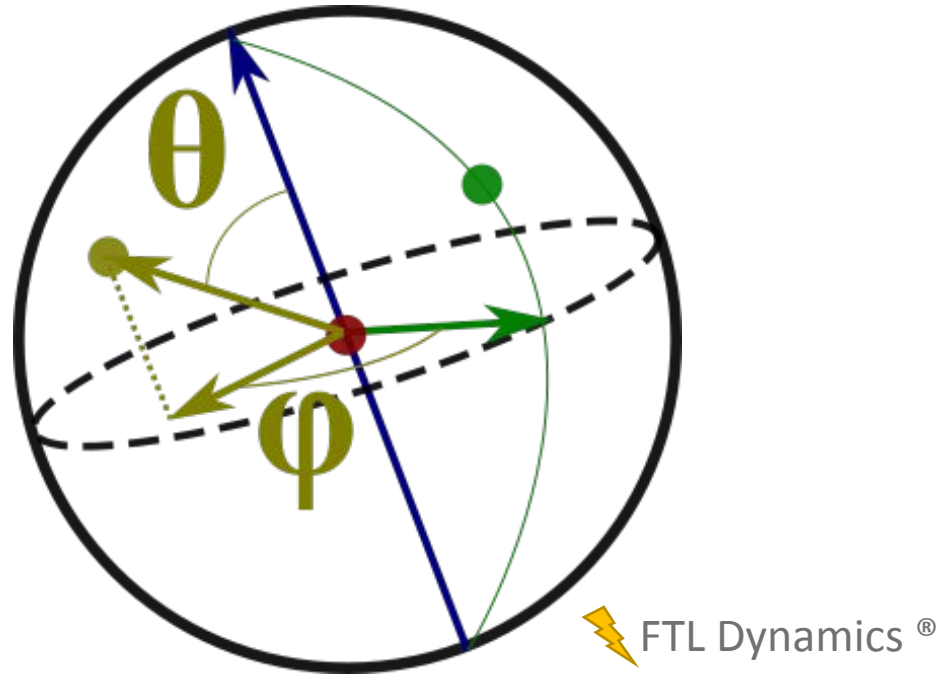
Planetary stations

- How to define a planet



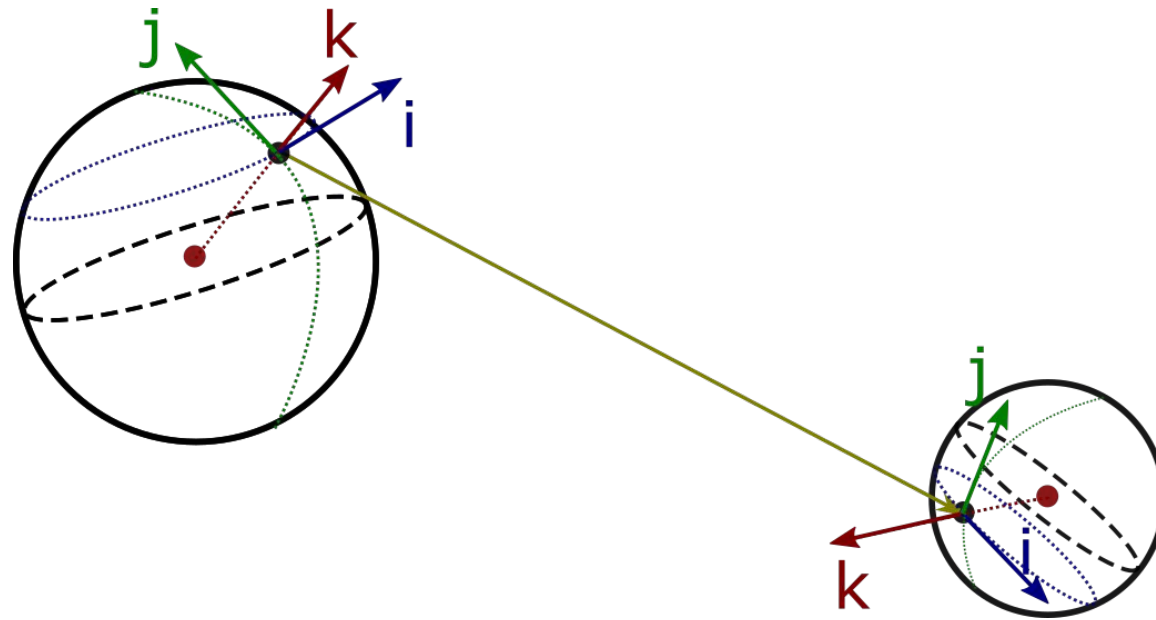
Planetary stations

- How to define a planet
- How to define a city



Planetary stations

- How to define a planet
- How to define a city
- Interplanetary connections



DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice, implement:

- **Data basics:** Matrices, homogeneous coordinates
- **Transformation matrices:**
 - Translation, rotation, scale
 - Change of basis, inverse transformation, combine transform matrices
 - Pretty stdout operator
- Use your new math powers on your job at FTL dynamics
- Recommended deadline: September 25th. Do you have **extra time**?
 - Go home and rest :)
 - Next session: read/write PPM images, tonemapping
 - Try programming ray-sphere intersections, test it with planetary stations

Lab #2 - Imaging

Informática Gráfica

Adolfo Muñoz - Julio Marco
Pablo Luesia - J. Daniel Subías – Óscar Pueyo



Before we begin...

- The problem:



Your final work's output
(without post-processing)

Before we begin...

- The problem:



Your final work's output
(without post-processing)

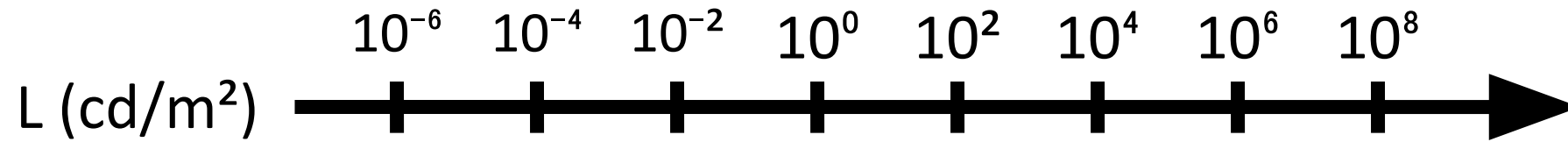


After tone mapping
(this session work!)

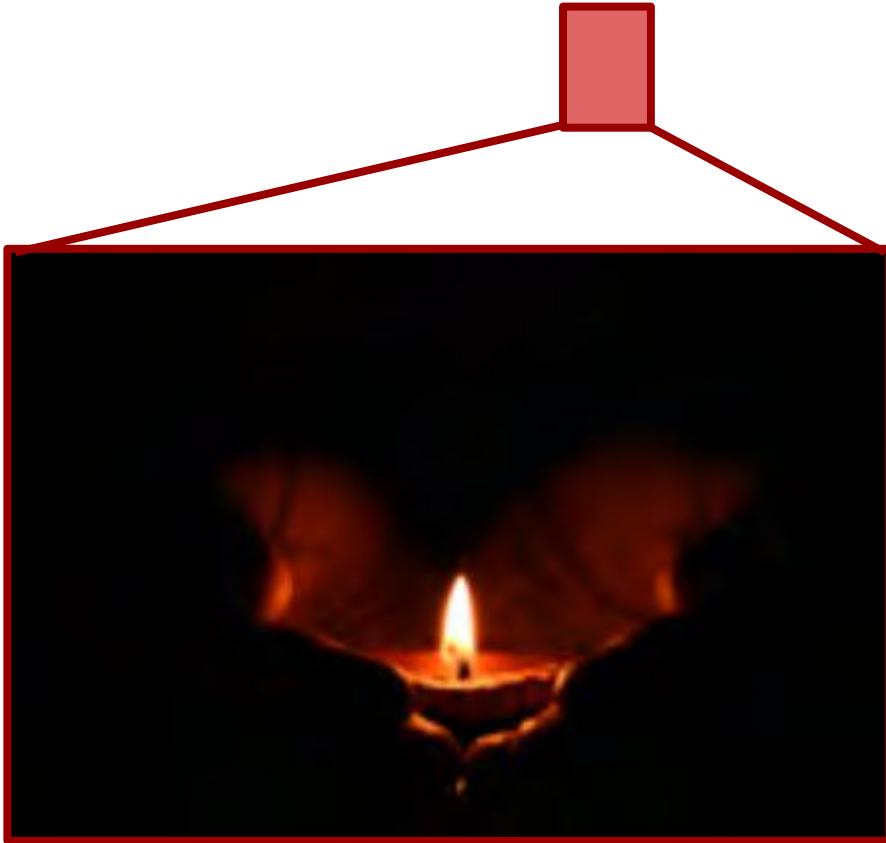
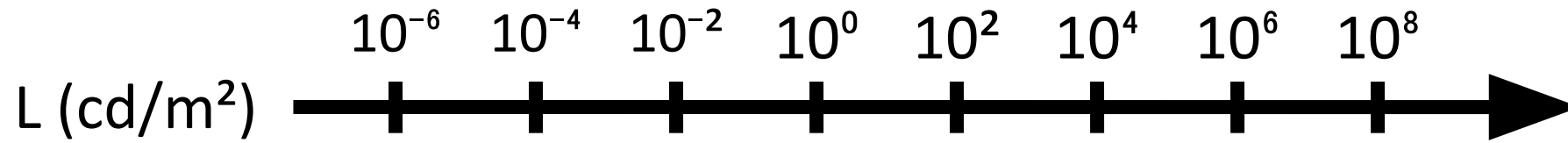
Before we begin...

- Intermediate assignment, again
 - No submission required
- To be completed before **October 2nd (without extensions)**
 - **Highly recommended**
- Remember: Final work is 80% of the final grade

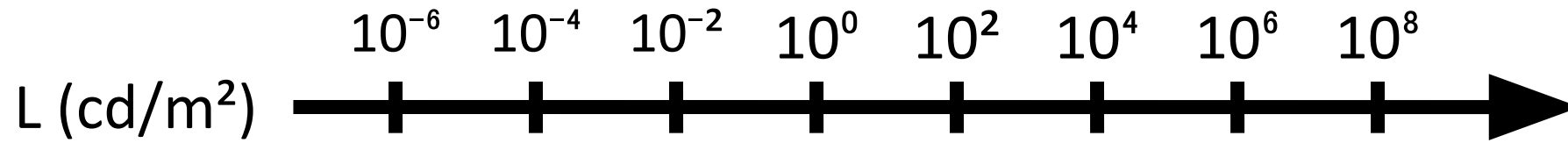
Problem statement, again



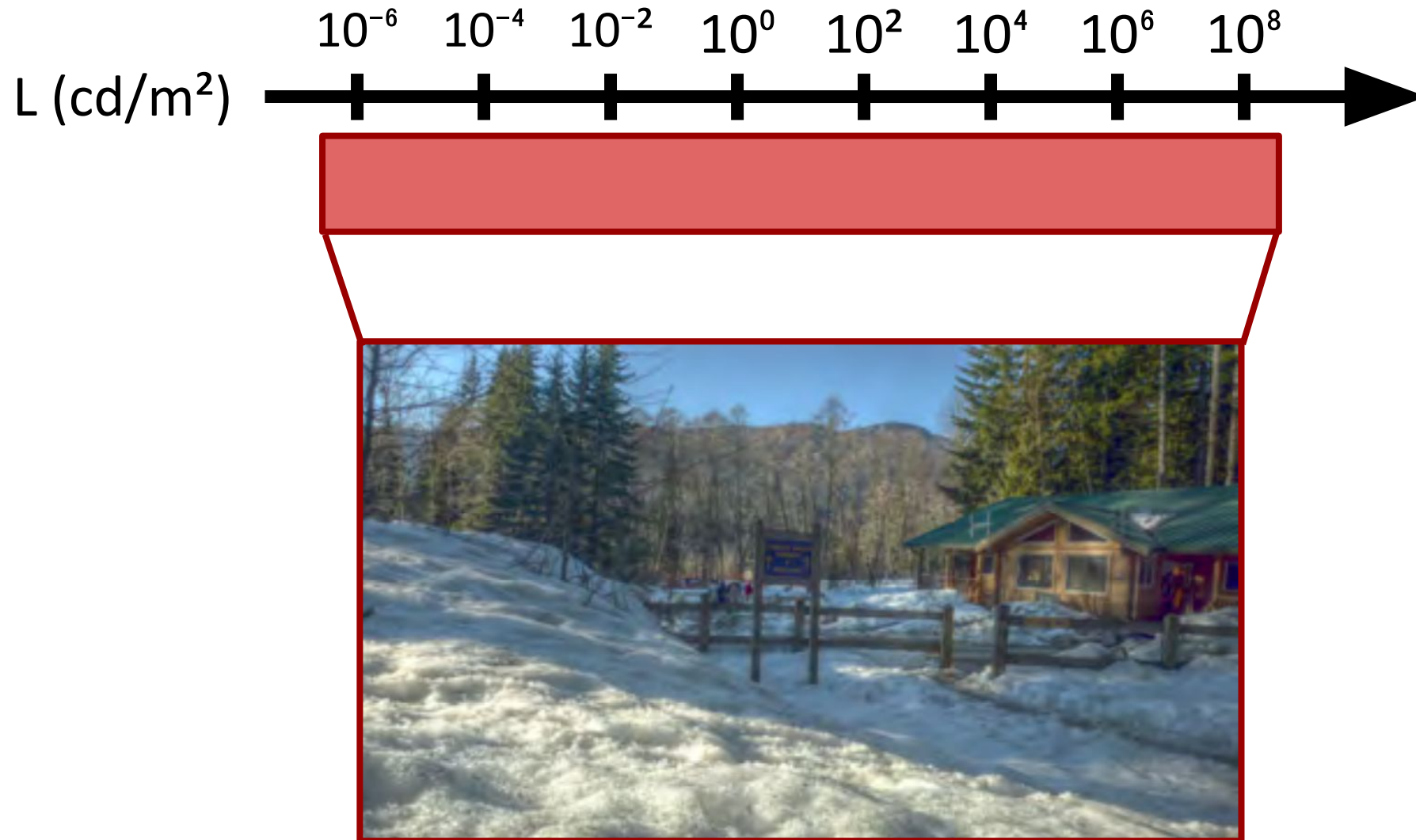
Problem statement, again



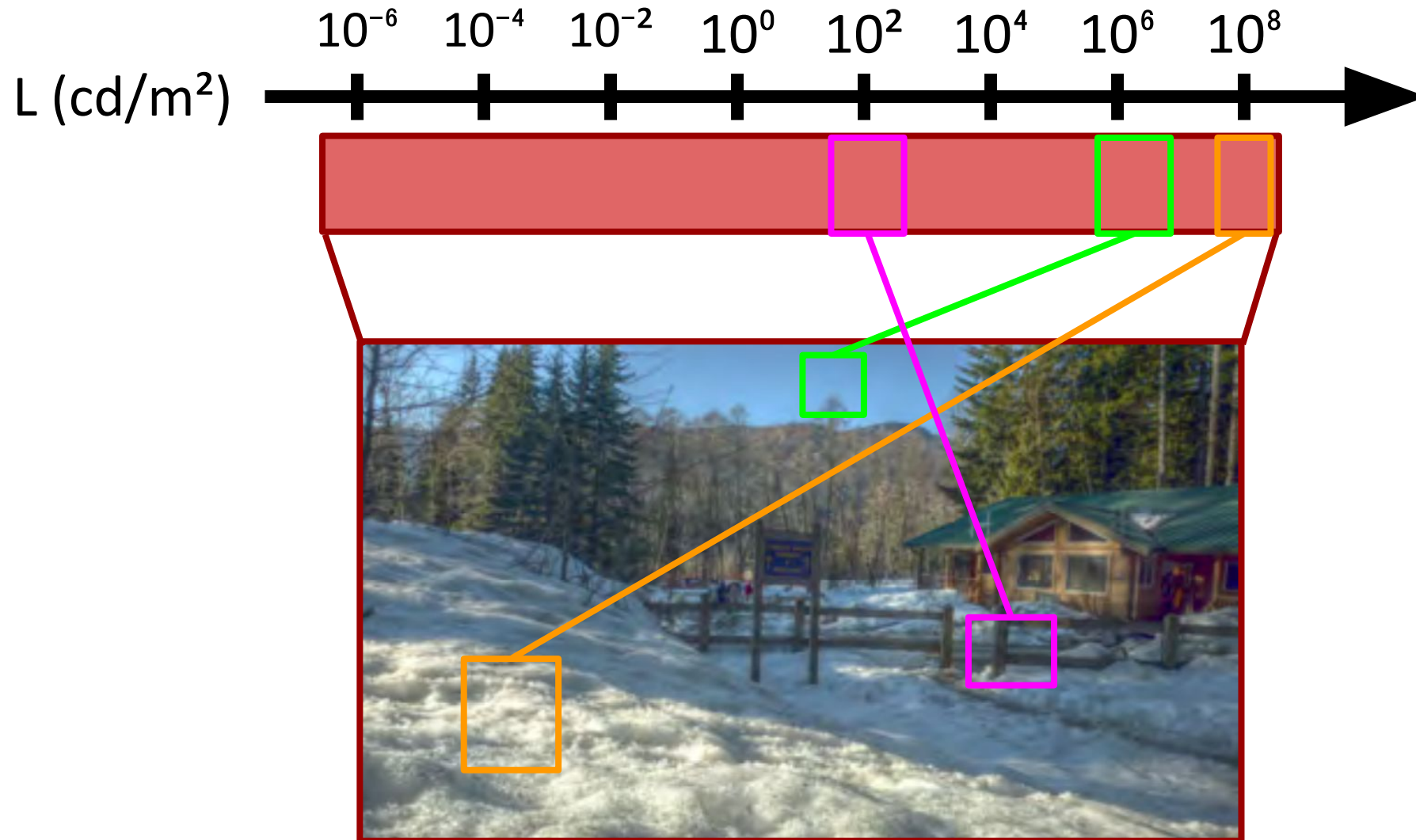
Problem statement, again



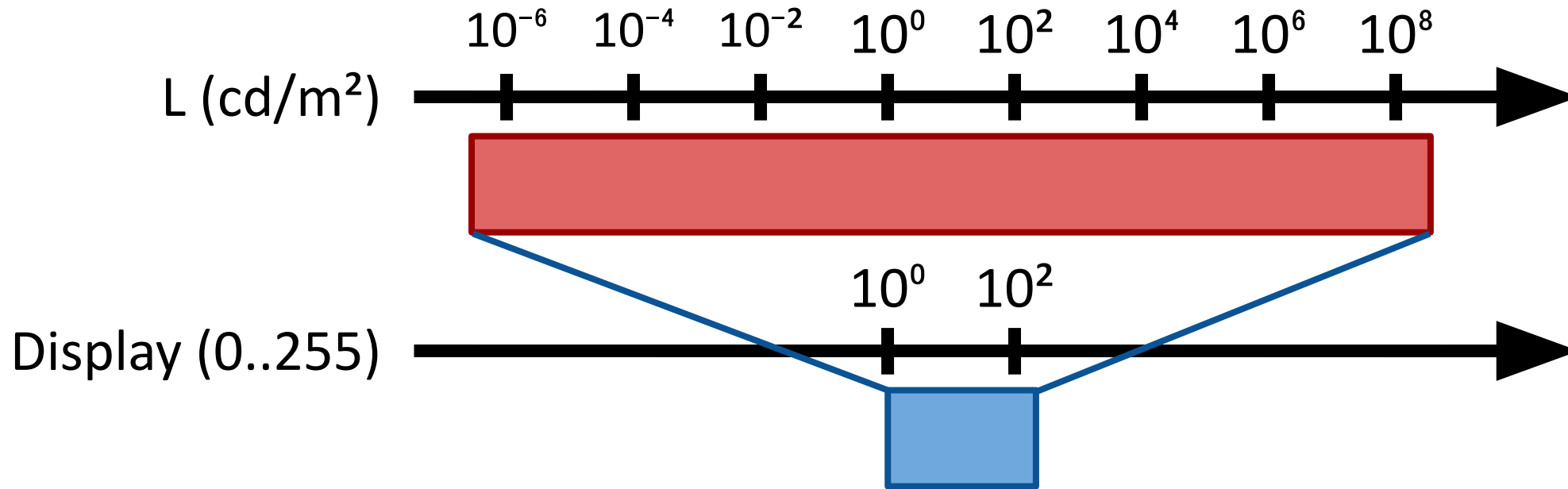
Problem statement, again



Problem statement, again

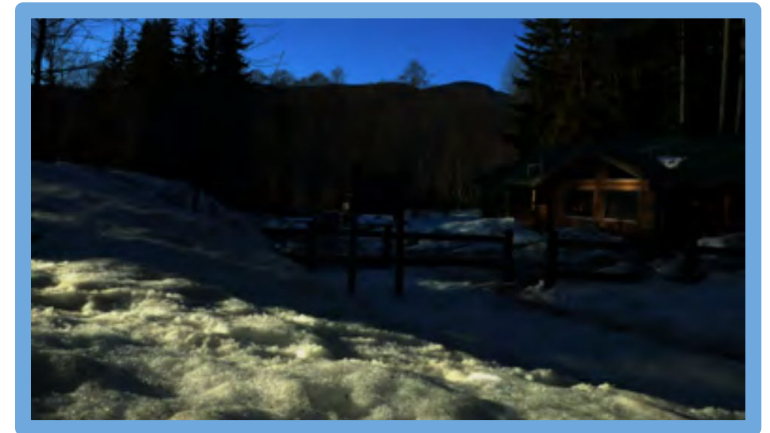
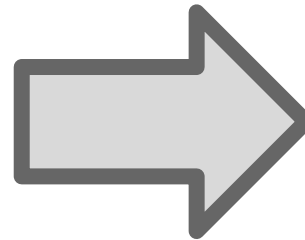
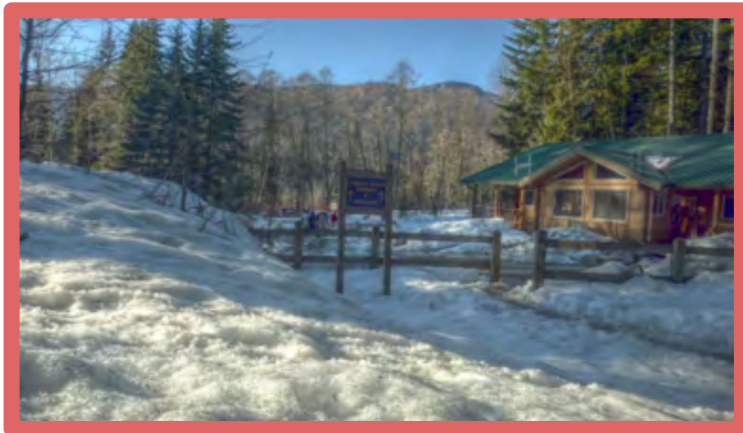
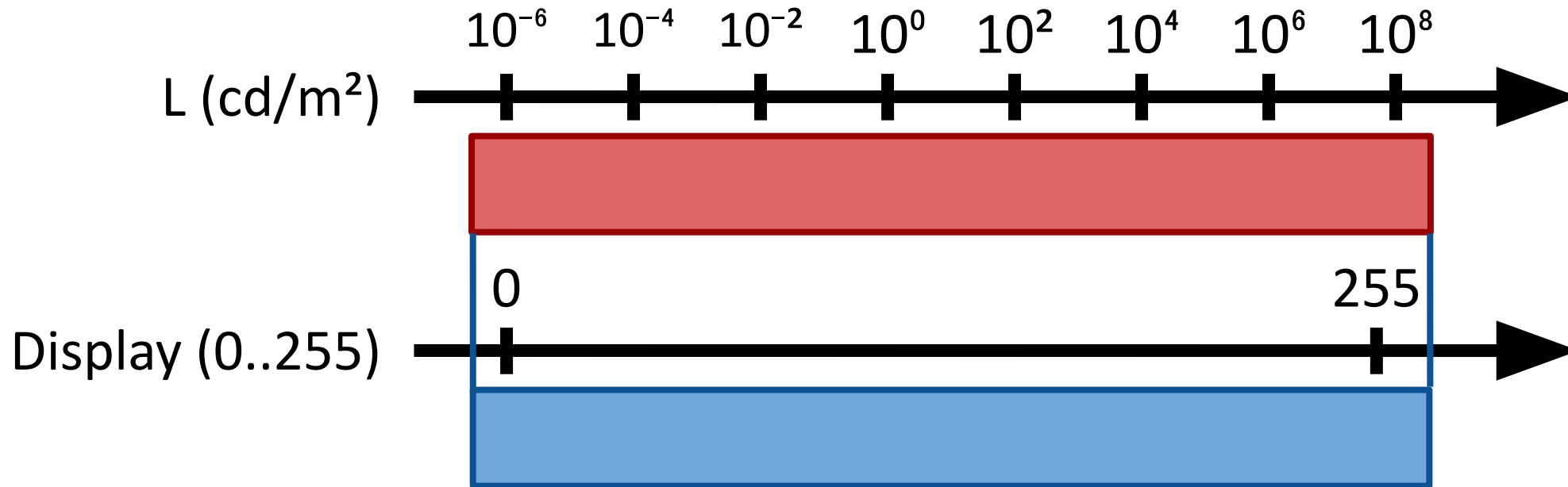


Problem statement, again

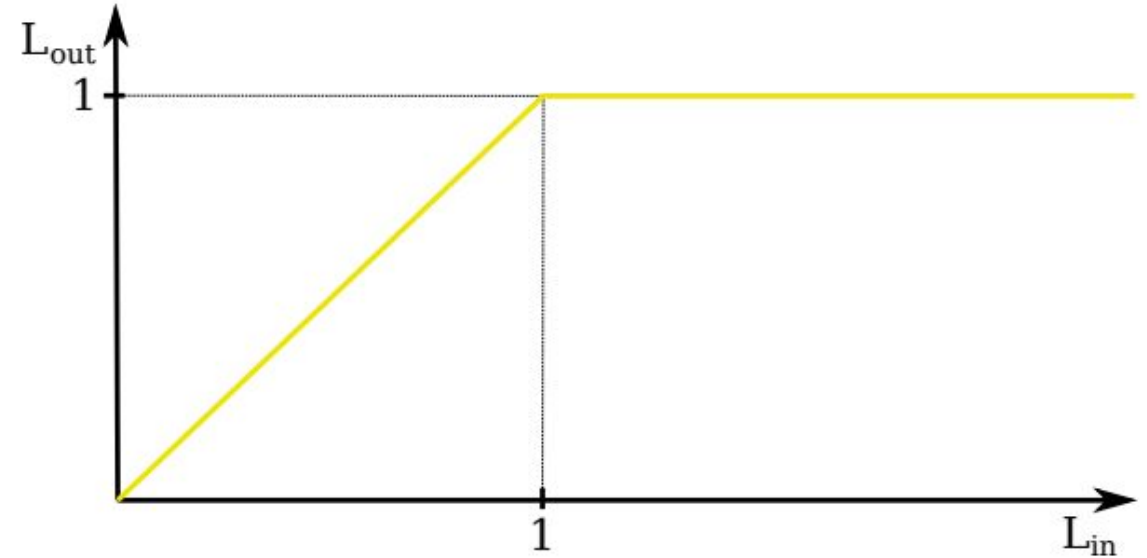
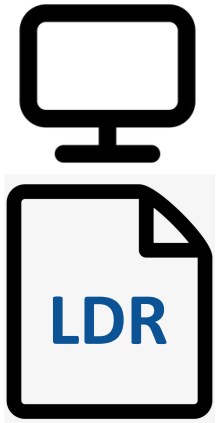


- A (typical) display only has 255 possible values for each color channel
- How to map the **red values** to the **blue values**?

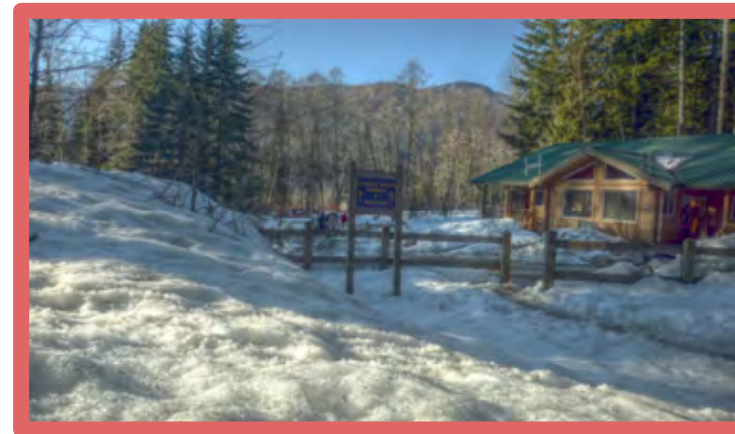
Problem statement, again



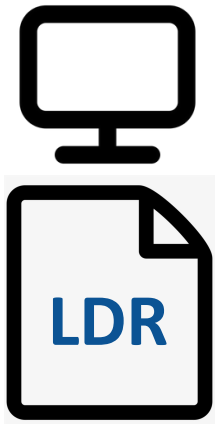
Tone mapping operators



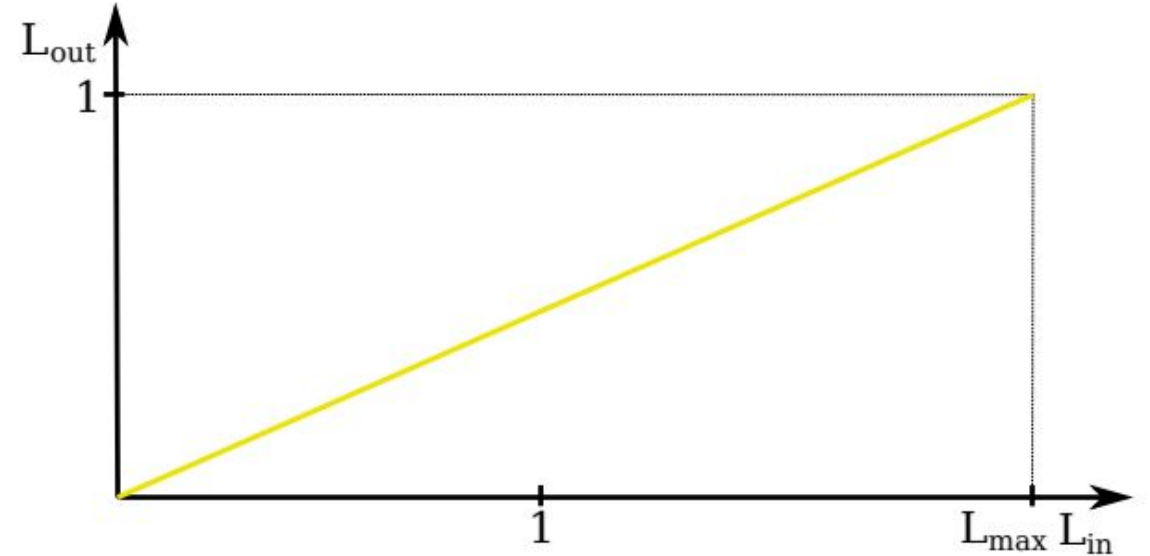
Clamping operator



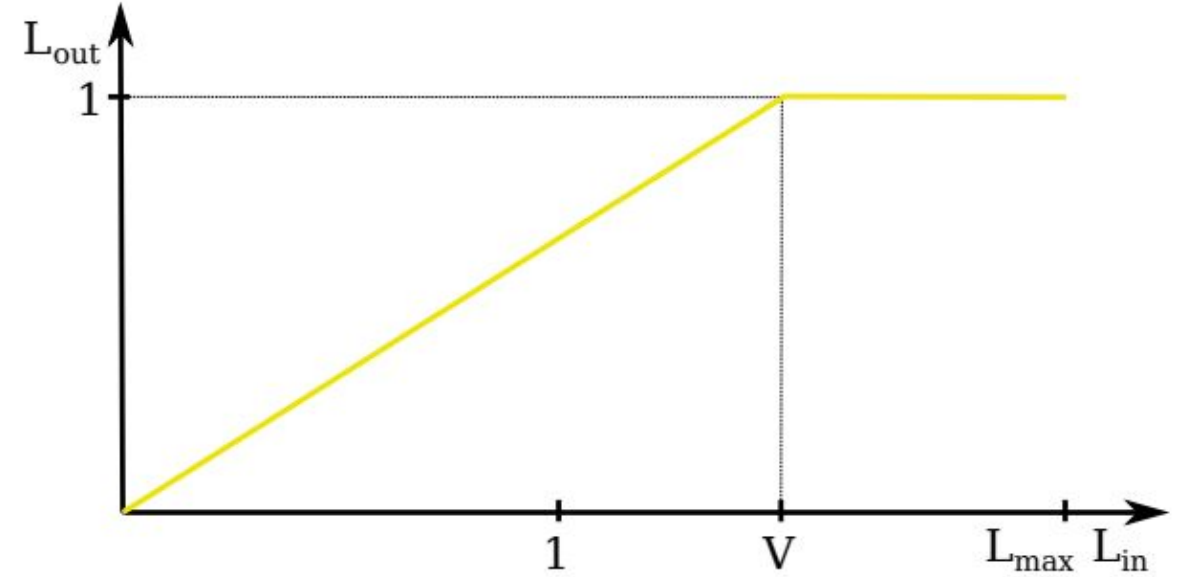
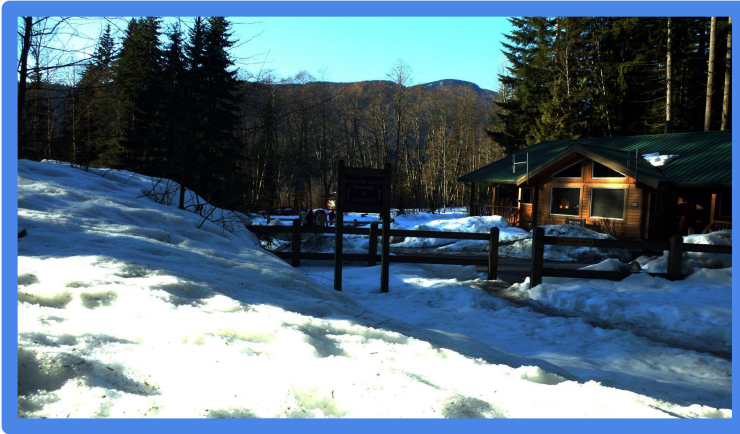
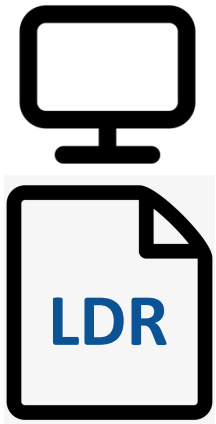
Tone mapping operators



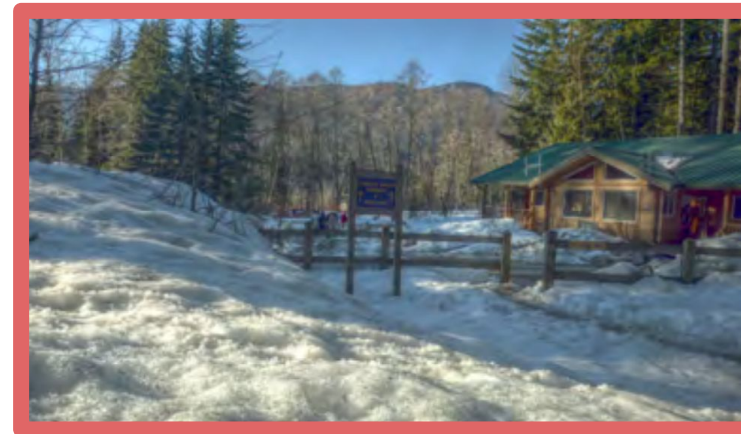
Equalization operator



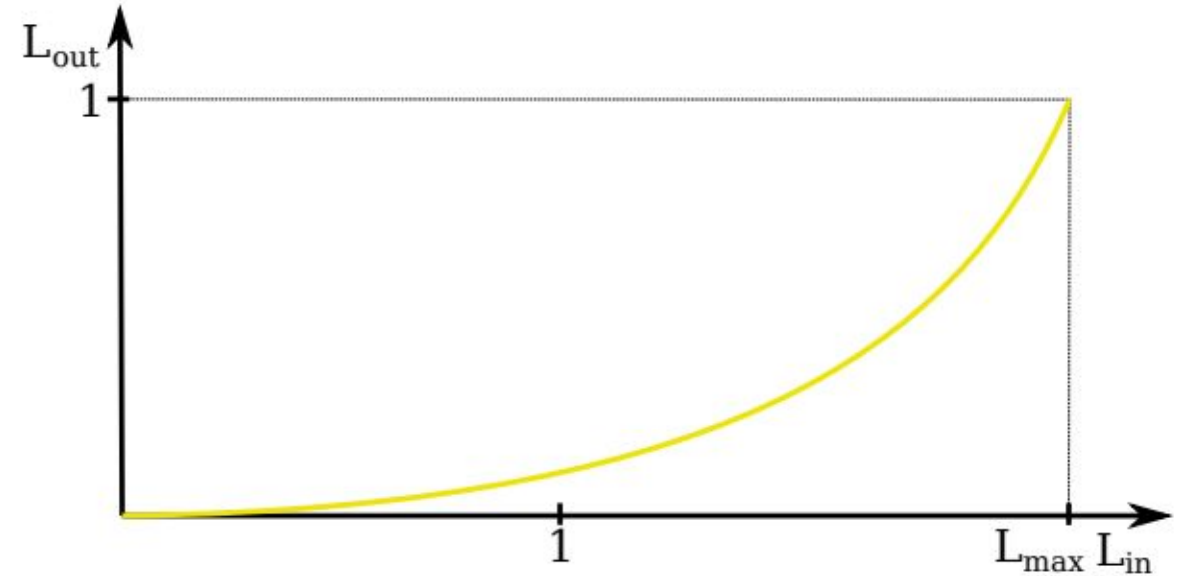
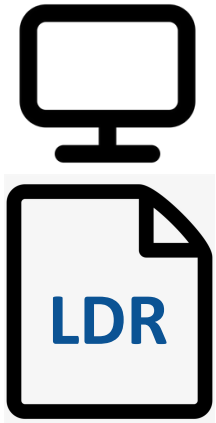
Tone mapping operators



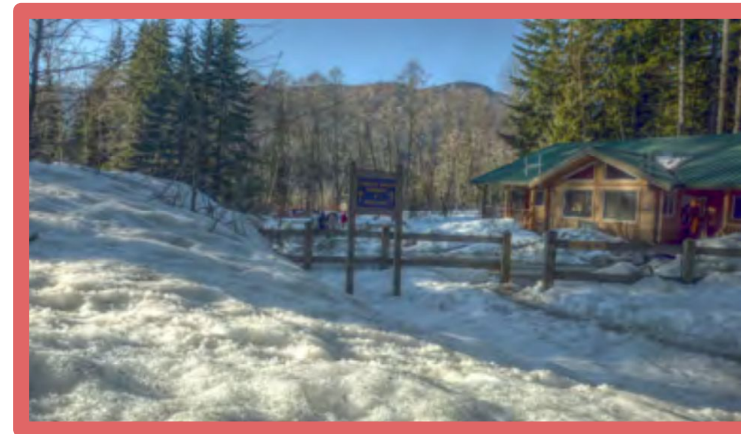
(Equalization + clamping)
operator



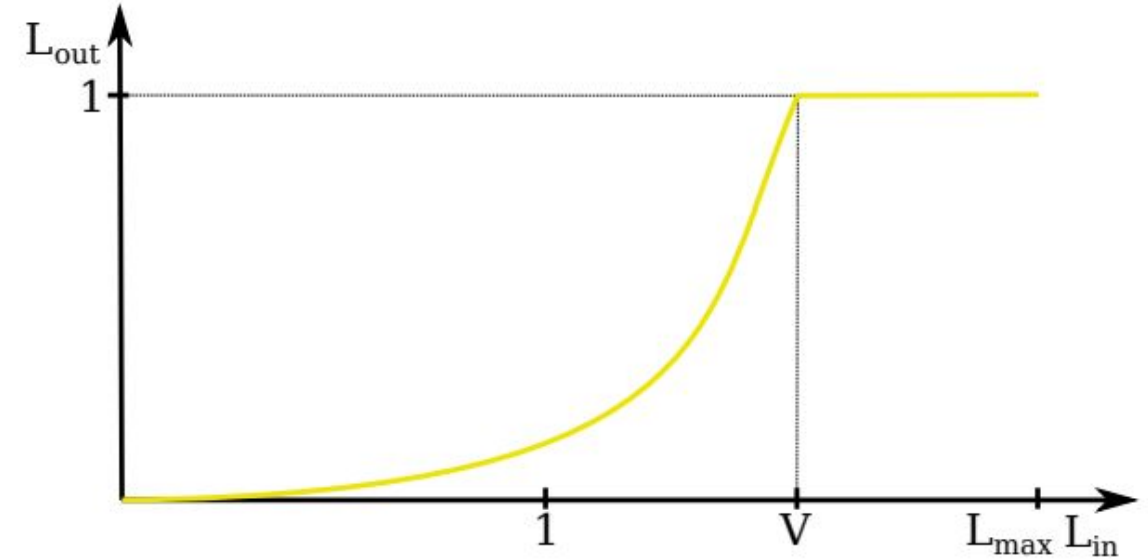
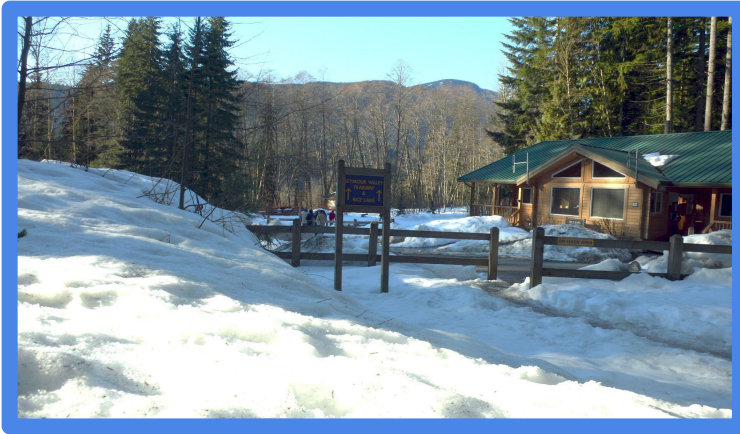
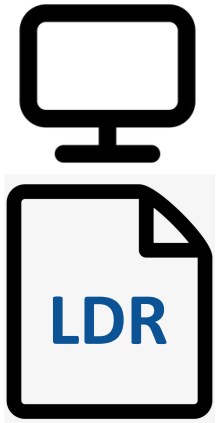
Tone mapping operators



Gamma operator



Tone mapping operators



(Gamma + clamping)
operator



But... how do we store it?

But... how do we store it?

PPM file format

PPM files

- Readable by most viewers
- Simple text file

```
1  P3
2  # feep.ppm
3  4 4
4  15
5  0 0 0      0 0 0      0 0 0      15 0 15
6  0 0 0      0 15 7    0 0 0      0 0 0
7  0 0 0      0 0 0      0 15 7    0 0 0
8  15 0 15     0 0 0      0 0 0      0 0 0
```

Details in: <https://netpbm.sourceforge.net/doc/ppm.html>

PPM files

The diagram illustrates the P3 PPM file format with the following content and annotations:

```

1 P3
2 # feep.ppm
3 4 4
4 15
5 0 0 0 0 0 0 0 0 0 0 0 0
6 0 0 0 0 15 7 0 0 0 0 0 0
7 0 0 0 0 0 0 0 0 15 7 0 0
8 15 0 15 0 0 0 0 0 0 0 0 0

```

Annotations:

- Format:** Points to the `P3` header.
- Comment: line starts with #**: Points to the comment line `# feep.ppm`.
- Size resolution: width and height in that order**: Points to the resolution values `4 4`.
- Color resolution (its maximum value)**: Points to the maximum color value `15`.
- A pixel**: Points to a group of three values (0 0 0) representing a single pixel.

Color channels are indicated by letters above the values in the data rows:

- R (Red):** Values 15, 0, 0, 0, 0, 0, 0, 0.
- G (Green):** Values 0, 0, 0, 0, 15, 7, 0, 0.
- B (Blue):** Values 15, 0, 0, 0, 0, 0, 15, 7.

The color maximum problem

- Problem 1: PPM standard and HDR files
 - Color stored in disk: [0..15] - Color stored in memory: [0..1]
 - Pixel value [0..15] divided by maximum (15)
 - The standard only allows maximum up to 65535.
 - We will ignore that for HDR images.
 - HDR maximum up to 2^{30}

```
1  P3
2  # feep.ppm
3  4 4
4  15
5  0 0 0 0 0 0 0 0 0 15 0 15
6  0 0 0 0 15 7 0 0 0 0 0 0
```

The memory maximum problem

- Problem 2: Memory maximum value (v) is still only 1
 - Define a custom maximum in your PPM file reader
 - **Optional comment #MAX=<real maximum, decimal number>**

```
1  P3
2  # feep.ppm
3  #MAX=18.35
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

The memory maximum problem

- Problem 2: Memory maximum value v is still only 1
 - Define a custom maximum in your PPM file reader
 - **Optional comment #MAX=<real maximum, decimal number>**

$$v = \frac{s}{\frac{m}{c}} \quad \text{READING FILE}$$

$$s = v \frac{c}{m} \quad \text{SAVING FILE}$$

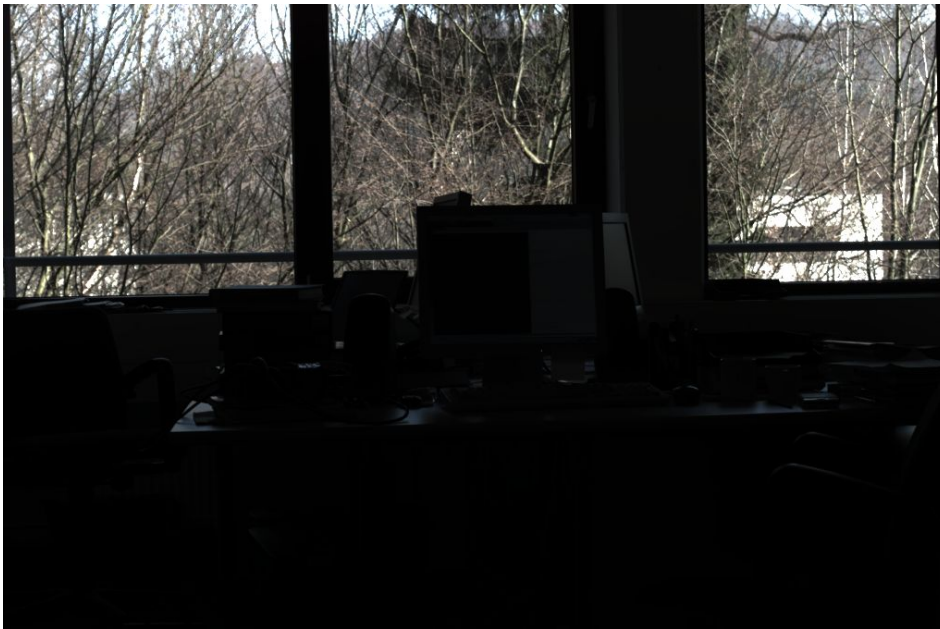
```
1  P3
2  # feep.ppm
3  #MAX=18.35
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

Example files

- There are some .ppm files in Moodle you can test with

Example files

- There are some .ppm files in Moodle you can test with
- Some examples (tone mapping RGB channels jointly)
 - **Clamp**



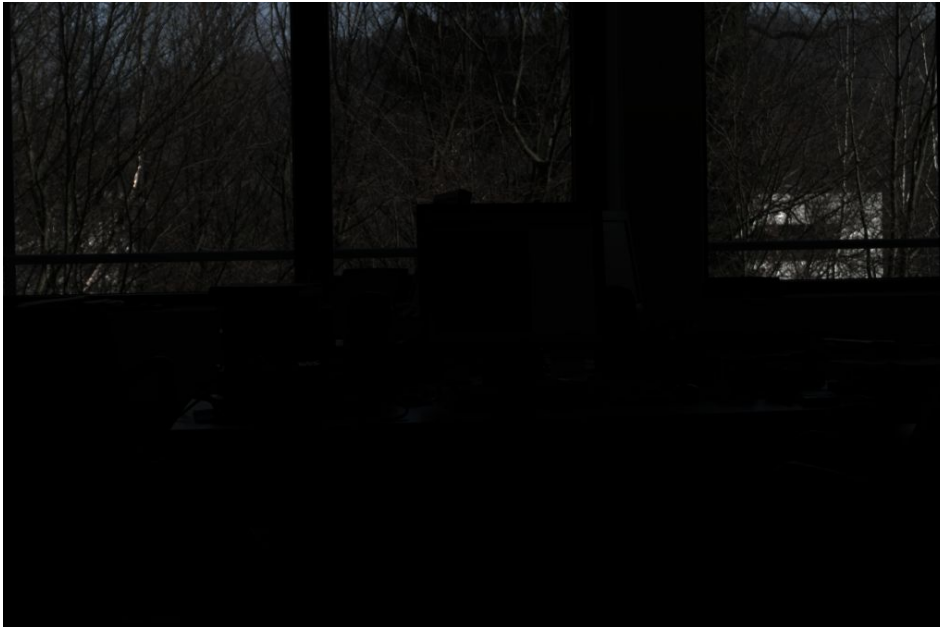
mpi_office.ppm



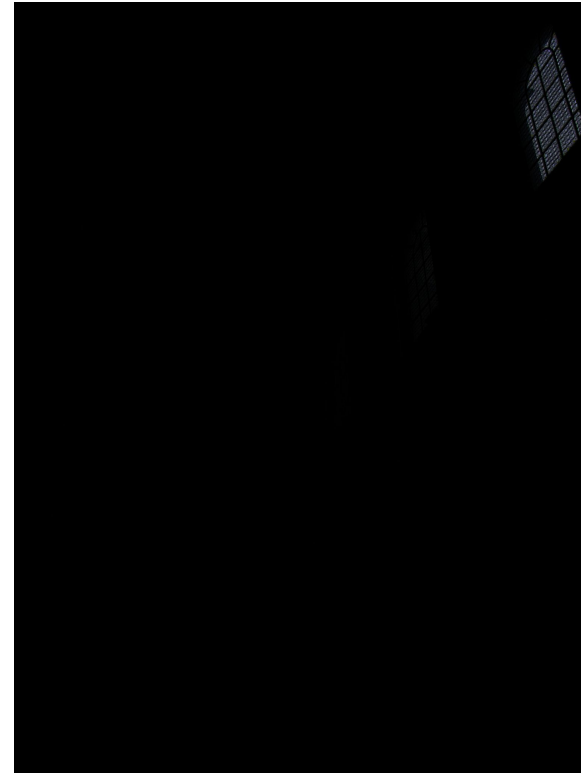
nancy_church_2.ppm

Example files

- There are some .ppm files in Moodle you can test with
- Some examples (tone mapping RGB channels jointly)
 - **Equalization**



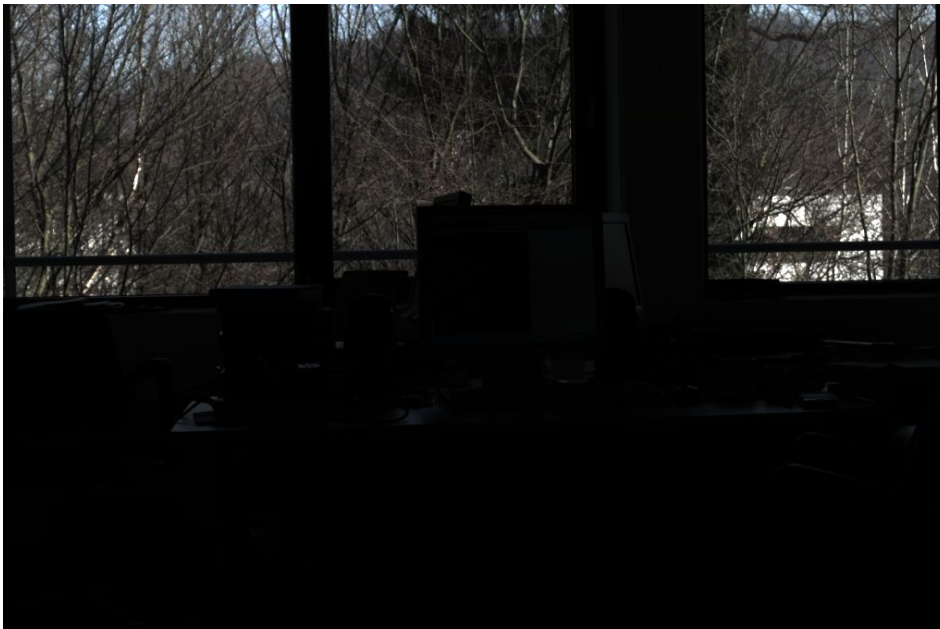
mpi_office.ppm



nancy_church_2.ppm

Example files

- There are some .ppm files in Moodle you can test with
- Some examples (tone mapping RGB channels jointly)
 - **Equalization + clamping**



mpi_office.ppm



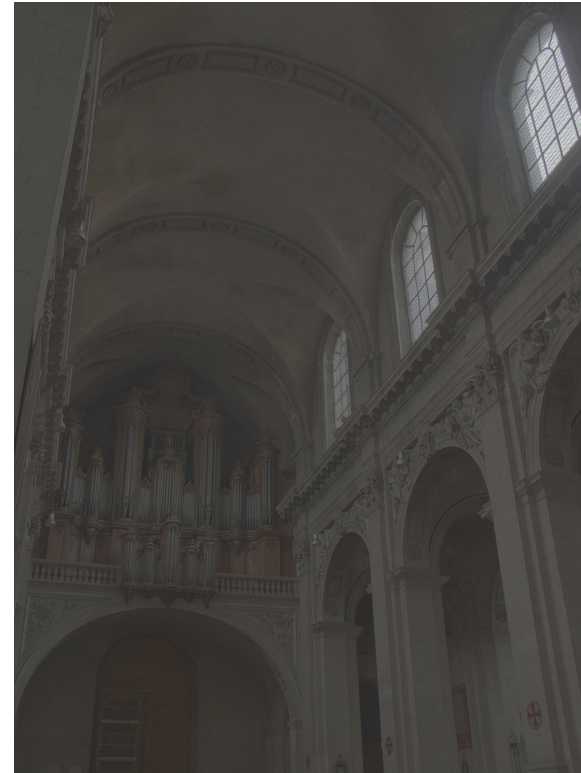
nancy_church_2.ppm

Example files

- There are some .ppm files in Moodle you can test with
- Some examples (tone mapping RGB channels jointly)
 - **Gamma**



mpi_office.ppm



nancy_church_2.ppm

Example files

- There are some .ppm files in Moodle you can test with
- Some examples (tone mapping RGB channels jointly)
 - **Clamp + gamma**



mpi_office.ppm



nancy_church_2.ppm

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice, implement:

- **PPM file reader and writer**
 - With disk maximum >65535 and memory maximum >1
- **Implement tone mapping operators**
 - Clamping, equalization, gamma, combinations
- Recommended deadline: **October 2nd**. Extensions:
 - Use other color spaces (HSV, HSL) for tone mapping
 - New tone mapping operators (check class slides)
 - Other LDR formats to save the image
 - You can use existing programs (e.g. convert PPM to PNG), just keep in mind that we are not strictly following the PPM standard

Lab #3 - Ray tracing (part 1)

Informática Gráfica

Adolfo Muñoz - Julio Marco

Pablo Luesia - J. Daniel Subías – Óscar Pueyo



Before we begin...

- Requirements for this lab:
 - Points, directions, matrices and their operations

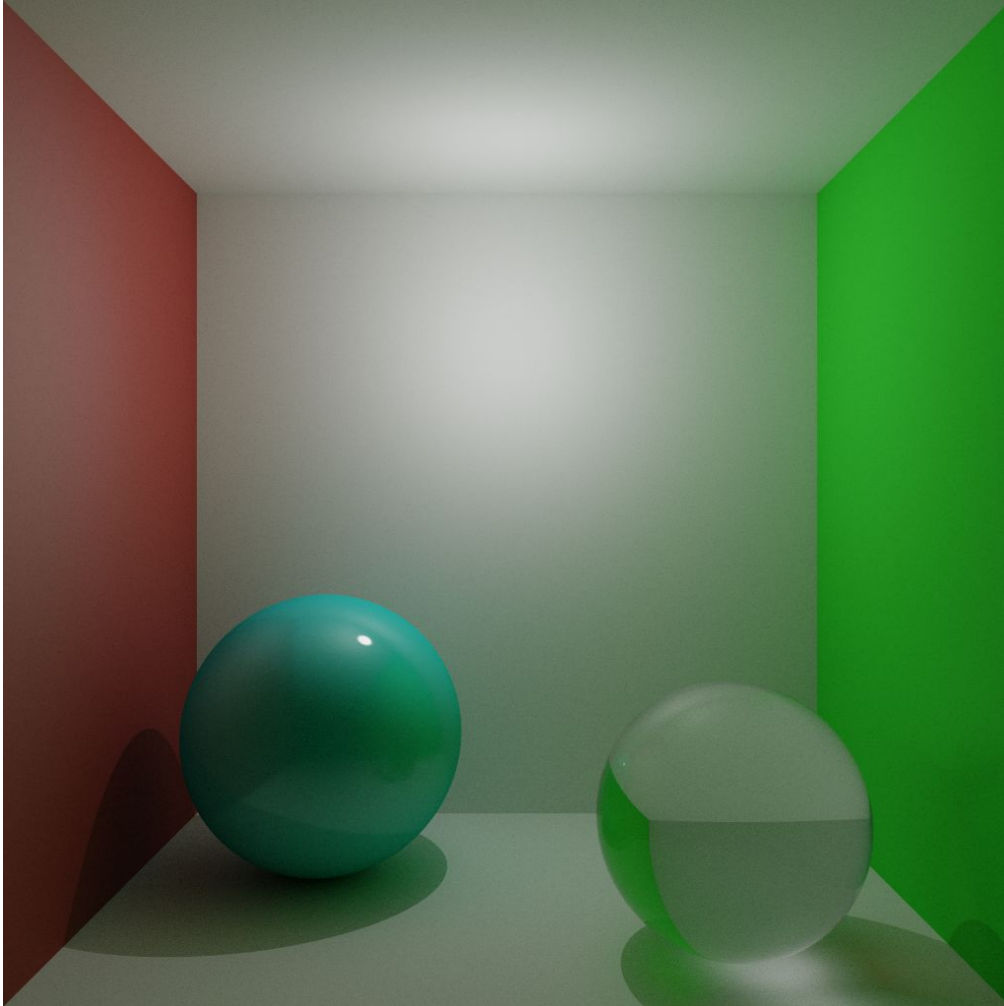
Before we begin...

- Requirements for this lab:
 - Points, directions, matrices and their operations
- Careful with the submissions
 - Lab 3 (ray tracing) does not need to be submitted
 - Recommended deadline: **October 18th**

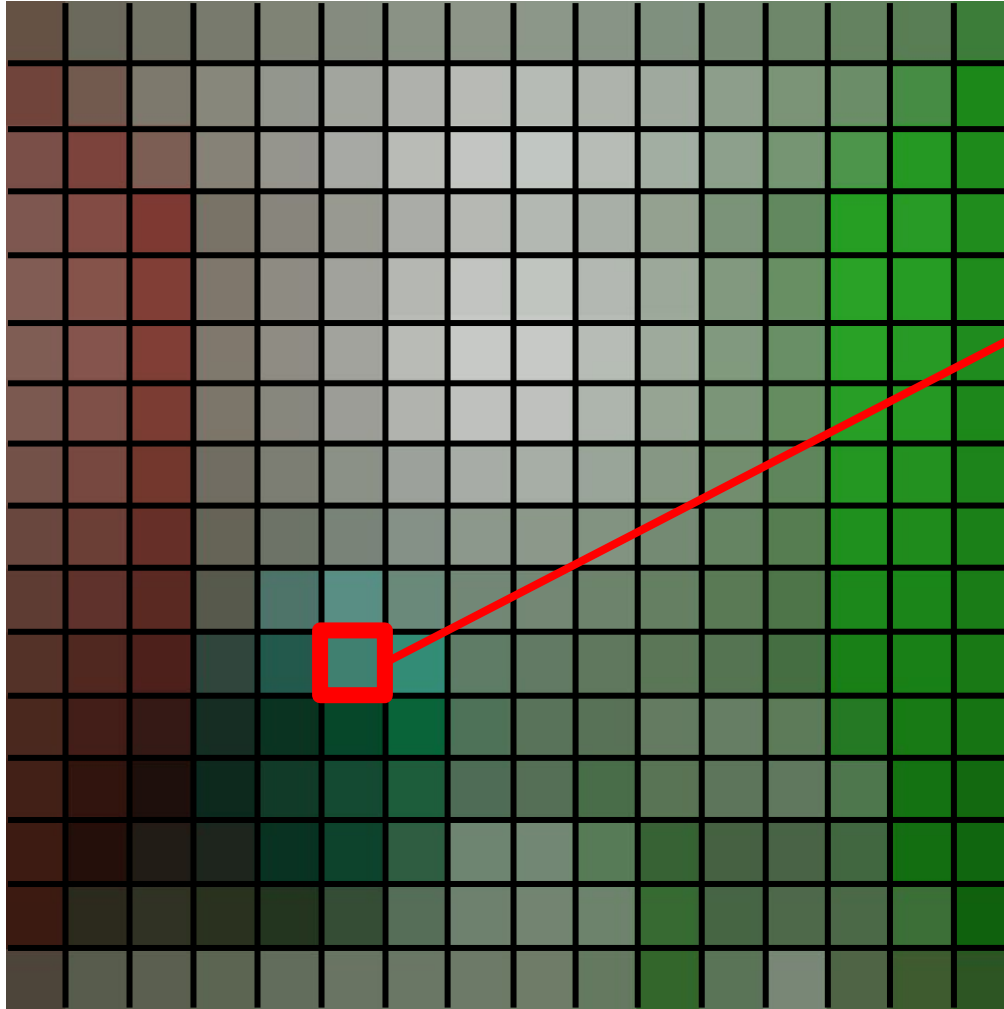
Before we begin...

- Requirements for this lab:
 - Points, directions, matrices and their operations
- Careful with the submissions
 - Lab 3 (ray tracing) does not need to be submitted
 - Recommended deadline: **October 18th**
 - Lab 4 (path tracing) **will be submitted** at the end of the course
 - All of the code you write today will be used for Lab 4
 - Recommended deadline: November 13th (moodle: January 11th, this is only a recommendation)
- Remember: Final work is 80% of the final grade

Which color do we fill each pixel with?



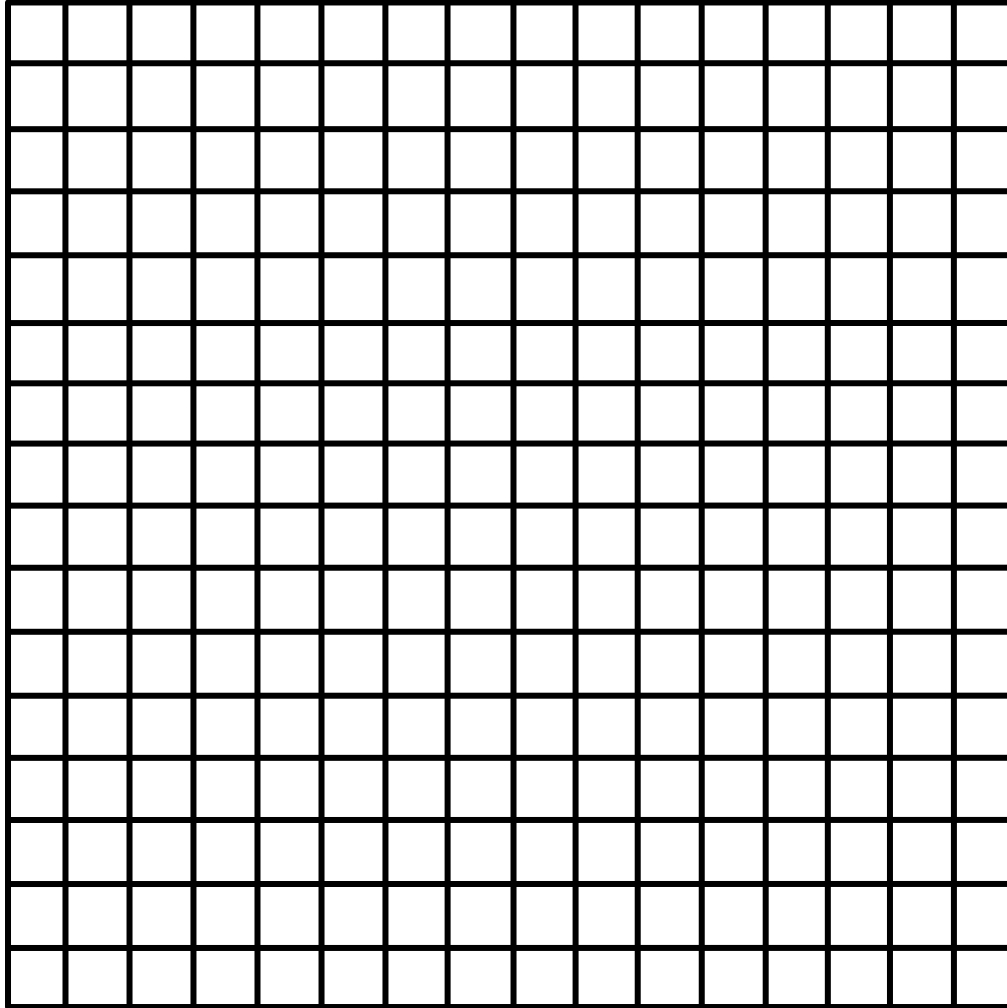
Which color do we fill each pixel with?



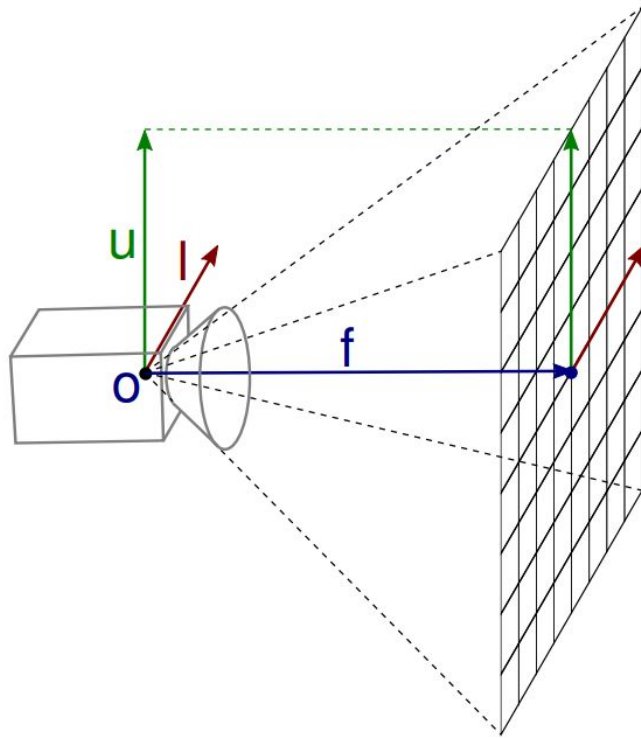
$R = 0.25$ $G = 0.5$ $B = 0.45$

But how?

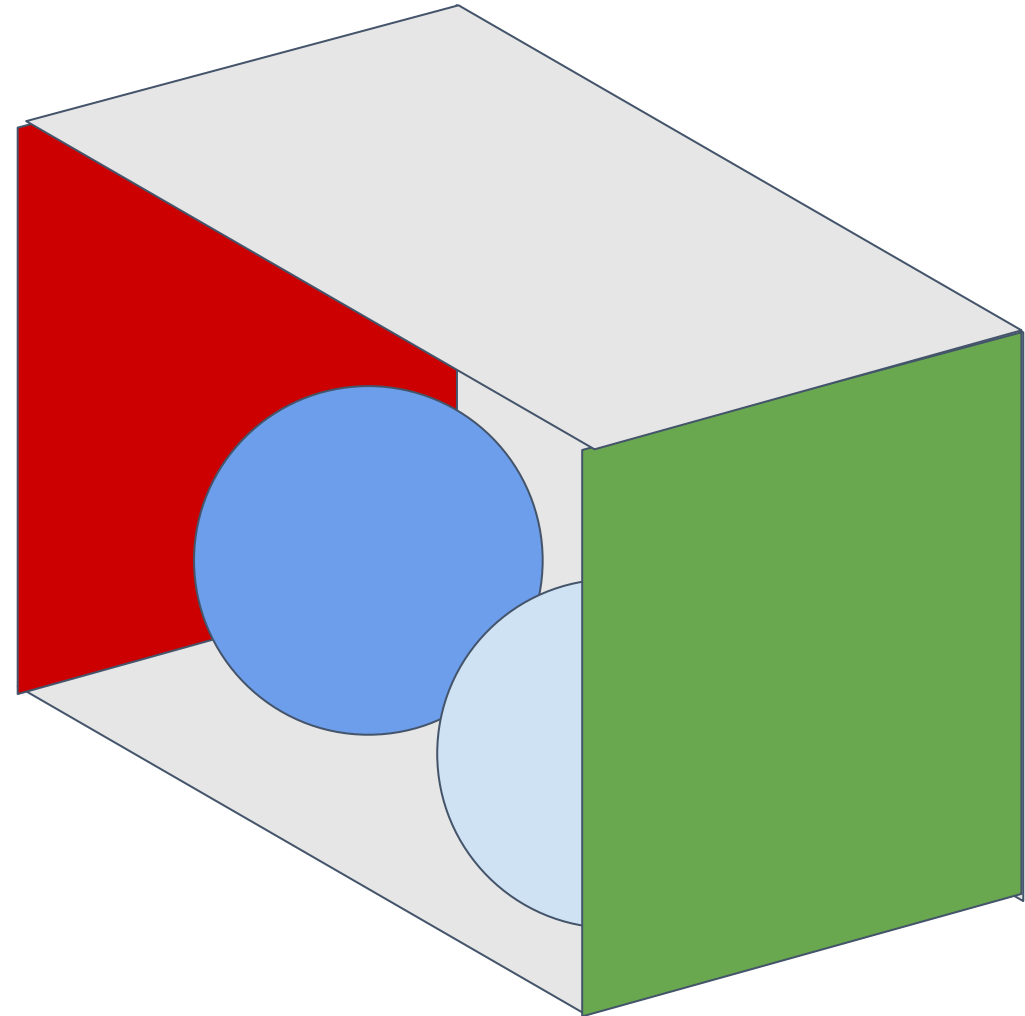
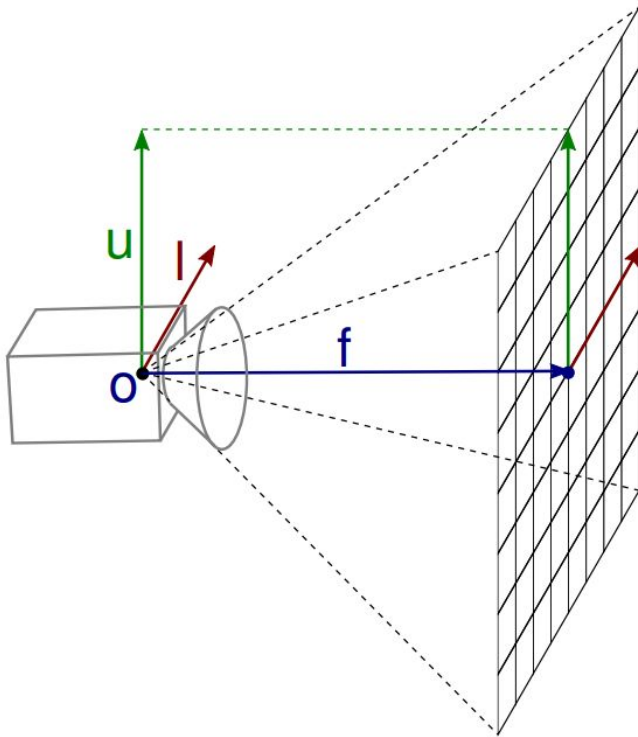
Which color do we fill each pixel with?



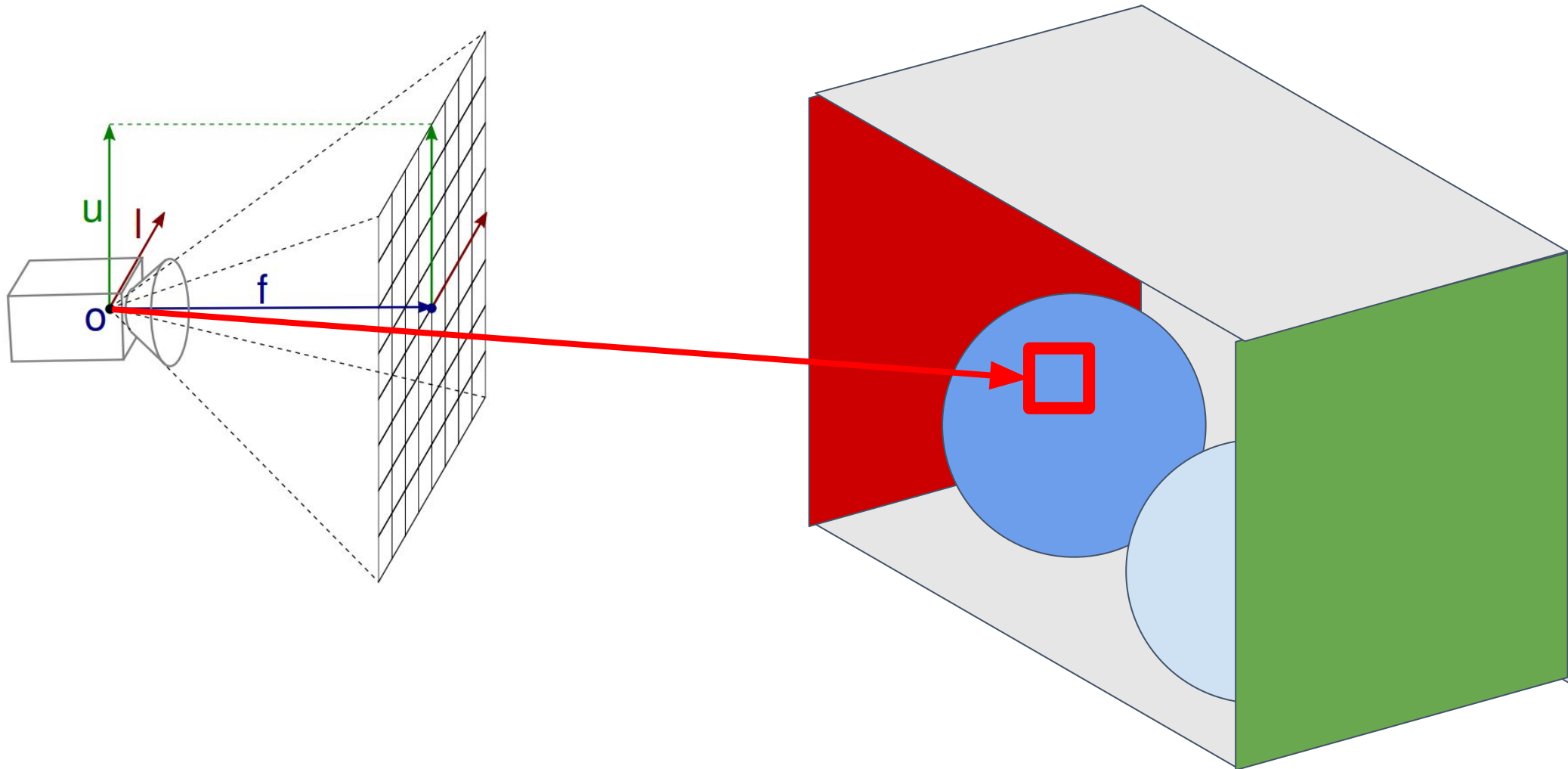
Which color do we fill each pixel with?



Which color do we fill each pixel with?

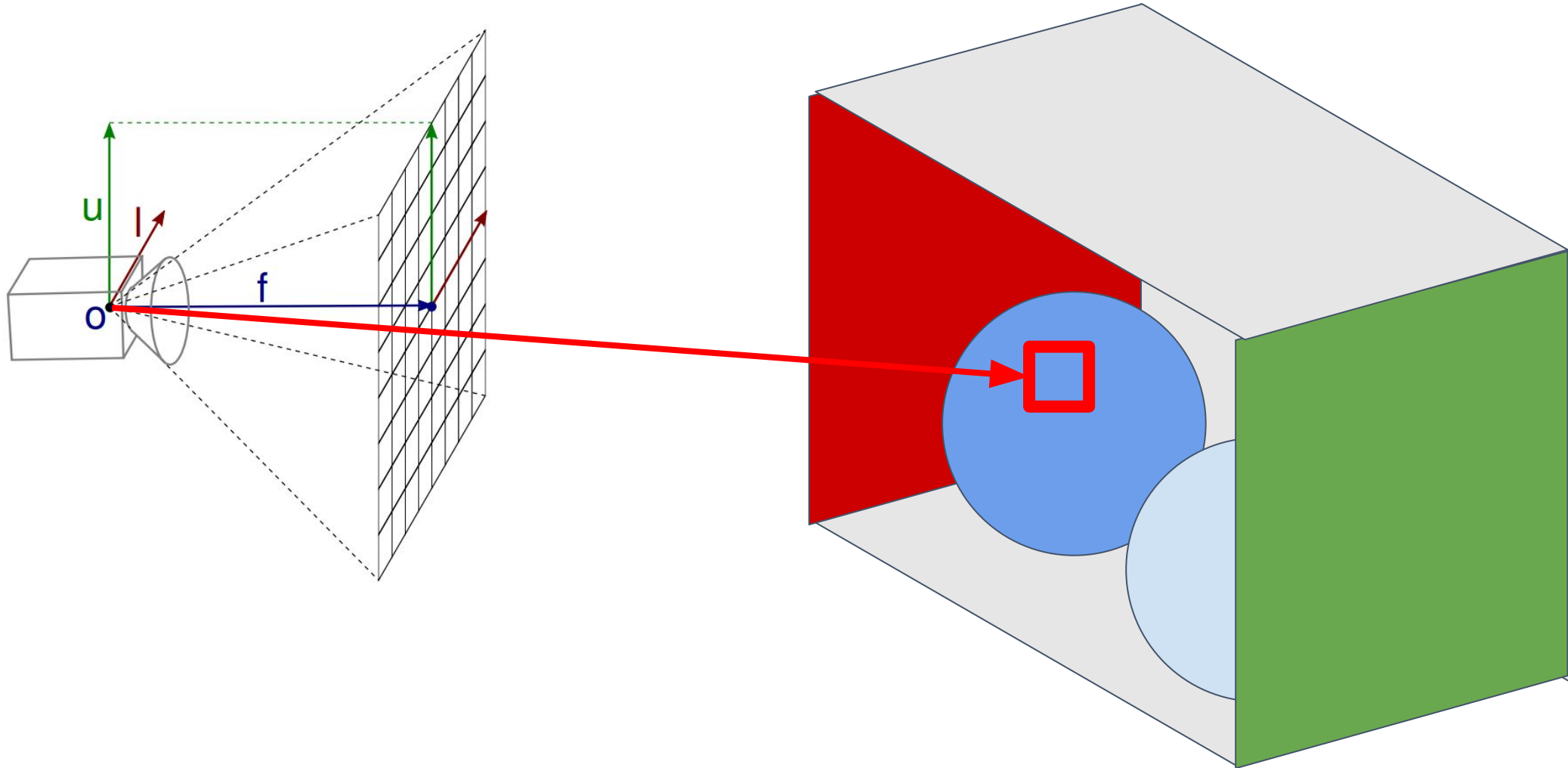


Which color do we fill each pixel with?



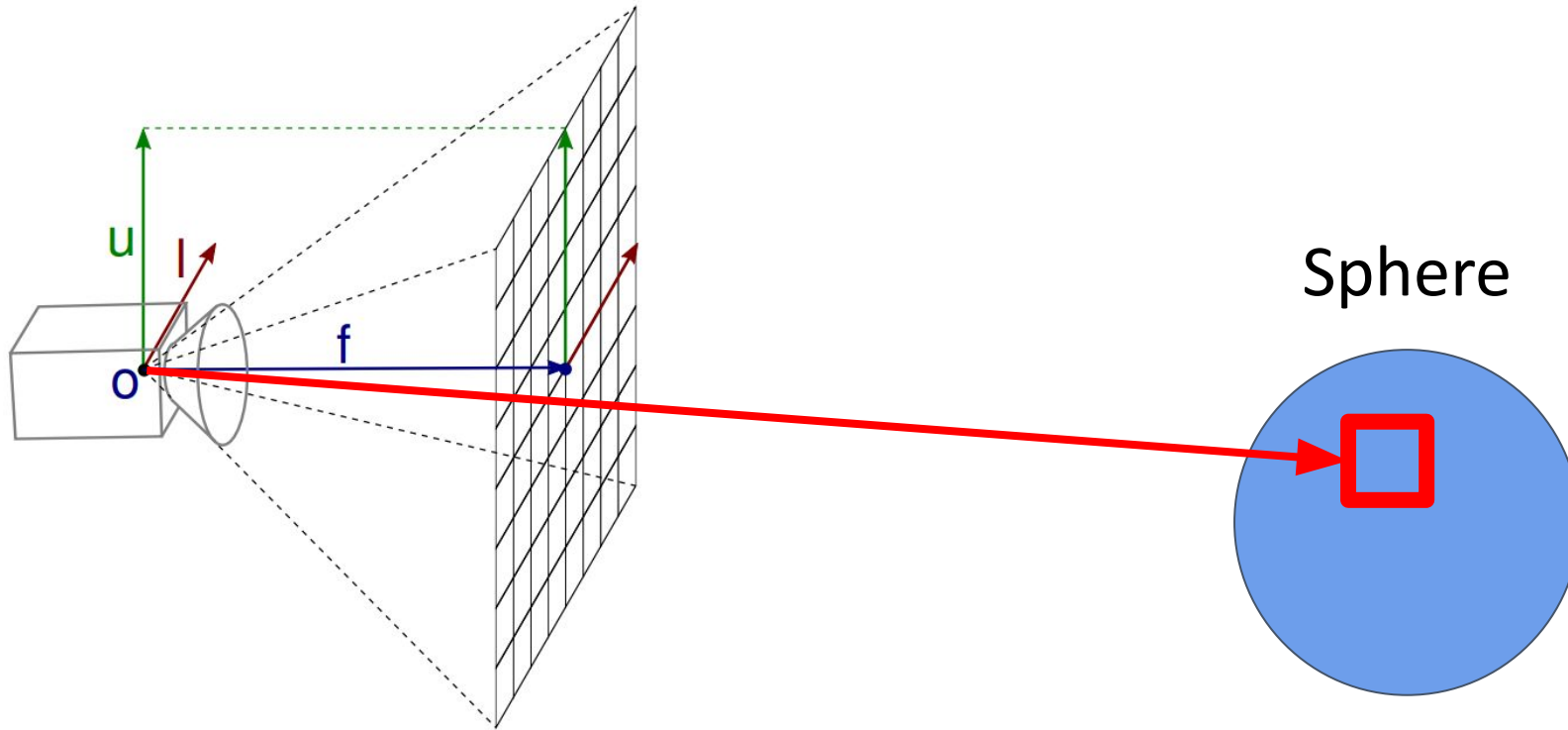
Which color do we fill each pixel with?

- Let's simplify



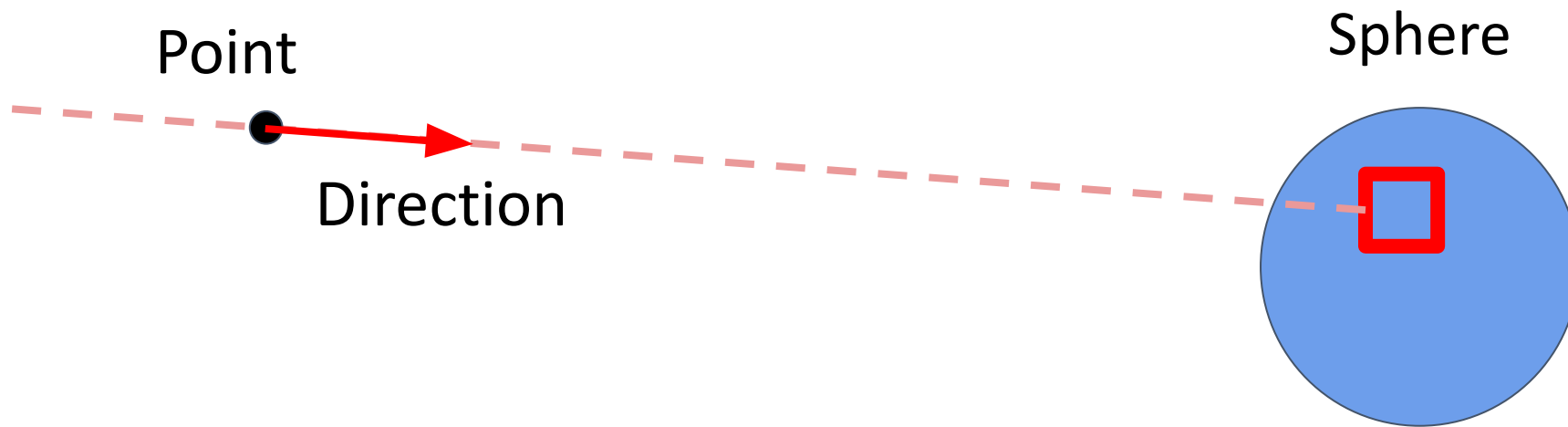
Which color do we fill each pixel with?

- Let's simplify



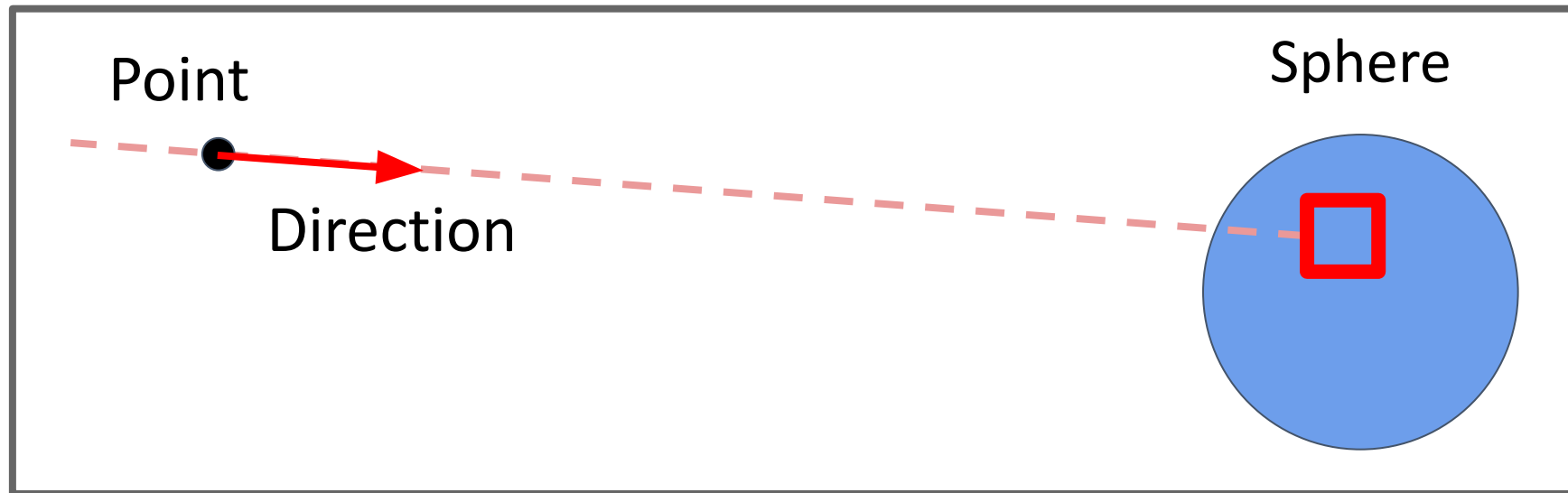
Which color do we fill each pixel with?

- Let's simplify



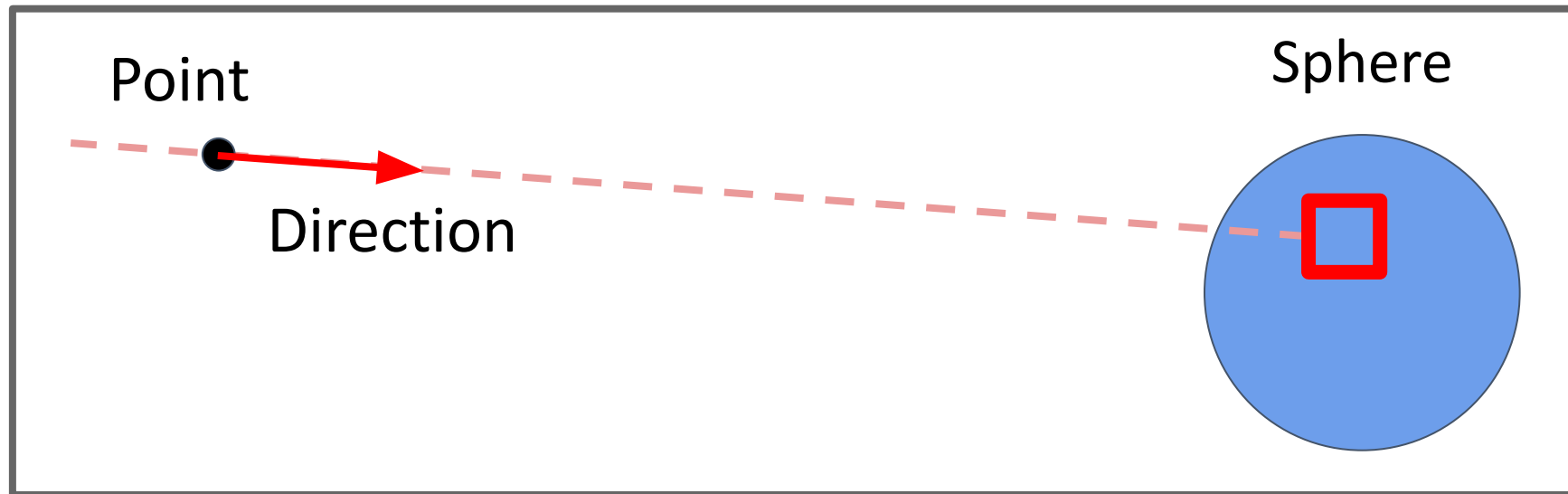
Ray intersection problem

- This session we focus on ray intersections
- Inputs: ray (origin + direction), geometry (spheres or planes)



Ray intersection problem

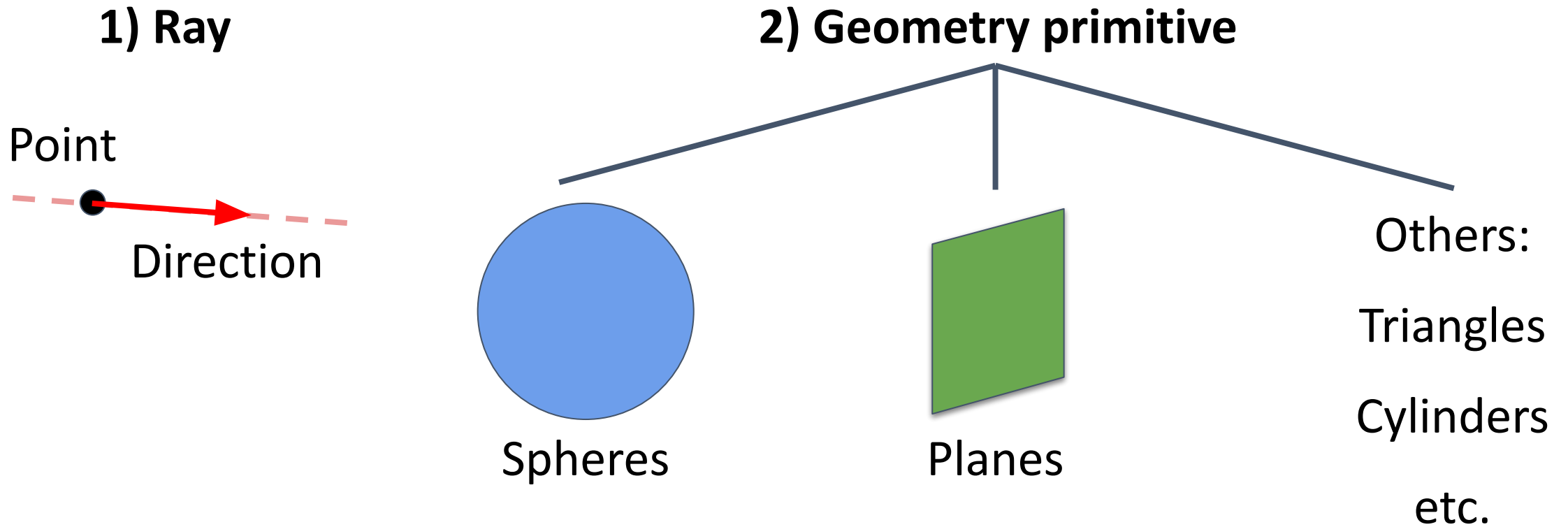
- This session we focus on ray intersections
- Inputs: ray (origin + direction), geometry (spheres or planes)



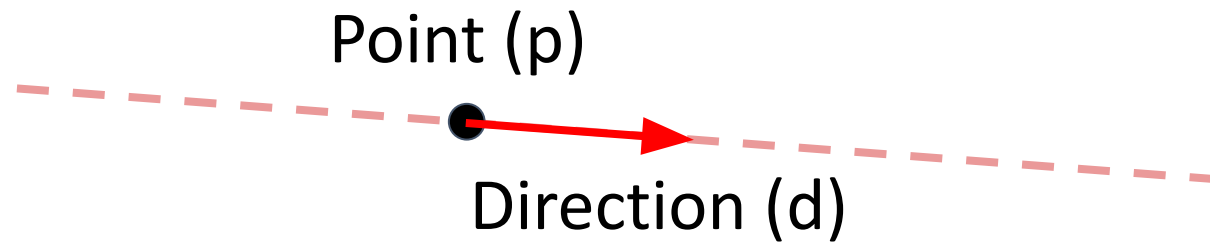
- Output: does the ray intersect? if yes, where?
 - Additional information: travel distance, surface normal, etc.

Inputs

- Your intersection function should have at least two inputs:



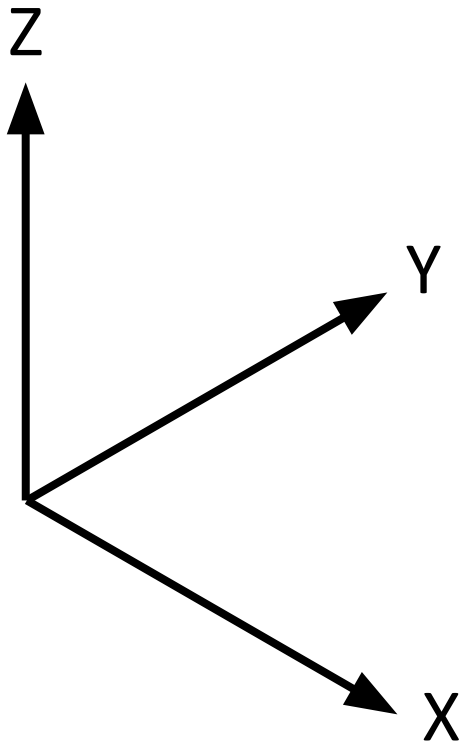
- How to define a ray:



- The line  is defined by
 - $p + d \cdot t$
 - $t \in \mathbb{R}$

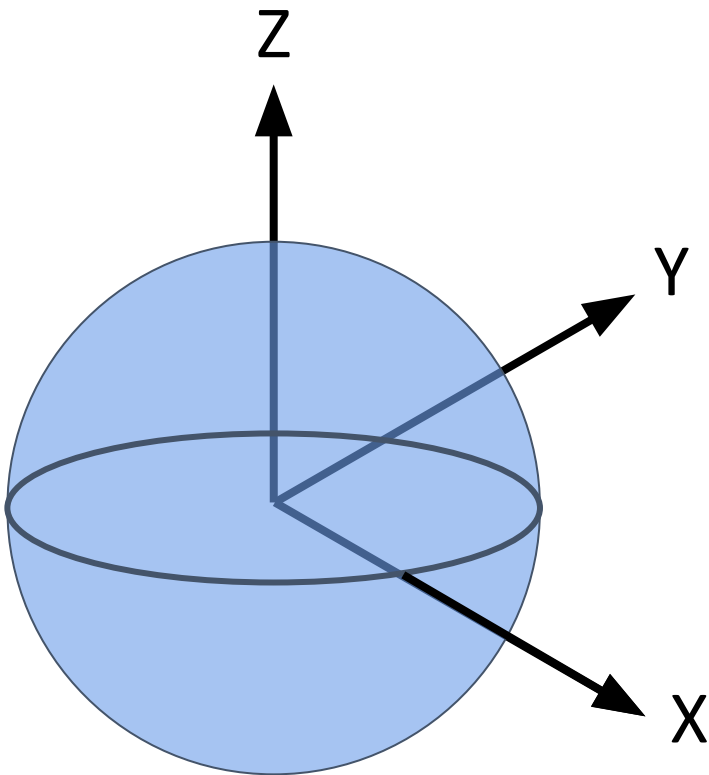
- How to define a geometric primitive:
 - Implicit equation

- How to define a geometric primitive:
 - Implicit equation $f(x, y, z)$



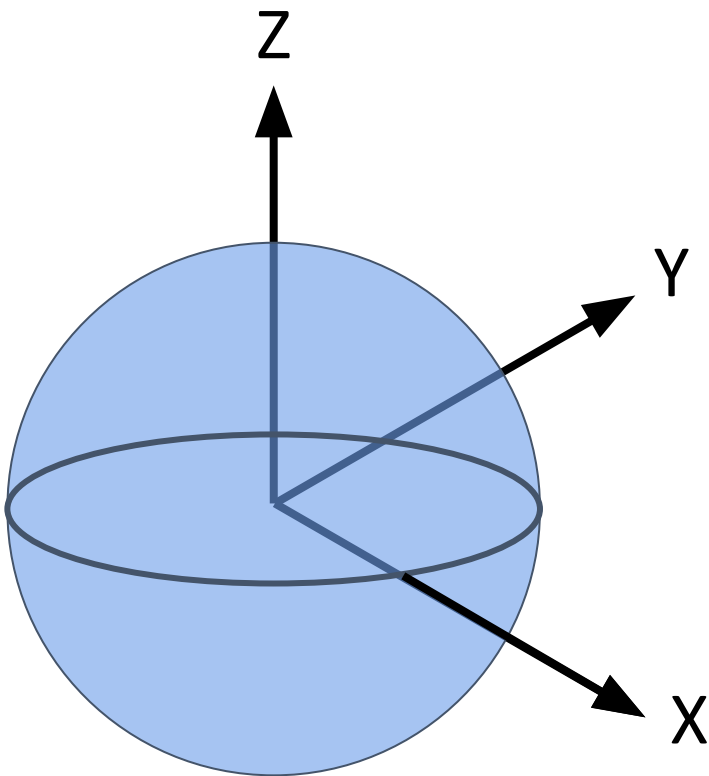
- The surface of the geometry is defined by all points (x, y, z) such that $f(x, y, z) = 0$

- How to define a geometric primitive:
 - Implicit equation $f(x, y, z)$



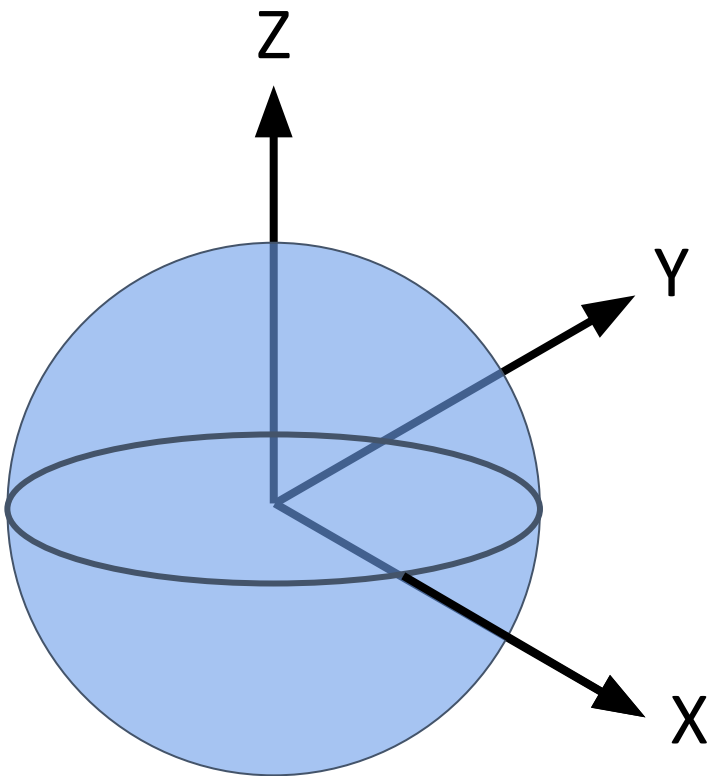
- The surface of the geometry is defined by all points (x, y, z) such that $f(x, y, z) = 0$
- Sphere:
 - Defined by center (c_x, c_y, c_z) and radius (r)
 - $f(x, y, z) = (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 - r^2$

- How to define a geometric primitive:
 - Implicit equation $f(x, y, z)$



- $f(x, y, z) = (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 - r^2$
- Points in the ray: $p + d \cdot t$

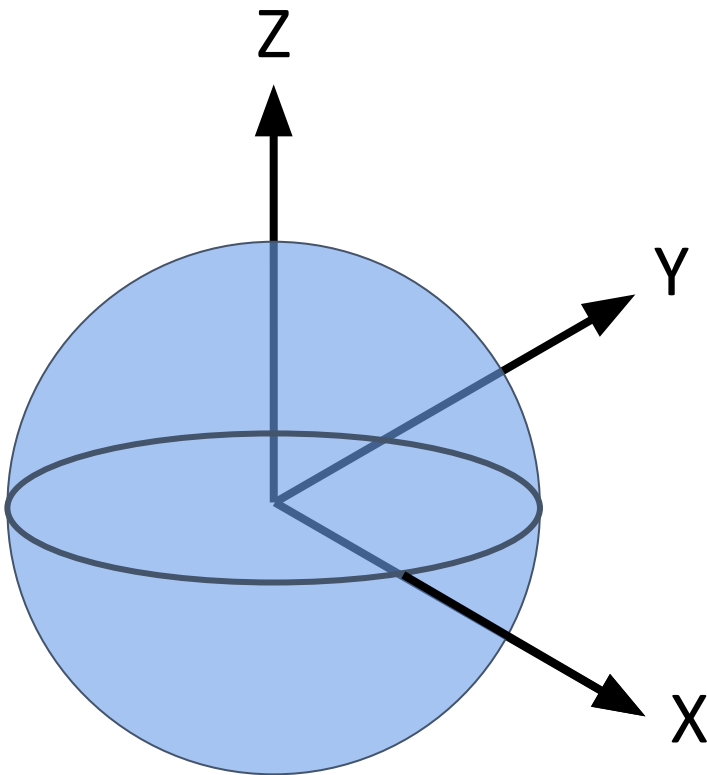
- How to define a geometric primitive:
 - Implicit equation $f(x, y, z)$



- $f(x, y, z) = (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 - r^2$
- Points in the ray: $p + d \cdot t$
- Intersection at:
 - $f(p + d \cdot t) = 0$
 - Solve for t
 - Can have 0, 1 or 2 solutions

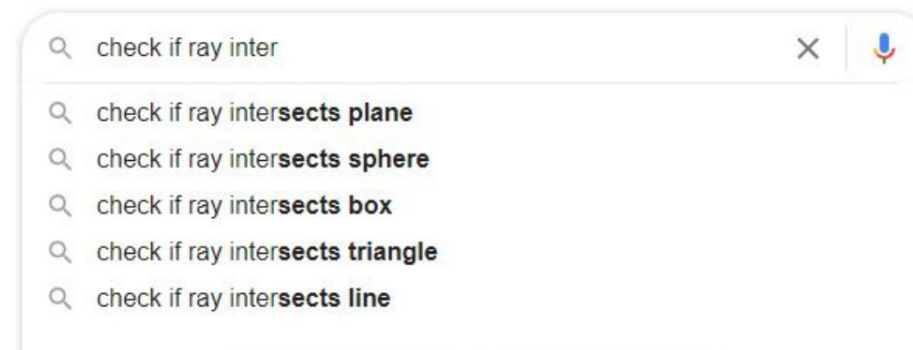
Inputs

- How to define a geometric primitive:
 - Implicit equation $f(x, y, z)$



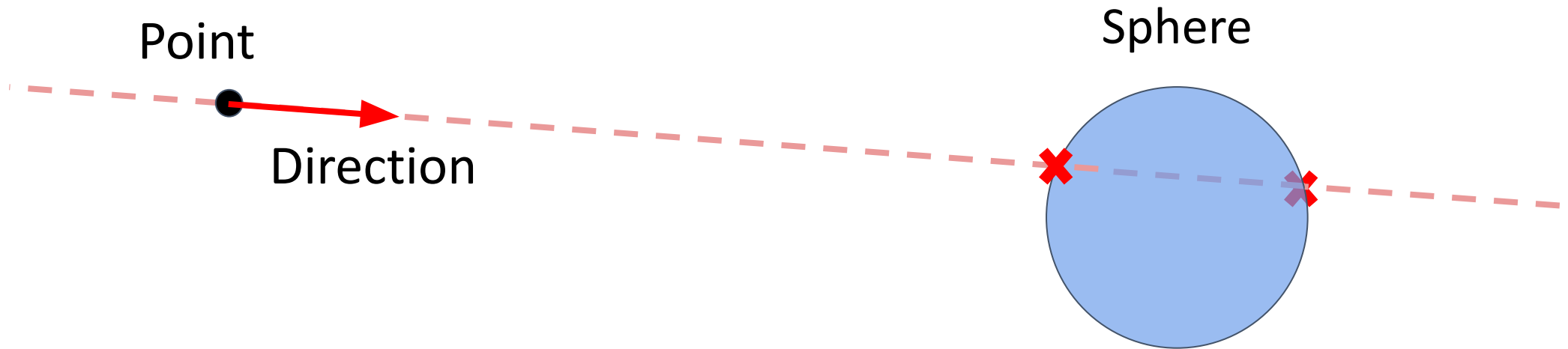
- $f(x, y, z) = (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 - r^2$
- Points in the ray: $p + d \cdot t$

Google



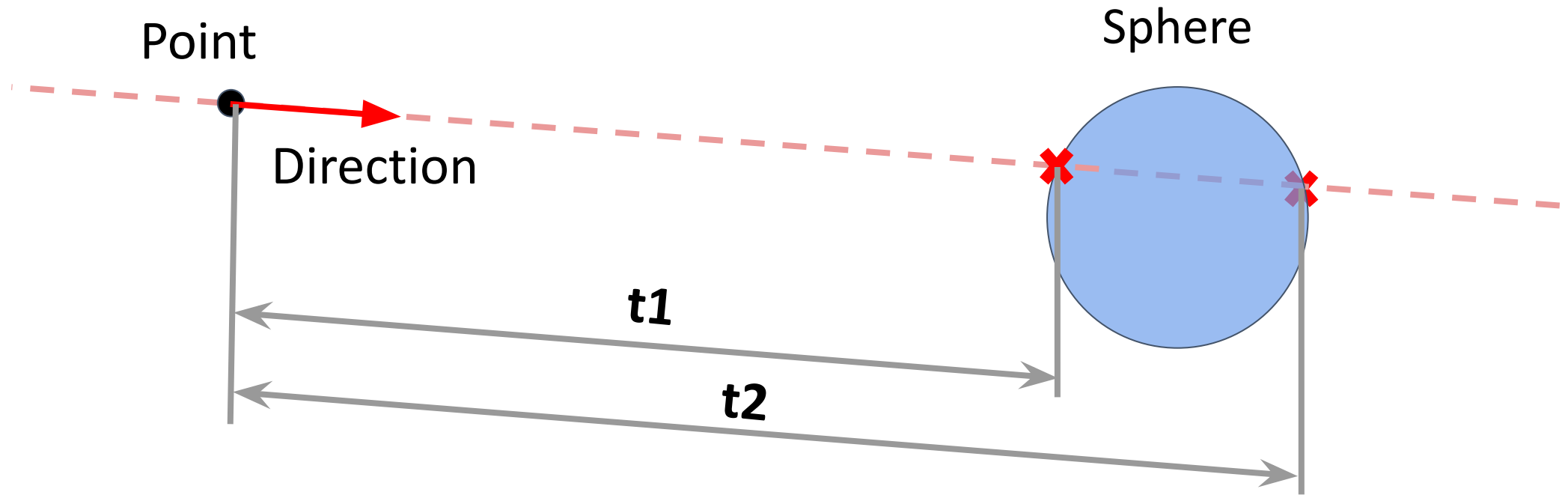
Outputs

- Solving $p + d \cdot t$ gives the distance to the sphere



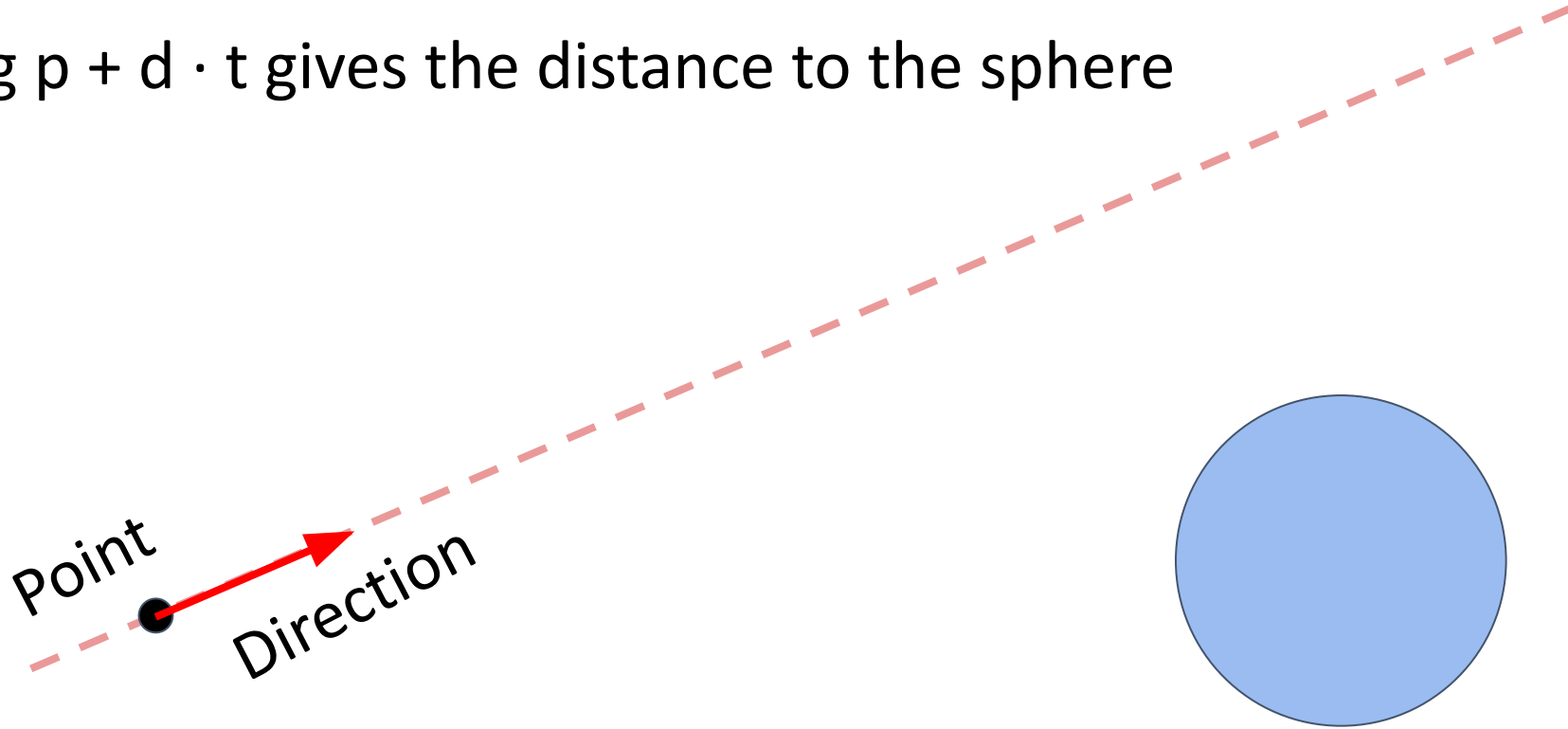
Outputs

- Solving $p + d \cdot t$ gives the distance to the sphere



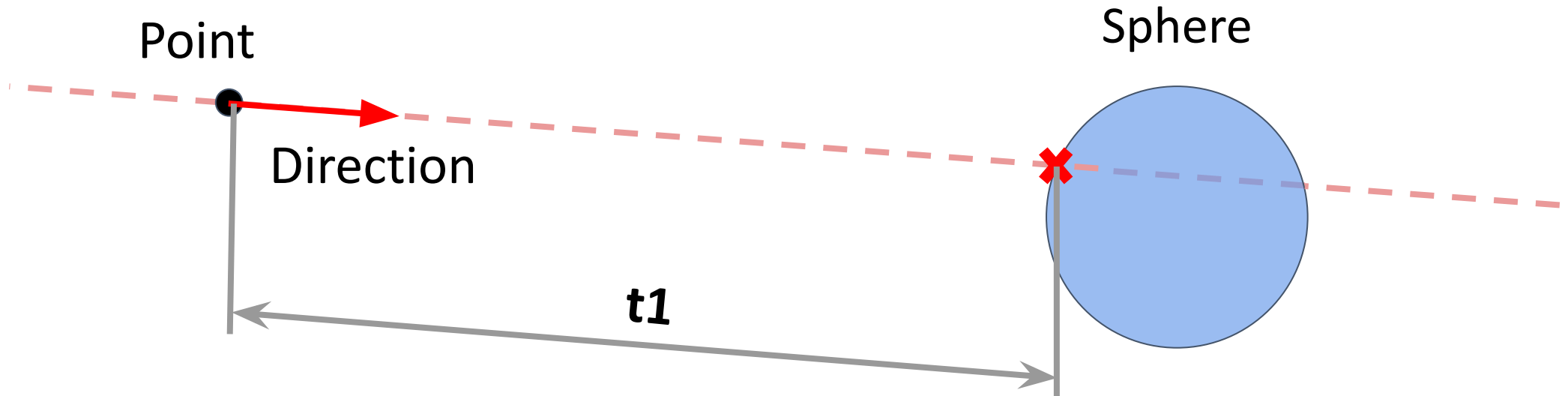
Outputs

- Solving $p + d \cdot t$ gives the distance to the sphere



Outputs

- Solving $p + d \cdot t$ gives the distance to the sphere



- Distance: $t1$
- Intersection point: $p + d \cdot t1$
- Surface normal at $t1$, etc.

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice, implement:

- **Primitives:** spheres and planes
- **Ray:** point + direction + other stuff you may want
- One **intersect function** per type of primitive
 - Inputs: Ray, one primitive
 - Output: Does the ray intersect? If yes, where? + additional info
- Recommended deadline: **October 18th.**
 - **Next lab:** define a camera and generate an image
 - **Extensions** (do not count towards deadline):
 - Other primitives: cones, cylinders, ellipsoids, disks or triangles
 - Acceleration structures: bounding volumes, multi-threading, etc.
 - Constructive solid geometry: google it or ask us :)

Lab #3 - Ray tracing (part 2)

Informática Gráfica

Adolfo Muñoz - Julio Marco

Pablo Luesia - J. Daniel Subías – Óscar Pueyo

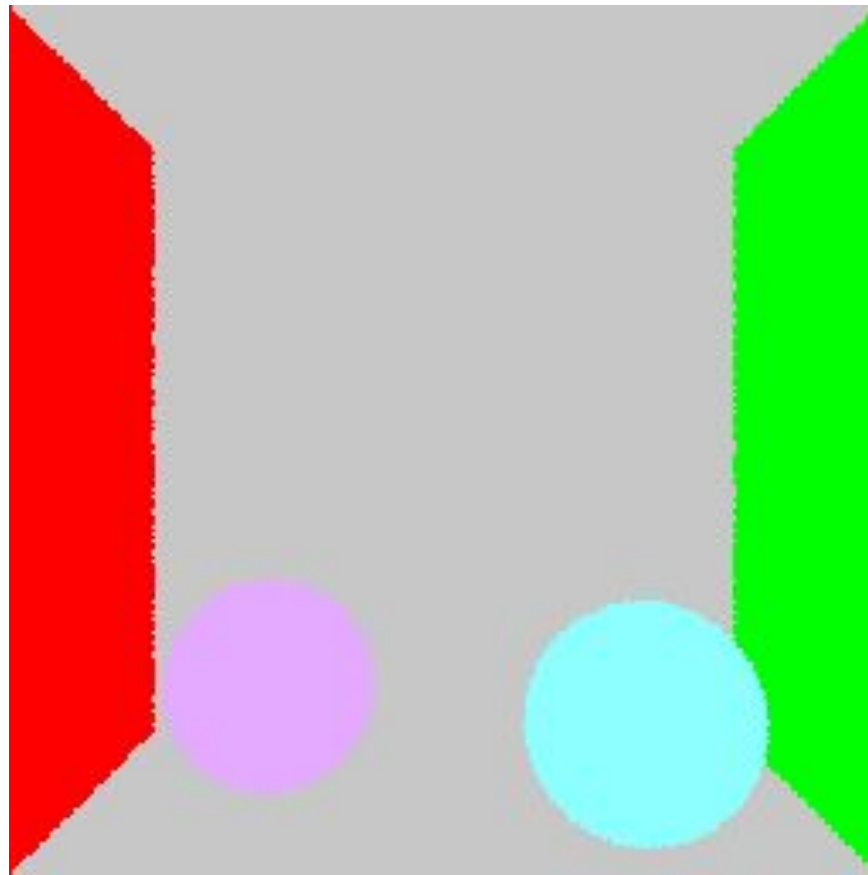


Before we begin...

- You can generate your first image today :)

Before we begin...

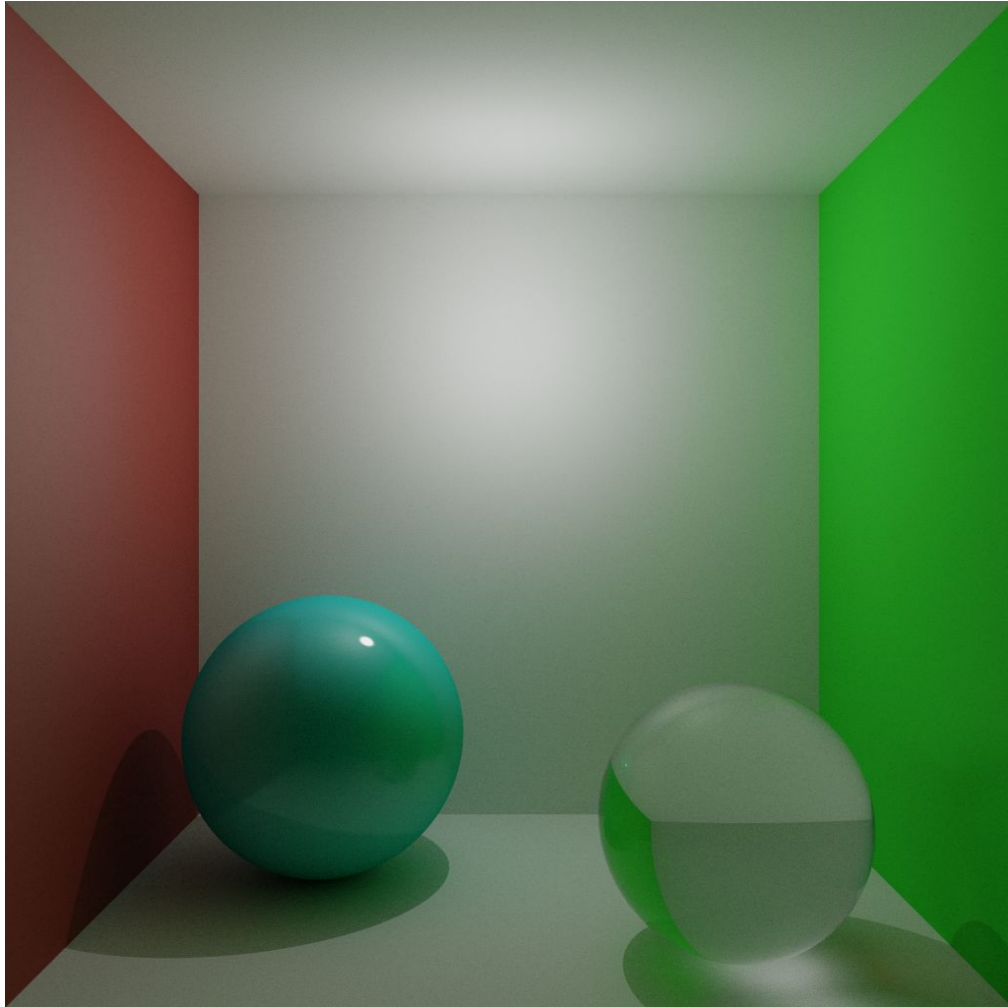
- You can generate your first image today :)



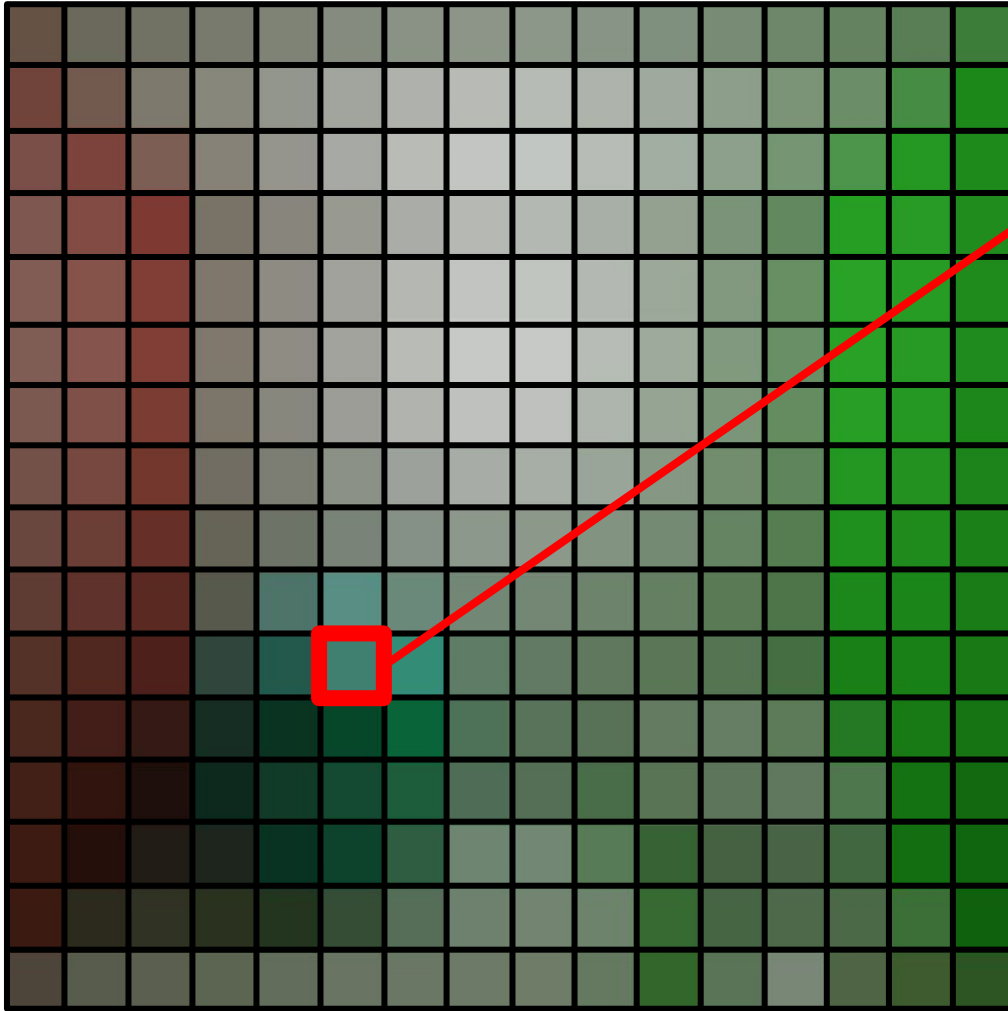
Before we begin...

- You can generate your first image today :)
- Careful with the submissions
 - Lab 3 (ray tracing) does not need to be submitted
 - Recommended deadline: **October 18th**
 - Lab 4 (path tracing) **will be submitted** at the end of the course
 - All of the code you write today will be used for Lab 4
 - Recommended deadline: November 13th (moodle: January 11th, this is only a recommendation)
- Remember: Final work is 80% of the final grade

Which color do we fill each pixel with?



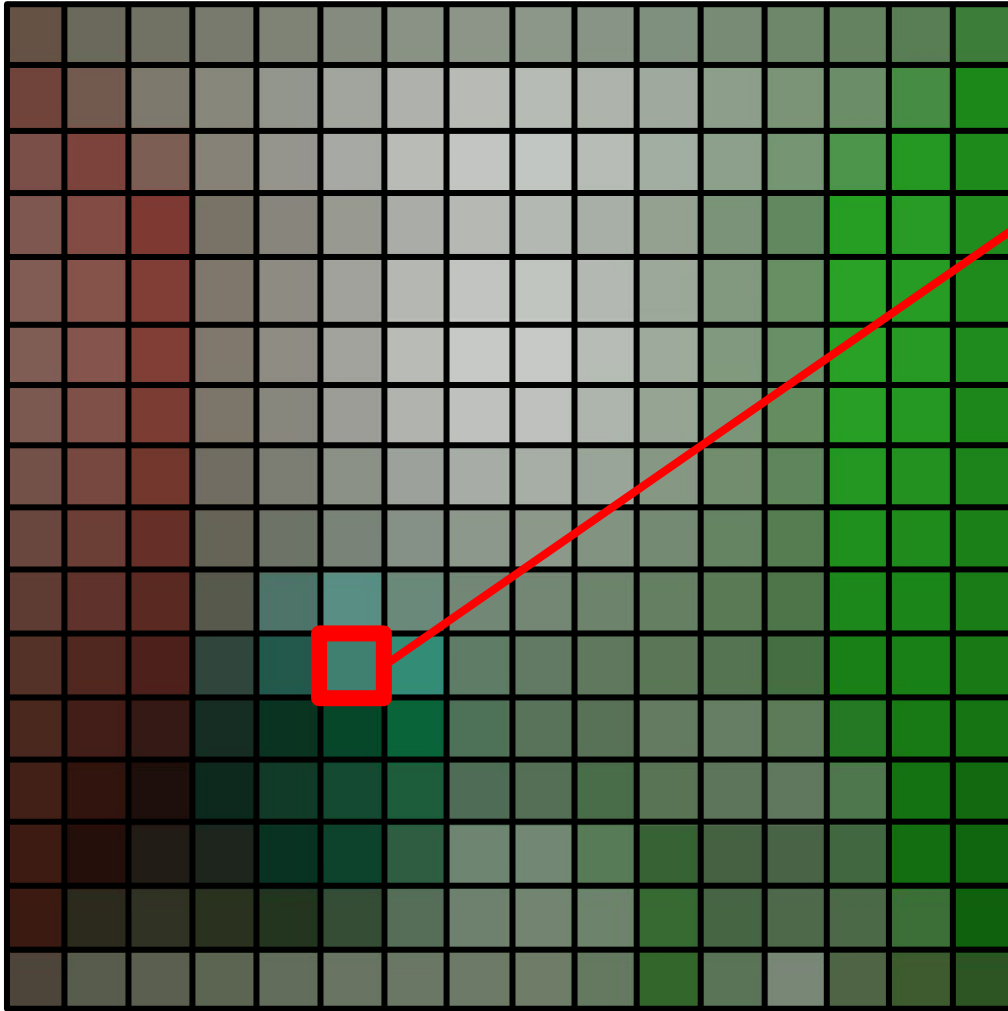
Which color do we fill each pixel with?



$R = 0.25$ $G = 0.5$ $B = 0.45$

But how?

Which color do we fill each pixel with?

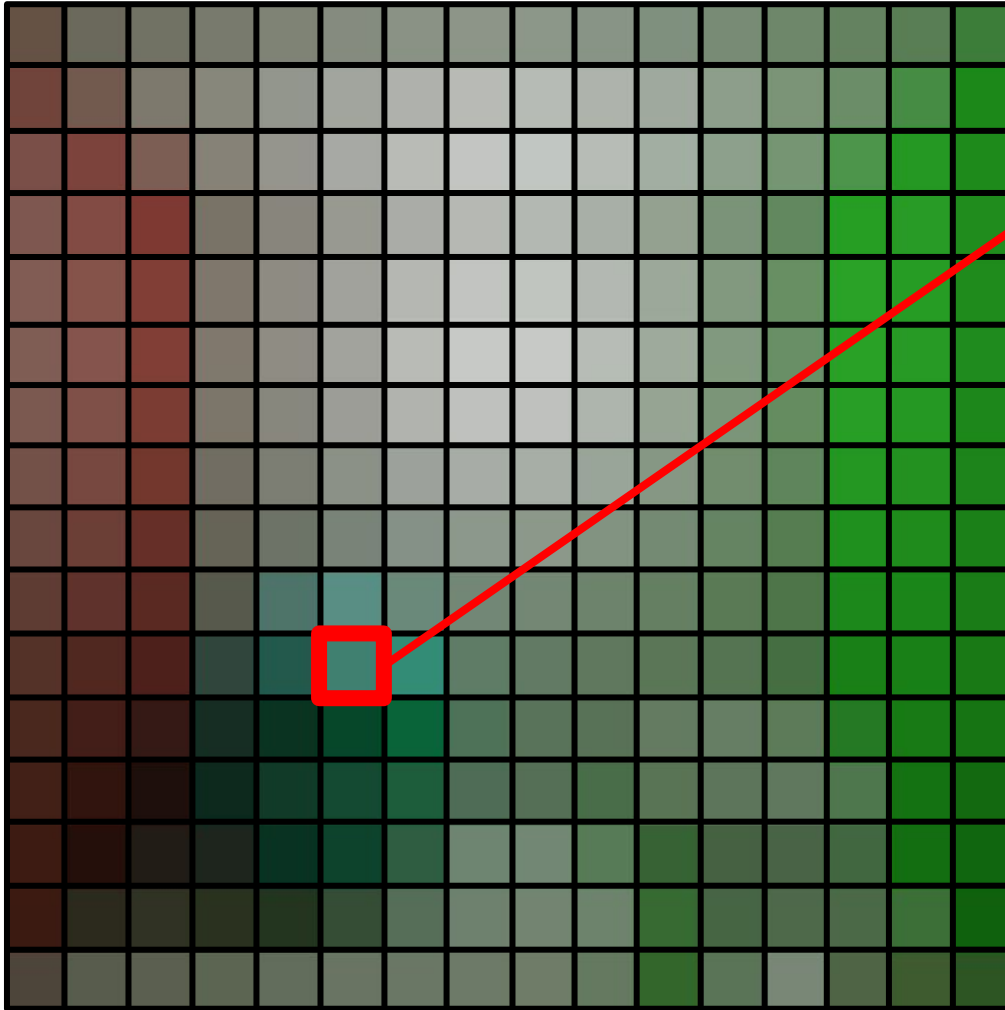


$R = 0.25 \quad G = 0.5 \quad B = 0.45$

But how?

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Which color do we fill each pixel with?



R = 0.25 G = 0.5 B = 0.45

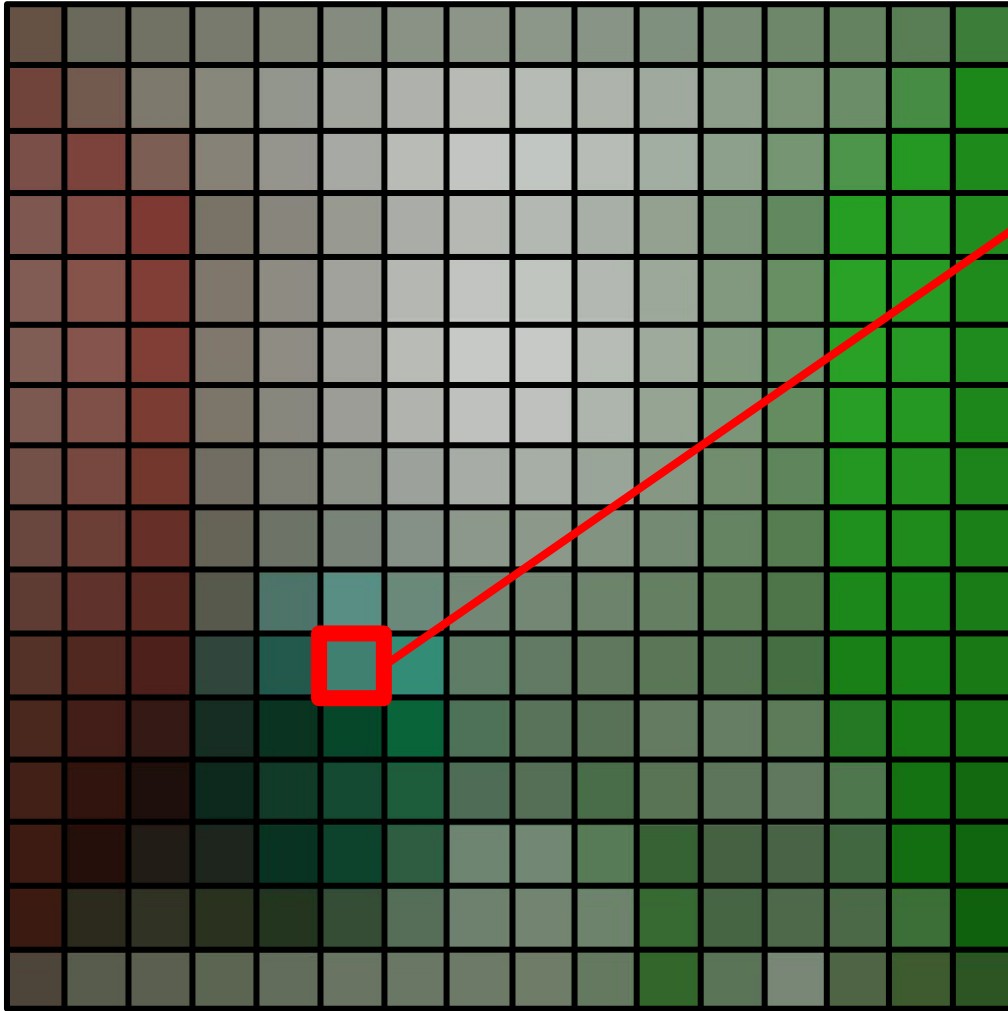
But how?

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

In this lab,

we simplify the render equation:

Which color do we fill each pixel with?



R = 0.25 G = 0.5 B = 0.45

But how?

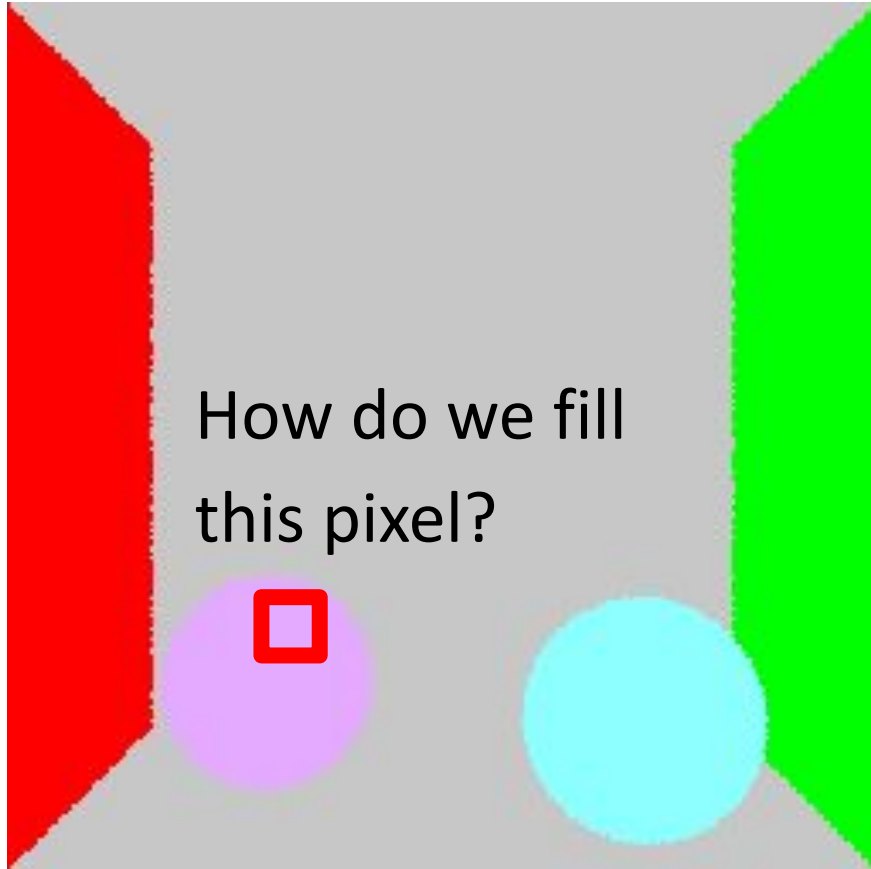
$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

In this lab,

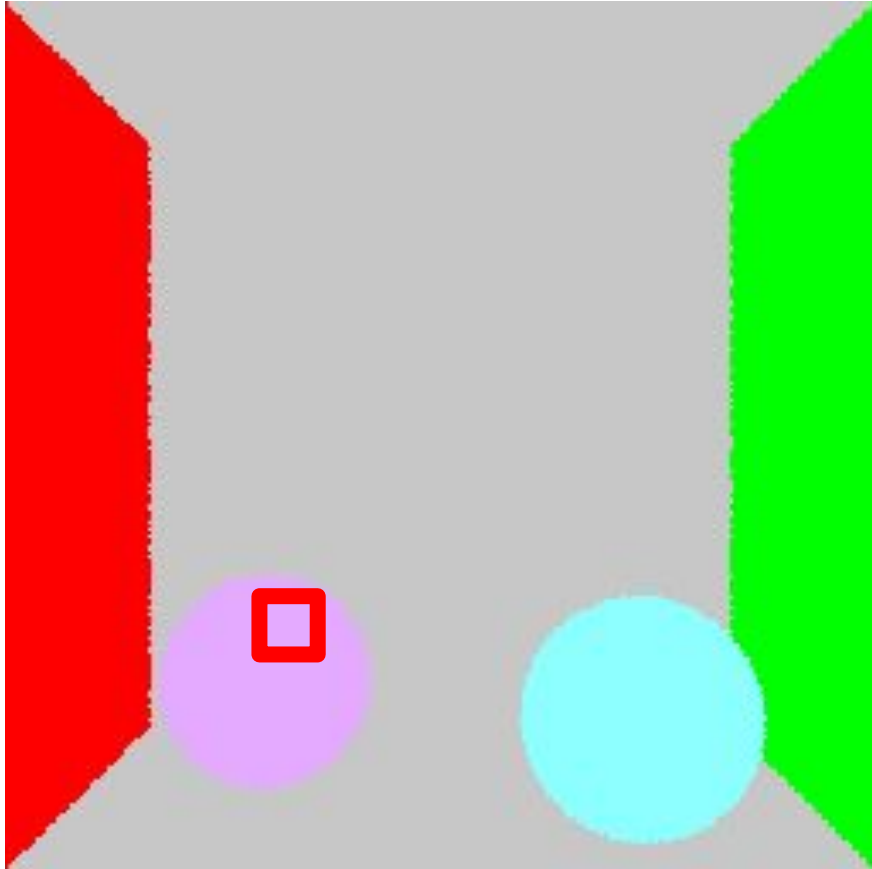
we simplify the render equation:

Everything is an area light

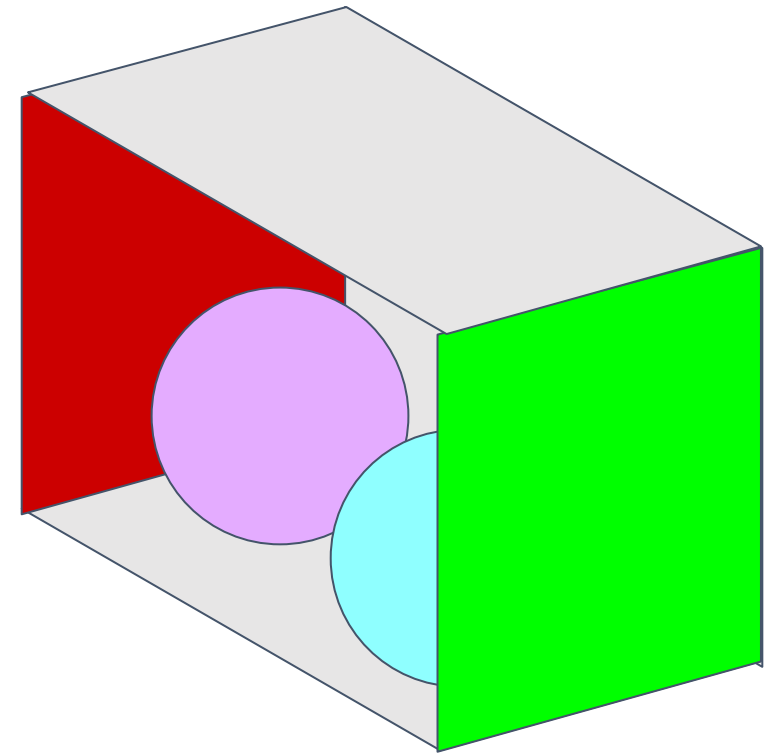
Which color do we fill each pixel with?



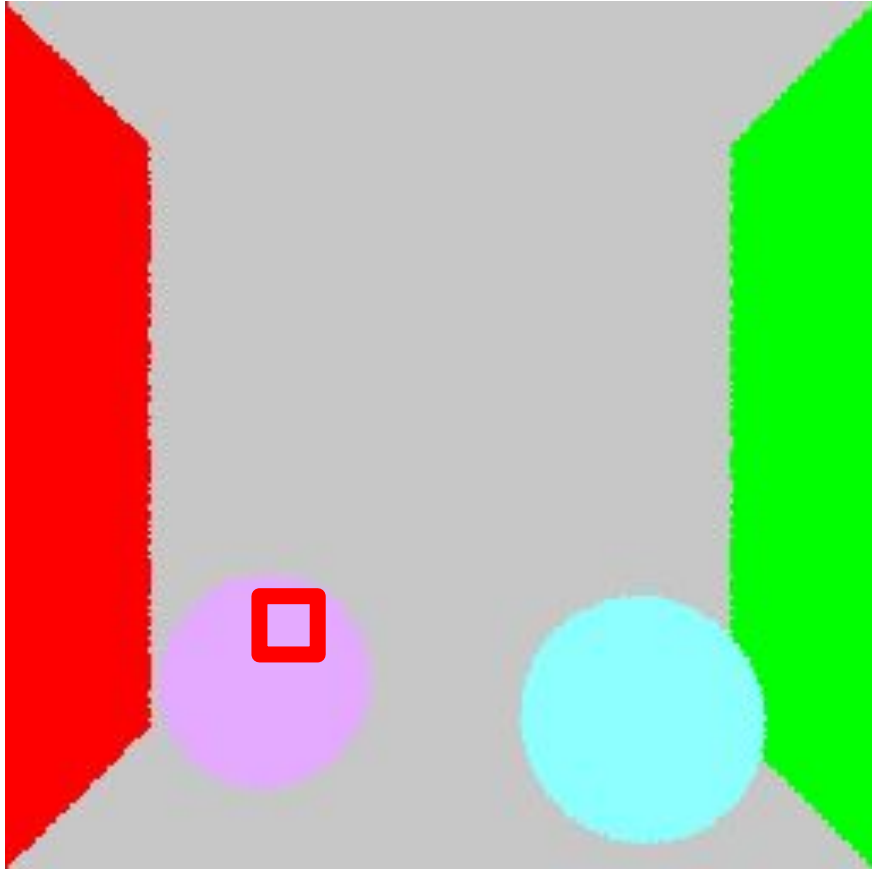
Which color do we fill each pixel with?



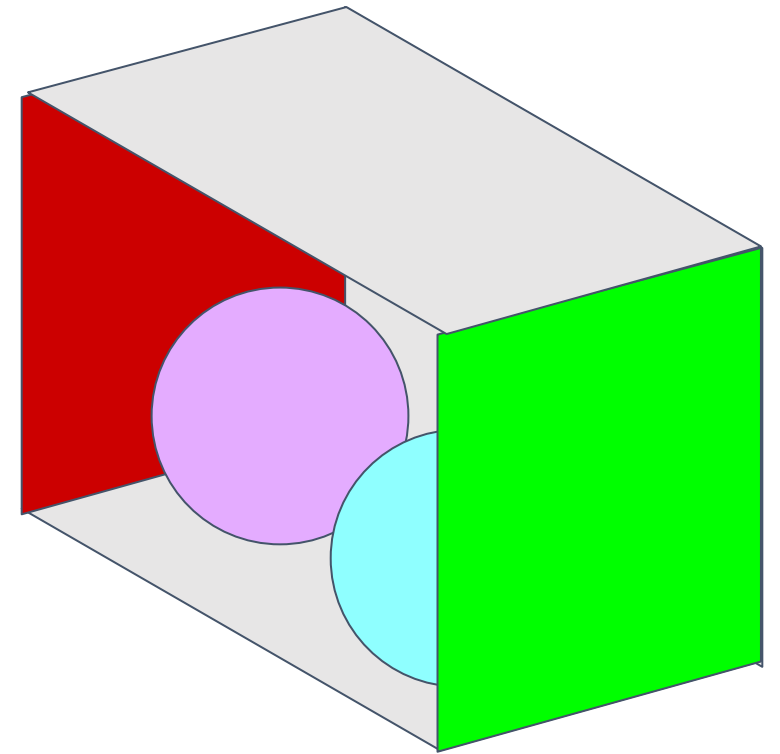
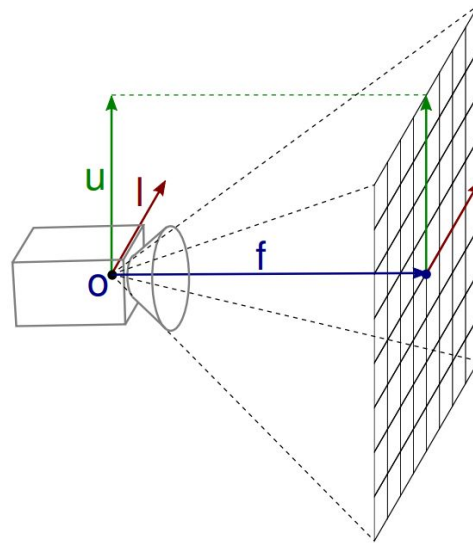
Define your scene's geometry (planes, spheres)



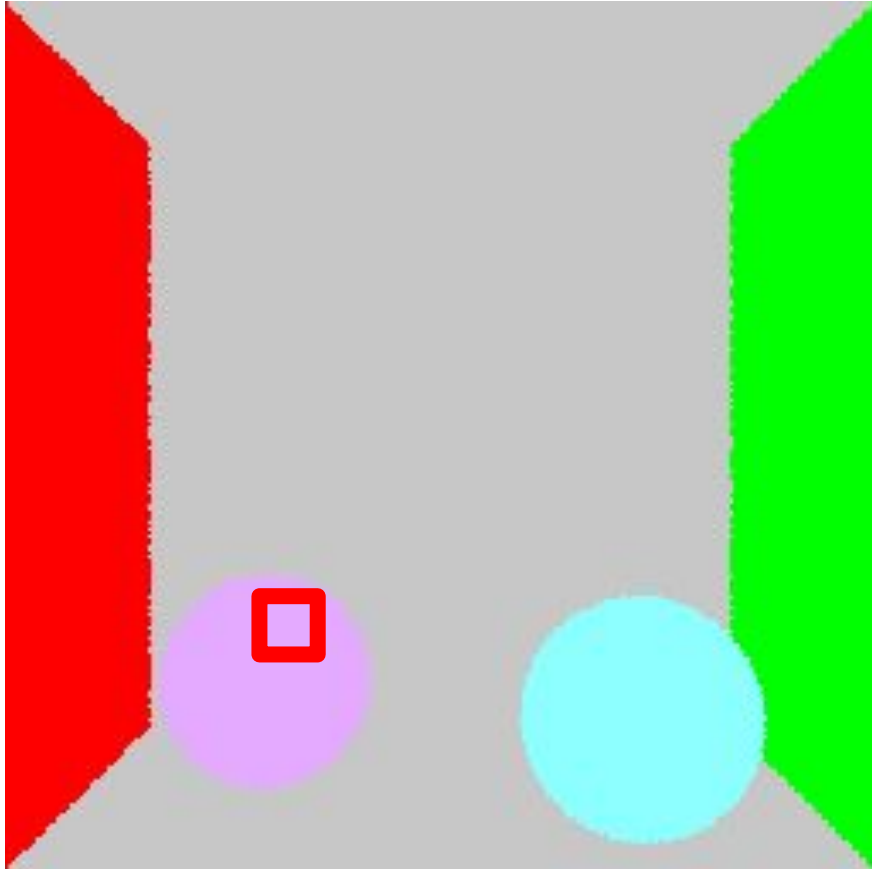
Which color do we fill each pixel with?



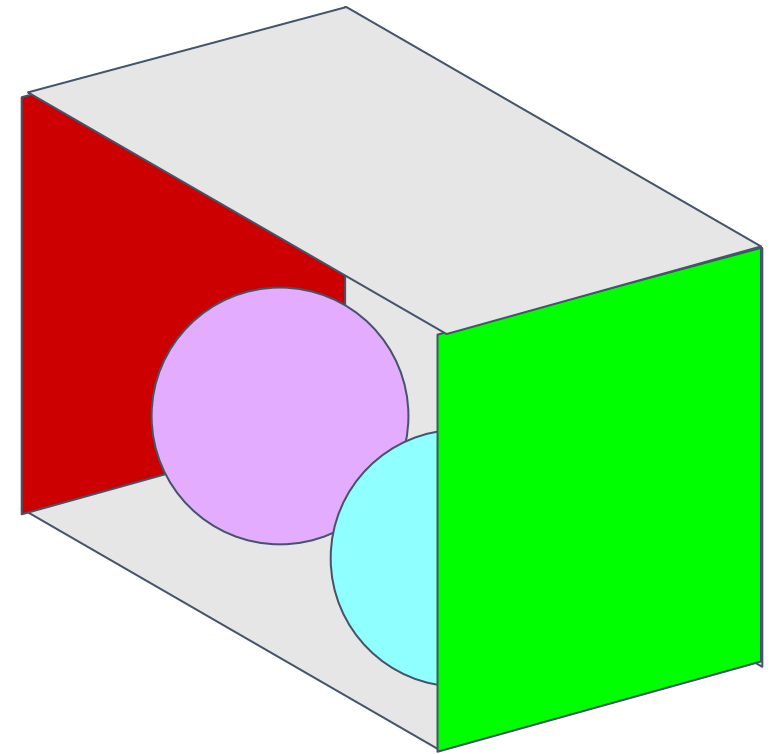
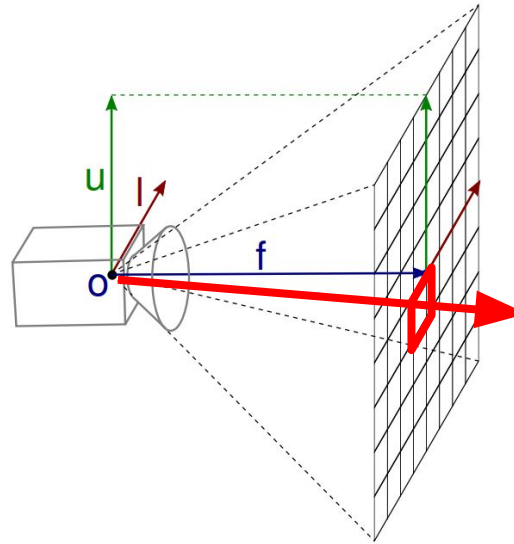
Define your scene's camera (image plane, film)



Which color do we fill each pixel with?

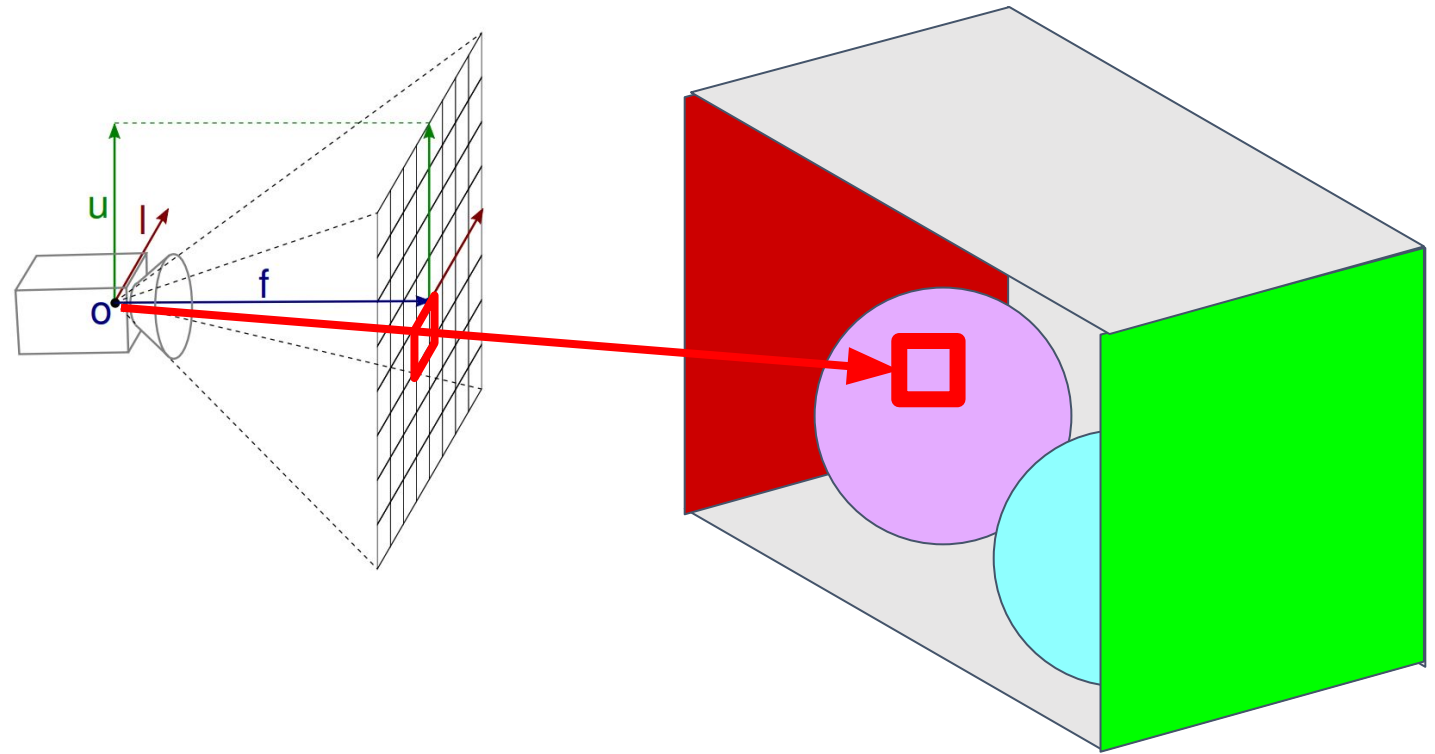
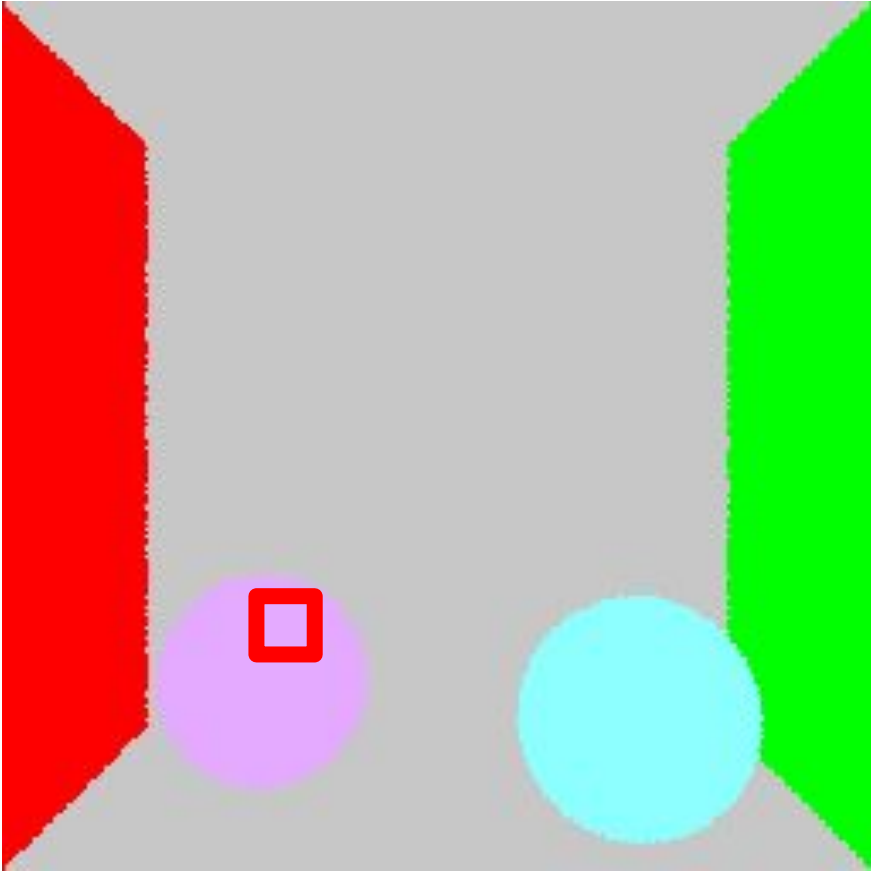


Generate a ray → that passes through 

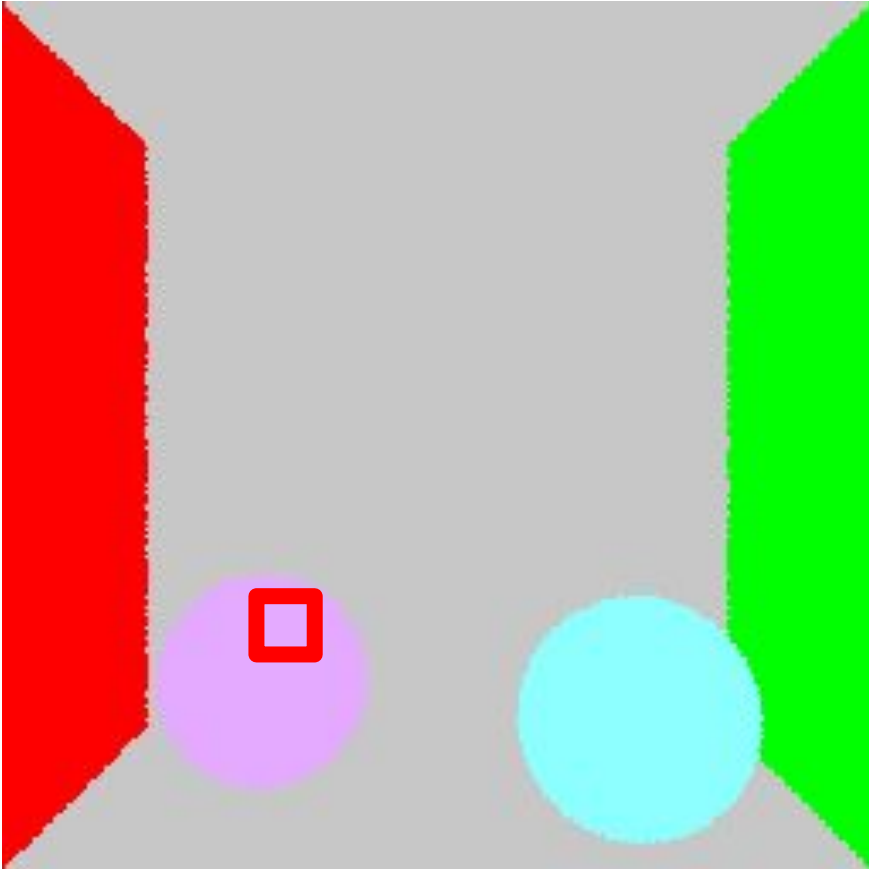


Which color do we fill each pixel with?

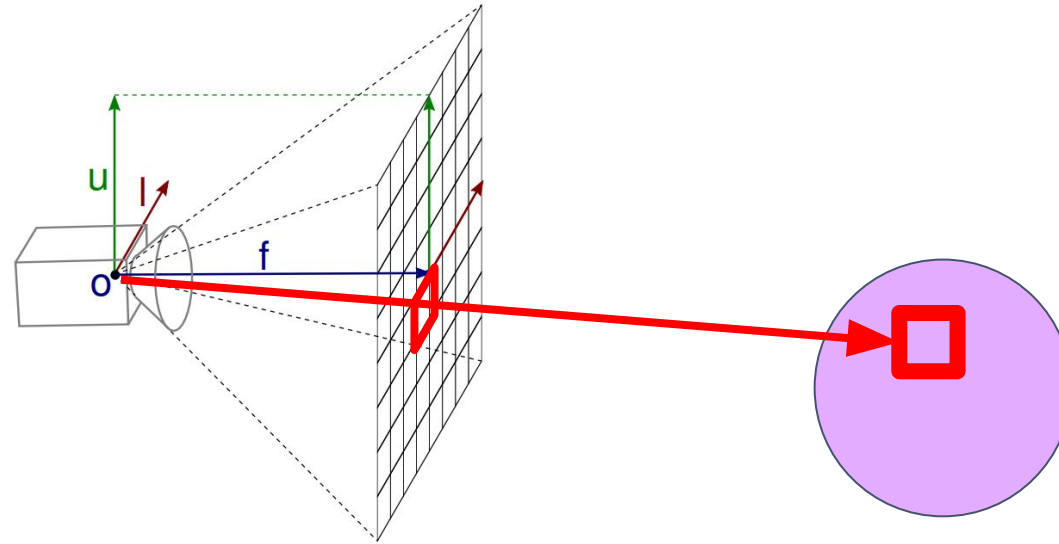
Intersect this ray with your scene



Which color do we fill each pixel with?



Return your sphere's emission color
(everything is an area light in this lab)

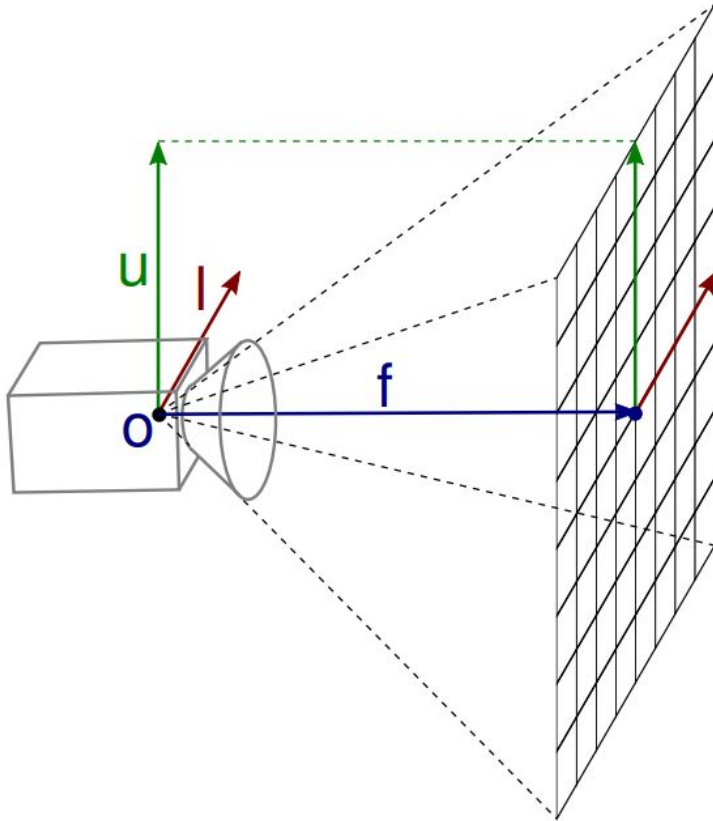


- Center: (x, y, z)
- Radius: d
- Emission: (r, g, b)

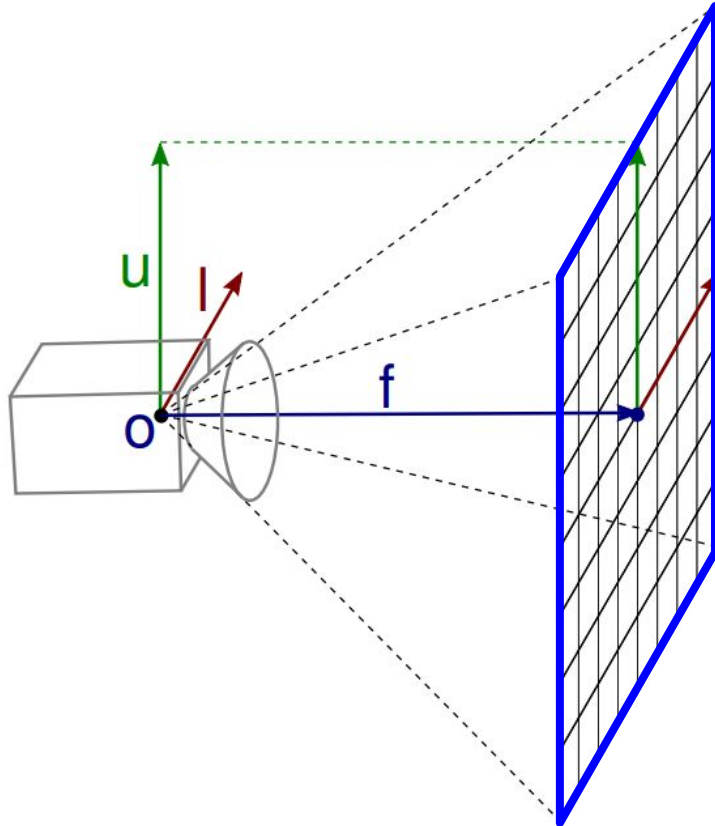
Considerations about the camera

A camera is defined by:

- Local coordinate system
(left L, up U, forward F, origin O)



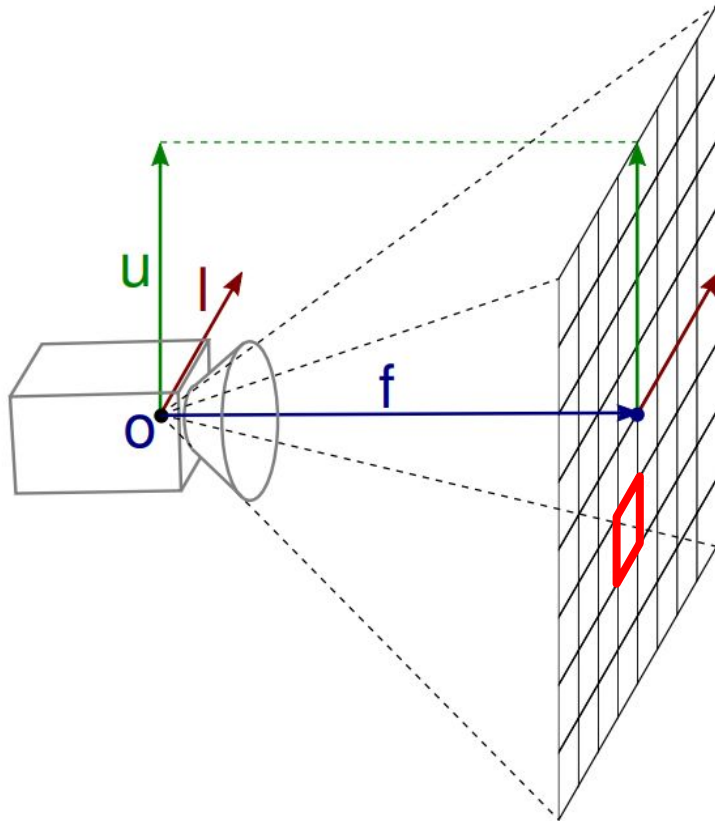
Considerations about the camera



A camera is defined by:

- Local coordinate system
(left **L**, up **U**, forward **F**, origin **O**)
- Forward, up & left also define the image plane
 - Divided in pixels (width, height)
 - Pixels should be square (not necessarily)

Considerations about the camera

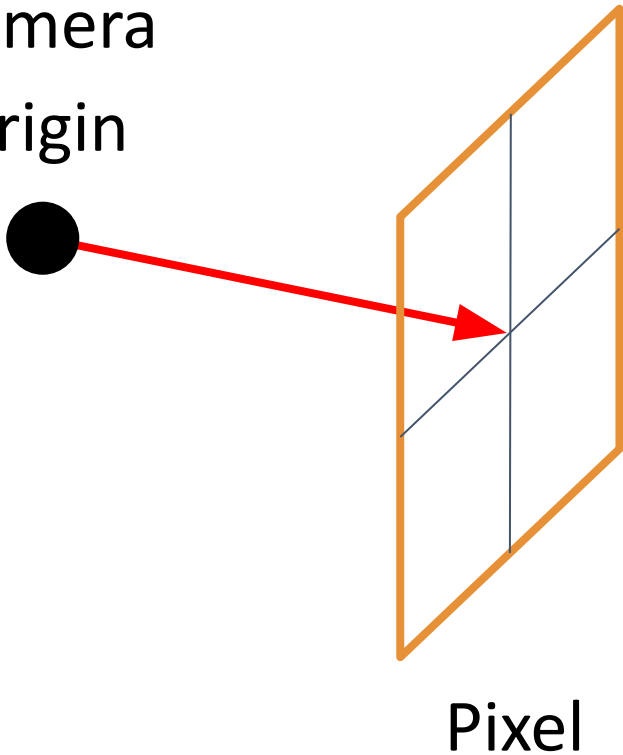


A camera is defined by:

- Local coordinate system
(left L, up U, forward F, origin O)
- Forward, up & left also define the image plane
 - Divided in pixels (width, height)
 - Pixels should be square (not necessarily)
- Calculate the boundaries of a pixel
 - Rays: origin O, direction towards pixel

Considerations about the camera

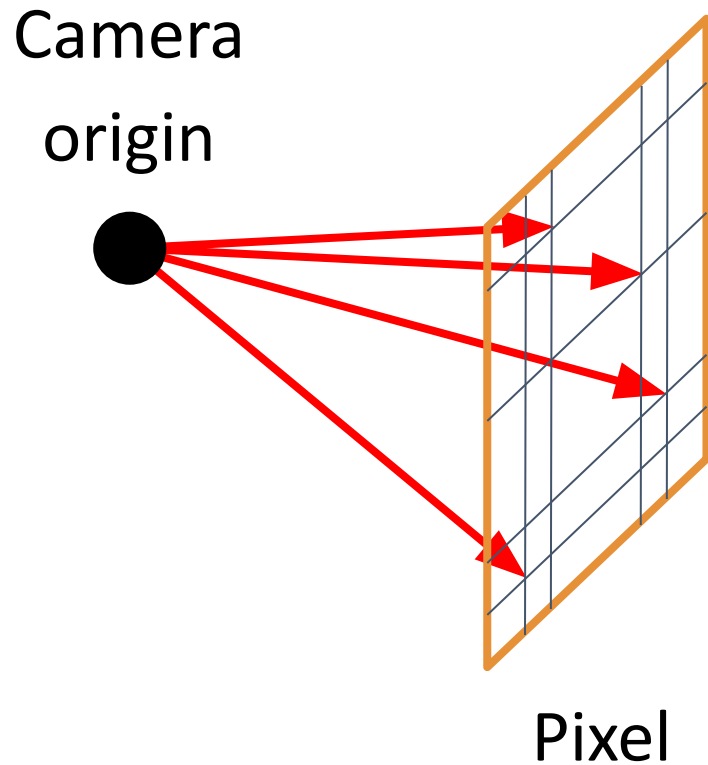
Camera
origin



A camera is defined by:

- Local coordinate system
(left L, up U, forward F, origin O)
- Forward, up & left also define the image plane
 - Divided in pixels (width, height)
 - Pixels should be square (not necessarily)
- Calculate the boundaries of a pixel
 - Rays: origin O, direction towards pixel

Considerations about the camera

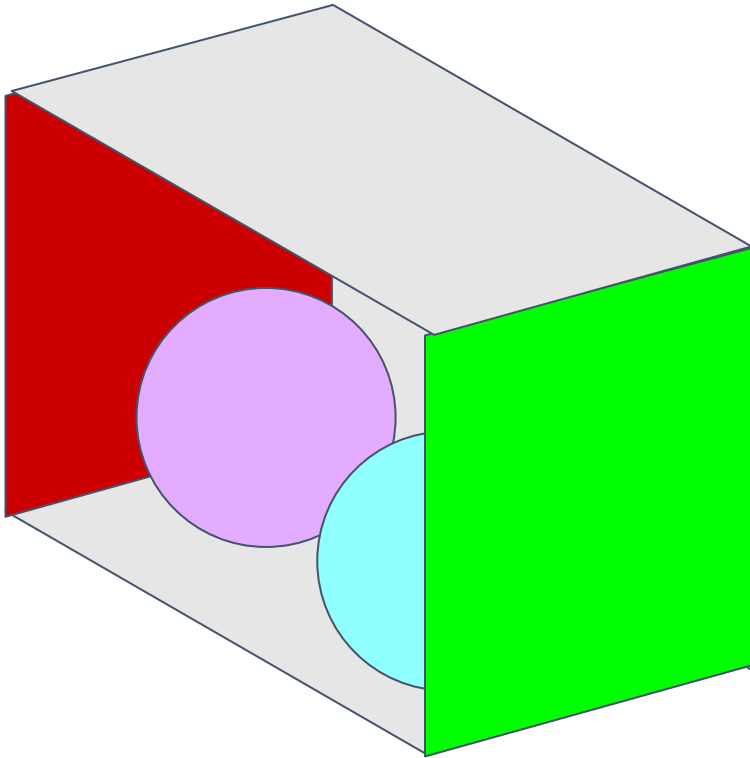


A camera is defined by:

- Local coordinate system
(left L, up U, forward F, origin O)
- Forward, up & left also define the image plane
 - Divided in pixels (width, height)
 - Pixels should be square (not necessarily)
- Calculate the boundaries of a pixel
 - Rays: origin O, direction towards pixel

Example scene: Cornell Box

- **Geometry**



Planes defined by normal (n) and distance (d)

Left plane $n = (1, 0, 0), d = 1$

Right plane $n = (-1, 0, 0), d = 1$

Floor plane $n = (0, 1, 0), d = 1$

Ceiling plane $n = (0, -1, 0), d = 1$

Back plane $n = (0, 0, -1), d = 1$

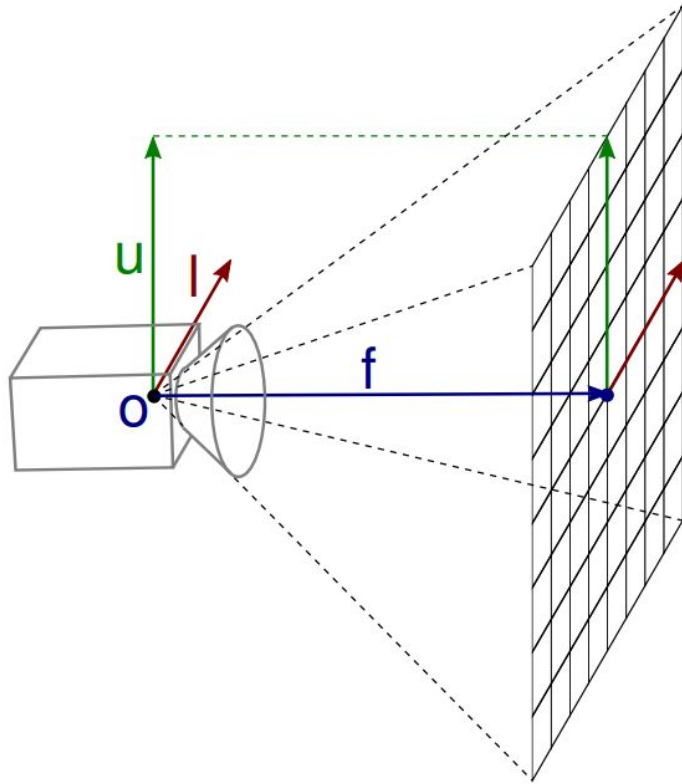
Spheres defined by center (c) and radius (r)

Left sphere $c = (-0.5, -0.7, 0.25), r = 0.3$

Right sphere $c = (0.5, -0.7, -0.25), r = 0.3$

Example scene: Cornell Box

- Camera



Camera and image plane defined by

Origin $O = (0, 0, -3.5)$

Left $L = (-1, 0, 0)$

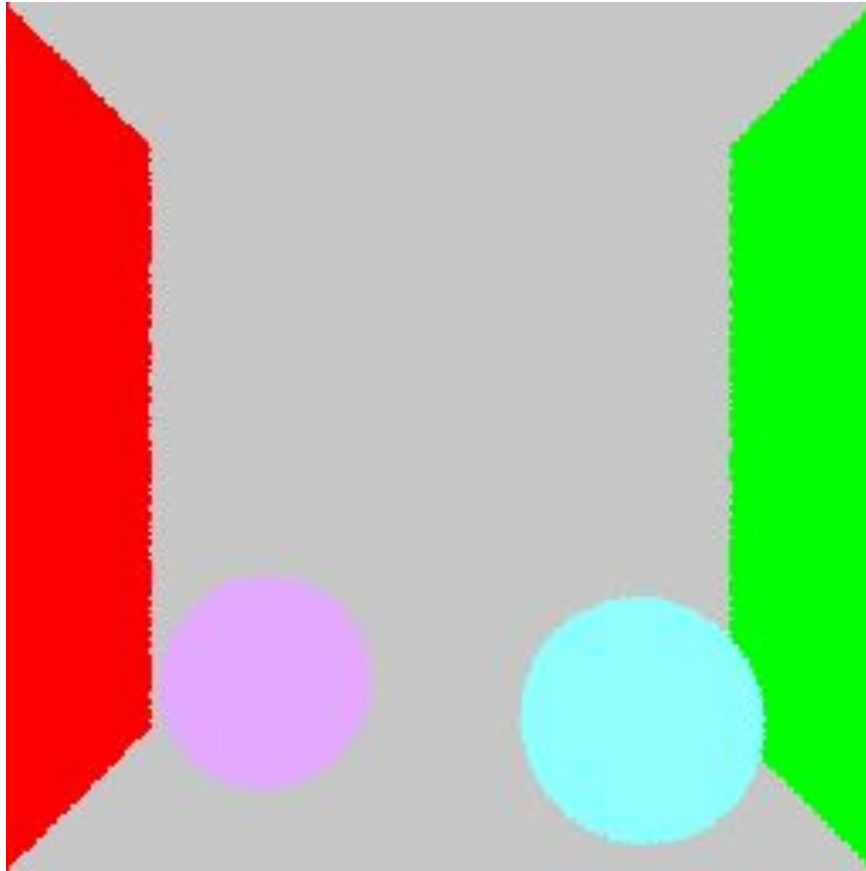
Up $U = (0, 1, 0)$

Forward $F = (0, 0, 3)$

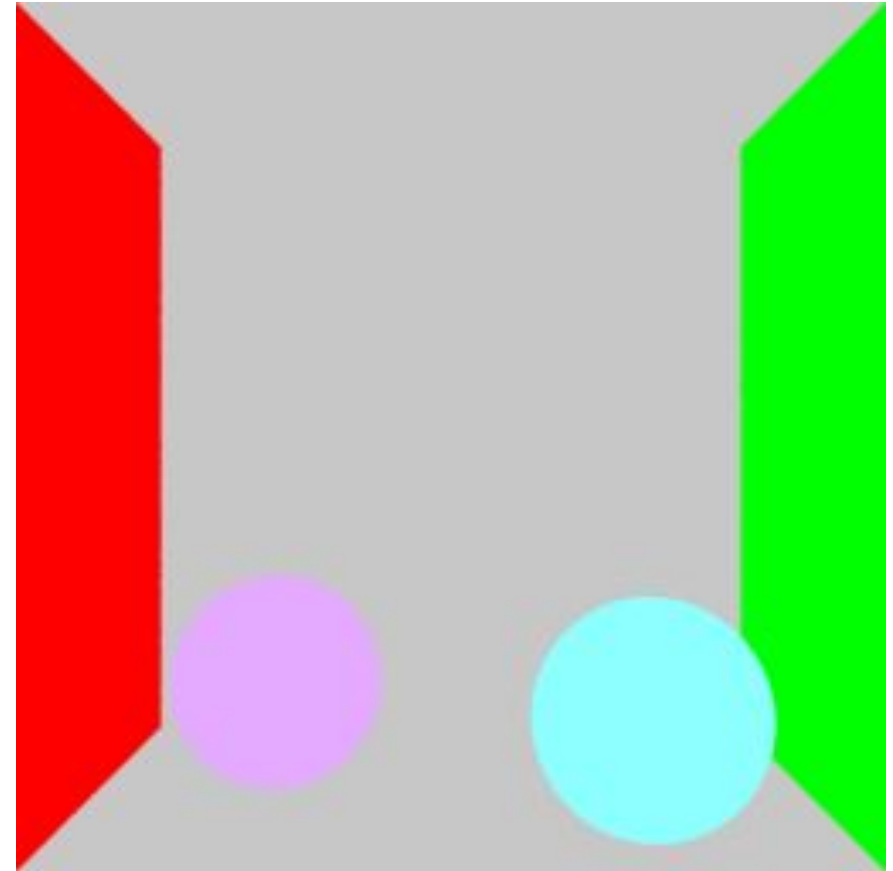
Size 256x256 pixels

Example scene: Cornell Box

- Results



1 ray per pixel



64 rays per pixel

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice, implement:

- **Scene (collection of geometry primitives)**
 - Spheres, planes, etc. (position, emission) should be hardcoded
- **Pinhole camera**
 - Generate a ray for a specific pixel
 - Intersect this ray with your scene
 - Generate an image
- Recommended deadline: **October 18th.**
 - **Extensions** (do not count towards deadline):
 - Other primitives: cones, cylinders, ellipsoids, disks or triangles
 - Acceleration structures: bounding volumes, multi-threading, etc.
 - Constructive solid geometry: google it or ask us :)

Lab #4 – Path tracing (part 1)

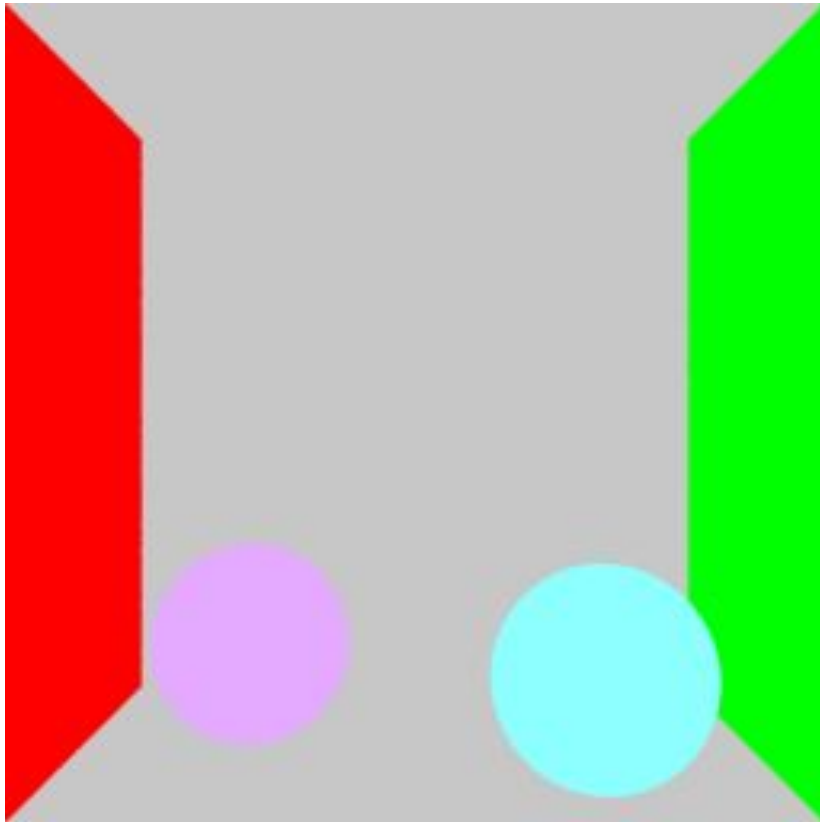
Informática Gráfica

Adolfo Muñoz - Julio Marco
Pablo Luesia - J. Daniel Subías – Óscar Pueyo

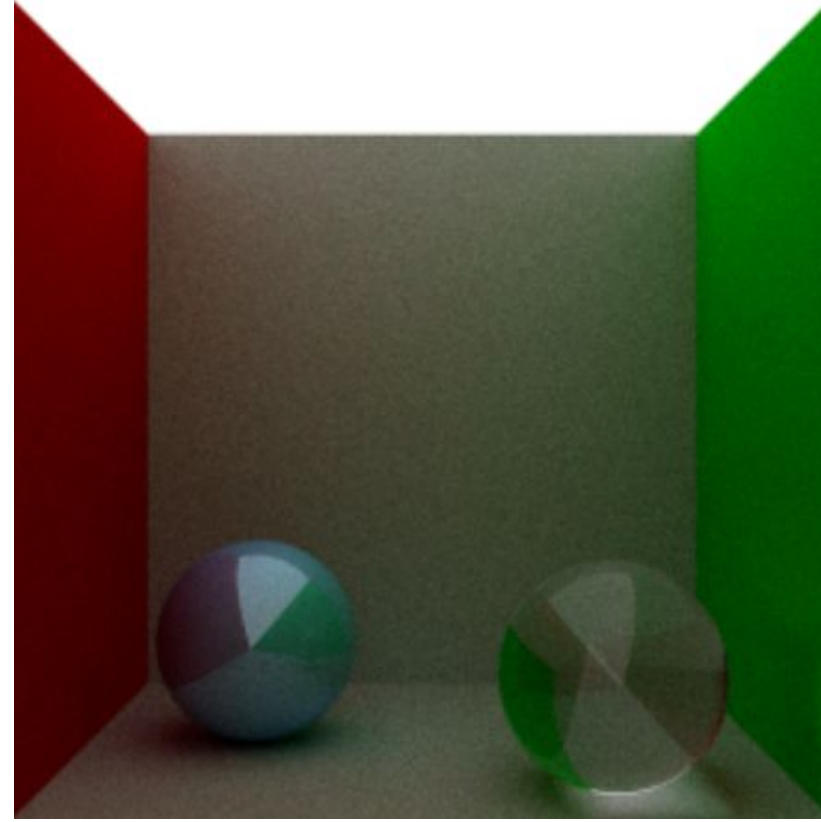


Before we begin...

- Begin extending ray tracing to the full path tracing algorithm



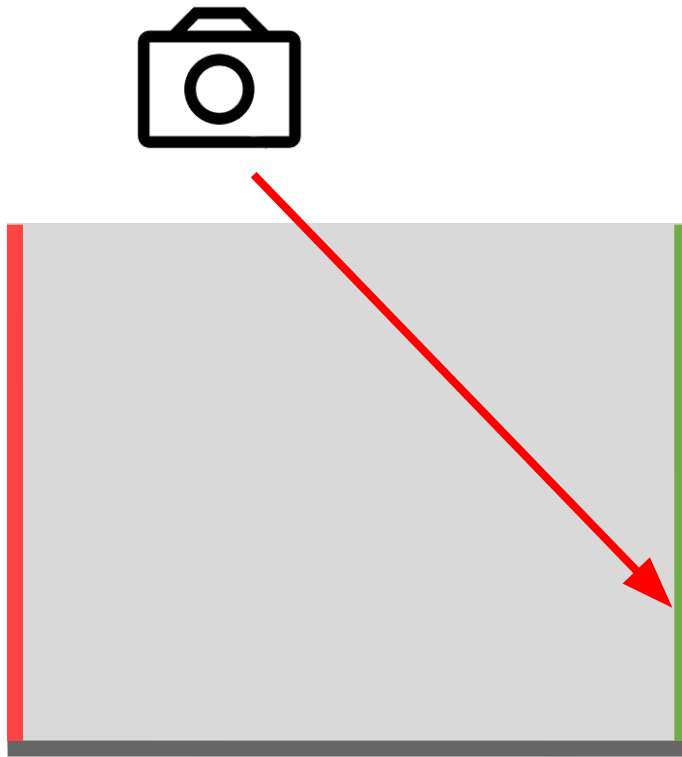
Lab 3 (ray tracing)



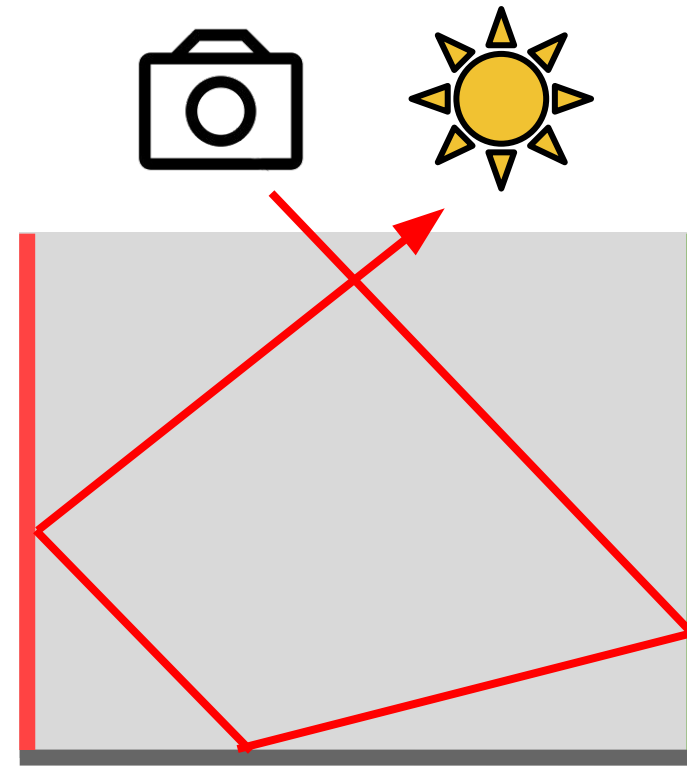
Lab 4 (path tracing)

Before we begin...

- Begin extending ray tracing to the full path tracing algorithm



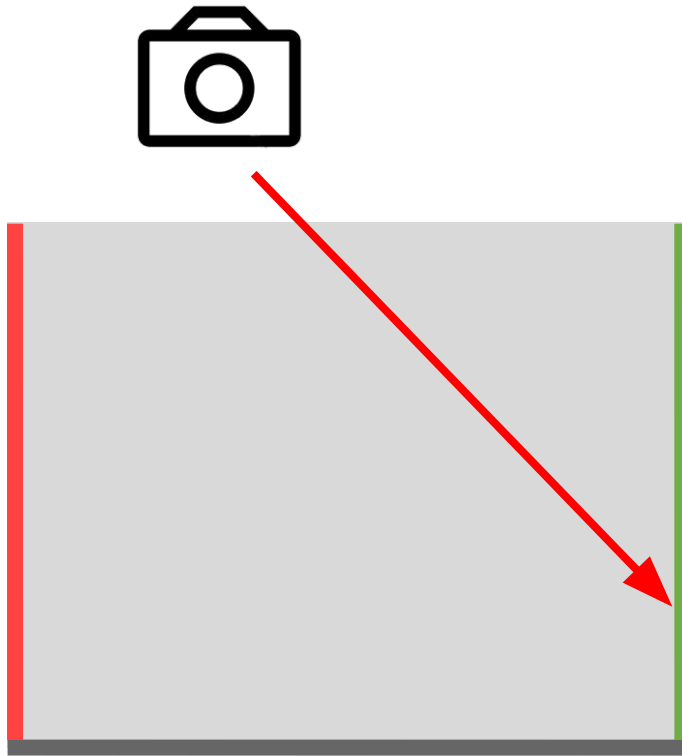
Lab 3 (ray tracing)



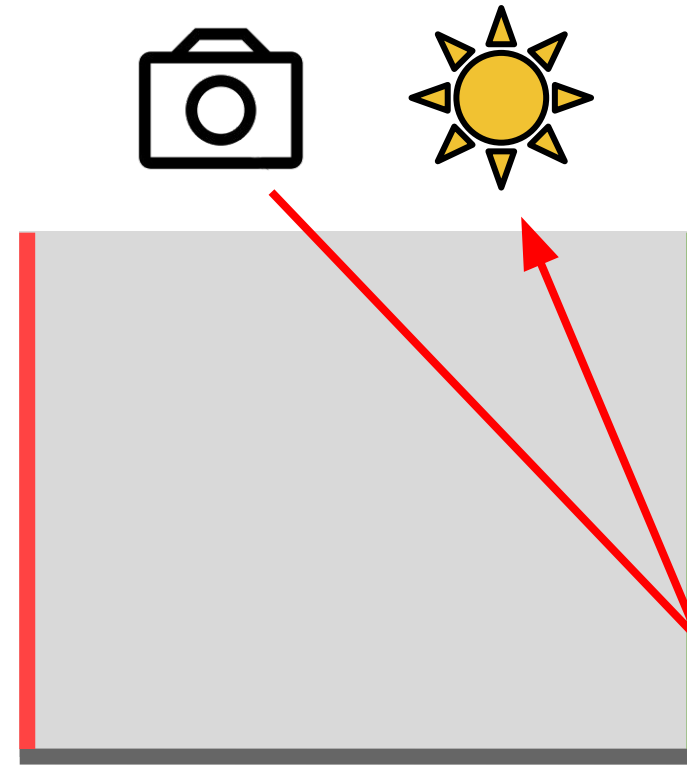
Lab 4 (path tracing)

Before we begin...

- Begin extending ray tracing to the full path tracing algorithm



Lab 3 (ray tracing)

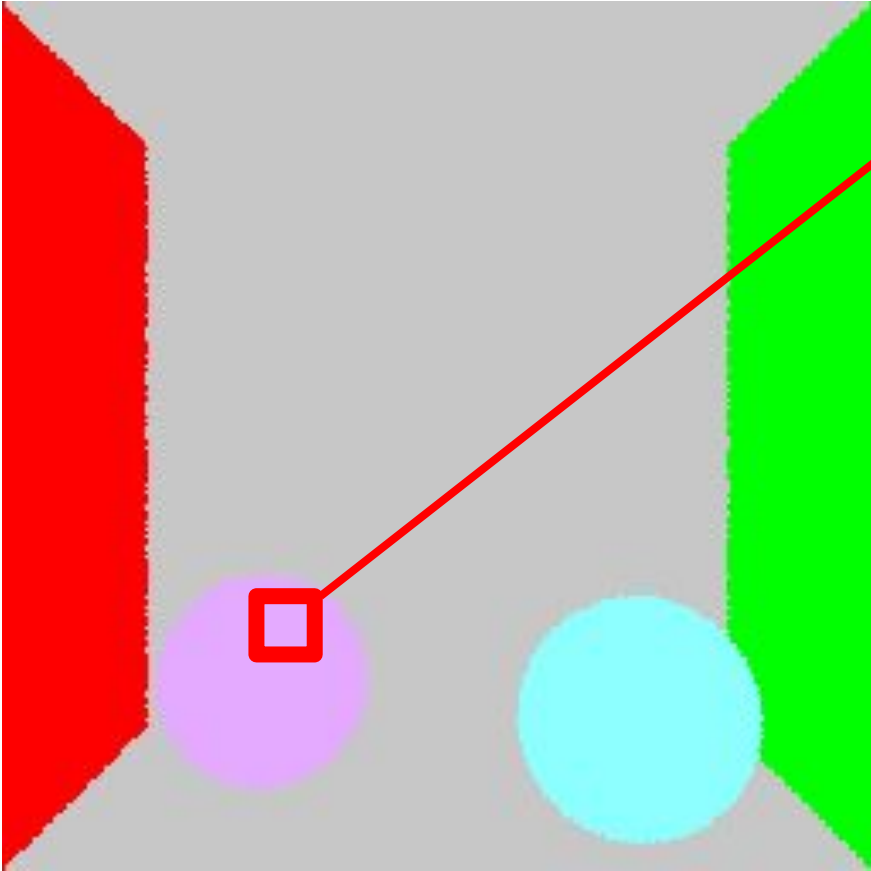


This session

Before we begin...

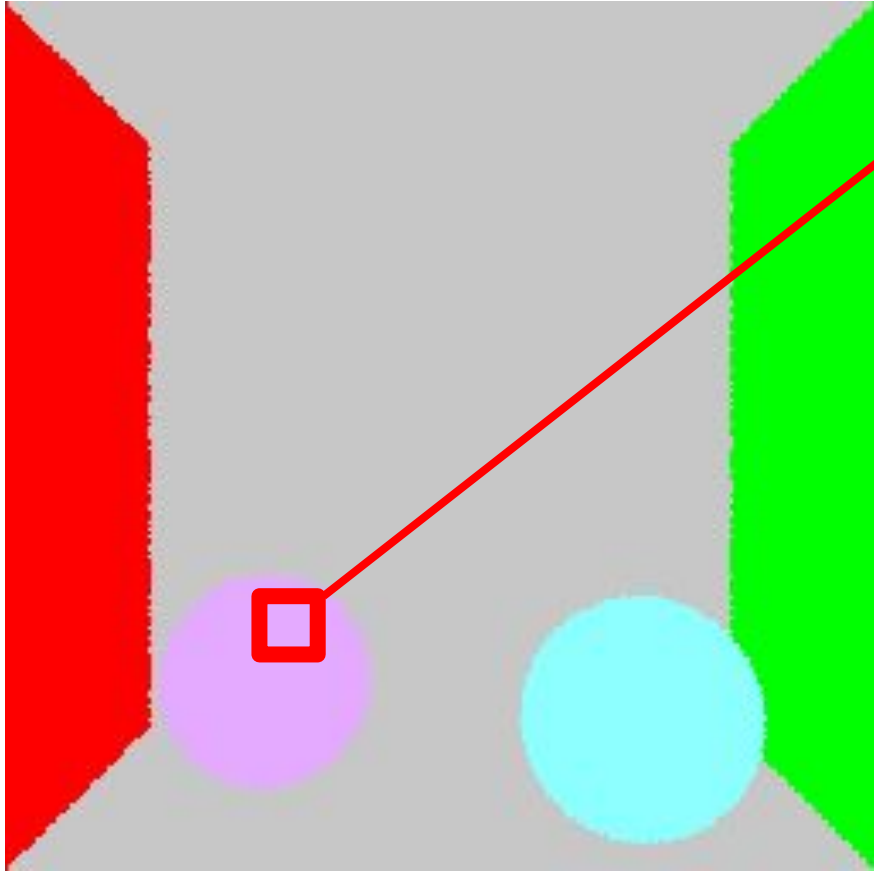
- Begin extending ray tracing to the full path tracing algorithm
- Lab 4 (path tracing) **is the first submitted work**
 - You will have 3 sessions before the recommended deadline
 - Recommended deadline: November 13th
 - Moodle: January 11th
 - You will use most of today's code for Lab 5 (photon mapping) too
- Remember: Final work is 80% of the final grade

Previously: everything is an area light

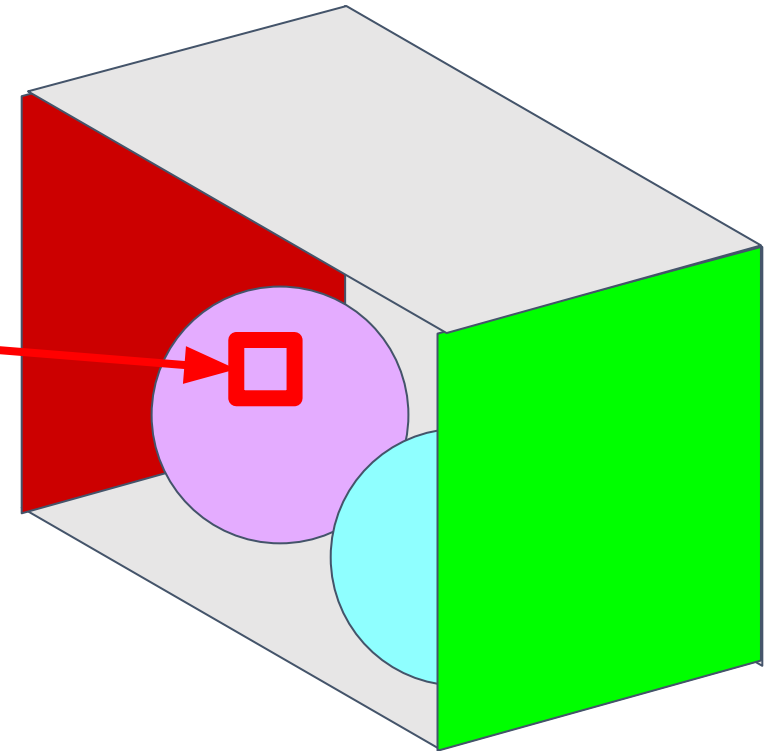
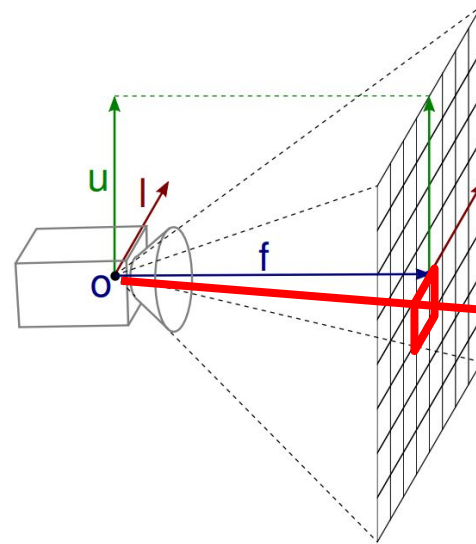


$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) \cancel{f_r(\mathbf{x}, \omega_i, \omega_o)} |\mathbf{n} \cdot \omega_i| d\omega_i$$

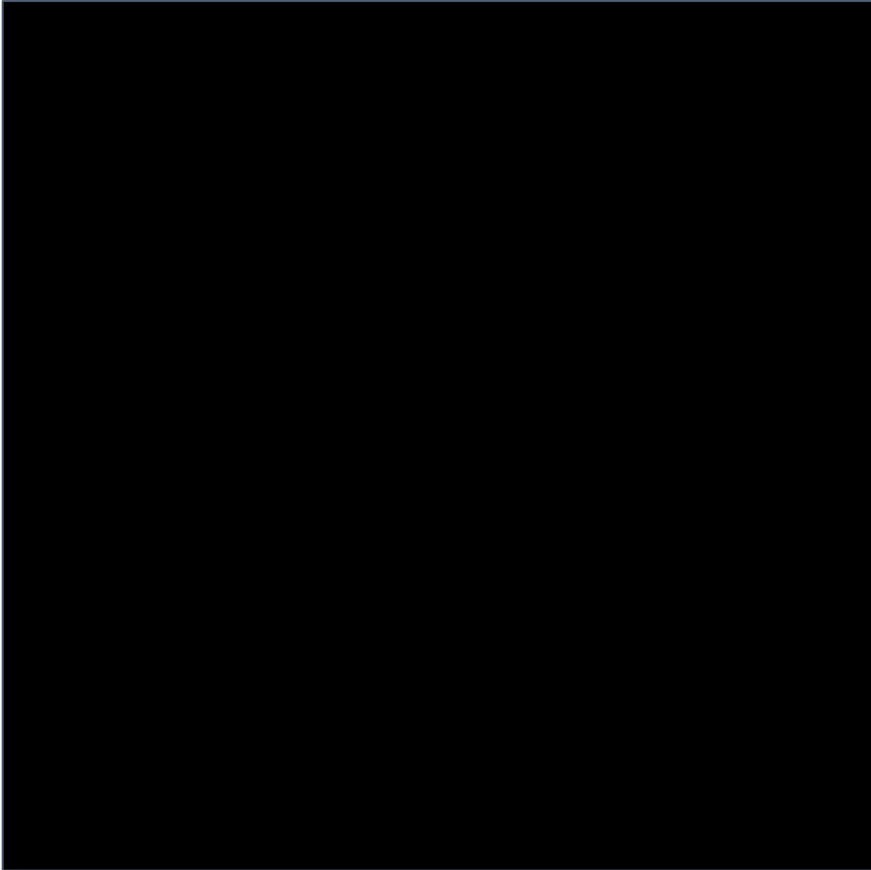
Previously: everything is an area light



$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \cancel{\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i}$$

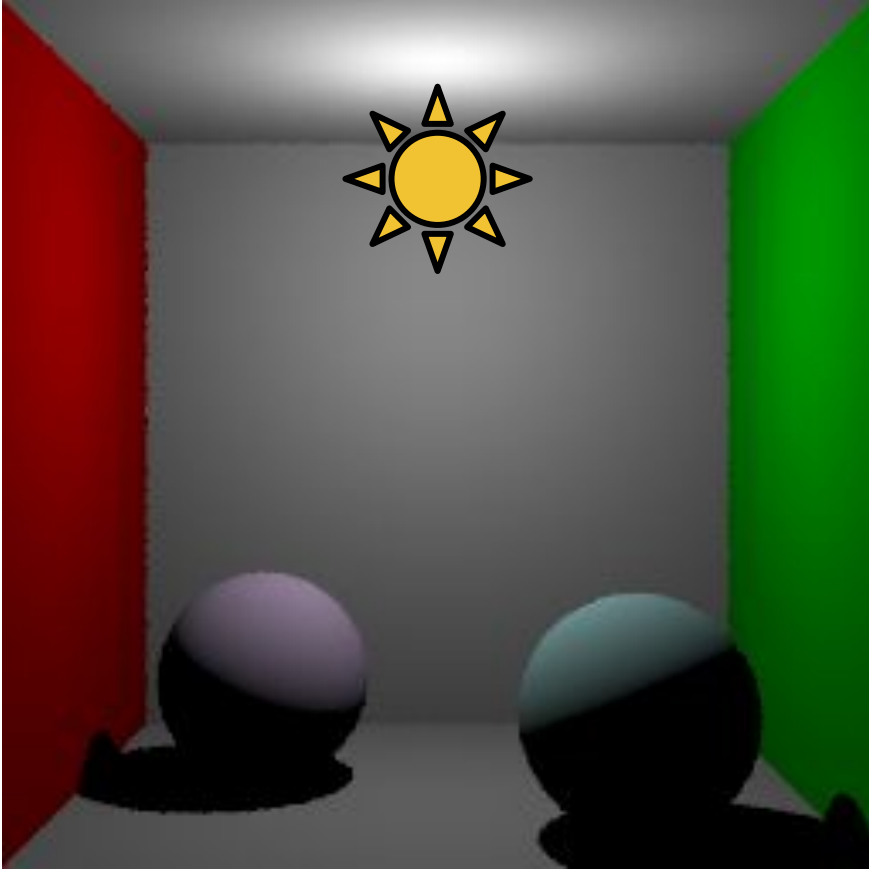


Which color do we fill each pixel with?

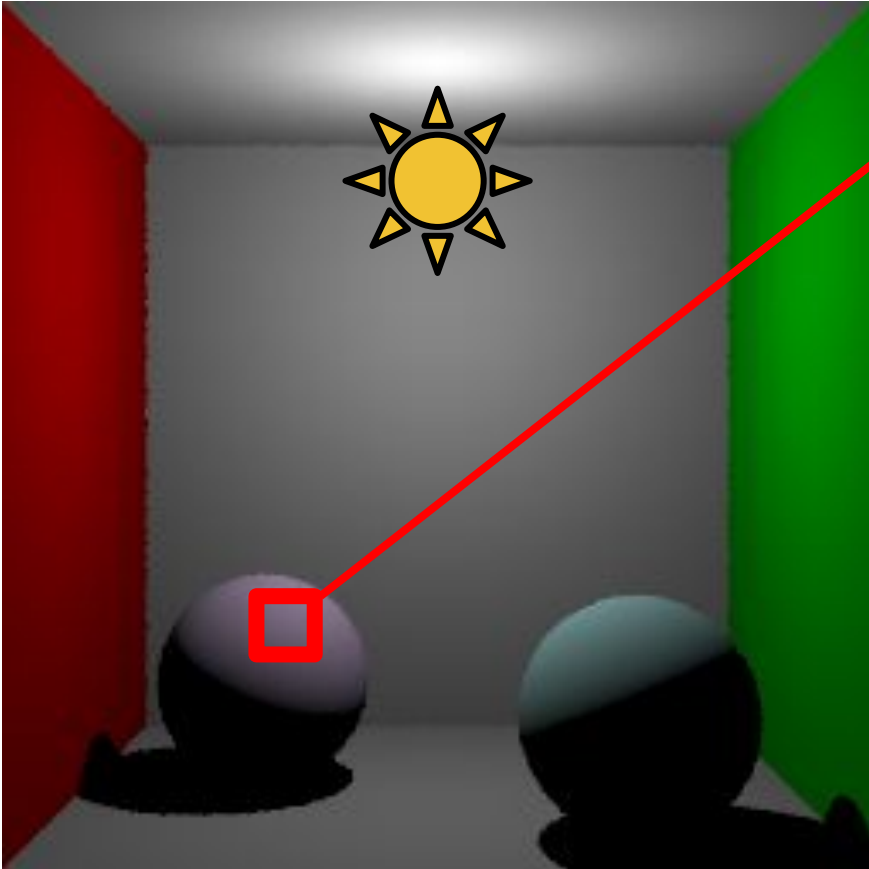


$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} \cancel{L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|} d\omega_i$$

Which color do we fill each pixel with?

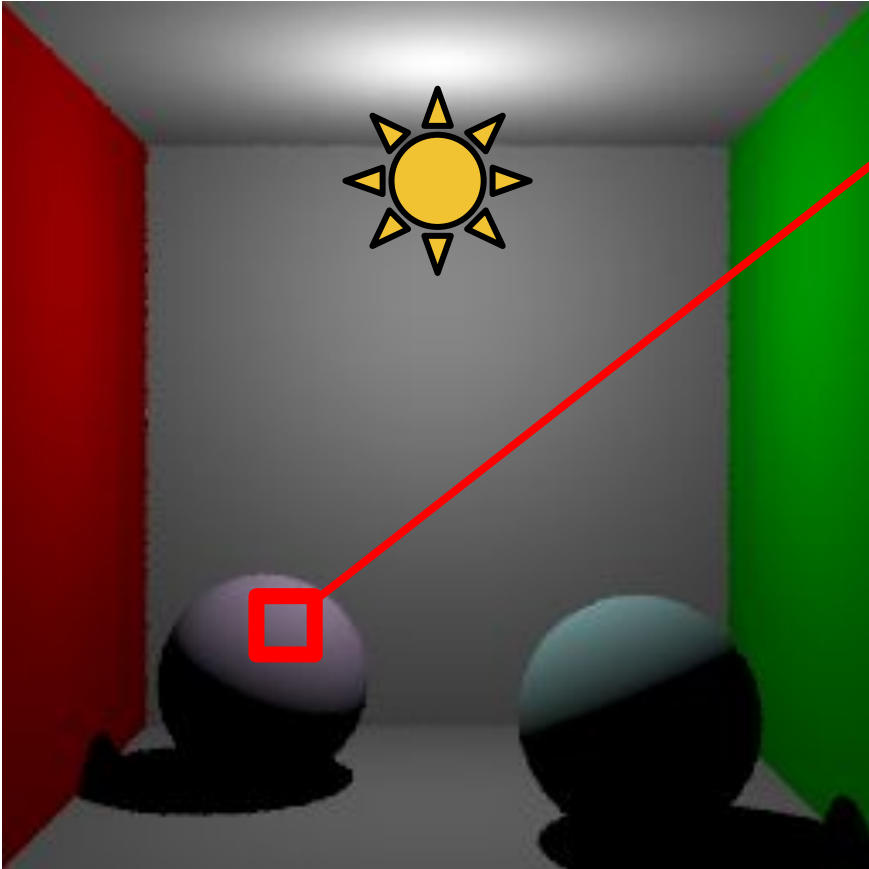


Which color do we fill each pixel with?



$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Which color do we fill each pixel with?

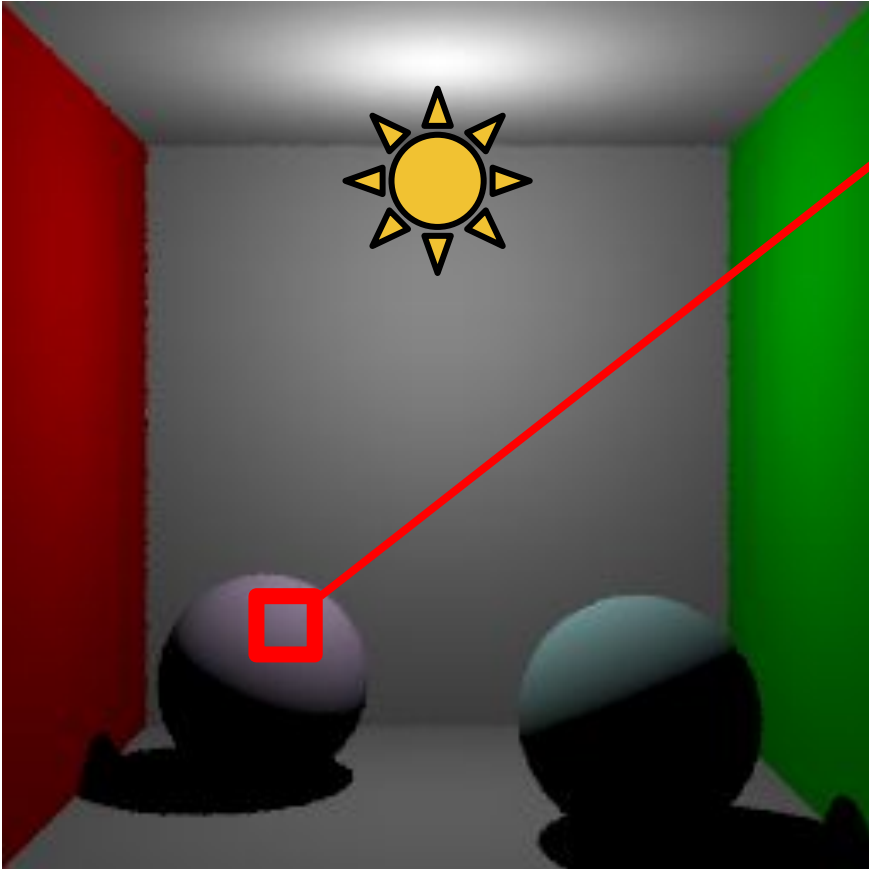


$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$



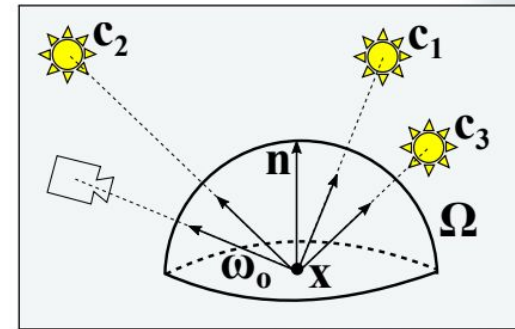
But just the direct light

Which color do we fill each pixel with?



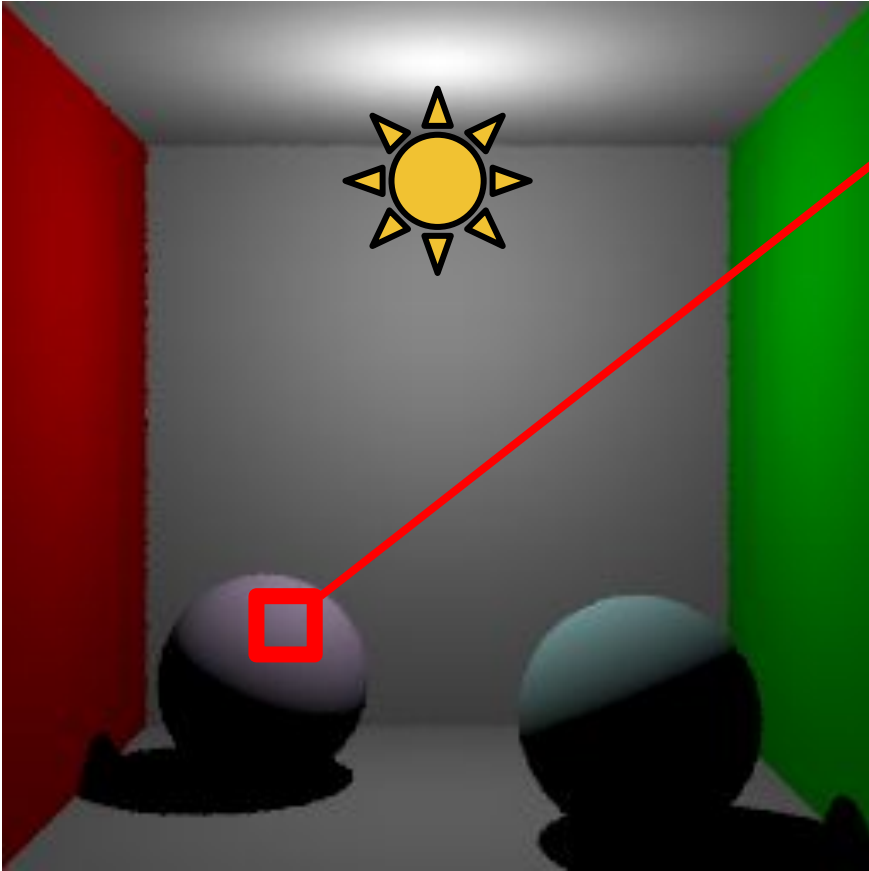
$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

But just the direct light



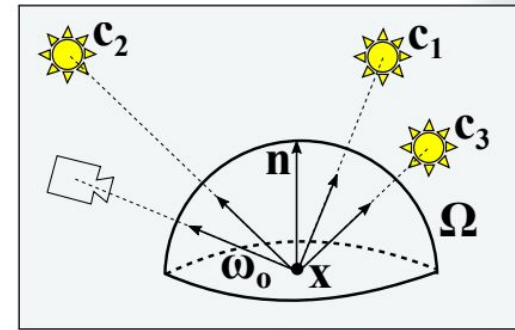
$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$

Which color do we fill each pixel with?



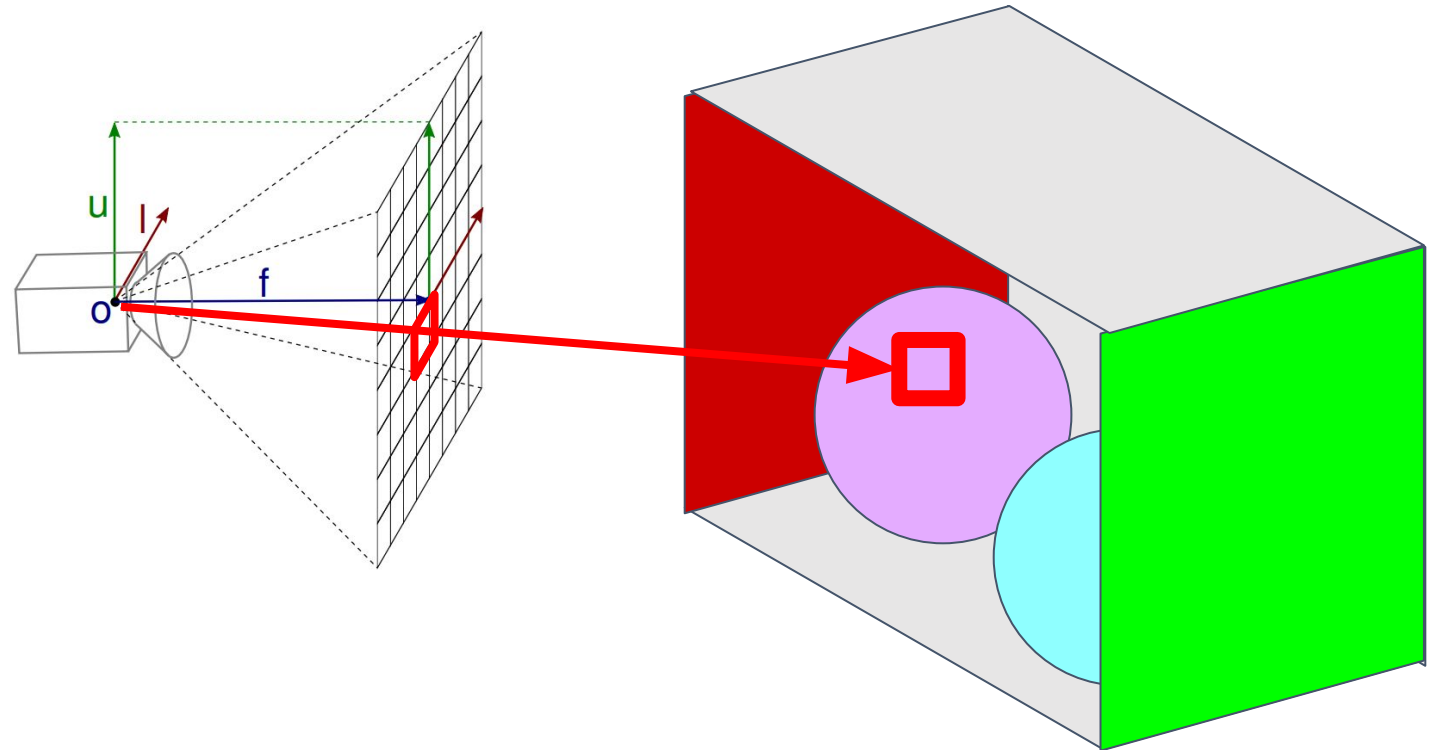
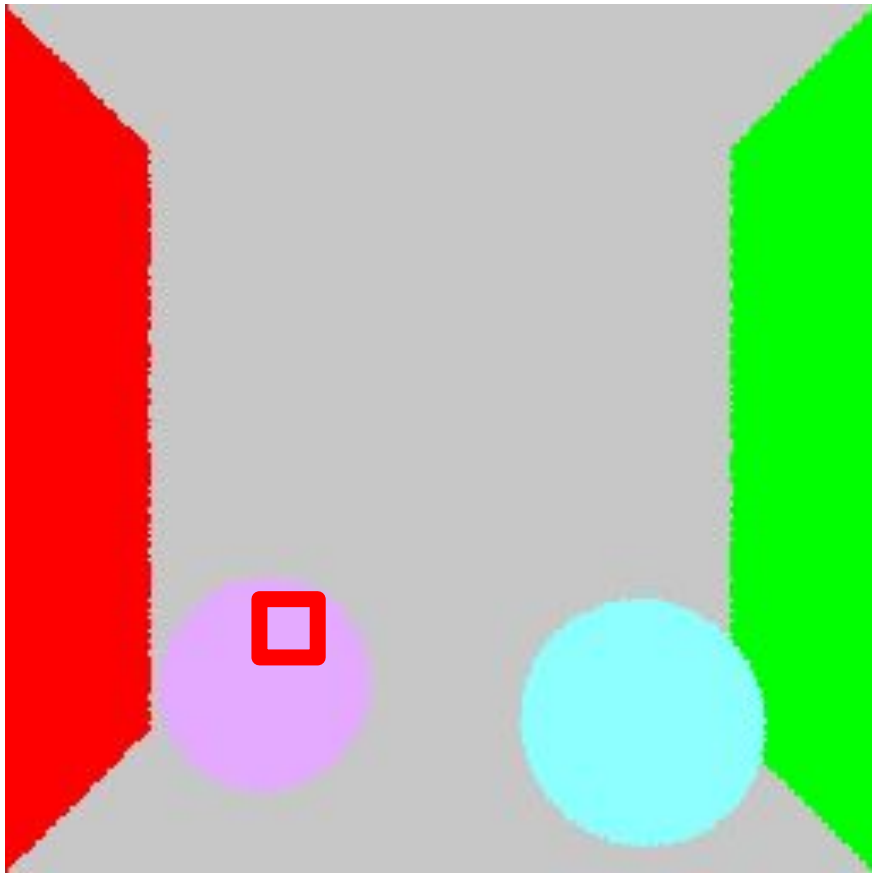
$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \omega_i)}_{\text{orange}} \underbrace{f_r(\mathbf{x}, \omega_i, \omega_o)}_{\text{blue}} \underbrace{|\mathbf{n} \cdot \omega_i|}_{\text{green}} d\omega_i$$

But just the direct light

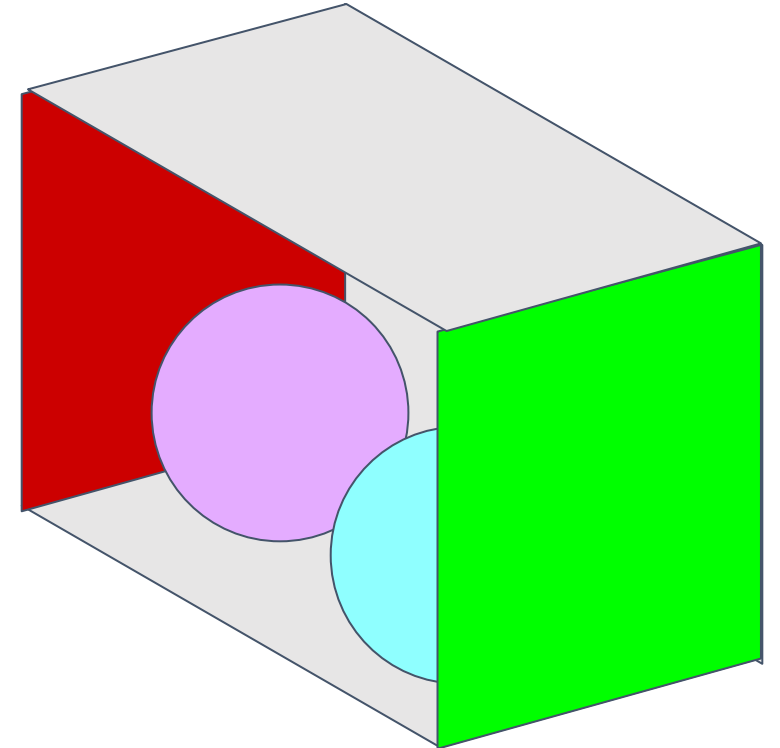
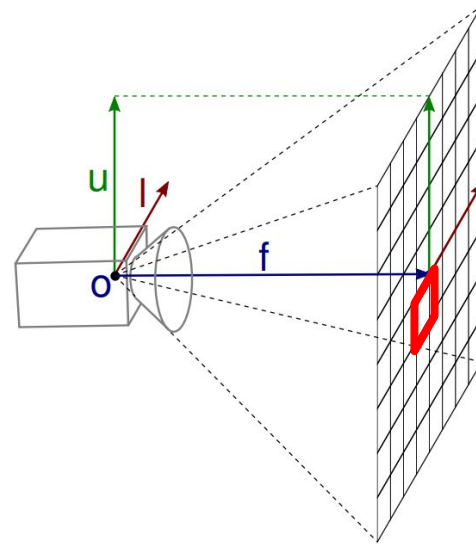
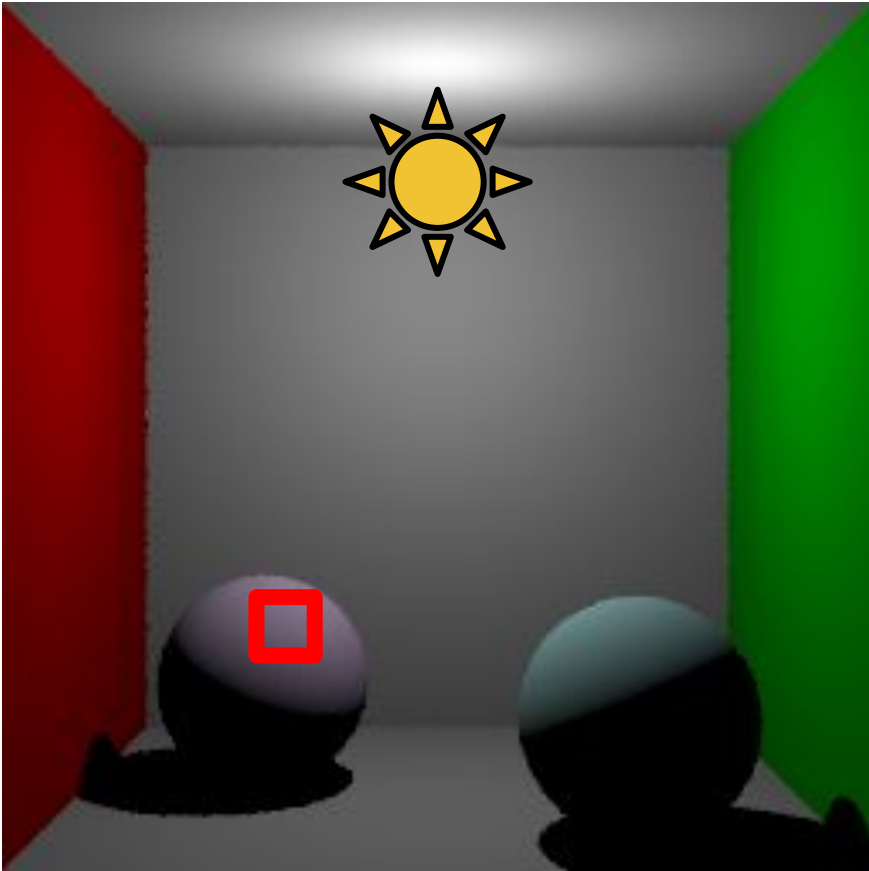


$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \underbrace{\frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2}}_{\text{orange}} \underbrace{f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right)}_{\text{blue}} \underbrace{\left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|}_{\text{green}}$$

Previously: return emission color

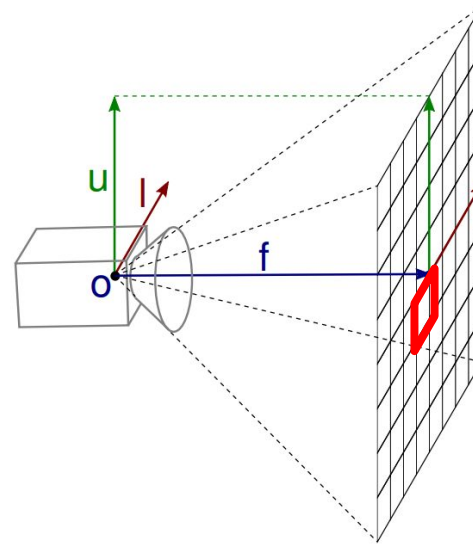
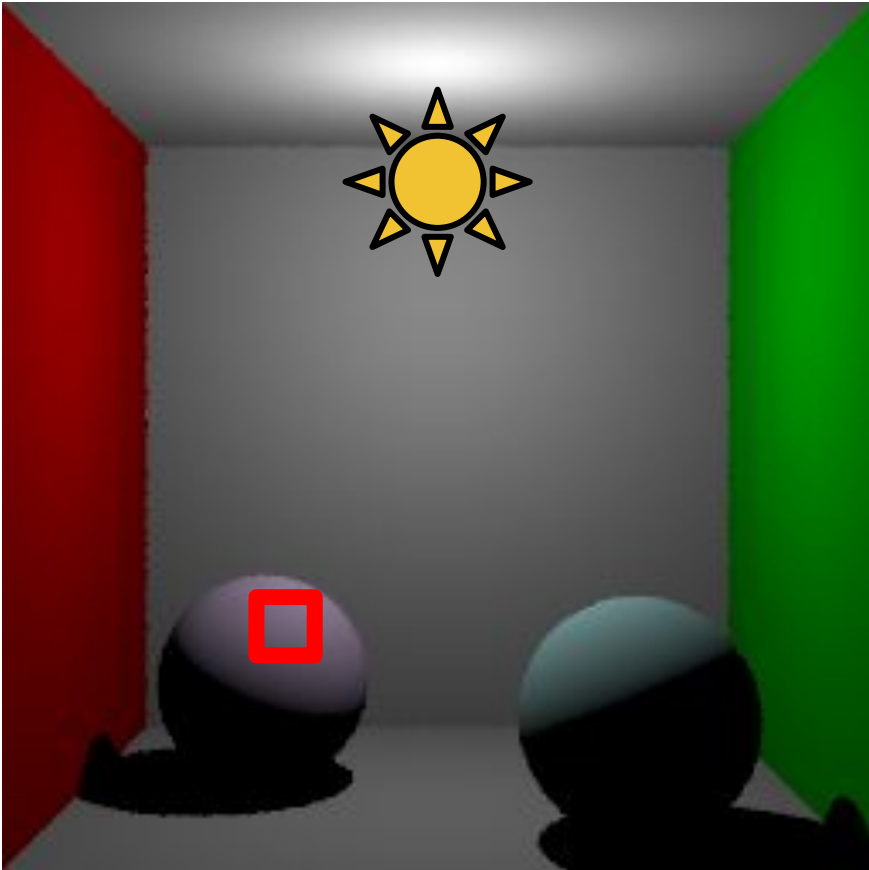


Which color do we fill each pixel with?

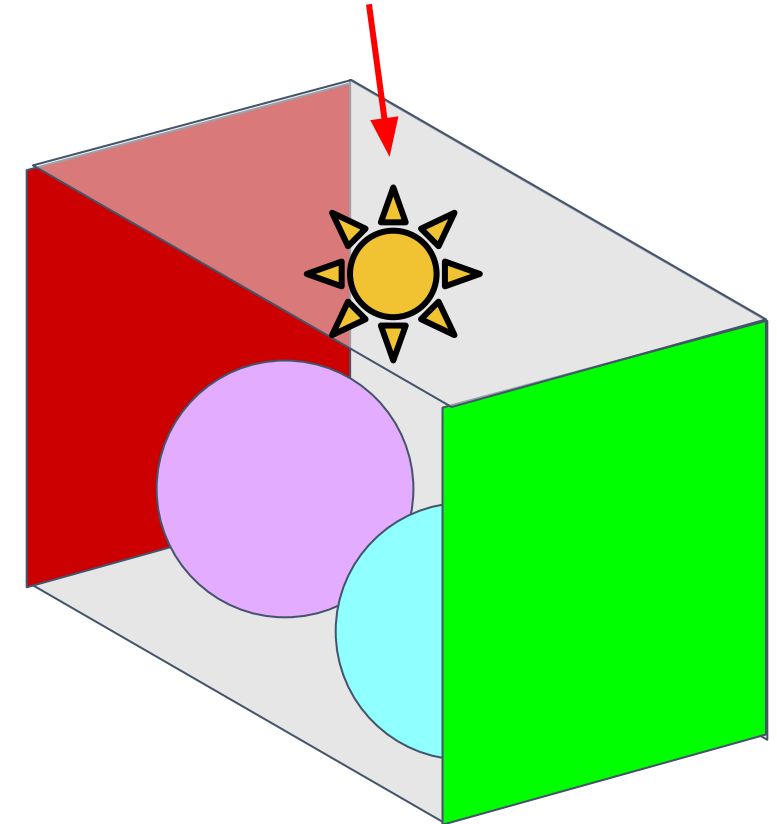


Which color do we fill each pixel with?

- Define your scene's point light sources

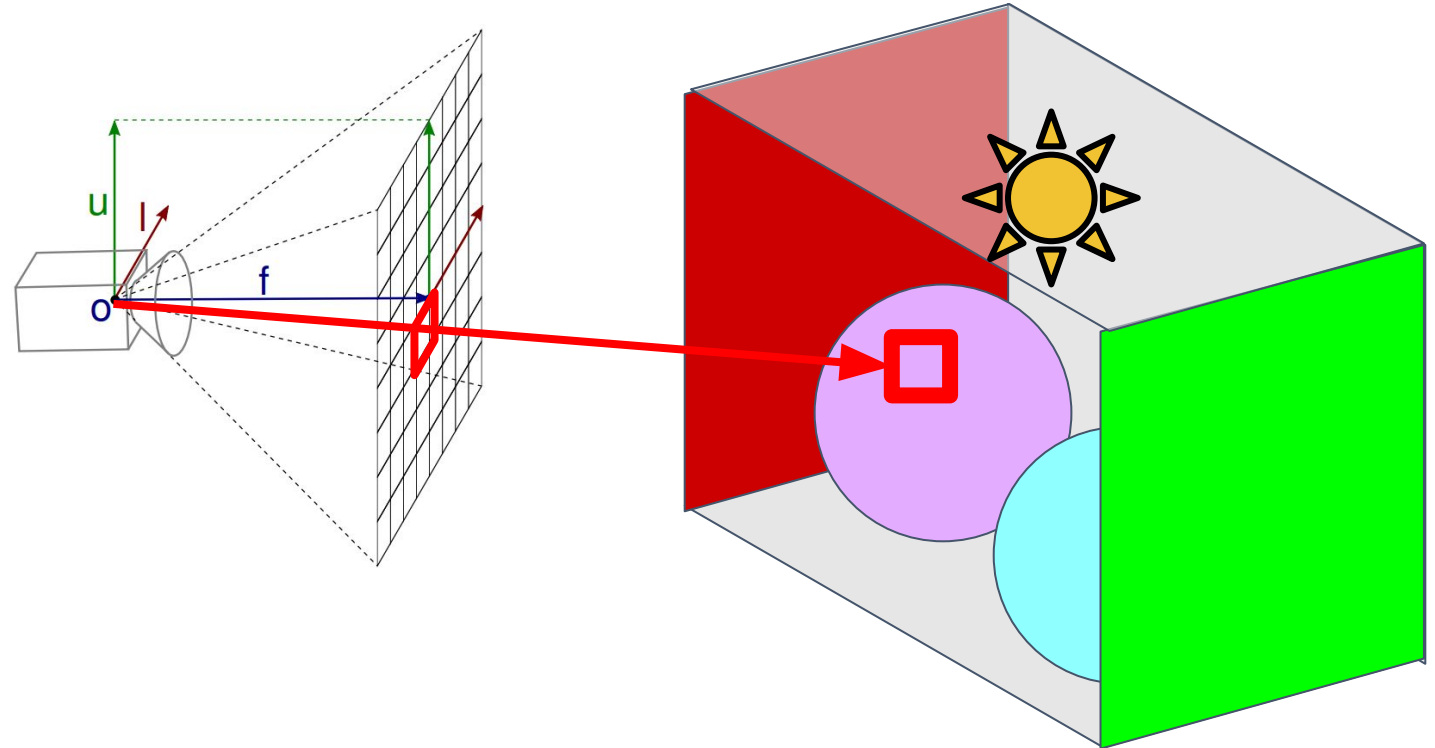
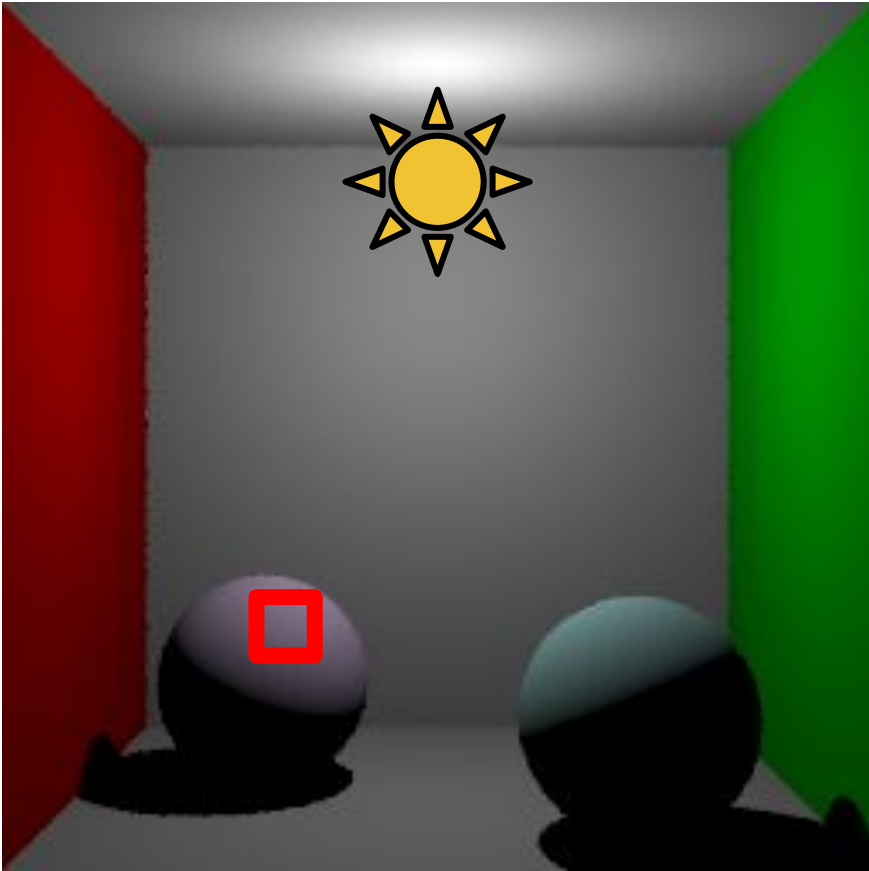


- Center: (x, y, z)
- Power: (r, g, b)



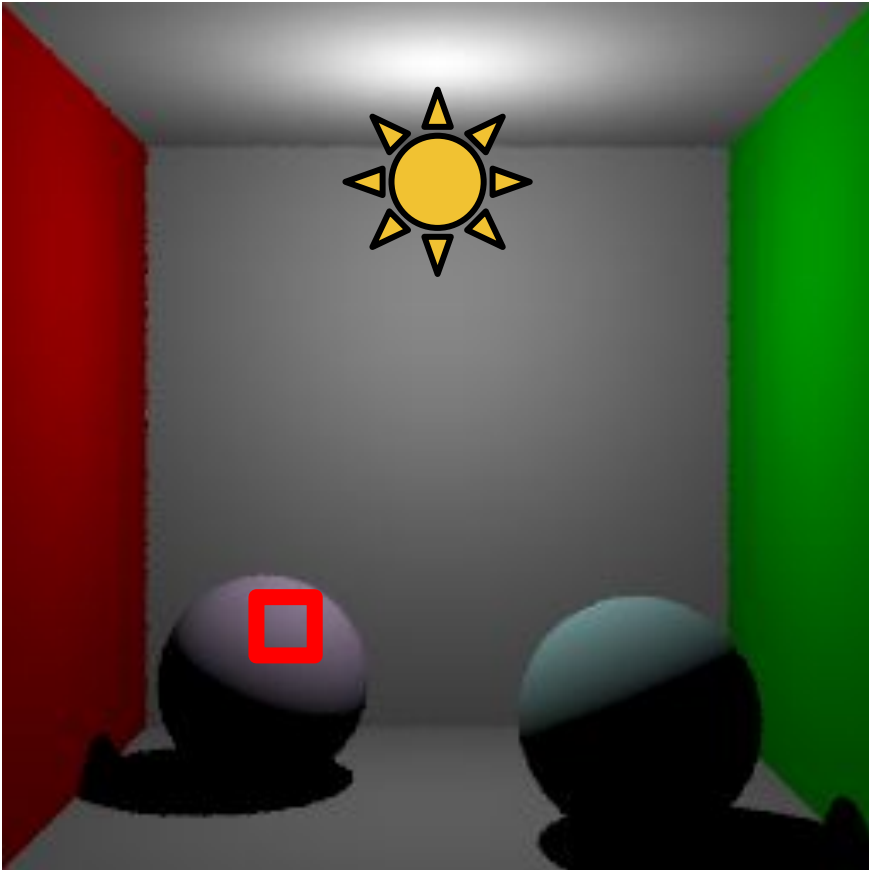
Which color do we fill each pixel with?

- Generate a ray from the camera and intersect

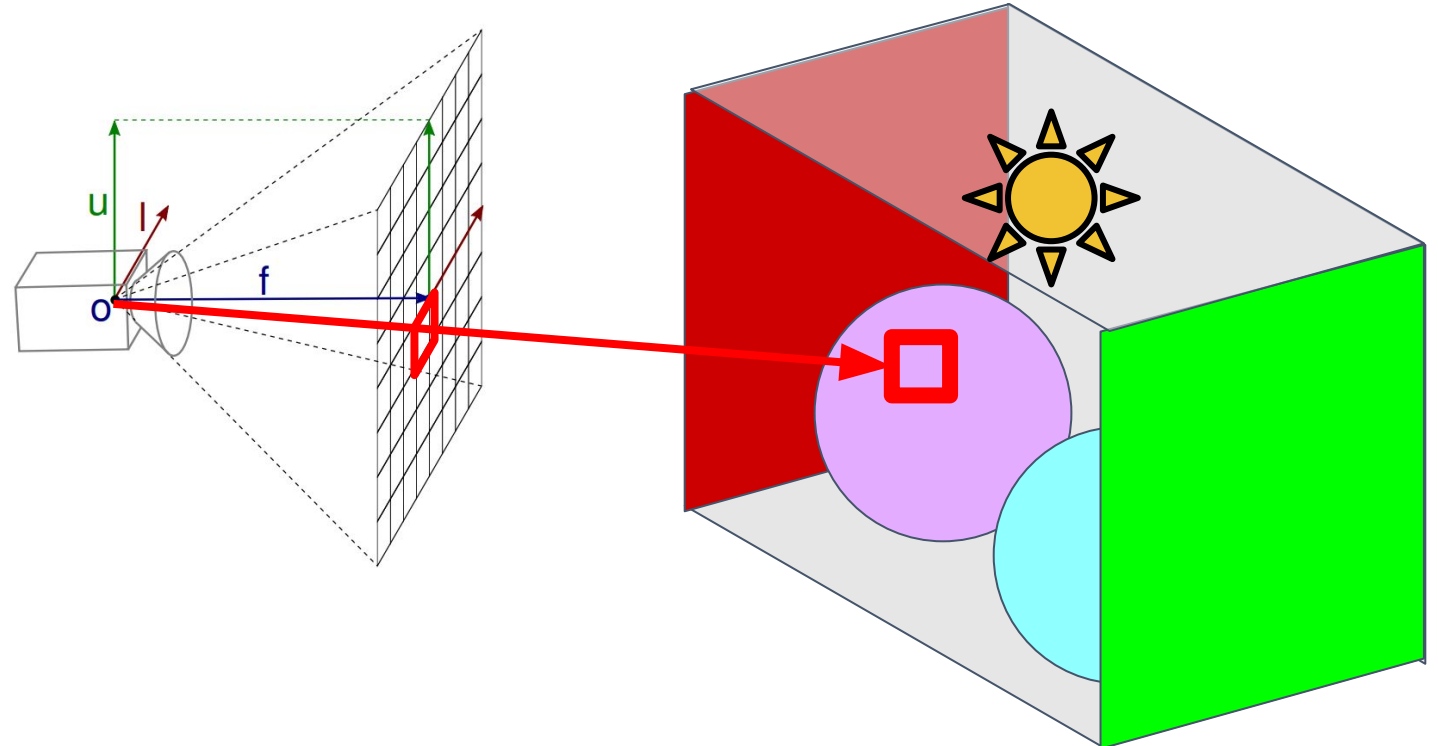


Which color do we fill each pixel with?

- Calculate direct light

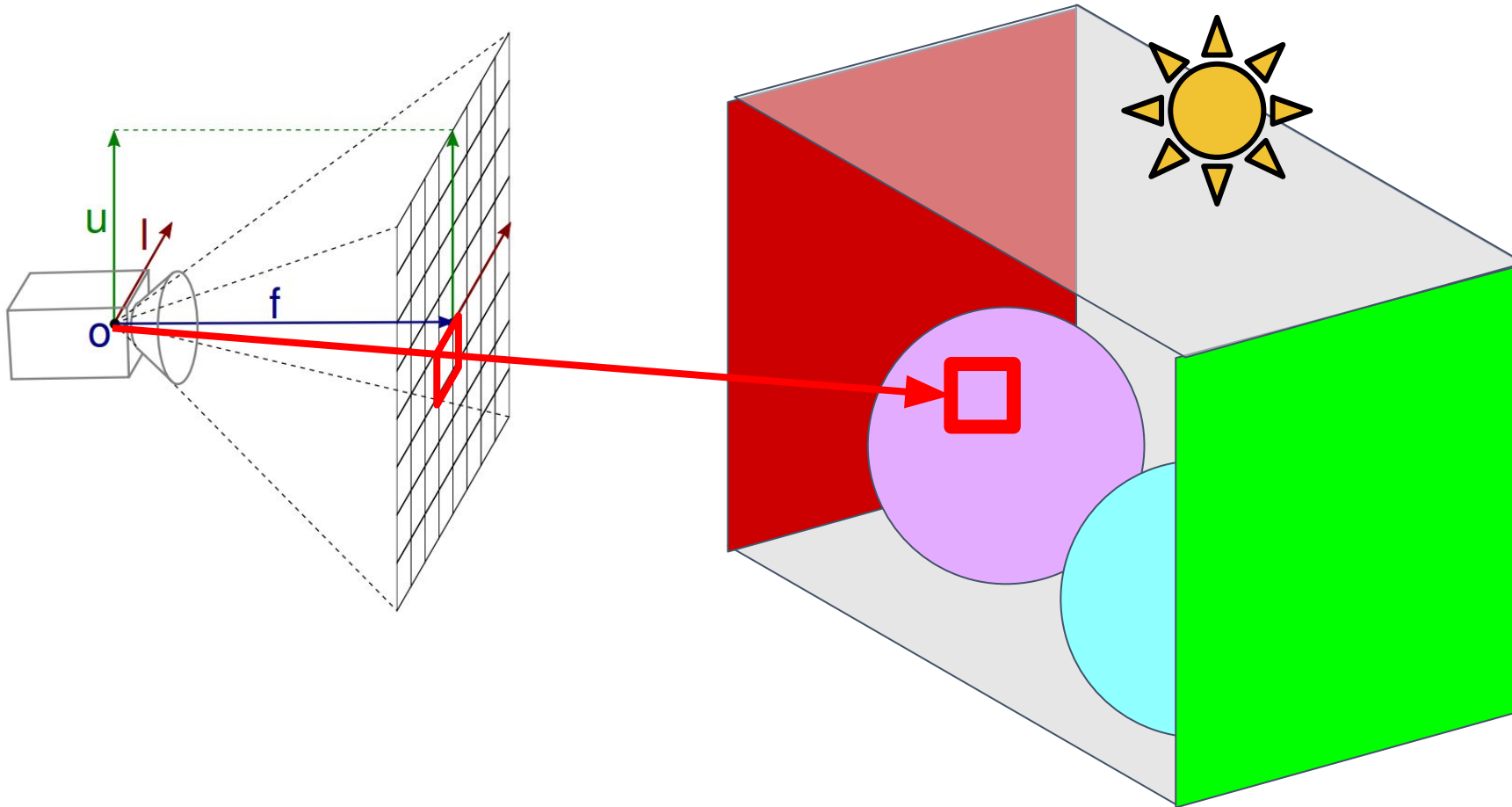


$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



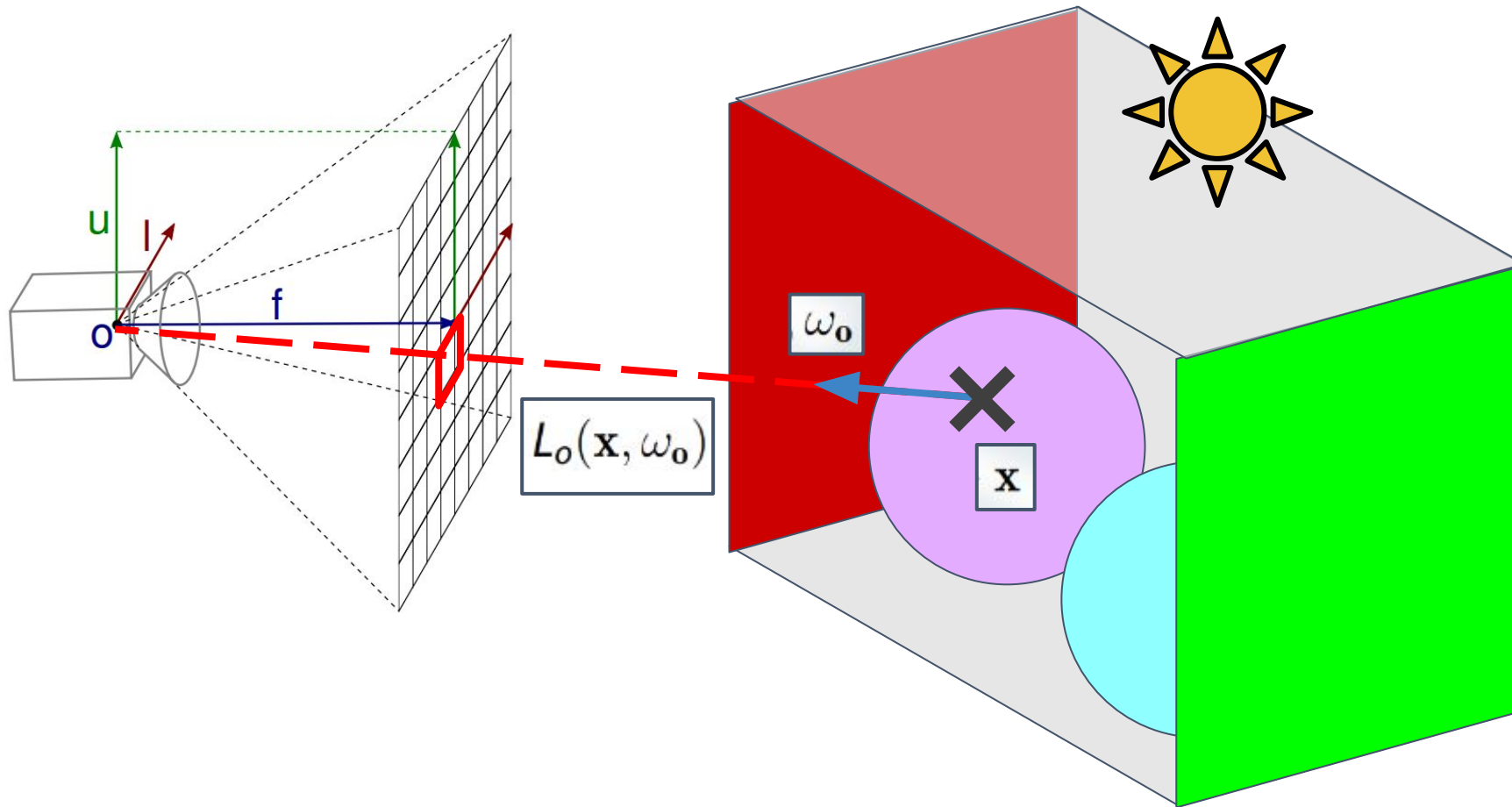
Which color do we fill each pixel with?

$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



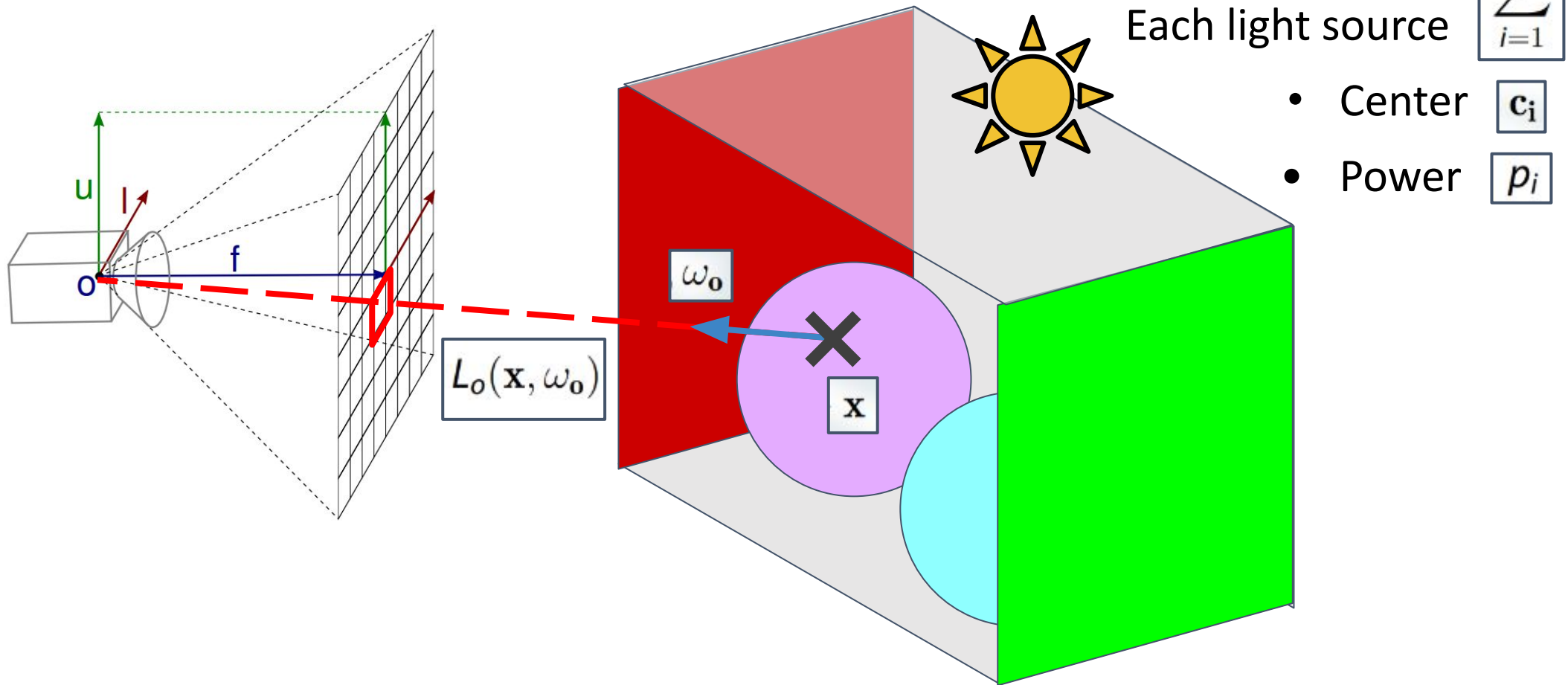
Which color do we fill each pixel with?

$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



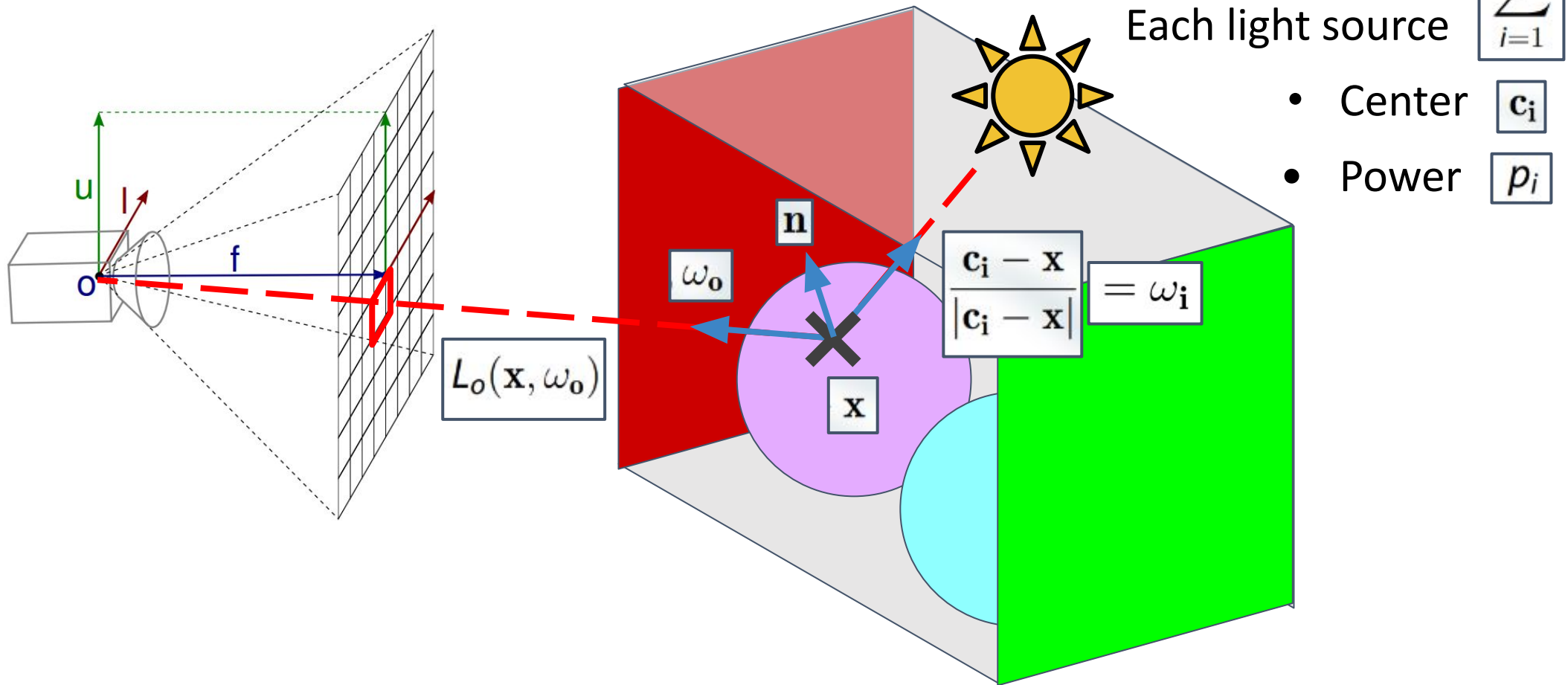
Which color do we fill each pixel with?

$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



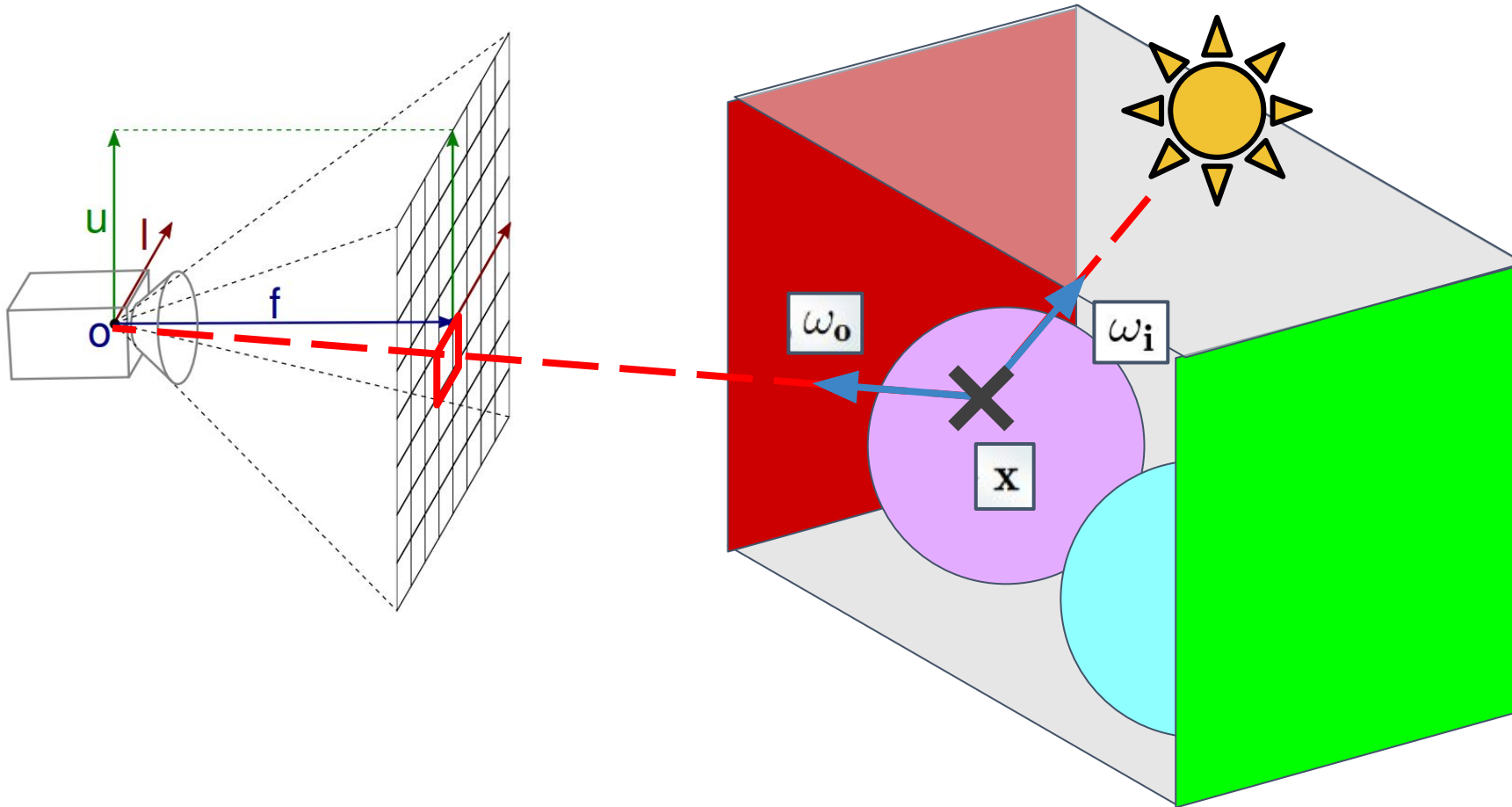
Which color do we fill each pixel with?

$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



Which color do we fill each pixel with?

$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



Which color do we fill each pixel with?

$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$

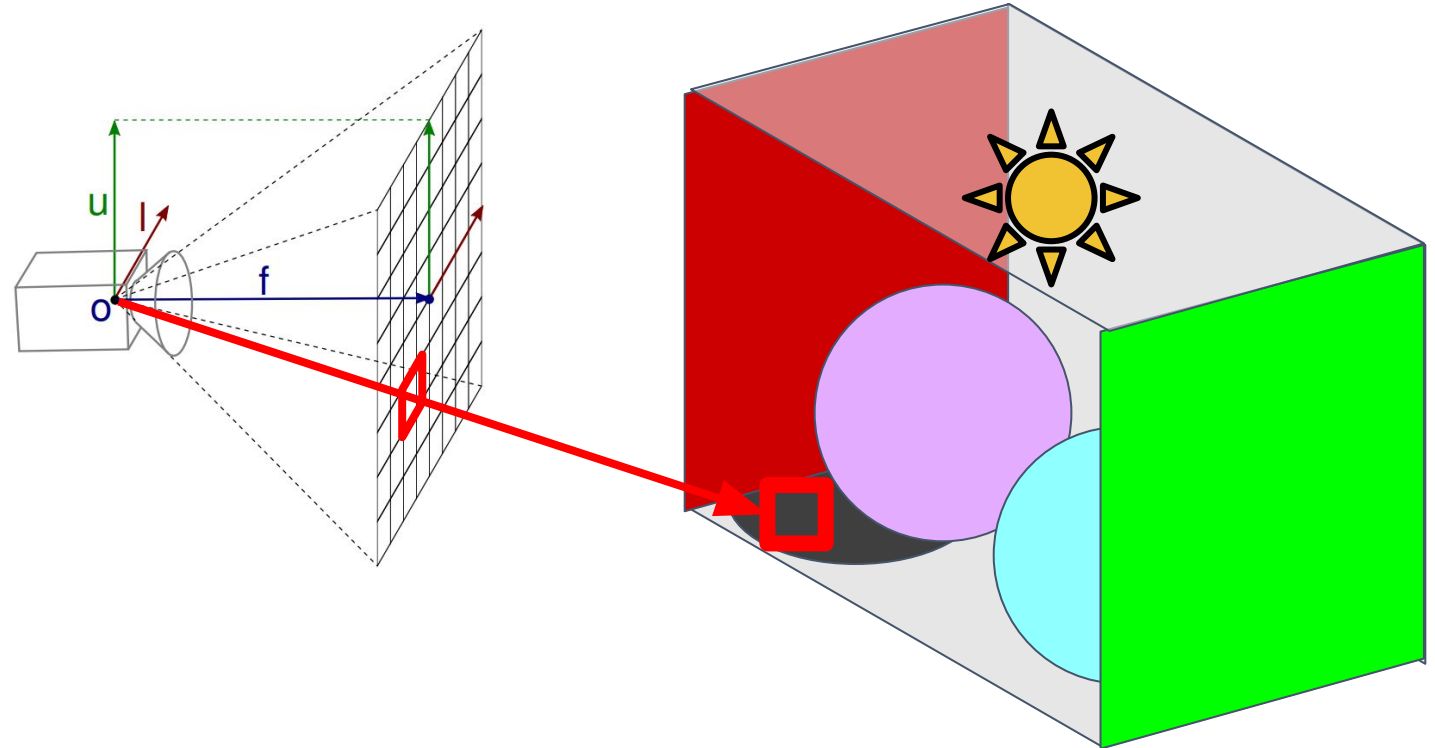
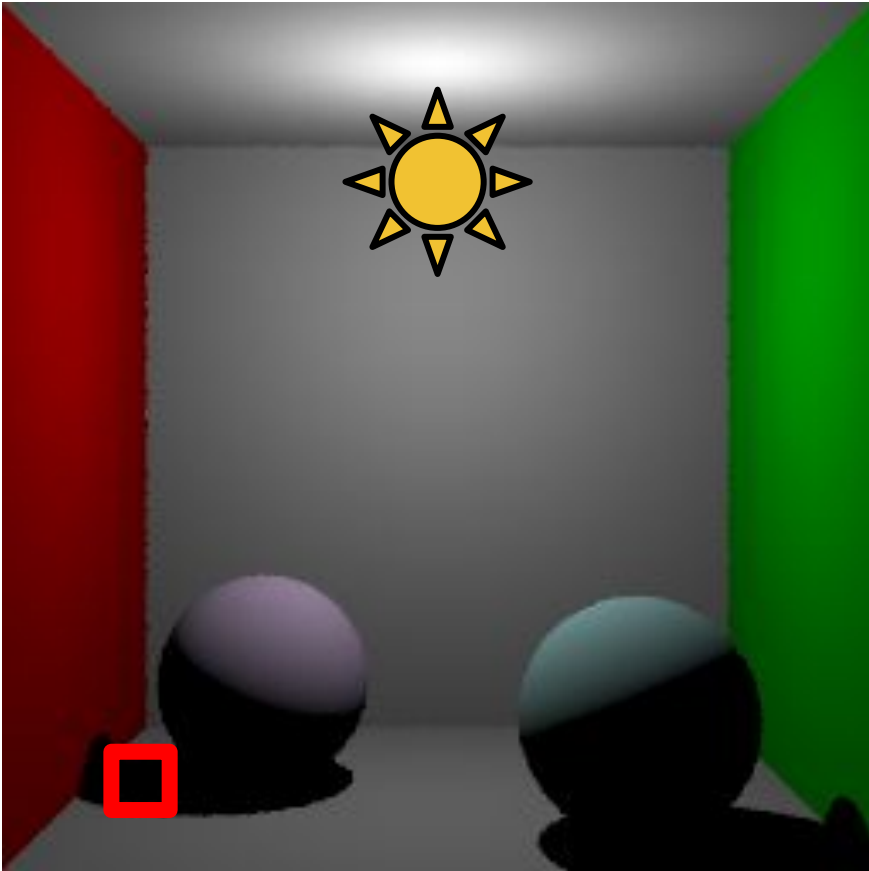
$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{k_d}{\pi}$$

k_d is the *diffuse coefficient*.

Lambertian BRDF is constant. Therefore lighting depends only on the cosine and it is independent from the point of view.

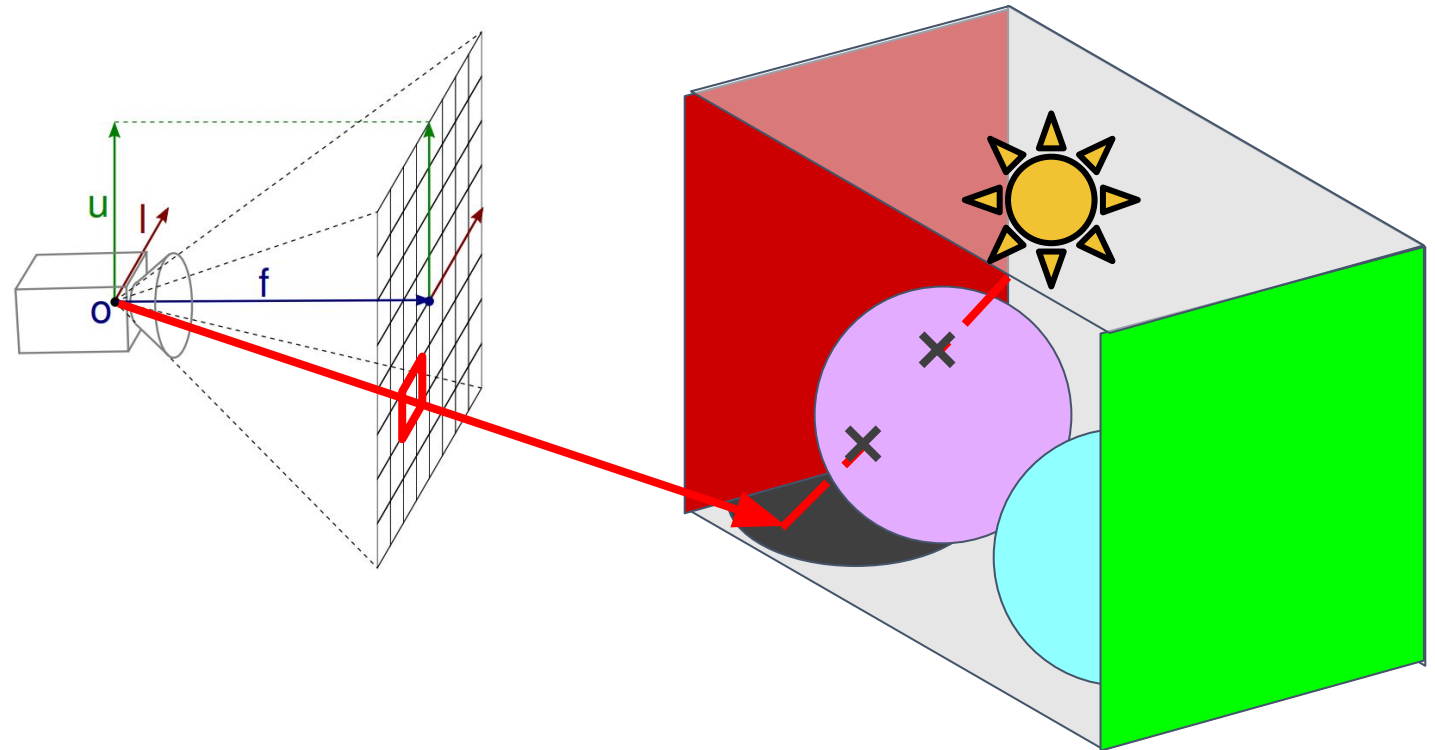
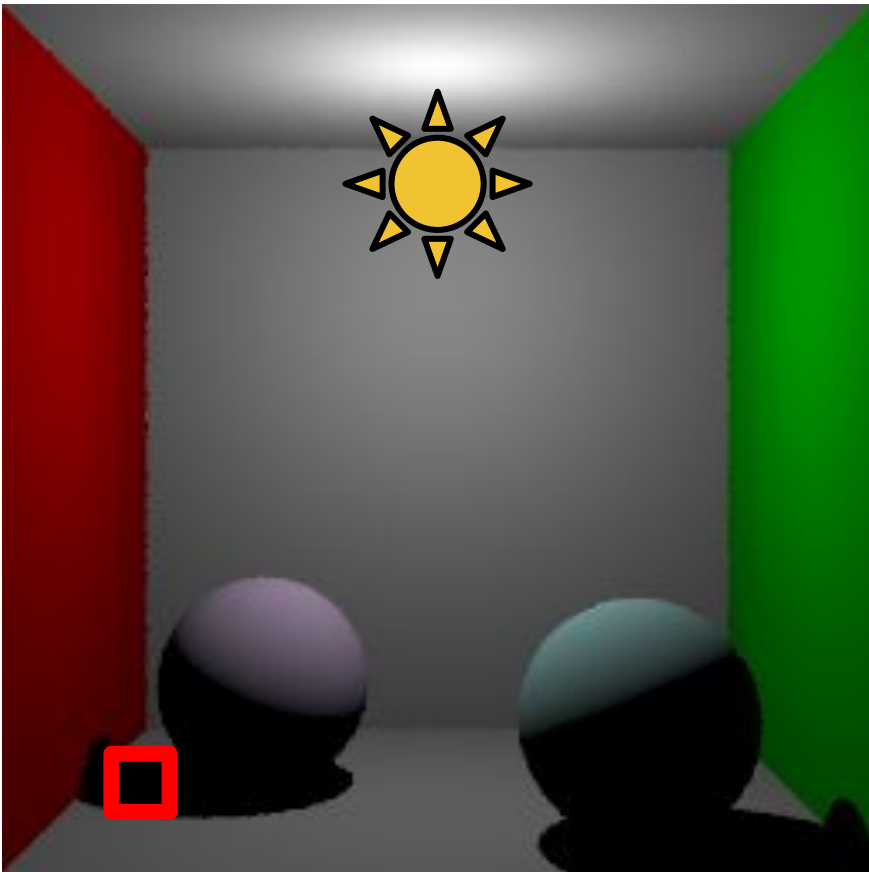
Which color do we fill each pixel with?

- Remember to cast **shadow rays**



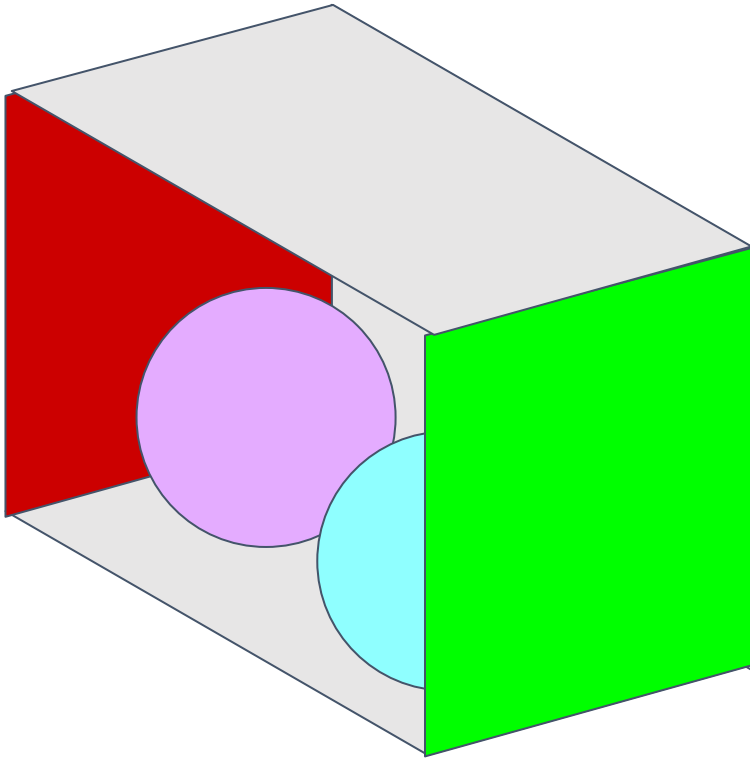
Which color do we fill each pixel with?

- Remember to cast **shadow rays**



Example scene: Cornell Box

- **Geometry**



Planes defined by normal (n) and distance (d)

Left plane $n = (1, 0, 0)$, $d = 1$

Right plane $n = (-1, 0, 0)$, $d = 1$

Floor plane $n = (0, 1, 0)$, $d = 1$

Ceiling plane $n = (0, -1, 0)$, $d = 1$

Back plane $n = (0, 0, -1)$, $d = 1$

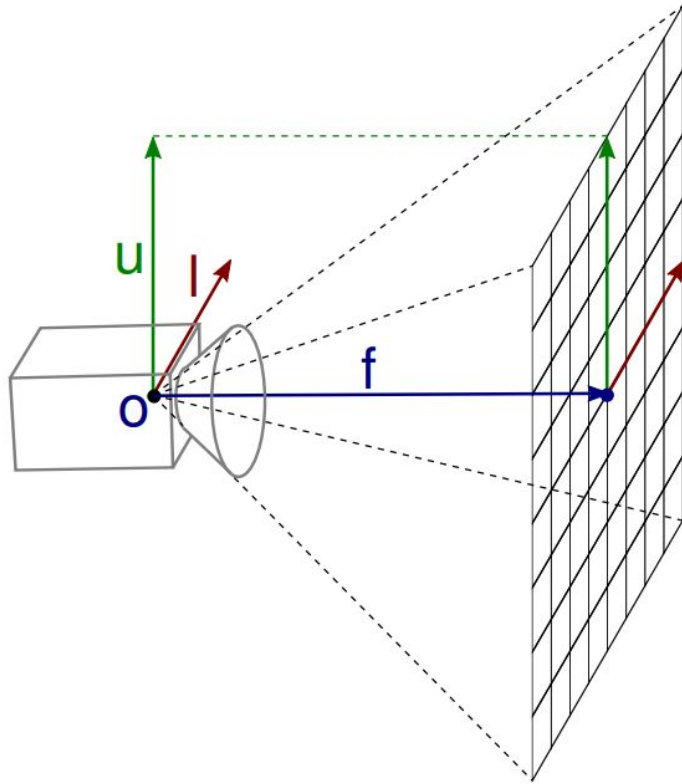
Spheres defined by center (c) and radius (r)

Left sphere $c = (-0.5, -0.7, 0.25)$, $r = 0.3$

Right sphere $c = (0.5, -0.7, -0.25)$, $r = 0.3$

Example scene: Cornell Box

- Camera & light sources



Camera and image plane defined by

Origin $O = (0, 0, -3.5)$

Left $L = (-1, 0, 0)$

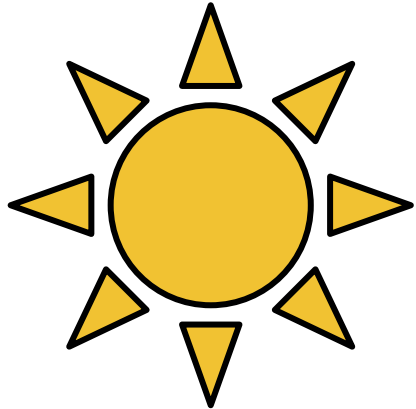
Up $U = (0, 1, 0)$

Forward $F = (0, 0, 3)$

Size 256x256 pixels

Example scene: Cornell Box

- Light sources



Center and power (emission)

Center $c = (0, 0.5, 0)$

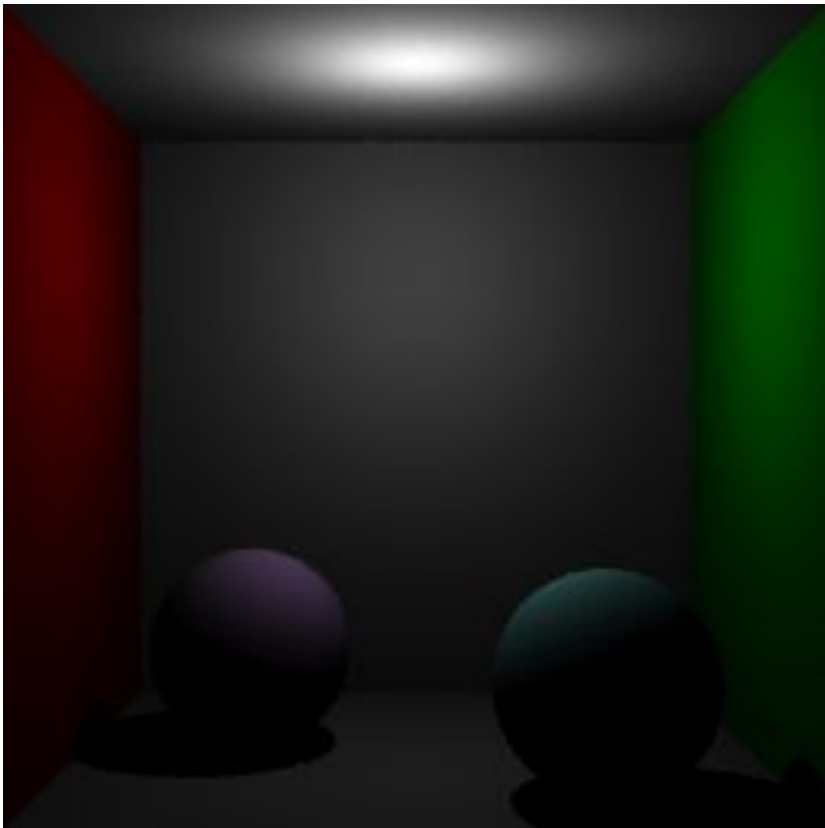
Power can be any number e.g. $p = (1, 1, 1)$

Just be careful with the #MAX

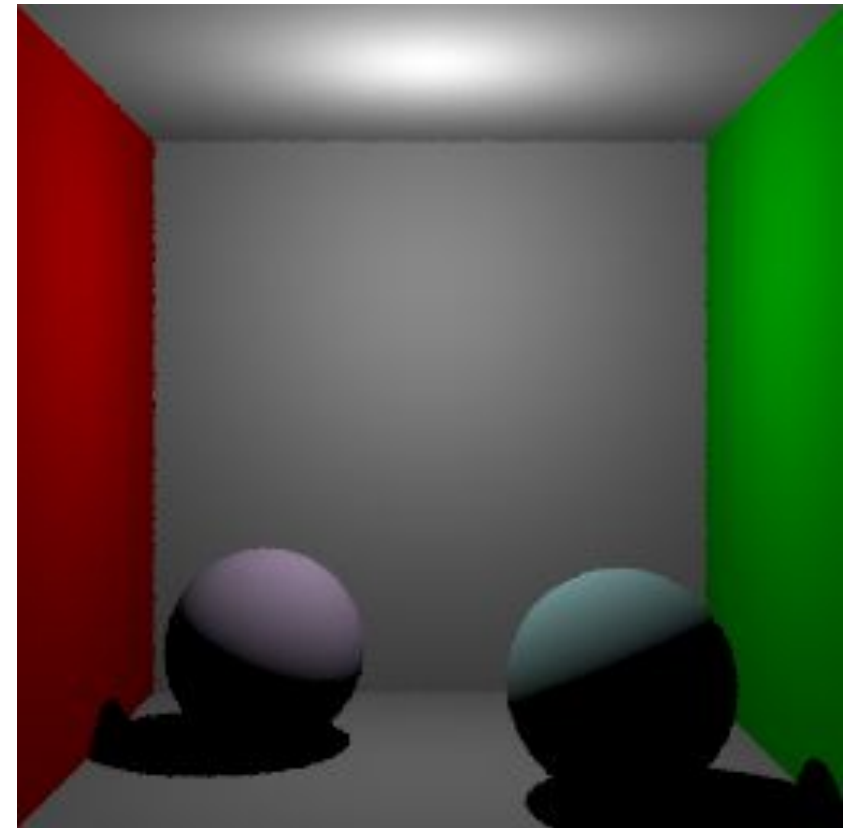
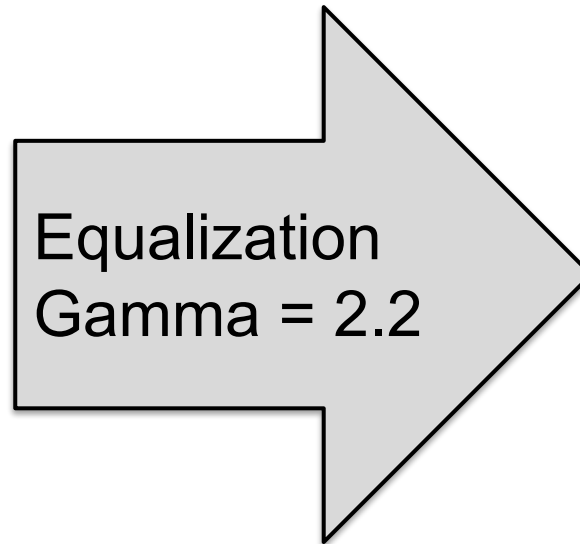
```
1  P3
2  # feep.ppm
3  #MAX=<maximum of your RGB memory values>
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

Example scene: Cornell Box

- Results



Today's result



With tone mapping

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice implement:

- Add **point light sources** to your scene
 - Point sources are defined by **center** (x, y, z) and **power** (r, g, b)
- Extend ray tracing to **calculate direct light** from point lights
 - Cast shadow rays, calculate direct light contribution if path is not occluded
- Recommended deadline: November 13th (moodle: January 11th)
 - Extensions (do not count towards recommended deadline):
 - **Textures:** make diffuse coefficient k_d depend on hit position $k_d(\mathbf{x})$
 - **Fresnel effects:** make diffuse coefficient k_d depend on viewing direction $k_d(\omega_o)$
 - **Parallelization:** divide work between several threads, estimate time to finish execution
 - **More:** importance sampling next-event estimation, etc. (see the lab assignment or ask us)

Lab #4 – Path tracing (part 2)

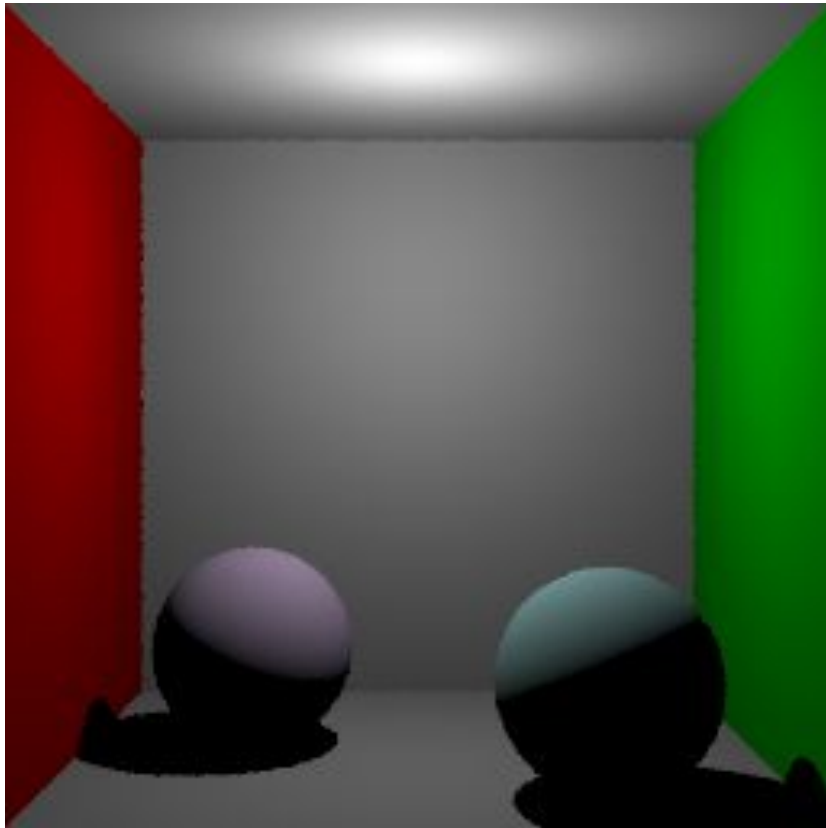
Informática Gráfica

Adolfo Muñoz - Julio Marco
Pablo Luesia - J. Daniel Subías – Óscar Pueyo

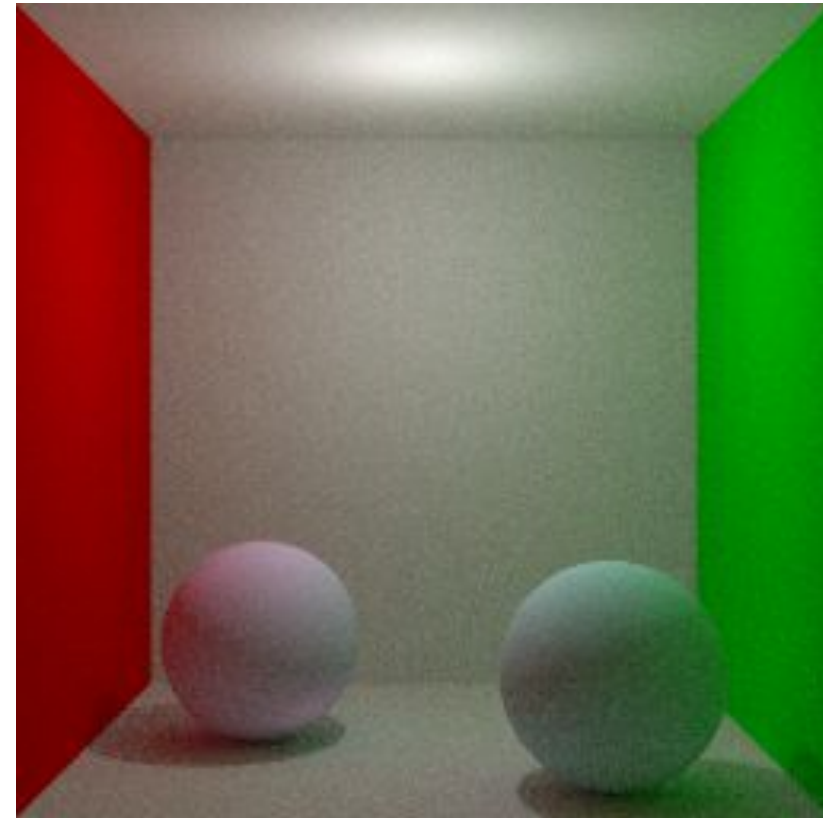


Before we begin...

- Today: calculate direct illumination **and** indirect illumination



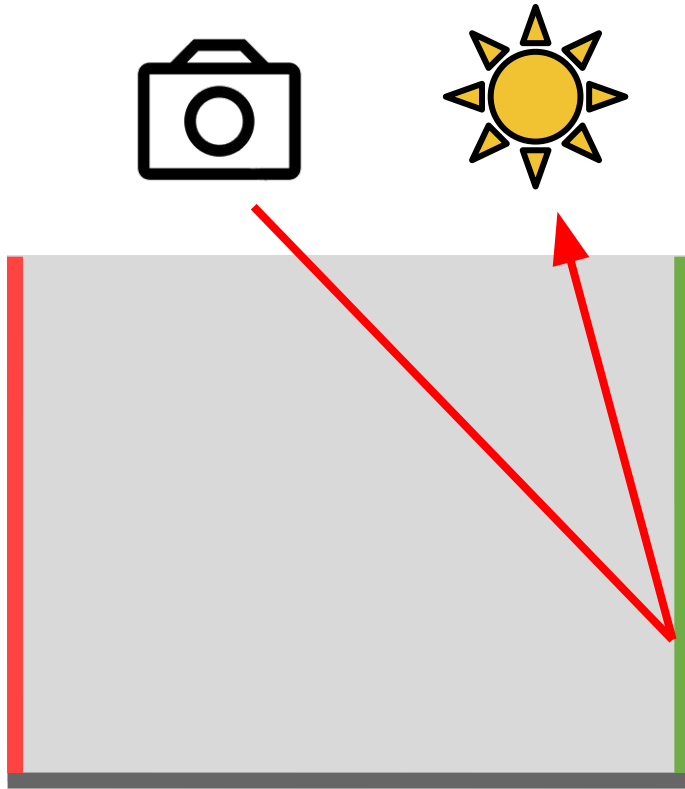
Previous session (direct light)



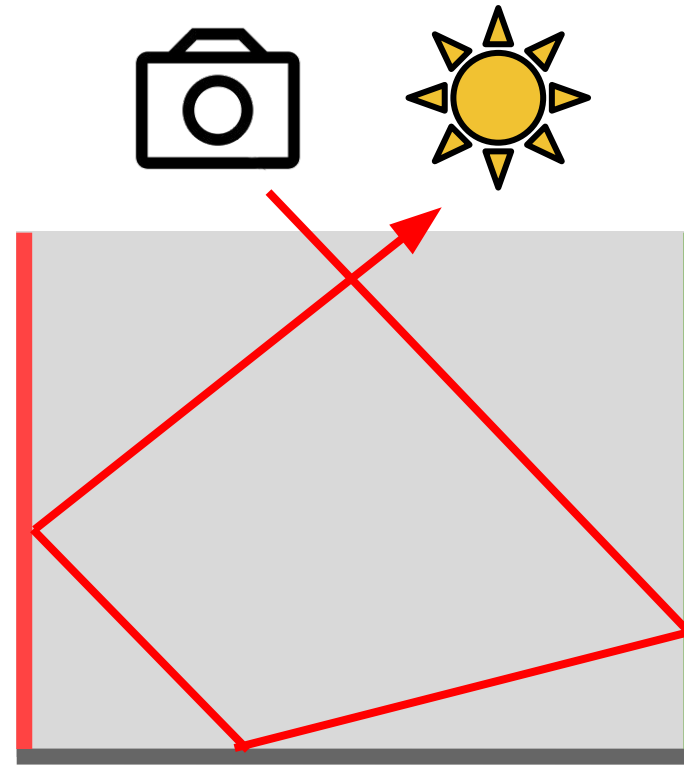
Full path tracing (+ indirect light)

Before we begin...

- Today: calculate direct illumination **and** indirect illumination



Previous session (direct light)

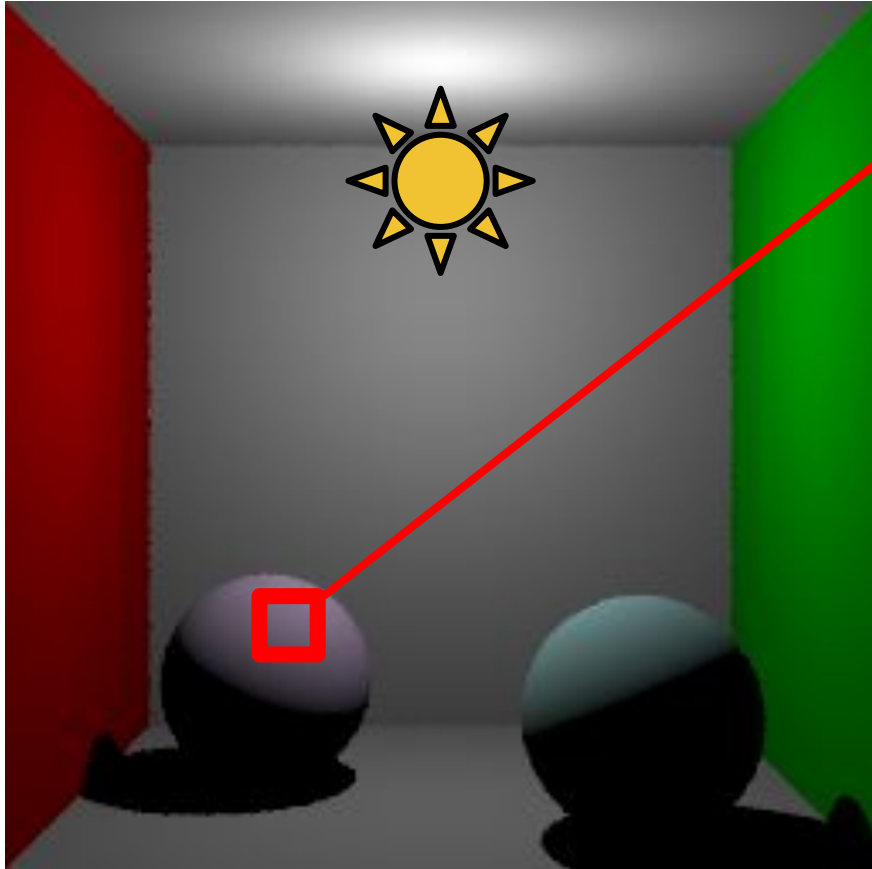


Full path tracing (+ indirect light)

Before we begin...

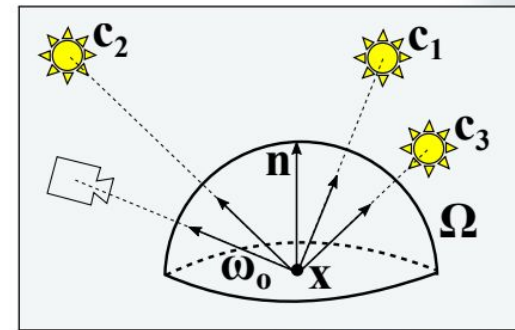
- Warning: there is a lot of theory on these slides, handle with care
 - If you want to go straight to the point/programming, go to the end
- Lab 4 (path tracing) **is the first submitted work**
 - Recommended deadline: November 13th
 - Moodle: January 11th
 - You will use most of today's code for Lab 5 (photon mapping) too
- Remember: Final work is 80% of the final grade

Previously: only direct illumination



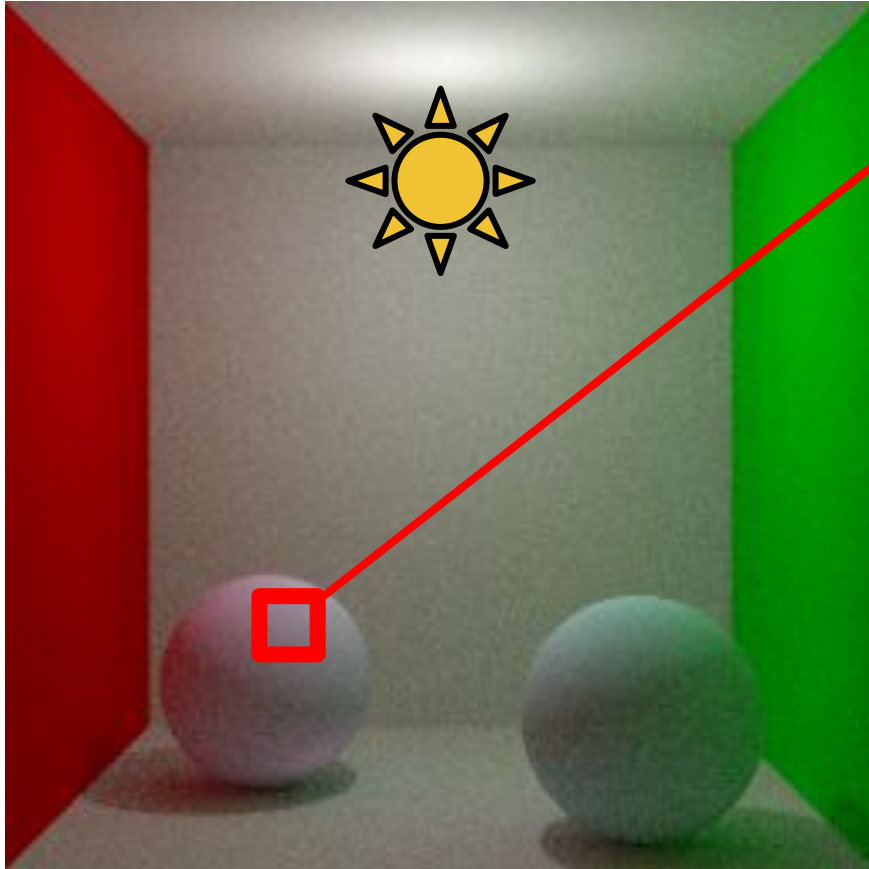
$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

But just the direct light



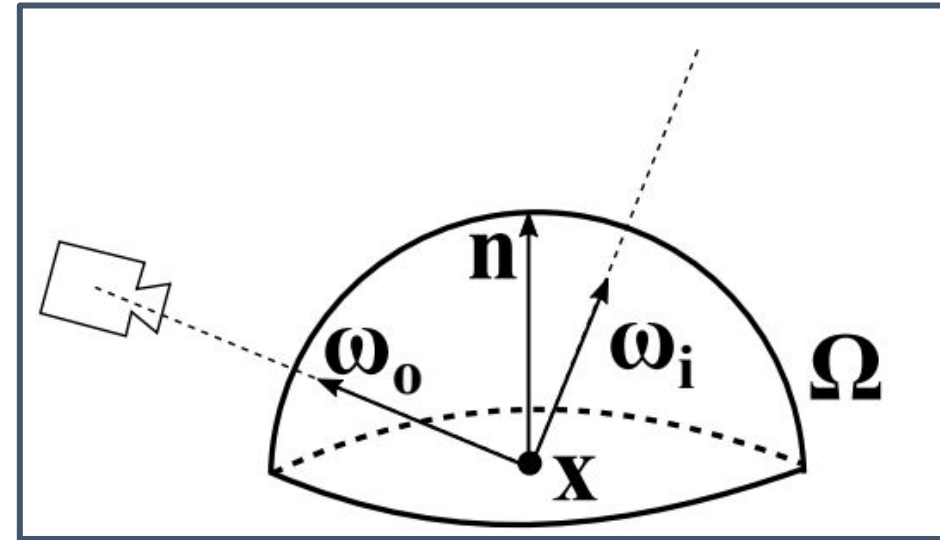
$$L_o(\mathbf{x}, \omega_o) = \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$

Which color do we fill each pixel with?



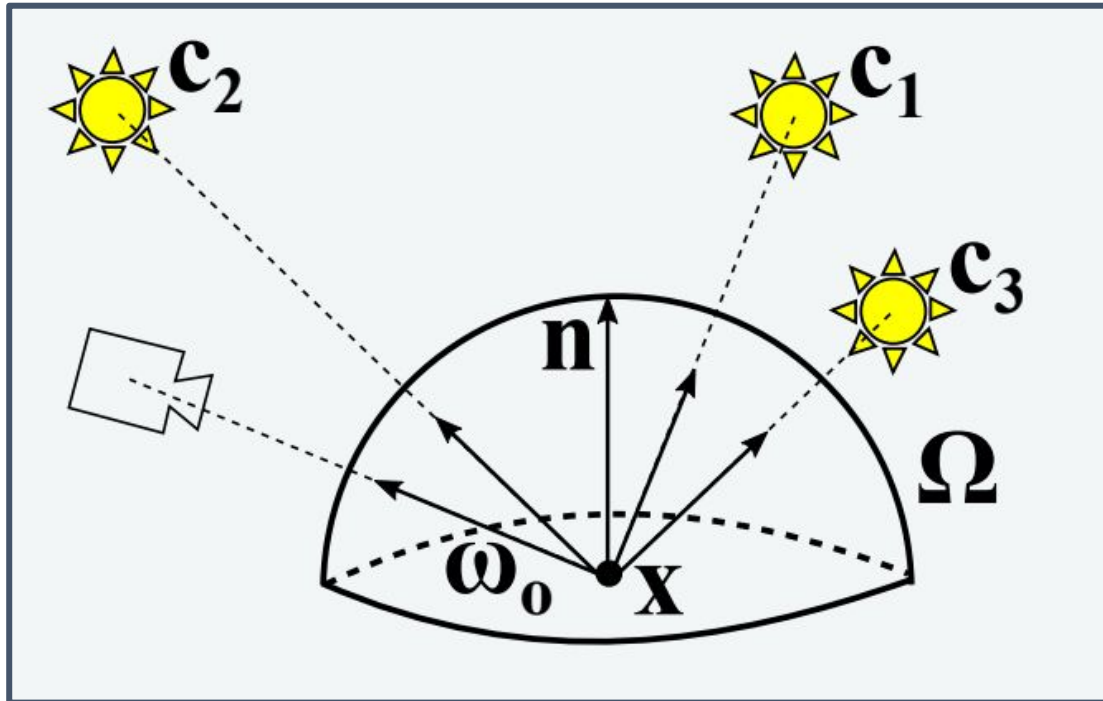
$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

The full integral



Computing the path integral

- How do we compute $\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$?
- On the previous session (direct light):



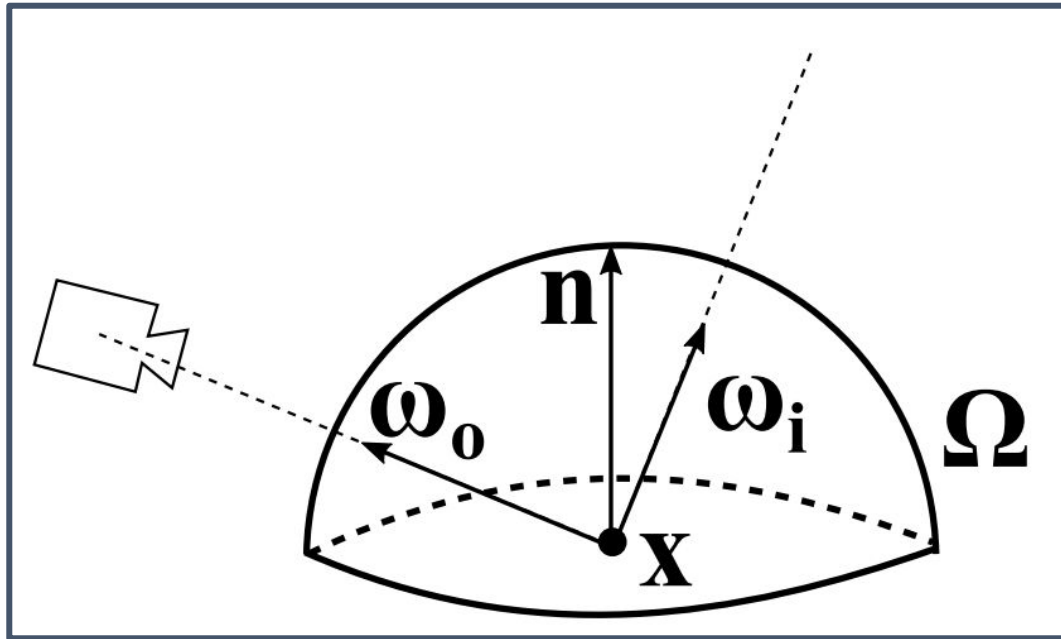
$$= \sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$

- Paths have exactly one bounce
- Closed-form solution

Computing the path integral

- How do we compute $\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$?

- Today (direct light + indirect light):



$$= \int_{\Omega_1} L_i(\mathbf{x}_1, \omega_{i1}) f_r(\mathbf{x}_1, \omega_{i1}, \omega_{o1}) |\mathbf{n}_1 \cdot \omega_{i1}| d\omega_{i1}$$

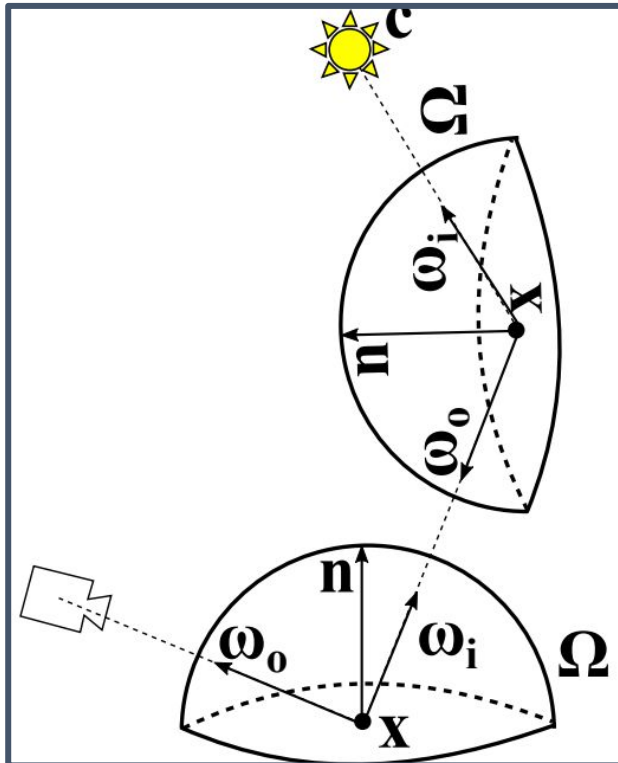
- Integrate over the whole hemisphere

Computing the path integral

- How do we compute

$$\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i \quad ?$$

- Today (direct light + indirect light):



$$= \int_{\Omega_1} \left(\int_{\Omega_2} L_i(\mathbf{x}_2, \omega_{i2}) f_r(\mathbf{x}_2, \omega_{i2}, -\omega_{i1}) |\mathbf{n}_2 \cdot \omega_{i2}| d\omega_{i1} \right) f_r(\mathbf{x}_1, -\omega_{i1}, \omega_{o1}) |\mathbf{n}_1 \cdot \omega_{i1}| d\omega_{i1}$$

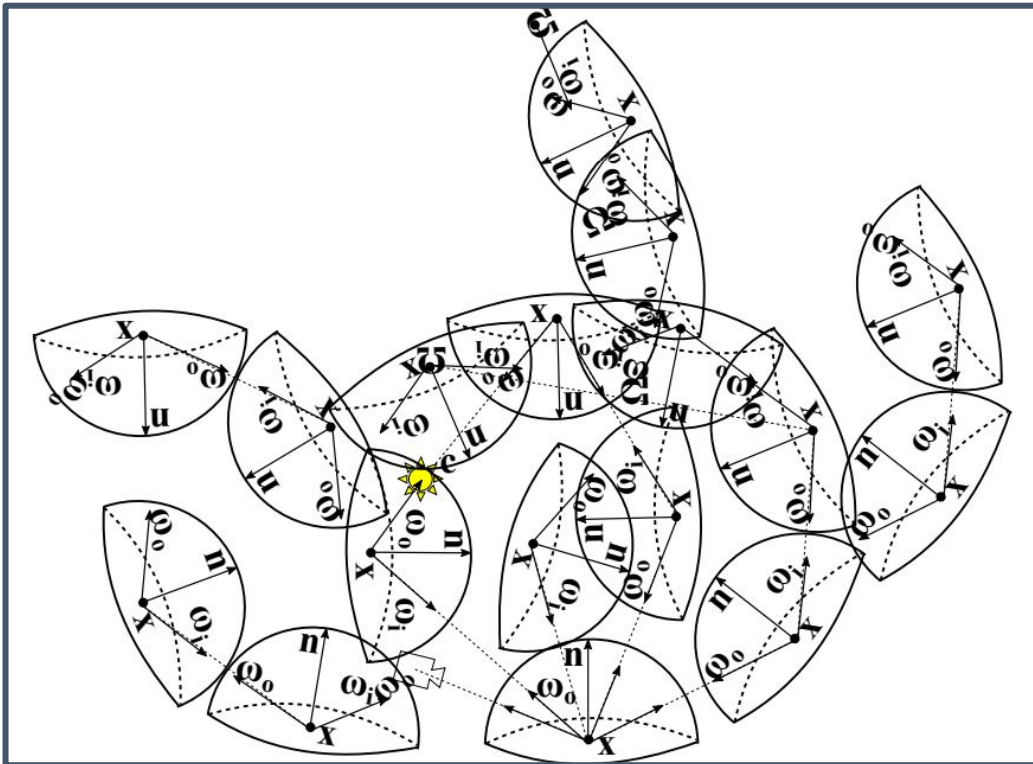
- Integrate over the whole hemisphere²

Computing the path integral

- How do we compute

$$\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i \quad ?$$

- Today (direct light + indirect light):



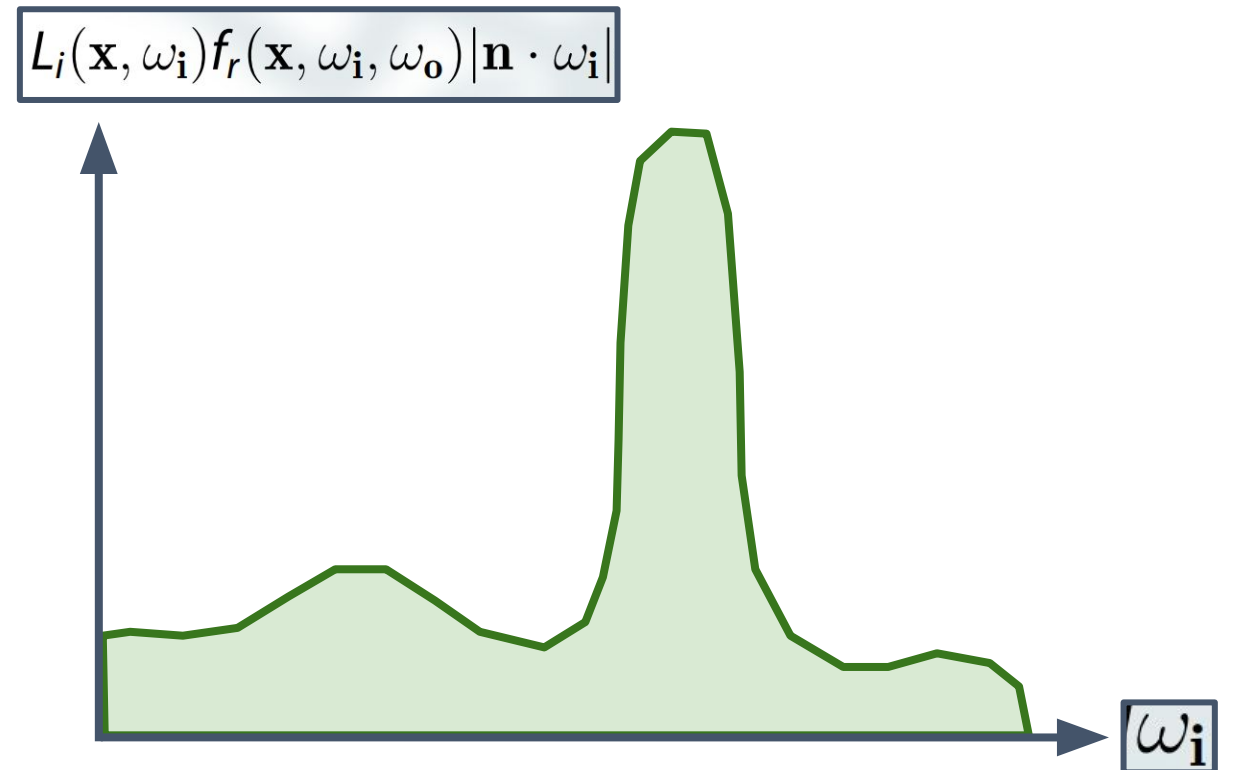
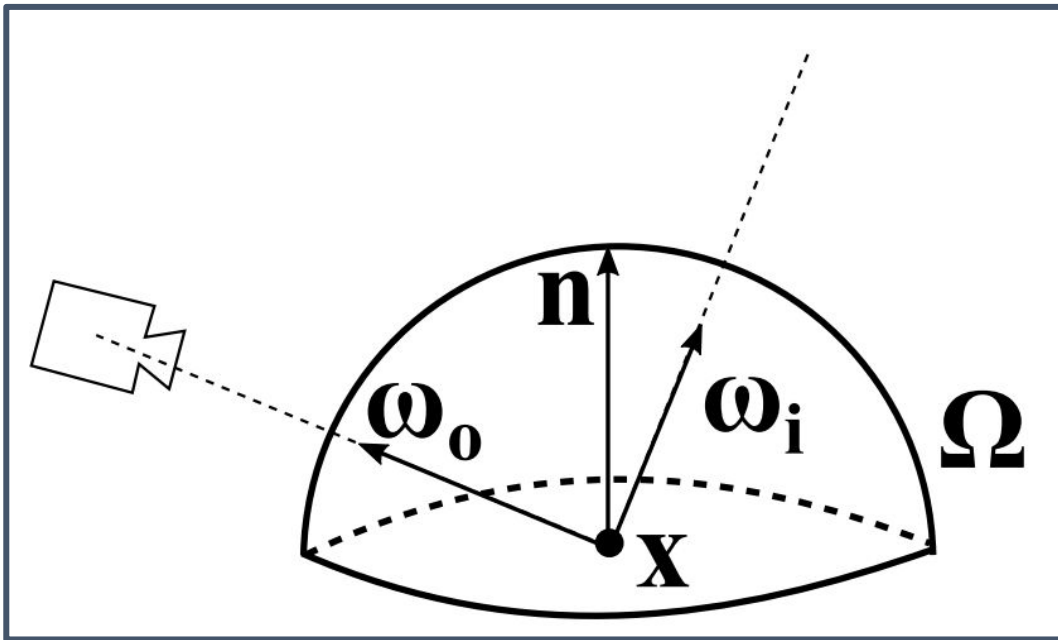
$$= \int_{\Omega_1} \cdots \int_{\Omega_N} L_i(\mathbf{x}_N, \omega_{N2}) \left(\prod_{k=2}^N f_r(\mathbf{x}_k, \omega_{ik}, -\omega_{i(k-1)}) \right) f_r(\mathbf{x}_1, \omega_{i1}, \omega_{o1})$$
$$\left(\prod_{k=1}^N |\mathbf{n}_k \cdot \omega_{ik}| \right) d\omega_{iN} \cdots d\omega_{i1}$$

- Paths can have 1..N bounces
- How do we solve this integral?

Approximating one integral

- How do we compute

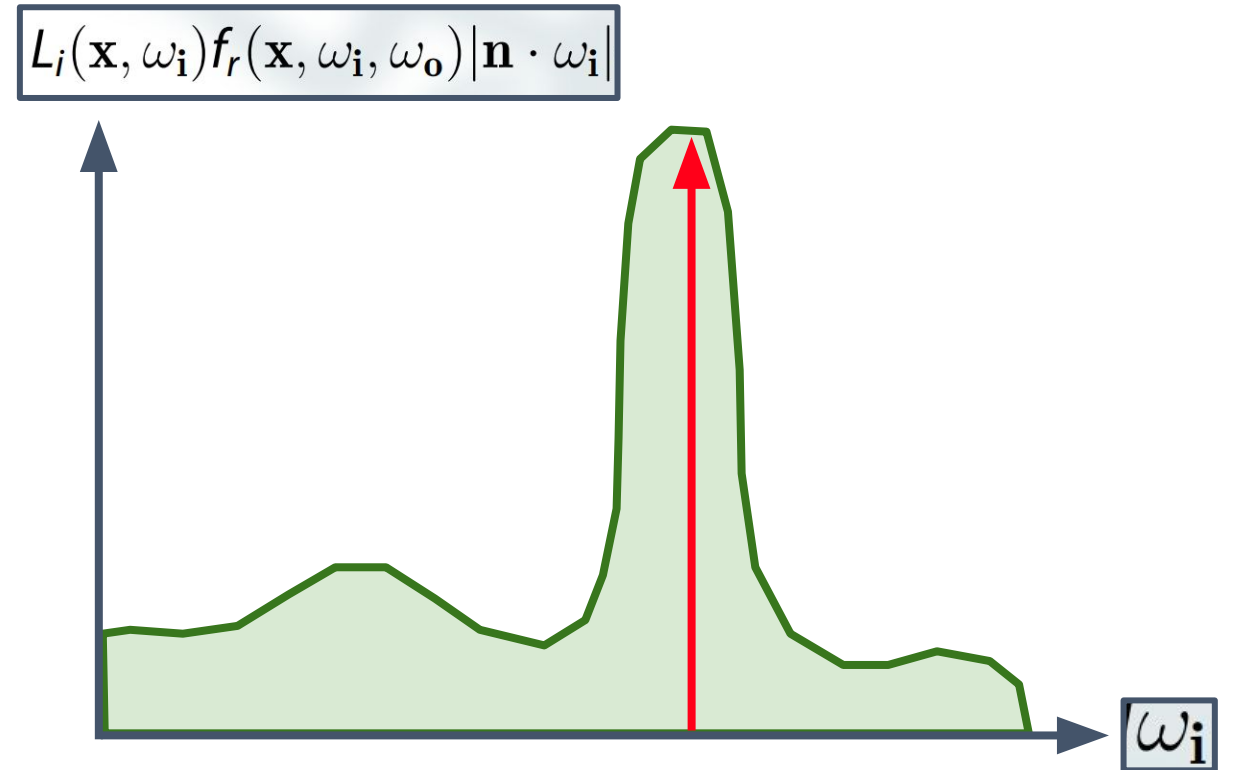
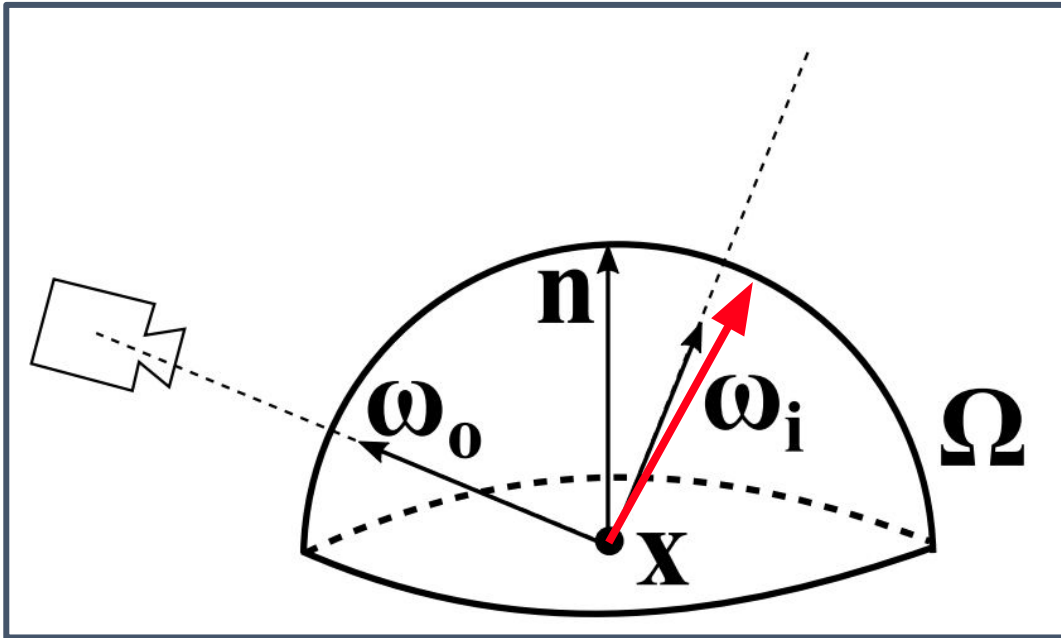
$$\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i \quad ?$$



Approximating one integral

- How do we compute

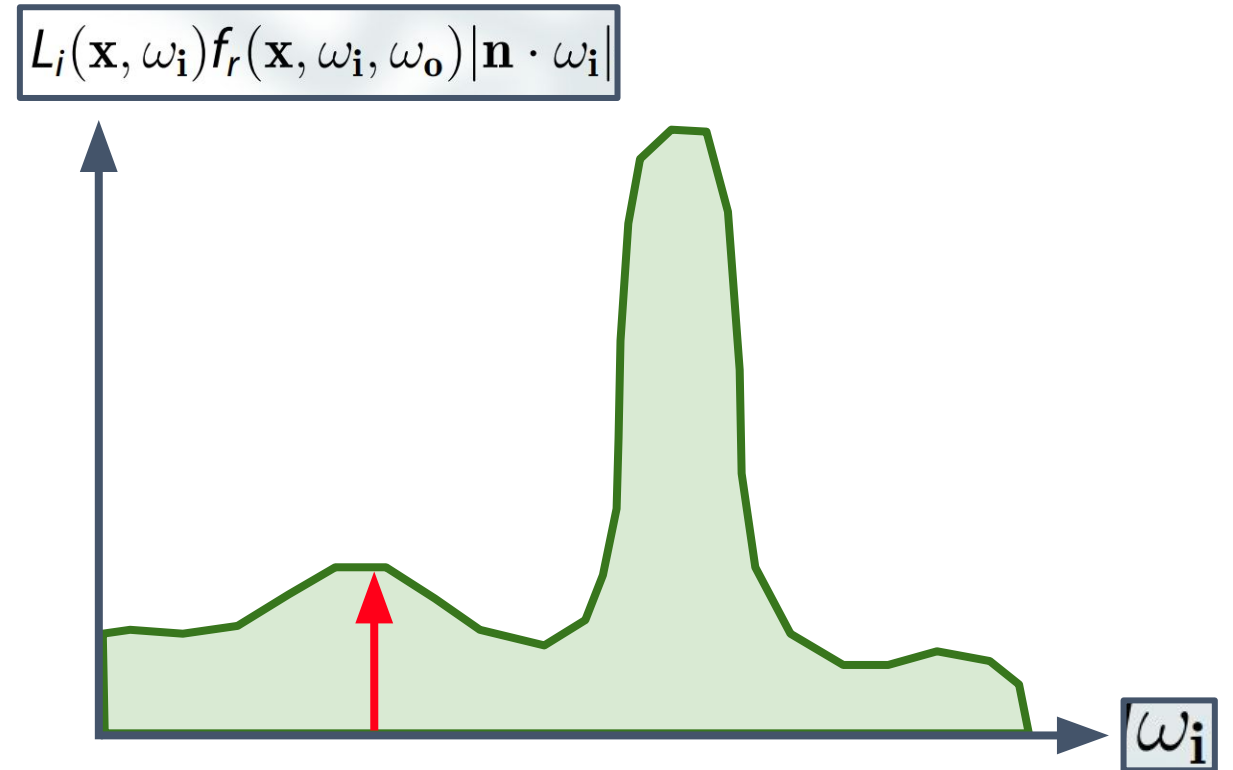
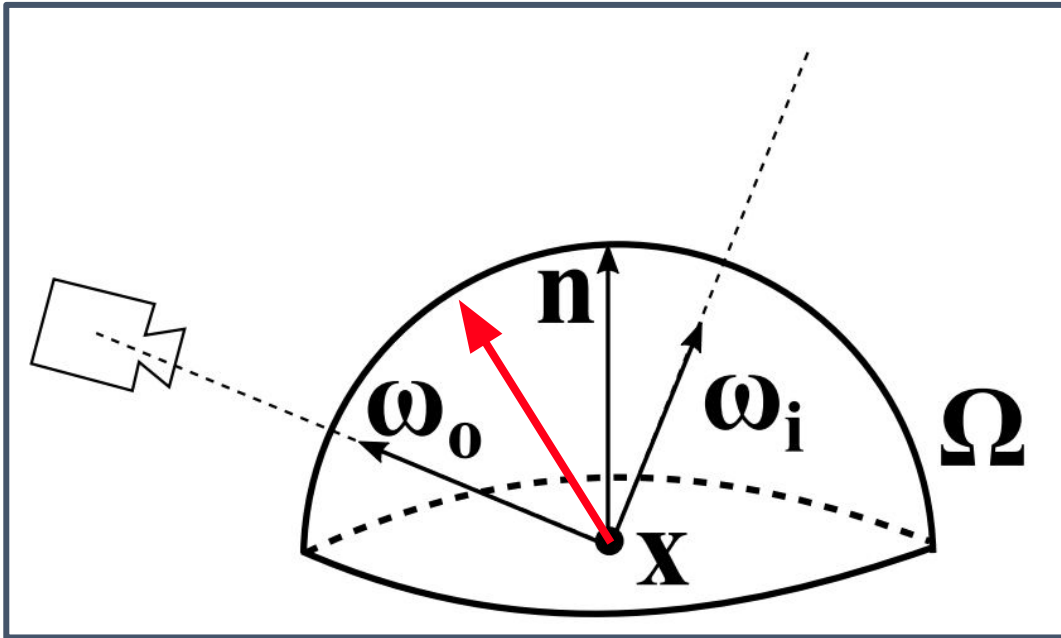
$$\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i \quad ?$$



Approximating one integral

- How do we compute

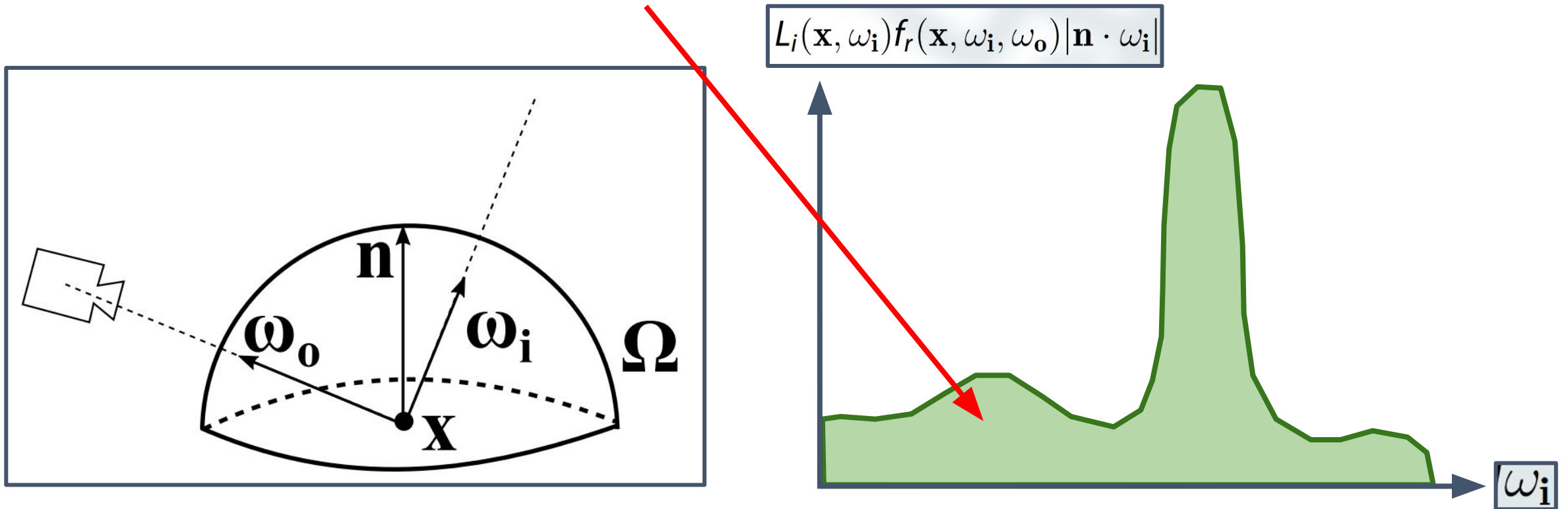
$$\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i \quad ?$$



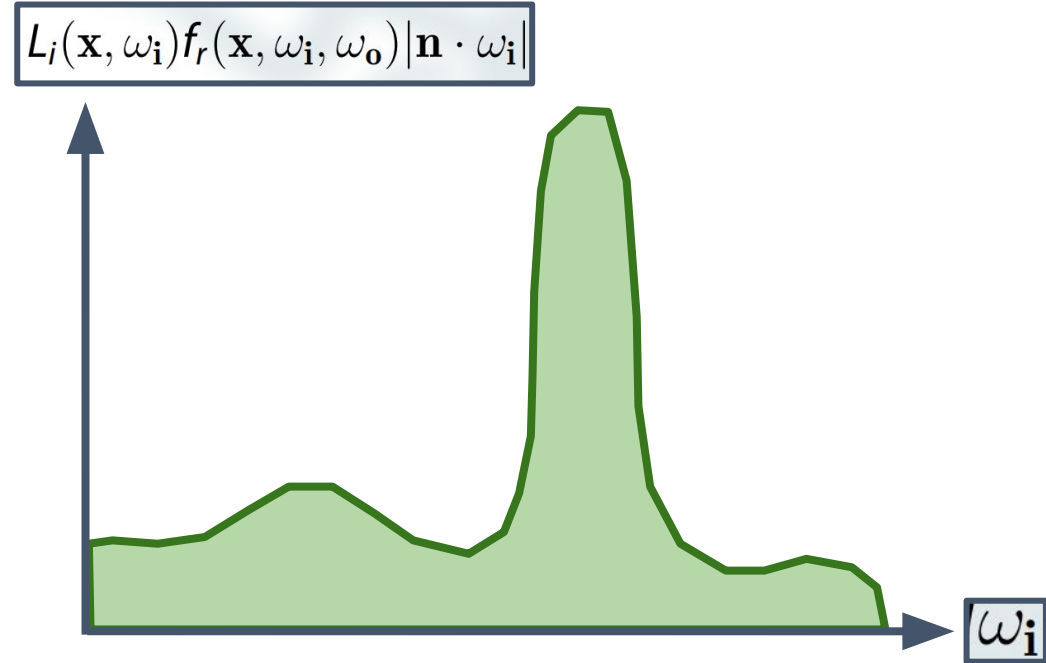
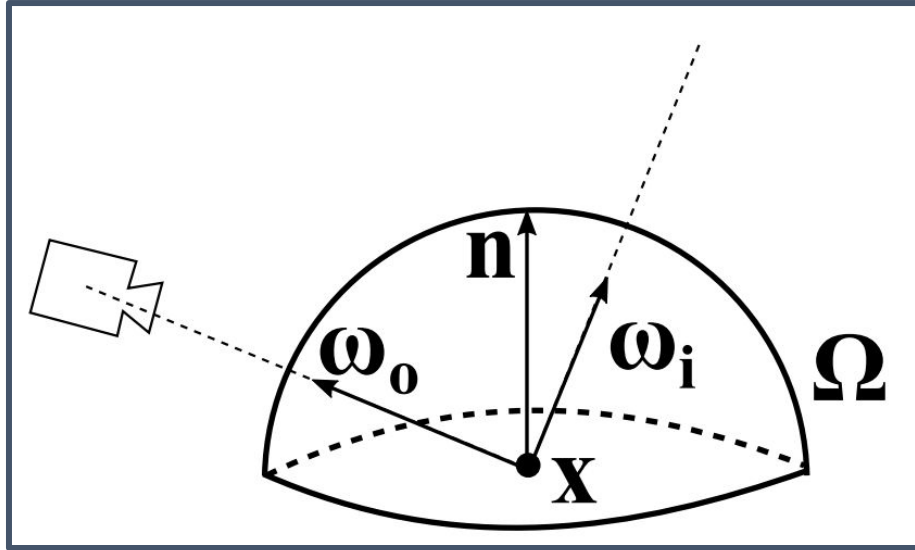
Approximating one integral

- How do we compute $\int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$?

- Compute this green area under the curve

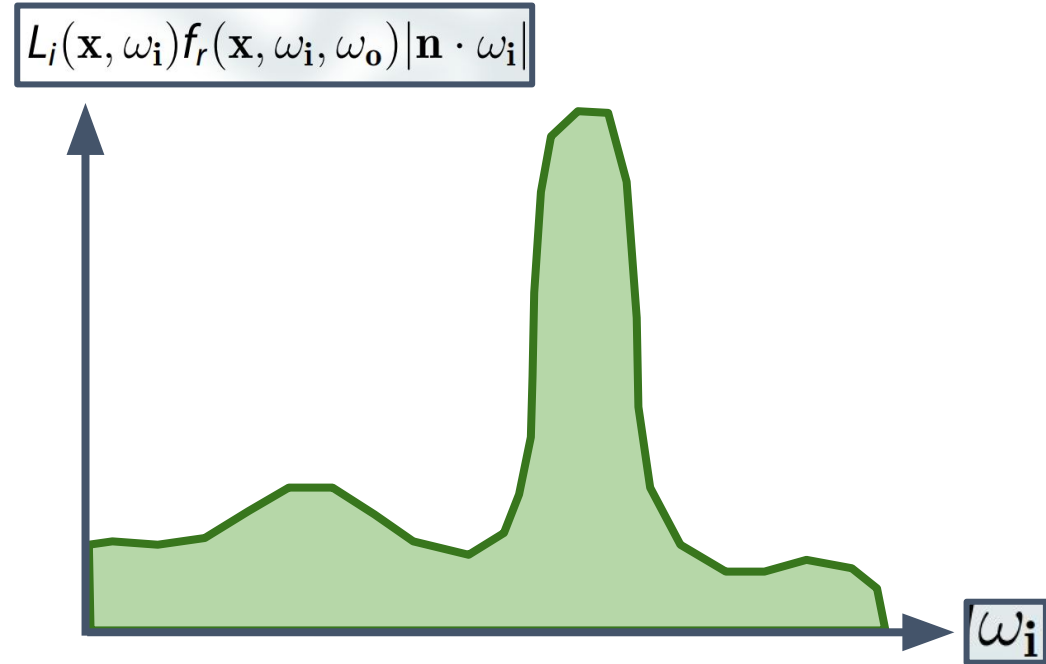
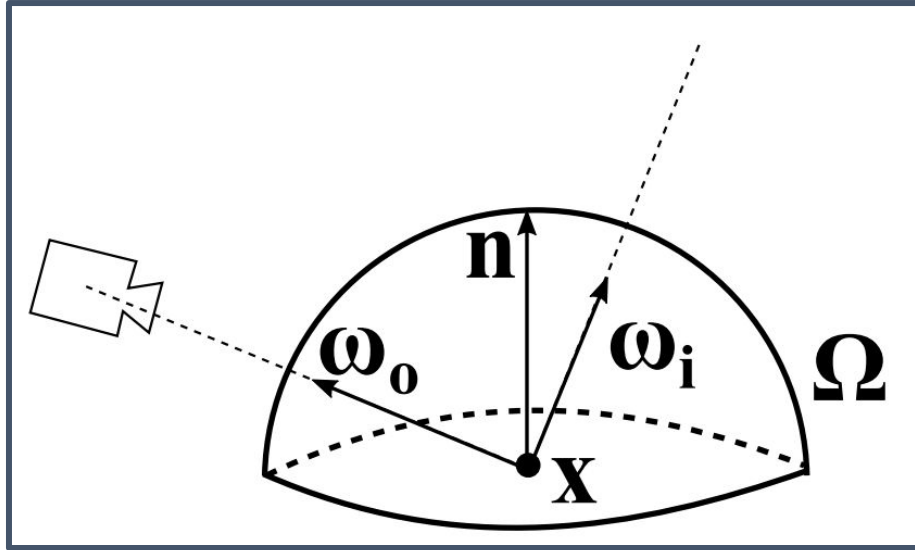


Approximating one integral



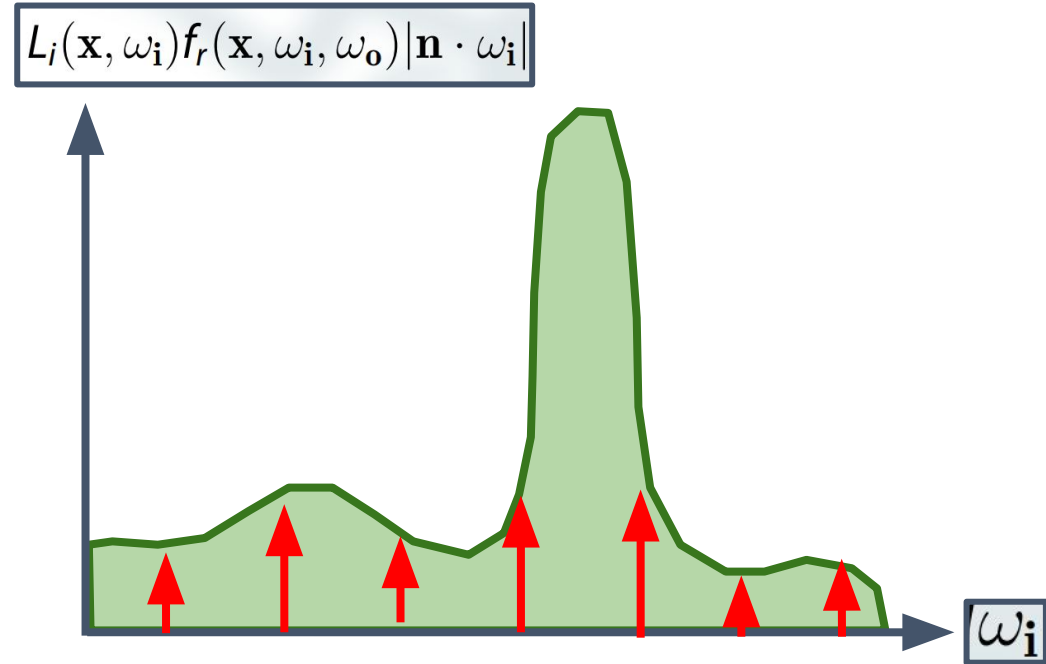
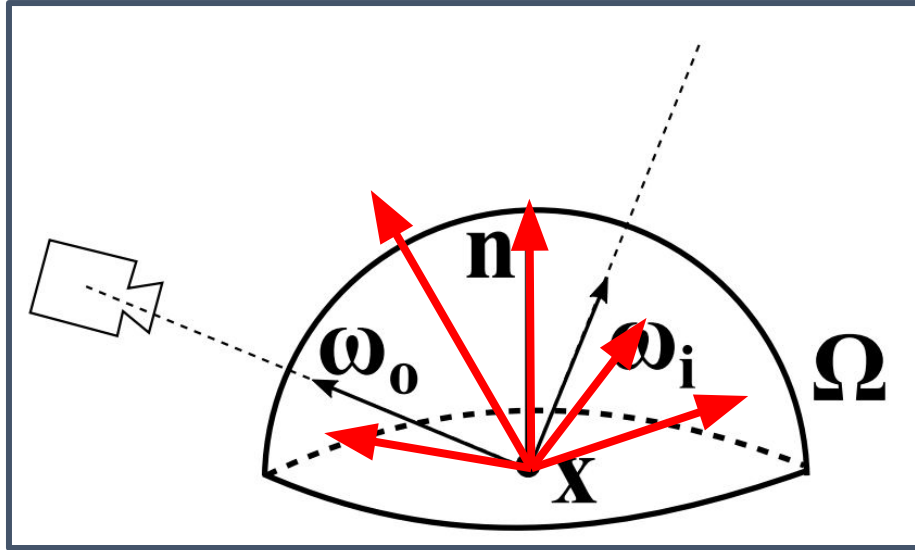
- There are infinite values for ω_i

Approximating one integral



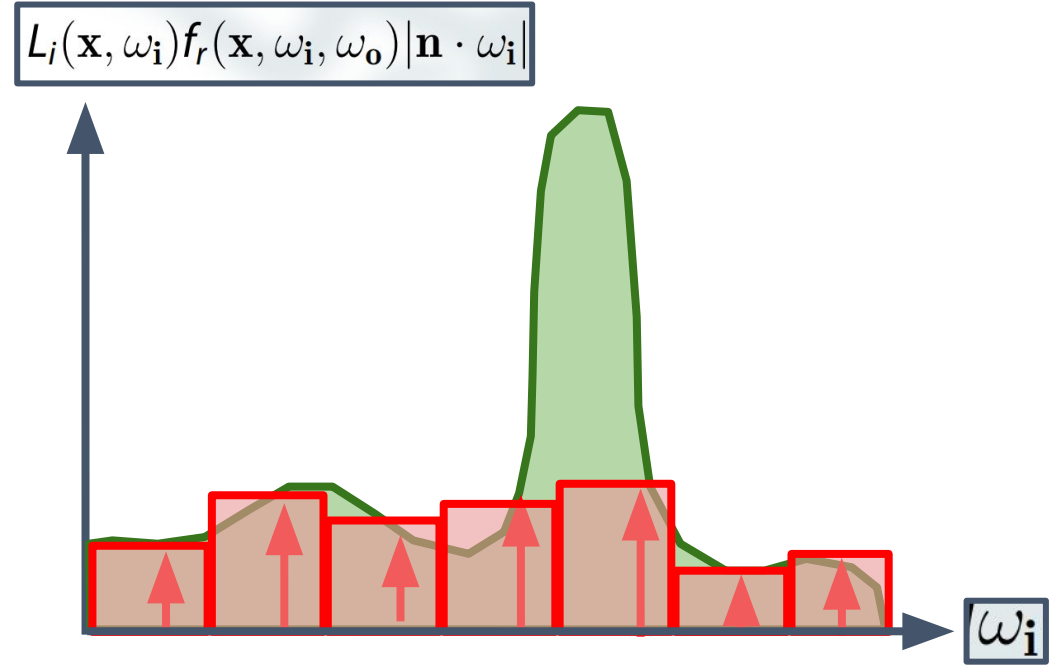
- There are infinite values for ω_i
- Idea 1: approximate using a finite amount of (evenly spaced) samples

Approximating one integral



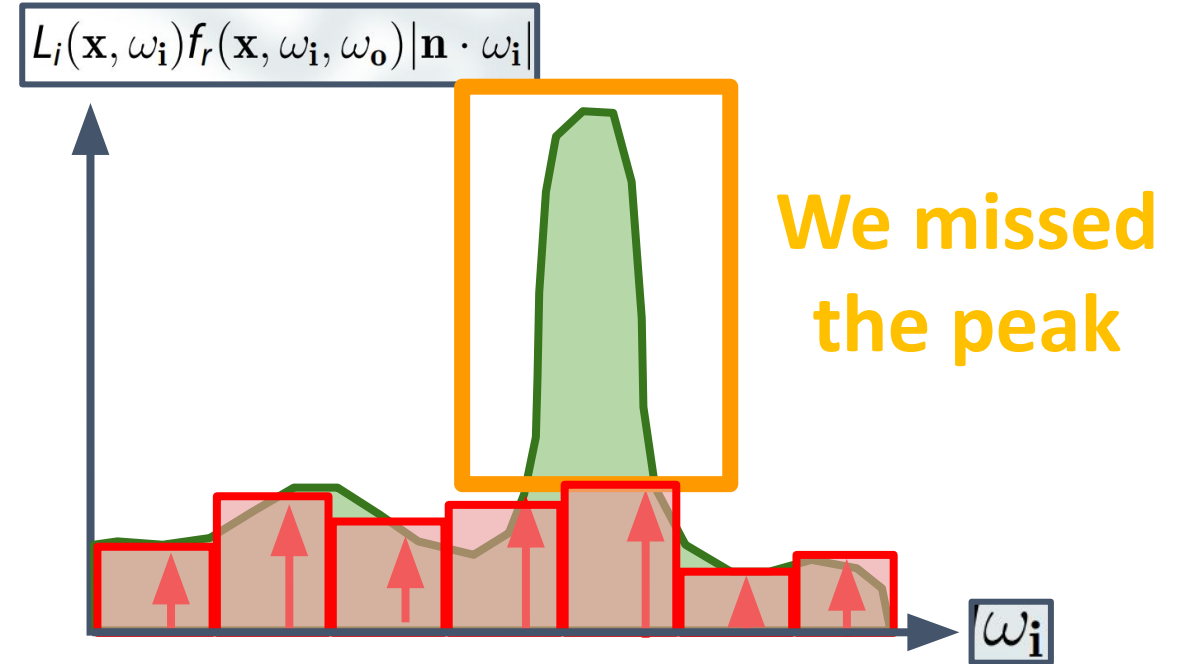
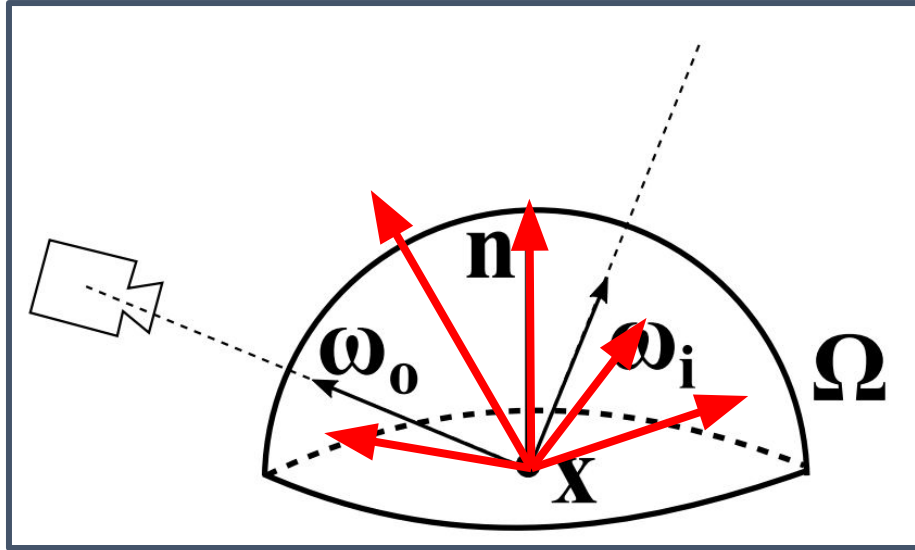
- There are infinite values for ω_i
- Idea 1: approximate using a finite amount of (evenly spaced) samples

**Graphics and
Imaging Lab**



-
- A diagram illustrating the approximation of a continuous function by a sum of rectangular pulses. On the left, a green-filled area under a curve represents a continuous function. This is followed by an approximation symbol $\approx$. To the right, a sequence of seven red-outlined rectangles of varying widths and heights are shown, separated by plus signs, representing the sum of rectangular pulses that approximate the continuous function.

Approximating one integral

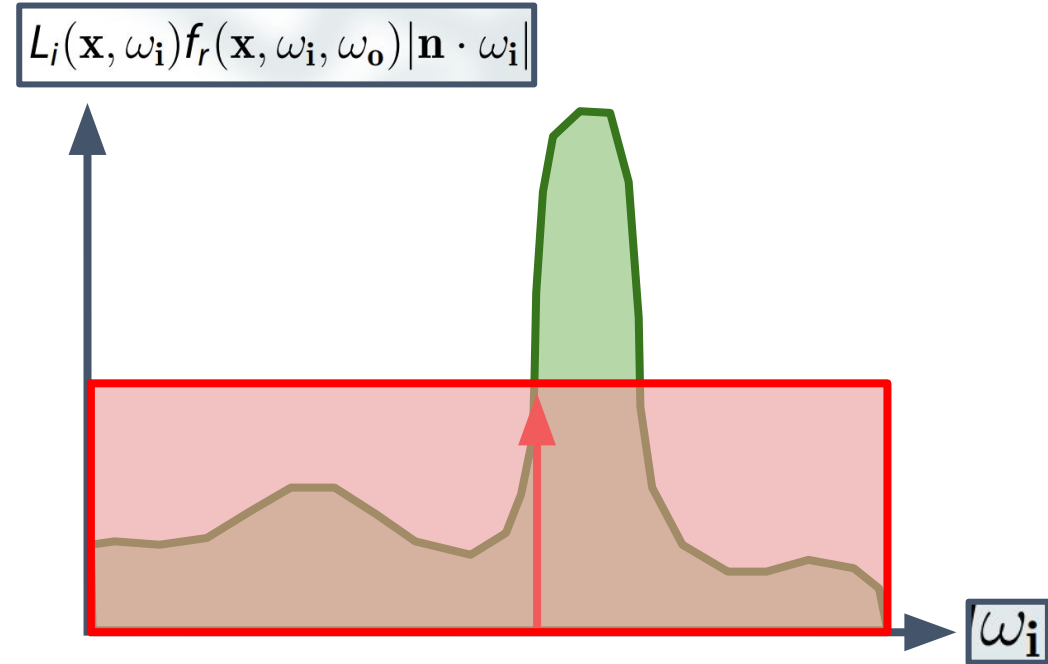
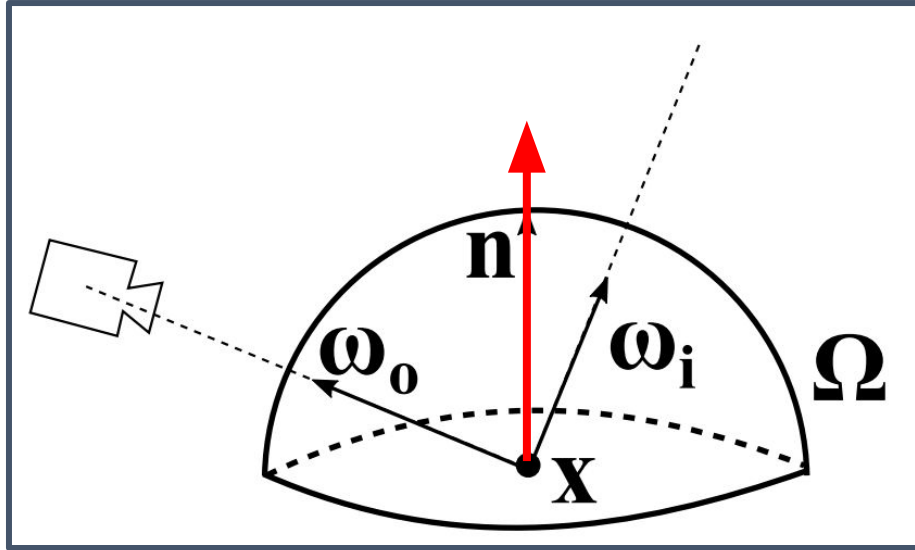


- There are infinite values for ω_i
- Idea 1: approximate using a finite amount of (evenly spaced) samples

A diagram illustrating the approximation of a function (green curve) using a sum of rectangles (red boxes). The equation is:

$$\text{Function} \approx \text{rectangle}_1 + \text{rectangle}_2 + \text{rectangle}_3 + \text{rectangle}_4 + \text{rectangle}_5 + \text{rectangle}_6 + \text{rectangle}_7$$

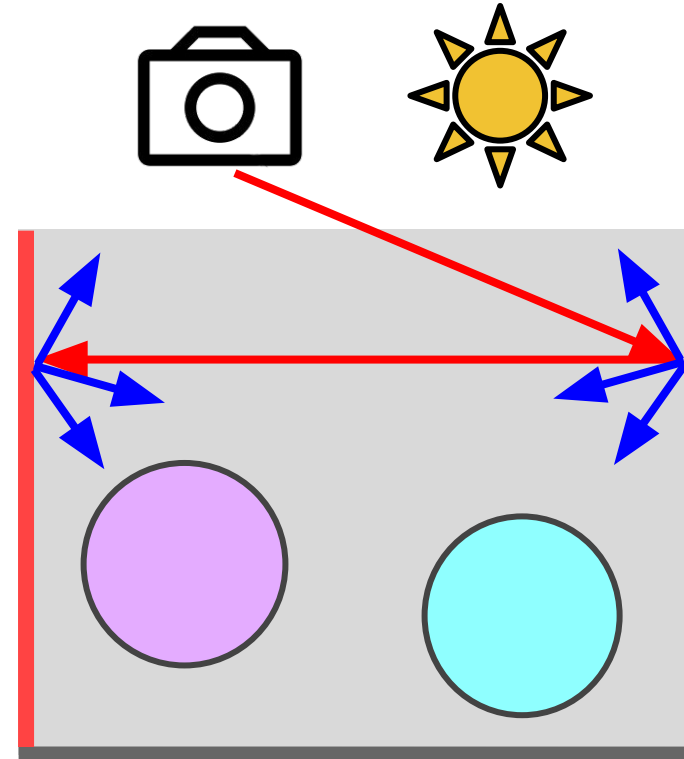
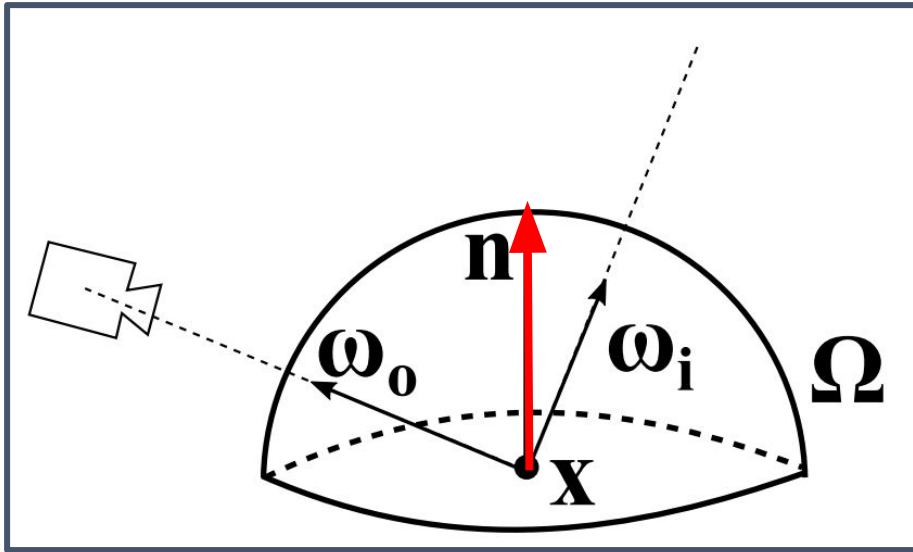
Approximating one integral



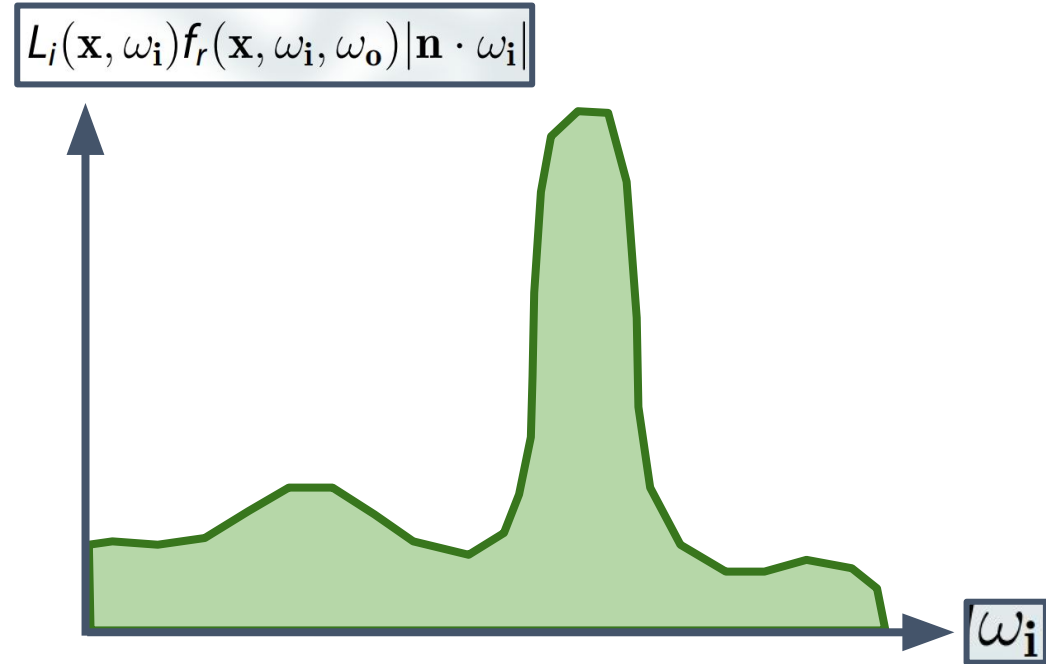
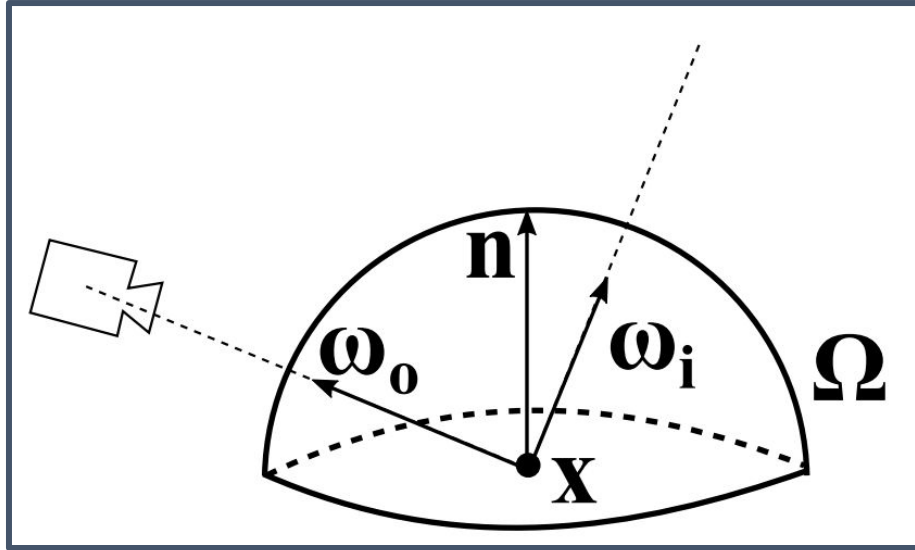
- There are infinite values for ω_i
- Idea 1: approximate using a finite amount of (evenly spaced) samples
 - Example of one deterministic sample

Approximating one integral in practice

- If all rays go towards $\mathbf{n}$, you ignore the rest of the hemisphere
 - Notice this will also happen with N samples
- Produces what's called a biased result (ignores )

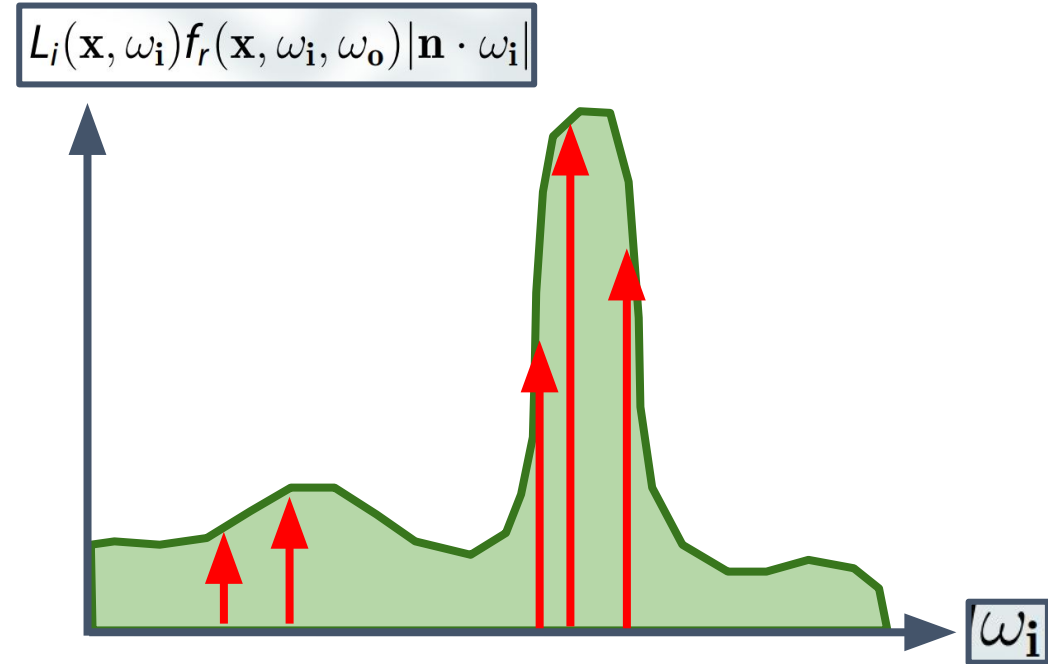
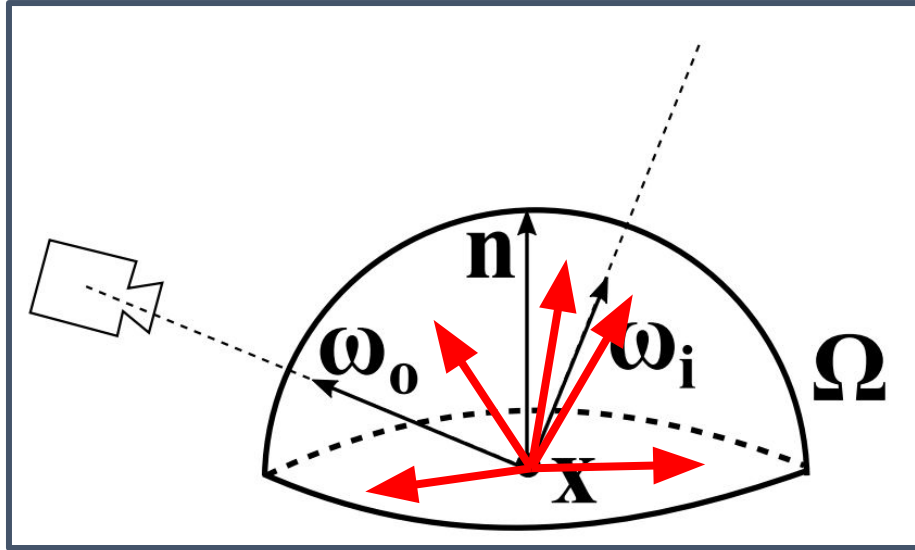


Approximating one integral



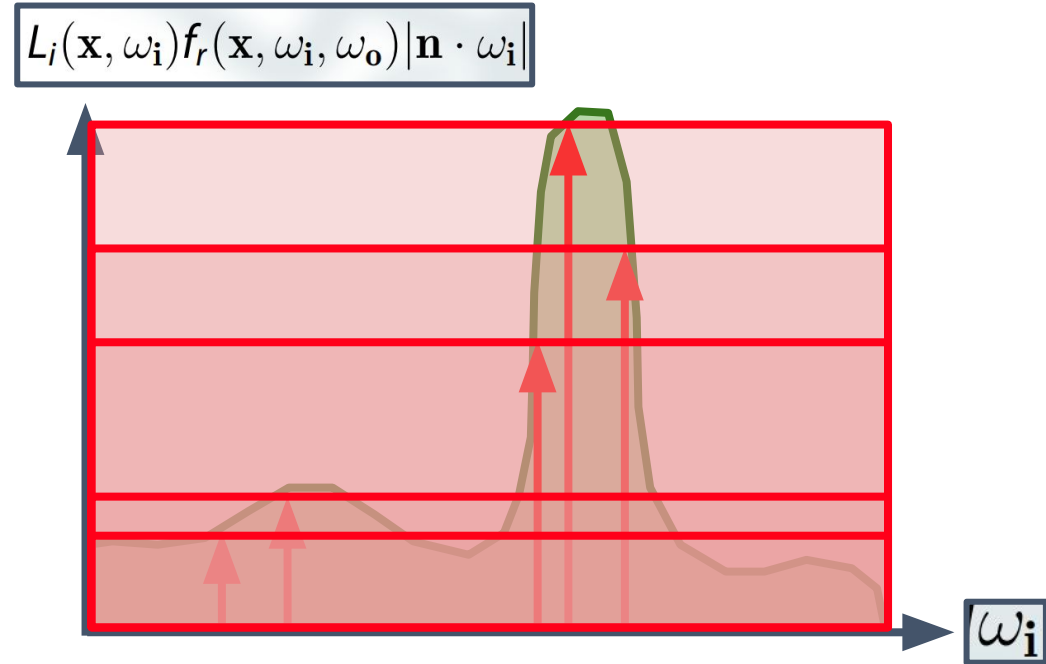
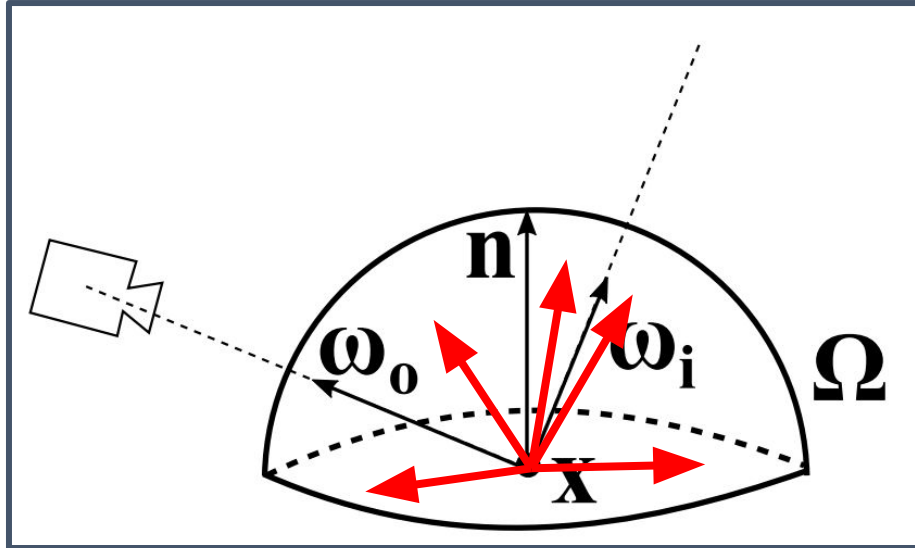
- There are infinite values for ω_i
- ~~Idea 1: approximate using a finite amount of (evenly spaced) samples~~
- Idea 2: **Monte Carlo estimator**, use the mean of $\mathbf{N}$ random samples

Approximating one integral



- There are infinite values for ω_i
- ~~Idea 1: approximate using a finite amount of (evenly spaced) samples~~
- Idea 2: **Monte Carlo estimator**, use the mean of N random samples

Approximating one integral

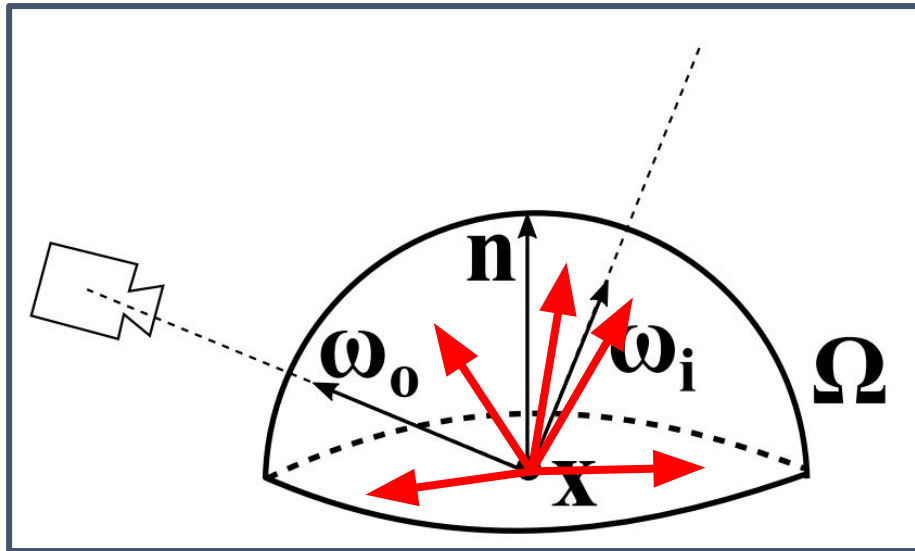


- There are infinite values for ω_i
- Idea 2: **Monte Carlo estimator**, use the mean of N random samples

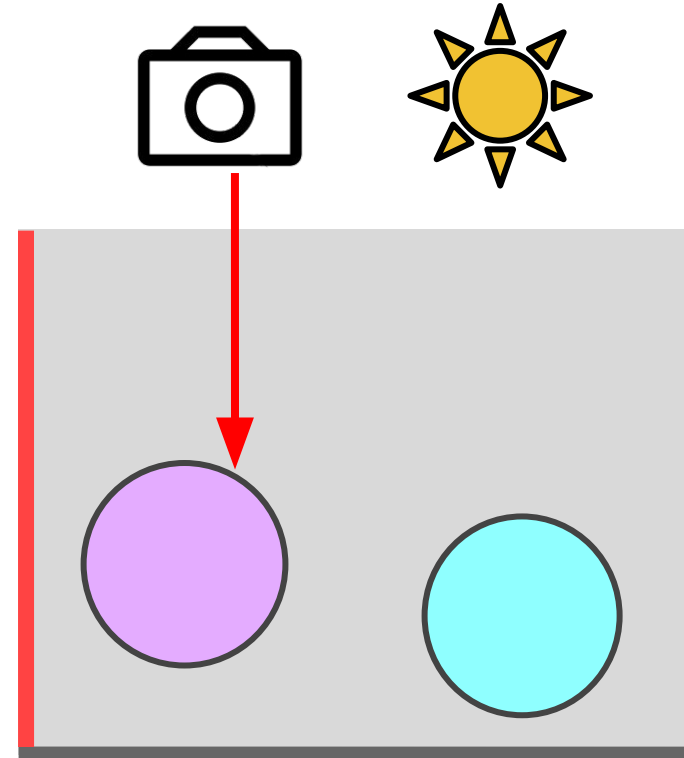
$$\approx (\text{rectangle 1} + \text{rectangle 2} + \text{rectangle 3} + \text{rectangle 4} + \text{rectangle 5}) / N$$

Approximating one integral in practice

- What happens if you approximate integrals using N samples?

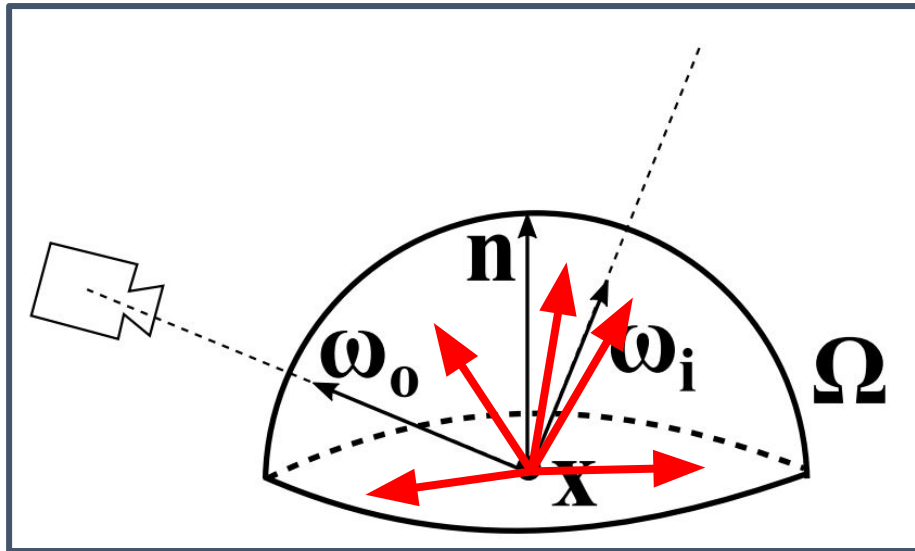


One ray from the camera

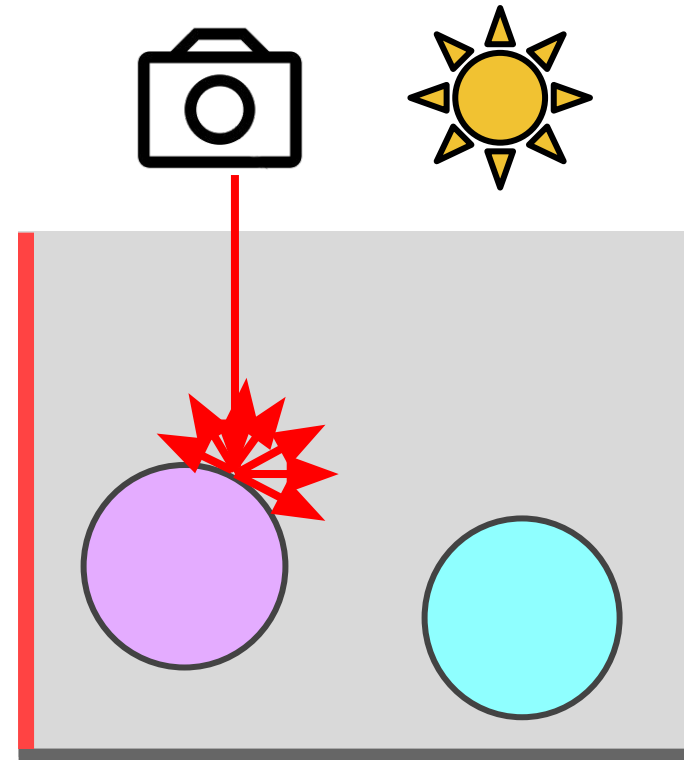


Approximating one integral in practice

- What happens if you approximate integrals using N samples?

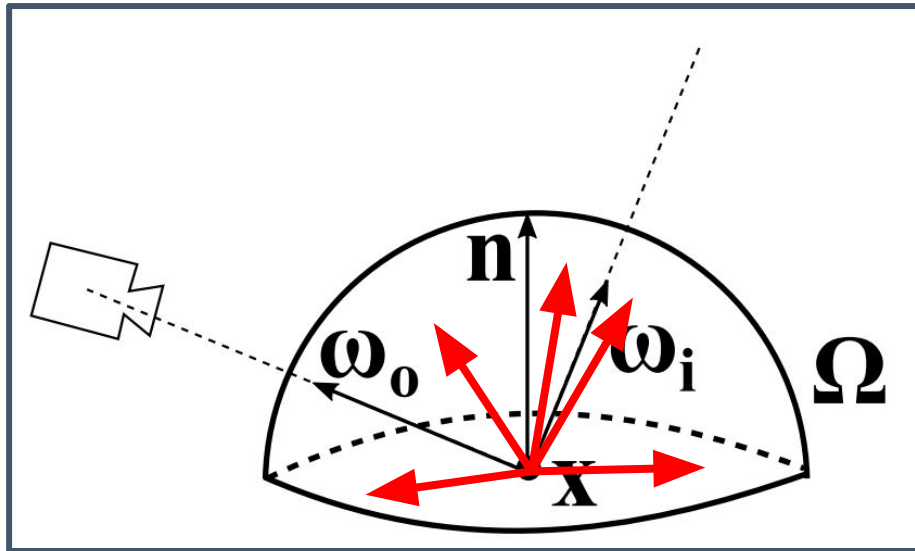


N rays for the first bounce

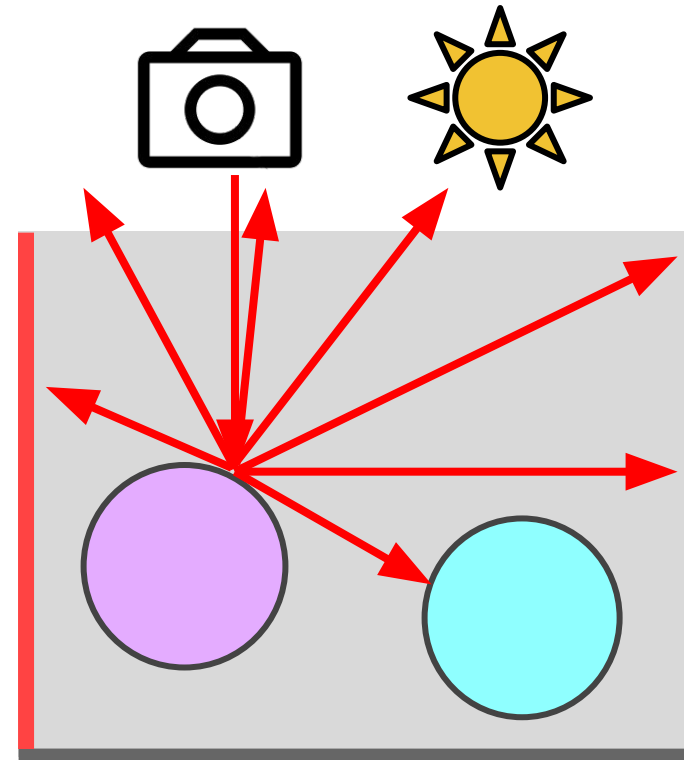


Approximating one integral in practice

- What happens if you approximate integrals using N samples?



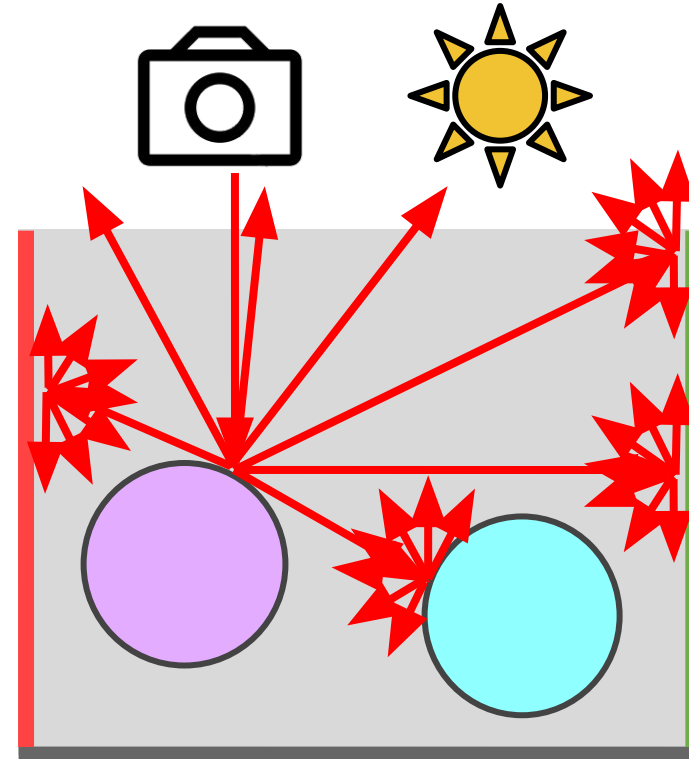
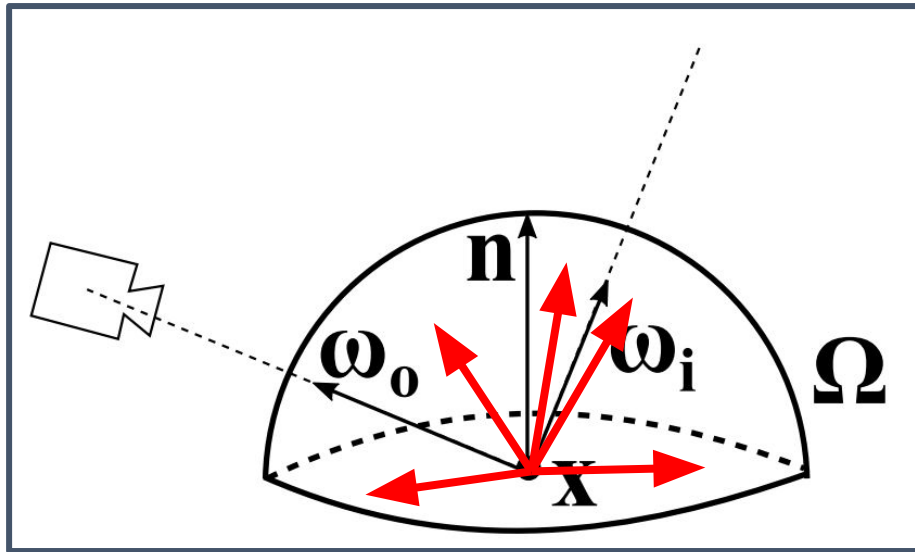
N rays for the first bounce



Approximating one integral in practice

- What happens if you approximate integrals using N samples?

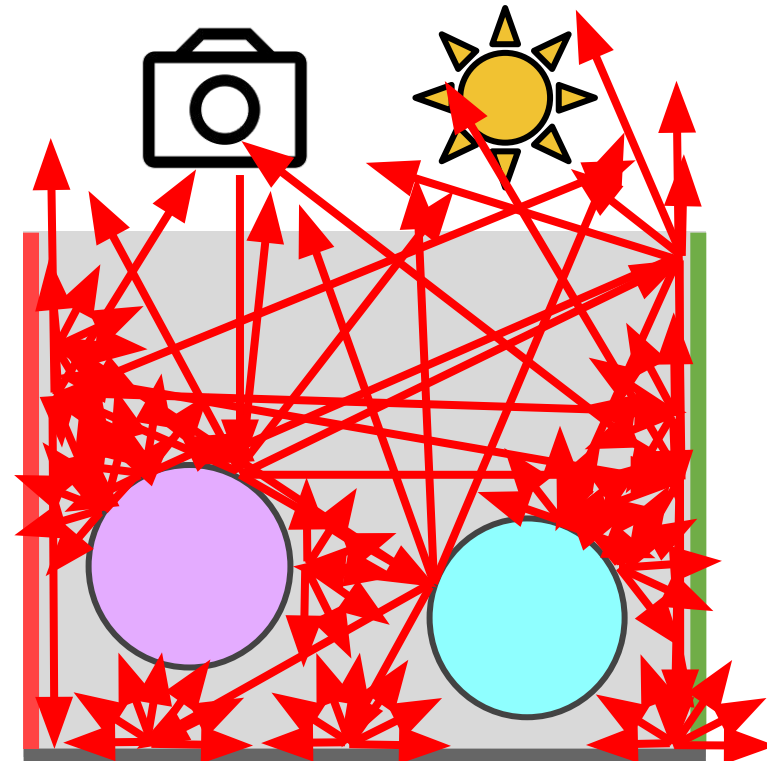
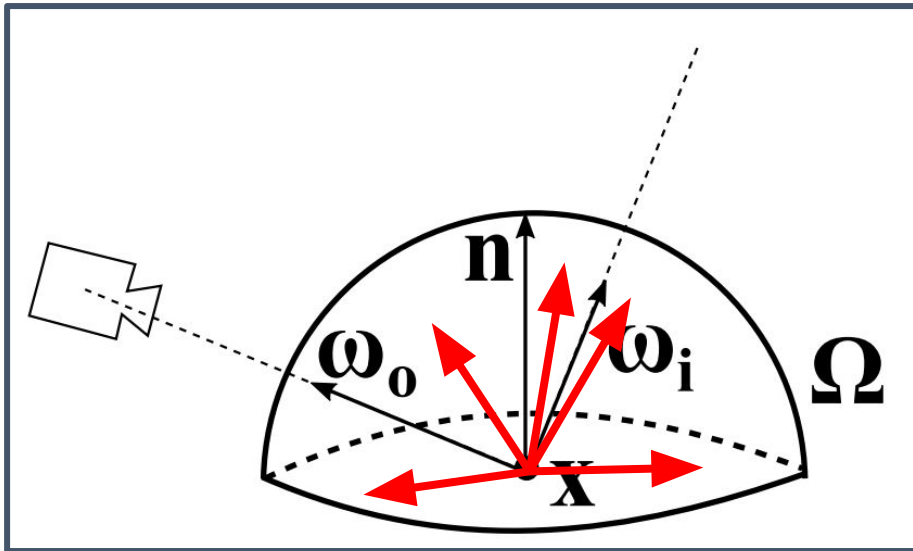
N^2 rays for the second bounce



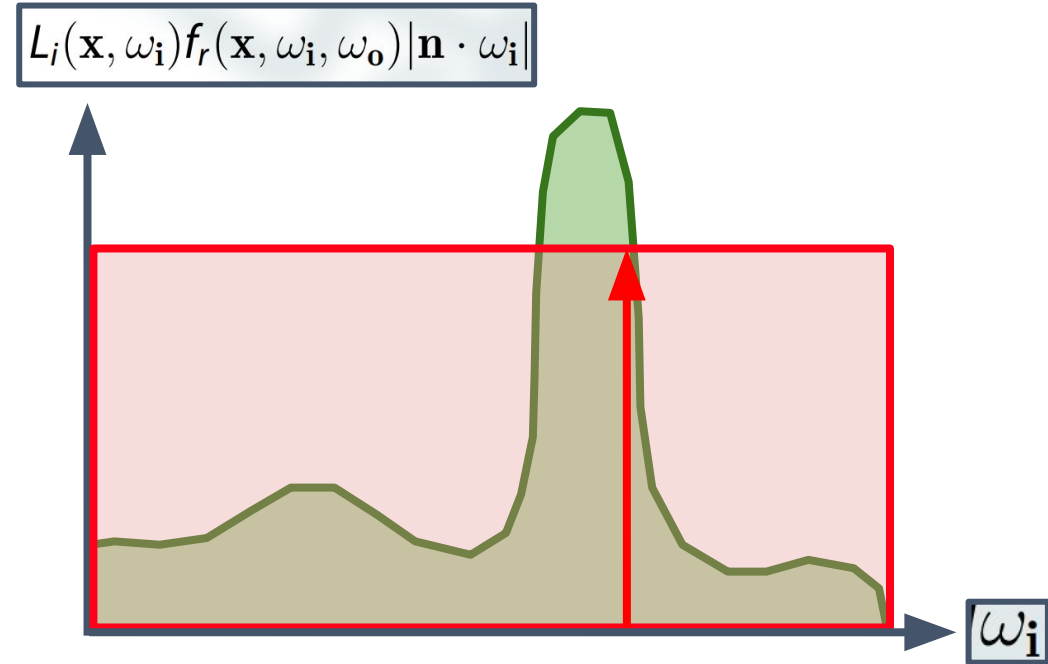
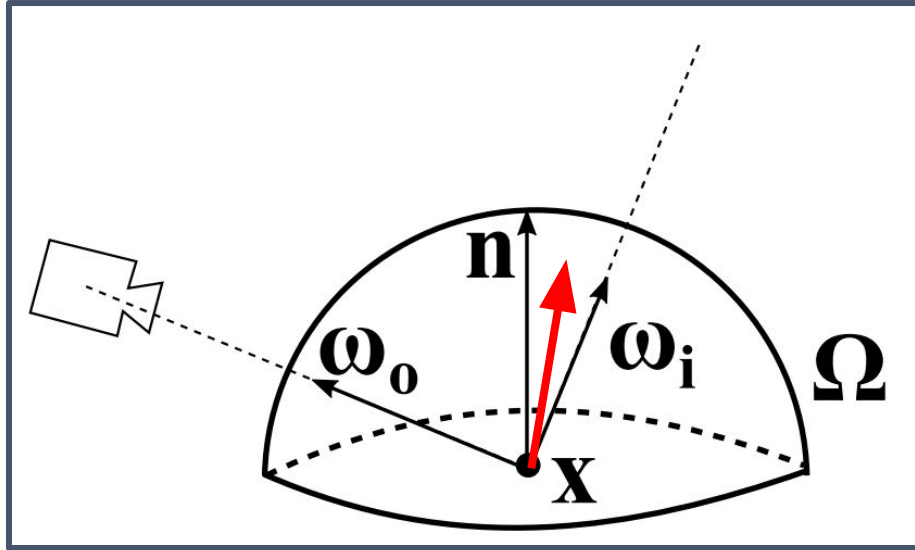
Approximating one integral in practice

- What happens if you approximate integrals using N samples?

N^N rays for the N th bounce



Approximating one integral

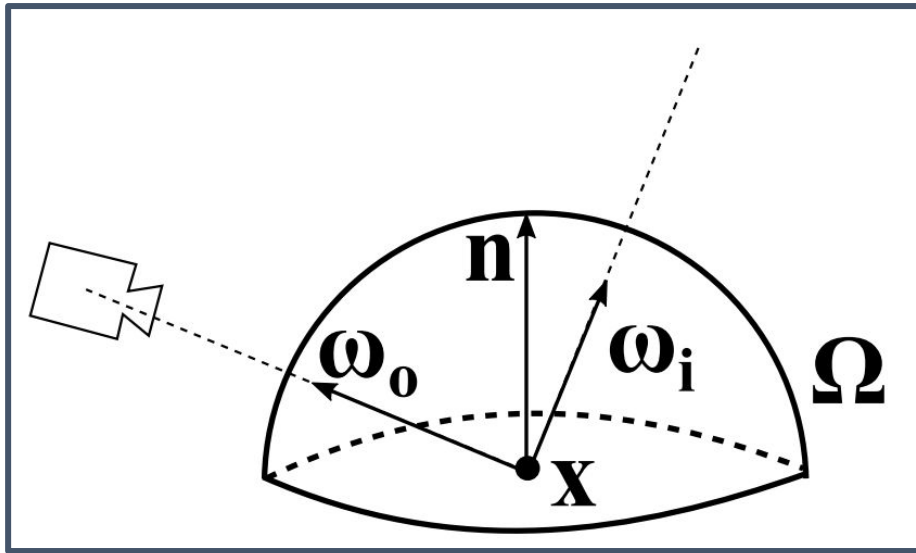


- There are infinite values for ω_i
- Idea 3: **Monte Carlo estimator**, use the mean of $N = 1$ random sample



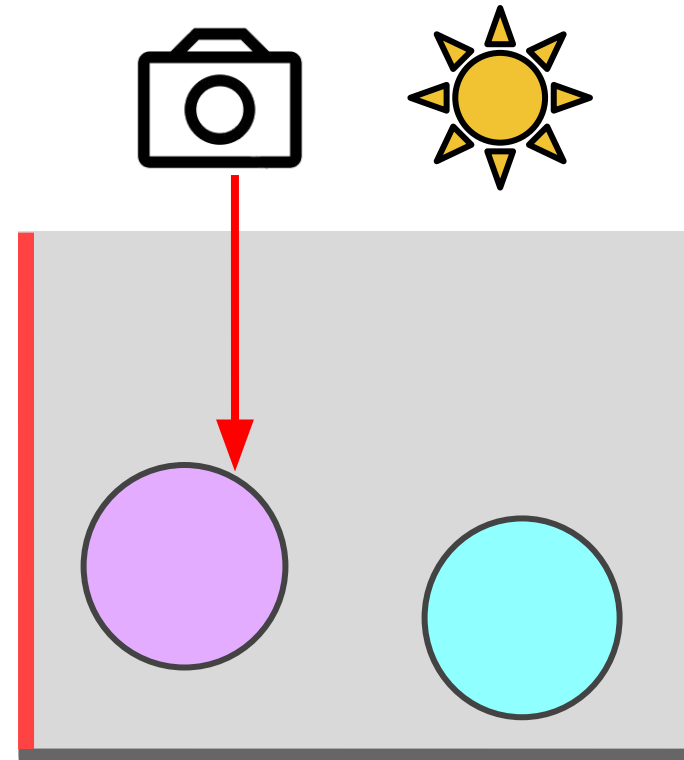
Approximating one integral in practice

- Monte Carlo estimation for the path integral



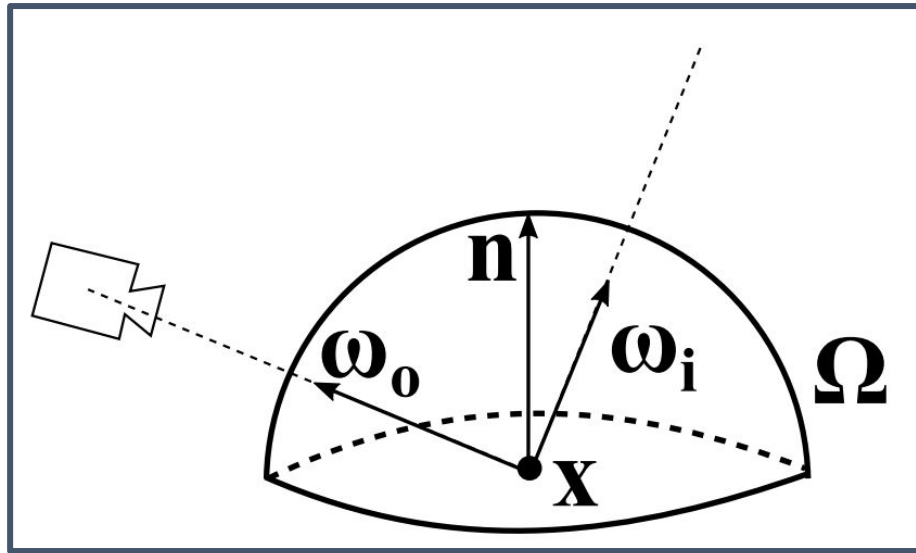
One random ω_i on each bounce

One ray from the camera



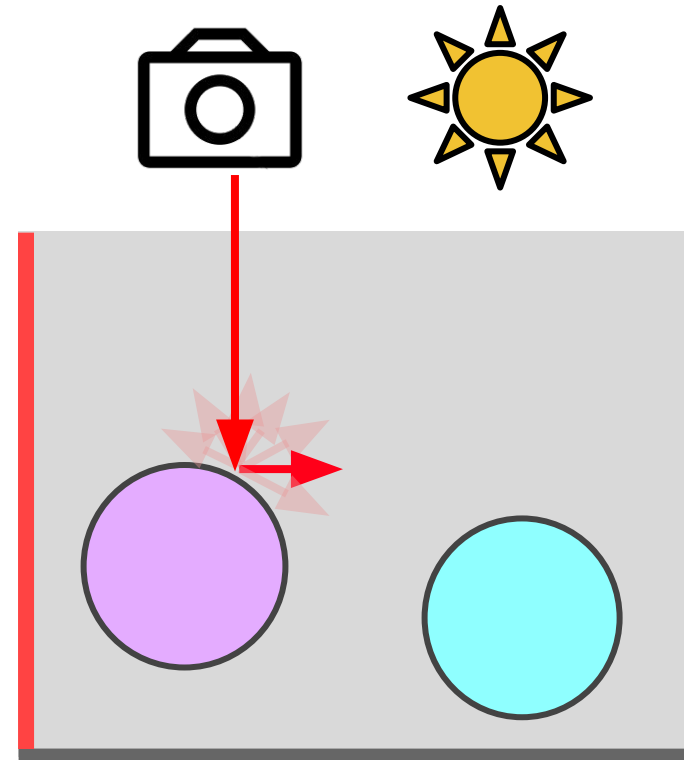
Approximating one integral in practice

- Monte Carlo estimation for the path integral



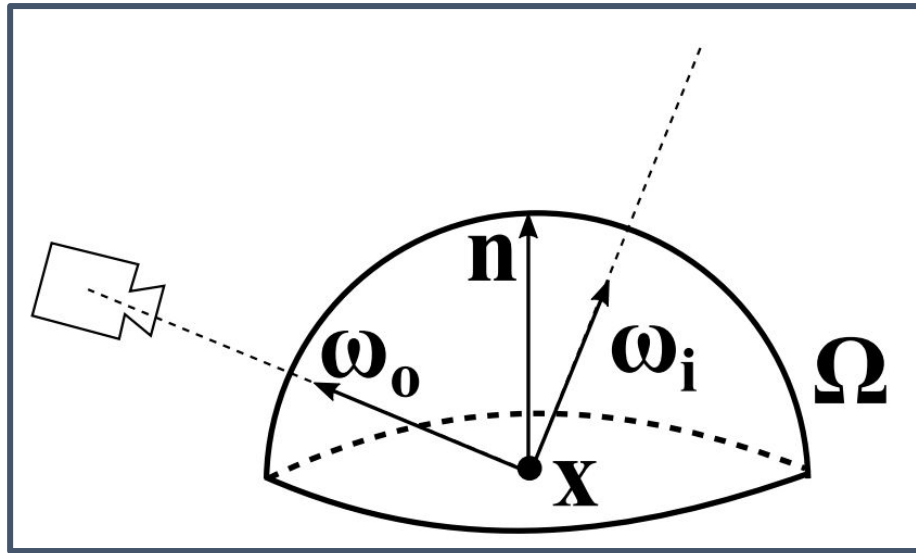
One random ω_i on each bounce

One random ray for the first bounce



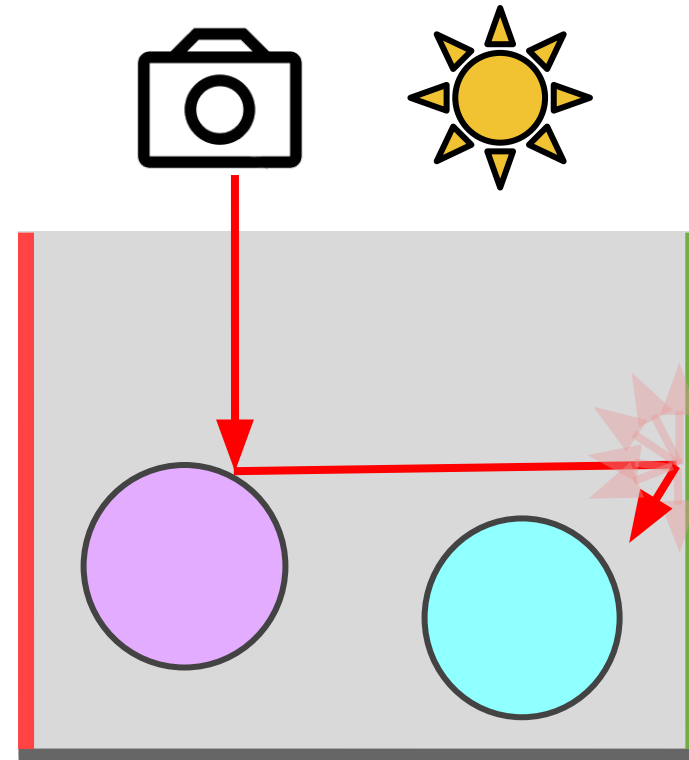
Approximating one integral in practice

- Monte Carlo estimation for the path integral



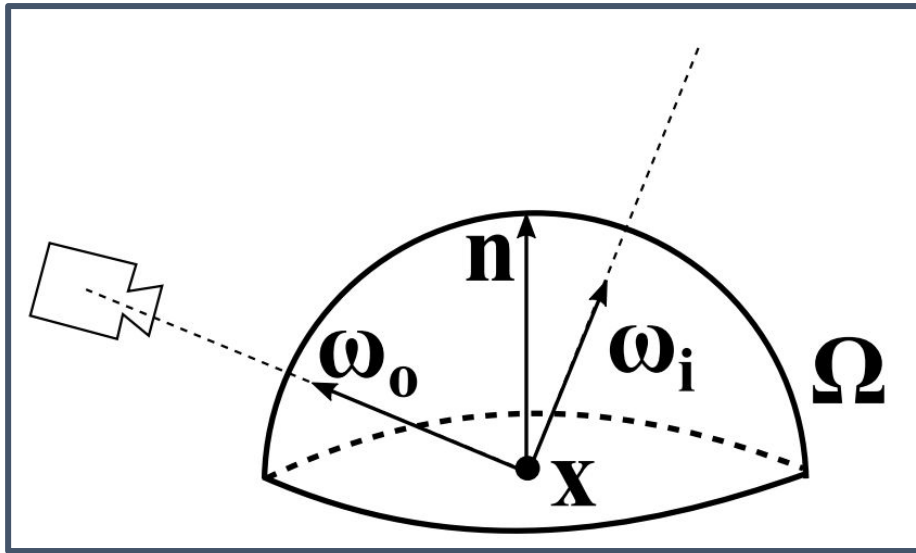
One random ω_i on each bounce

One random ray for the second bounce



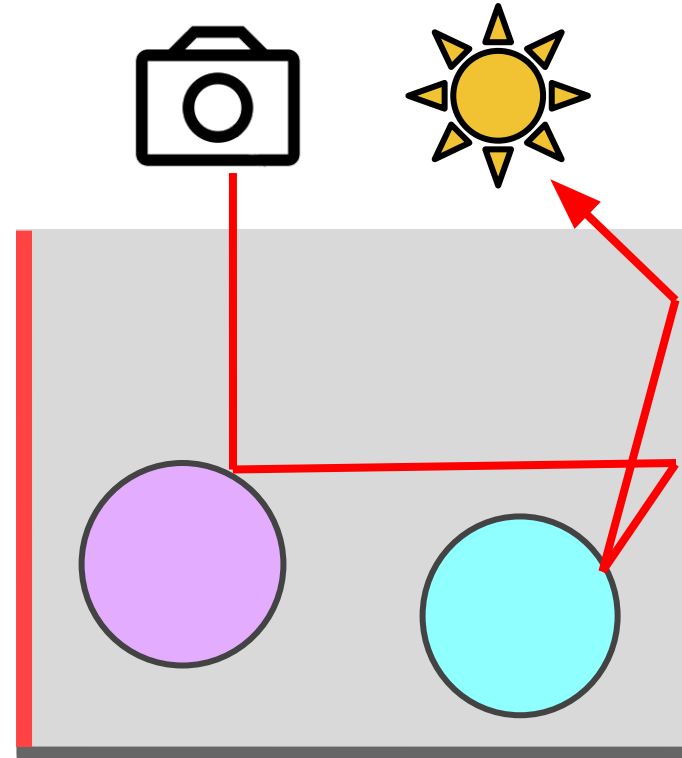
Approximating one integral in practice

- Monte Carlo estimation for the path integral



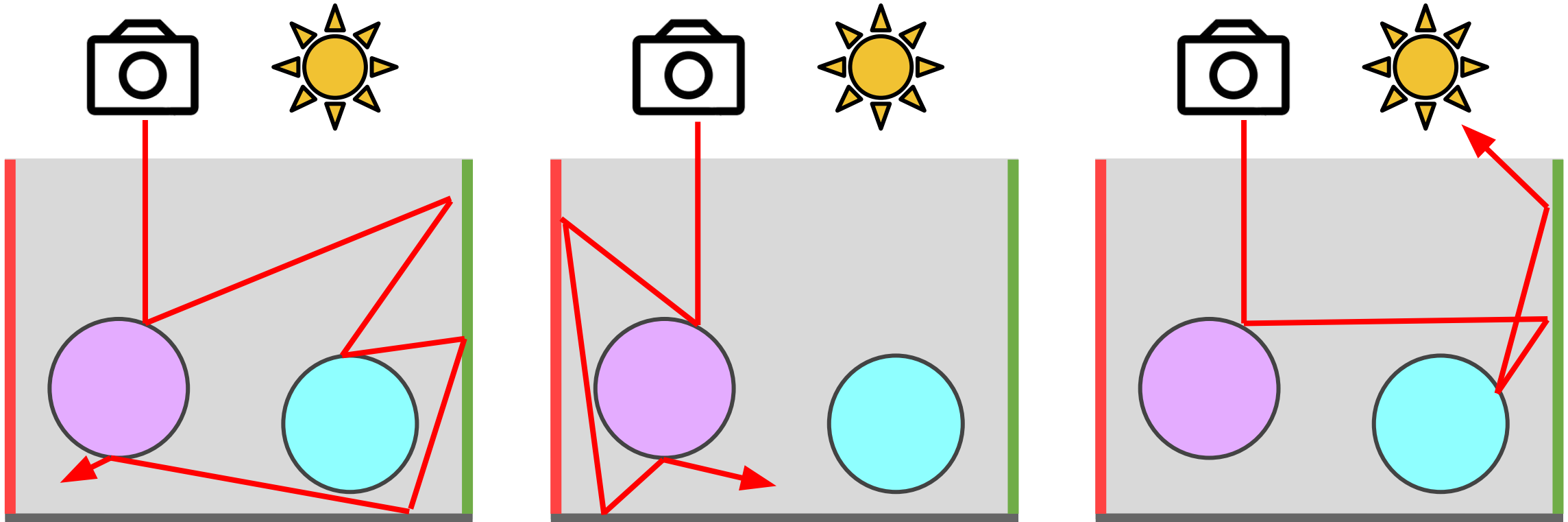
One random ω_i on each bounce

One random path



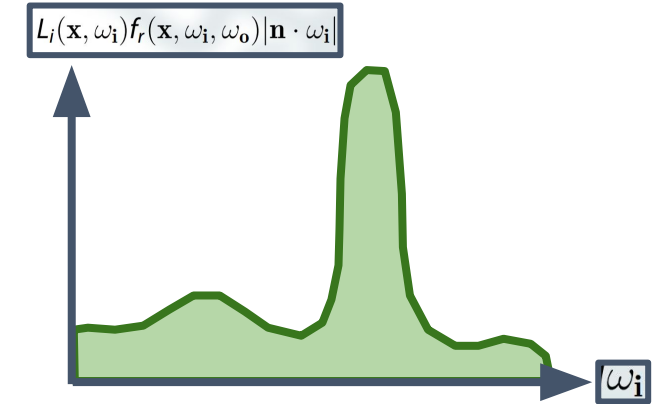
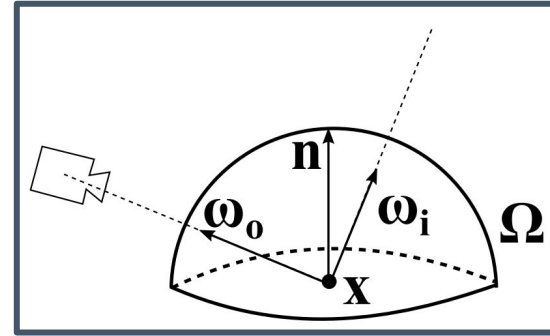
Approximating one integral in practice

- Monte Carlo estimation for the path integral
 - Sum of multiple random paths
 - More paths $\rightarrow$ better approximation of the integral (better result)



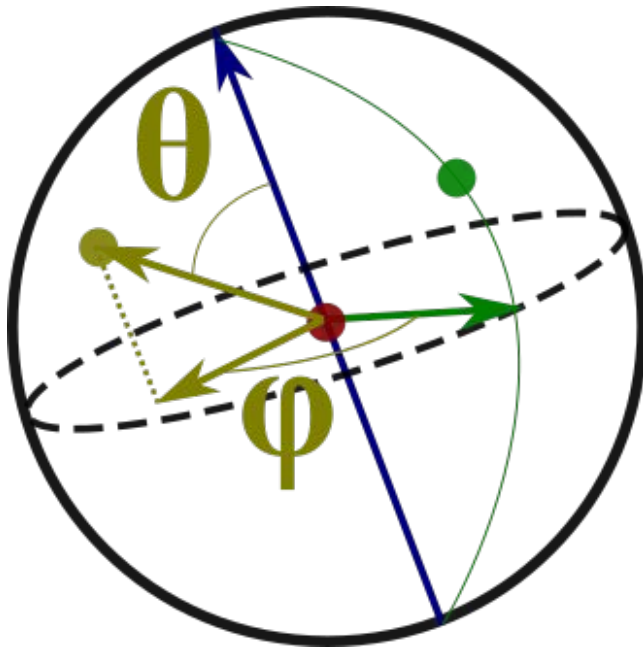
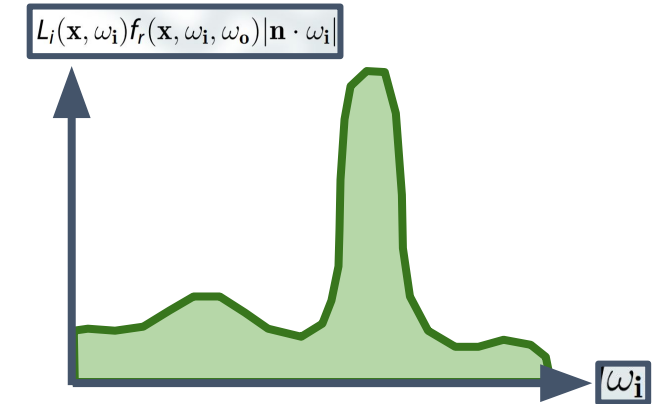
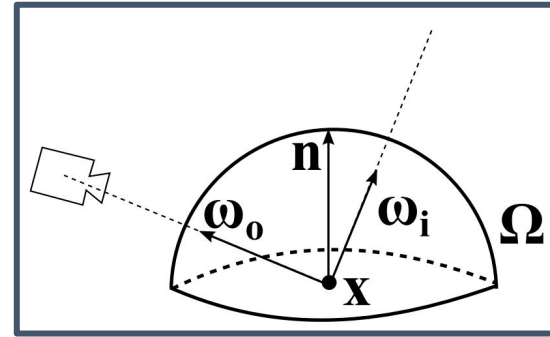
Generating random paths

- How to generate a value for ω_i ?



Generating random paths

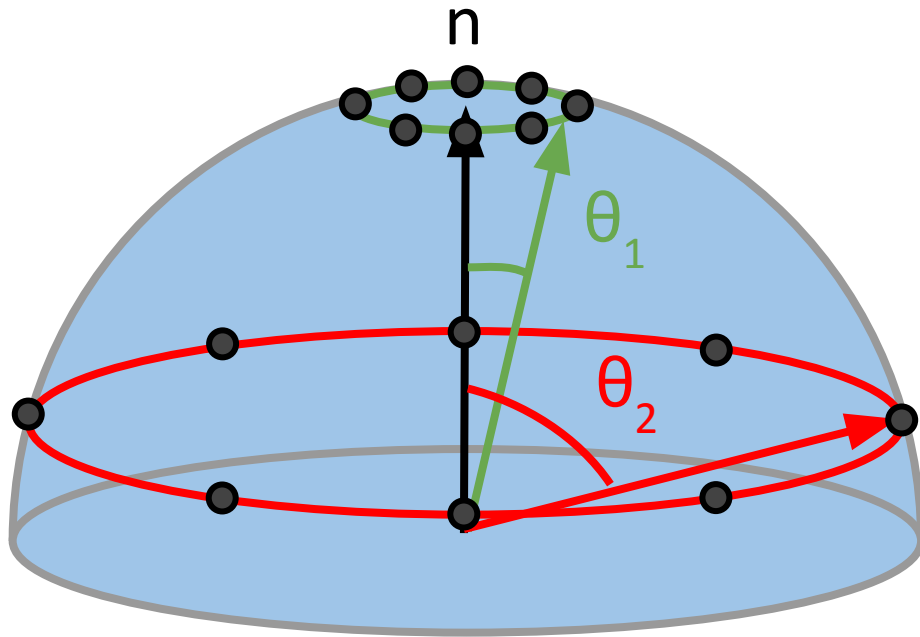
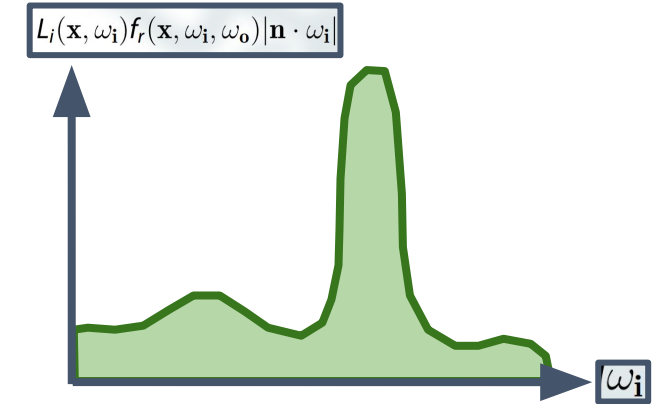
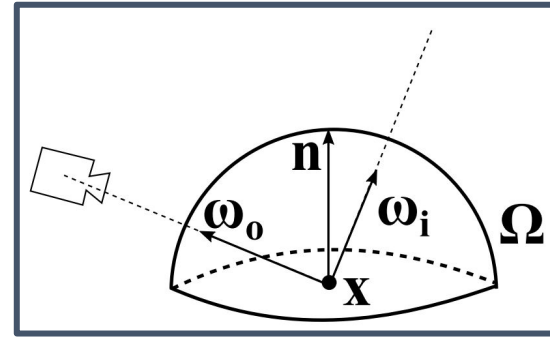
- How to generate a value for ω_i ?
- Remember your old job:



- Spherical coordinates:
 $\theta \in [0, \frac{\pi}{2})$ (**hemisphere**)
 $\varphi \in [0, 2\pi)$
- Convert spherical to cartesian

Generating random paths

- How to generate a value for ω_i ?
- Remember your old job:

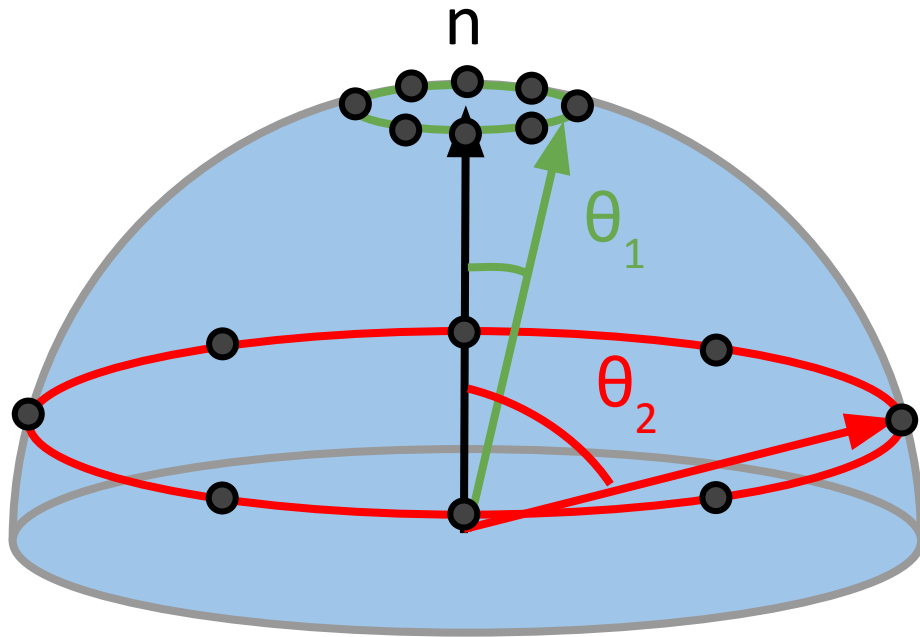
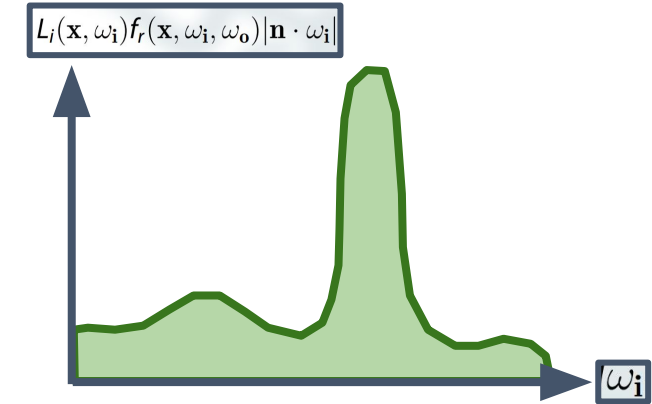
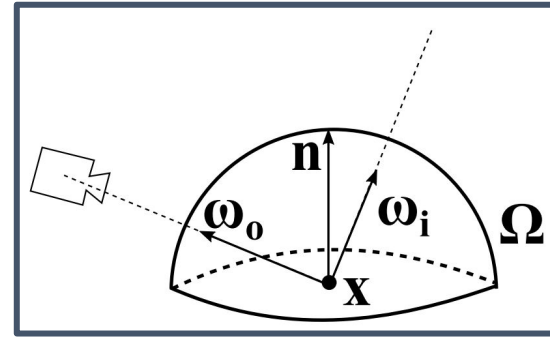


Uniform θ , $\varphi \neq$ Uniform solid angle ω_i

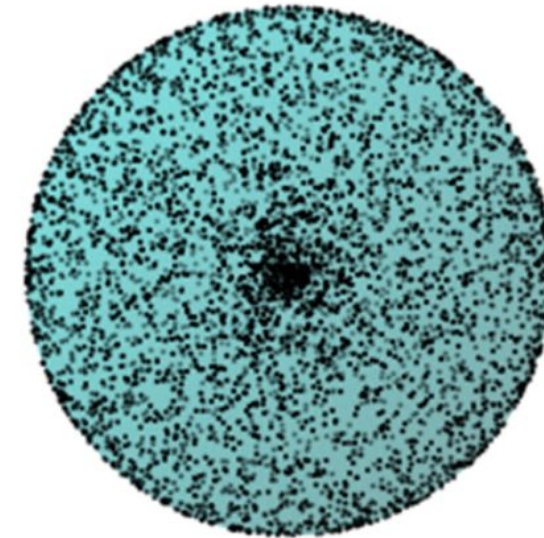
- Spherical coordinates:
 $\theta \in [0, \frac{\pi}{2})$ (**hemisphere**)
 $\varphi \in [0, 2\pi)$
- Convert spherical to cartesian

Generating random paths

- How to generate a value for ω_i ?
- Remember your old job:



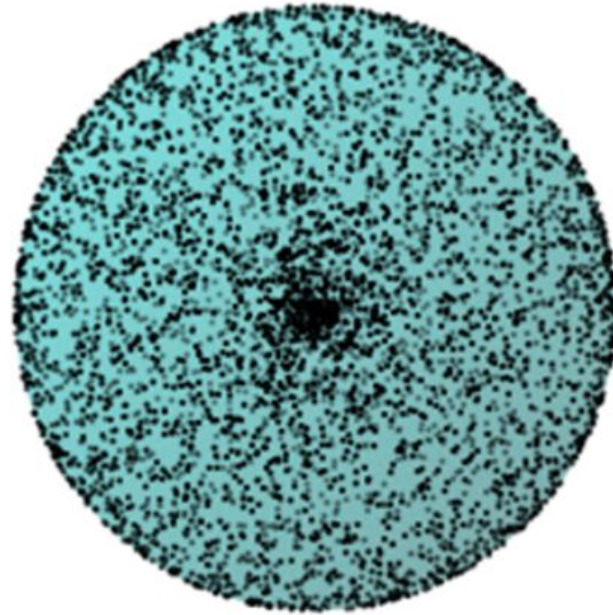
Uniform θ , $\varphi \neq$ Uniform solid angle ω_i



Top view (more samples near $\mathbf{n}$)

Uniform angle vs uniform solid angle

Uniform angle sampling



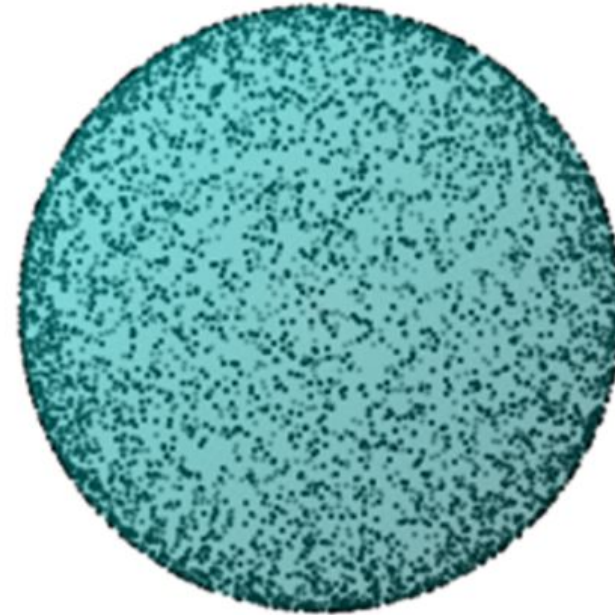
$$\xi_\theta \in [0, 1)$$

$$c^{-1}(\xi_{\theta_i}) = \frac{\pi}{2}\xi_{\theta_i}$$

$$\xi_\phi \in [0, 1)$$

$$c^{-1}(\xi_{\phi_i}) = 2\pi\xi_{\phi_i}$$

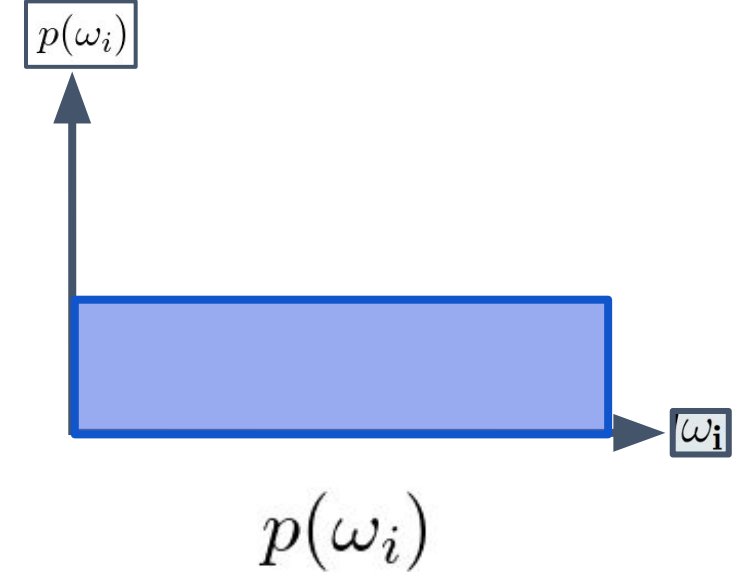
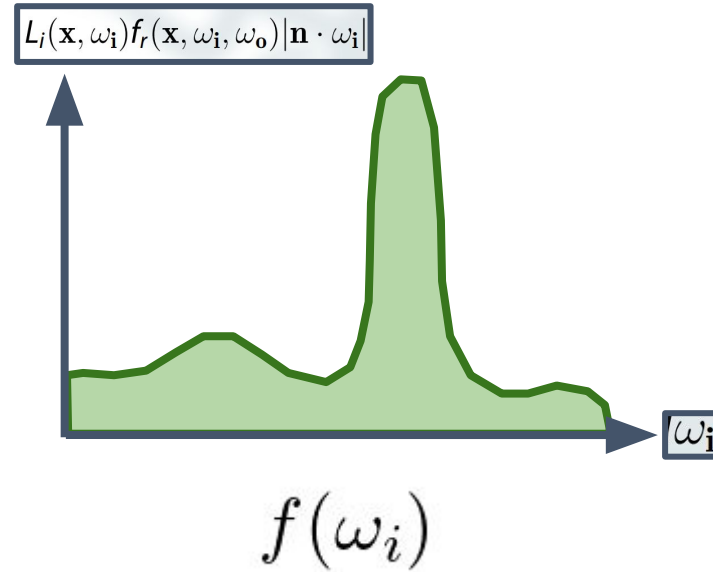
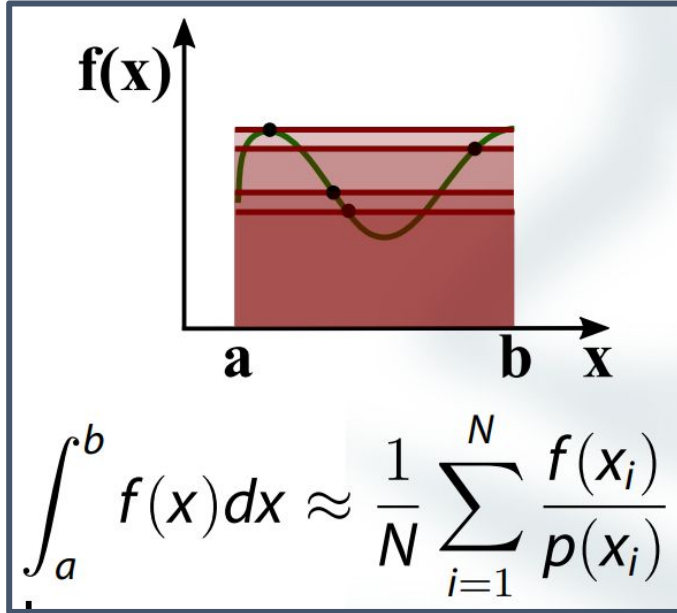
Uniform solid angle sampling



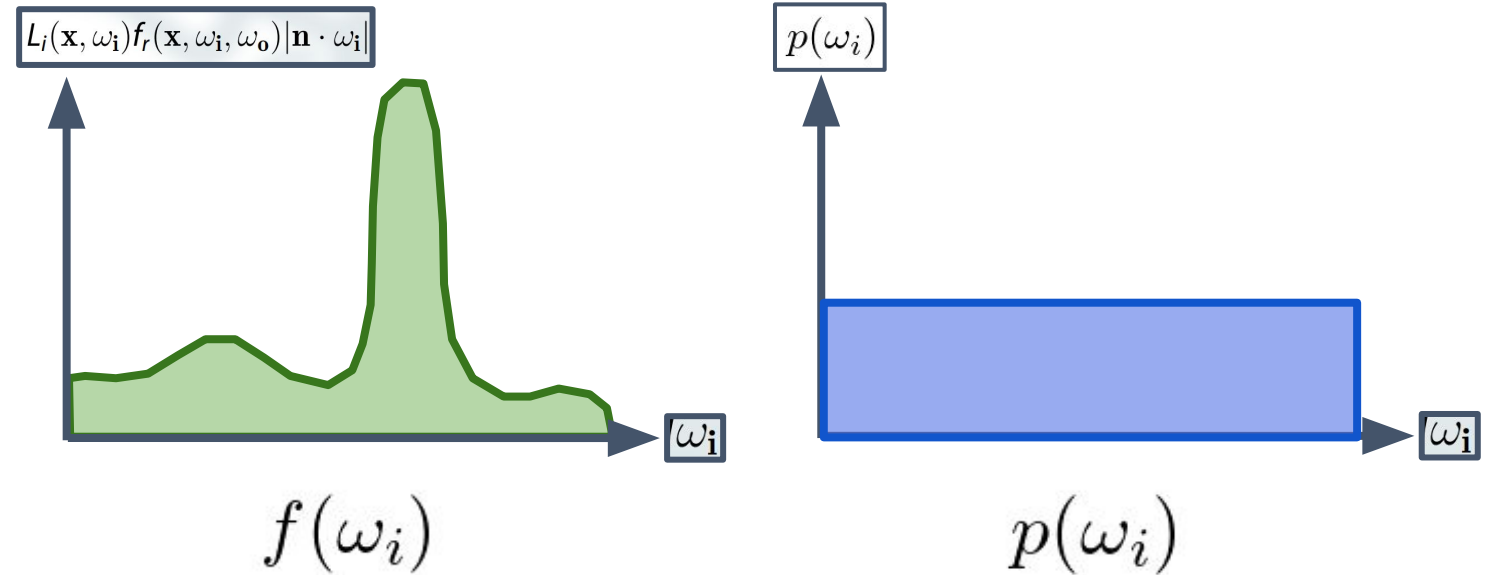
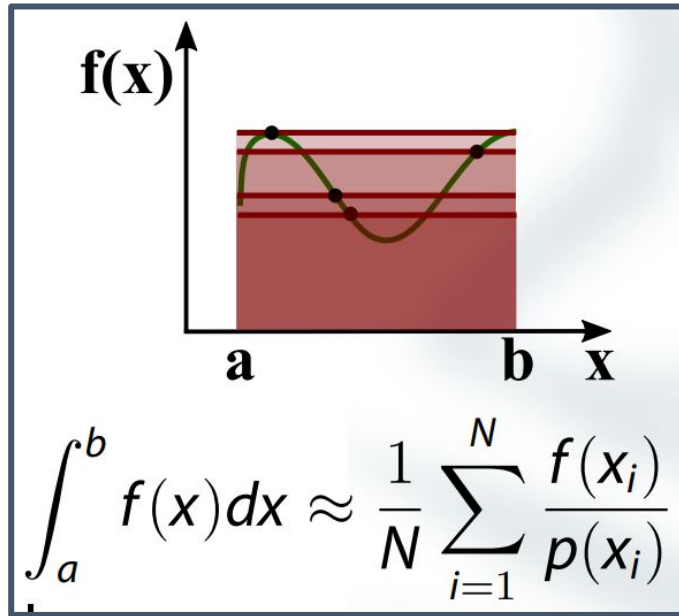
$$c^{-1}(\xi_{\theta_i}) = \arccos \xi_{\theta_i}$$

$$c^{-1}(\xi_{\phi_i}) = 2\pi\xi_{\phi_i}$$

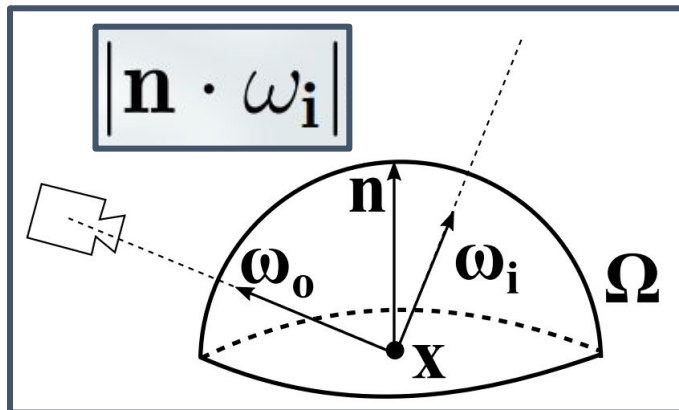
Importance sampling



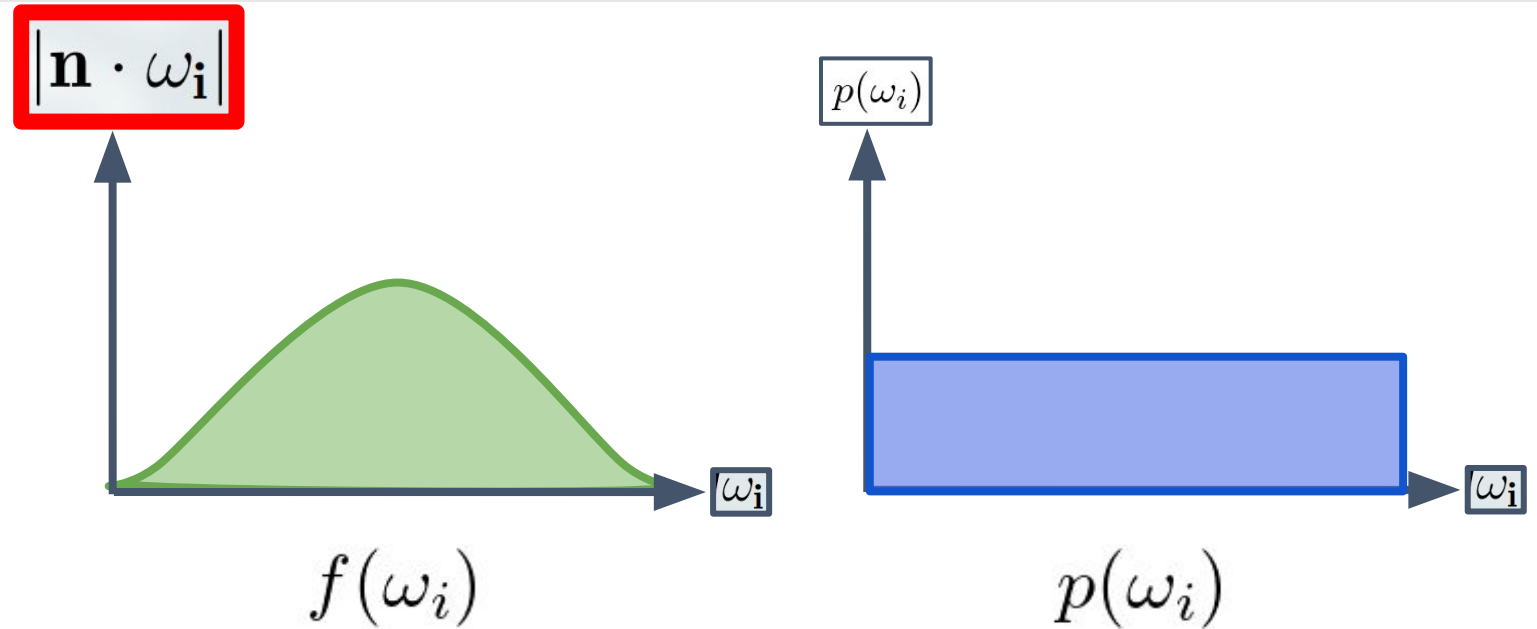
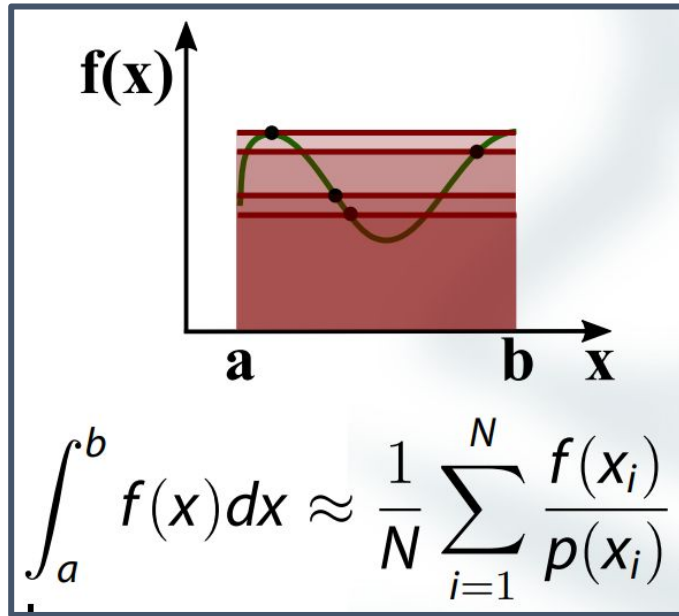
Importance sampling



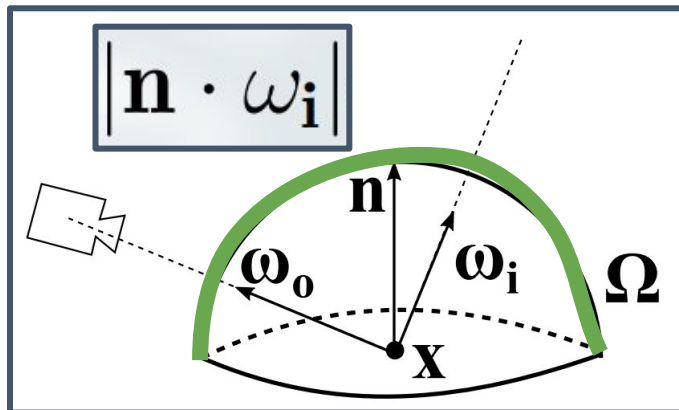
- Monte Carlo works better when f and p have similar distributions



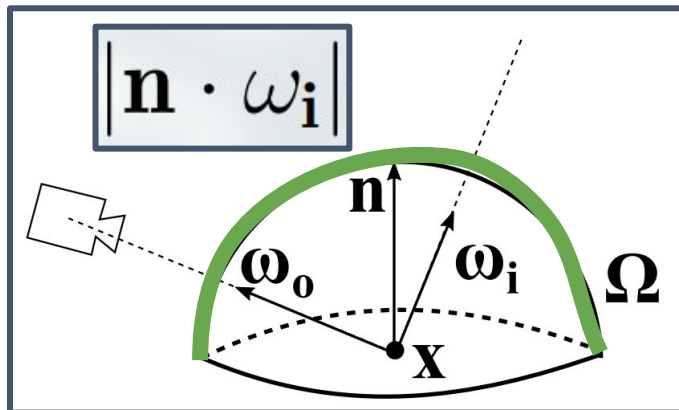
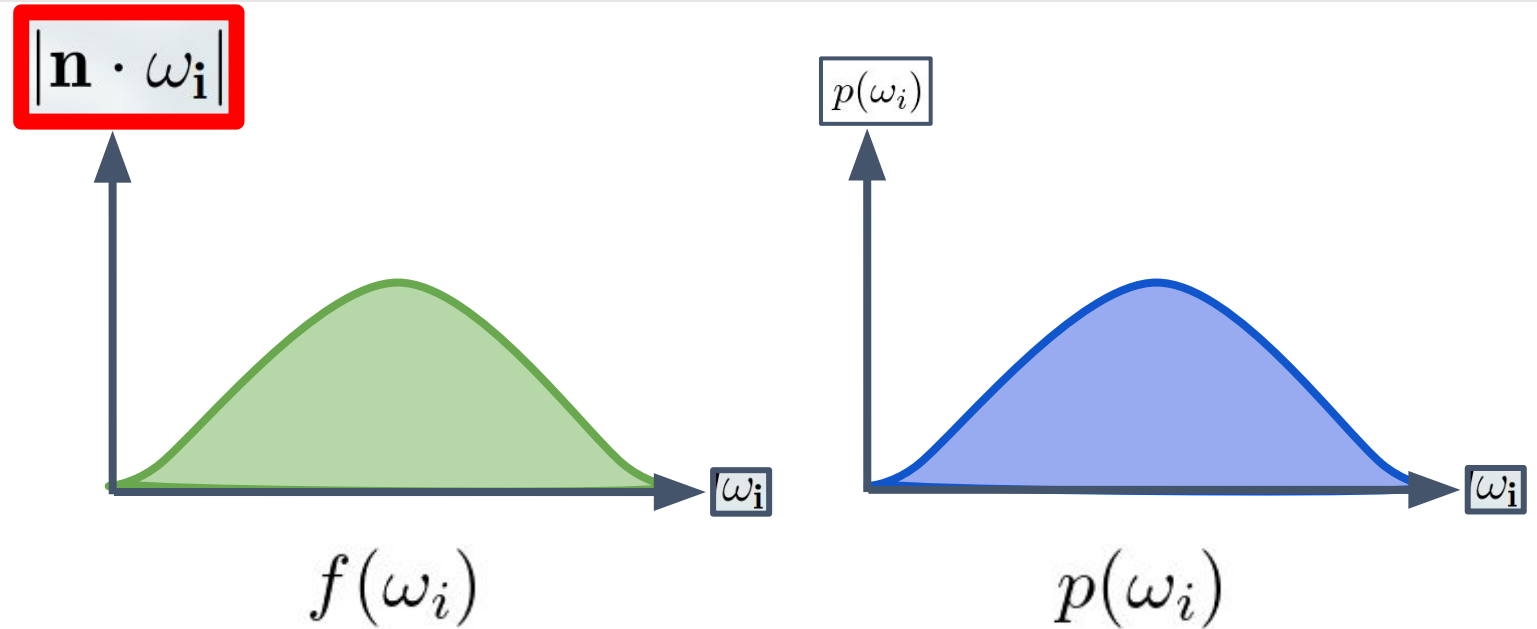
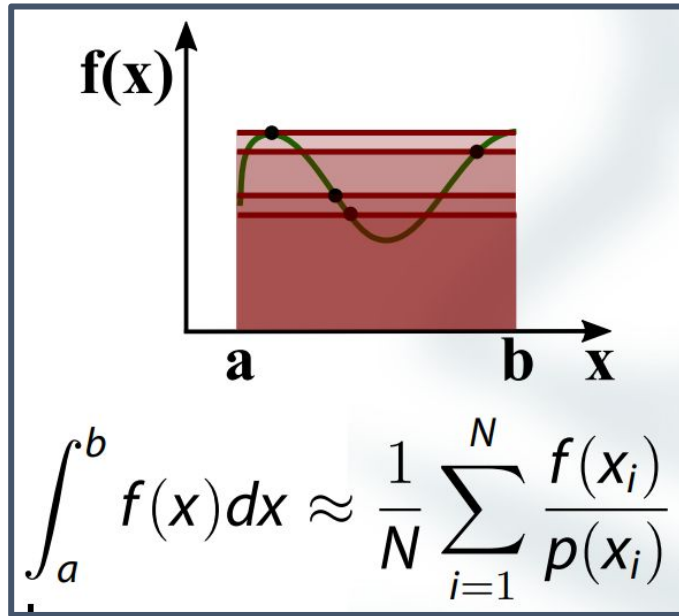
Importance sampling: diffuse materials



- Monte Carlo works better when f and p have similar distributions



Importance sampling: diffuse materials



- Monte Carlo works better when f and p have similar distributions
- Make p follow $|\mathbf{n} \cdot \omega_i|$
- Cosine-weighted

$$c^{-1}(\xi_{\theta_i}) = \arccos \sqrt{1 - \xi_{\theta_i}}$$

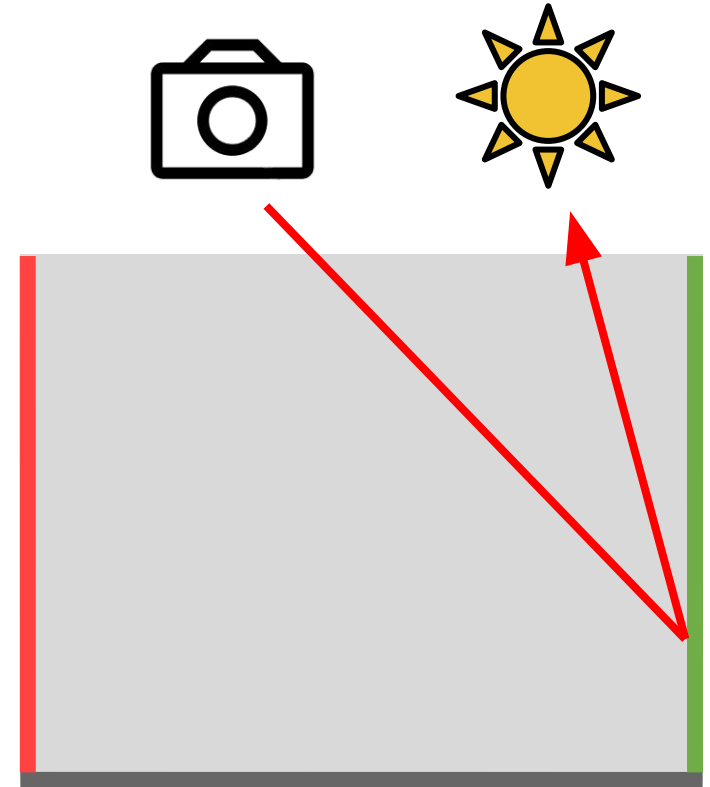
$$c^{-1}(\xi_{\phi_i}) = 2\pi\xi_{\phi_i}$$

Programming recommendations

- Enough maths, let's code

Programming recommendations

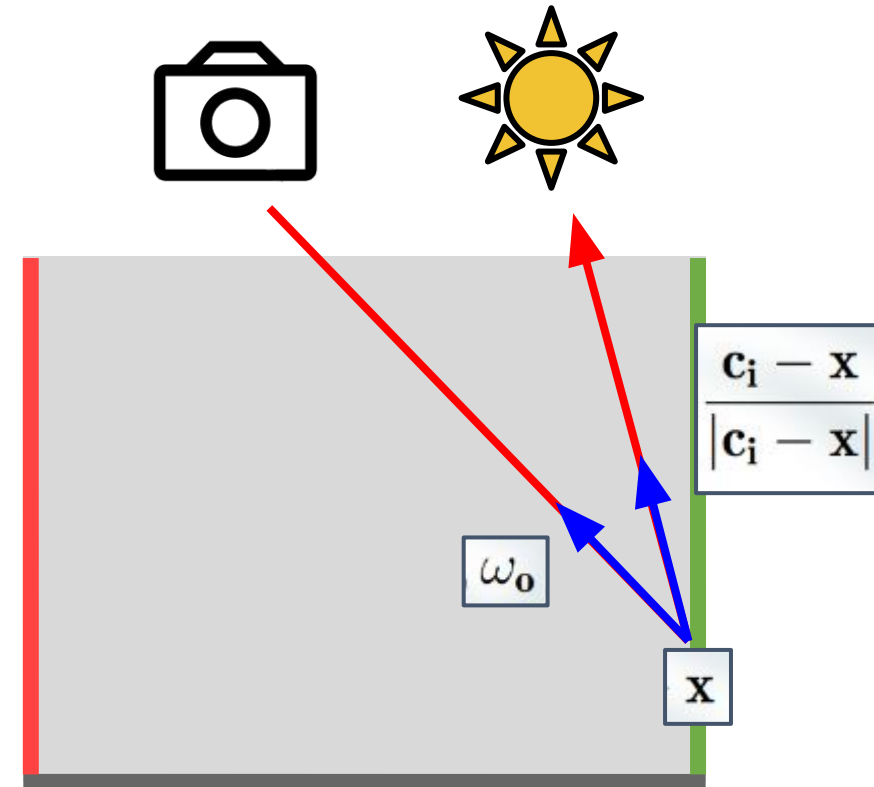
- Enough maths, let's code
- **Recommendation:** you can separate your code in functions



Programming recommendations

- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF** evaluation function

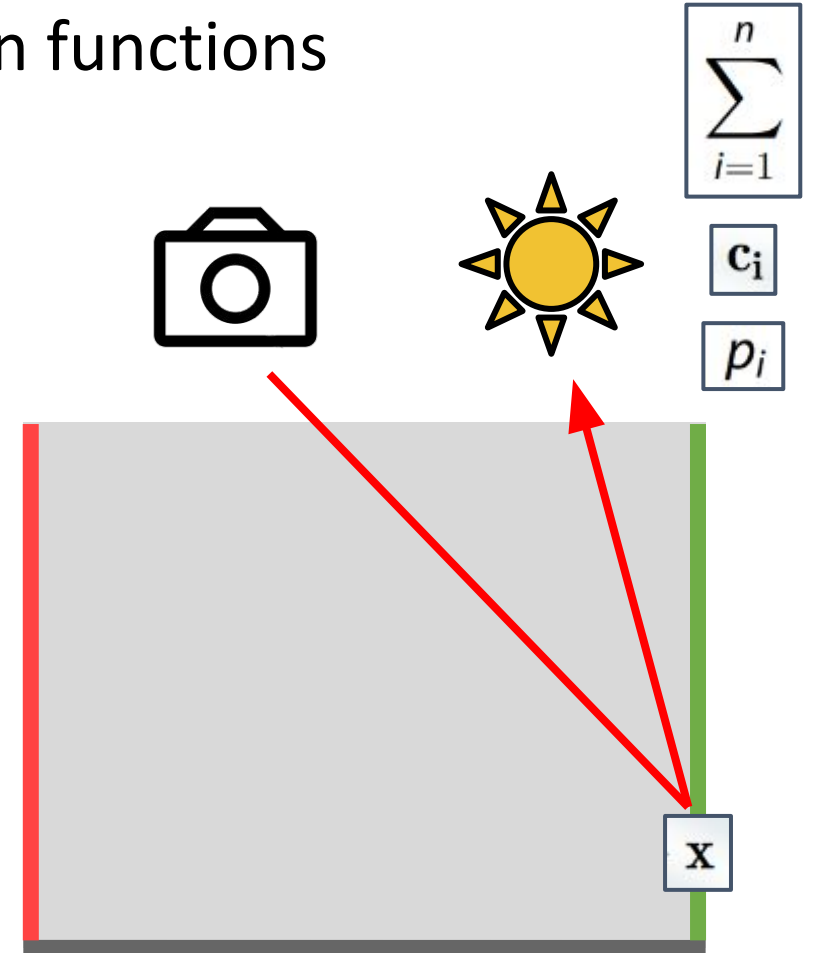
$$f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right)$$



Programming recommendations

- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF evaluation** function
 - Next-event estimation (direct lights at $\mathbf{x}$)

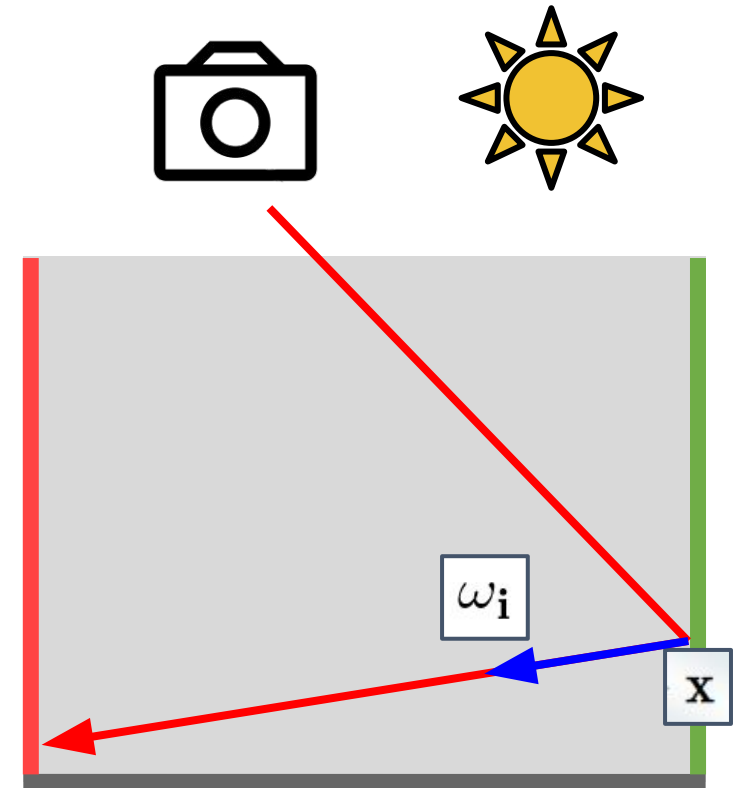
$$\sum_{i=1}^n \frac{p_i}{|\mathbf{c}_i - \mathbf{x}|^2} f_r \left(\mathbf{x}, \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|}, \omega_o \right) \left| \mathbf{n} \cdot \frac{\mathbf{c}_i - \mathbf{x}}{|\mathbf{c}_i - \mathbf{x}|} \right|$$



Programming recommendations

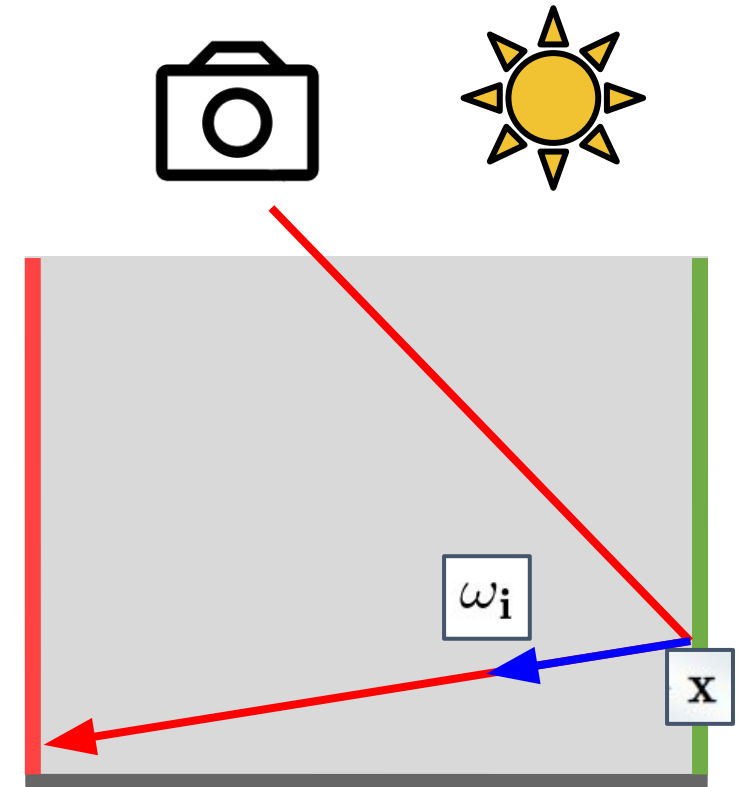
- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF evaluation** function
 - Next-event estimation (direct lights at $\mathbf{x}$)
 - Today's session
 - Diffuse **BRDF sample** function

$$\begin{array}{l} \xi_\theta \in [0, 1) \\ \xi_\varphi \in [0, 1) \end{array} \begin{array}{c} \longrightarrow \\ \longrightarrow \end{array} \begin{array}{l} \theta \in [0, \frac{\pi}{2}) \\ \varphi \in [0, 2\pi) \end{array} \begin{array}{c} \longrightarrow \\ \longrightarrow \end{array} \omega_i$$



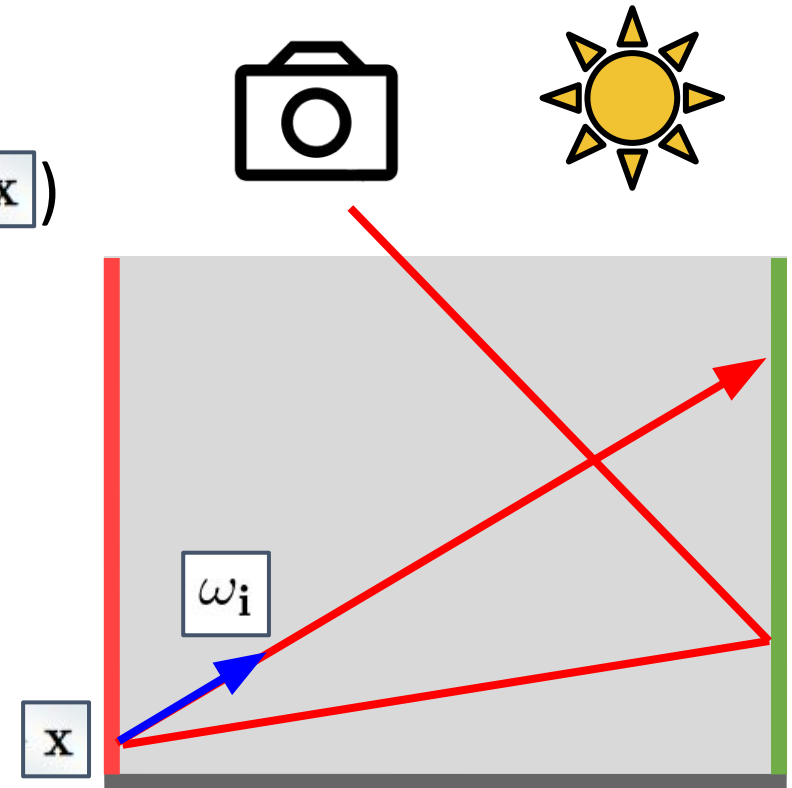
Programming recommendations

- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF evaluation** function
 - Next-event estimation (direct lights at $\mathbf{x}$)
 - Today's session
 - Diffuse **BRDF sample** function
 - Return ω_i and $f_r(\mathbf{x}, \omega_i, \omega_o)$
 - Call it on every bounce
 - Intersect ray $(\mathbf{x}, \omega_i)$ with geometry



Programming recommendations

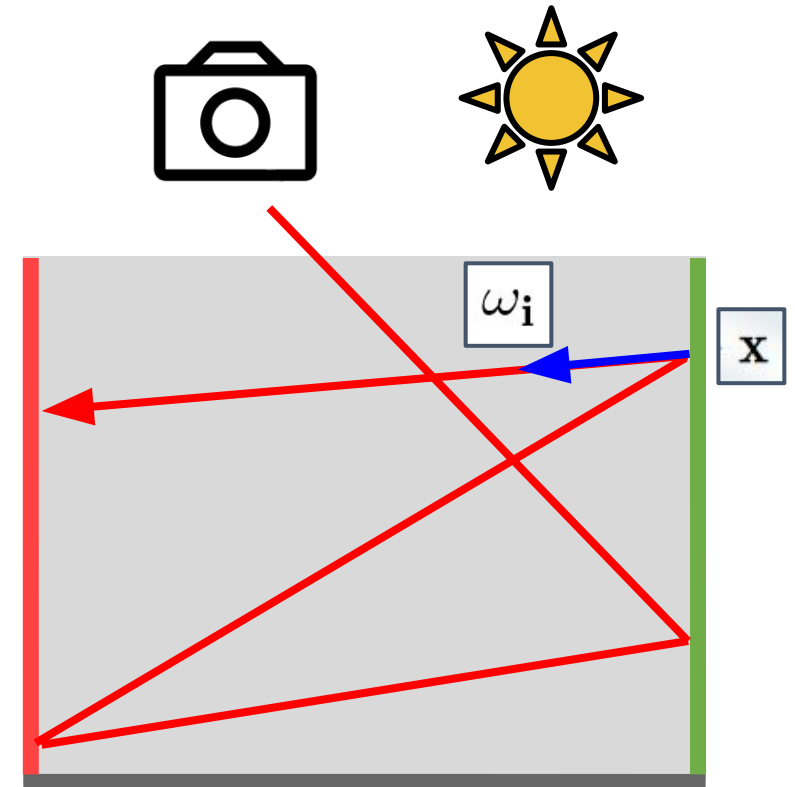
- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF evaluation** function
 - Next-event estimation (direct lights at $\mathbf{x}$)
 - Today's session
 - Diffuse **BRDF sample** function
 - Return ω_i and $f_r(\mathbf{x}, \omega_i, \omega_o)$
 - Call it on every bounce
 - Intersect ray ($\mathbf{x}, \omega_i$) with geometry



Programming recommendations

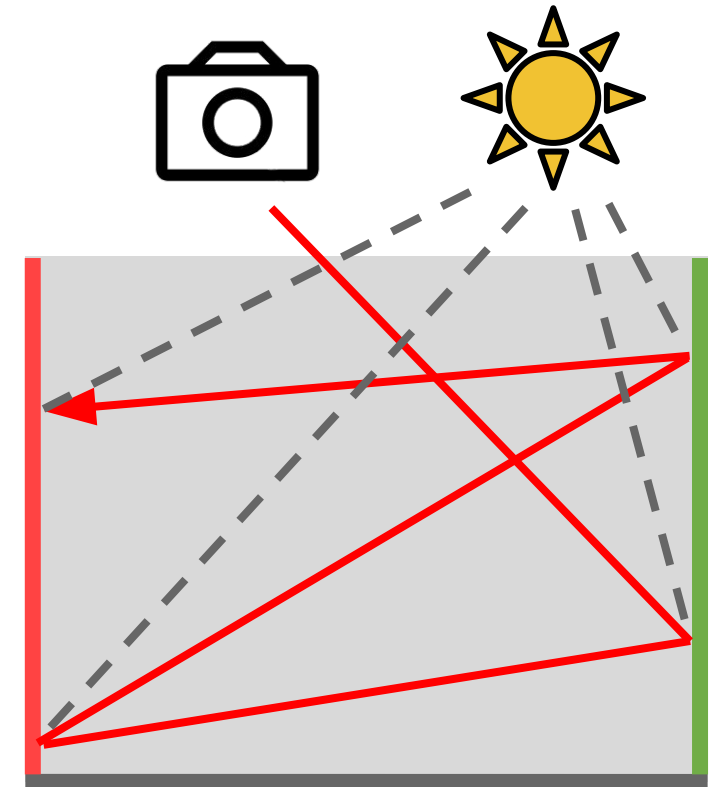
- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF evaluation** function
 - Next-event estimation (direct lights at $\mathbf{x}$)
 - Today's session
 - Diffuse **BRDF sample** function

Problem: it is **impossible** to hit a point light with a random ray ($\mathbf{x}$, ω_i)



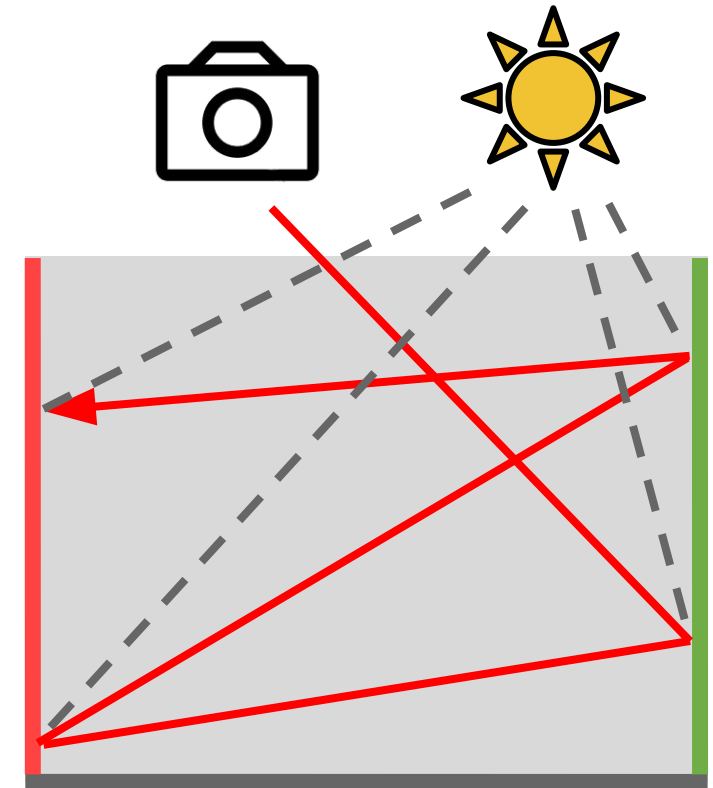
Programming recommendations

- Enough maths, let's code
- **Recommendation:** you can separate your code in functions
 - From previous session
 - Diffuse **BRDF evaluation** function
 - Next-event estimation (direct lights at $\mathbf{x}$)
 - Today's session
 - Diffuse **BRDF sample** function
 - Generate ω_i and intersect ray
 - Call next-event estimation on every bounce $\mathbf{x}$



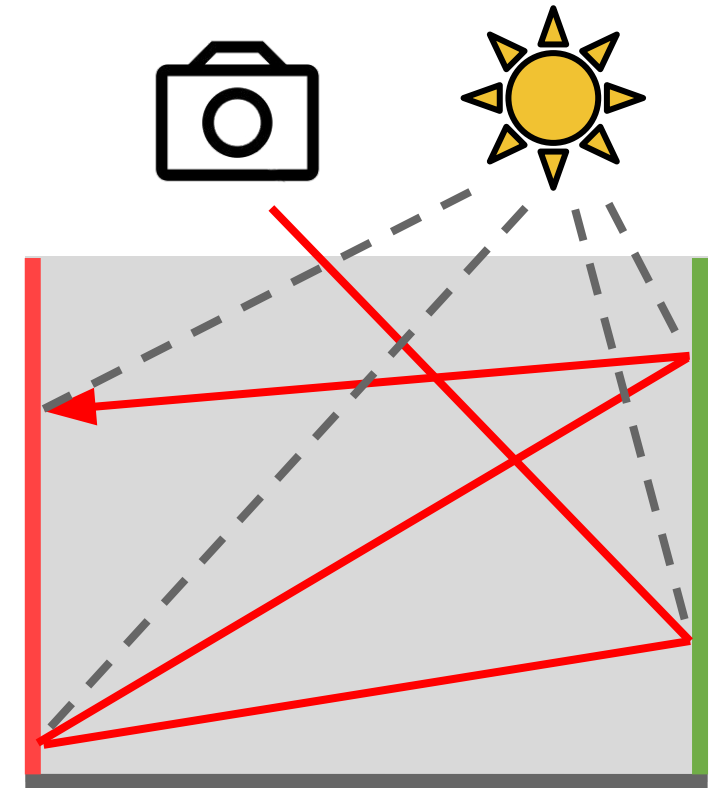
Programming recommendations

- Path tracing algorithm: Want to calculate $L_o(\mathbf{x}, \omega_o)$
 - Intersect ray with geometry at point $\mathbf{x}$ with its own BRDF
 - Search for light sources at $\mathbf{x}$
 - Sample BRDF $\mathbf{x}$ for a new ray $(\mathbf{x}, \omega_i)$
 - Repeat the same steps again



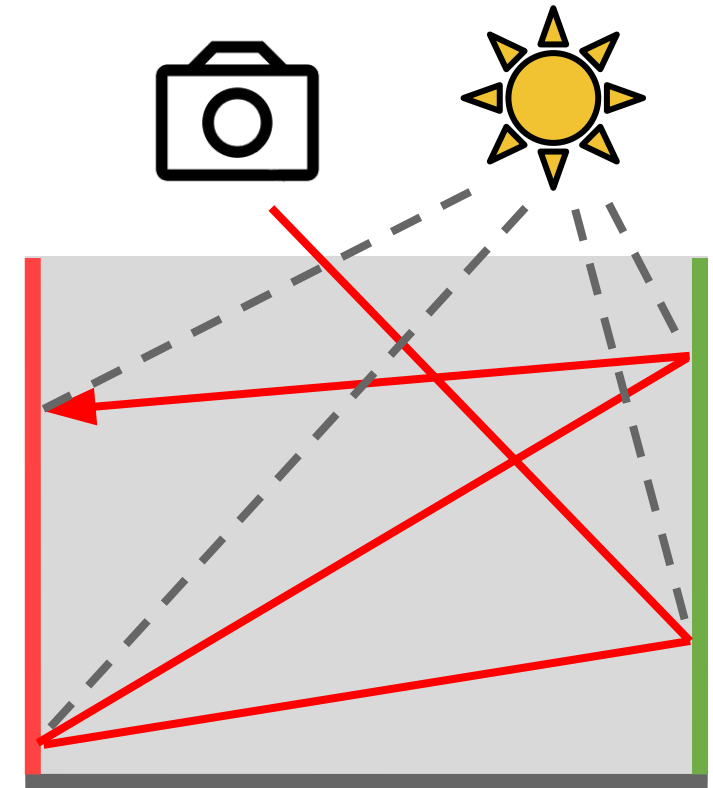
Programming recommendations

- Path tracing algorithm: Want to calculate $L_o(\mathbf{x}, \omega_o)$
 - Intersect ray with geometry at point $\mathbf{x}$ with its own BRDF
 - Search for light sources at $\mathbf{x}$
 - Sample BRDF $\mathbf{x}$ for a new ray $(\mathbf{x}, \omega_i)$
 - Repeat the same steps again
- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)



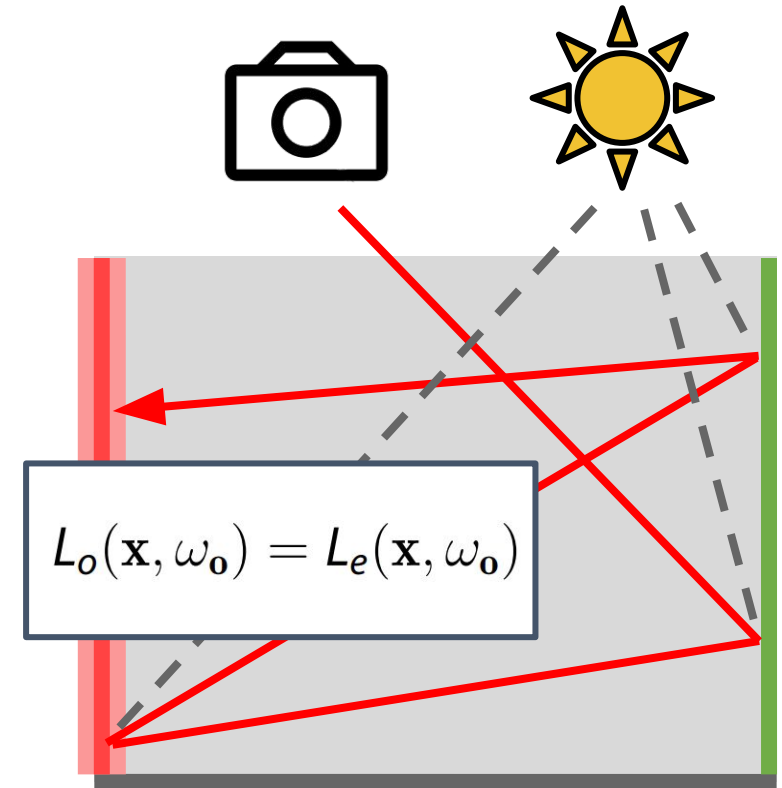
Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- When do we stop generating rays?
(aka. path termination conditions)



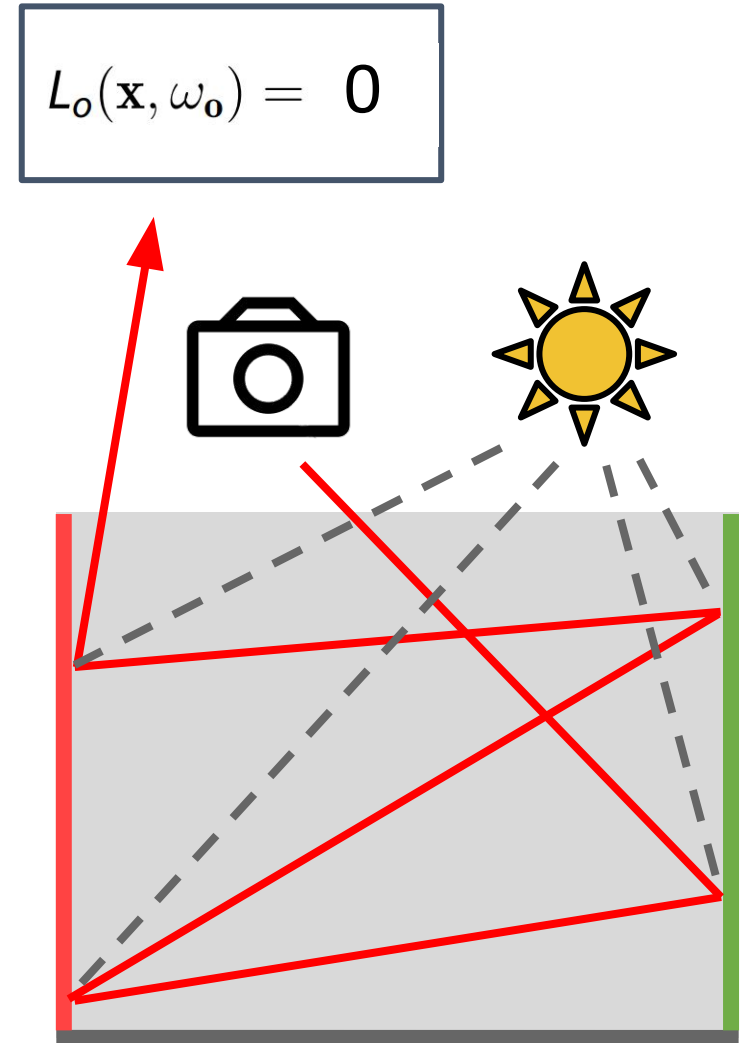
Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- When do we stop generating rays?
(aka. path termination conditions)
 - **(1) When hitting an area light source**



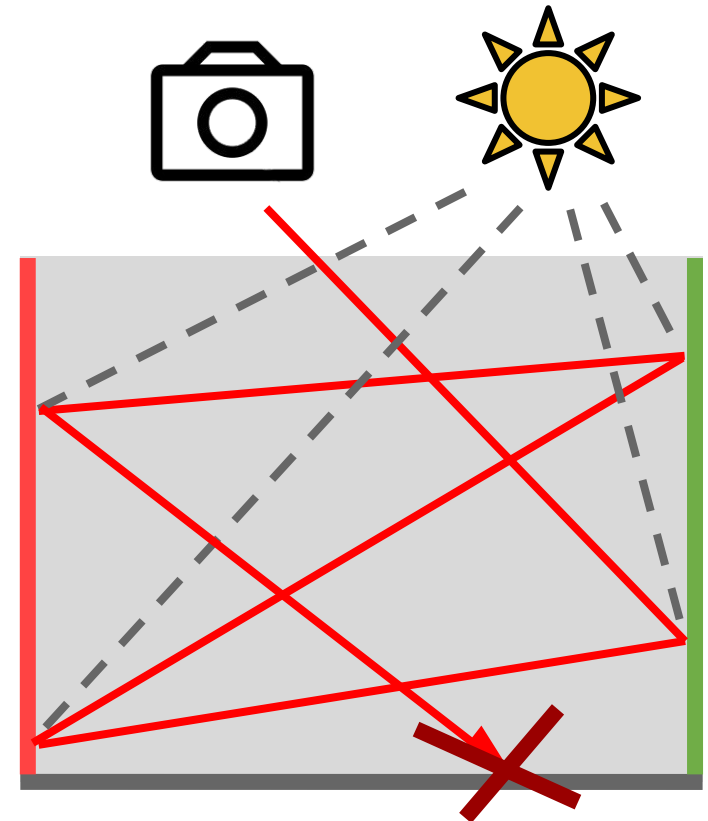
Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- When do we stop generating rays?
(aka. path termination conditions)
 - (1) When hitting an area light source
 - **(2) When ray does not hit anything**



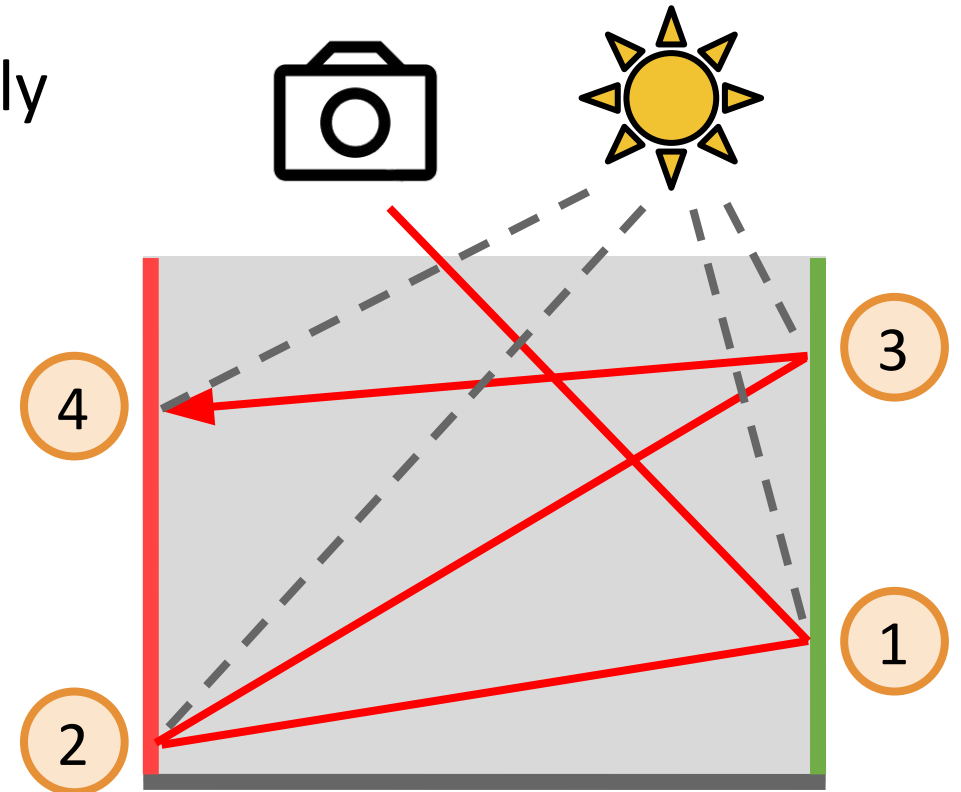
Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- When do we stop generating rays?
(aka. path termination conditions)
 - (1) When hitting an area light source
 - (2) When ray does not hit anything
 - **(3) When there are $>N$ bounces**



Programming recommendations

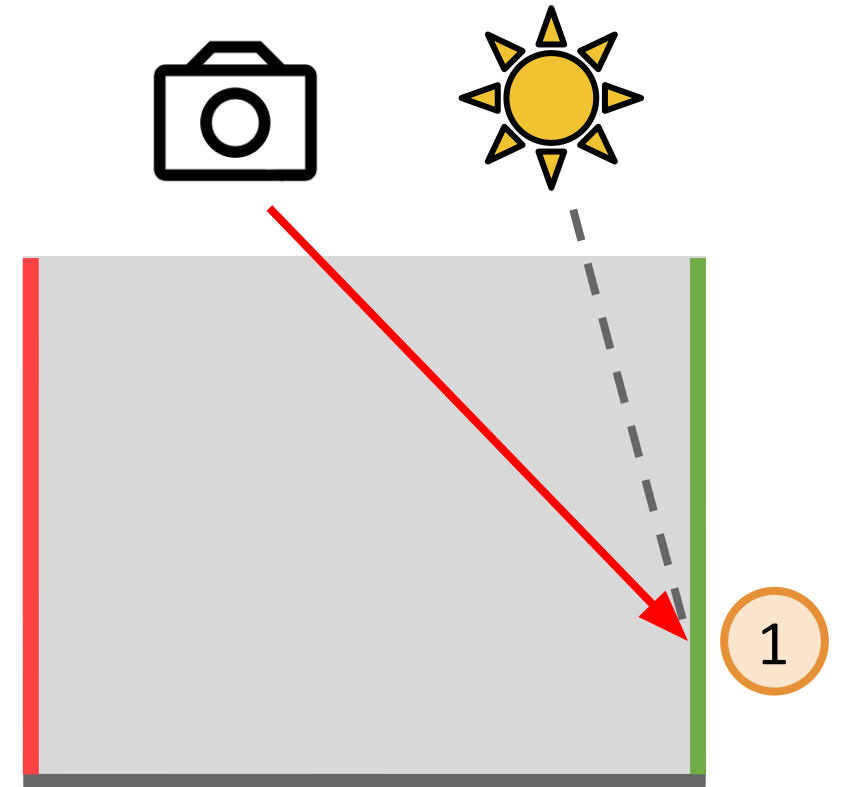
- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- Important: account for light bounces correctly
 - You should **sum** four contributions



Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- Important: account for light bounces correctly
 - In this example, you should **sum** four contributions
 - Light at 1 should be multiplied by BRDF/cosine terms at 1

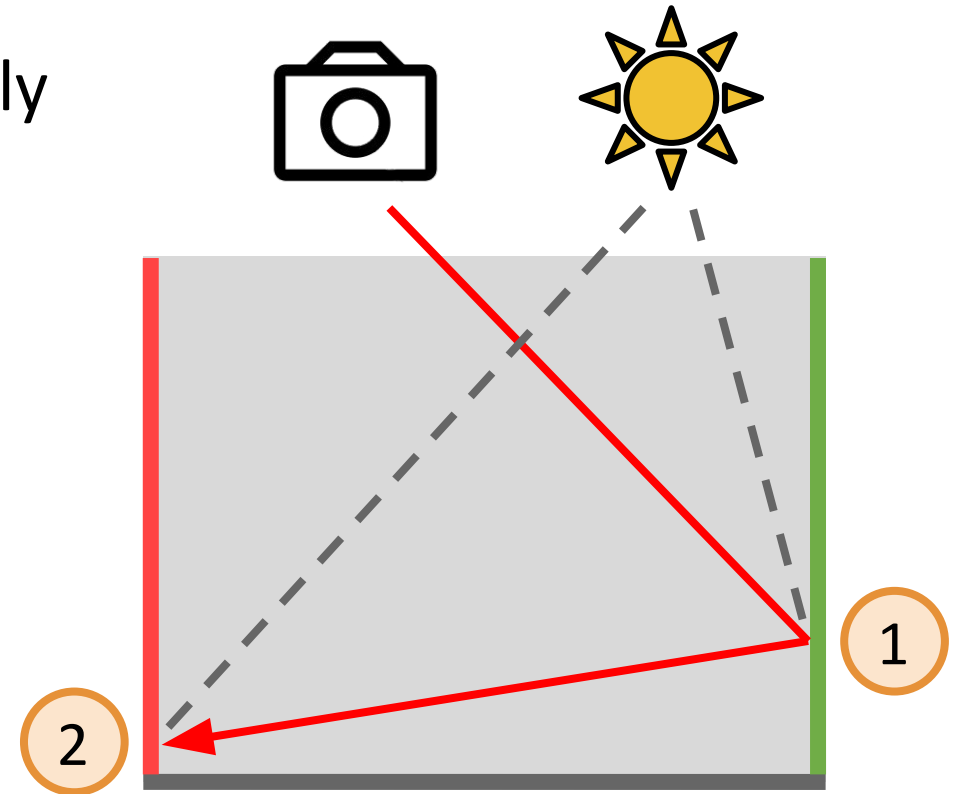
$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|$$



Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- Important: account for light bounces correctly
 - In this example, you should **sum** four contributions
 - Light at 2 should be multiplied by BRDF/cosine terms at 2 1

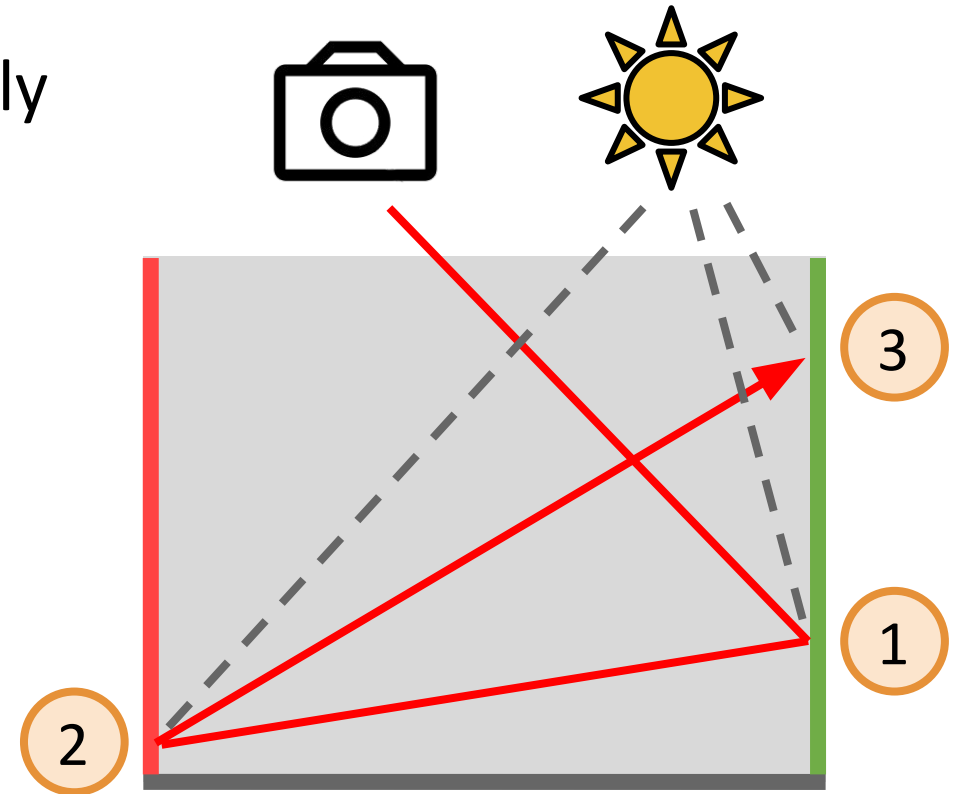
$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|$$



Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- Important: account for light bounces correctly
 - In this example, you should **sum** four contributions
 - Light at 3 should be multiplied by BRDF/cosine terms at 3 2 1

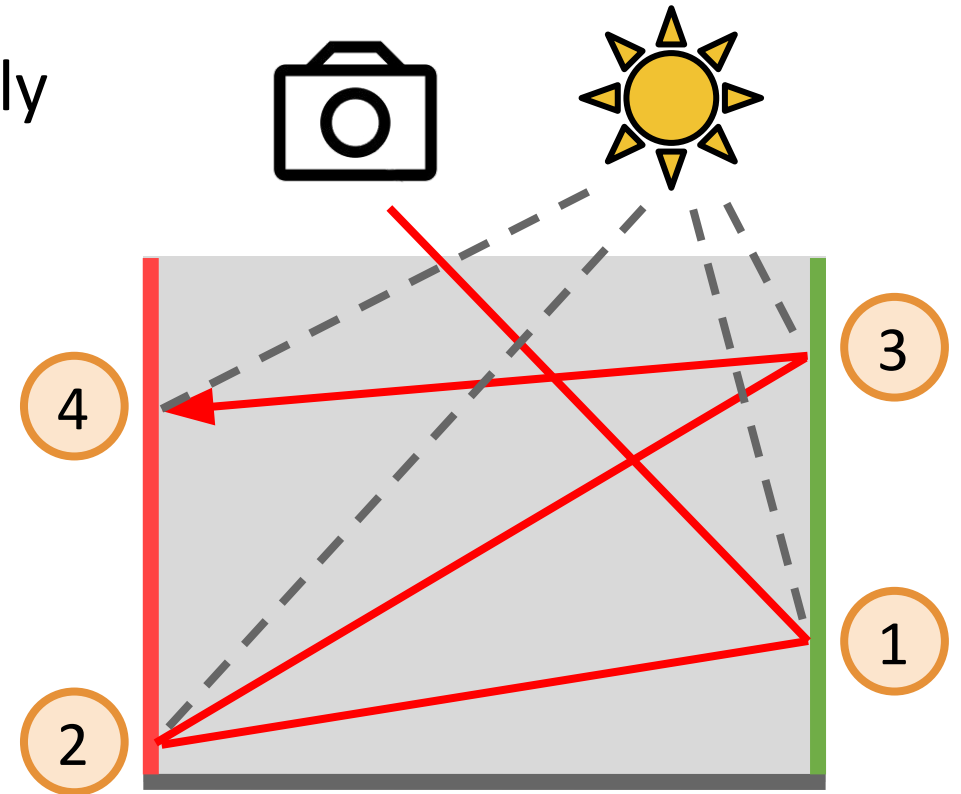
$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|$$



Programming recommendations

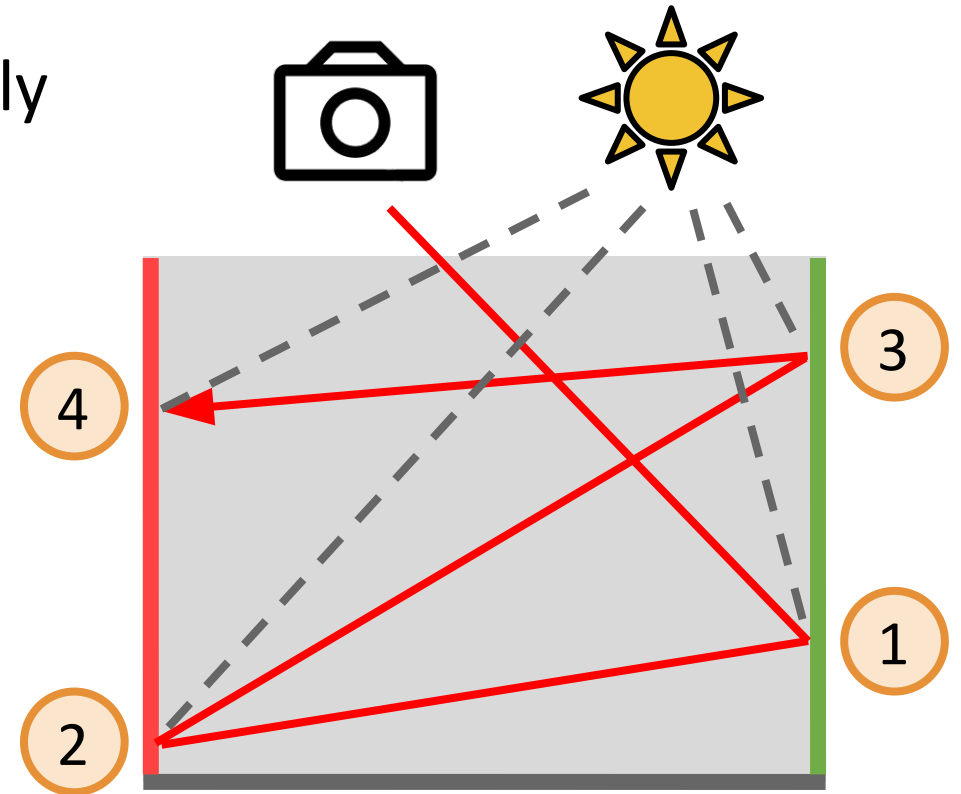
- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- Important: account for light bounces correctly
 - In this example, you should **sum** four contributions
 - Light at 4 should be multiplied by BRDF/cosine terms at 4 3 2 1

$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|$$



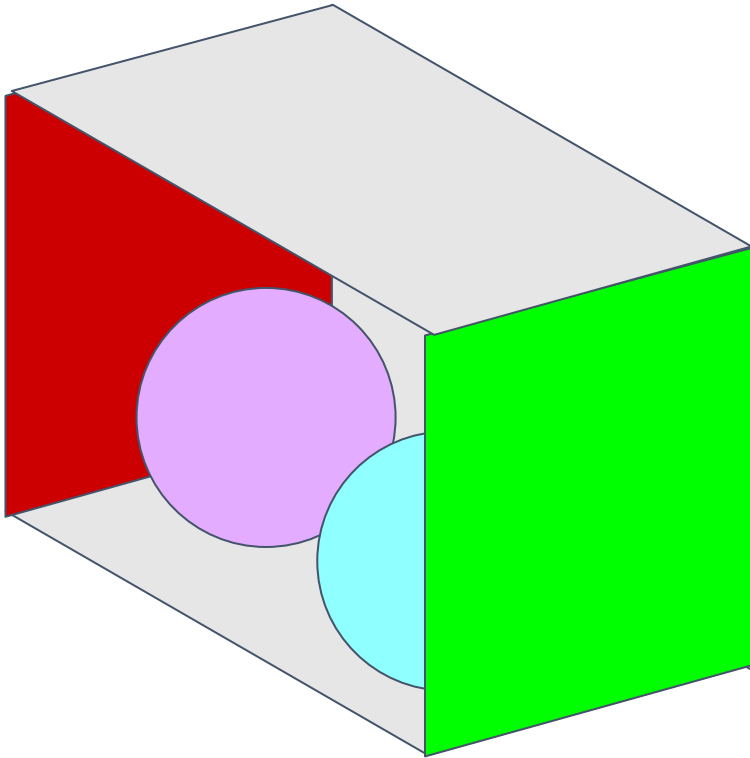
Programming recommendations

- Function can be written as:
 - Recursive (recommended)
 - Iterative (faster, caution)
- Important: account for light bounces correctly
 - In this example, you should **sum** four contributions
 - Total: $\textcircled{1} + \textcircled{2} + \textcircled{3} + \textcircled{4}$
 - Easy to do recursively
 - Tricky to do iteratively



Example scene: Cornell Box

- **Geometry**



Planes defined by normal (n) and distance (d)

Left plane $n = (1, 0, 0), d = 1$

Right plane $n = (-1, 0, 0), d = 1$

Floor plane $n = (0, 1, 0), d = 1$

Ceiling plane $n = (0, -1, 0), d = 1$

Back plane $n = (0, 0, -1), d = 1$

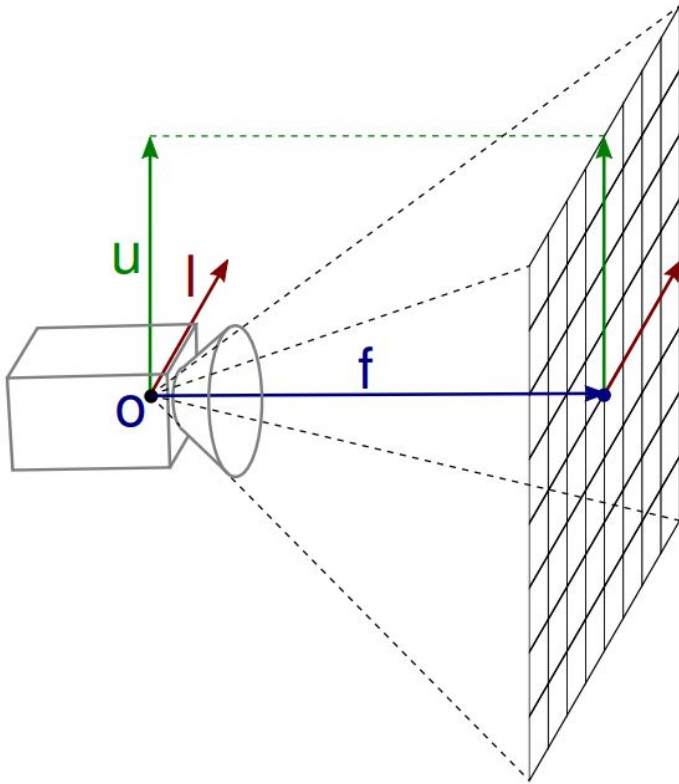
Spheres defined by center (c) and radius (r)

Left sphere $c = (-0.5, -0.7, 0.25), r = 0.3$

Right sphere $c = (0.5, -0.7, -0.25), r = 0.3$

Example scene: Cornell Box

- Camera & light sources



Camera and image plane defined by

Origin $O = (0, 0, -3.5)$

Left $L = (-1, 0, 0)$

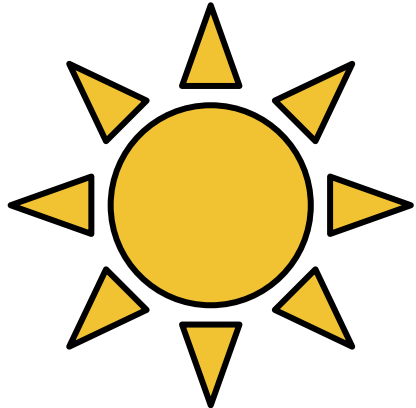
Up $U = (0, 1, 0)$

Forward $F = (0, 0, 3)$

Size 256x256 pixels

Example scene: Cornell Box

- Light sources



Center and power (emission)

Center $c = (0, 0.5, 0)$

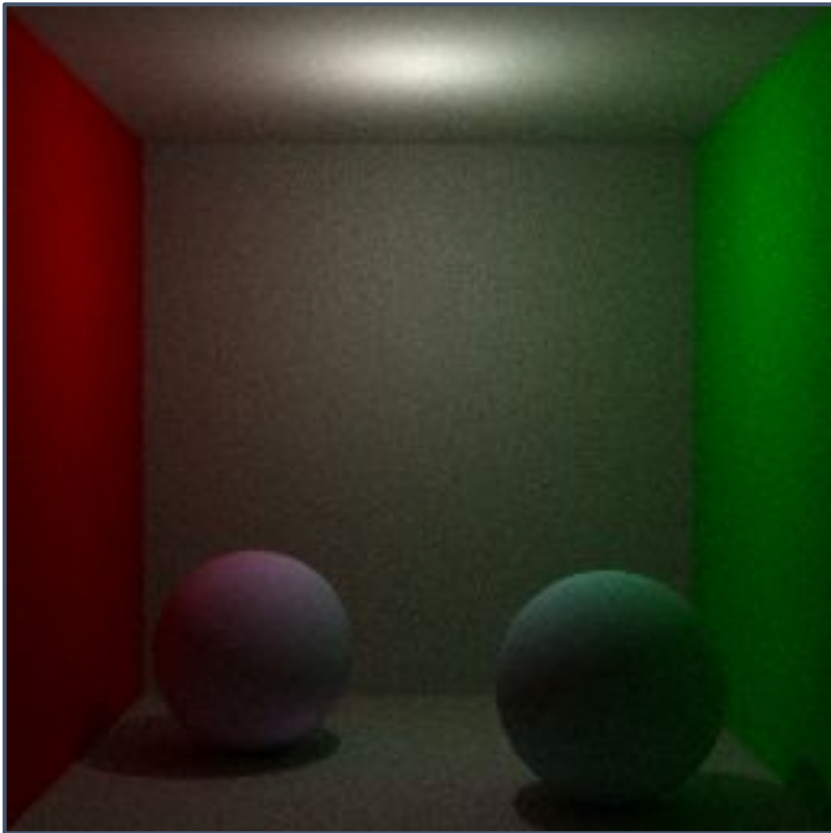
Power can be any number e.g. $p = (1, 1, 1)$

Just be careful with the #MAX

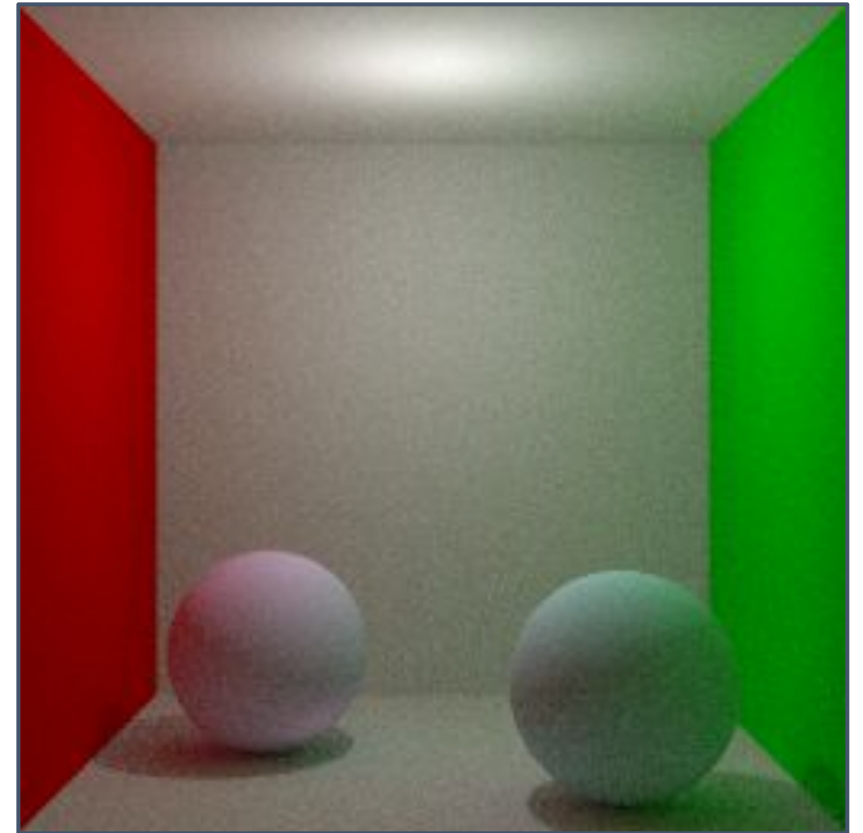
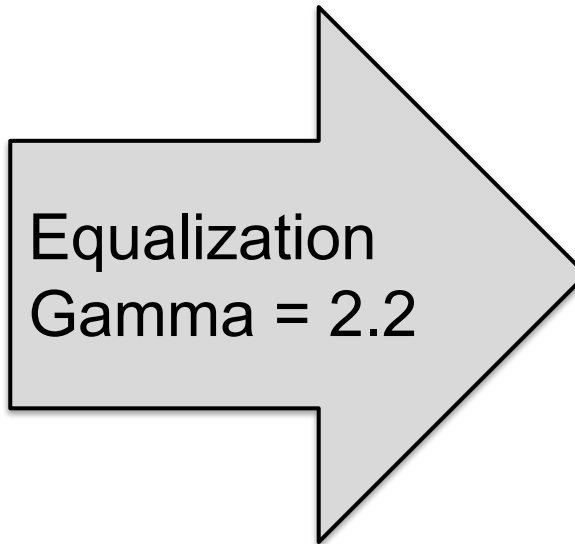
```
1  P3
2  # feep.ppm
3  #MAX=<maximum of your RGB memory values>
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

Example scene: Cornell Box

- Results (no area lights + point light)



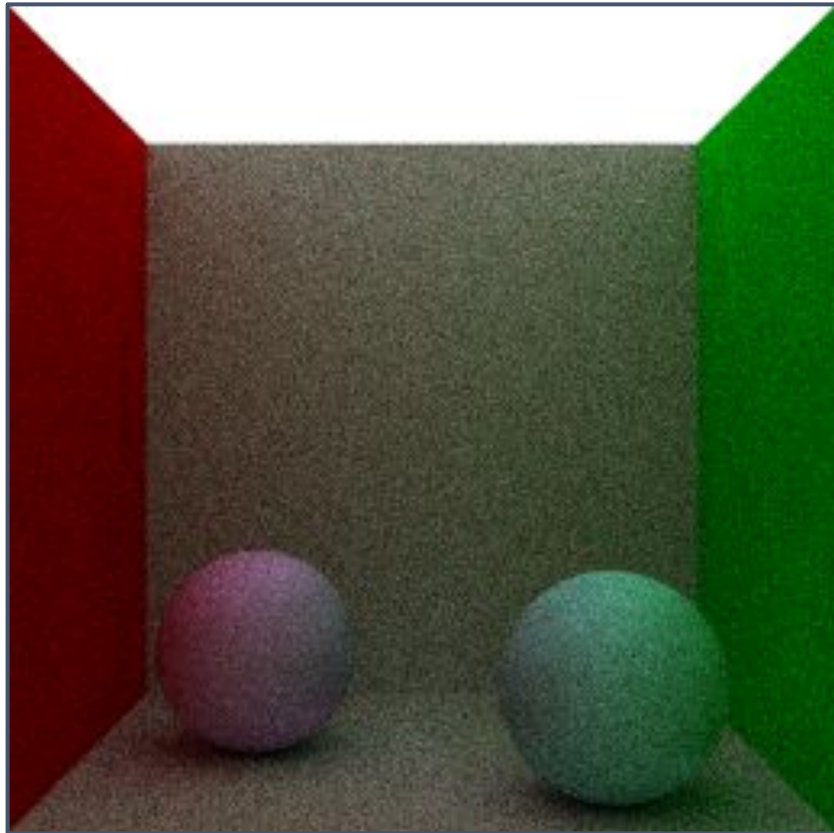
Using a point light



With tone mapping

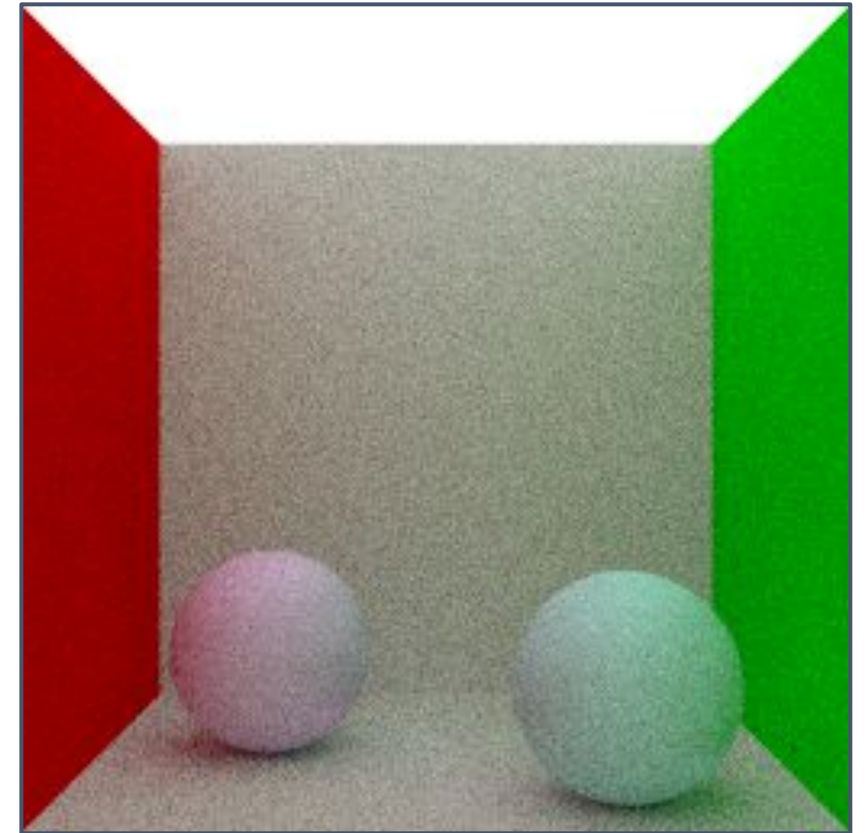
Example scene: Cornell Box

- Results (no point light + ceiling plane is an area light)



Using an area light

Equalization
 $\text{Gamma} = 2.2$



With tone mapping

Questions

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice implement:

- Diffuse BRDF (k_d for each material)
 - **“Evaluate” function:** Return $f(\mathbf{x}, \omega_i, \omega_o)$ (you should have already programmed this)
 - **“Sample” function:** Return random direction ω_i and $f(\mathbf{x}, \omega_i, \omega_o)$
- Find point light sources using **next-event estimation** on each bounce
- Terminate paths when: (1) no intersection (2) area light is hit (3) $>N$ bounces
- Recommended deadline: November 13th (moodle: January 11th)
 - Extensions (do not count towards recommended deadline):
 - **Textures:** make diffuse coefficient k_d depend on hit position $k_d(\mathbf{x})$
 - **Fresnel effects:** make diffuse coefficient k_d depend on viewing direction $k_d(\omega_o)$
 - **Parallelization:** divide work between several threads, estimate time to finish execution
 - **More:** importance sampling next-event estimation, etc. (see the lab assignment or ask us)

Lab #4 – Path tracing (part 3)

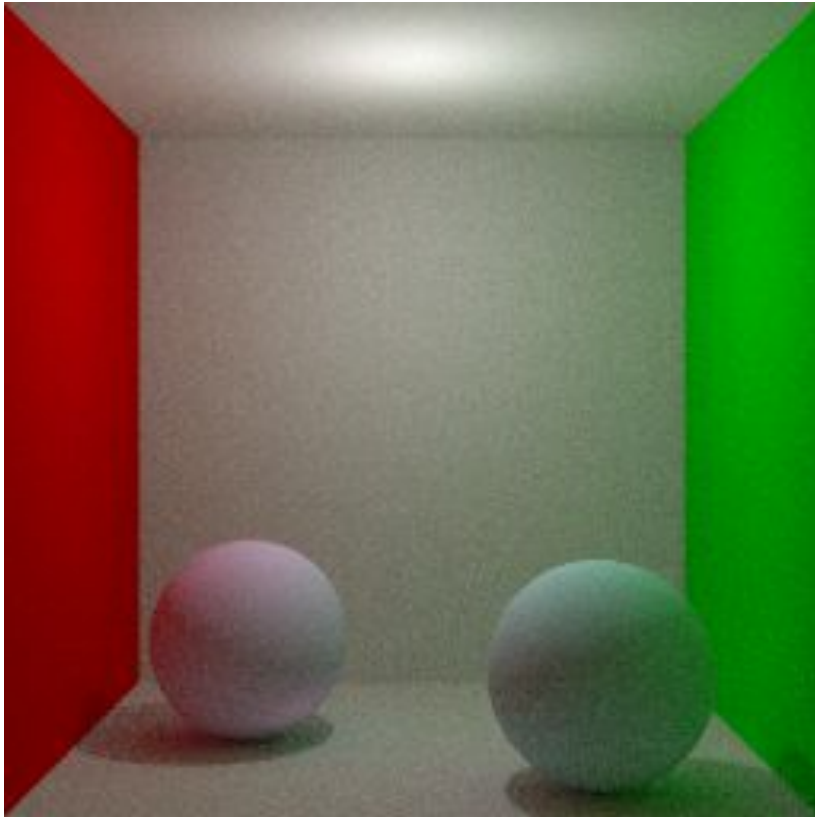
Informática Gráfica

Adolfo Muñoz - Julio Marco
Pablo Luesia - J. Daniel Subías – Óscar Pueyo

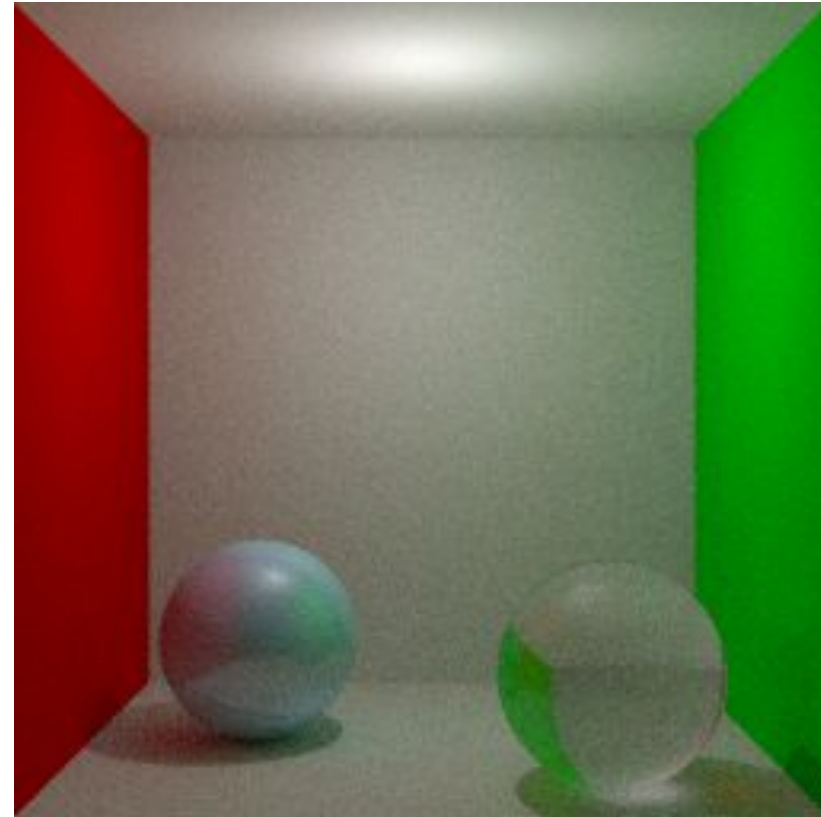


Before we begin...

- Today: add **more materials** to the path tracing algorithm



Previous session (diffuse)

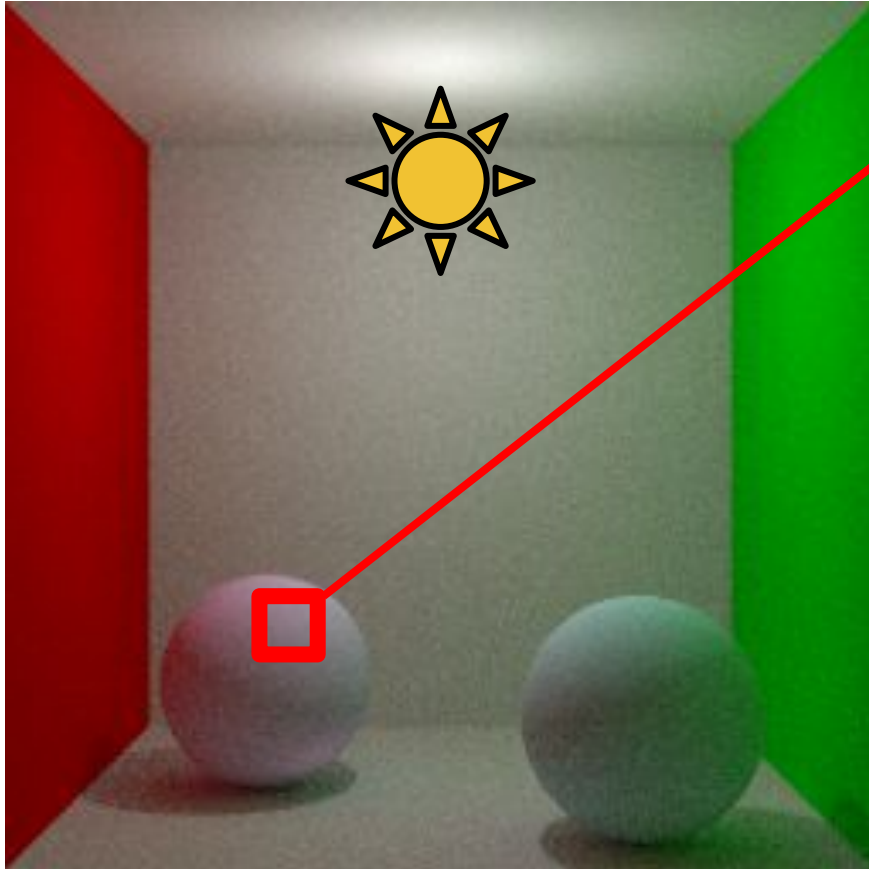


+ Perfect specular and refraction

Before we begin...

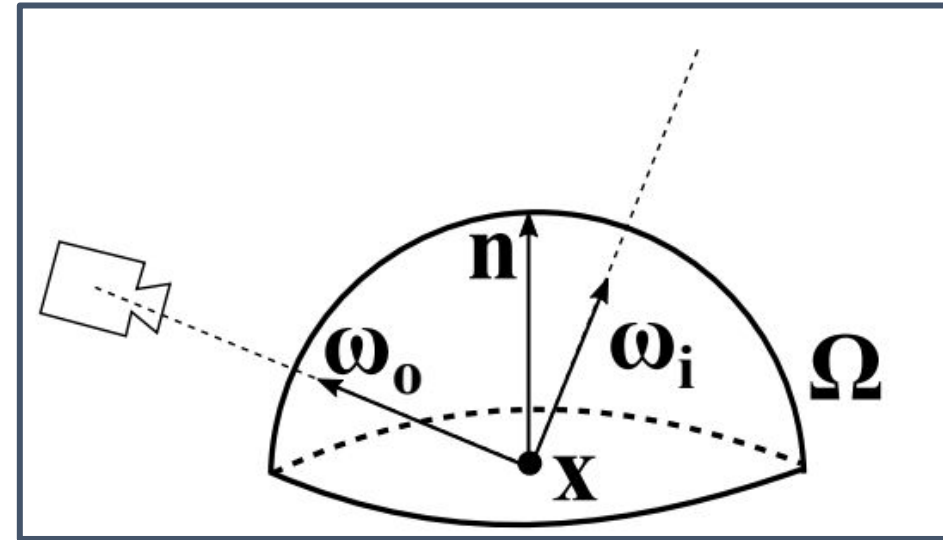
- Lab 4 (path tracing) **is the first submitted work**
 - Recommended deadline: November 13th
 - Moodle: January 11th
 - You will use most of today's code for Lab 5 (photon mapping) too
- Remember: Final work is 80% of the final grade

Recap: Which color do we fill each pixel with?

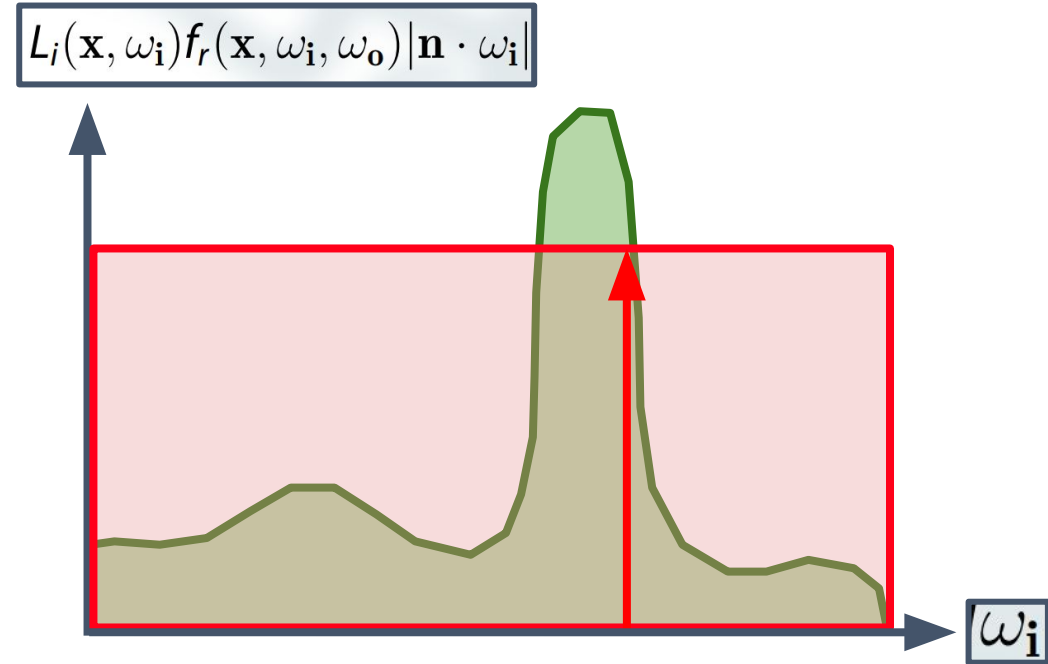
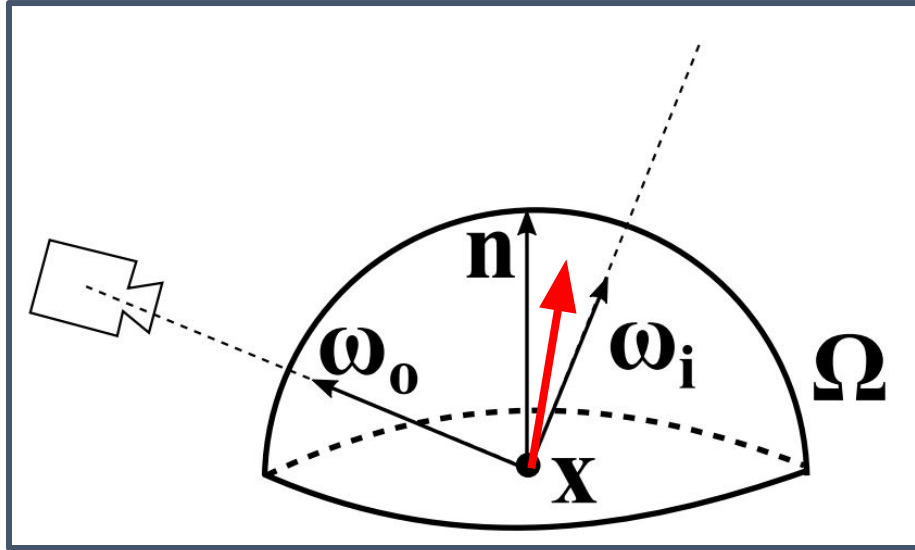


$$L_o(\mathbf{x}, \omega_o) = \cancel{L_e(\mathbf{x}, \omega_o)} + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

The full integral



Recap: Approximating one integral

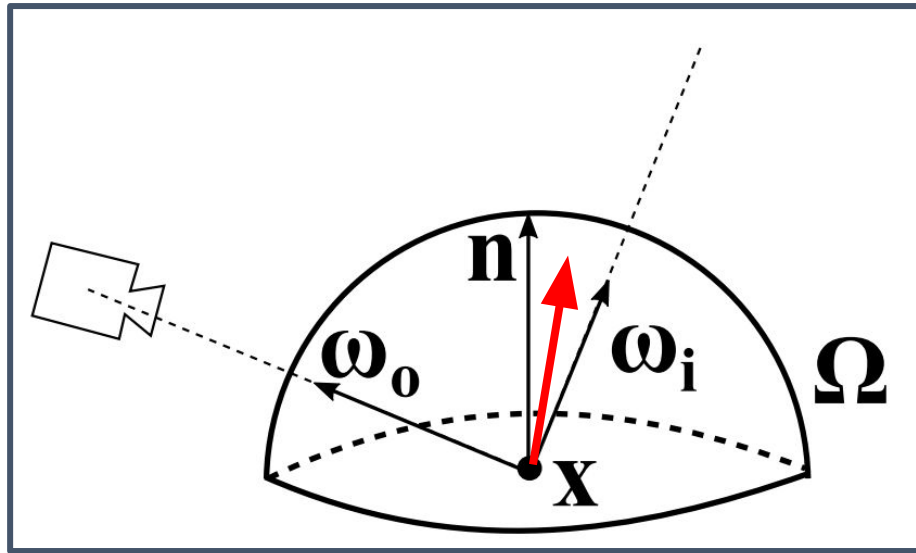


- There are infinite values for ω_i
- Idea 3: **Monte Carlo estimator**, use the mean of $N = 1$ random sample



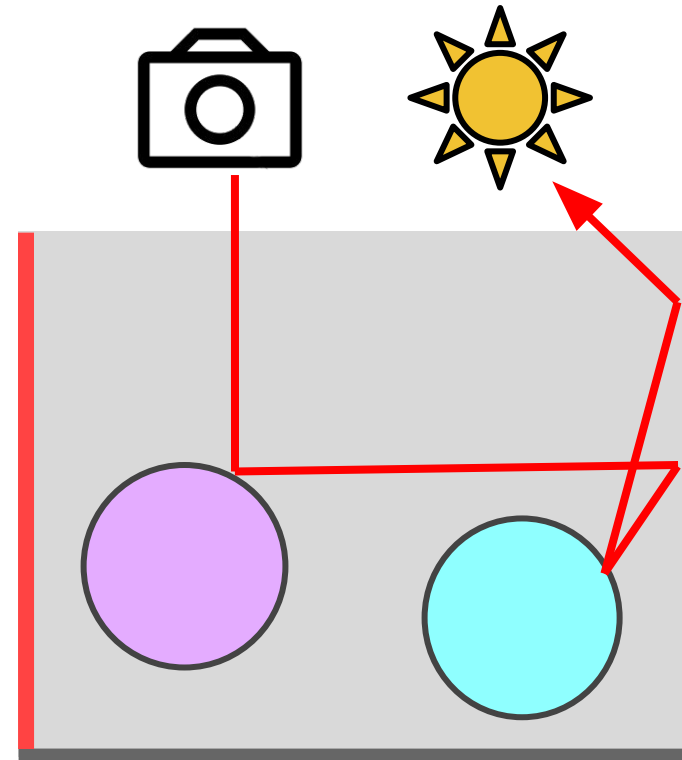
Recap: Approximating one integral in practice

- Monte Carlo estimation for the path integral



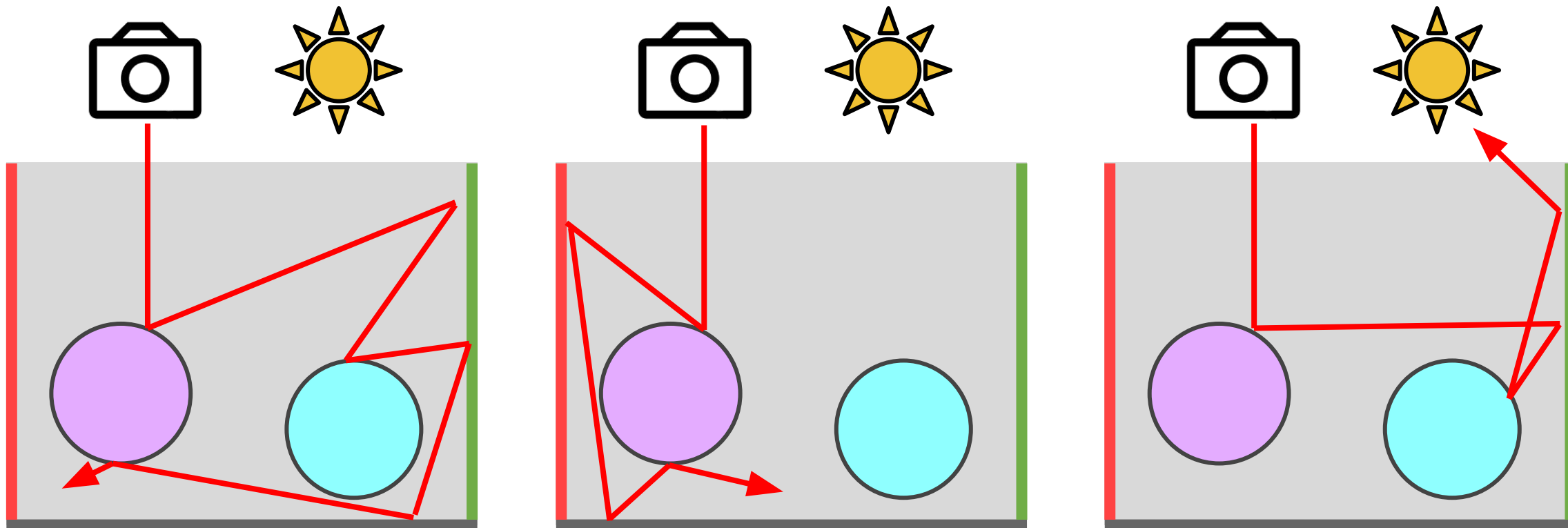
One random ω_i on each bounce

One random path



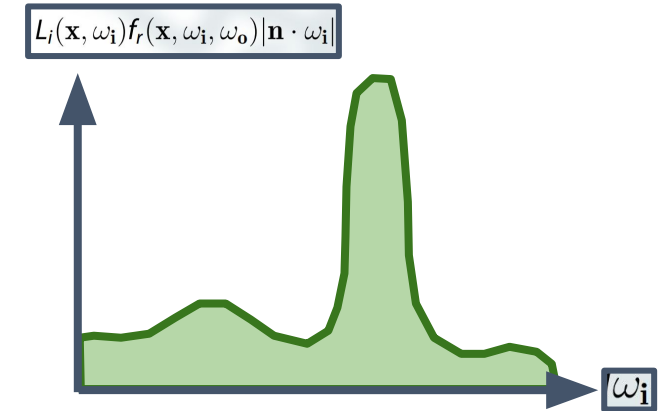
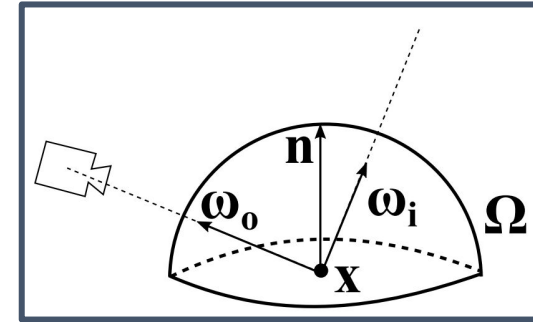
Recap: Approximating one integral in practice

- Monte Carlo estimation for the path integral
 - Sum of multiple random paths
 - More paths $\rightarrow$ better approximation of the integral (better result)



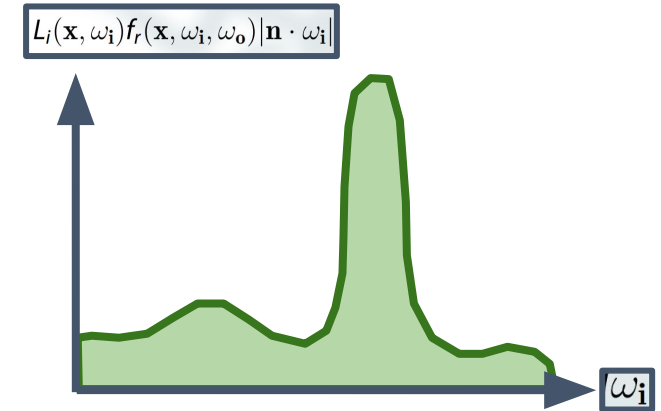
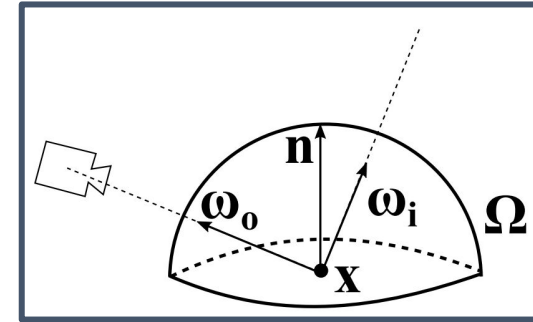
Recap: Sampling directions in Ω

- How to generate a value for ω_i ?
- Use the sampling method (probability distribution of ω_i) that benefits you



Recap: Sampling directions in Ω

- How to generate a value for ω_i ?
- Use the sampling method (probability distribution of ω_i) that benefits you



Uniform solid angle sampling

$$p(\theta_i) = \sin \theta_i$$

$$p(\phi_i) = \frac{1}{2\pi}$$

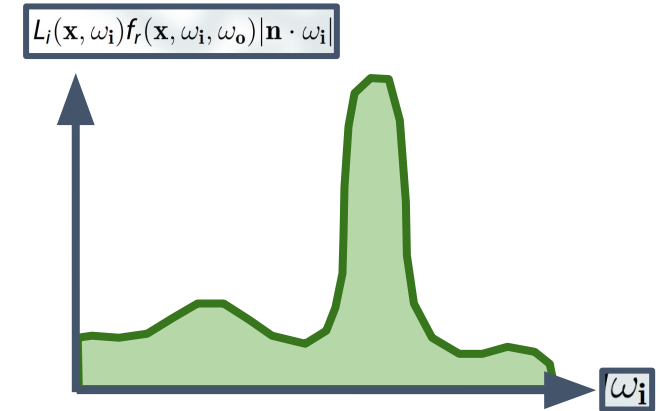
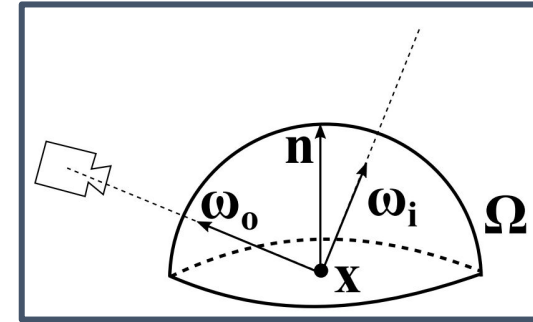
Uniform cosine sampling

$$p(\theta_i) = 2 \sin \theta_i \cos \theta_i$$

$$p(\phi_i) = \frac{1}{2\pi}$$

Recap: Sampling directions in Ω

- How to generate a value for ω_i ?
- Use the sampling method (probability distribution of ω_i) that benefits you



Uniform solid angle sampling

$$p(\theta_i) = \sin \theta_i$$

$$p(\phi_i) = \frac{1}{2\pi}$$

$$c^{-1}(\xi_{\theta_i}) = \arccos \xi_{\theta_i}$$

$$c^{-1}(\xi_{\phi_i}) = 2\pi \xi_{\phi_i}$$

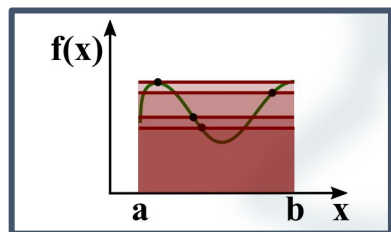
Uniform cosine sampling

$$p(\theta_i) = 2 \sin \theta_i \cos \theta_i \quad c^{-1}(\xi_{\theta_i}) = \arccos \sqrt{1 - \xi_{\theta_i}}$$

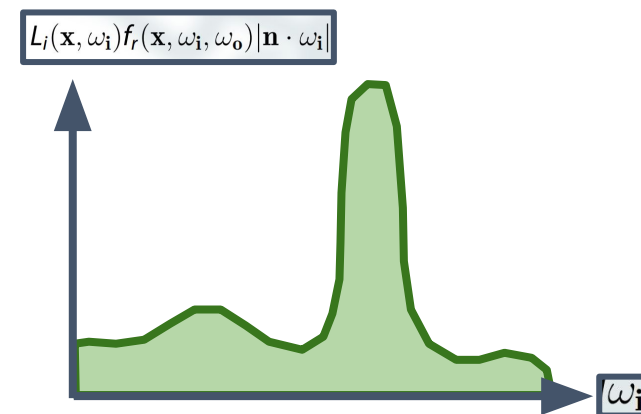
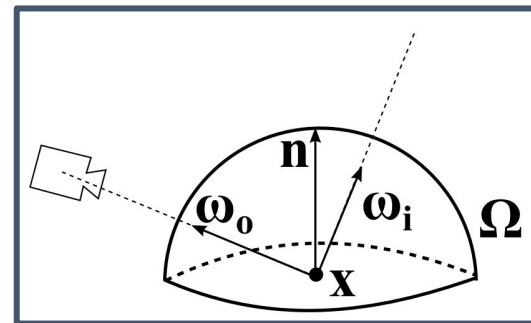
$$p(\phi_i) = \frac{1}{2\pi}$$

$$c^{-1}(\xi_{\phi_i}) = 2\pi \xi_{\phi_i}$$

Recap: Sampling directions in Ω



$$\int_a^b f(x) dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$



Uniform solid angle sampling

$$p(\theta_i) = \sin \theta_i$$

$$p(\phi_i) = \frac{1}{2\pi}$$

$$c^{-1}(\xi_{\theta_i}) = \arccos \xi_{\theta_i}$$

$$c^{-1}(\xi_{\phi_i}) = 2\pi \xi_{\phi_i}$$

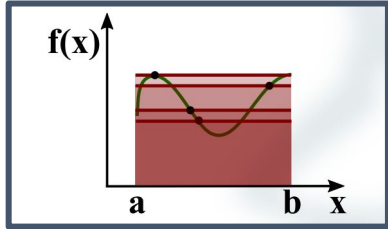
Uniform cosine sampling

$$p(\theta_i) = 2 \sin \theta_i \cos \theta_i \quad c^{-1}(\xi_{\theta_i}) = \arccos \sqrt{1 - \xi_{\theta_i}}$$

$$p(\phi_i) = \frac{1}{2\pi}$$

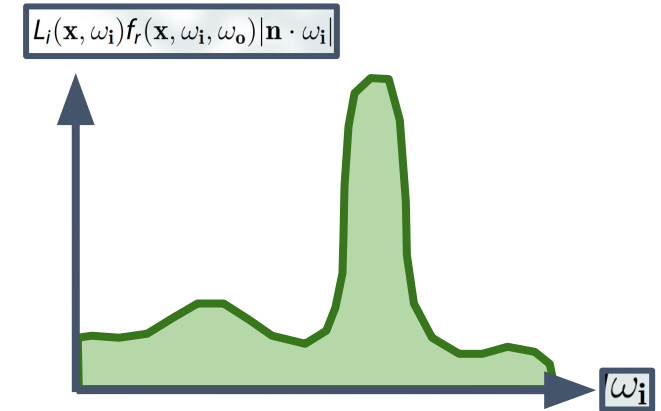
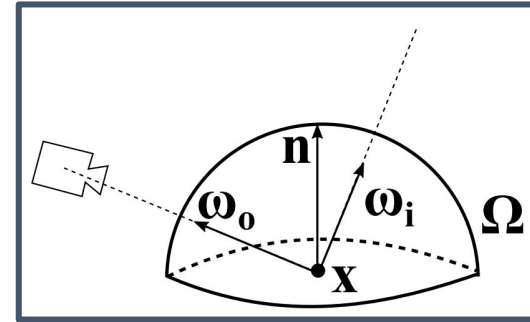
$$c^{-1}(\xi_{\phi_i}) = 2\pi \xi_{\phi_i}$$

Recap: Sampling directions in Ω



$$\int_a^b f(x) dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$

**Remember
to simplify**



Uniform solid angle sampling

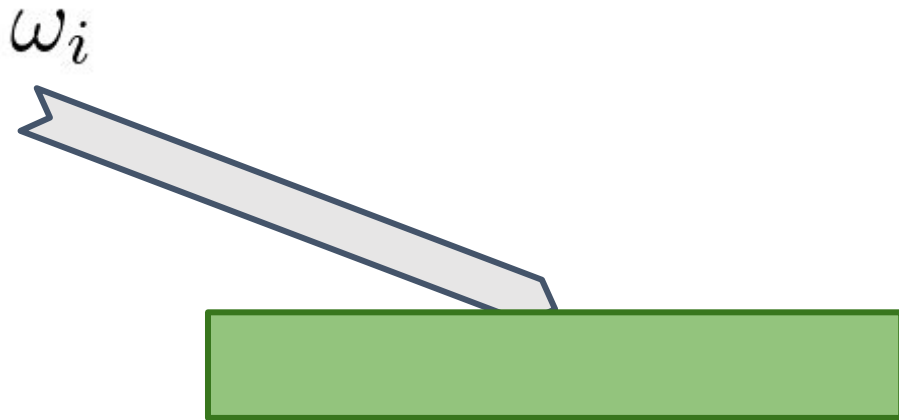
$$L_o(\mathbf{x}, \omega_o) \approx \sum_{i=1}^N \frac{2\pi L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) \cos \theta_i \sin \theta_i}{\sin \theta_i}$$

Uniform cosine sampling

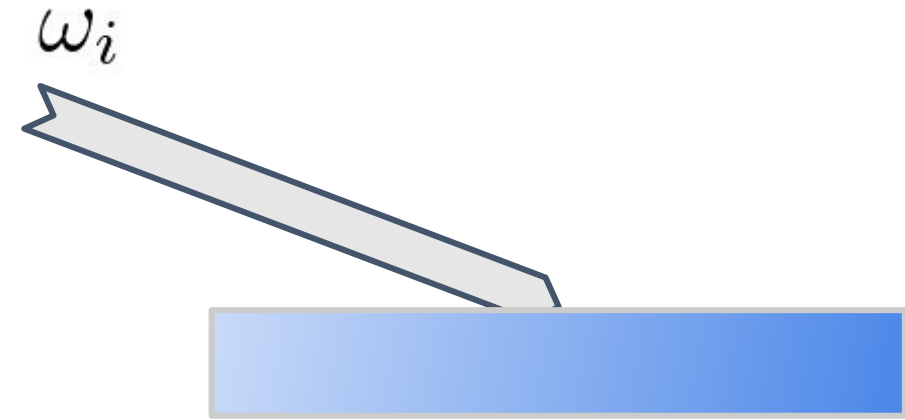
$$L_o(\mathbf{x}, \omega_o) \approx \sum_{i=1}^N \frac{2\pi L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) \cos \theta_i \sin \theta_i}{2 \sin \theta_i \cos \theta_i}$$

Diffuse and specular BRDFs

Diffuse material



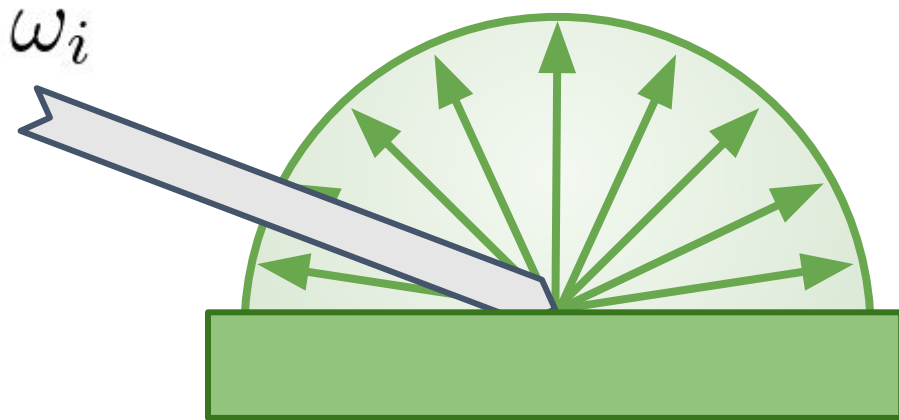
Perfect specular material



Diffuse and specular BRDFs

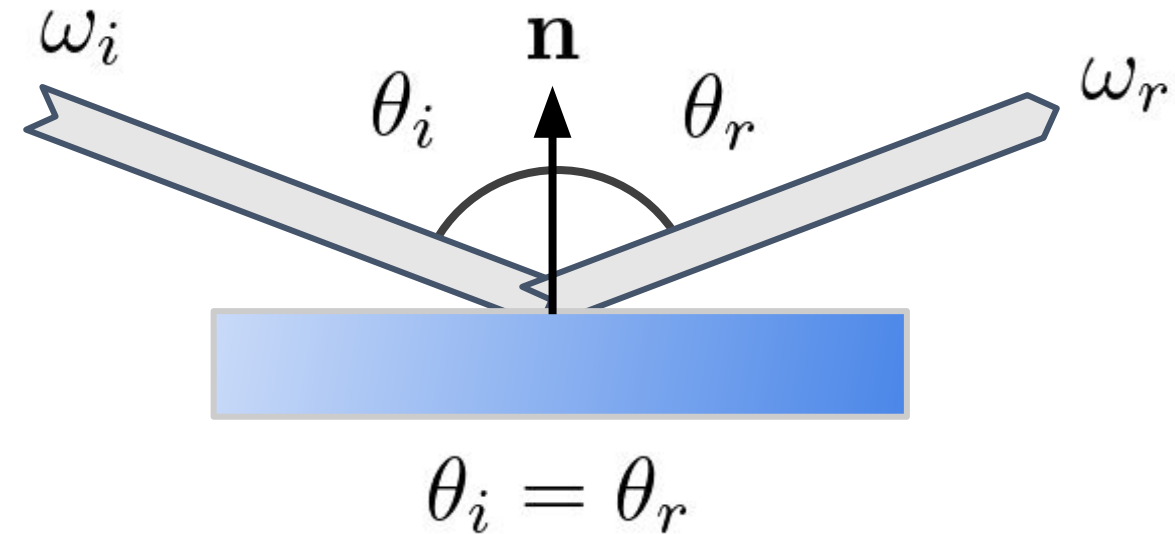
Diffuse material

Light is reflected in all directions equally



Perfect specular material

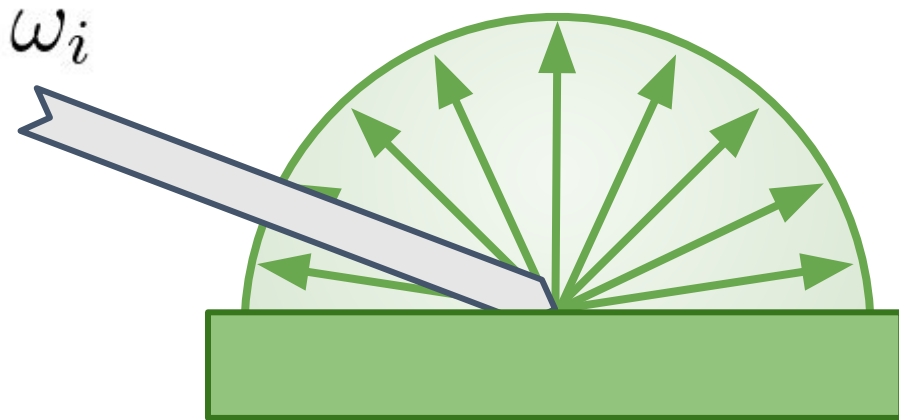
All light is (perfectly) reflected towards one direction ω_r



Diffuse and specular BRDFs

Diffuse material

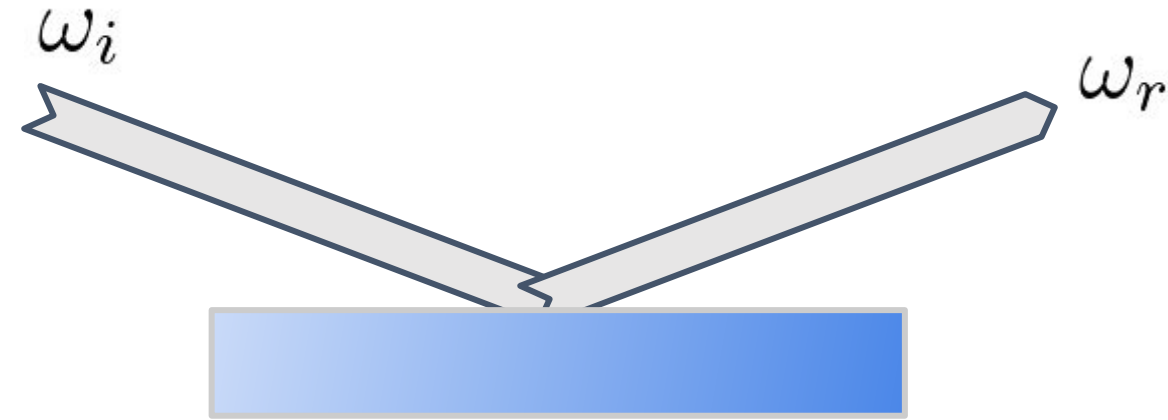
Light is reflected in all directions equally



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{k_d}{\pi}$$

Perfect specular material

All light is (perfectly) reflected towards one direction ω_r

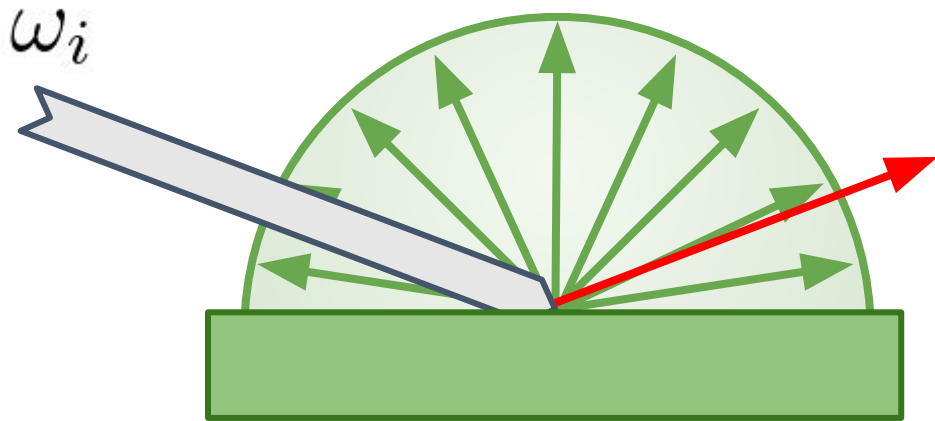


$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_r}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

Diffuse and specular BRDFs

Diffuse material

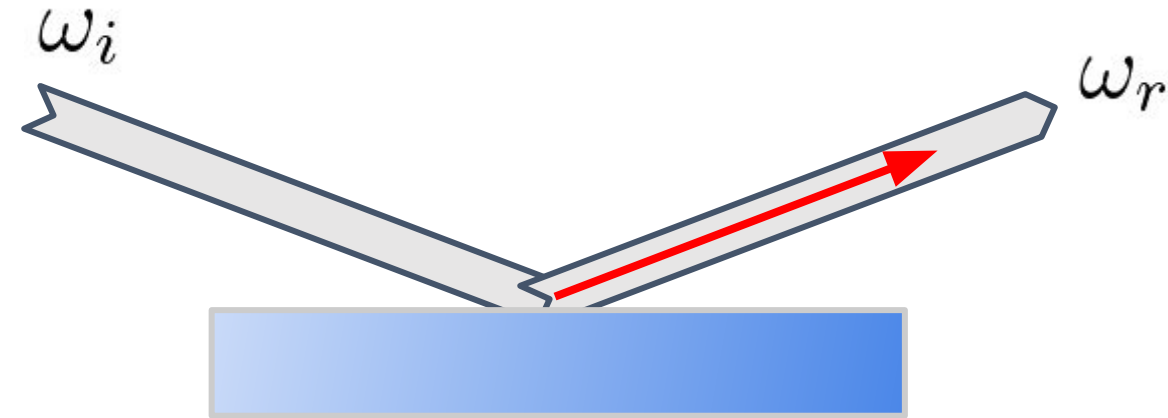
Light is reflected in all directions equally



$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| = \frac{k_d}{\pi} |\mathbf{n} \cdot \omega_i|$$

Perfect specular material

All light is (perfectly) reflected towards one direction ω_r

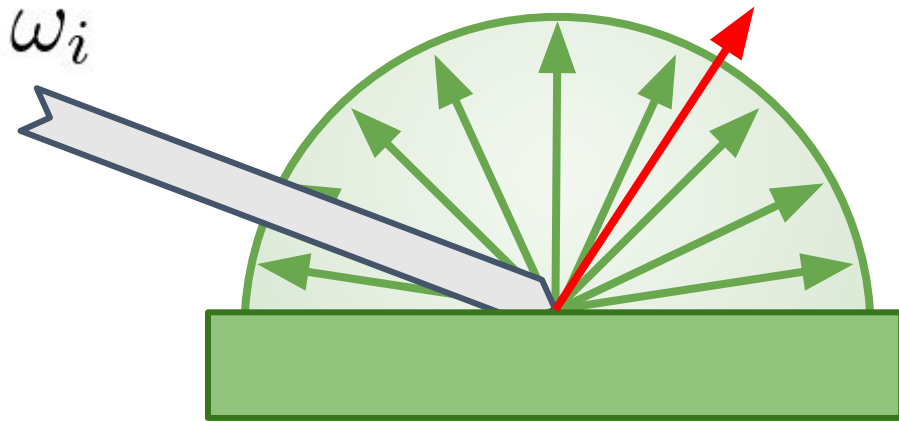


$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| = 1$$

Diffuse and specular BRDFs

Diffuse material

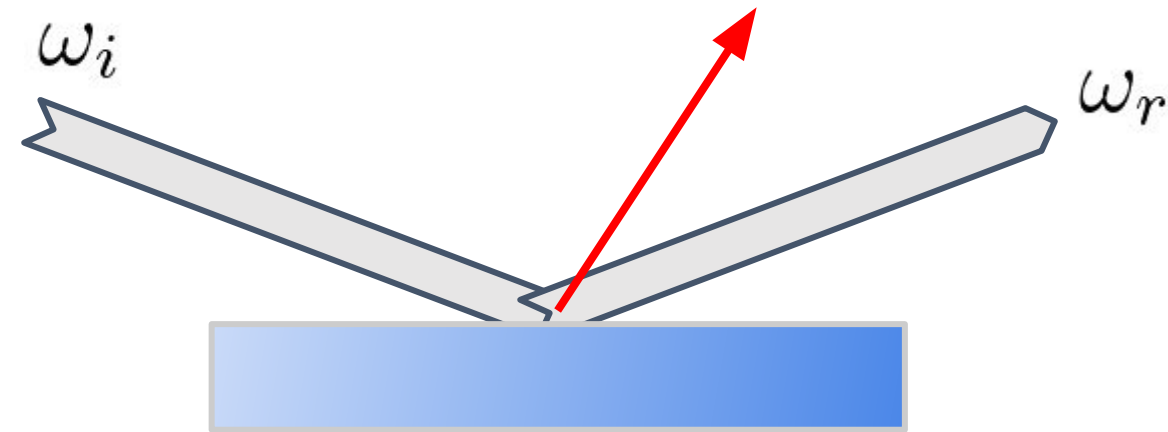
Light is reflected in all directions equally



$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| = \frac{k_d}{\pi} |\mathbf{n} \cdot \omega_i|$$

Perfect specular material

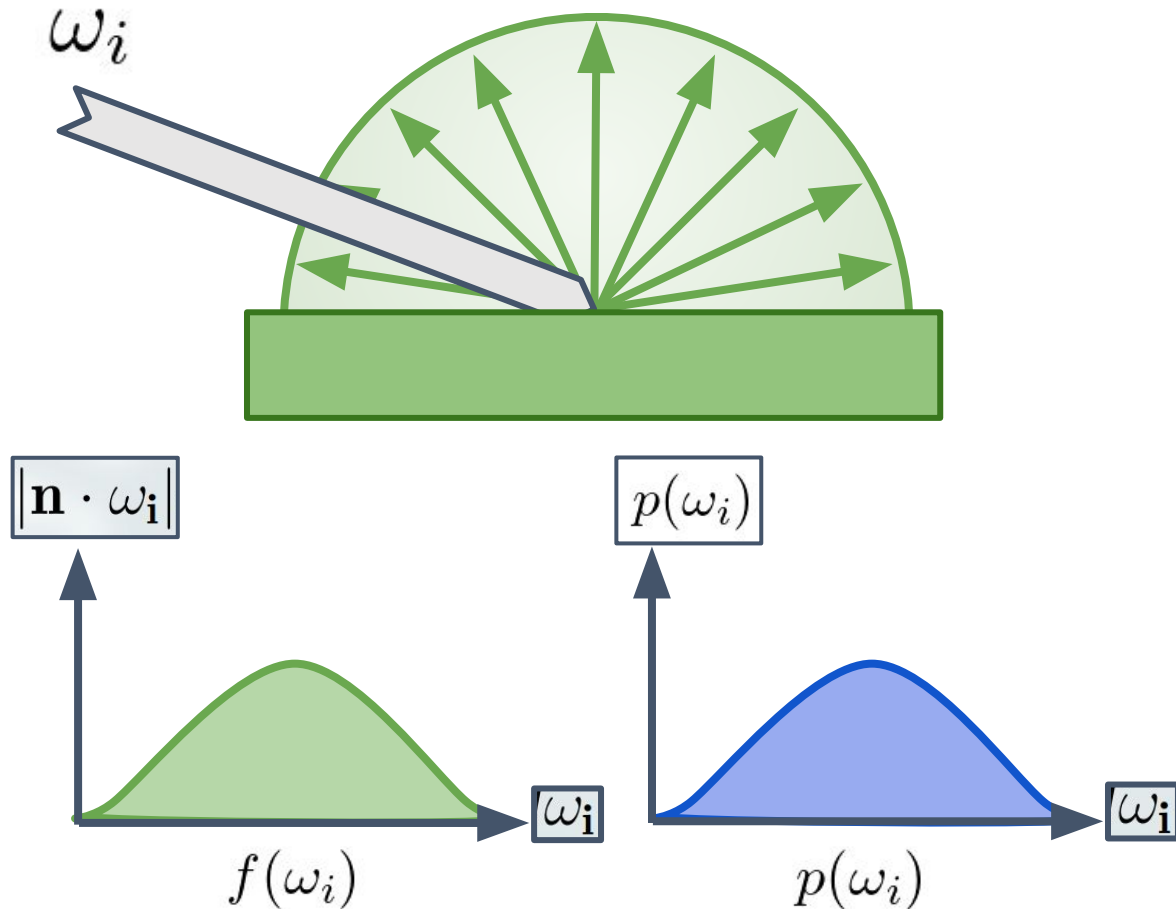
All light is (perfectly) reflected towards one direction ω_r



$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| = 0$$

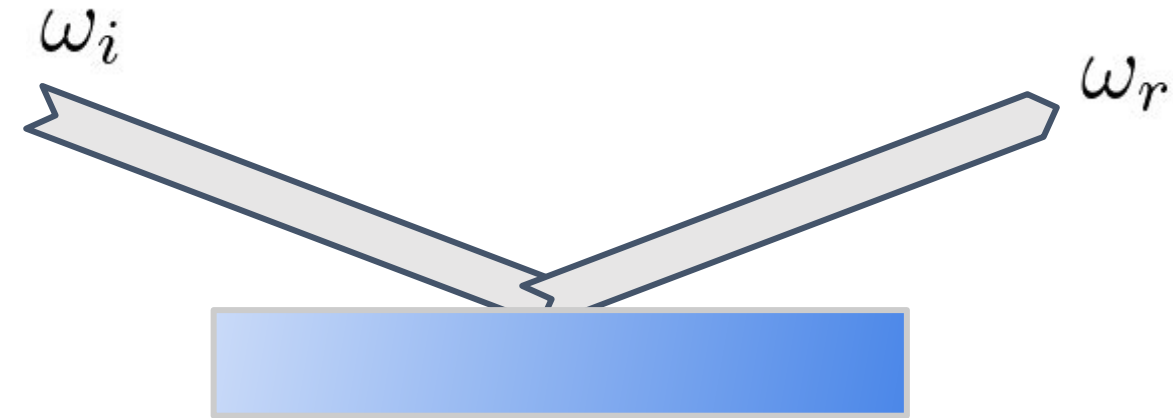
Diffuse and specular BRDFs

Diffuse material



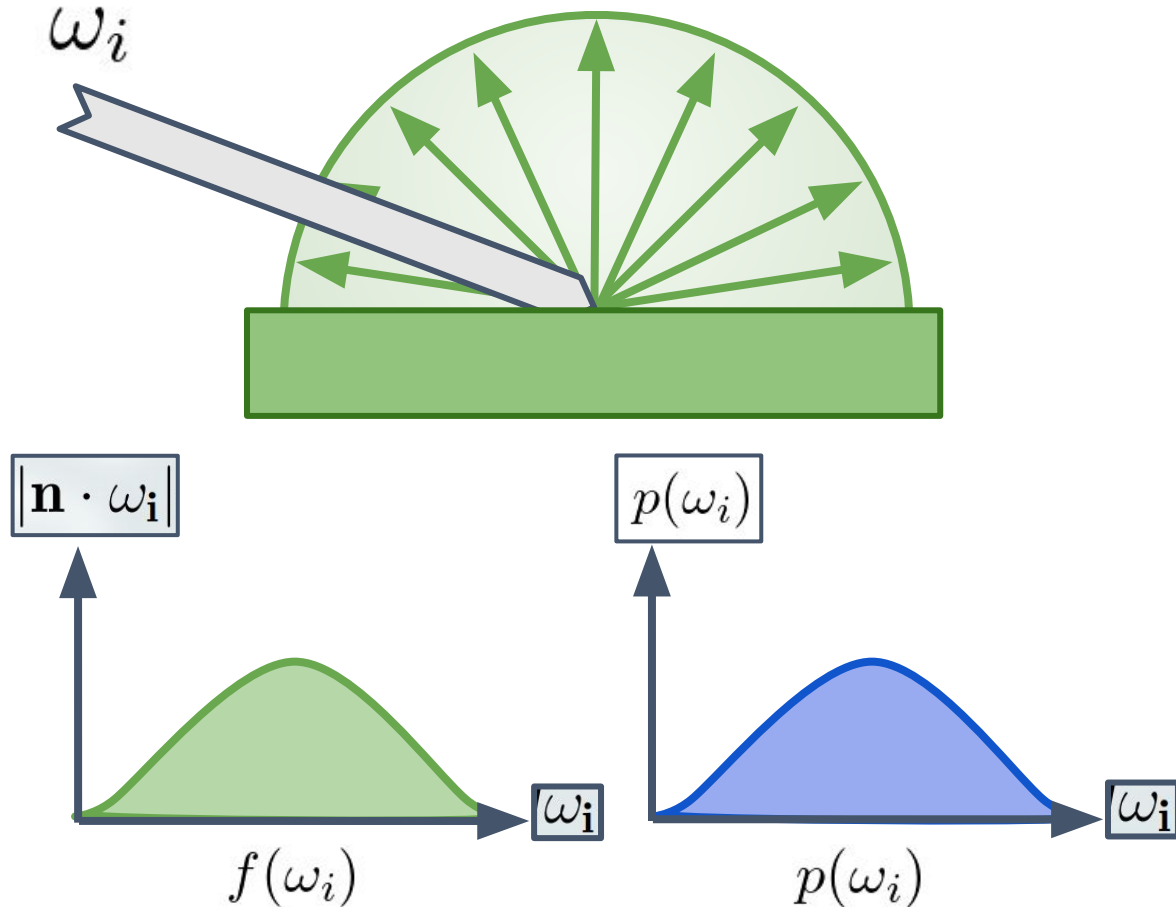
Uniform cosine sampling is good

Perfect specular material



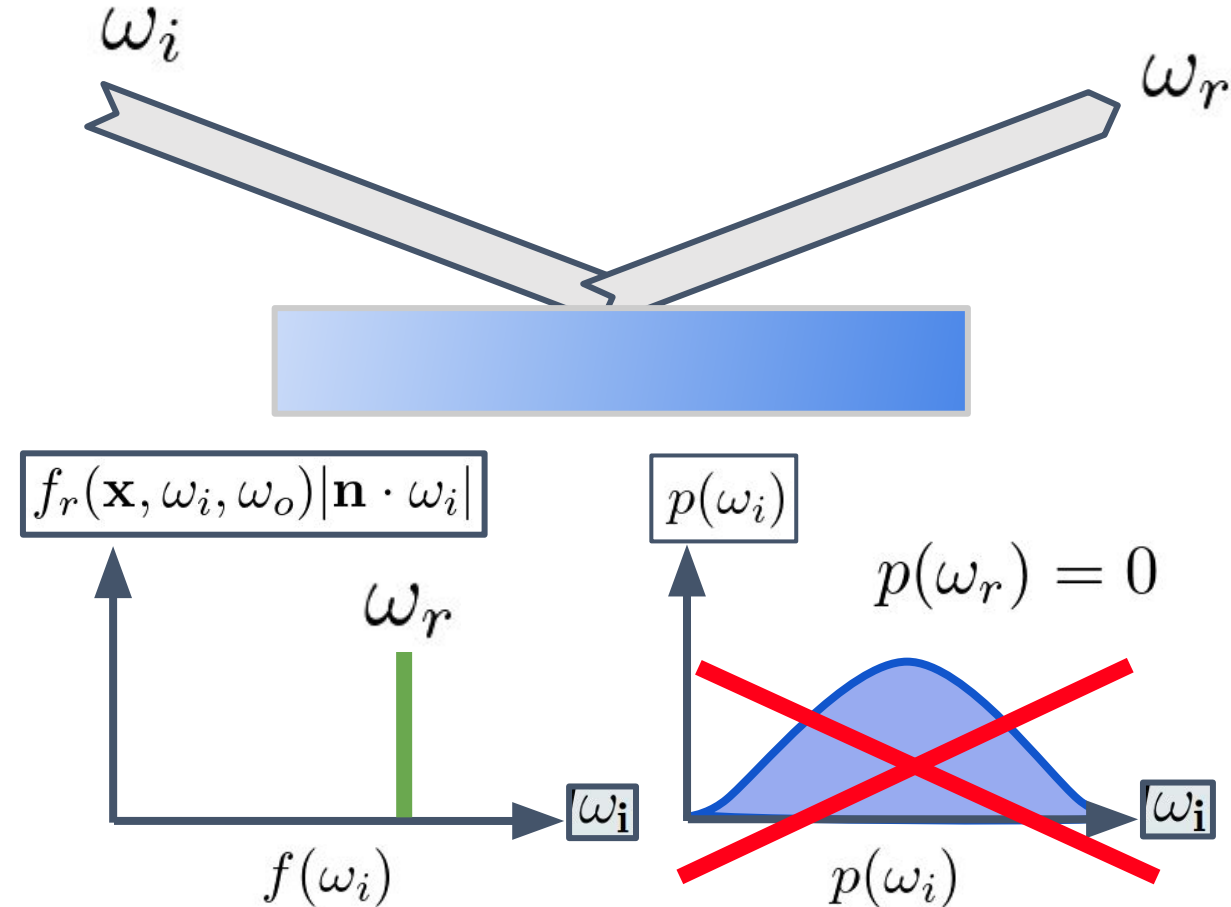
Diffuse and specular BRDFs

Diffuse material



Uniform cosine sampling is good

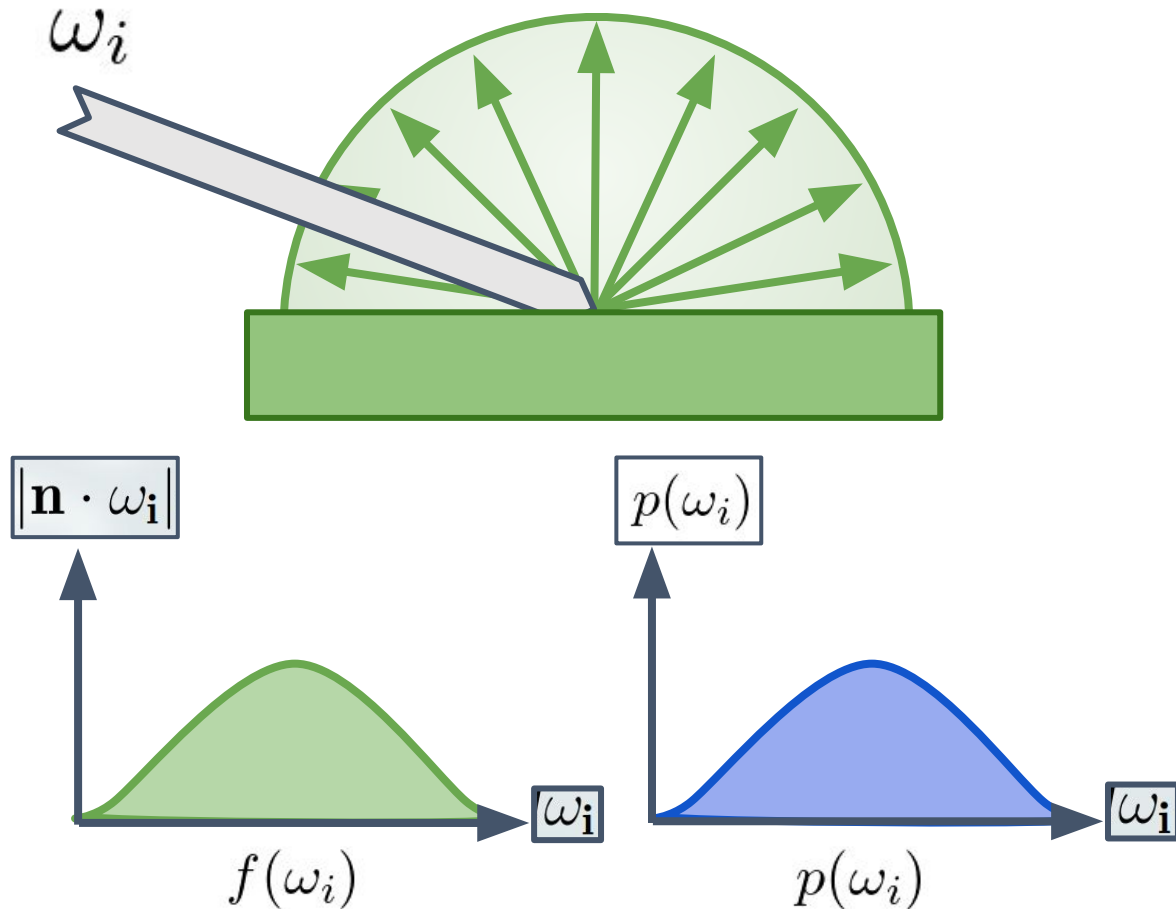
Perfect specular material



Uniform cosine sampling is wrong

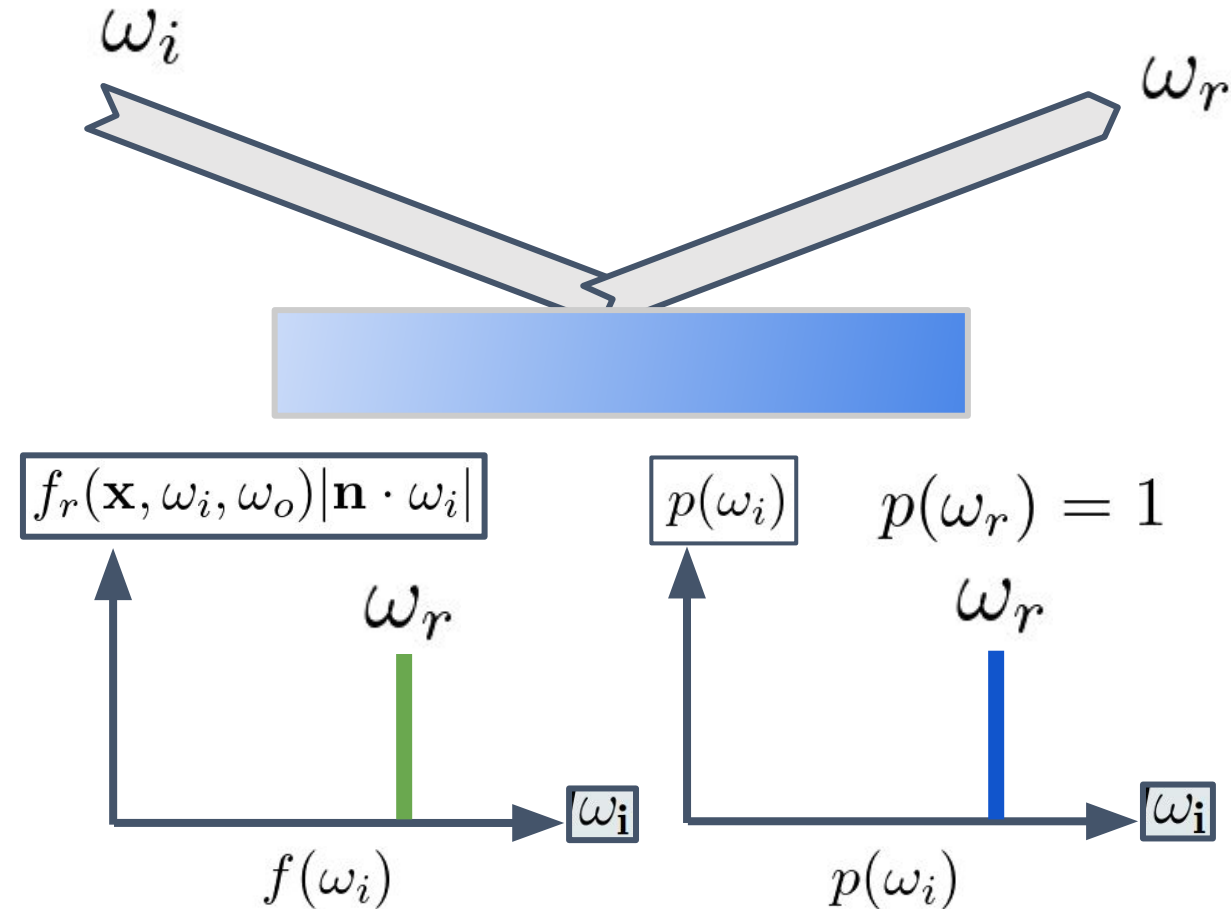
Diffuse and specular BRDFs

Diffuse material



Uniform cosine sampling is good

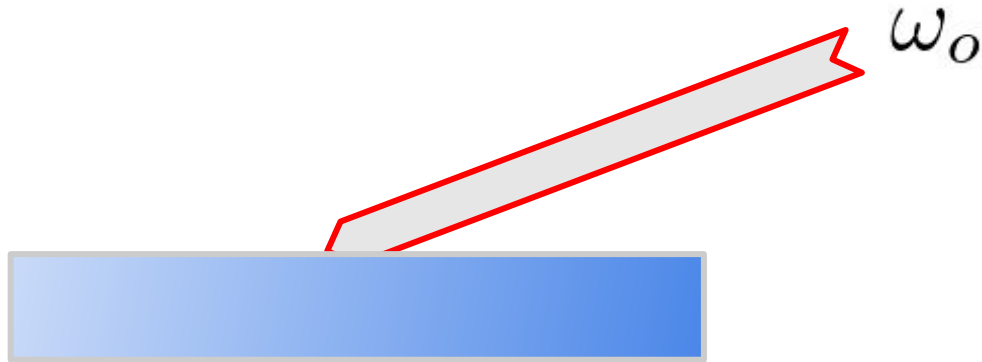
Perfect specular material



Deterministic sampling (not Monte Carlo)

Perfect specular (delta BRDF) sampling

- **Remember:** paths go from the camera to the light source

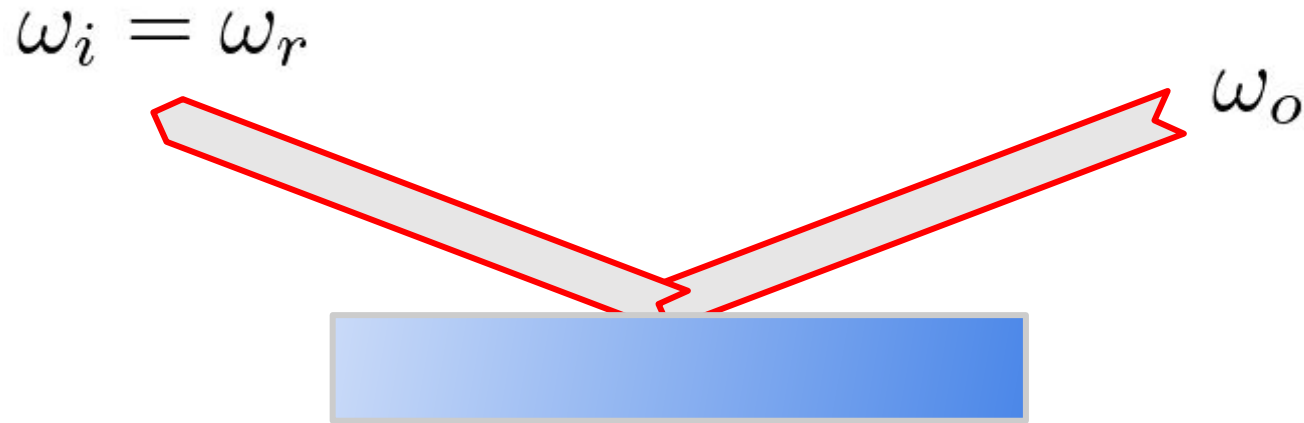


$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_r}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

Always sample the perfect specular direction ω_r

Perfect specular (delta BRDF) sampling

- **Remember:** paths go from the camera to the light source



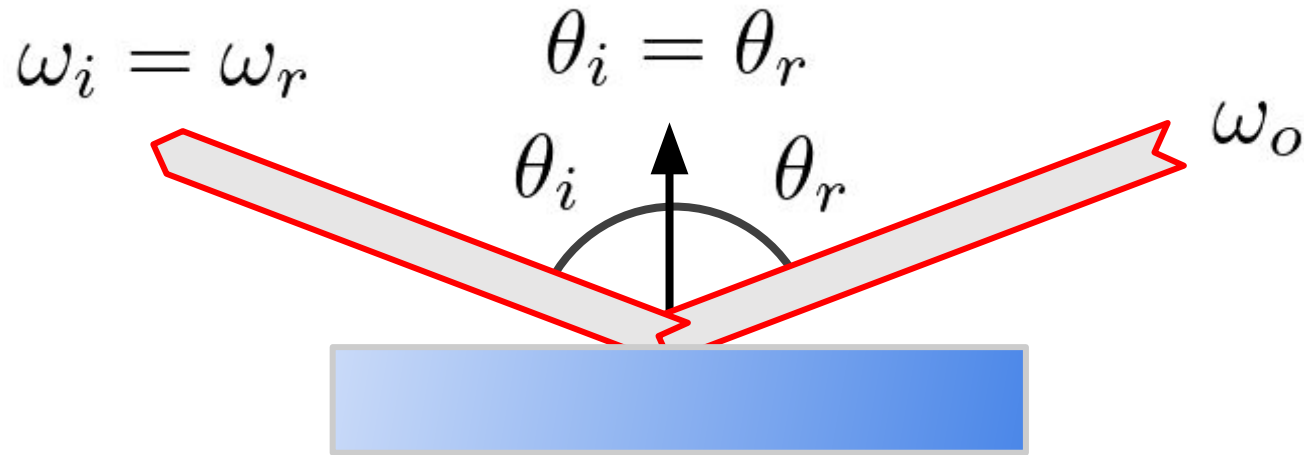
$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_r}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

Always sample the perfect specular direction ω_r

$$\omega_i = \omega_r = \omega_o - 2\mathbf{n}(\omega_o \cdot \mathbf{n})$$

Perfect specular (delta BRDF) sampling

- **Remember:** paths go from the camera to the light source



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_r}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

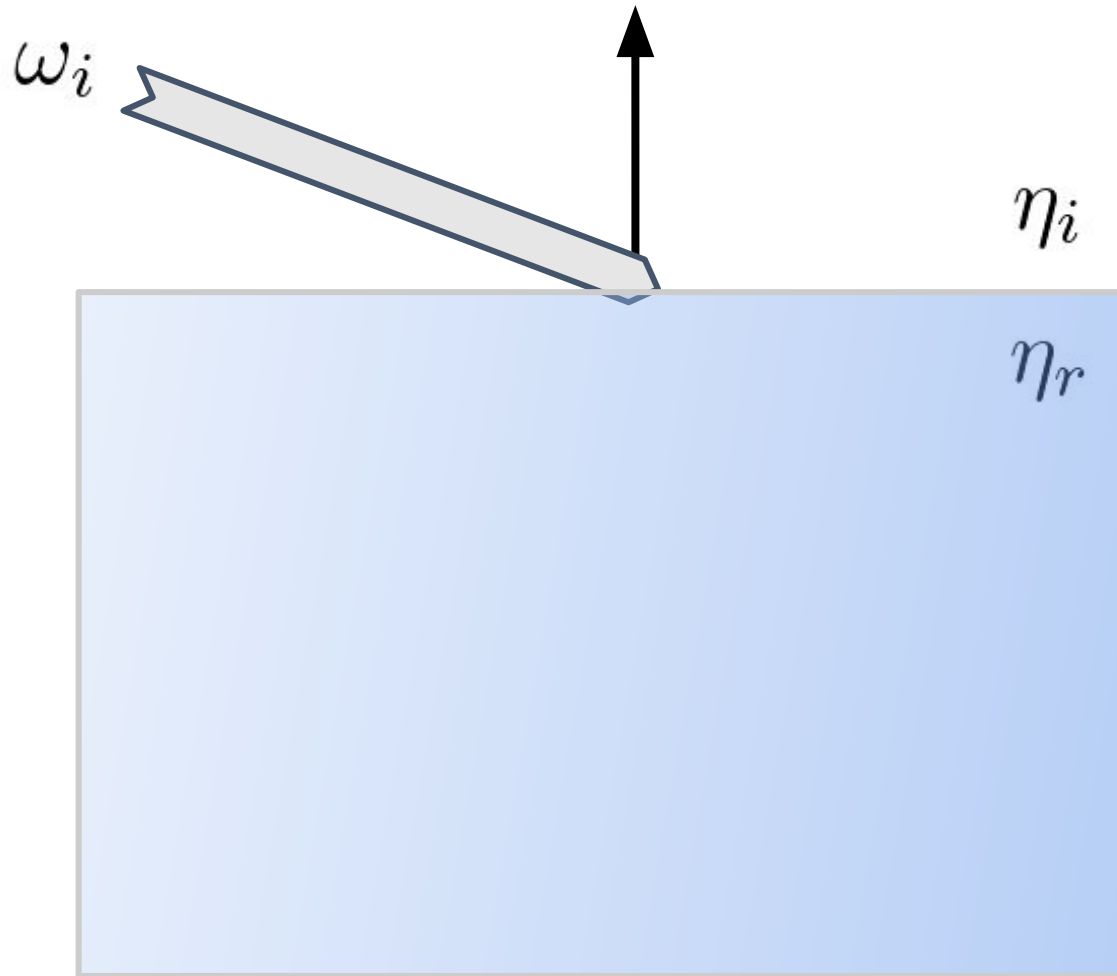
Always sample the perfect specular direction ω_r

$$\omega_i = \omega_r = \omega_o - 2\mathbf{n}(\omega_o \cdot \mathbf{n})$$

$$p(\omega_r) = 1$$

Perfect refraction (delta BTDF) sampling

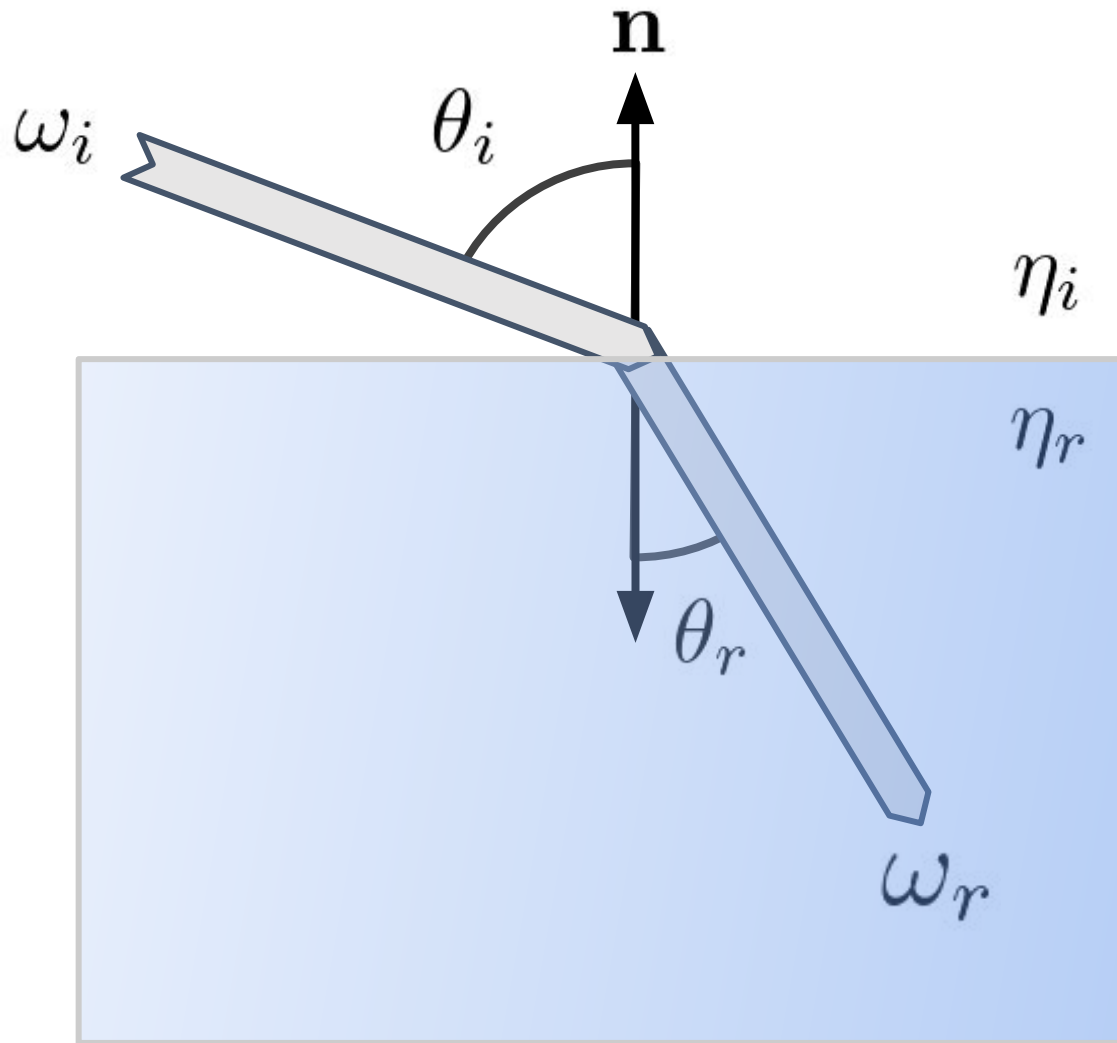
- The material does not **reflect** light, instead it (perfectly) **refracts** light



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_t}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

Perfect refraction (delta BTDF) sampling

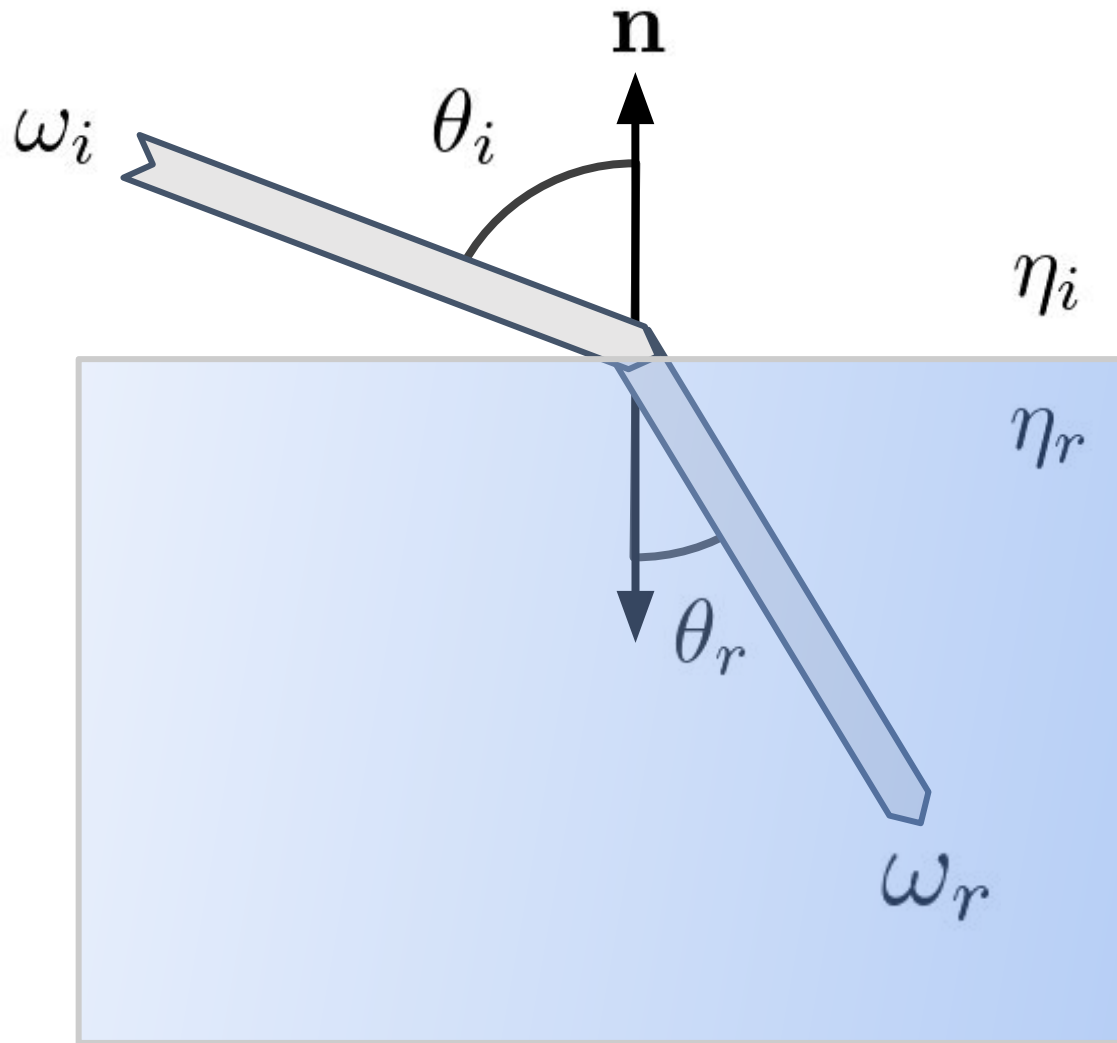
- The material does not **reflect** light, instead it (perfectly) **refracts** light



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_t}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

Perfect refraction (delta BTDF) sampling

- The material does not **reflect** light, instead it (perfectly) **refracts** light



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_t}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

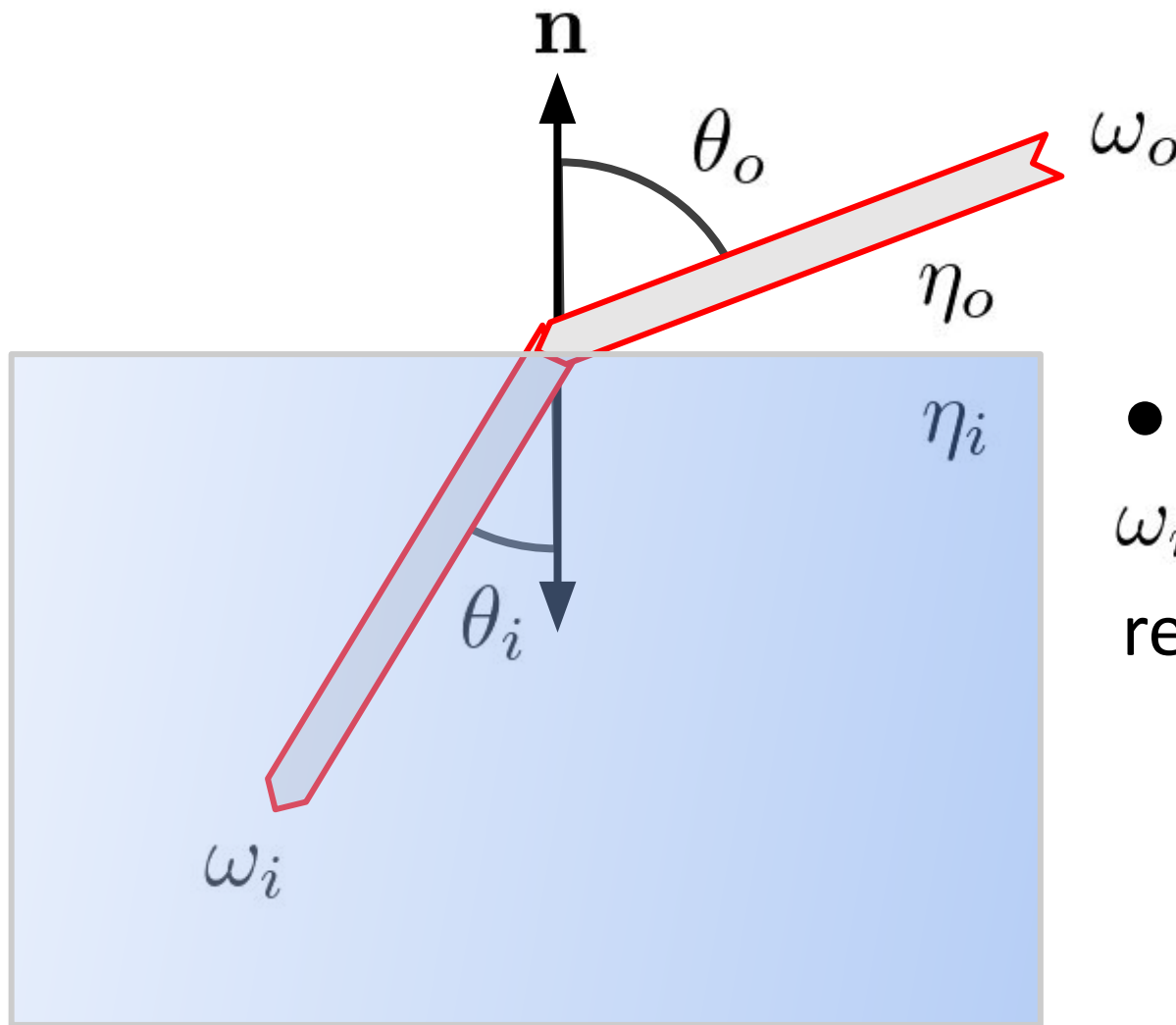
- **Snell's law:**

ω_r depends on the index of refraction η_i , η_r of the two media

$$\eta_i \sin \theta_i = \eta_r \sin \theta_r$$

Perfect refraction (delta BTDF) sampling

- The material does not **reflect** light, instead it (perfectly) **refracts** light



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_t}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

- Snell's law:**

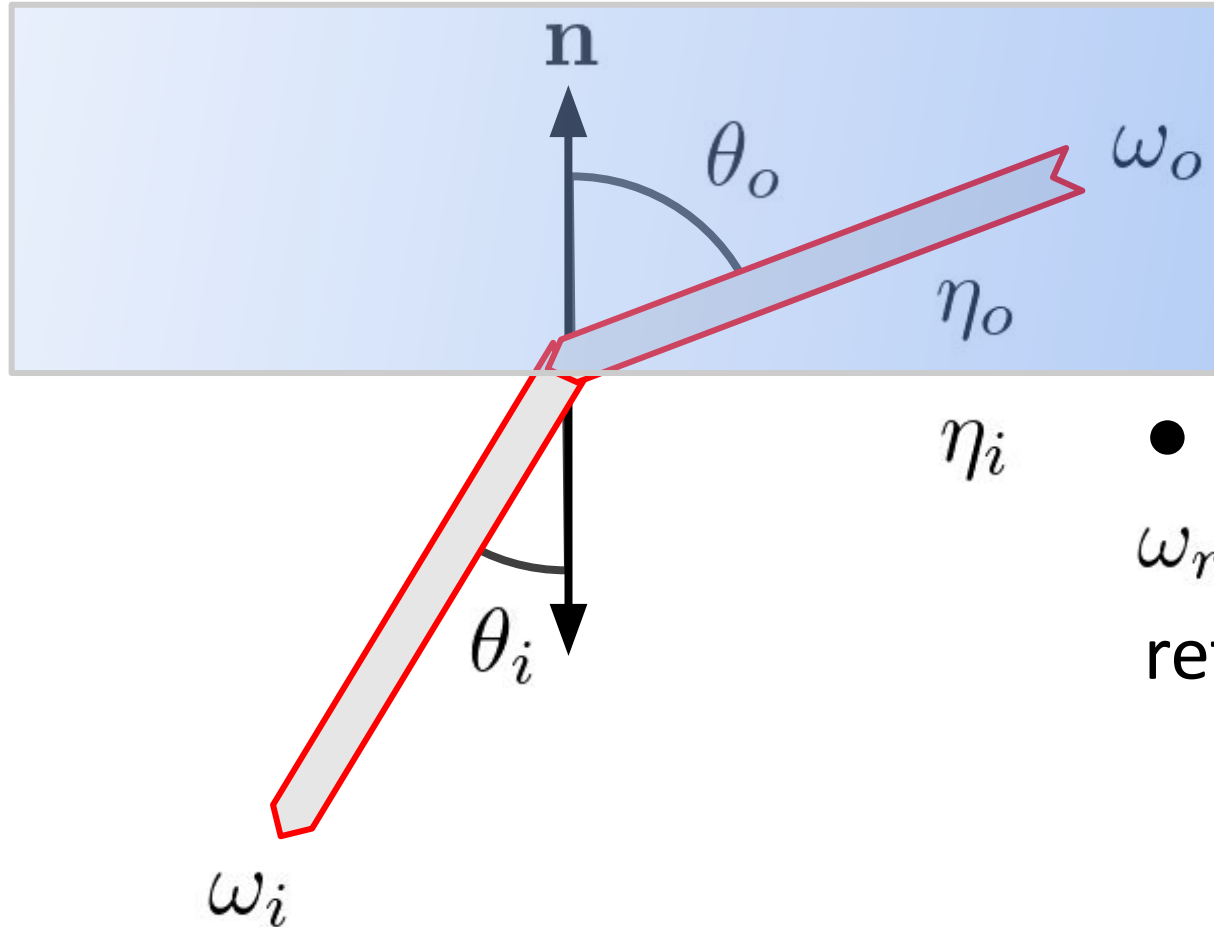
ω_r depends on the index of refraction η_i, η_r of the two media

$$\sin \theta_i = \sin \theta_r = \frac{\eta_o}{\eta_i} \sin \theta_o$$

$$p(\omega_r) = 1$$

Perfect refraction (delta BTDF) sampling

- The material does not **reflect** light, instead it (perfectly) **refracts** light



$$f_r(\mathbf{x}, \omega_i, \omega_o) = \frac{\delta_{\omega_t}(\omega_i)}{(\omega_i \cdot \mathbf{n})}$$

- Snell's law:**

ω_r depends on the index of refraction η_i, η_r of the two media

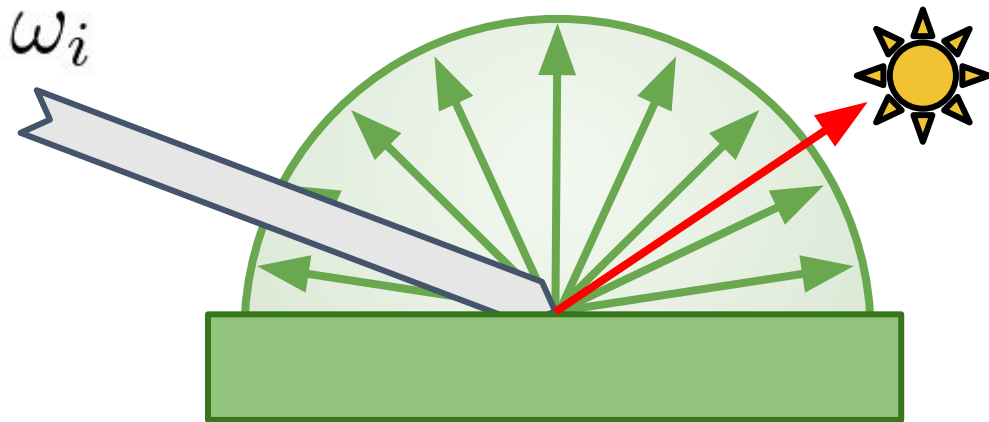
$$\sin \theta_i = \sin \theta_r = \frac{\eta_o}{\eta_i} \sin \theta_o$$

$$p(\omega_r) = 1$$

Next-event estimation and delta BRDF

Diffuse material

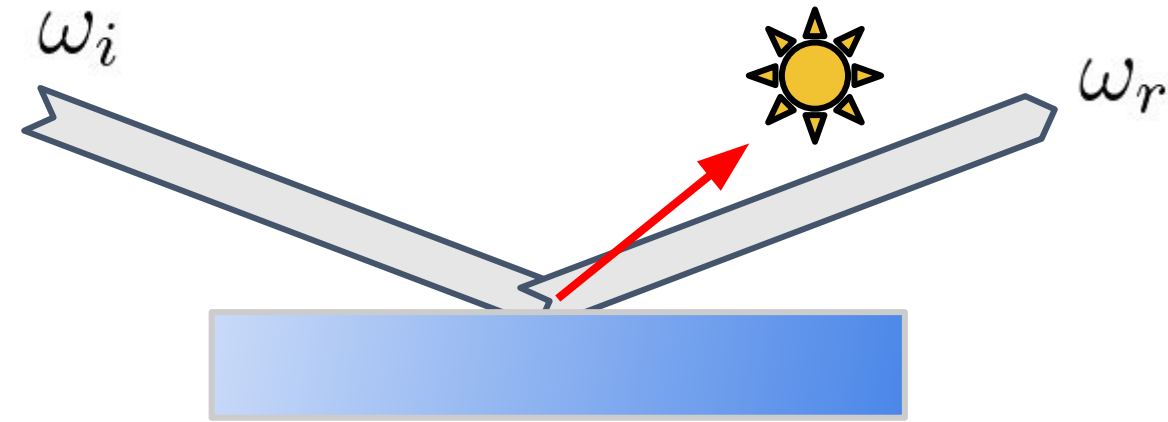
Light is reflected in all directions equally



$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| = \frac{k_d}{\pi} |\mathbf{n} \cdot \omega_i|$$

Perfect specular material

All light is (perfectly) reflected towards one direction ω_r



$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| = 0$$

**Next-event estimation will
always return 0**

The full BSDF

- Combining these three properties:

$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

- Each geometry primitive/material must have three BSDF coefficients:
 - k_d : Lambertian diffuse
 - k_s : Perfect specular reflectance (delta BRDF, law of reflection)
 - k_t : Perfect refraction (delta BTDF, Snell's law)

- Combining these three properties:

$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

- For physical correctness:
 - $k_d + k_s + k_t \leq 1$ on all channels (RGB)
- Recommended, not mandatory:
 - Emissive objects do not reflect/refract, and vice versa
 - Area lights with emission coefficient ($k_e > 0$)

Material coefficients

- Each primitive/material must have four coefficients in total:

- k_d lambertian diffuse

- k_s perfect specular

- k_t perfect refraction

- k_e emission coefficient

$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

$$L_e(\mathbf{x}, \omega_o)$$

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Material coefficients

- Each primitive/material must have four coefficients in total:

- k_d lambertian diffuse

- k_s perfect specular

- k_t perfect refraction

- k_e emission coefficient

$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

$$L_e(\mathbf{x}, \omega_o)$$

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Combining coefficients

- You can combine coefficients to get different materials

Diffuse



$$k_d > 0$$

Combining coefficients

- You can combine coefficients to get different materials

Diffuse



$$k_d > 0$$

Specular



$$k_s > 0$$

Combining coefficients

- You can combine coefficients to get different materials

Diffuse



$$k_d > 0$$

Specular



$$k_s > 0$$

Plastic



$$k_d, k_s > 0$$

Combining coefficients

- How do we sample w_i when there is more than one strategy?

Diffuse



Uniform cosine
sampling

Specular



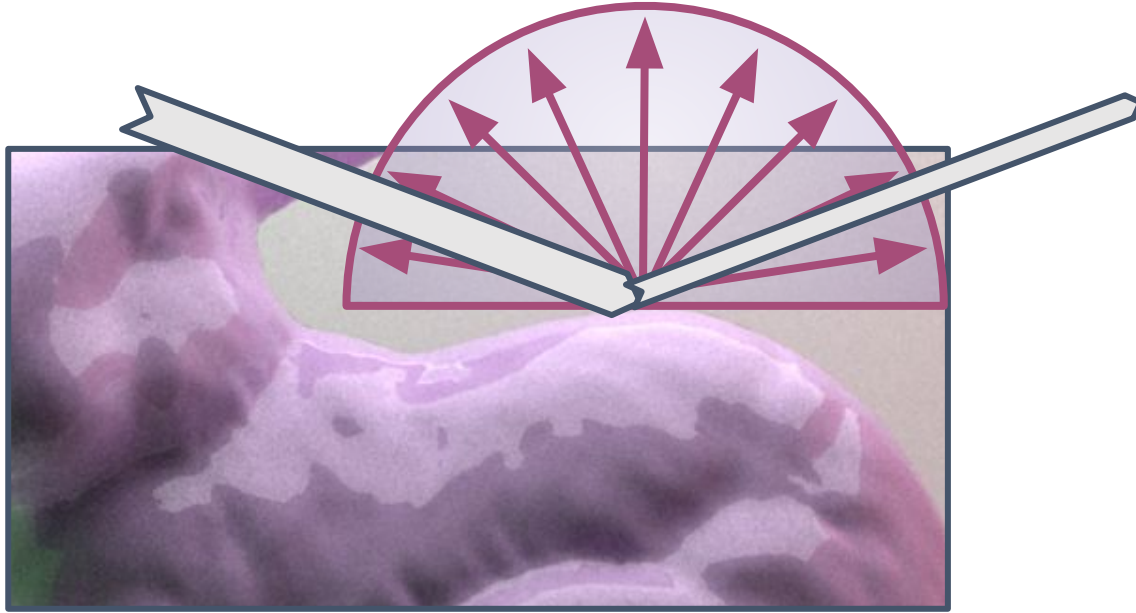
Deterministic
(delta BRDF)

Plastic

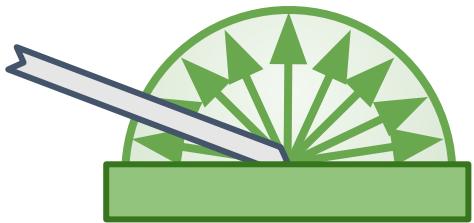


???

Russian Roulette: motivation

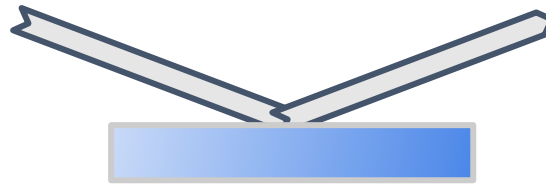


Plastic



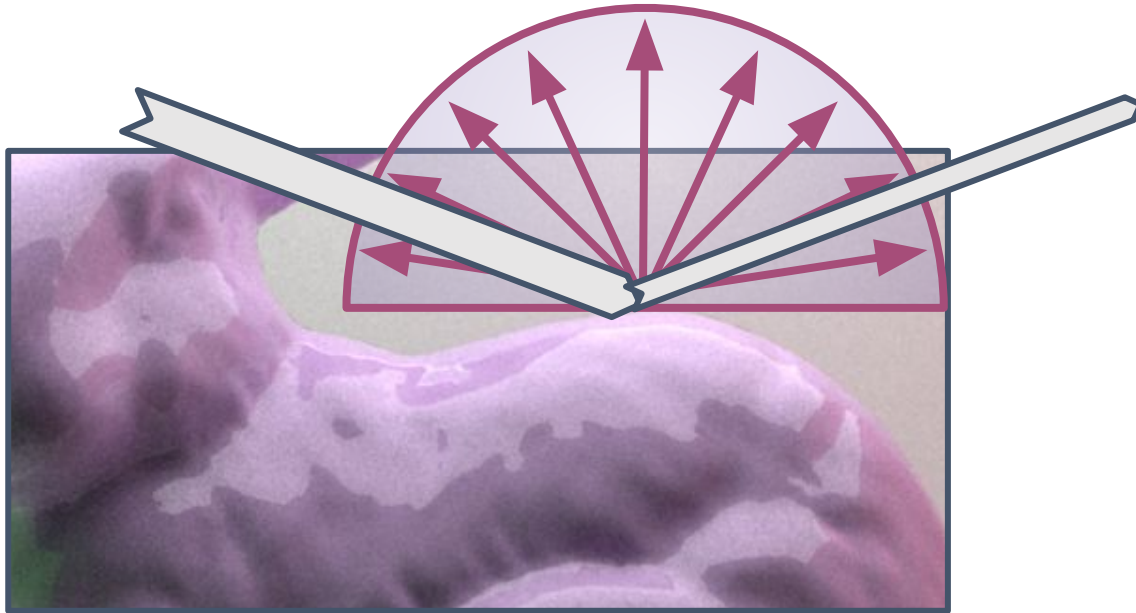
Diffuse

+



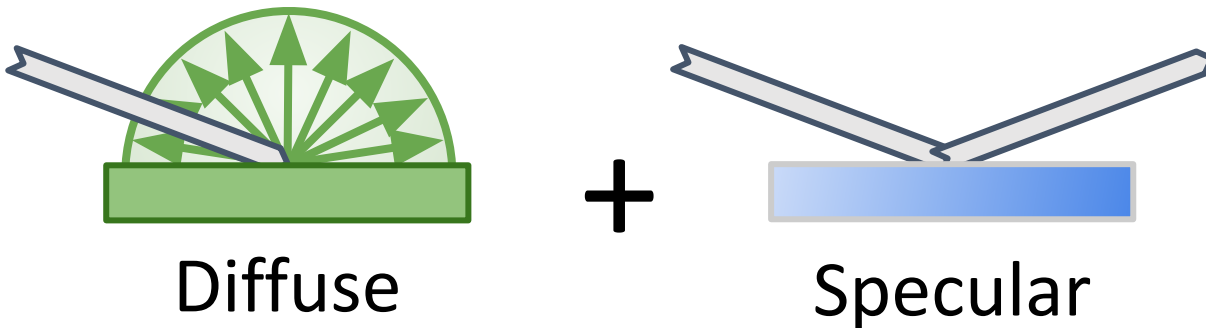
Specular

Russian Roulette: motivation

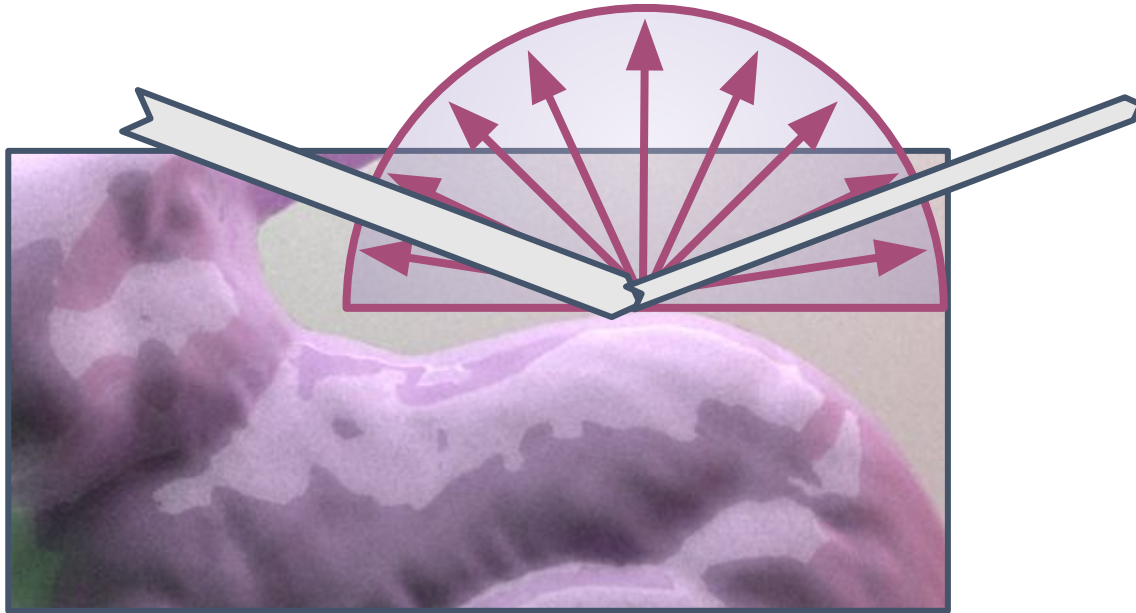


Plastic

- Only sample diffuse
 - Ignores specular part
- Only sample specular
 - Ignores diffuse part

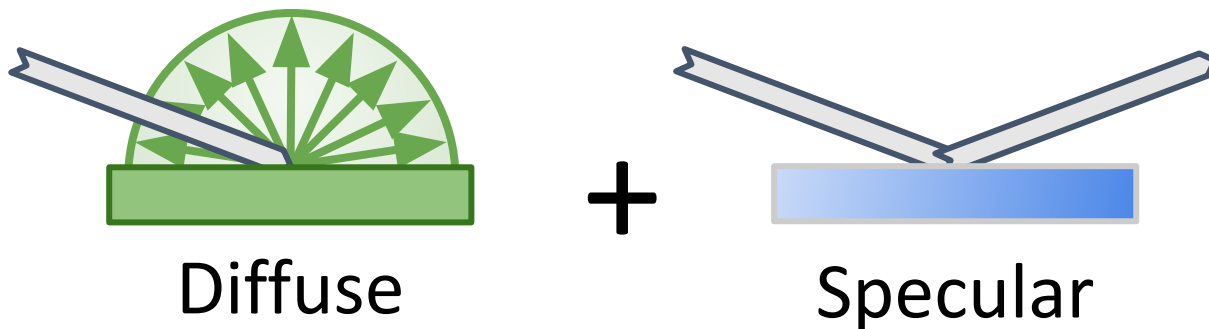


Russian Roulette: motivation

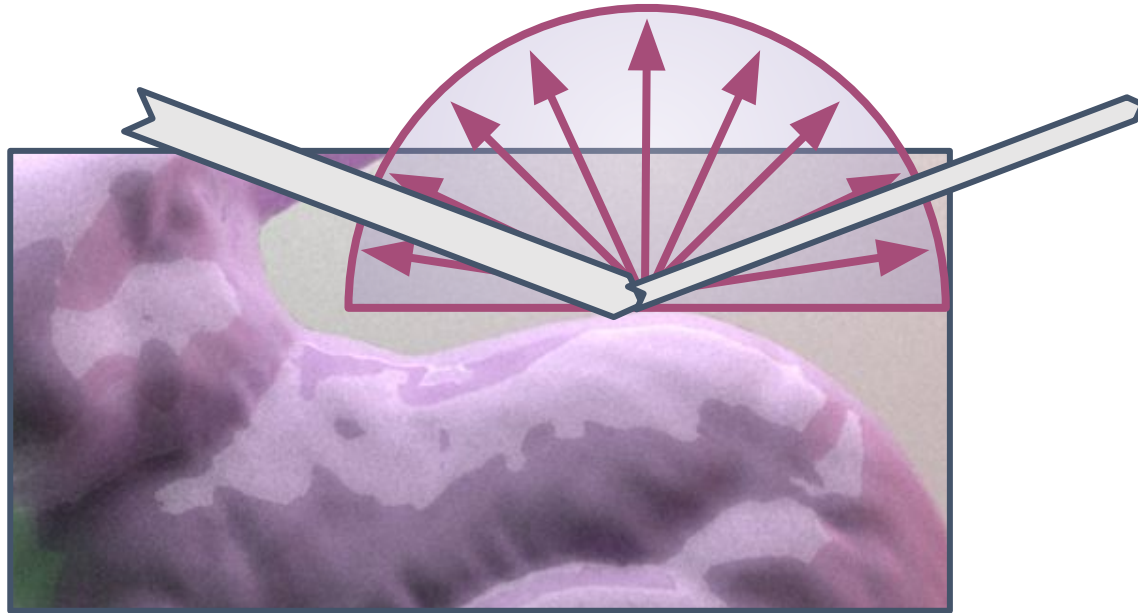


Plastic

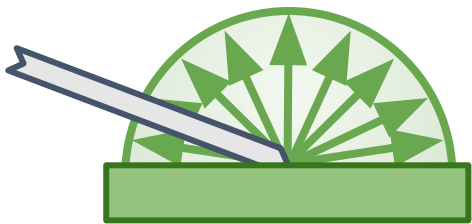
- ~~Only sample diffuse~~
 - Ignores specular part
- ~~Only sample specular~~
 - Ignores diffuse part
- Sample both at the same time with two rays
 - Grows exponentially



Russian Roulette: motivation

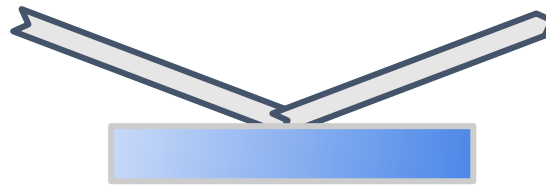


Plastic



Diffuse

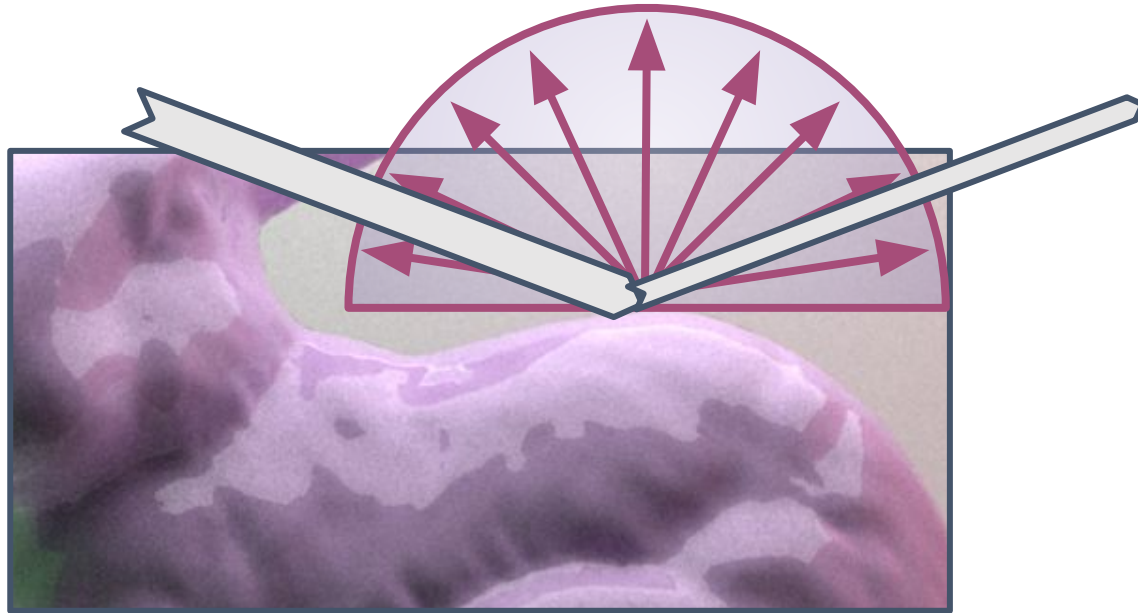
+



Specular

- ~~Only sample diffuse~~
 - Ignores specular part
- ~~Only sample specular~~
 - Ignores diffuse part
- ~~Sample both at the same time with two rays~~
 - Grows exponentially
- **Russian Roulette:** sample one random event (diffuse / specular)

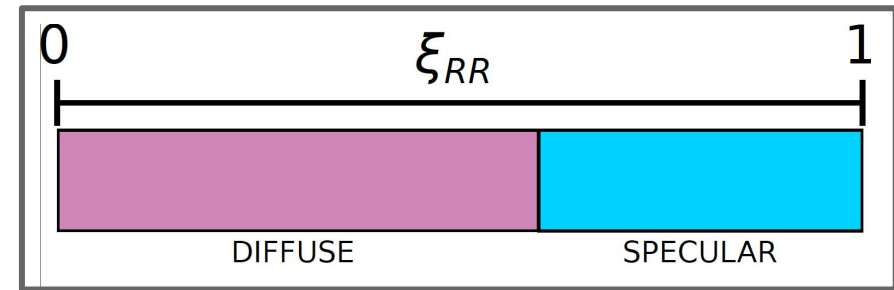
Russian Roulette: explanation



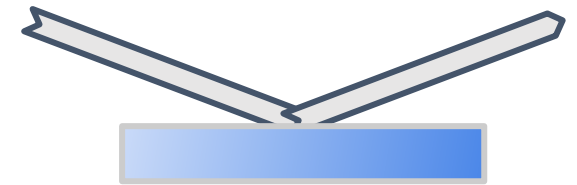
Plastic

- **Russian Roulette:** sample one random event (diffuse / specular)

Random number $\xi_{RR} \in [0, 1]$:

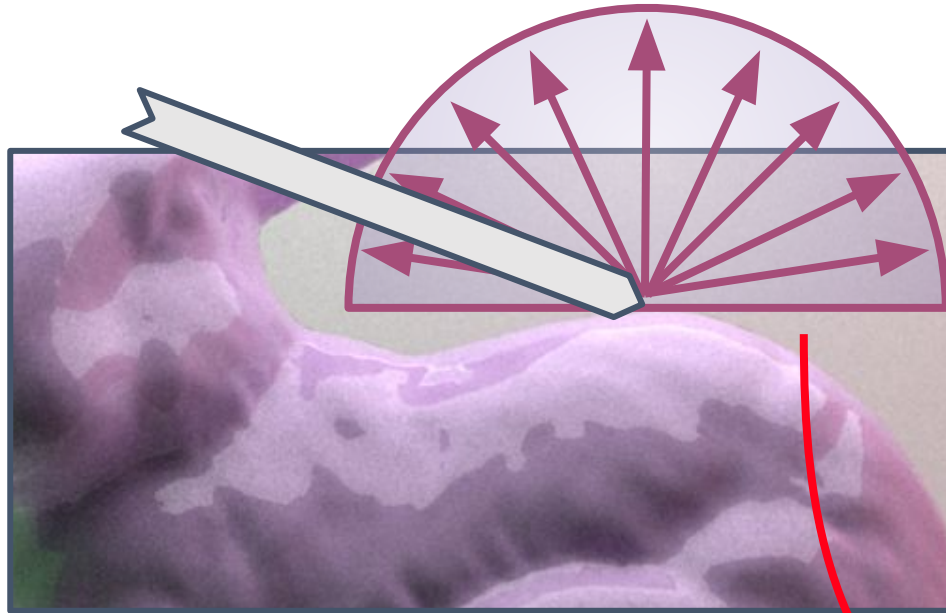


Diffuse



Specular

Russian Roulette: explanation

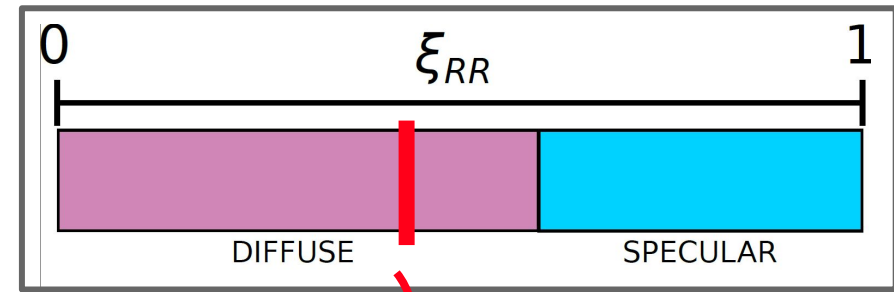


Plastic

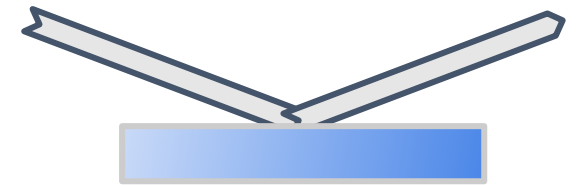
Sample ω_i based on your strategy for **diffuse** BRDFs

- **Russian Roulette:** sample one random event (diffuse / specular)

Random number $\xi_{RR} \in [0, 1]$:

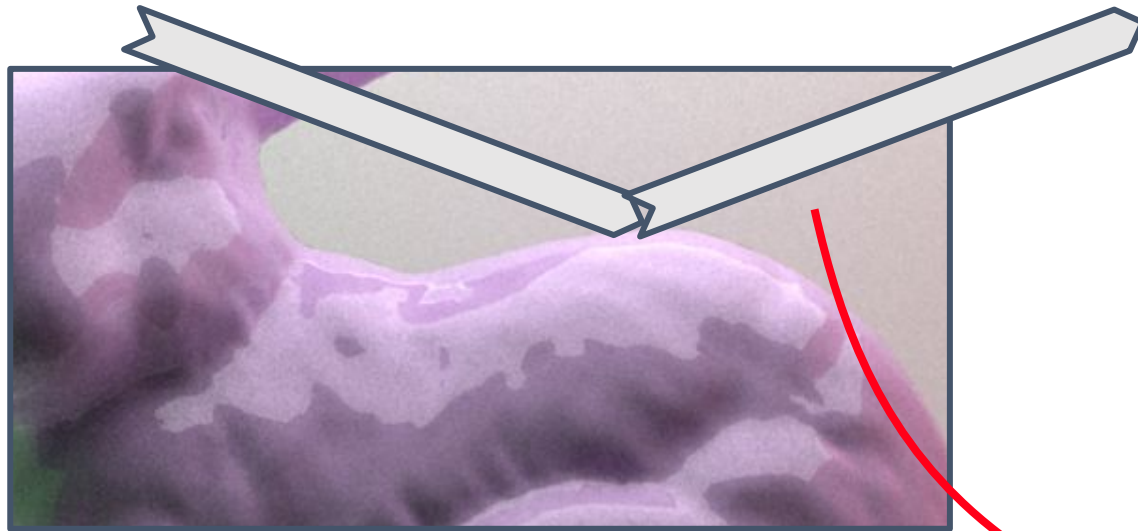


Diffuse



Specular

Russian Roulette: explanation

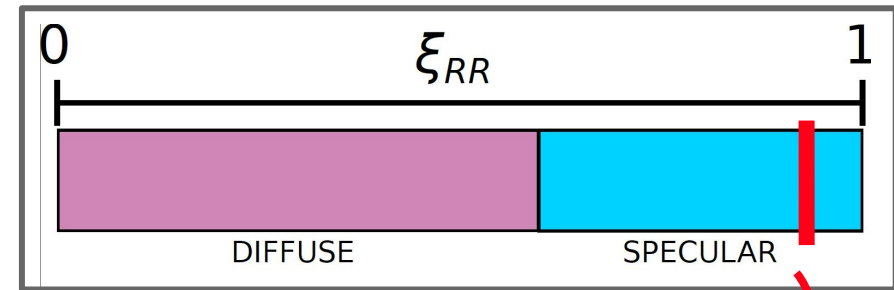


Plastic

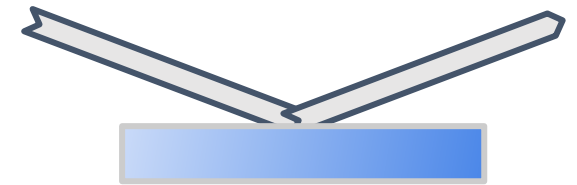
Sample ω_i based on your strategy for **specular** BRDFs

- **Russian Roulette:** sample one random event (diffuse / specular)

Random number $\xi_{RR} \in [0, 1]$:



Diffuse



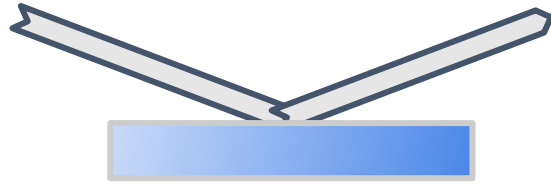
Specular

Russian Roulette: probabilities



Diffuse

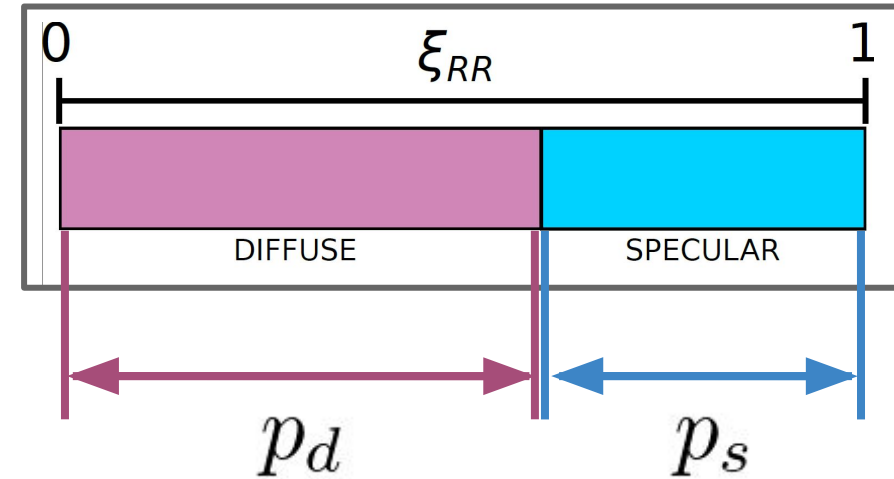
k_d



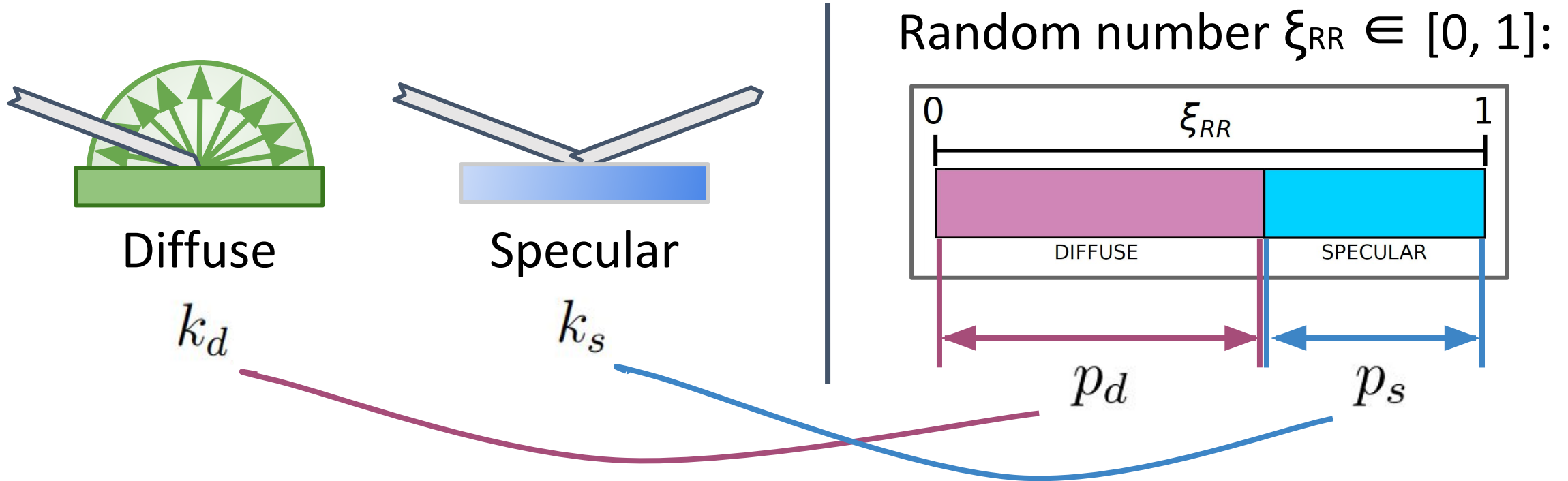
Specular

k_s

Random number $\xi_{RR} \in [0, 1]$:

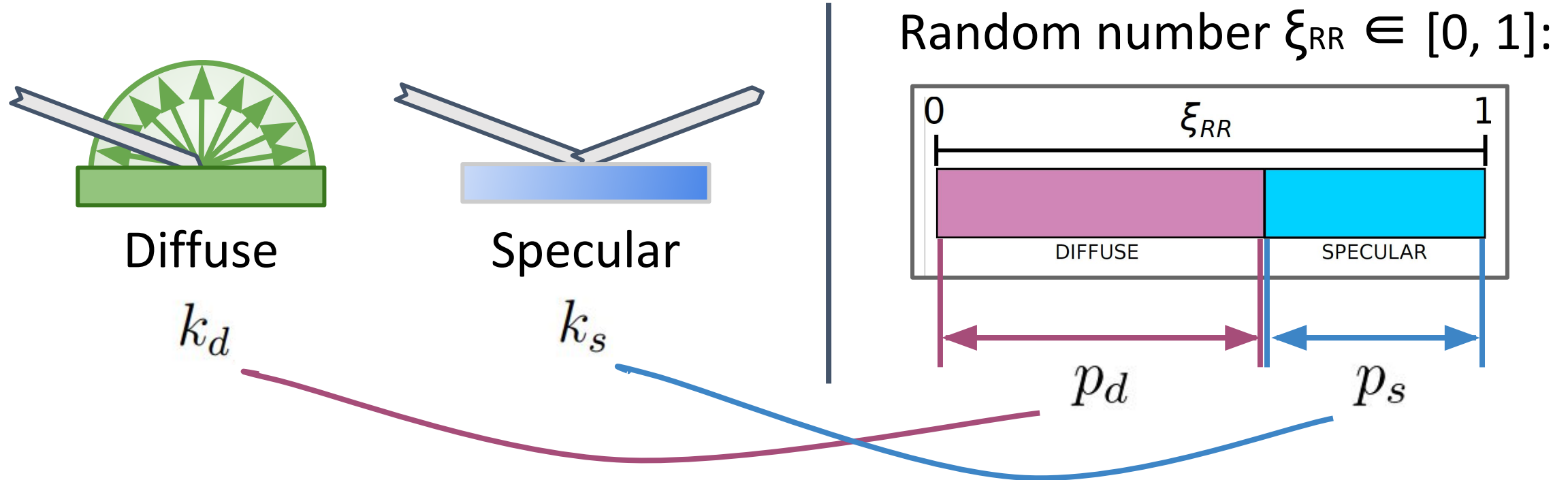


Russian Roulette: probabilities



- How to obtain probabilities p_d, p_s based on RGB coefficients k_d, k_s ?

Russian Roulette: probabilities



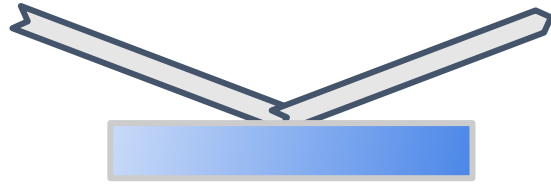
- How to obtain probabilities p_d, p_s based on RGB coefficients k_d, k_s ?
 - Example: $p_i = \max k_i$ (maximum of RGB channels)
 - Remember: $k_d + k_s + k_t \leq 1$ on all channels (RGB)

Russian Roulette: probabilities



Diffuse

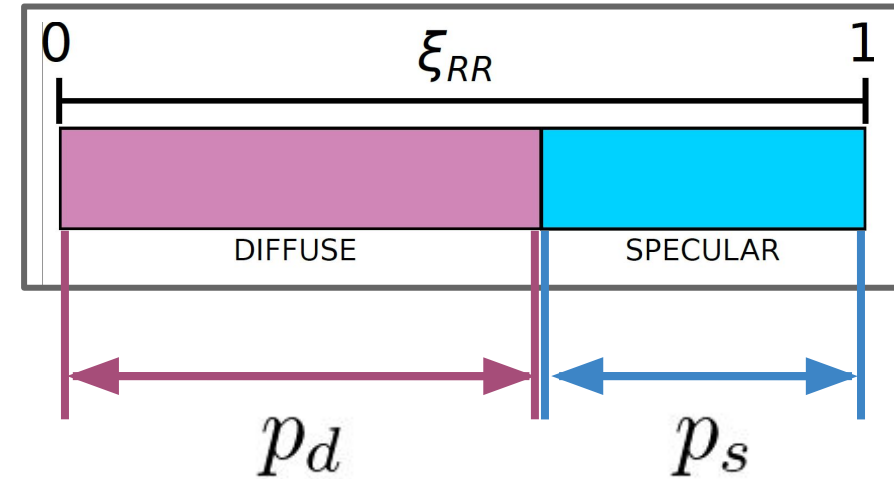
k_d



Specular

k_s

Random number $\xi_{RR} \in [0, 1]$:



- After evaluating each event, divide by its probability:

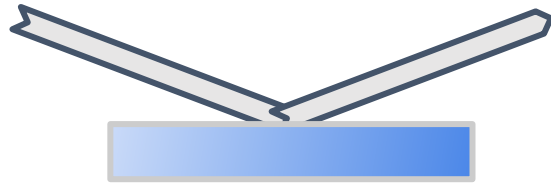
$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

Russian Roulette: probabilities



Diffuse

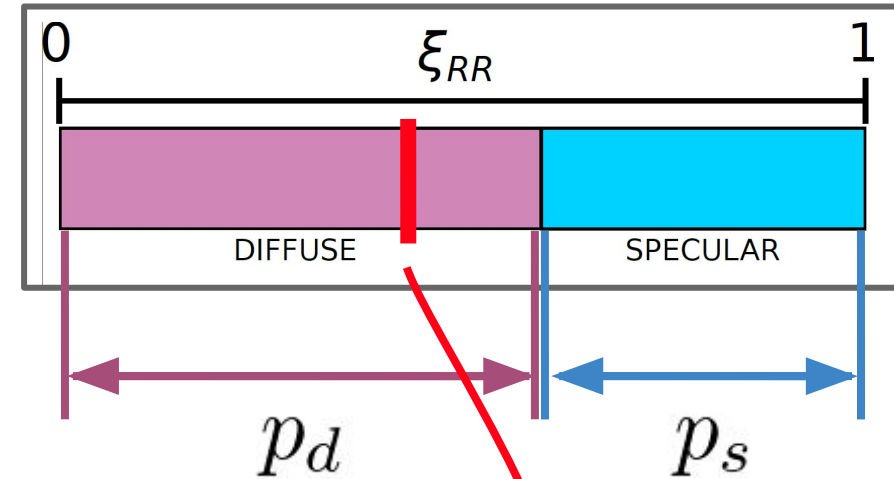
k_d



Specular

k_s

Random number $\xi_{RR} \in [0, 1]$:



- After evaluating each event, divide by its probability:

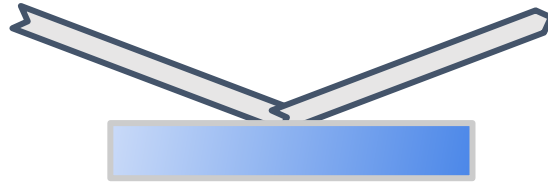
$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} \cdot \frac{1}{p_d}$$

Russian Roulette: probabilities



Diffuse

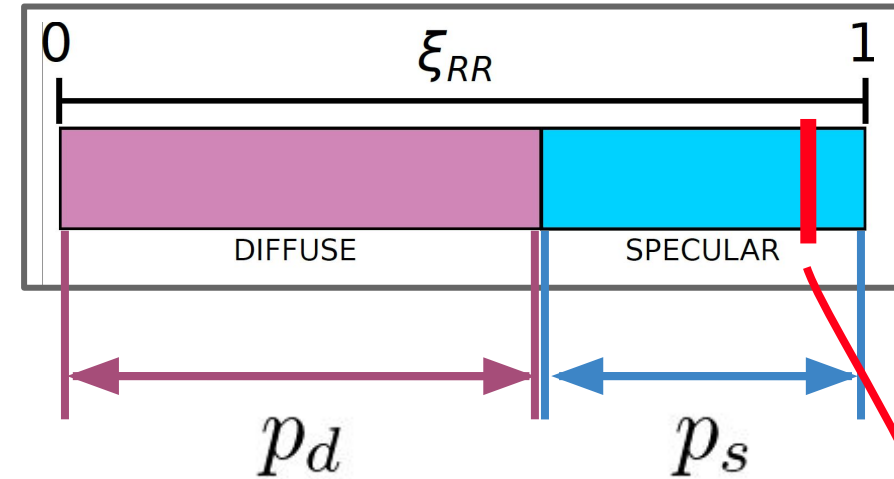
k_d



Specular

k_s

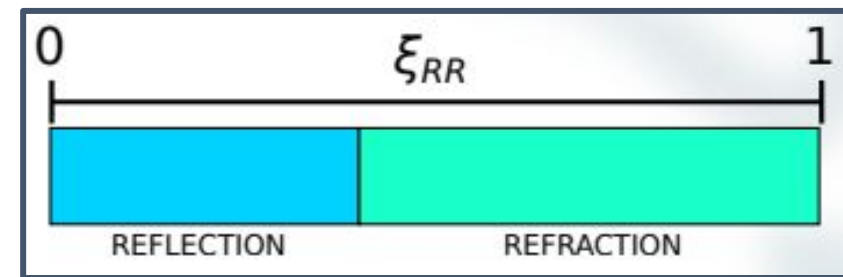
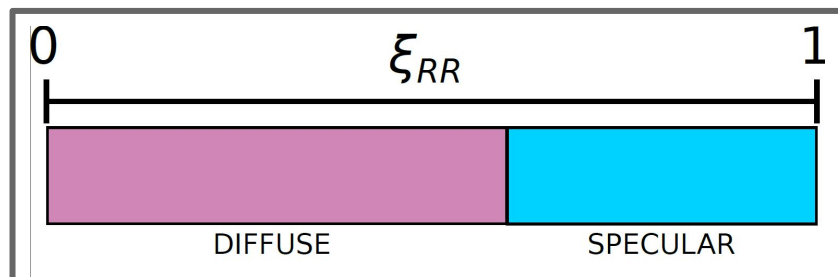
Random number $\xi_{RR} \in [0, 1]$:



- After evaluating each event, divide by its probability:

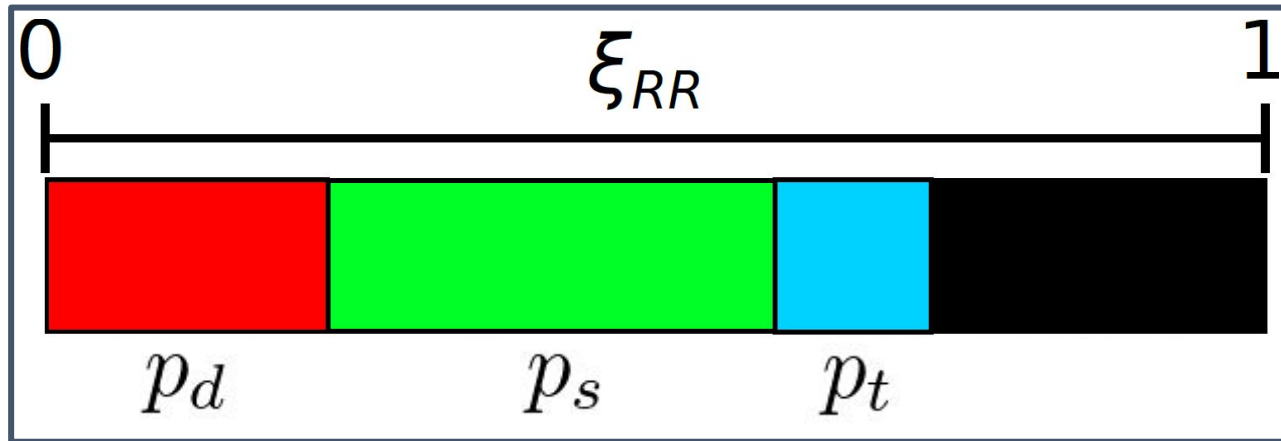
$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} \cdot \frac{1}{p_s}$$

Material combinations



Path termination

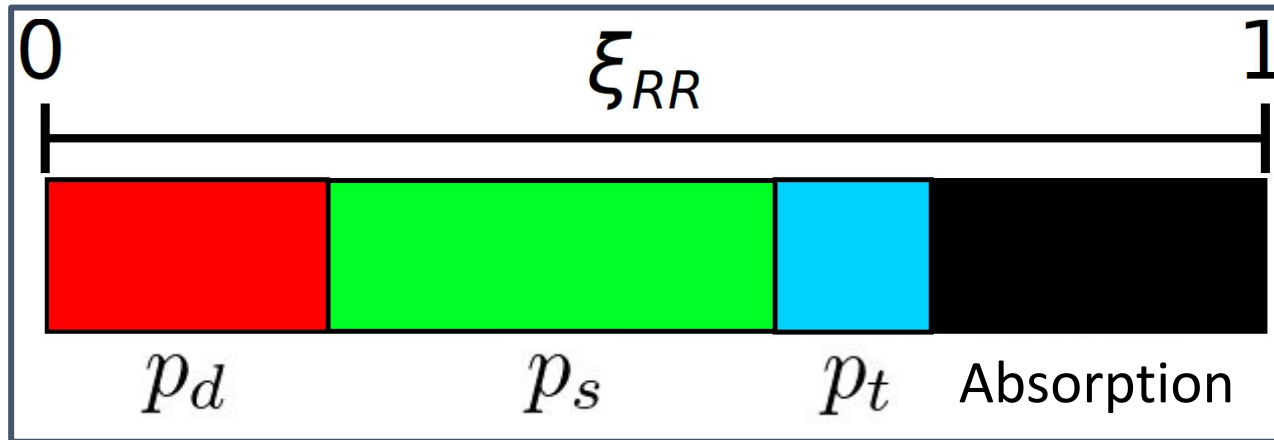
- Until now we have defined three events for Russian Roulette
 - Diffuse, perfect specular, perfect refractive



$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

Path termination

- Until now we have defined three events for Russian Roulette
 - Diffuse, perfect specular, perfect refractive
- New: “absorption” event as a path termination condition



$$1 - (p_d + p_s + p_t)$$

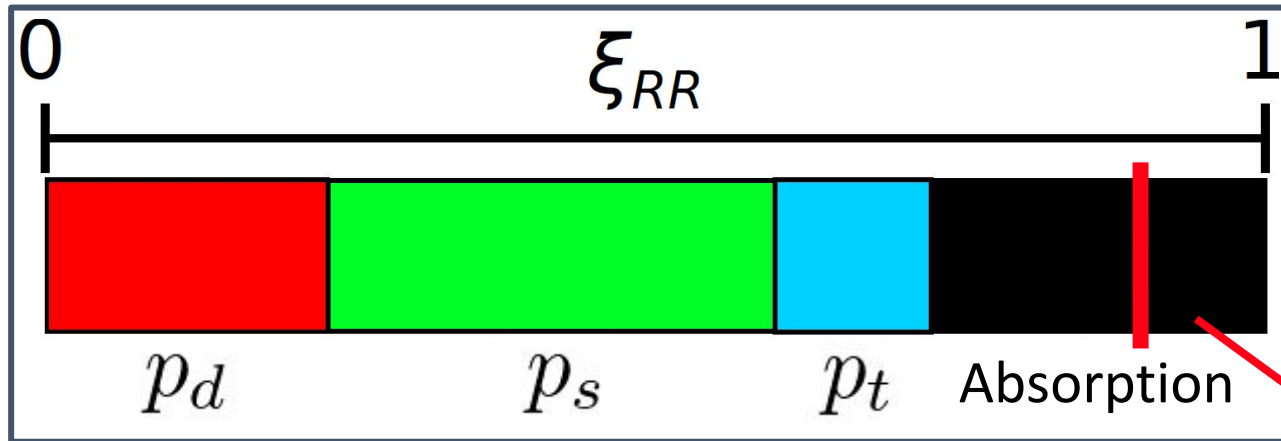
An arrow points from this equation to the black "Absorption" segment of the bar in the diagram above.

$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

Below the equation, there are three horizontal colored bars: a red bar under the first term, a green bar under the second term, and a blue bar under the third term.

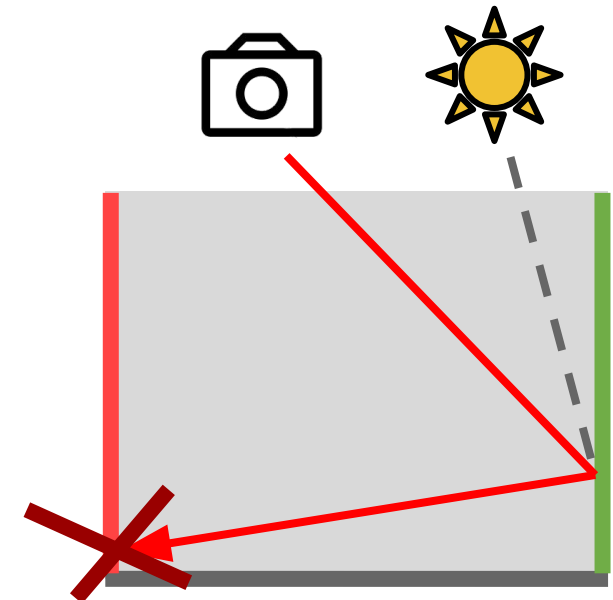
Path termination

- Until now we have defined three events for Russian Roulette
 - Diffuse, perfect specular, perfect refractive
- New: “absorption” event as a path termination condition



$$1 - (p_d + p_s + p_t)$$

$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

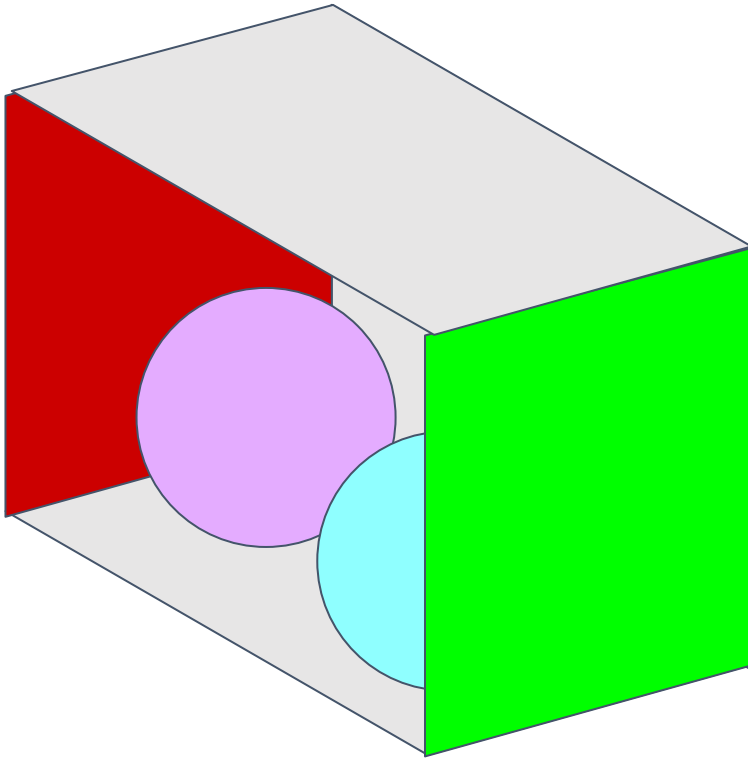


Path termination

- Until now we have defined three events for Russian Roulette
 - Diffuse, perfect specular, perfect refractive
- New: “absorption” event as a path termination condition
 - Remember previous termination path conditions:
 - Ray does not intersect
 - Ray intersects with an area light
 - Number of bounces $> N$ (not required with Russian Roulette)

Example scene: Cornell Box

- **Geometry**



Planes defined by normal (n) and distance (d)

Left plane $n = (1, 0, 0)$, $d = 1$

Right plane $n = (-1, 0, 0)$, $d = 1$

Floor plane $n = (0, 1, 0)$, $d = 1$

Ceiling plane $n = (0, -1, 0)$, $d = 1$

Back plane $n = (0, 0, -1)$, $d = 1$

Spheres defined by center (c) and radius (r)

Left sphere $c = (-0.5, -0.7, 0.25)$, $r = 0.3$

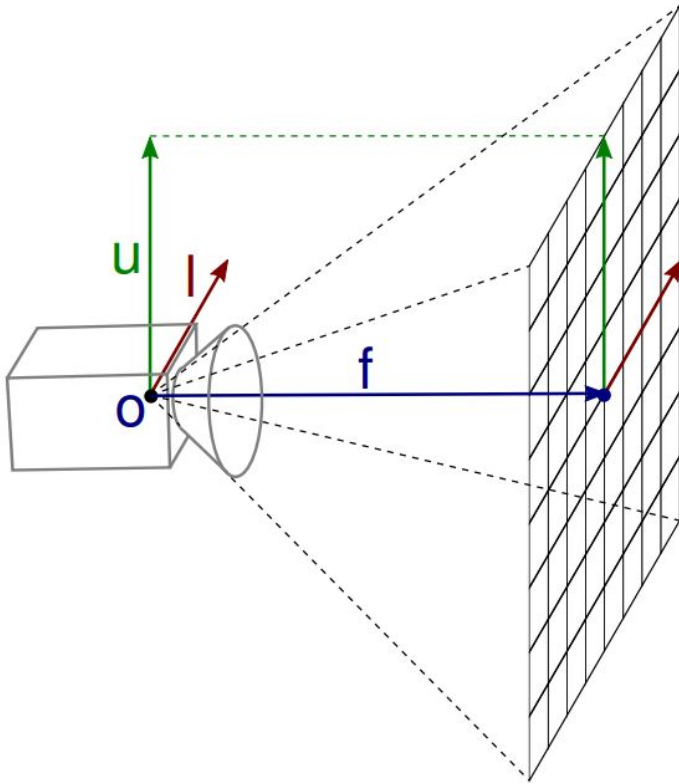
- Mix of blue diffuse + specular

Right sphere $c = (0.5, -0.7, -0.25)$, $r = 0.3$

- Mix of specular and refraction, $\eta = 1.5$

Example scene: Cornell Box

- Camera & light sources



Camera and image plane defined by

Origin $O = (0, 0, -3.5)$

Left $L = (-1, 0, 0)$

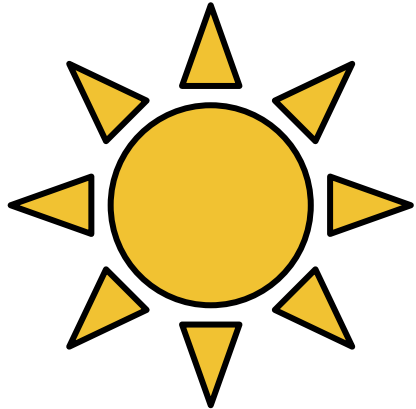
Up $U = (0, 1, 0)$

Forward $F = (0, 0, 3)$

Size 256x256 pixels

Example scene: Cornell Box

- Light sources



Center and power (emission)

Center $c = (0, 0.5, 0)$

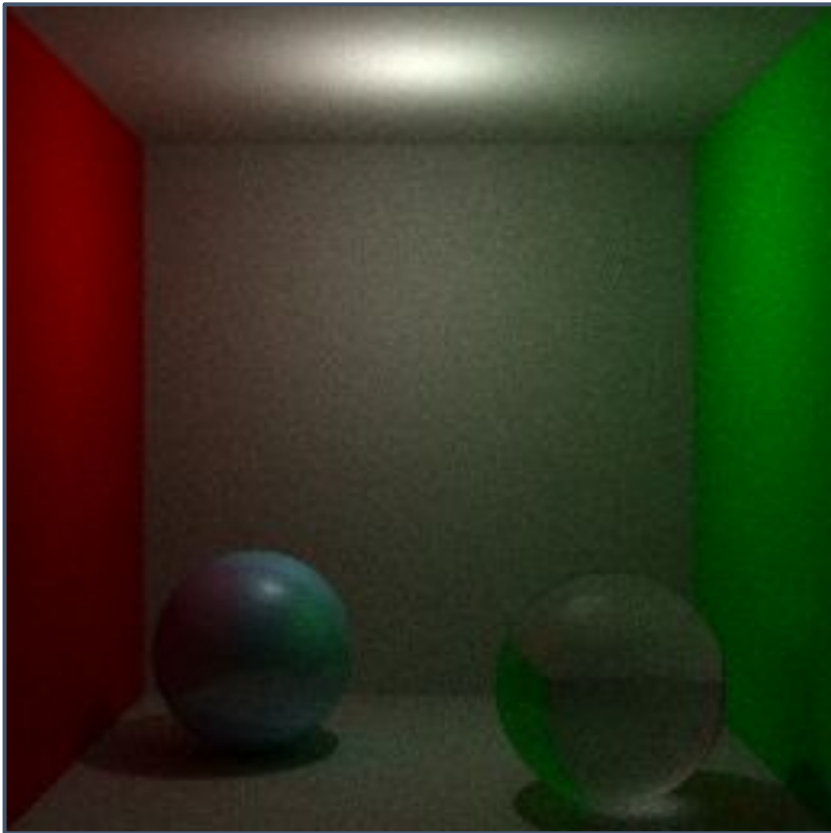
Power can be any number e.g. $p = (1, 1, 1)$

Just be careful with the #MAX

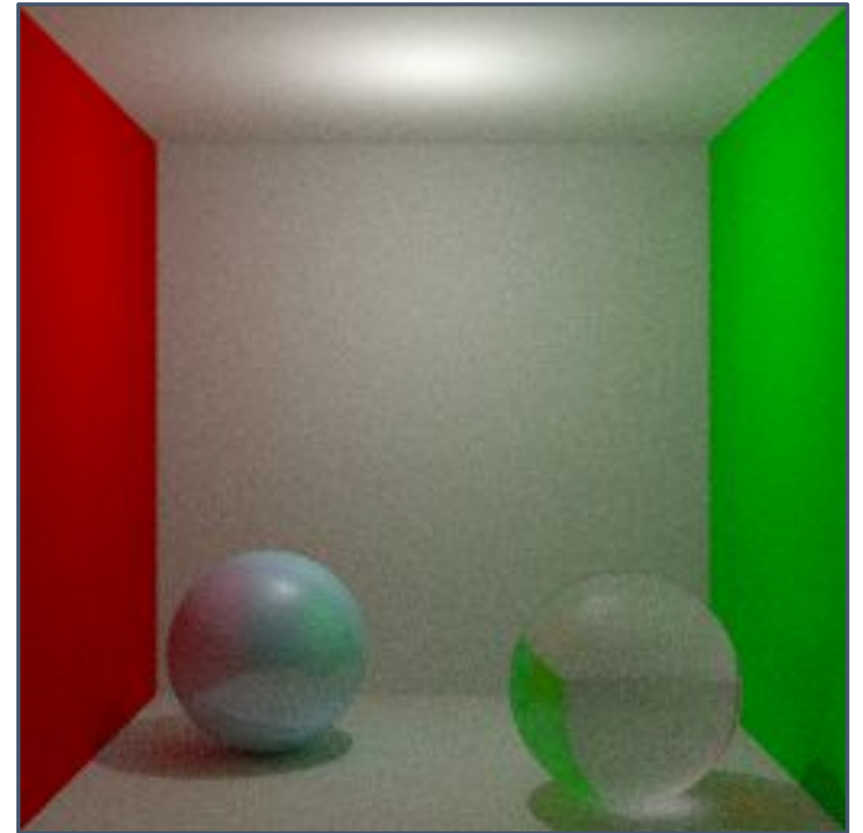
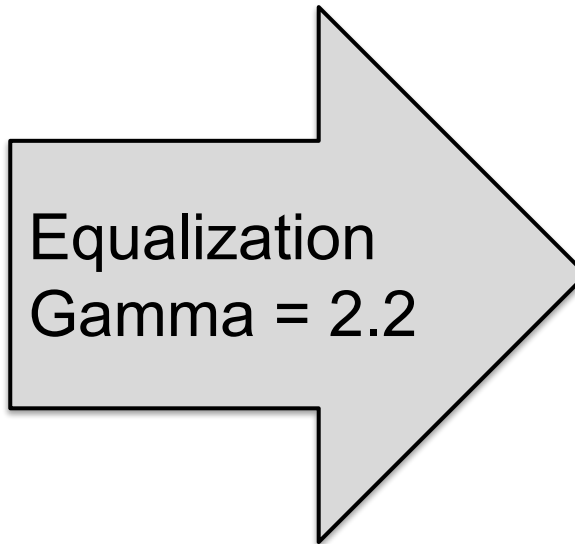
```
1  P3
2  # feep.ppm
3  #MAX=<maximum of your RGB memory values>
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

Example scene: Cornell Box

- Results (no area lights + point light)



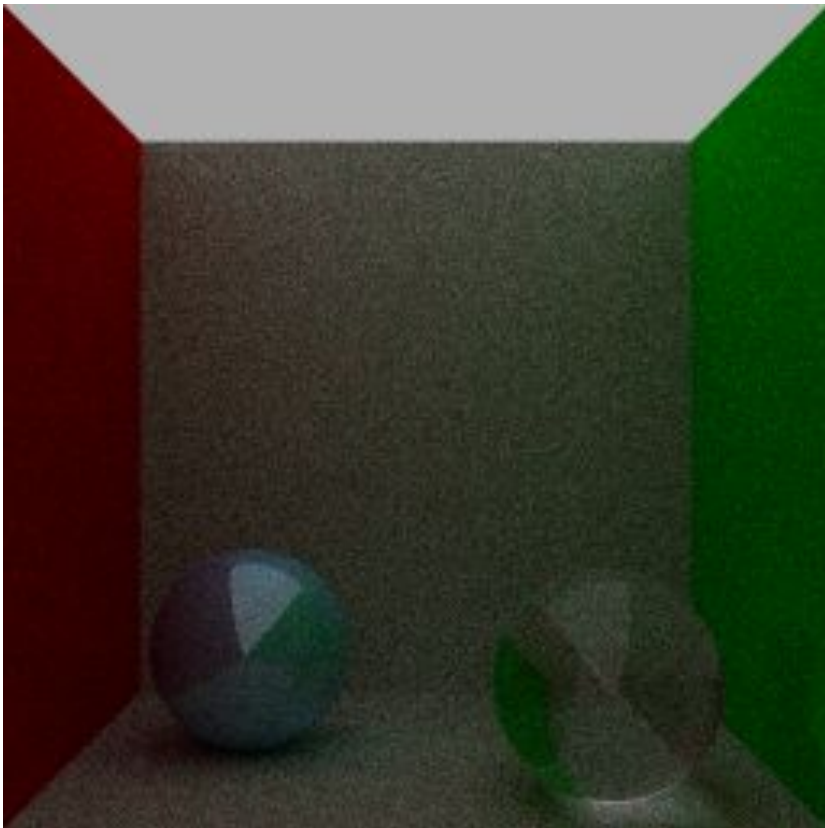
Using a point light



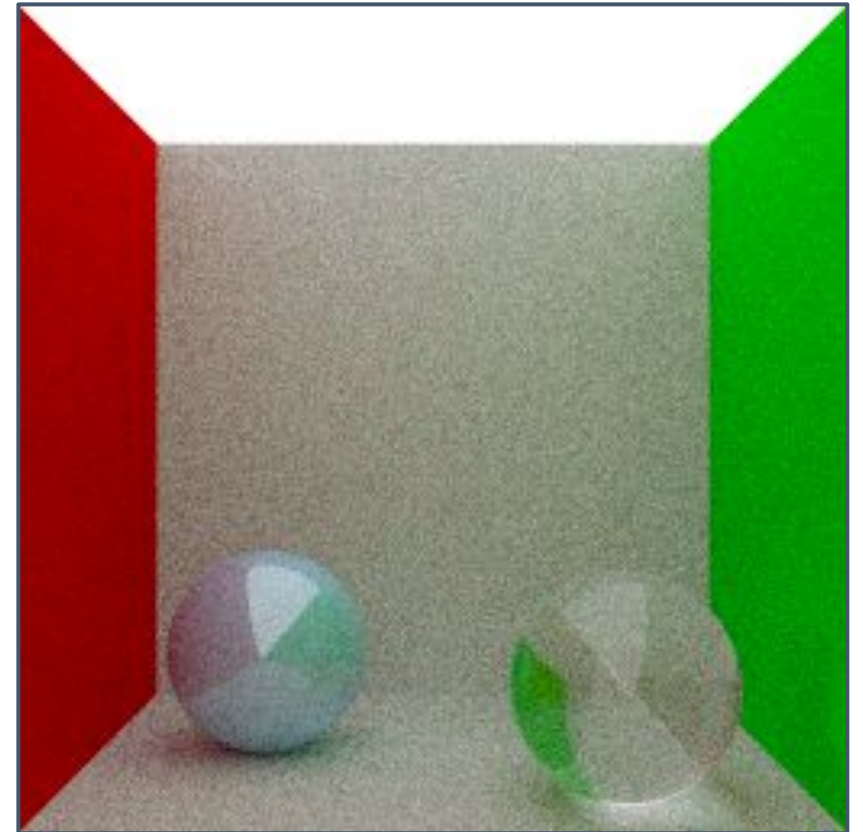
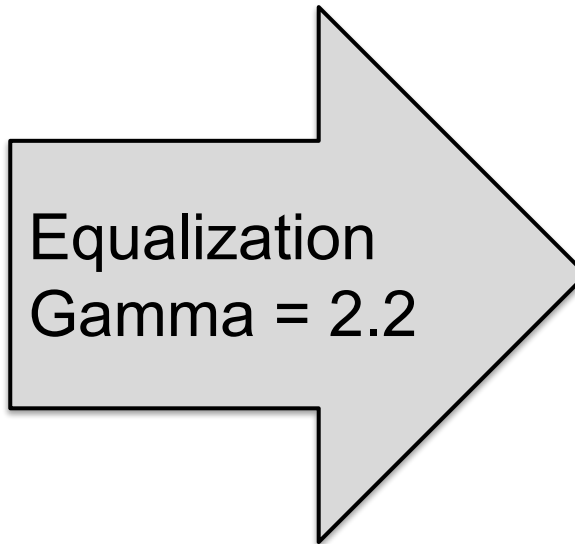
With tone mapping

Example scene: Cornell Box

- Results (no point light + ceiling plane is an area light)



Using an area light



With tone mapping

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice implement:

- Perfect specular and refractive BSDFs (add k_s and k_t for each material)
 - Extend “**evaluate**” and “**sample**” functions
- Apply Russian Roulette **at each intersection** to select an event:
 - (1) Diffuse, (2) Perfect specular, (3) Perfect refractive, (4) Absorption
 - Sample BSDF based on selected event, evaluate and divide by event’s probability
- Recommended deadline: November 13th (moodle: January 11th)
 - Extensions (do not count towards recommended deadline):
 - **Recommended to finish base path tracer for next week before any optionals**
 - **Realistic coefficients:** Modulate specular/refractive coefficients k_s and k_t using Fresnel equations
 - **Textures:** make material coefficients on hit position $\mathbf{x}$
 - **Fresnel effects:** make material coefficients depend on viewing direction ω_o

Lab #5 – Photon mapping (part 1)

Informática Gráfica

Adolfo Muñoz - Julio Marco

Pablo Luesia - J. Daniel Subías – Óscar Pueyo

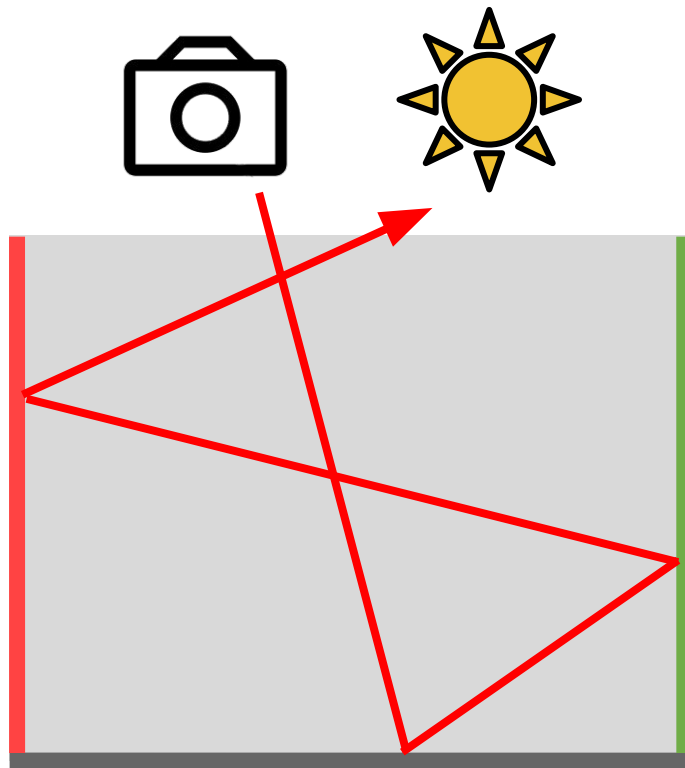


Before we begin...

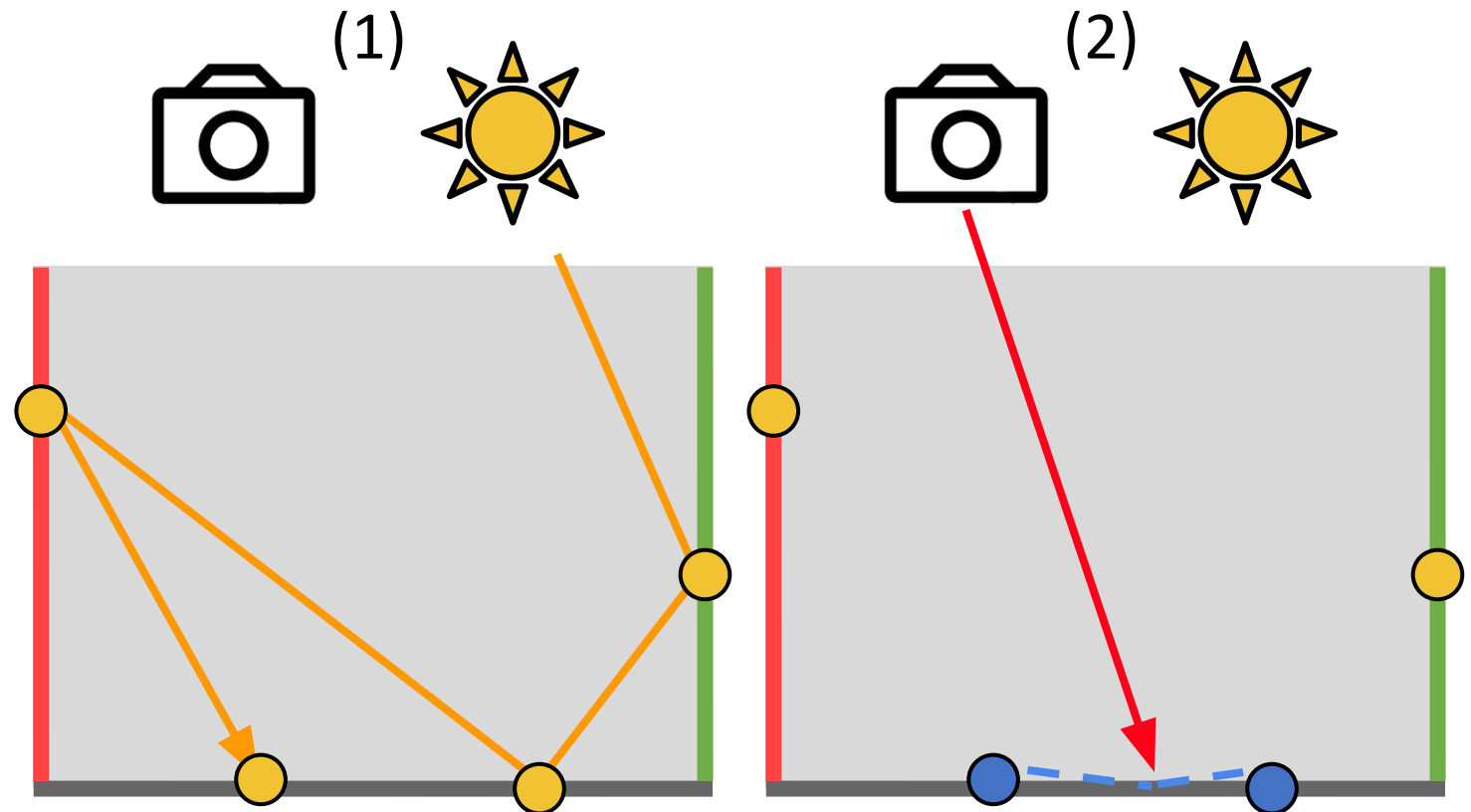
- Lab 5 (photon mapping) **is the second submitted work**
 - Recommended deadline: December 4th
 - Moodle: January 11th
- You can probably **reuse most of your code** for this assignment
- Remember: Final work is 80% of the final grade

Photon mapping basics

- Photon mapping is a **two-pass** algorithm
 - Uses an **intermediate storage** known as the **photon map**



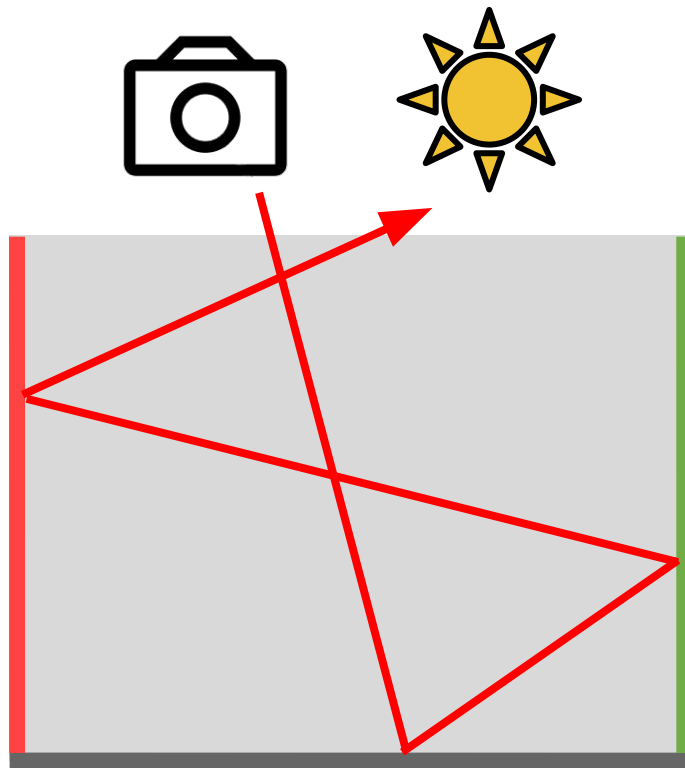
Lab 4 (path tracing)



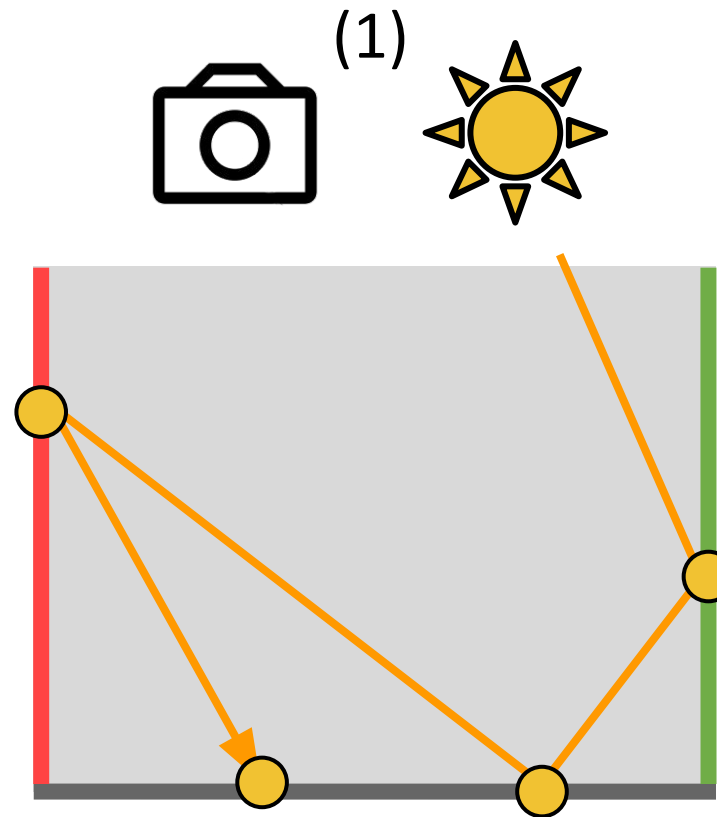
Lab 5 (photon mapping)

Photon mapping basics

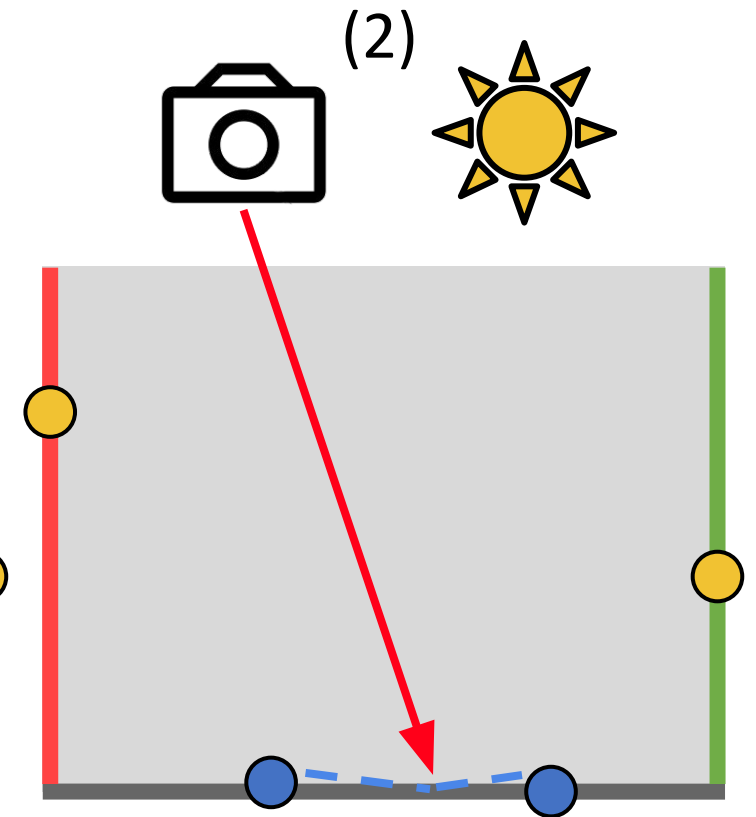
- Photon mapping is a **two-pass** algorithm
 - Uses an **intermediate storage** known as the **photon map**



Light→Camera



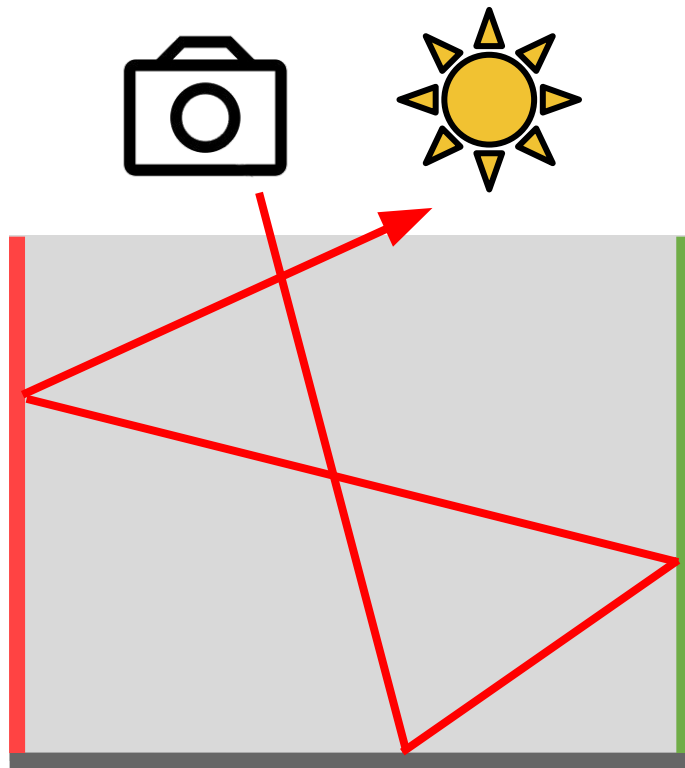
Light→Photon map



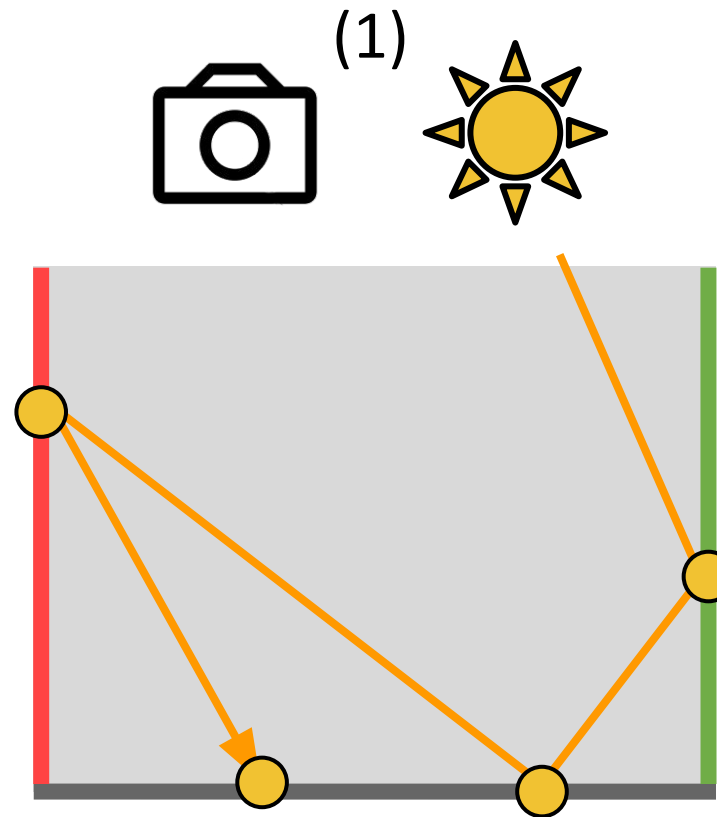
Photon map→Camera

Photon mapping basics

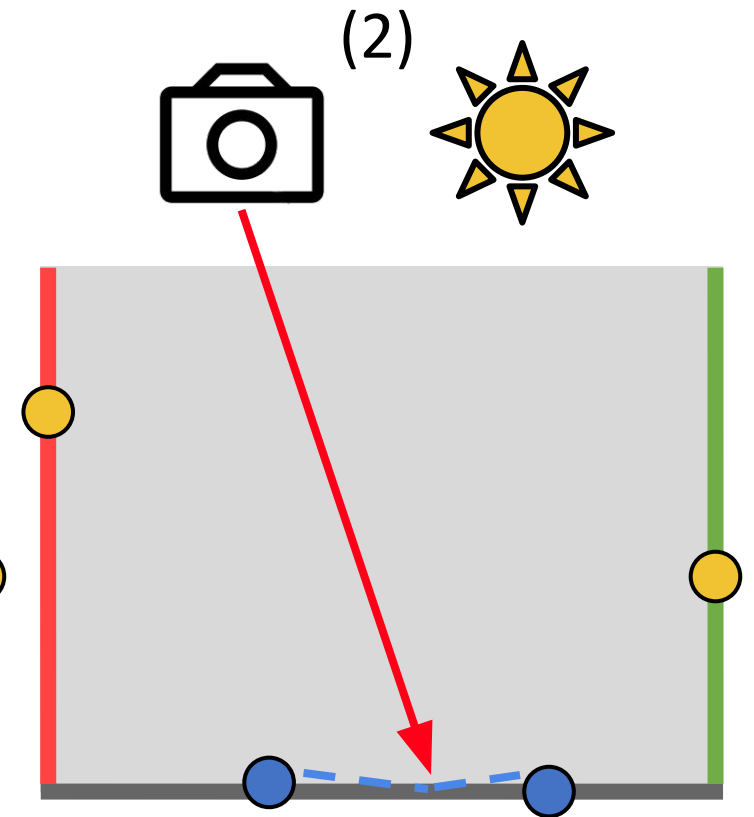
- Lights **write** into the photon map, camera rays **read** from the photon map
- One camera ray can read photons from **many light paths**



Light→Camera



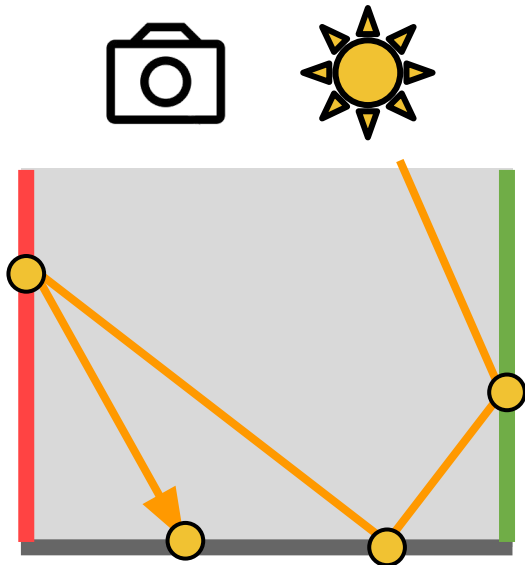
Light→Photon map



Photon map→Camera

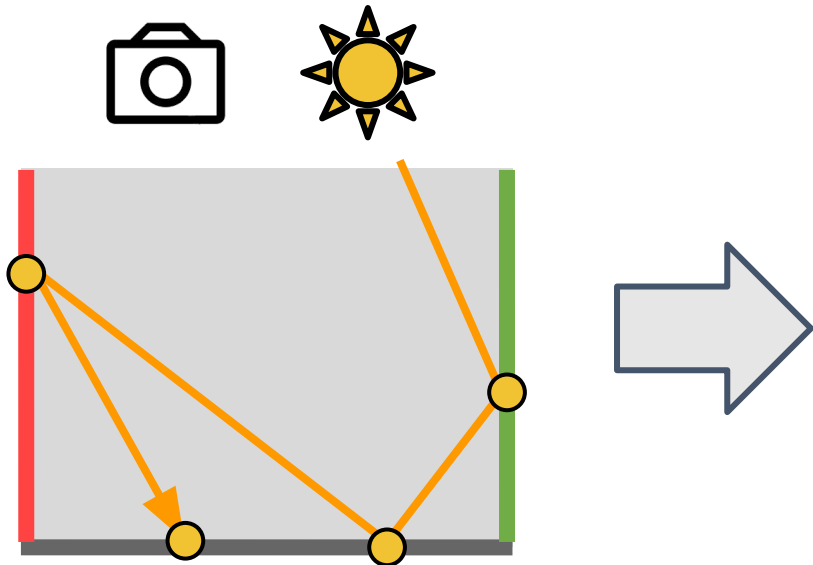
How to create the photon map

- Split a path into two:
 - (1) lights write into the photon map ←
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)



How to create the photon map

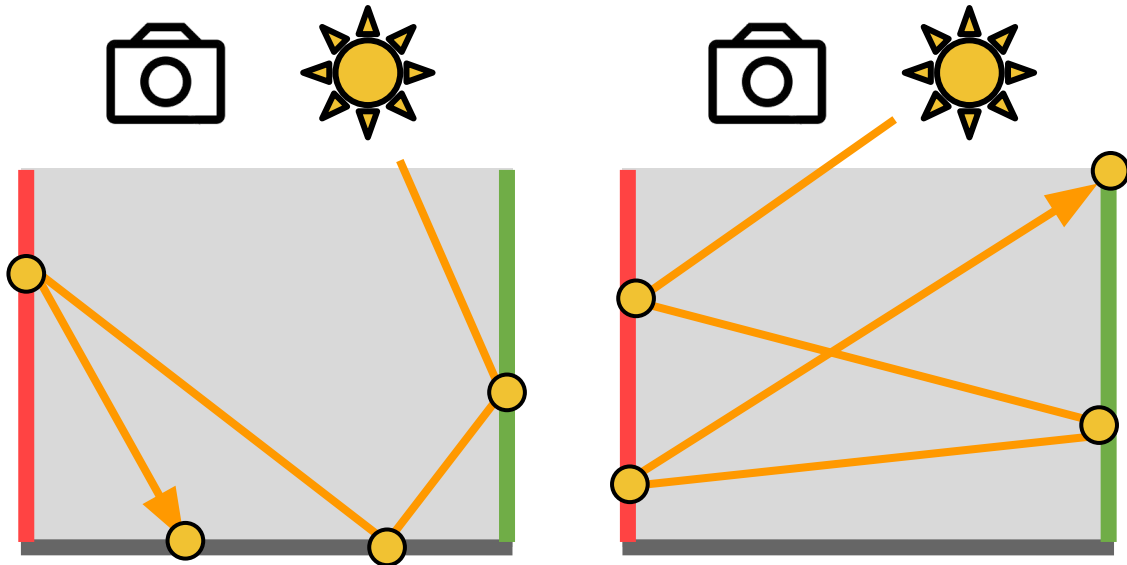
- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)



- Example: four photons ○ ○ ○ ○ this walk
- For each one:
 - Position $\mathbf{x}$
 - Incident direction at $\mathbf{x}$ ω_i
 - Flux (similar to radiance)

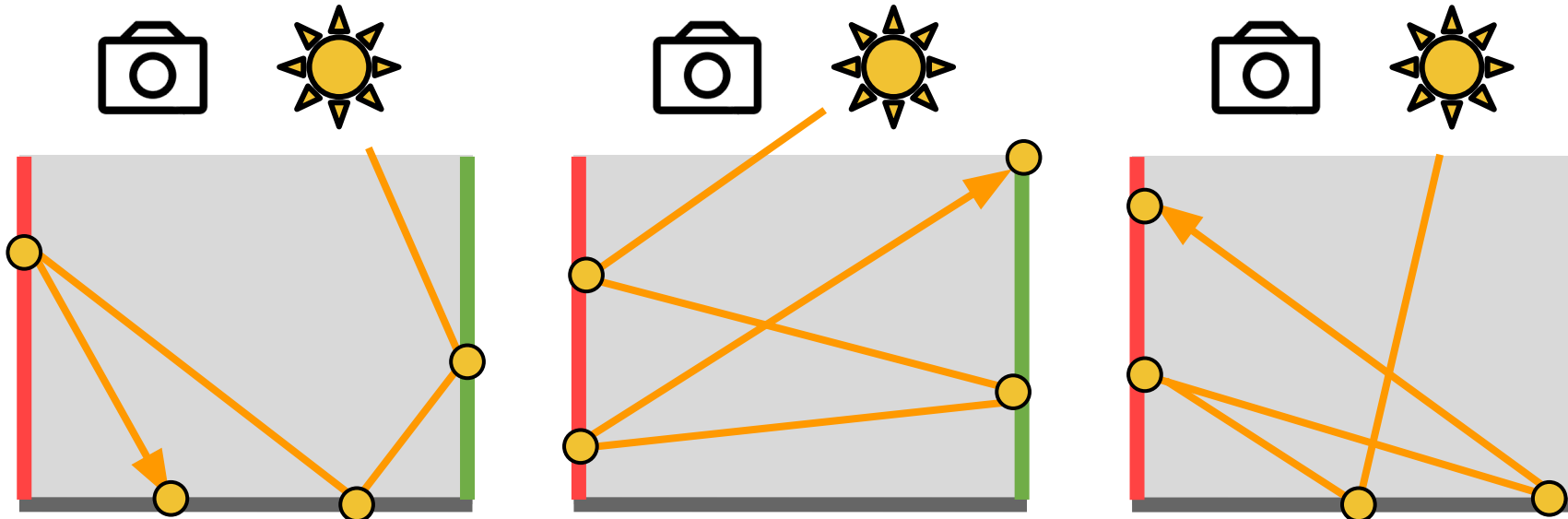
How to create the photon map

- Split a path into two:
 - (1) lights write into the photon map ←
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)



How to create the photon map

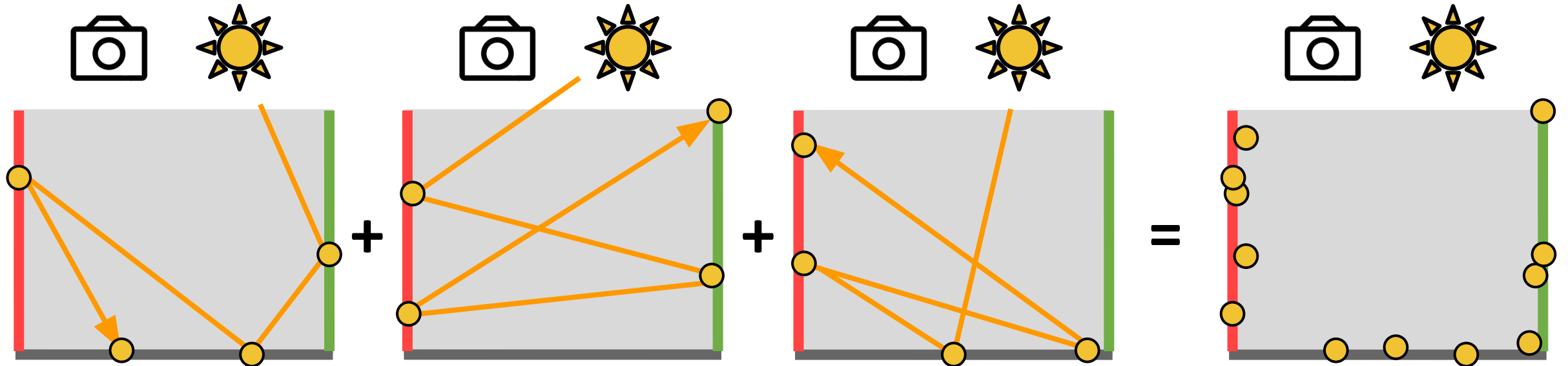
- Split a path into two:
 - (1) lights write into the photon map ←
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)



How to create the photon map

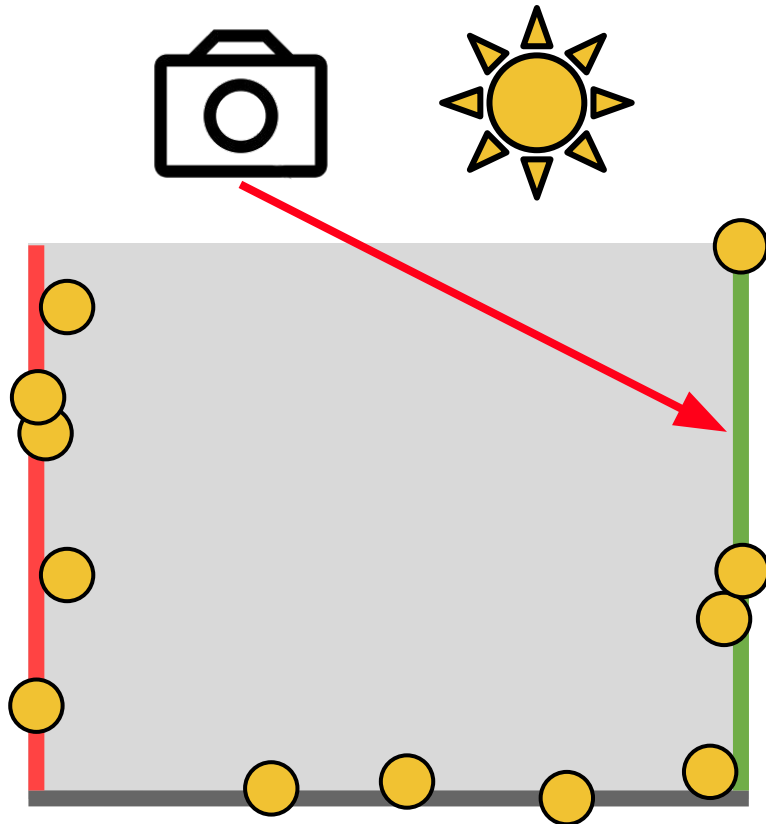
- Split a path into two:
 - (1) lights write into the photon map 
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)

Final photon map



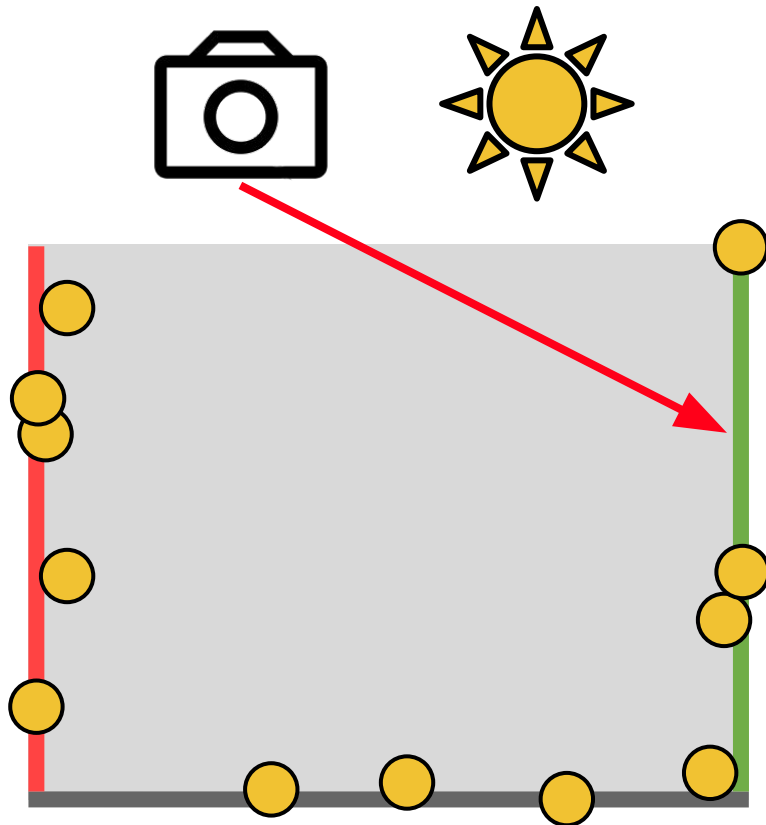
How the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map ←



How the photon map is used

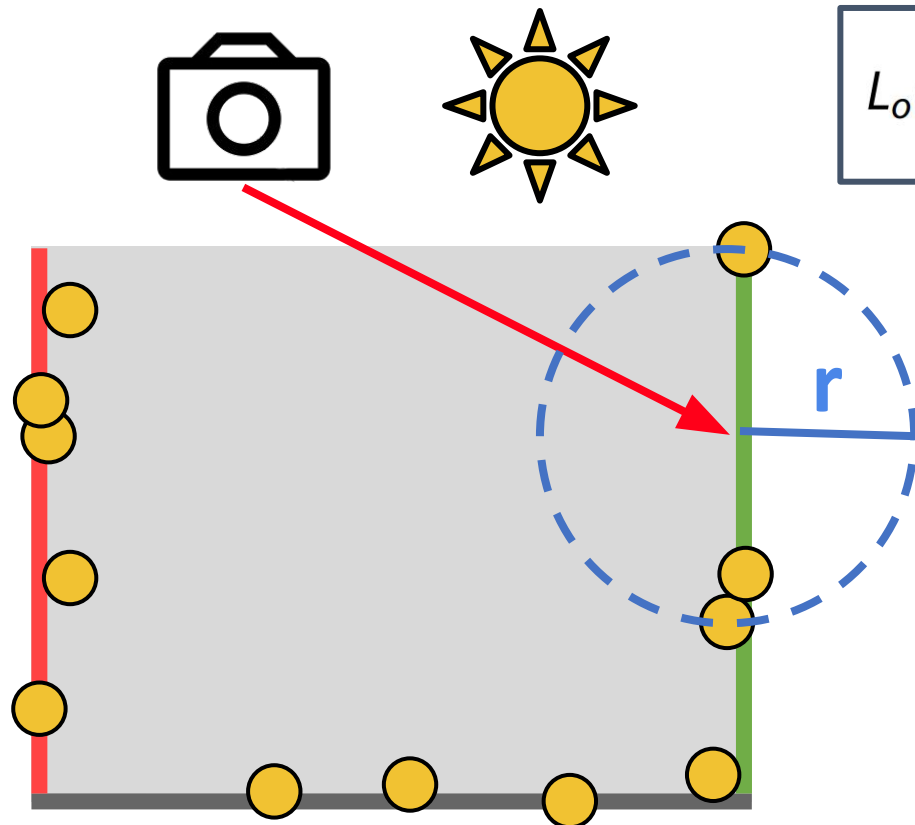
- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map ←



$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

How the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map 

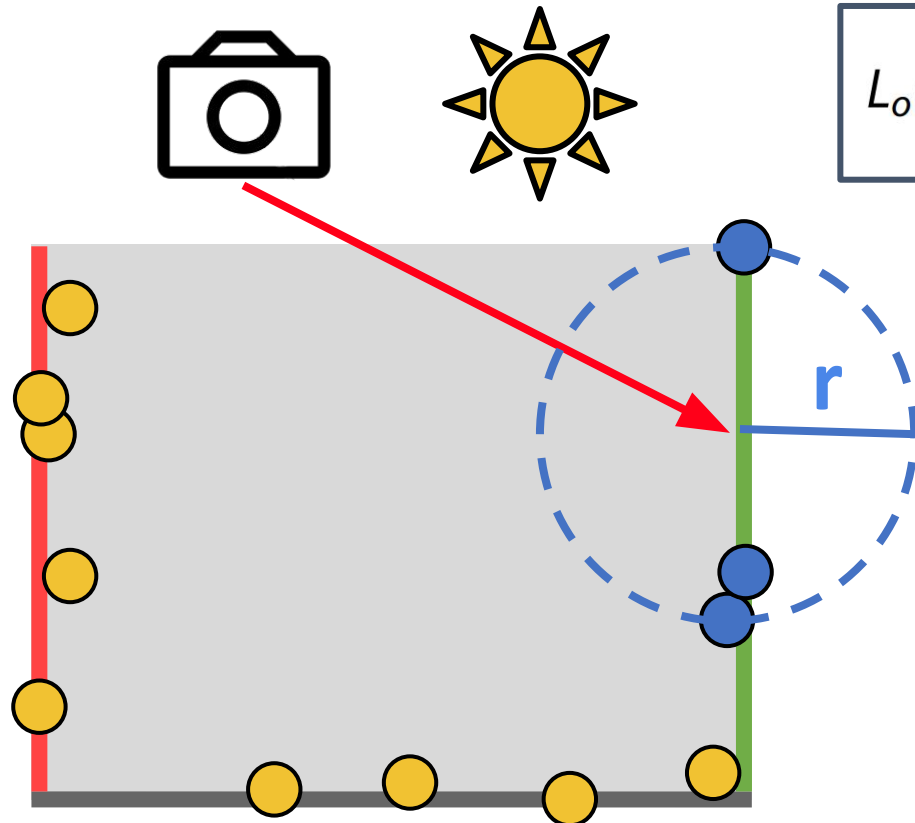


$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

- Idea: use **k nearby** photons (distance $< r$) to approximate the Render Equation

How the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map 

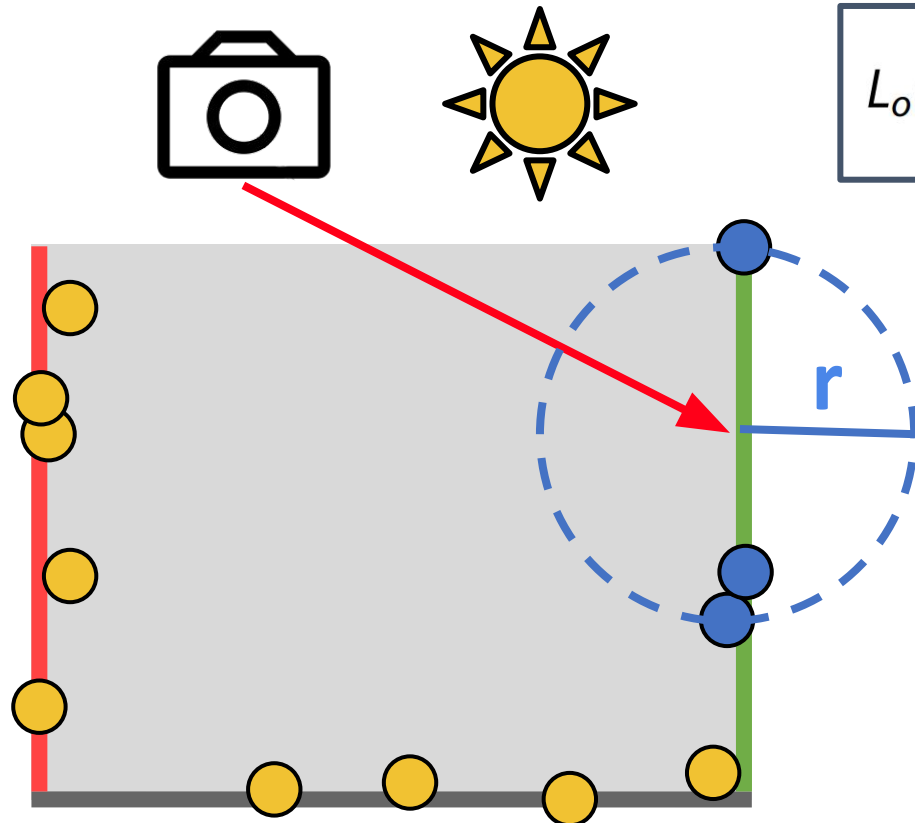


$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

- Idea: use **k nearby** photons (distance $< r$) to approximate the Render Equation

How the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map 



$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

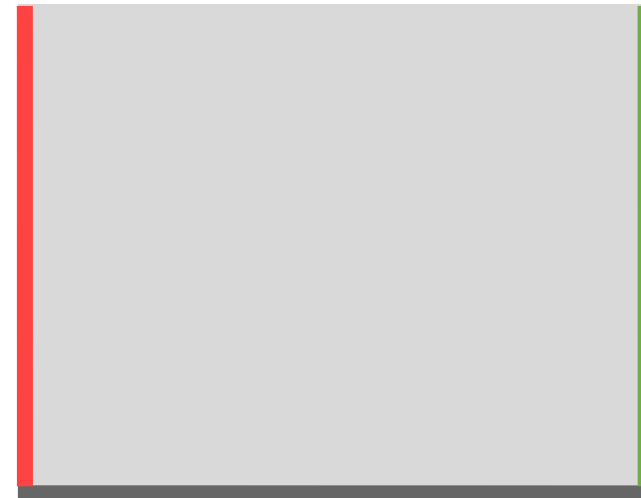
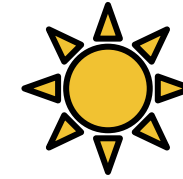
- Idea: use **k nearby** photons (distance $< r$) to approximate the Render Equation
- Integral is approximated as the **sum of k = 3 contributions**

Implementation of the photon map

- Photon map is a collection of N photons
 - Position
 - Incident direction
 - Flux

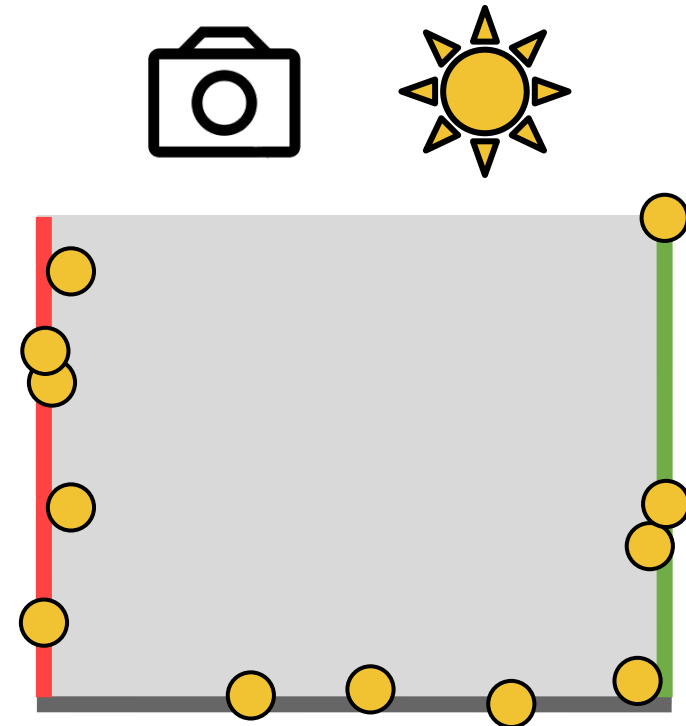
Implementation of the photon map

- Photon map is a collection of N photons
 - Position
 - Incident direction
 - Flux
- Operations:
 - Add a photon / a list of photons



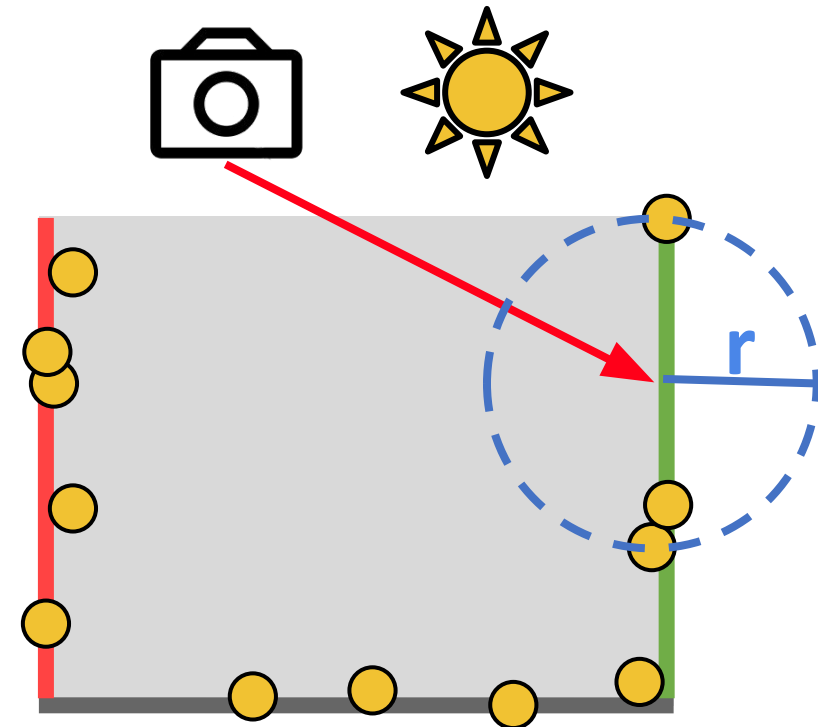
Implementation of the photon map

- Photon map is a collection of N photons
 - Position
 - Incident direction
 - Flux
- Operations:
 - Add a photon / a list of photons



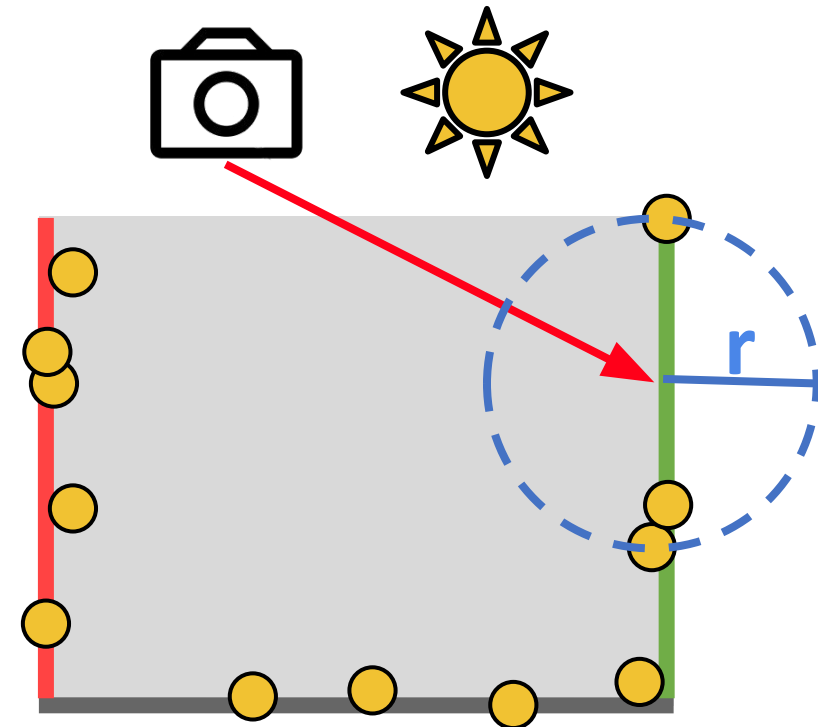
Implementation of the photon map

- Photon map is a collection of N photons
 - Position
 - Incident direction
 - Flux
- Operations:
 - Add a photon / a list of photons
 - Find photons near a point $\boxed{x}$
 - Filter by distance $< r$
 - Filter by k nearest photons



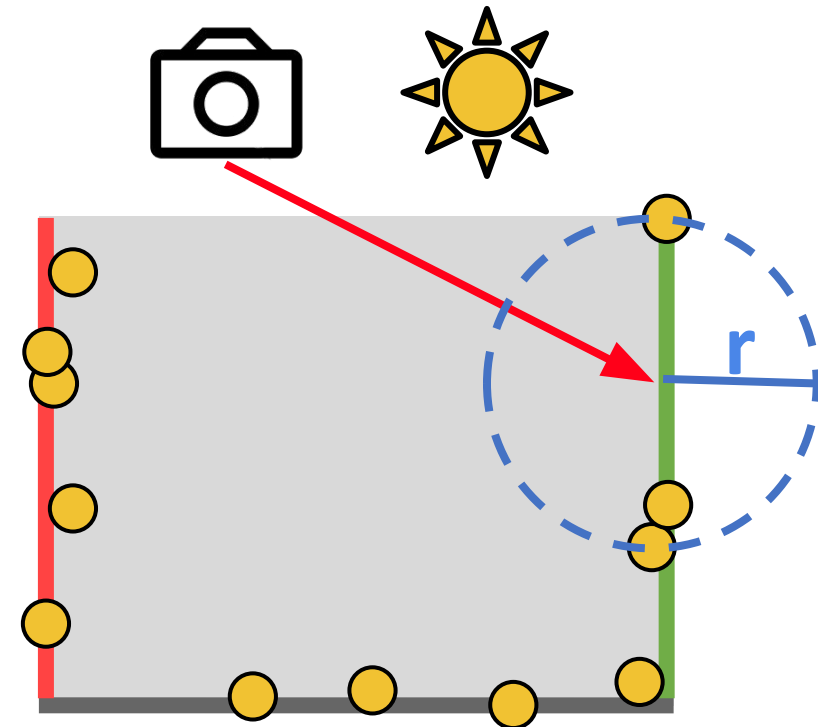
Implementation of the photon map

- Photon map is a collection of N photons
 - Position
 - Incident direction
 - Flux
- Operations:
 - Add a photon / a list of photons
 - Find photons near a point X
 - Filter by distance $< r$
 - Filter by k nearest photons
 - Usually the number of stored photons is huge (large N)



Implementation of the photon map

- Photon map is a collection of N photons
 - Position
 - Incident direction
 - Flux
- Operations:
 - Add a photon / a list of photons
 - Find photons near a point $\mathbf{x}$
 - Filter by distance $< r$
 - Filter by k nearest photons
 - Usually the number of stored photons is huge (large N)
- We need a data structure that implements these **two operations as fast as possible**



Implementation of the photon map

- Data structures for the photon map

Implementation of the photon map

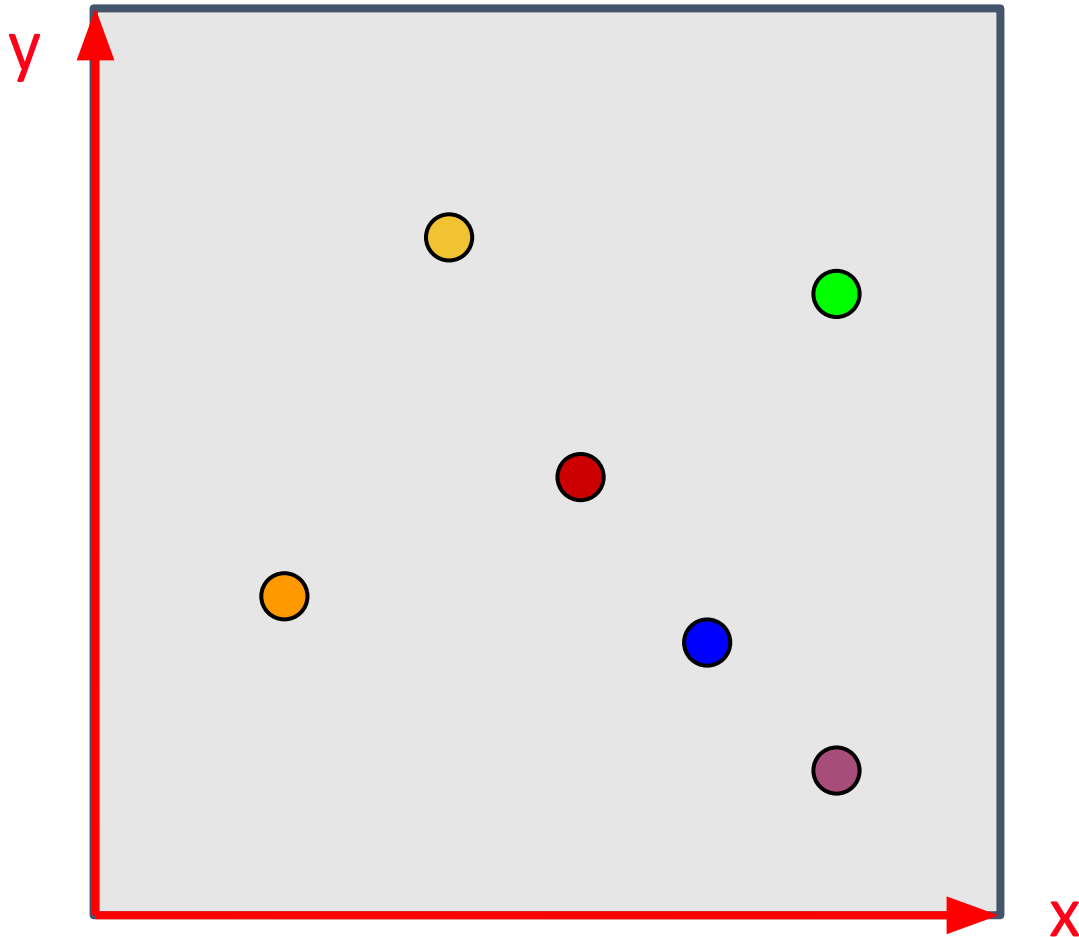
- Data structures for the photon map
- **Array**
 - Add photons → Add elements to the array
 - Search nearby photons → Loop through all N elements of the array, $O(N)$
 - **Very** inefficient

Implementation of the photon map

- Data structures for the photon map
- **Array**
 - Add photons → Add elements to the array
 - Search nearby photons → Loop through all N elements of the array, $O(N)$
 - **Very** inefficient
- **k-d tree**
 - Structure for organizing points (photons) in a k dimensional space
 - We use a space with $k = 3$ dimensions (x, y, z)

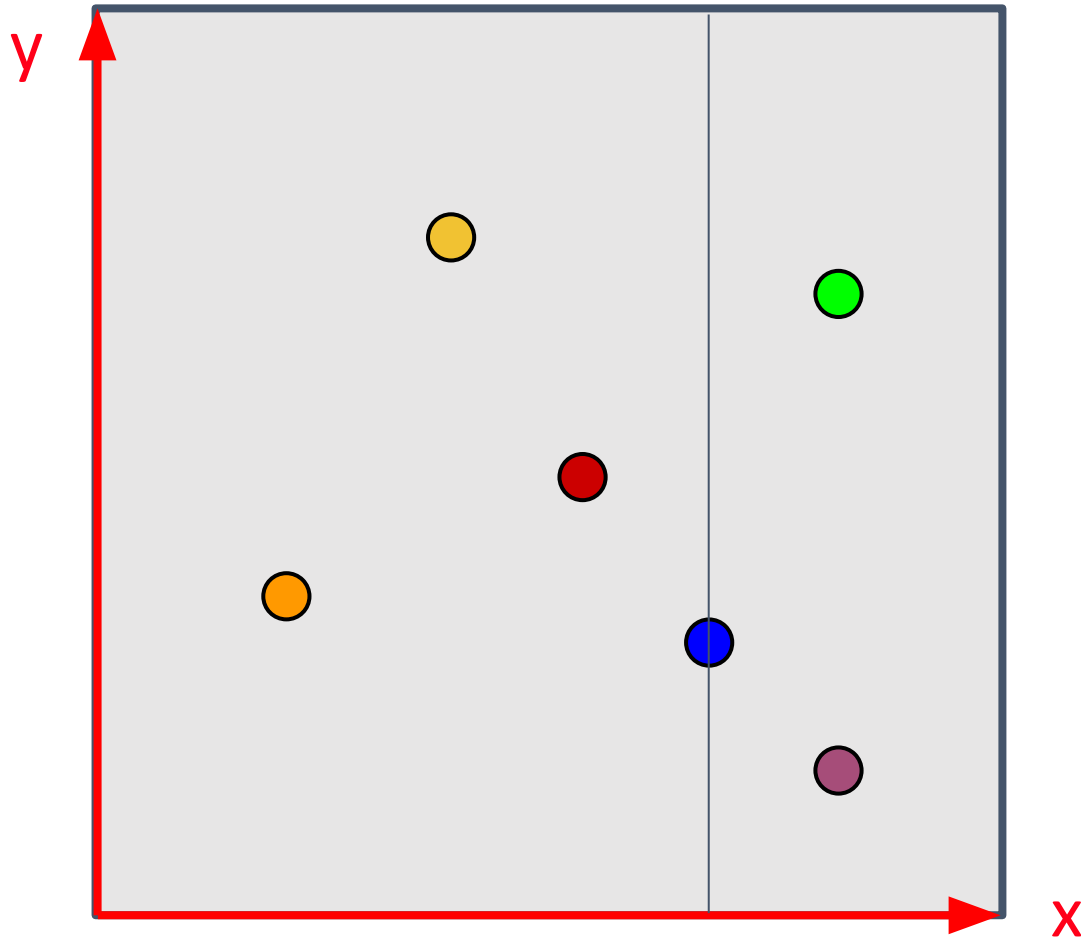
Implementation of the photon map

- k-d tree example with $k = 2$ dimensions



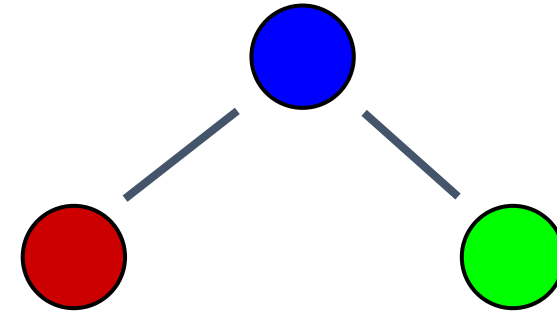
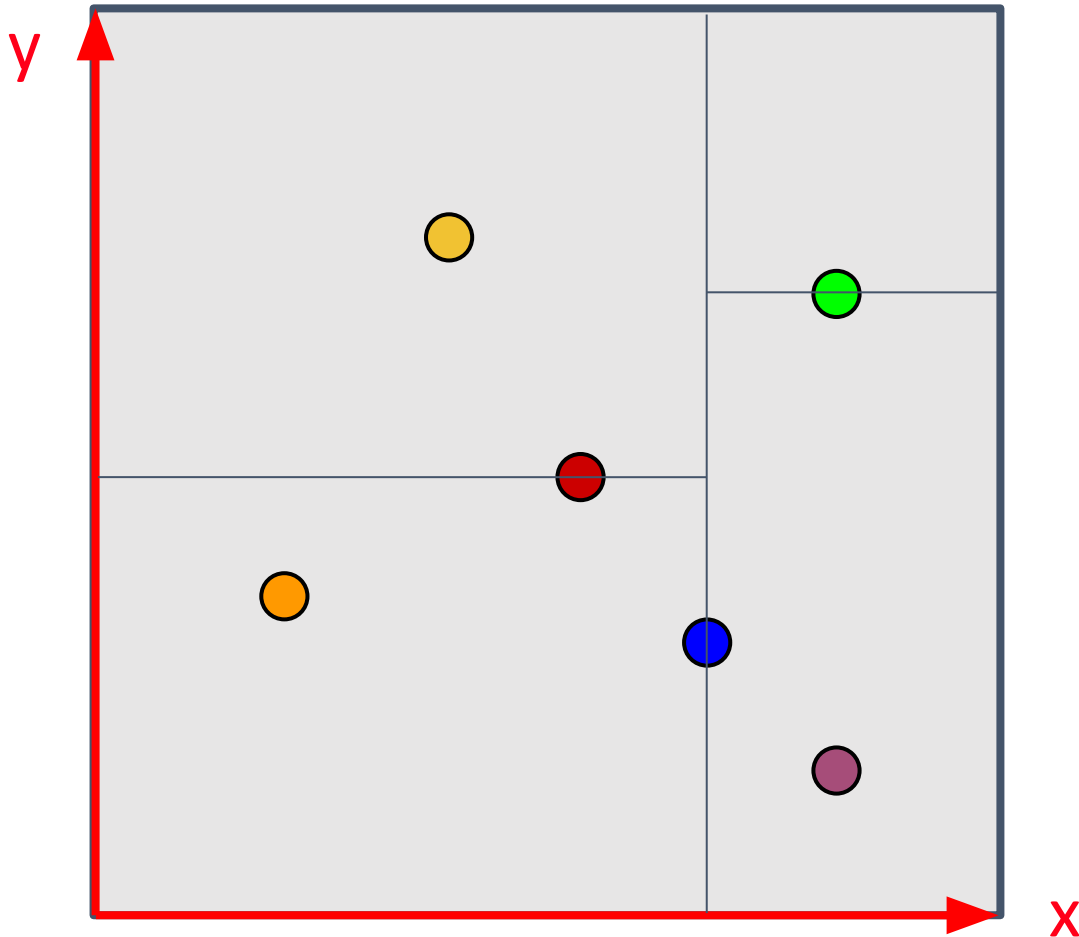
Implementation of the photon map

- k-d tree example with $k = 2$ dimensions



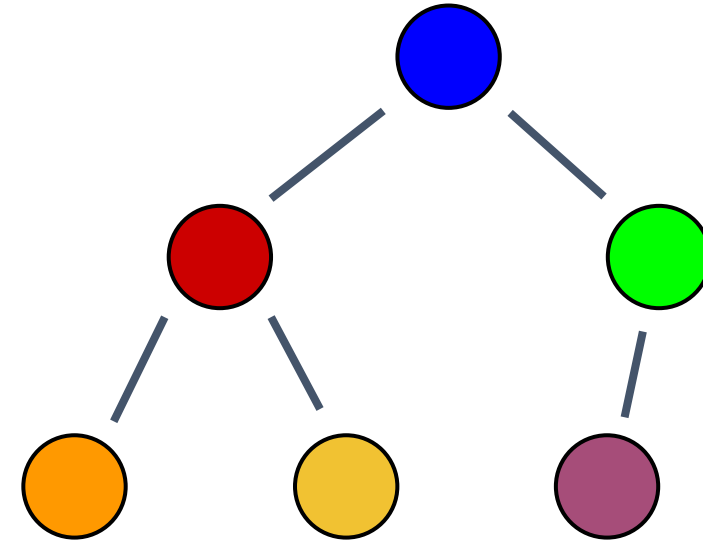
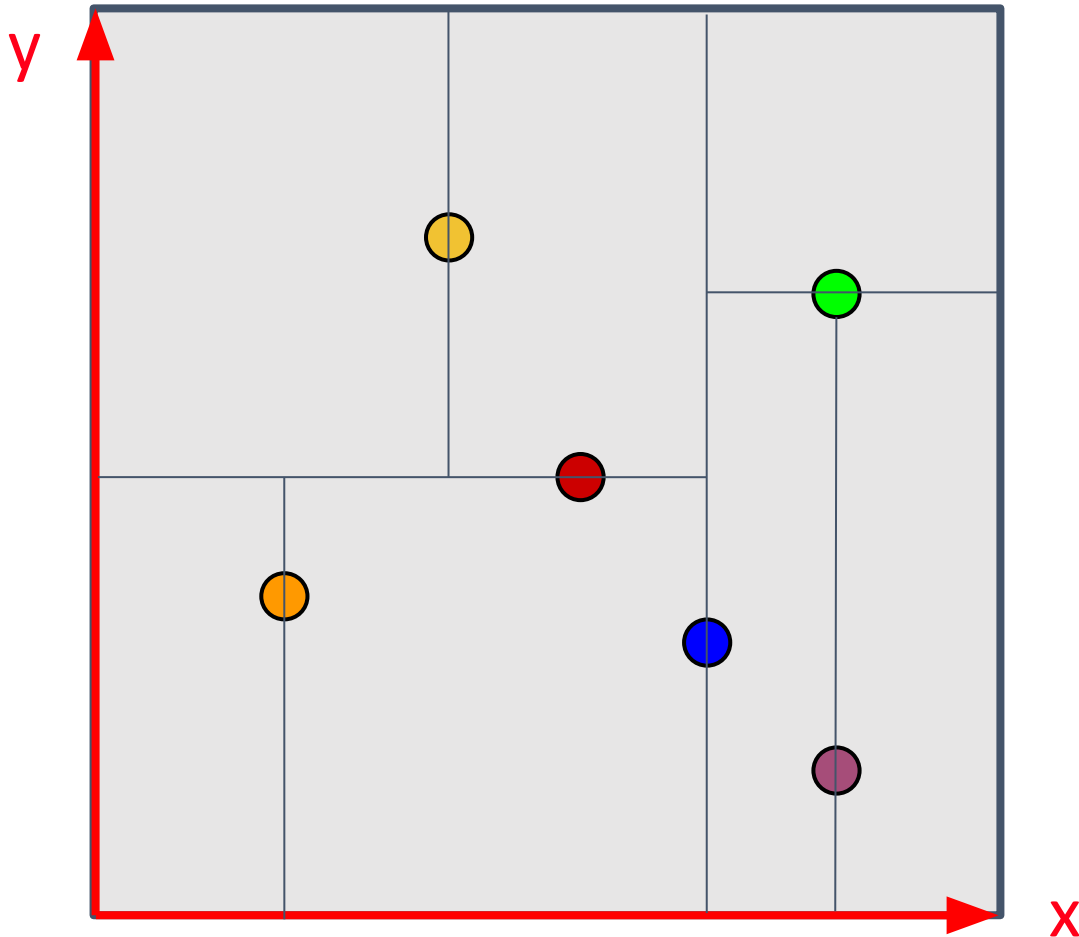
Implementation of the photon map

- k-d tree example with $k = 2$ dimensions



Implementation of the photon map

- k-d tree example with $k = 2$ dimensions



Implementation of the photon map

- Data structures for the photon map
- **Array**
 - Add photons → Add elements to the array
 - Search nearby photons → Loop through all N elements of the array, $O(N)$
 - **Very** inefficient
- **k-d tree**
 - Structure for organizing points (photons) in a k dimensional space
 - We use a space with $k = 3$ dimensions (x, y, z)
 - Add photons → Create a tree that partitions space, leaves/children = photons
 - Search nearby photons → **Space partitioning gives average time $O(\log N)$**

Today: Generating the photon map

Today we will focus on the generation of the photon map

- Photon random walk
 - 1) Select a (point) light source from the scene
 - 2) Begin the photon random walk with a random direction
 - 3) Scatter the photon through the scene
 - 4) Store the scattered photons in the photon map

Today: Generating the photon map

Today we will focus on the generation of the photon map

- Photon random walk
 - 1) Select a (point) light source from the scene
 - 2) Begin the photon random walk with a random direction
 - 3) Scatter the photon through the scene
 - 4) Store the scattered photons in the photon map
- Repeat until
 - Maximum number of photons in the photon map (map is full)
 - **Maximum number of random walks**

Today: Generating the photon map

Today we will focus on the generation of the photon map

- Photon random walk
 - 1) **Select a (point) light source from the scene**
 - 2) Begin the photon random walk with a random direction
 - 3) Scatter the photon through the scene
 - 4) Store the scattered photons in the photon map
- Repeat until
 - Maximum number of photons in the photon map (map is full)
 - Maximum number of random walks

1. Select a light source

The user will indicate the number of photon random walks N

- How many photons for each light source?

1. Select a light source

The user will indicate the number of photon random walks N

- How many photons for each light source?
 - Idea 1: all light sources emit same amount of photons

1. Select a light source

The user will indicate the number of photon random walks N

- How many photons for each light source?
 - Idea 1: all light sources emit same amount of photons



$N/2$ photons



$N/2$ photons

1. Select a light source

The user will indicate the number of photon random walks N

- How many photons for each light source?
 - Idea 1: all light sources emit same amount of photons



$N/2$ photons



$N/2$ photons

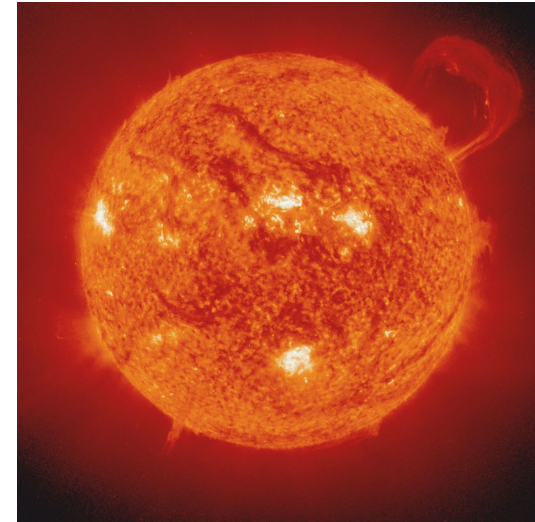
1. Select a light source

The user will indicate the number of photon random walks N

- How many photons for each light source?
 - Idea 1: all light sources emit same amount of photons



$N/2$ photons



$N/2$ photons

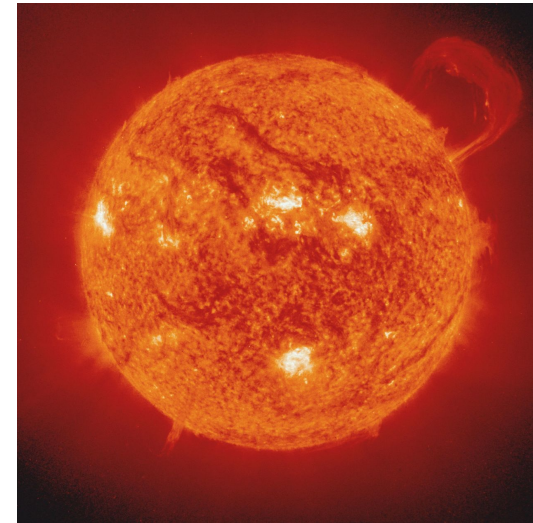
1. Select a light source

The user will indicate the number of photon random walks N

- How many photons for each light source?
 - ~~Idea 1: all light sources emit same amount of photons~~



$N/2$ photons



$N/2$ photons

1. Select a light source

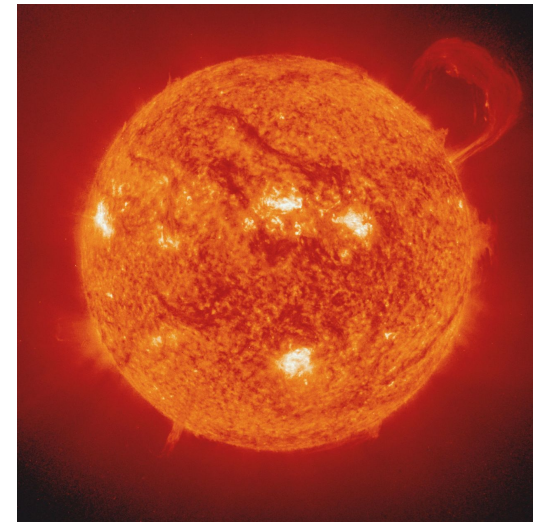
The user will indicate the number of photon random walks N

- How many photons for each light source?
 - Idea 2: Photons proportional to the emission/power of the light source



$N^*(p1/(p1 + p2))$
photons

p2



$N^*(p2/(p1 + p2))$
photons

Today: Generating the photon map

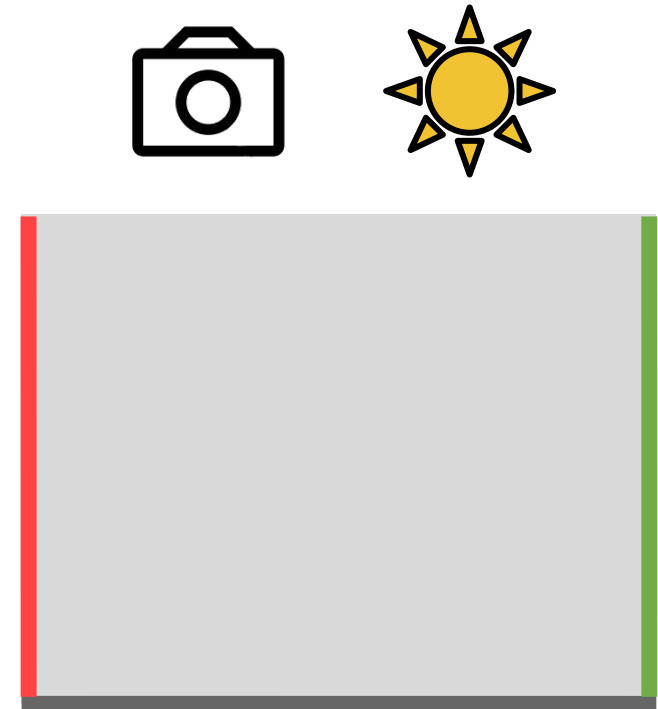
Today we will focus on the generation of the photon map

- Photon random walk
 - 1) Select a (point) light source from the scene
 - 2) Begin the photon random walk with a random direction**
 - 3) Scatter the photon through the scene
 - 4) Store the scattered photons in the photon map
- Repeat until
 - Maximum number of photons in the photon map (map is full)
 - **Maximum number of random walks**

2. Begin the random walk

A point light emits photons in all possible (spherical) directions

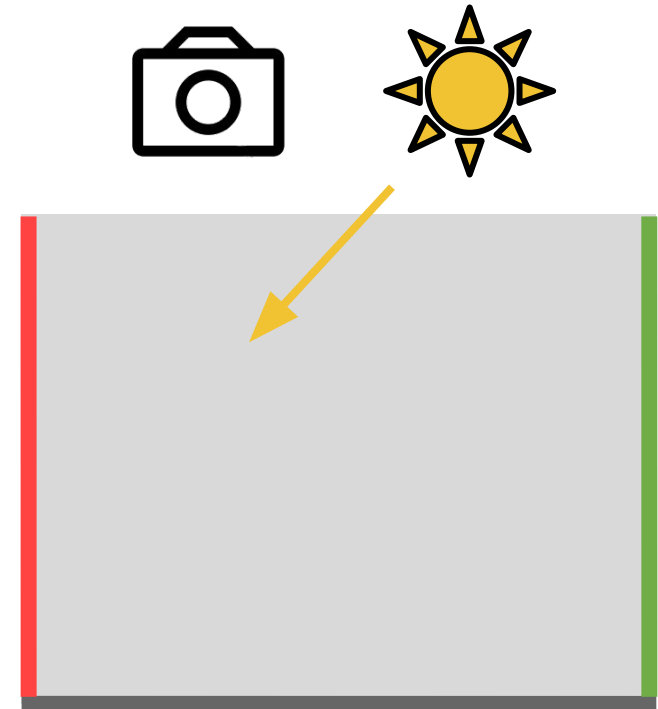
- How to choose a direction to emit each photon?



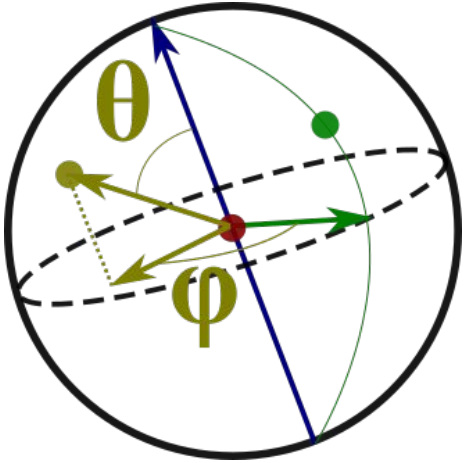
2. Begin the random walk

A point light emits photons in all possible (spherical) directions

- How to choose a direction to emit each photon?
 - Random direction (Monte Carlo)
 - **Sample a direction on the sphere**



2. Begin the random walk



Spherical coordinates:

- $\Theta \in [0, \pi)$
- $\varphi \in [0, 2\pi)$

Convert spherical to cartesian

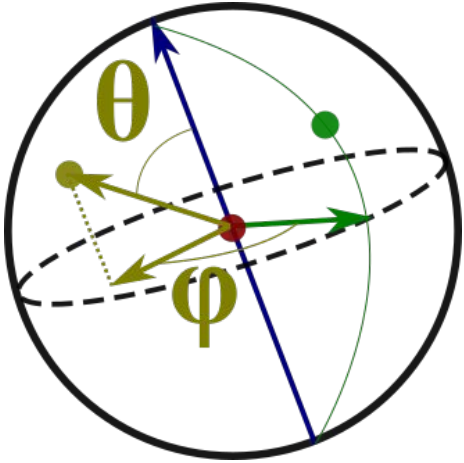
$\xi_\Theta \in [0,1]$ } Uniform
 $\xi_\varphi \in [0,1]$ } random

Uniform angle sampling **is bad**

$$\Theta = \pi \xi_\Theta$$

$$\varphi = 2\pi \xi_\varphi$$

2. Begin the random walk



Spherical coordinates:

- $\Theta \in [0, \pi)$
- $\varphi \in [0, 2\pi)$

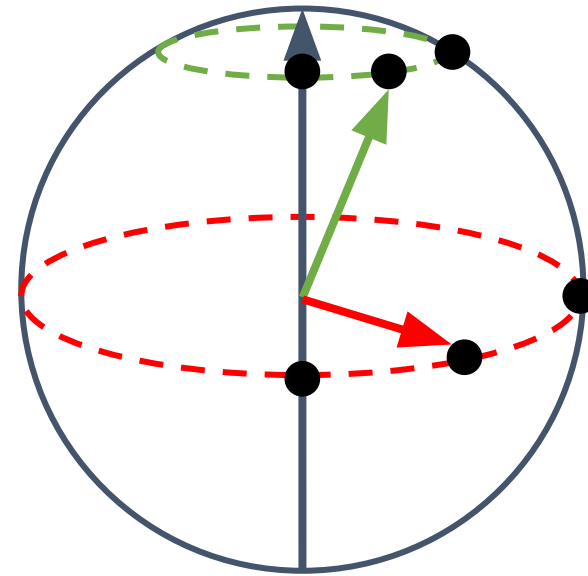
Convert spherical to cartesian

$\xi_\Theta \in [0, 1]$
 $\xi_\varphi \in [0, 1]$ } Uniform random

Uniform angle sampling **is bad**

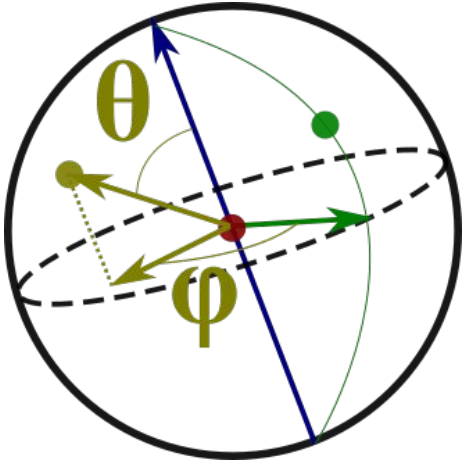
$$\Theta = \pi \xi_\Theta$$

$$\varphi = 2\pi \xi_\varphi$$



Uniform $\theta, \varphi \neq$ Uniform solid angle ω_i

2. Begin the random walk



Spherical coordinates:

- $\Theta \in [0, \pi)$
- $\varphi \in [0, 2\pi)$

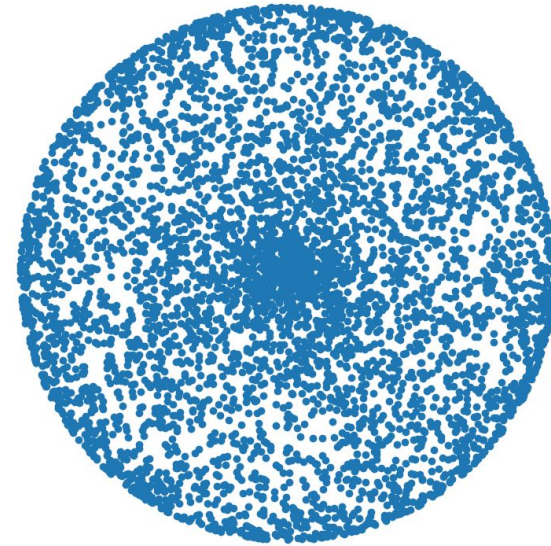
Convert spherical to cartesian

$\xi_\Theta \in [0, 1]$ } Uniform
 $\xi_\varphi \in [0, 1]$ } random

Uniform angle sampling **is bad**

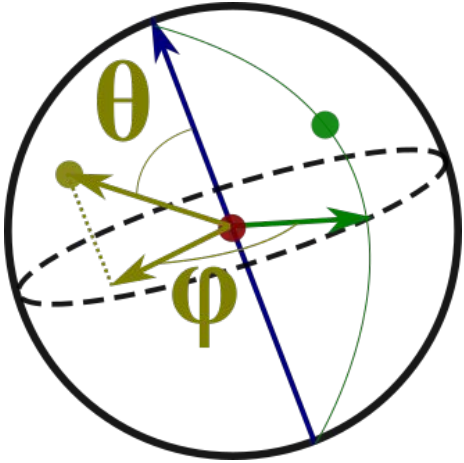
$$\Theta = \pi \xi_\Theta$$

$$\varphi = 2\pi \xi_\varphi$$



Top view (more samples near **poles**)

2. Begin the random walk



Spherical coordinates:

- $\Theta \in [0, \pi)$
- $\varphi \in [0, 2\pi)$

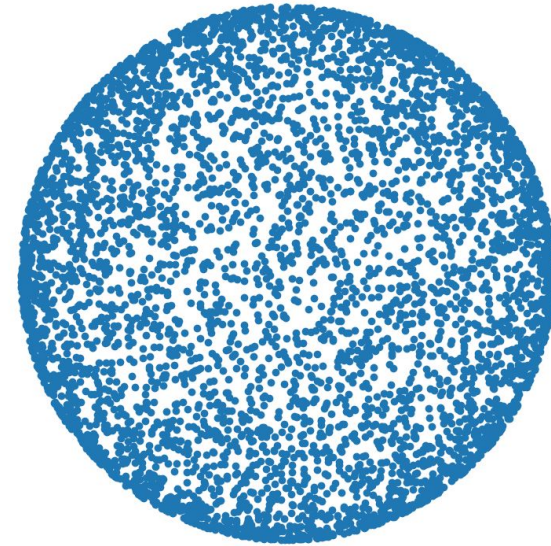
Convert spherical to cartesian

$\xi_\Theta \in [0, 1]$
 $\xi_\varphi \in [0, 1]$ } Uniform
random

Uniform solid angle sampling is good

$$\Theta = \arccos(2\xi_\Theta - 1)$$

$$\varphi = 2\pi\xi_\varphi$$



Top view (uniformly distributed)

Today: Generating the photon map

Today we will focus on the generation of the photon map

- Photon random walk
 - 1) Select a (point) light source from the scene
 - 2) Begin the photon random walk with a random direction
 - 3) Scatter the photon through the scene**
 - 4) Store the scattered photons in the photon map
- Repeat until
 - Maximum number of photons in the photon map (map is full)
 - **Maximum number of random walks**

3. Scatter the photon through the scene

The origin of the photons is the light source

- **Conservation of energy:**

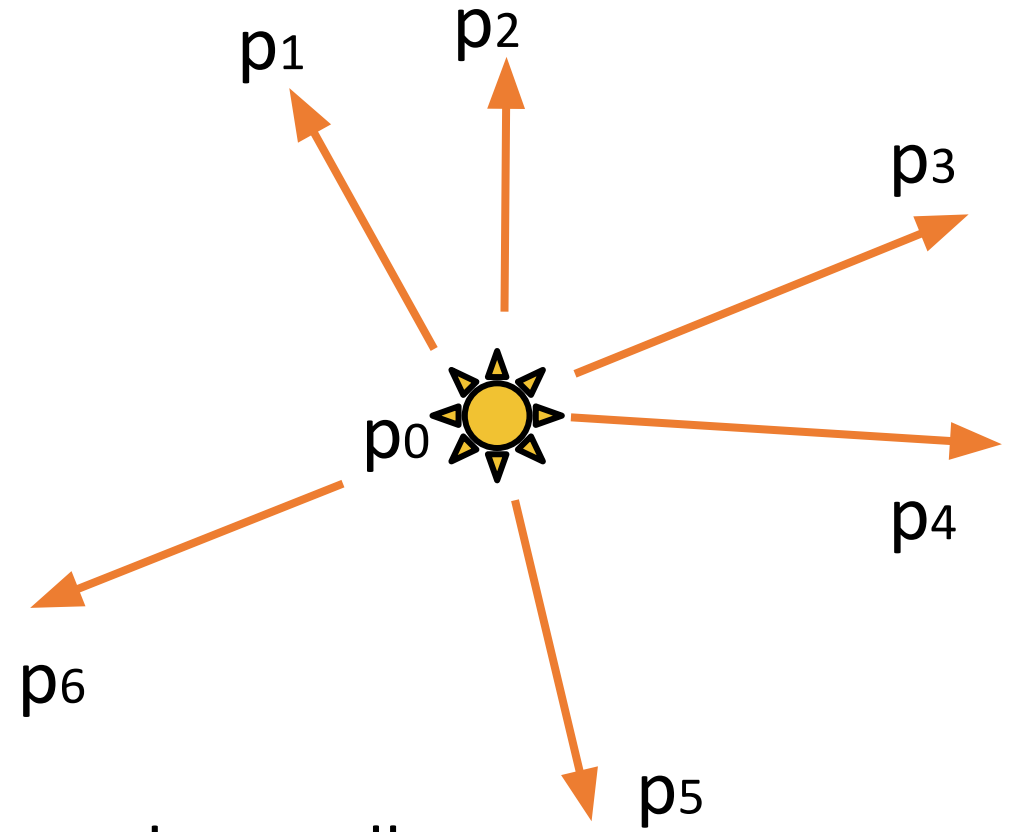
$$p_1 + p_2 + p_3 + p_4 + p_5 + p_6 = 4\pi p_0$$

The light source emitted S photons.

A **photon's flux** is $p_i = 4\pi p_0 / S$

Careful with the photon map's size limit

You might need to update S after doing the random walks

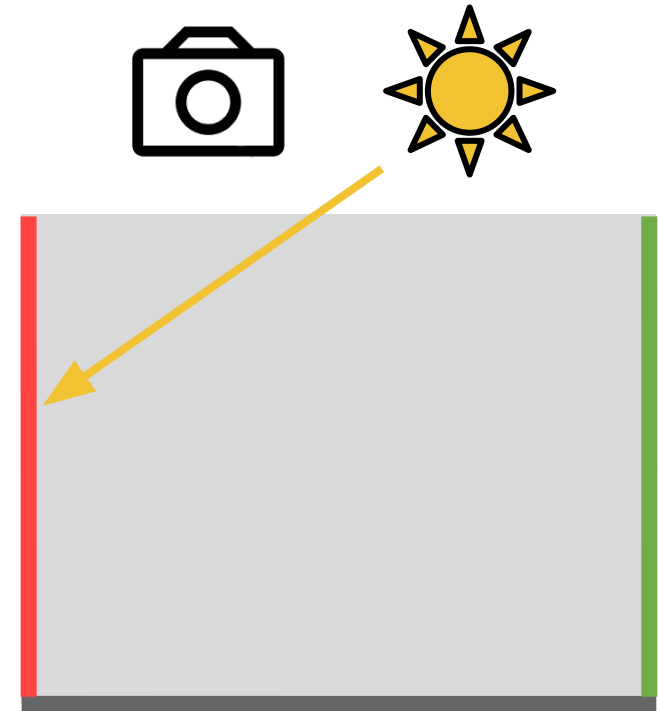


3. Scatter the photon through the scene

Photon hits a diffuse surface → Store photon in the photon map

Next: In which directions do photons scatter?

- Note: in this session we **only use diffuse materials**

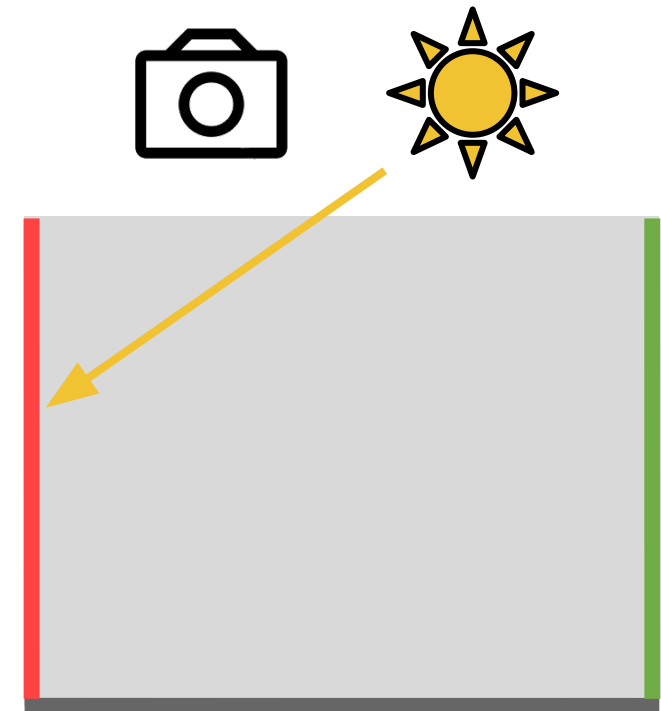
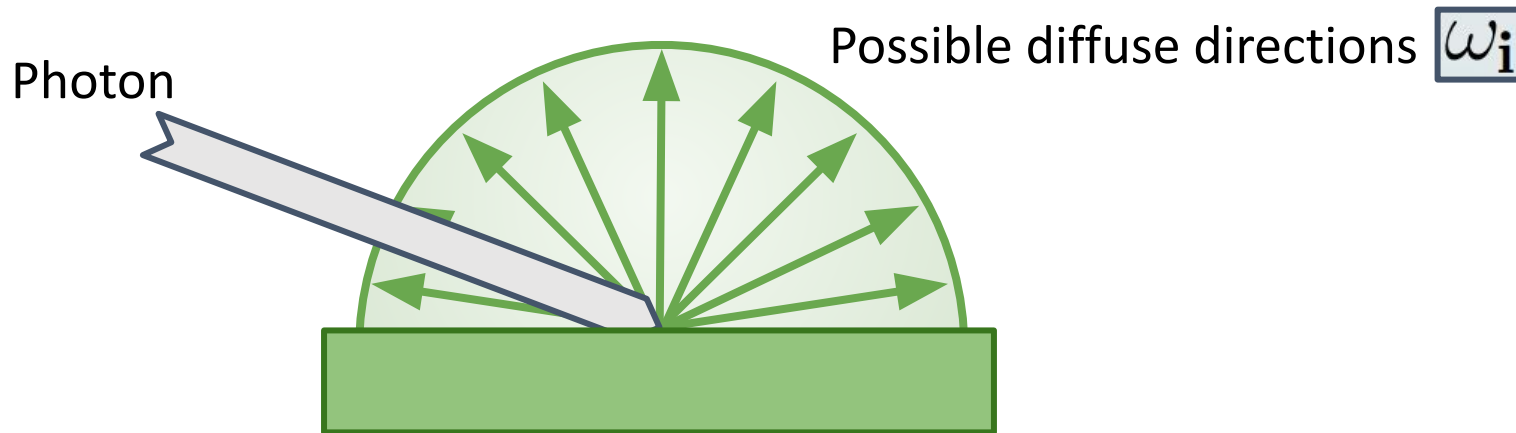


3. Scatter the photon through the scene

Photon hits a diffuse surface → Store photon in the photon map

Next: In which directions do photons scatter?

- Note: in this session we **only use diffuse materials**

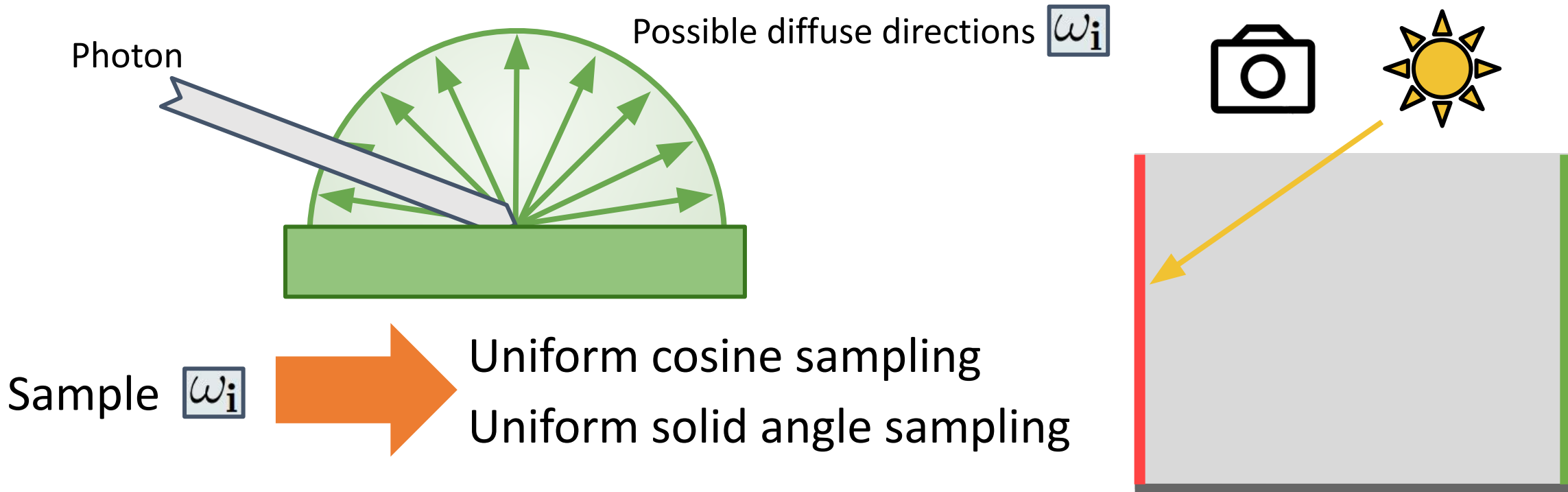


3. Scatter the photon through the scene

Photon hits a diffuse surface → Store photon in the photon map

Next: In which directions do photons scatter?

- Note: in this session we **only use diffuse materials**

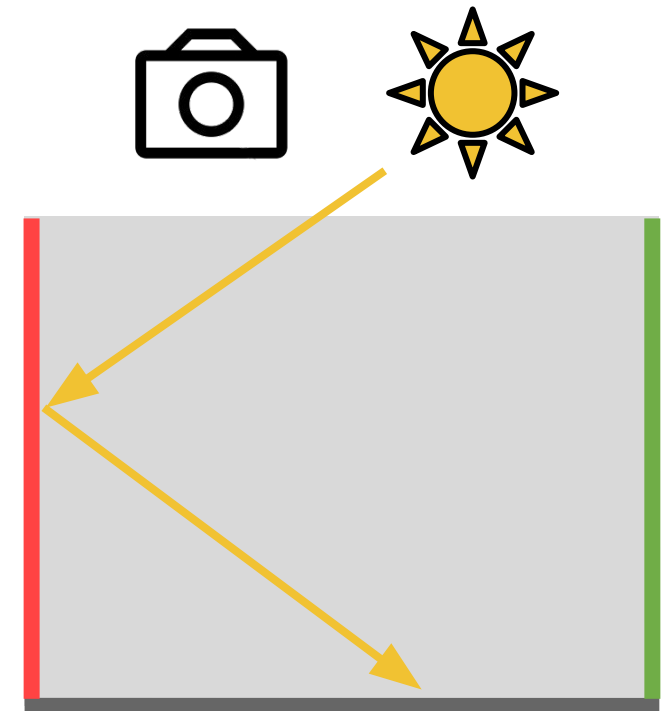


3. Scatter the photon through the scene

Photon hits a diffuse surface → Store photon in the photon map

Next: In which directions do photons scatter?

- Note: in this session we **only use diffuse materials**
- Sample ω_i
- Evaluate BRDF and cosine term
 $f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|$
- Compute radiance for scattered photon



3. Scatter the photon through the scene

Photon hits a diffuse surface → Store photon in the photon map

Next: In which directions do photons scatter?

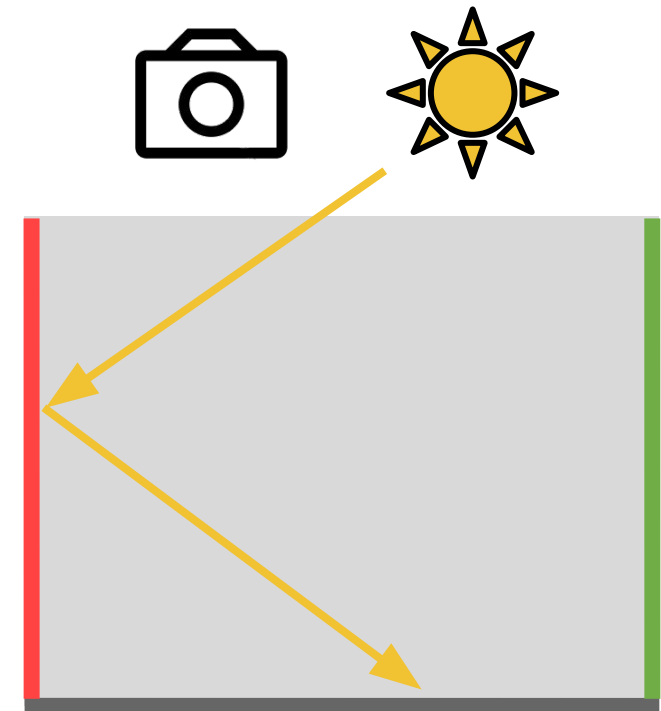
- Note: in this session we **only use diffuse materials**

- Sample ω_i
- Evaluate BRDF and cosine term

$$f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i|$$

- Compute radiance for scattered photon

**You already implemented this for the path tracer
(reuse your code)**



Today: Generating the photon map

Today we will focus on the generation of the photon map

- Photon random walk
 - 1) Select a (point) light source from the scene
 - 2) Begin the photon random walk with a random direction
 - 3) Scatter the photon through the scene
 - 4) Store the scattered photons in the photon map**
- Repeat until
 - Maximum number of photons in the photon map (map is full)
 - **Maximum number of random walks**

4. Store photons in the photon map

You can use the provided k-d tree:

- Available in Moodle:
 - `kd_tree.h`
- Implemented in C++
- You need to define some custom classes (see next slides)

4. Store photons in the photon map

```
/*
 * Your Photon class implementation, which stores each
 * photon walk interaction
 */
class YourPhoton {
    Vec3D position_;    // 3D point of the interaction
    ...
    // It returns the axis i position (x, y or z)
    float position(std::size_t i) const { return position_[i]}
    ...
}
```

Your photon class

```
/*
 * An additional struct that allows the KD-Tree to access your photon position
 */
struct PhotonAxisPosition {
    float operator()(const YourPhoton& p, std::size_t i) const {
        return p.position(i);
    }
};
```

An auxiliary struct to
access the position
in your photon class

4. Store photons in the photon map

Type definition (alias) as a shorter name

```
/*  
    The KD-Tree ready to work in 3 dimensions, with YourPhoton s, under a  
    brand-new name: YourPhotonMap  
*/  
using YourPhotonMap = nn::KDTree<YourPhoton,3,PhotonAxisPosition>;
```

4. Store photons in the photon map

You need two functions:

- 1. Construction of the k-d tree

```
/*  
    Example function to generate the photon map with the given photons  
*/  
YourPhotonMap generation_of_photon_map(...){  
    std::list<YourPhoton> photons = ...;           // Create a list of photons  
    map = YourPhotonMap(photons, PhotonAxisPosition())  
    return map  
}
```

4. Store photons in the photon map

You need two functions:

- 1. Construction of the k-d tree

```
map = YourPhotonMap(photons, PhotonAxisPosition())
```

- 2. Search for nearest neighbors in the k-d tree

```
// nearest is the nearest photons returned by the KDTree  
auto nearest = map.nearest_neighbors(position,  
| | | | | | | | | | nphotons_estimate,  
| | | | | | | | | | radius_estimate)
```

With the provided source you can find an example of usage:

- *kd_tree_example.cpp*

4. Store photons in the photon map

```
/*  
    Example method to search for the nearest neighbors of the photon map  
*/  
void search_nearest(YourPhotonMap map, ...){  
    // Position to look for the nearest photons  
    Vec3D query_position = ...;  
    // Maximum number of photons to look for  
    unsigned long nphotons_estimate = ...;  
    // Maximum distance to look for photons  
    float radius_estimate = ...;  
  
    // nearest is the nearest photons returned by the KDTree  
    auto nearest = map.nearest_neighbors(query_position,  
                                         nphotons_estimate,  
                                         radius_estimate)  
}
```

Careful!

Your Vec3D class
must implement
operator[]

4. Store photons in the photon map

```
// Position to look for the nearest photons  
Vec3D query_position = ...;
```

```
class Vec3D{  
private:  
    // 3 dimension axis  
    float x, y, z;
```



```
public:  
    // Overload the [] operator for indexing  
    const float& operator[](size_t t) const{  
        if (t == 0)  
            return x;  
        else if (t == 1)  
            return y;  
        else if (t == 2)  
            return z;  
        else  
            return;  
    }  
};
```

**For example*

4. Store photons in the photon map

To compile our code, you will need C++17. You can use the flag:

-std=c++17

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice implement:

- The generation of a photon map:
 - Select a (point) light source
 - Sampling a direction for light source to start the random walk
 - Photon scattering through the scene (**only diffuse or absorption events**)
 - Storage in the photon map: **k-d tree**
- Recommended deadline: December 4th (moodle: January 11th)
 - If you have more time, you can visualize your photon map:
 - Launch rays from the camera (exactly like path tracing, reuse your code)
 - At each intersection, find the nearest photon in the photon map
 - Use the flux from that photon to paint each pixel of your final image

Lab #5 – Photon mapping (part 2)

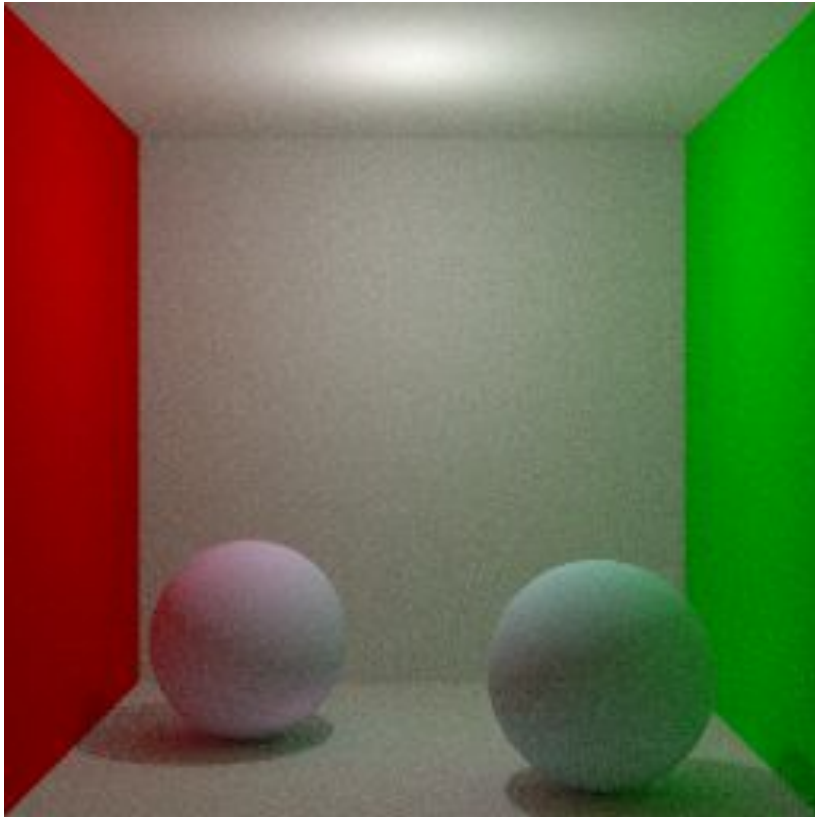
Informática Gráfica

Adolfo Muñoz - Julio Marco
Pablo Luesia - J. Daniel Subías – Óscar Pueyo

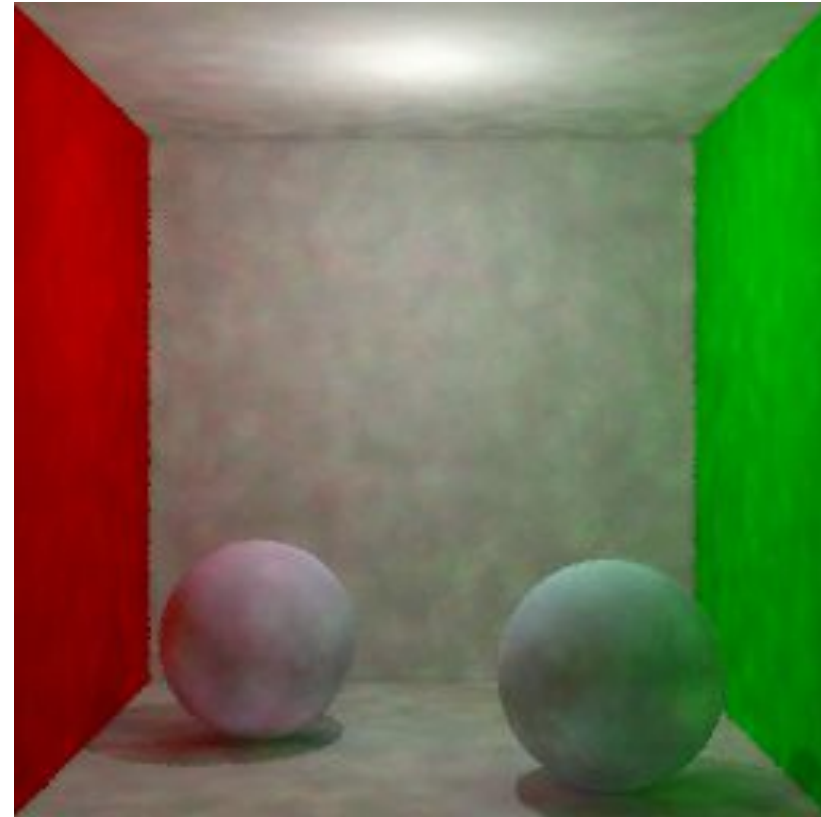


Before we begin...

- Today: render the first image with photon mapping



Path tracing



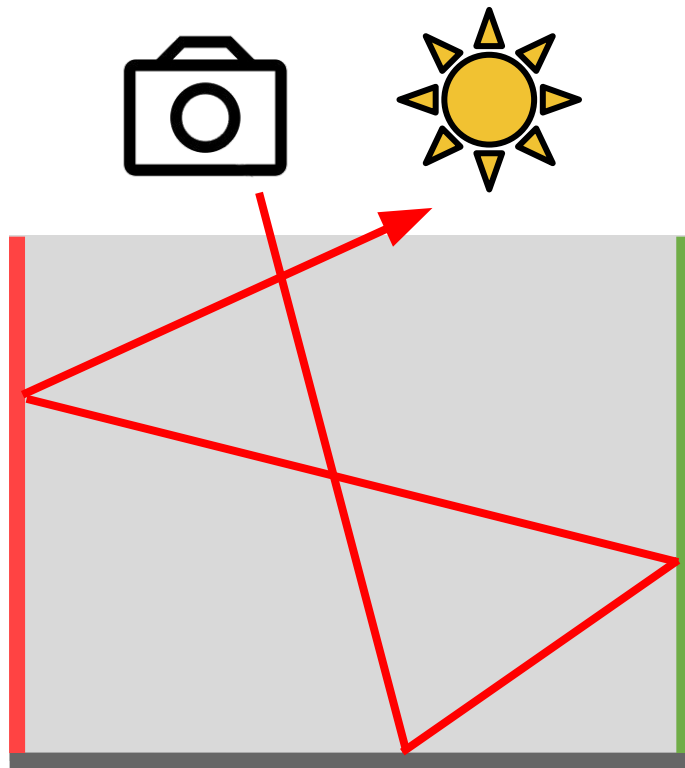
Photon mapping

Before we begin...

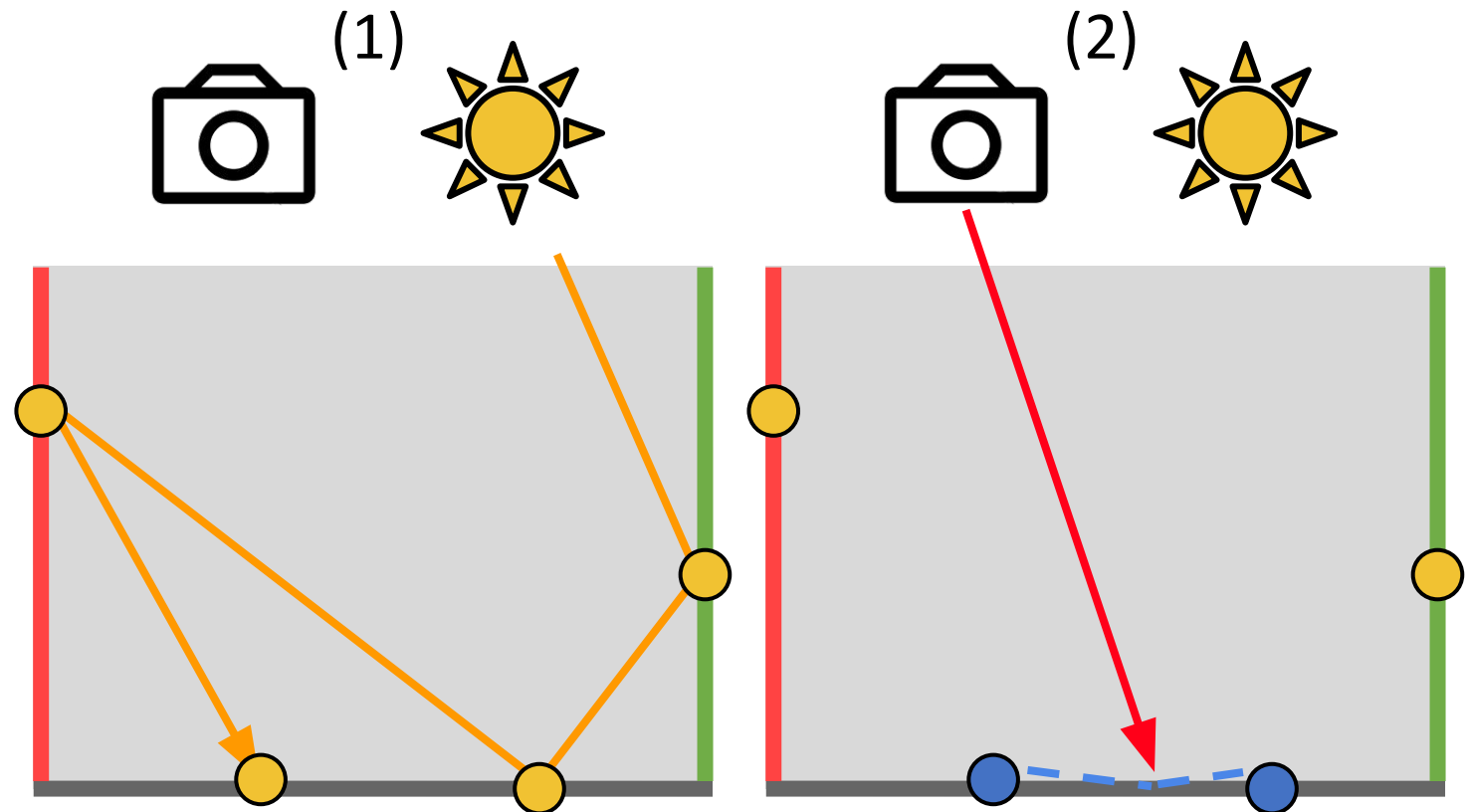
- Lab 5 (photon mapping) **is the second submitted work**
 - Recommended deadline: December 4th
 - Moodle: January 11th
- You can probably **reuse most of your code** for this assignment
- Remember: Final work is 80% of the final grade

Recap: photon mapping basics

- Photon mapping is a **two-pass** algorithm
 - Uses an **intermediate storage** known as the **photon map**



Lab 4 (path tracing)

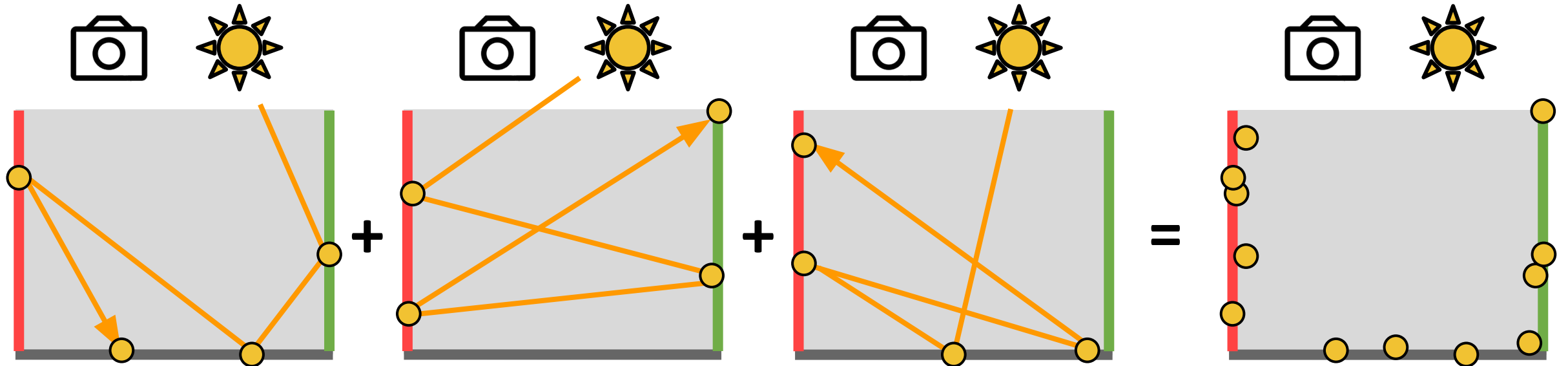


Lab 5 (photon mapping)

Recap: photon map construction

- Split a path into two:
 - (1) lights write into the photon map 
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)

Final photon map



Recap: photon map construction

The origin of the photons is the light source

- **Conservation of energy:**

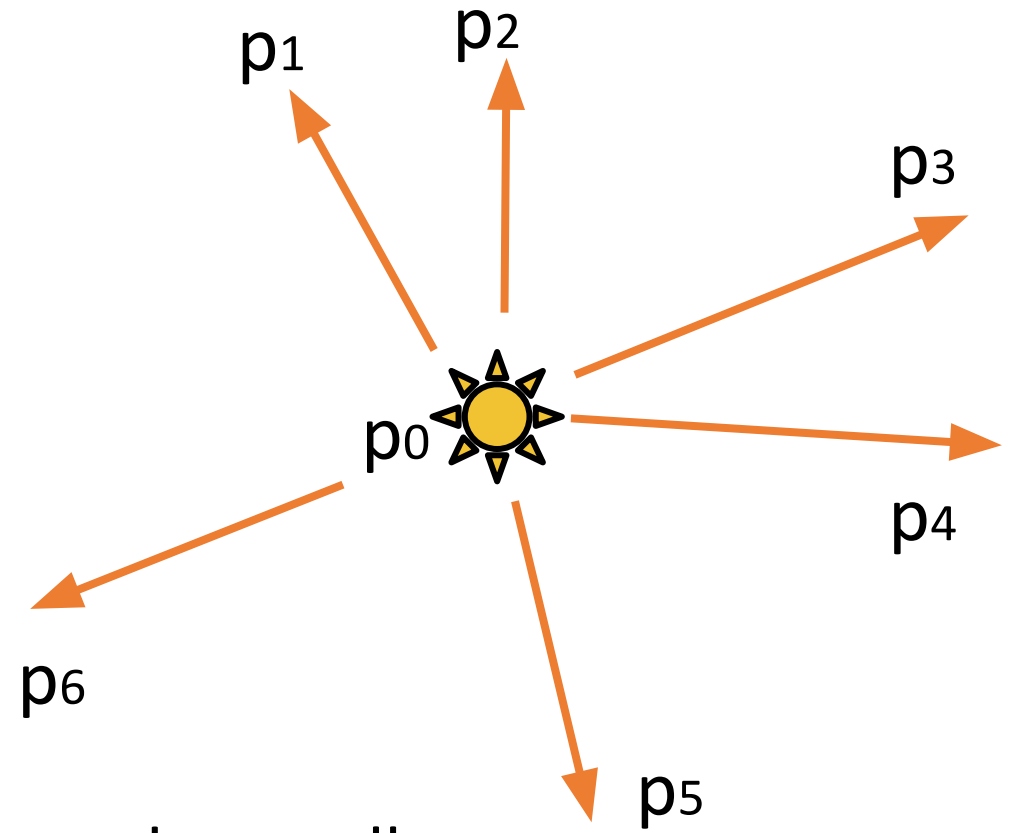
$$p_1 + p_2 + p_3 + p_4 + p_5 + p_6 = 4\pi p_0$$

The light source emitted S photons.

A **photon's flux** is $p_i = 4\pi p_0 / S$

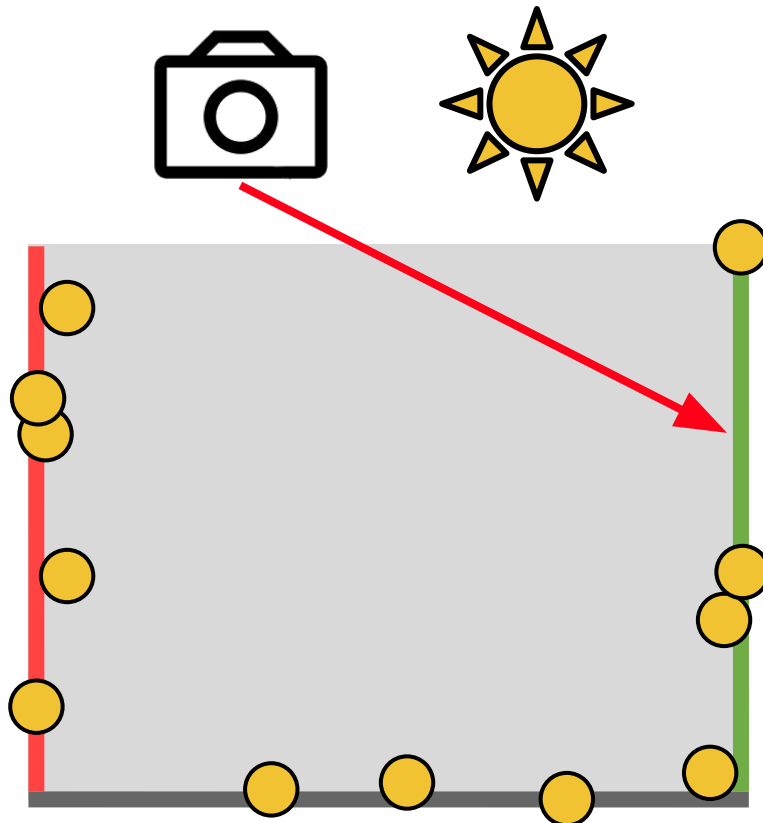
Careful with the photon map's size limit

You might need to update S after doing the random walks



Recap: how the photon map is used

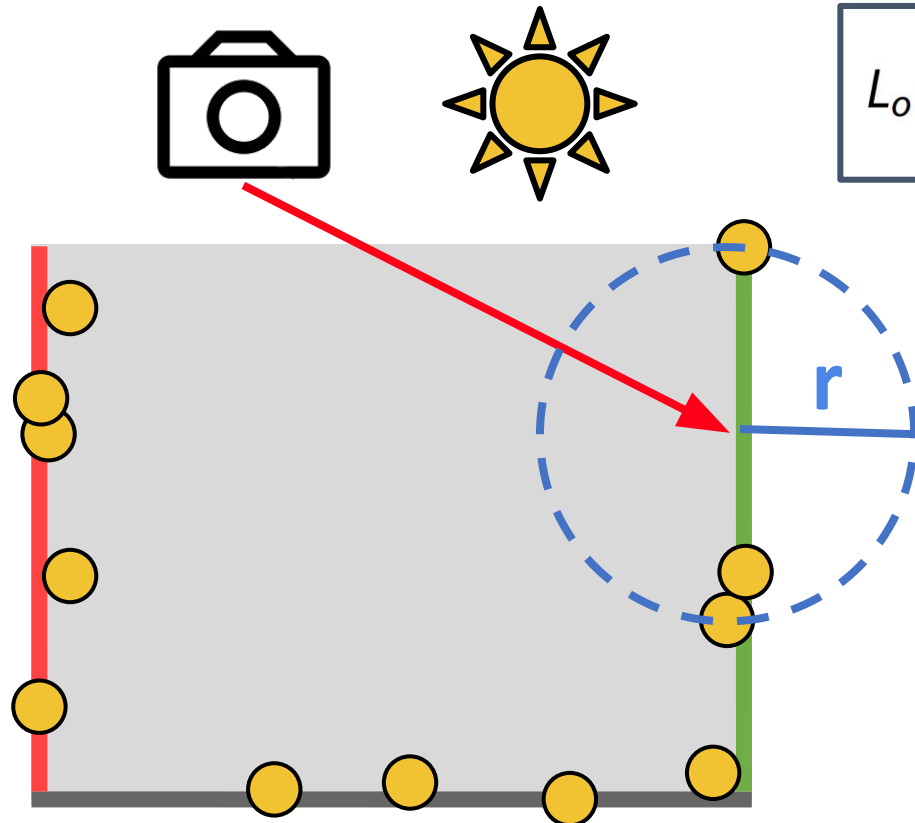
- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map ←



$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Recap: how the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map 

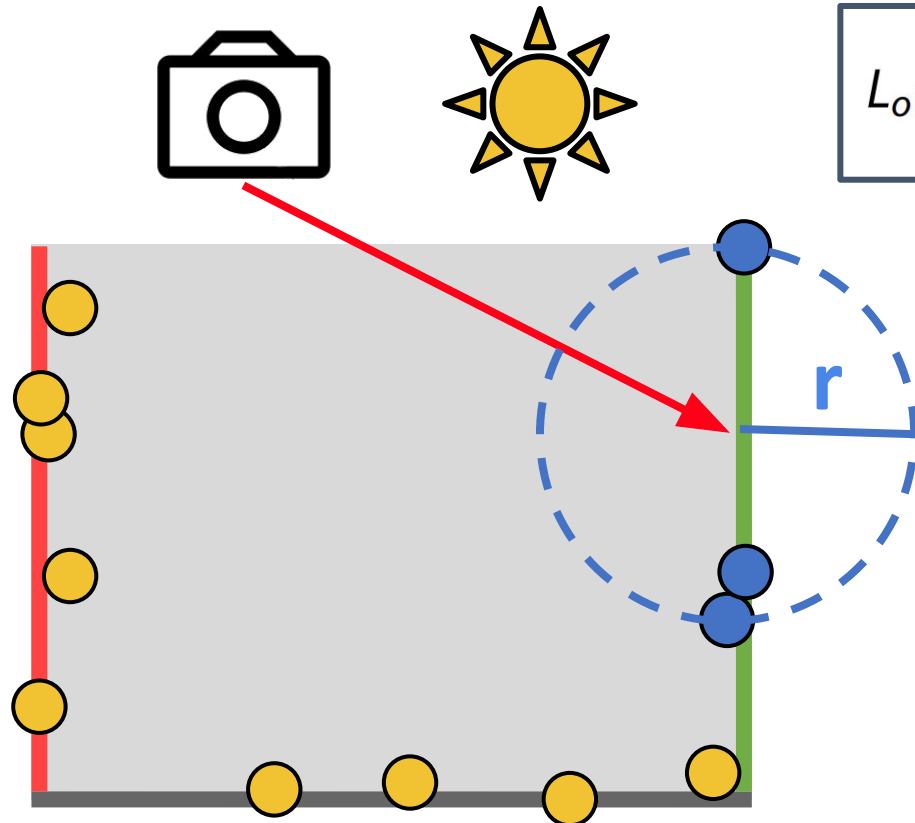


$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

- Idea: use **k nearby** photons (distance $< r$) to approximate the Render Equation

Recap: how the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map 

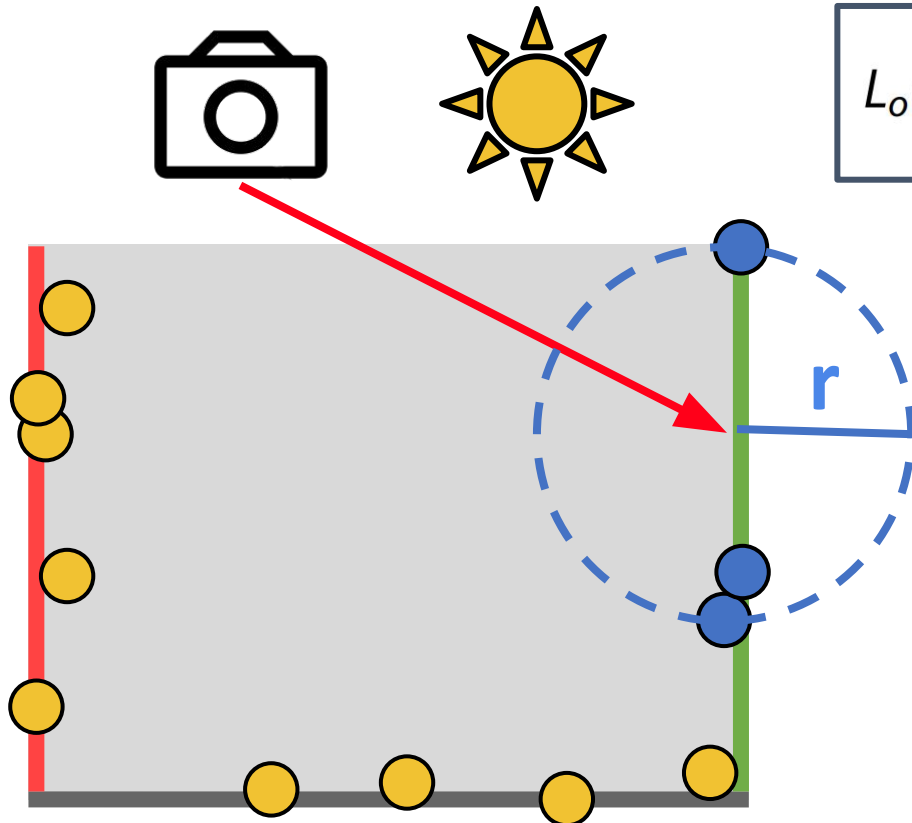


$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

- Idea: use **k nearby** photons (distance < r) to approximate the Render Equation

Recap: how the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map ←

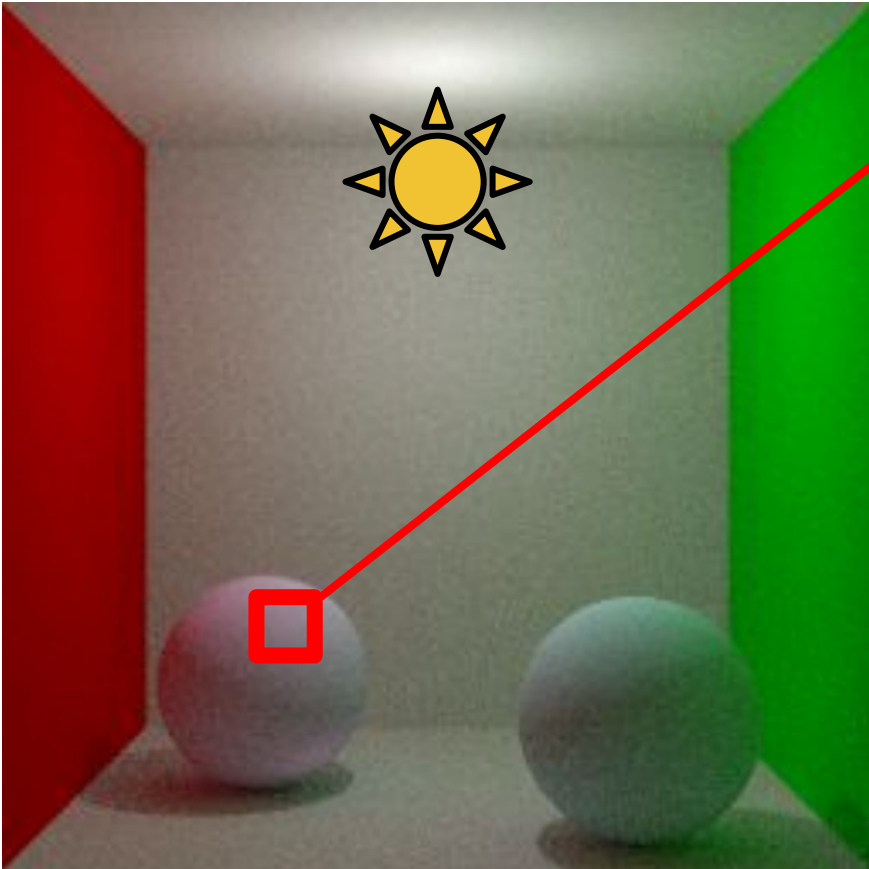


$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

- Idea: use **k nearby** photons (distance < r) to approximate the Render Equation
- Integral is approximated as the **sum of k = 3 contributions**

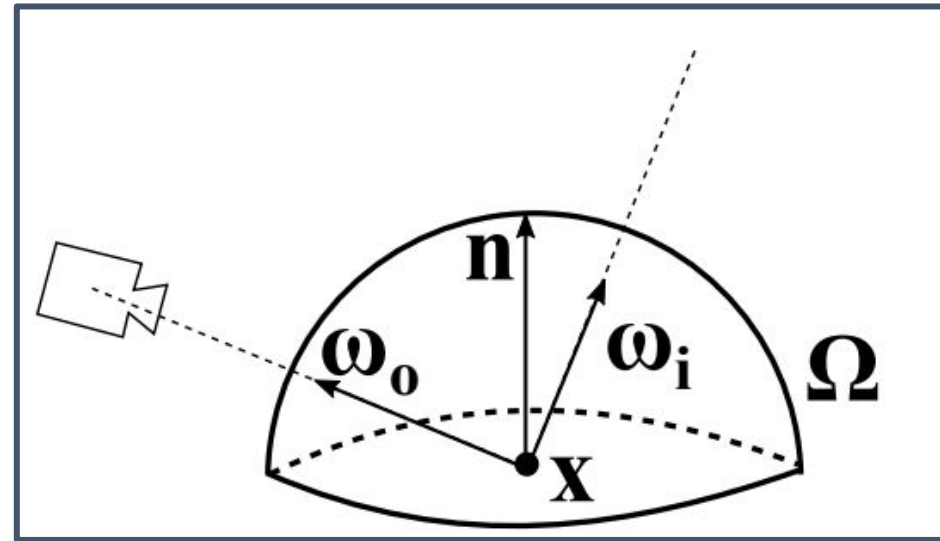
Which color do we fill each pixel with?

- For path tracing:



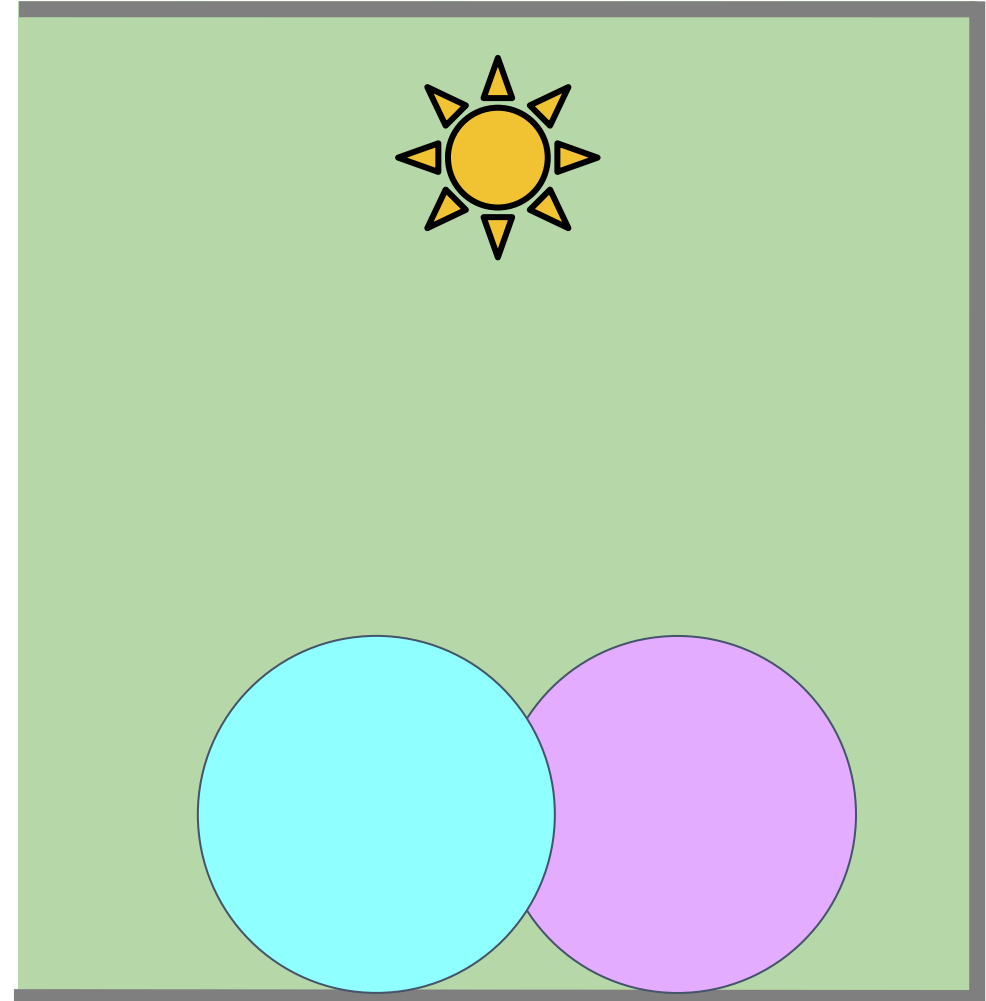
$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

The full integral



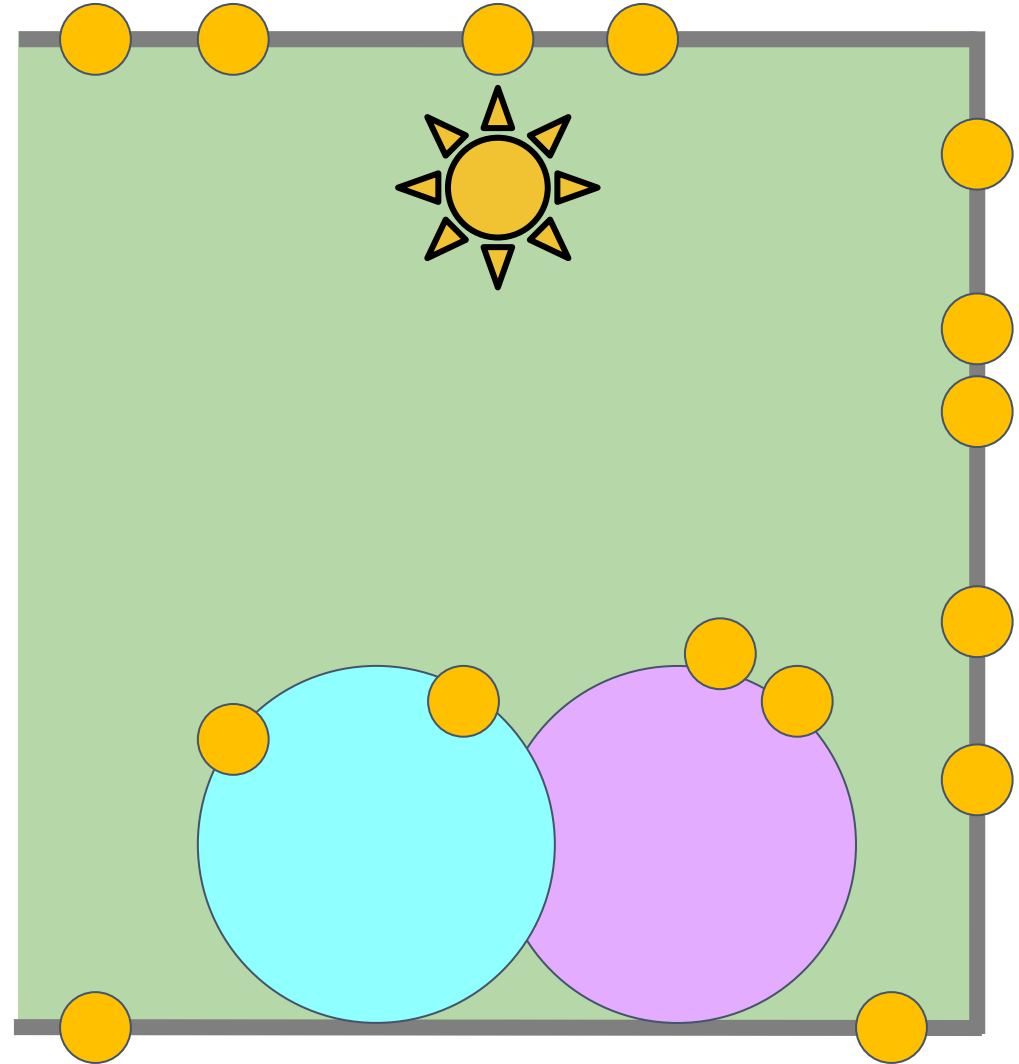
Which color do we fill each pixel with?

- For photon mapping:



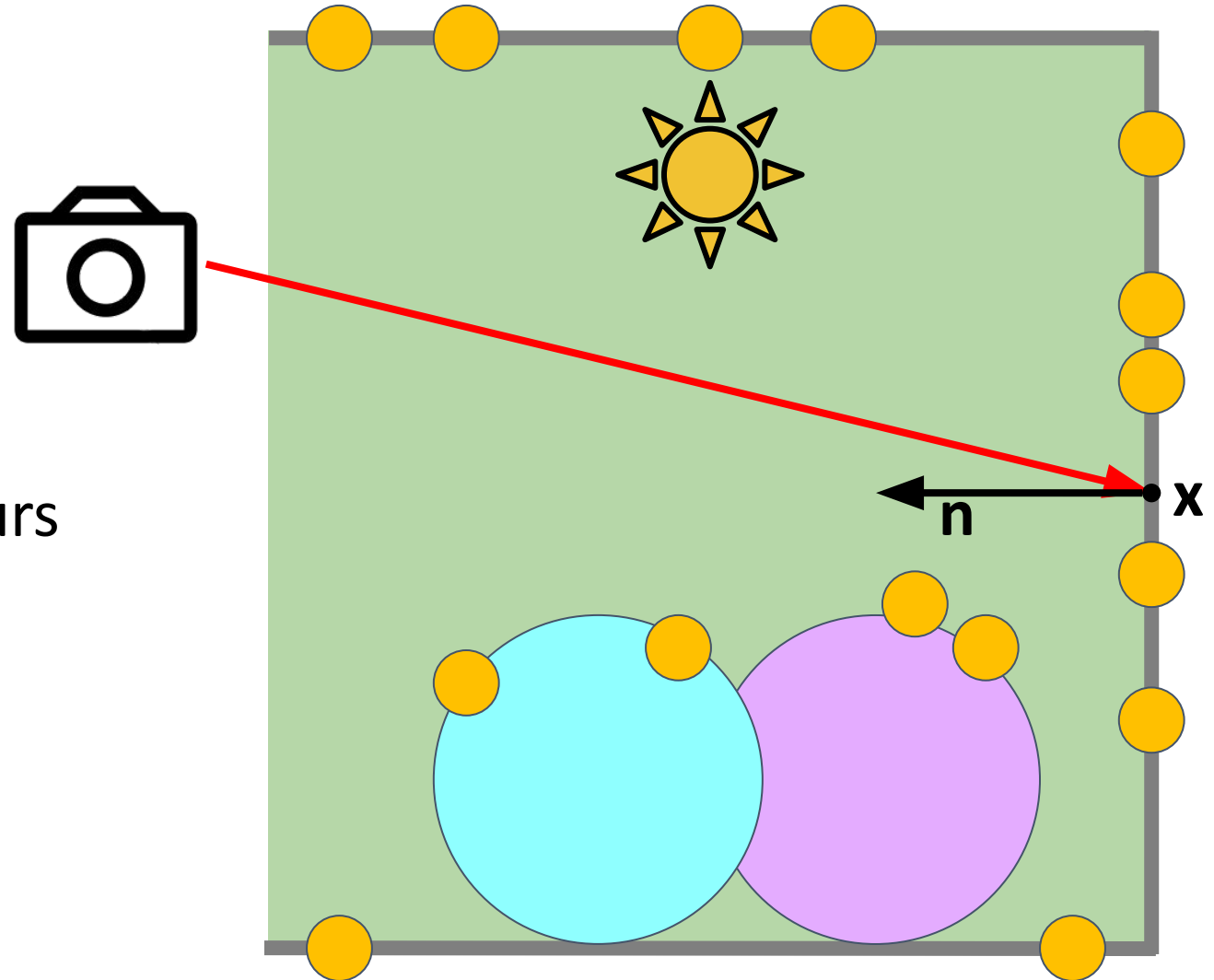
Which color do we fill each pixel with?

- For photon mapping:
1) Calculate the photon map
(previous session)



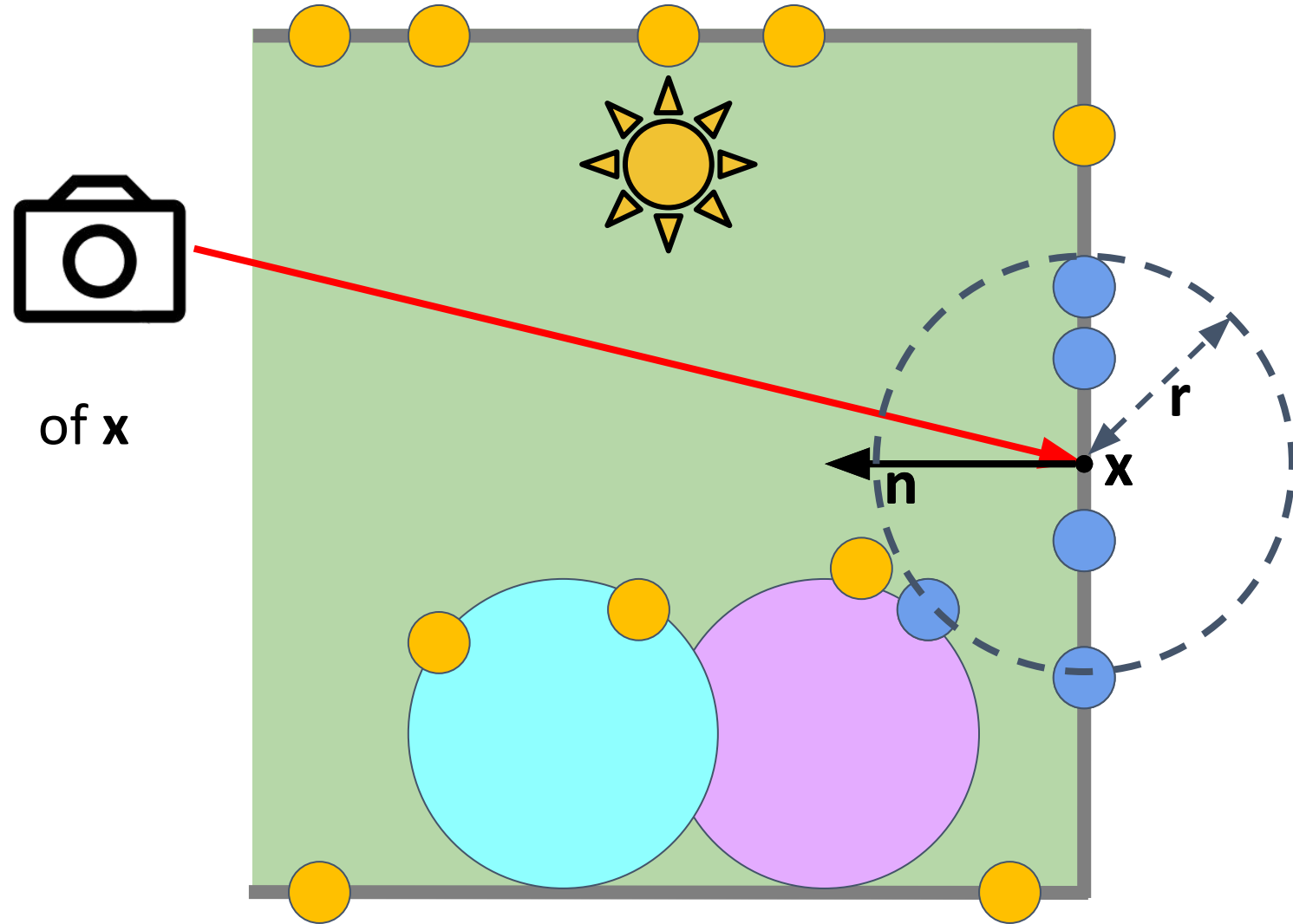
Which color do we fill each pixel with?

- For photon mapping:
 - 2) Launch rays from the camera (**same as path tracing**)
- Intersect at point $\mathbf{x}$
- Search for nearest neighbours of $\mathbf{x}$ in the photon map:



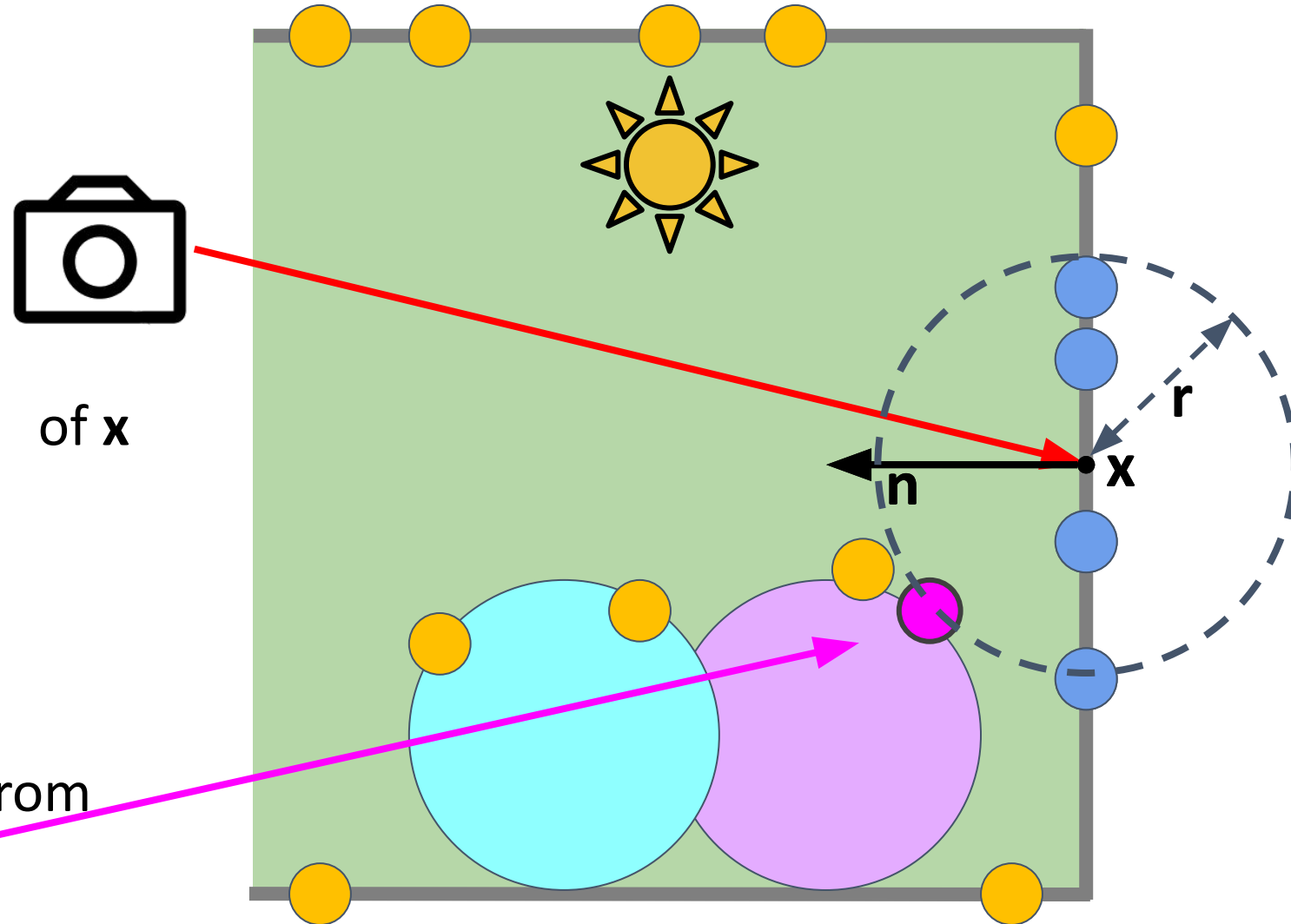
Which color do we fill each pixel with?

- For photon mapping:
 - 2) Launch rays from the camera (**same as path tracing**)
- Intersect at point $\mathbf{x}$
- Search for nearest neighbours of $\mathbf{x}$ in the photon map:
 - a) k nearest photons
 - b) photons at distance $\leq r$



Which color do we fill each pixel with?

- For photon mapping:
 - 2) Launch rays from the camera (**same as path tracing**)
 - Intersect at point x
 - Search for nearest neighbours of x in the photon map:
 - a) k nearest photons
 - b) photons at distance $\leq r$
 - Note how some photons are from **other surfaces**
- (you can try to solve this, or not)

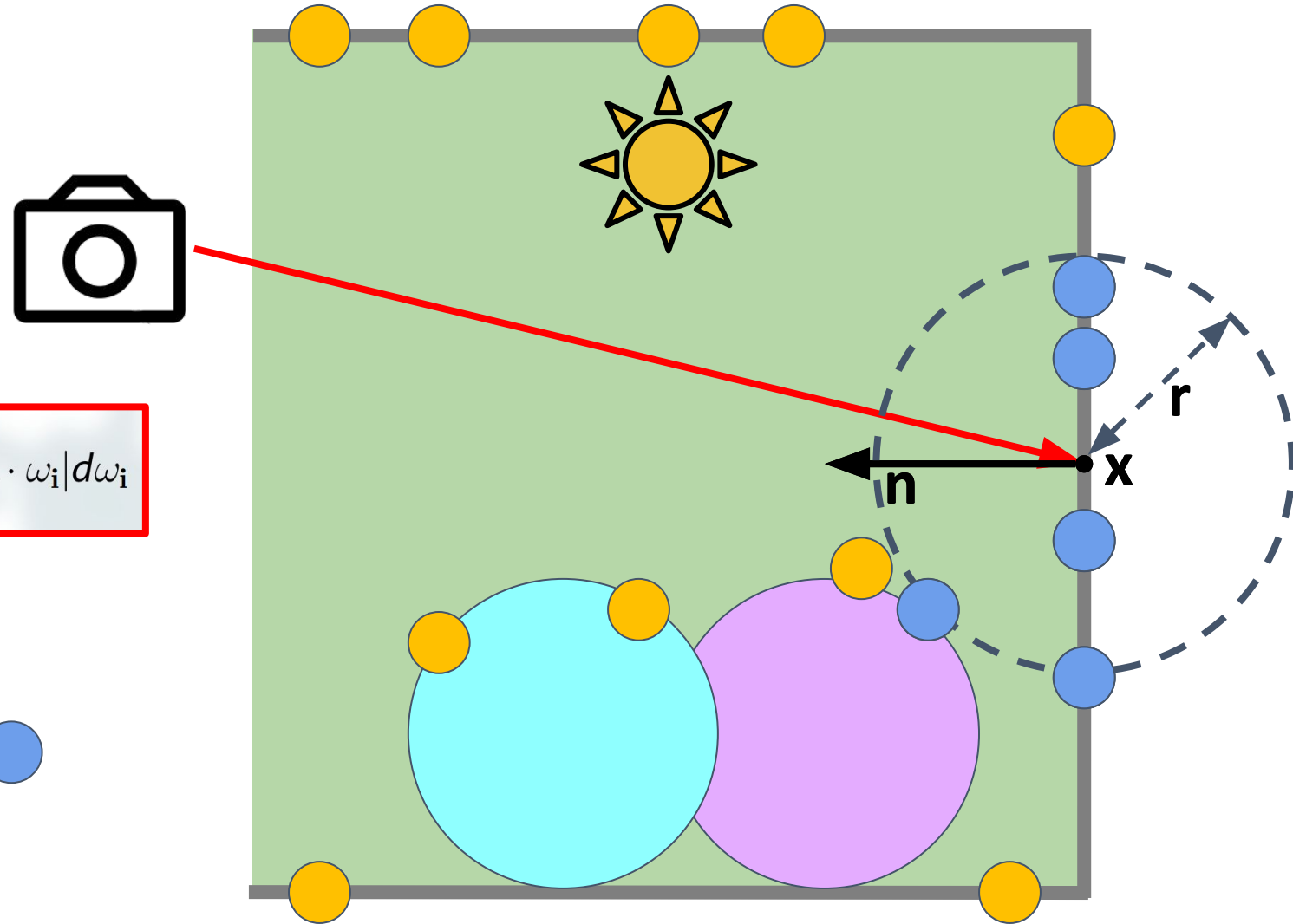
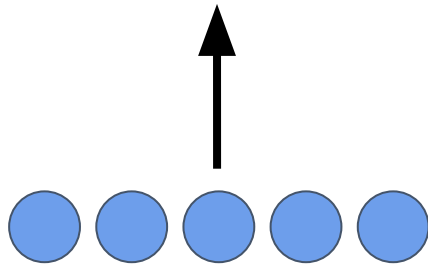


Which color do we fill each pixel with?

- For photon mapping:

3) Estimate color of the pixel with nearby photons

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

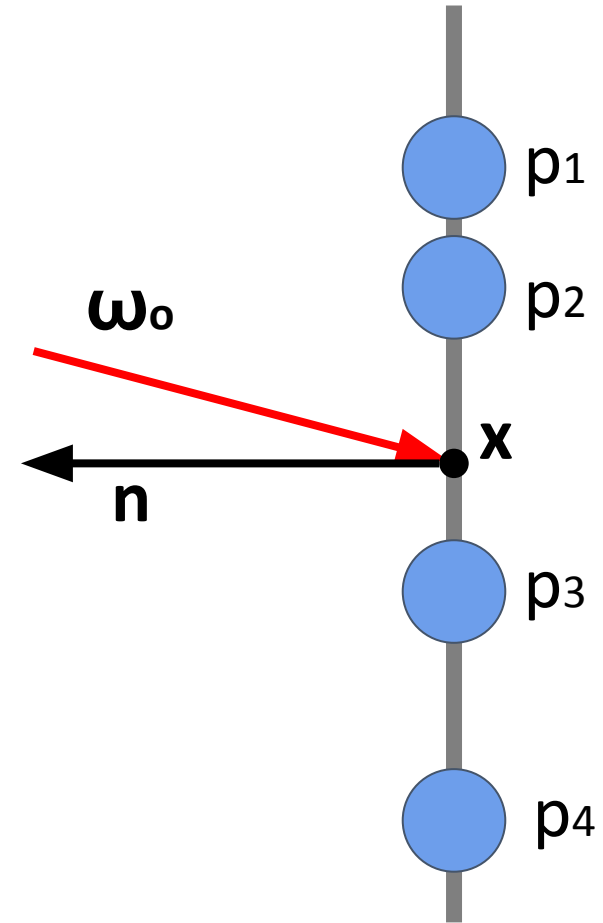


Kernel density estimation

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Each photon p contains:

- Φ_p flux
- $\mathbf{x}_p$ position
- ω_p direction



Kernel density estimation

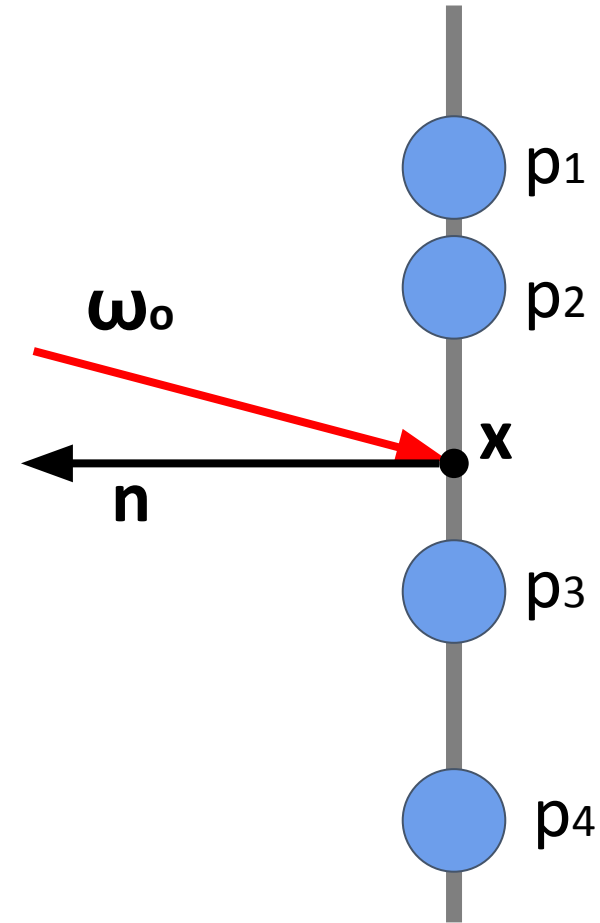
$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

Each photon p contains:

- Φ_p flux
- $\mathbf{x}_p$ position
- ω_p direction

Constant density estimation: box kernel

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$



Kernel density estimation

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

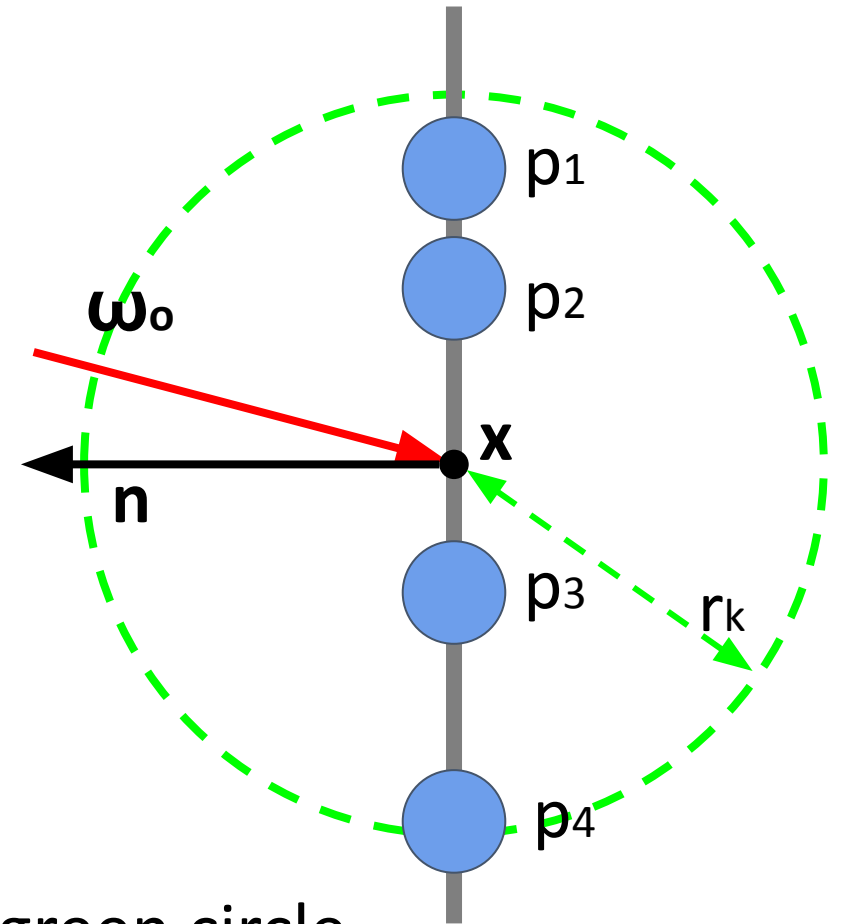
Each photon p contains:

- Φ_p flux
- $\mathbf{x}_p$ position
- ω_p direction

Constant density estimation: box kernel

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$

area of the green circle
 r_k is the radius that contains all photons



Kernel density estimation

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

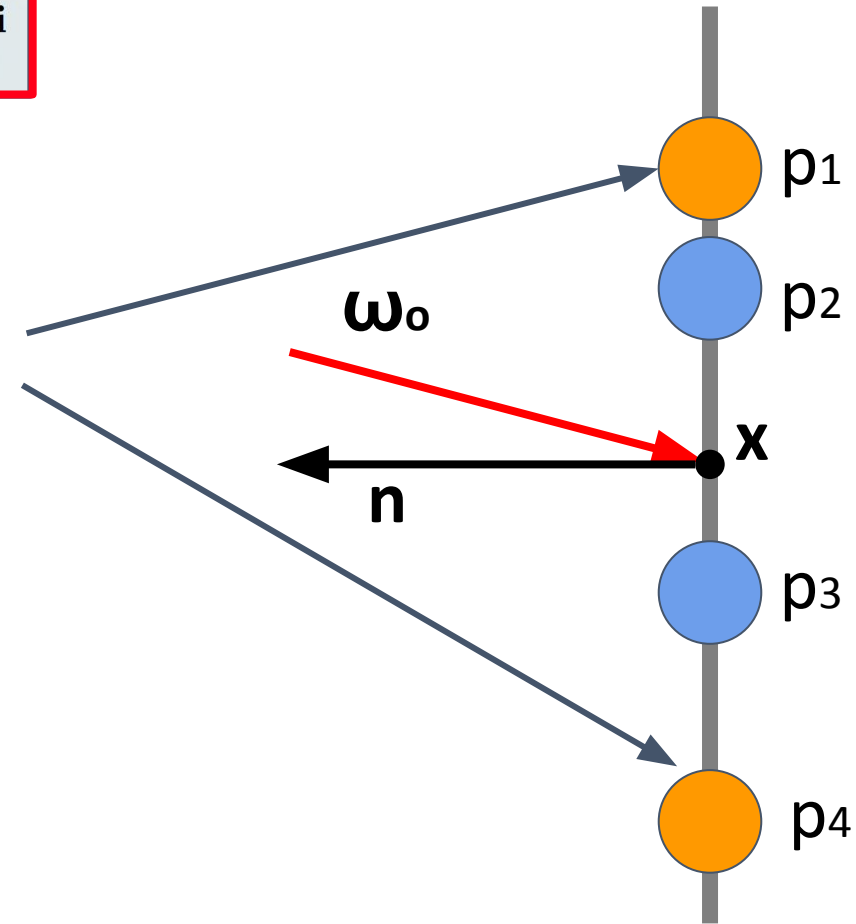
Each photon p contains:

- Φ_p flux
- $\mathbf{x}_p$ position
- ω_p direction

Far away from the intersection point

Constant density estimation: box kernel

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$



Kernel density estimation

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

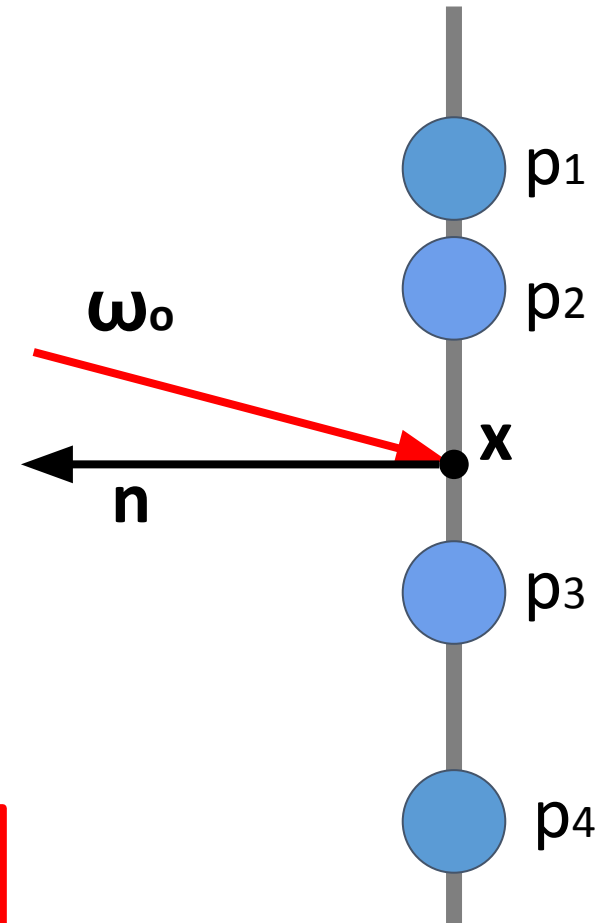
Each photon p contains:

- Φ_p flux
- $\mathbf{x}_p$ position
- ω_p direction

Non-constant density estimation (optional)

Gives more weight to photons closer to $\mathbf{x}$

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \Phi_p \underline{K_{2D}(|\mathbf{x} - \mathbf{x}_p|, r_k)}$$



Kernel density estimation

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

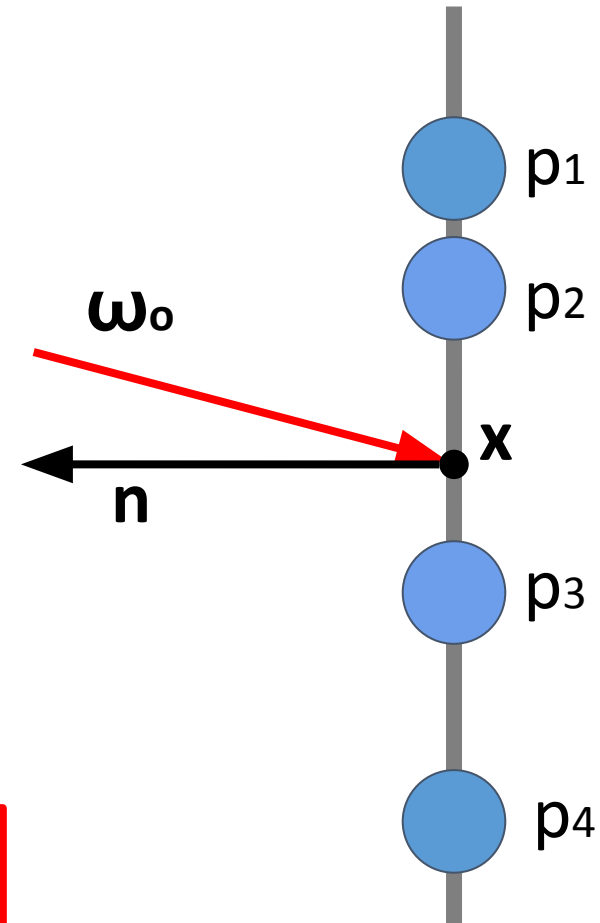
Each photon p contains:

- Φ_p flux
- $\mathbf{x}_p$ position
- ω_p direction

Non-constant density estimation (optional)

Gives more weight to photons closer to $\mathbf{x}$

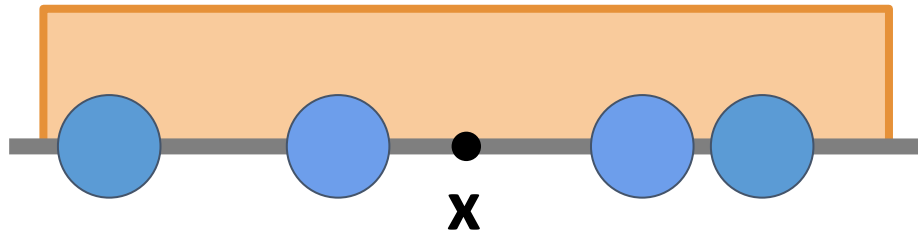
$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \Phi_p K_{2D}(\|\mathbf{x} - \mathbf{x}_p\|, r_k)$$



Kernel density estimation

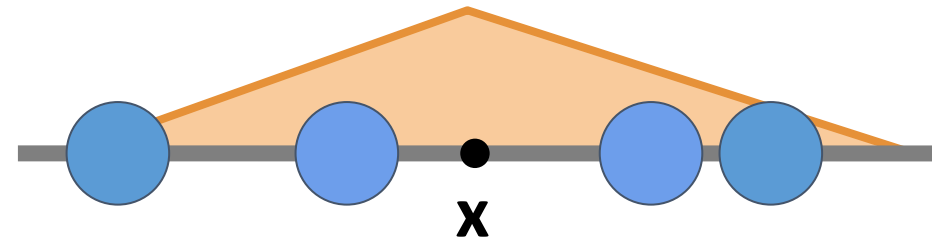
Constant density estimation

Box kernel

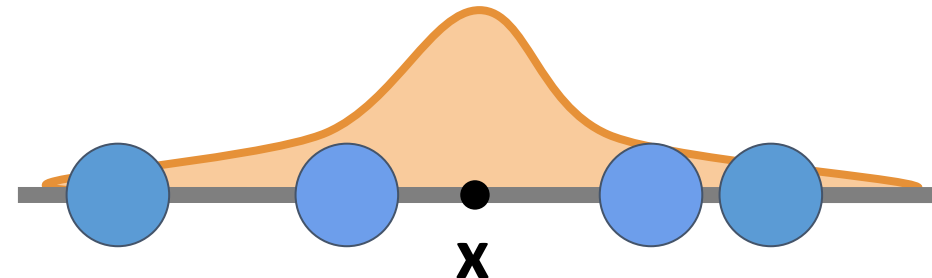


Non-constant density estimation

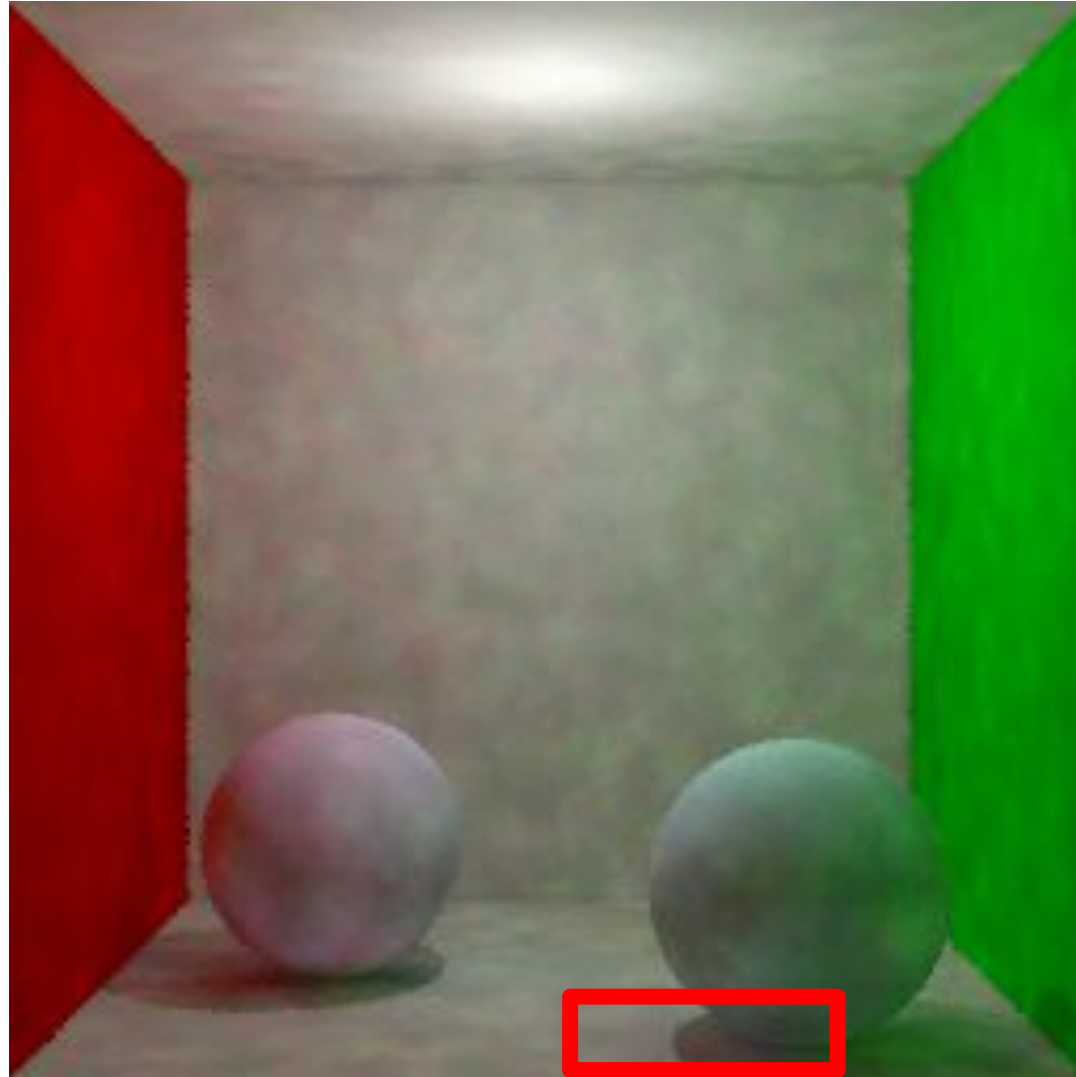
Cone kernel



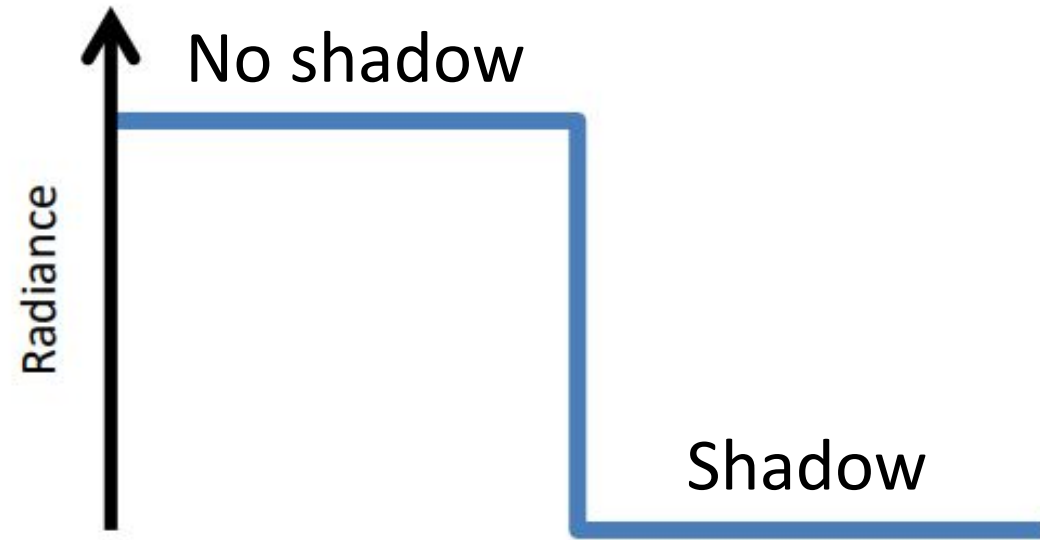
Gaussian kernel



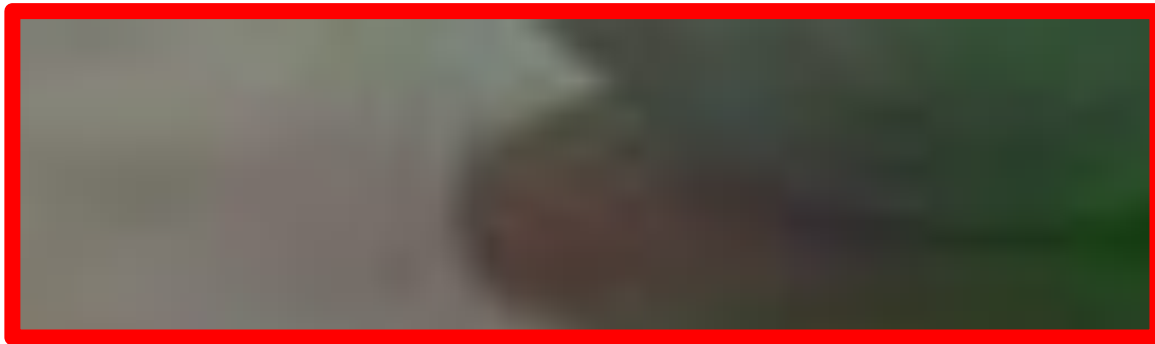
Photon mapping is biased



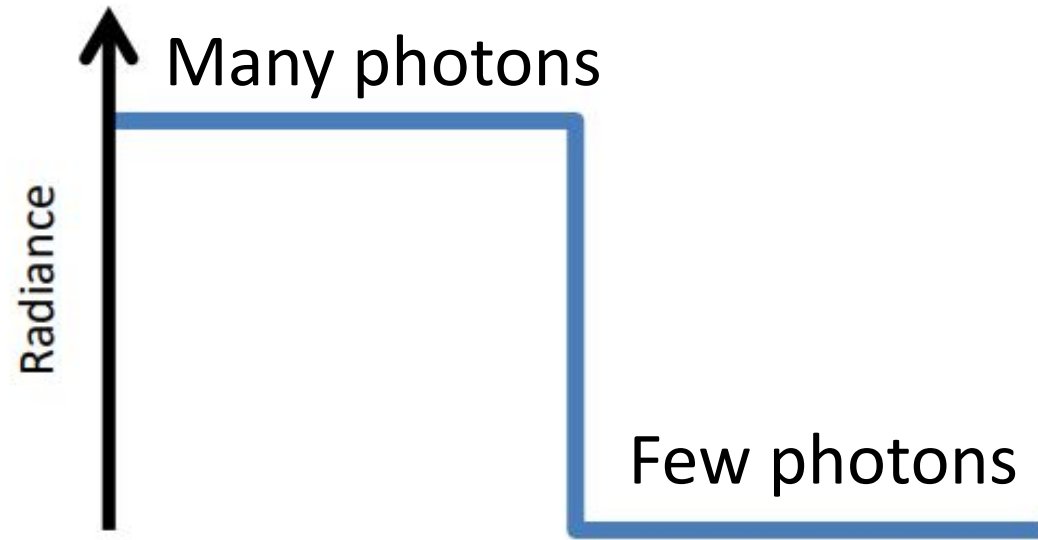
Photon mapping is biased



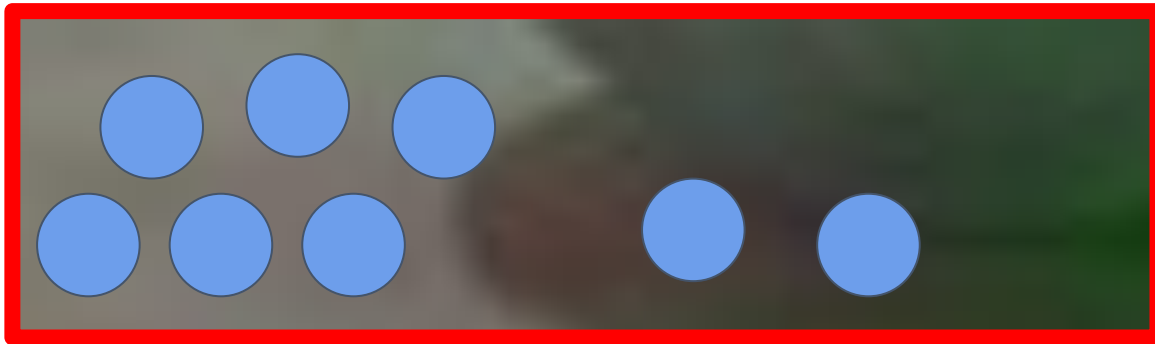
Example: area with hard shadows



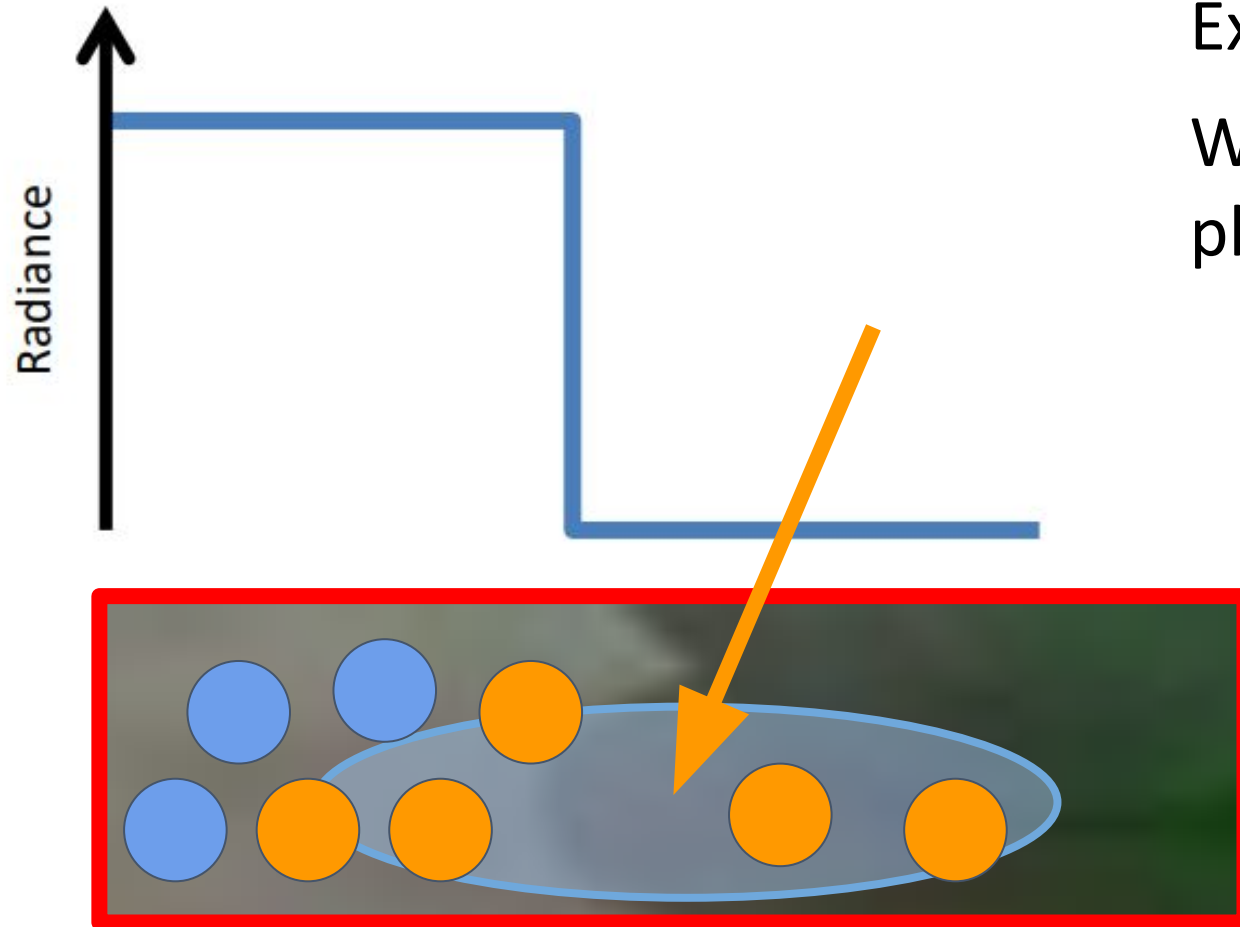
Photon mapping is biased



Example: area with hard shadows



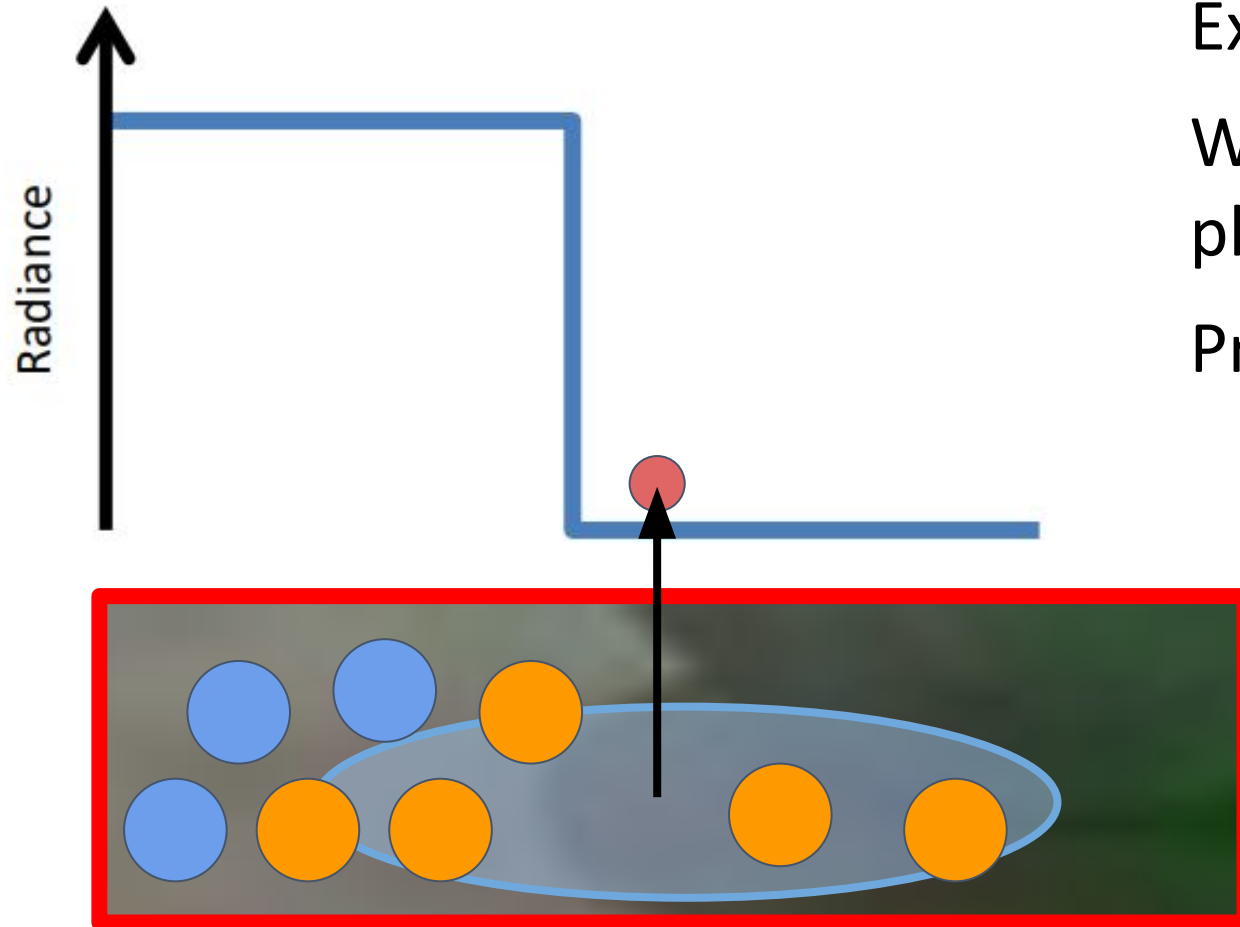
Photon mapping is biased



Example: area with hard shadows

When a ray intersects, it picks up photons from both areas

Photon mapping is biased

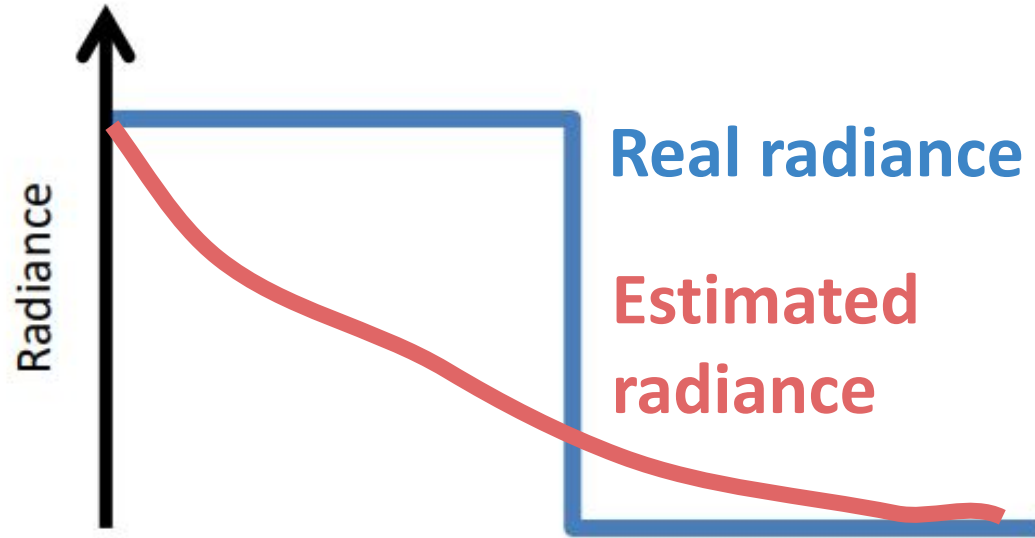


Example: area with hard shadows

When a ray intersects, it picks up photons from both areas

Produces a **biased estimation**

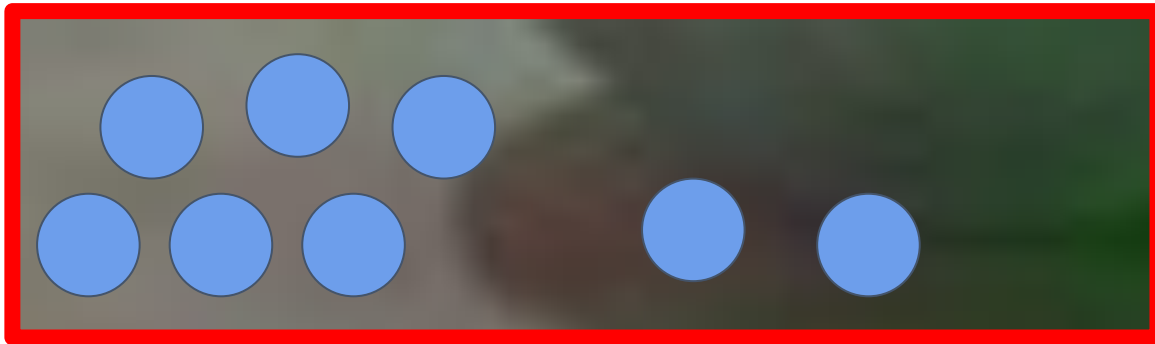
Photon mapping is biased



Example: area with hard shadows

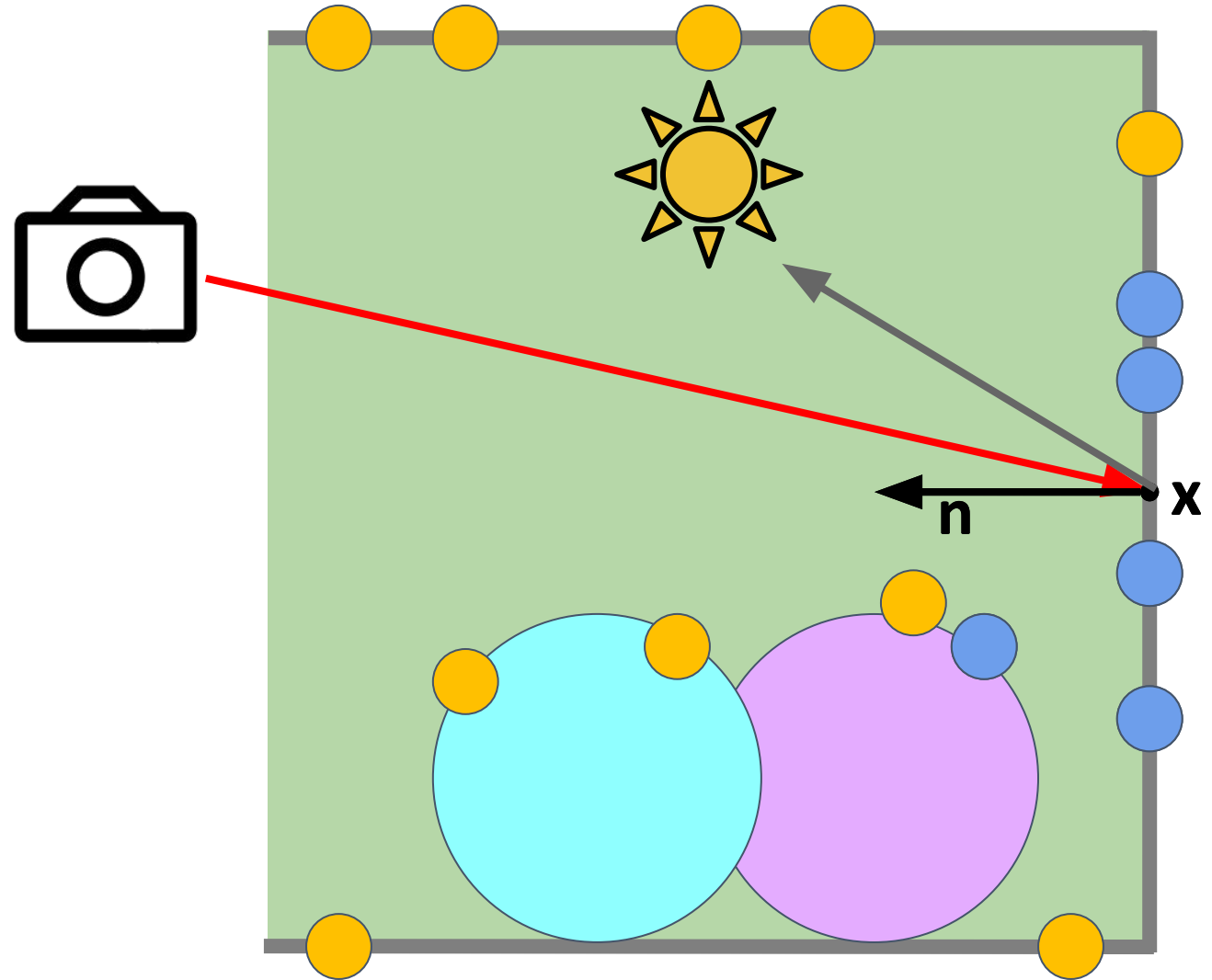
Estimated result is **biased**

(This did not happen for path tracing)



How to compute direct light

We can compute direct light
without using a photon map:

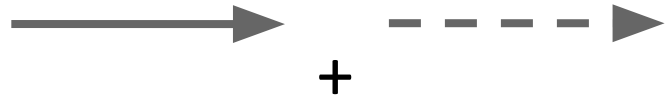


How to compute direct light

We can compute direct light
without using a photon map:

However

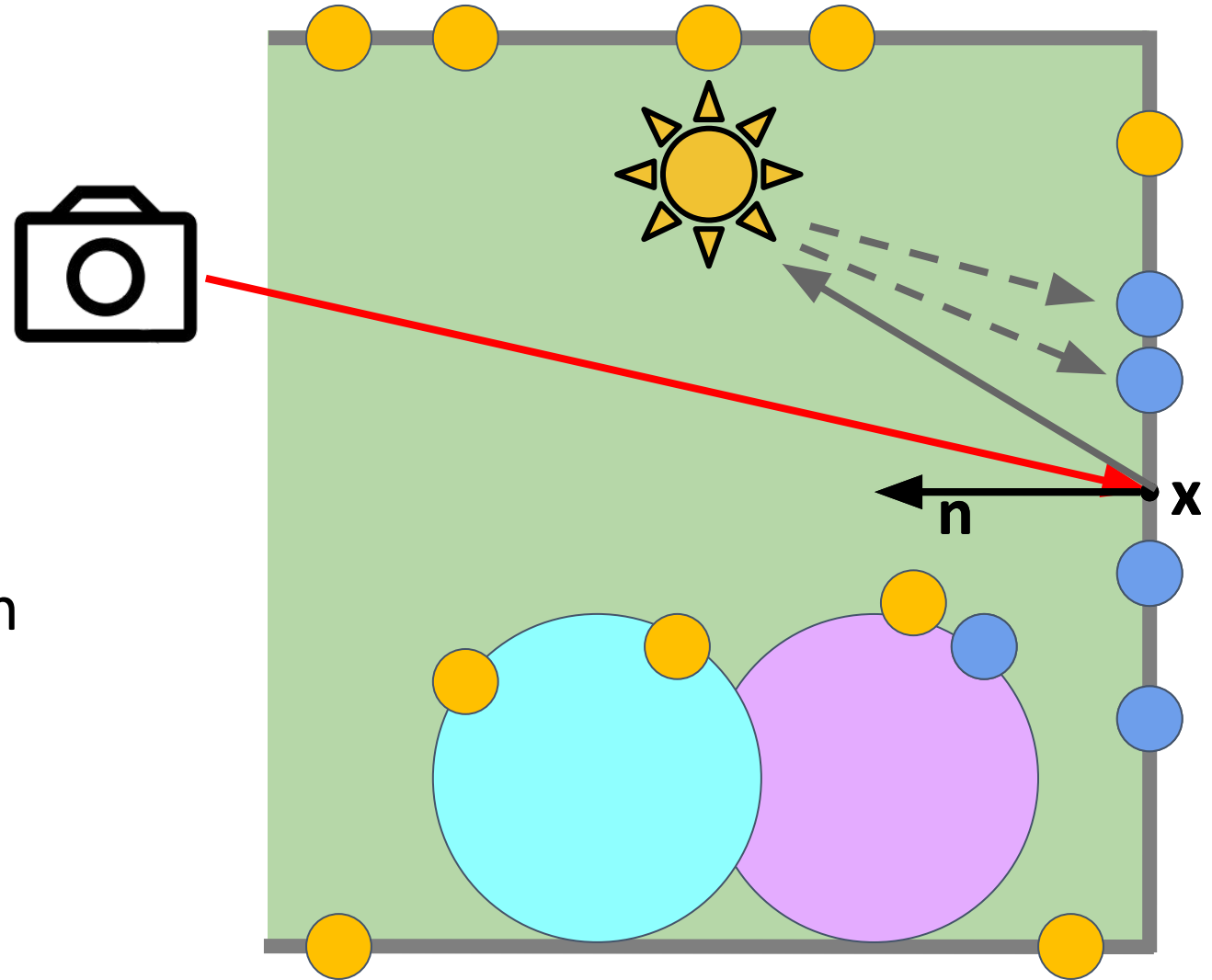
You are duplicating energy



Next-event estimation



First bounce photons

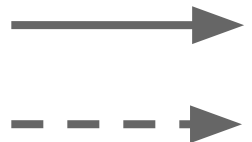


How to compute direct light

We can compute direct light
without using a photon map:

However

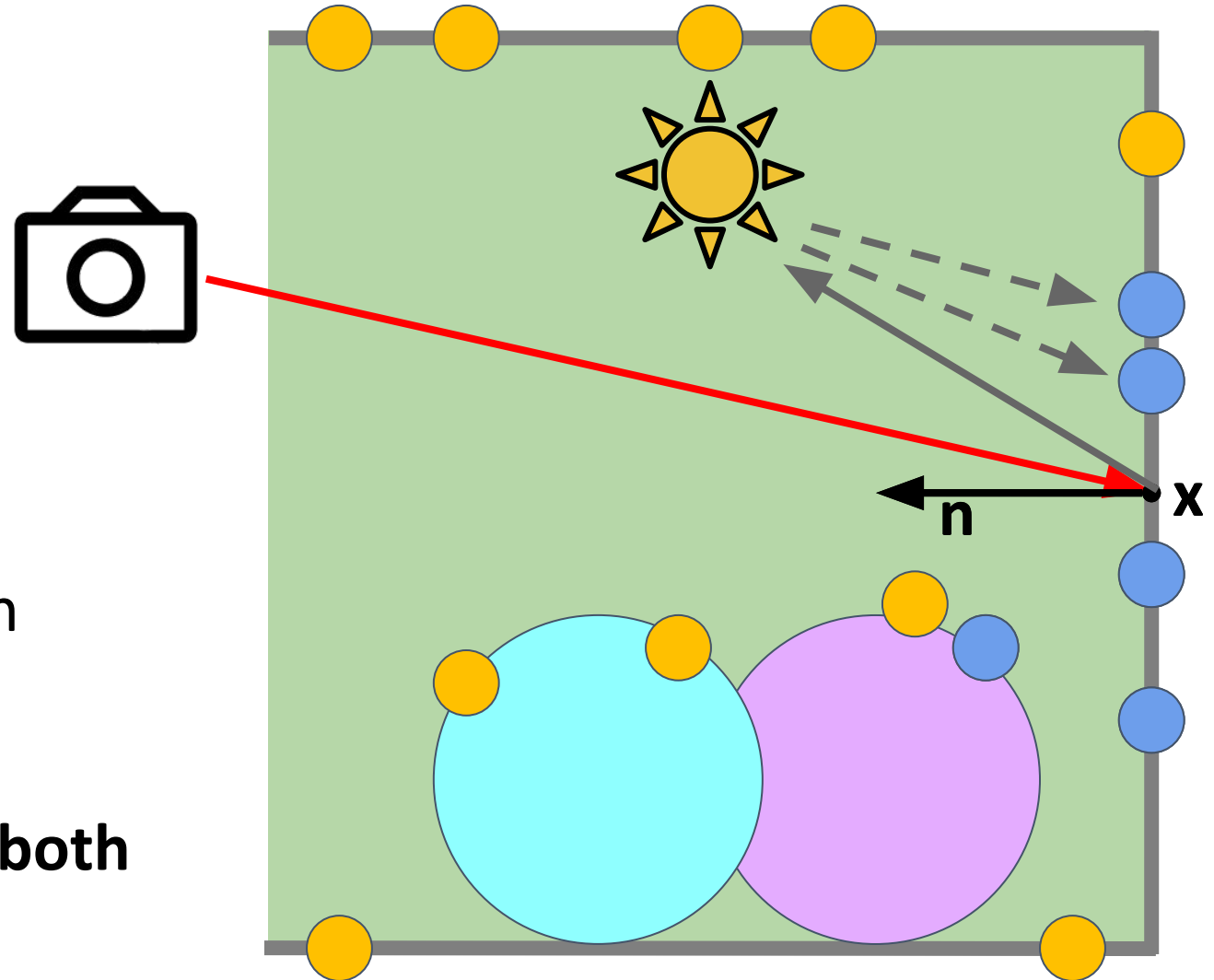
You are duplicating energy



Next-event estimation

First bounce photons

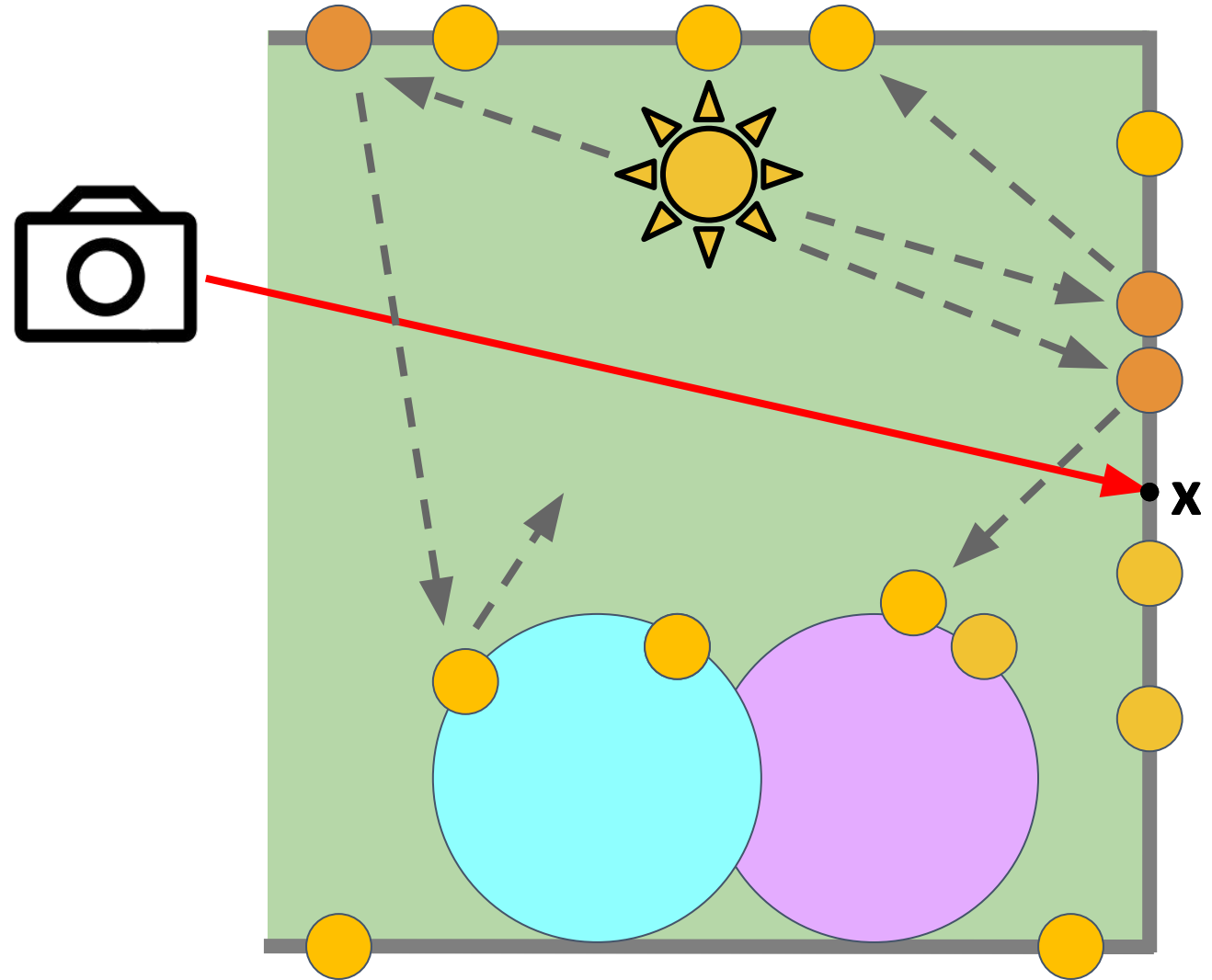
Only choose one method, not both



How to compute direct light

Method 1:

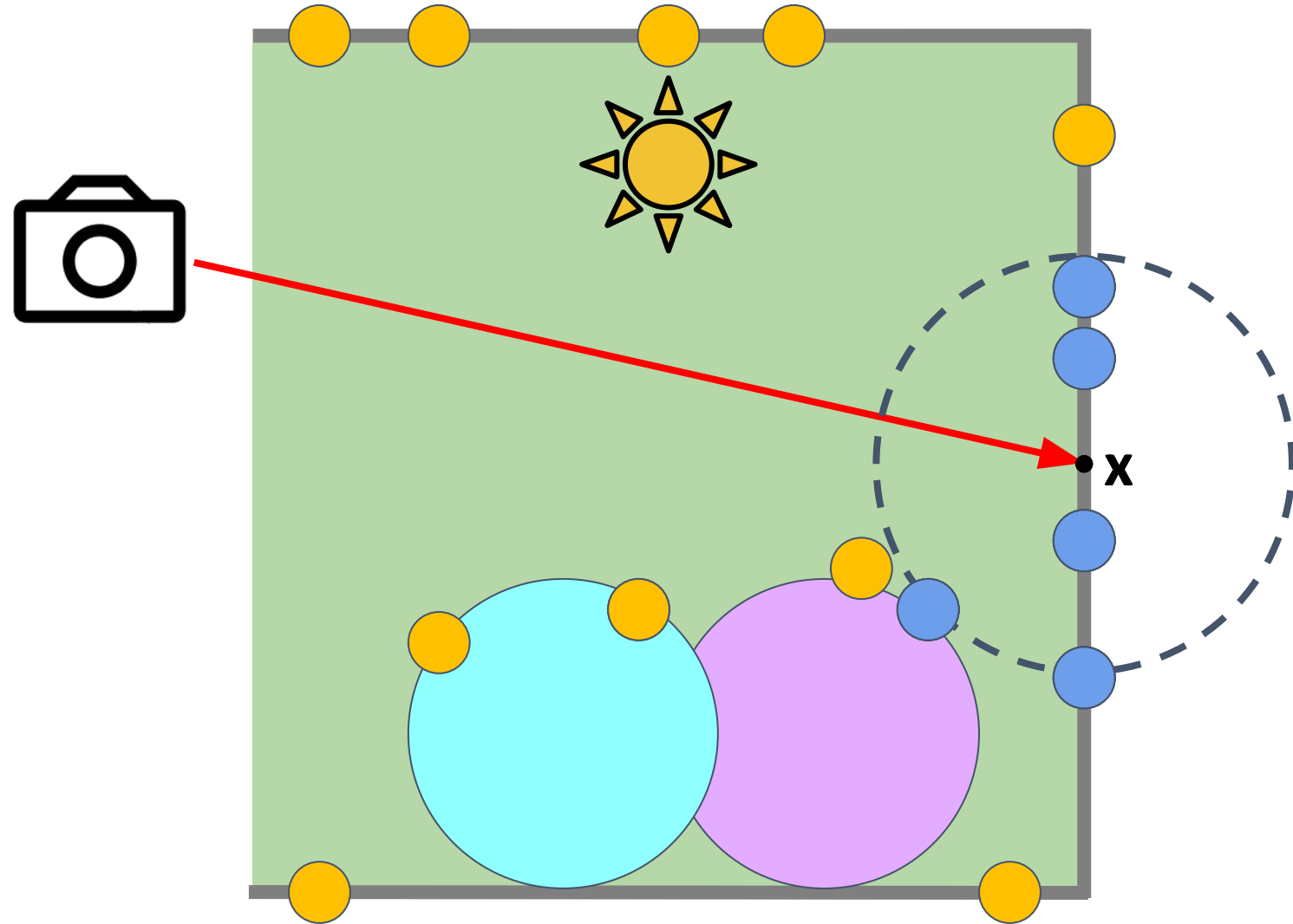
- When generating the photon map, store photons on every diffuse interaction, including the first one



How to compute direct light

Method 1:

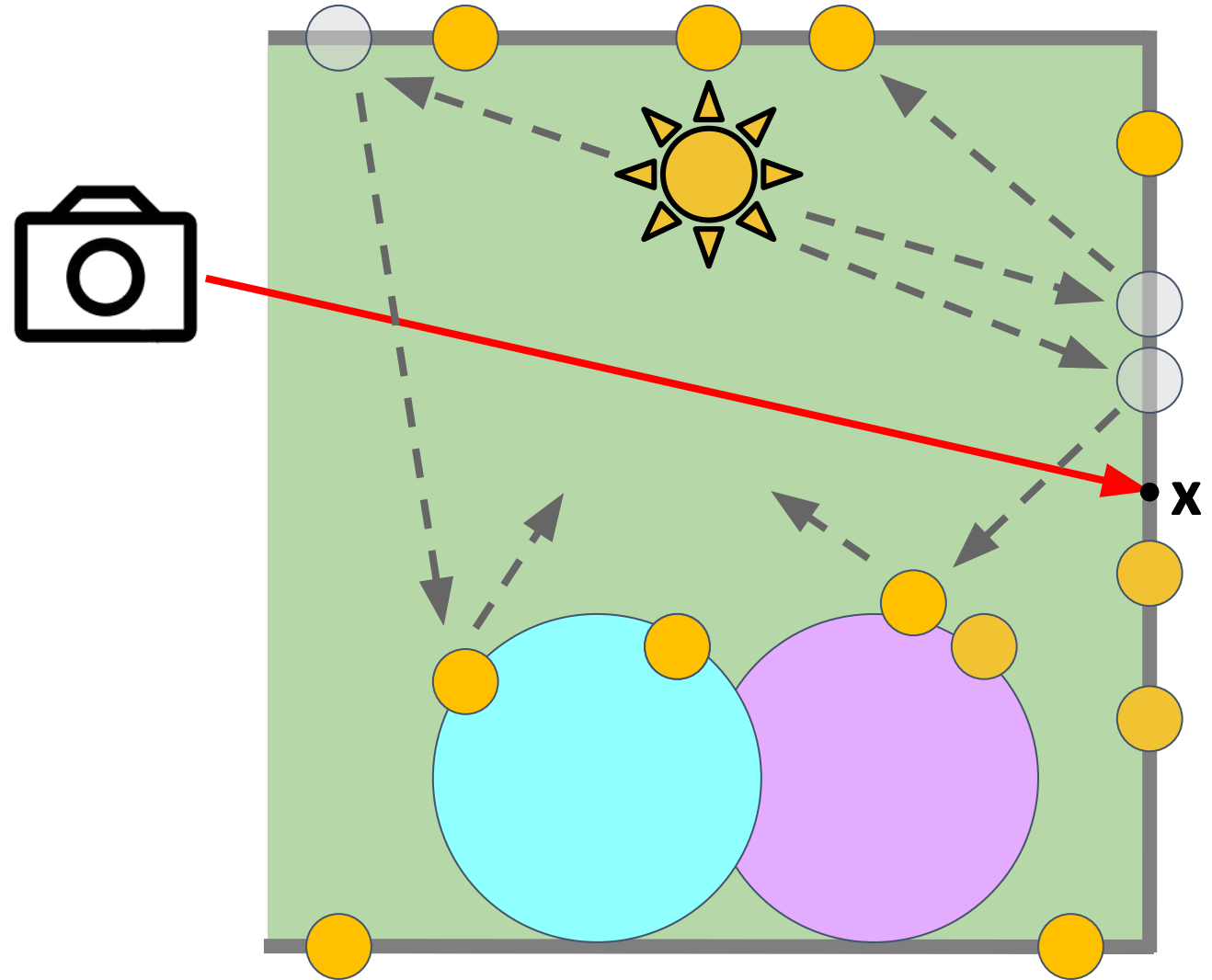
- When generating the photon map, store photons on every diffuse interaction, including the first one
- Later, compute direct light using kernel density estimation



How to compute direct light

Method 2:

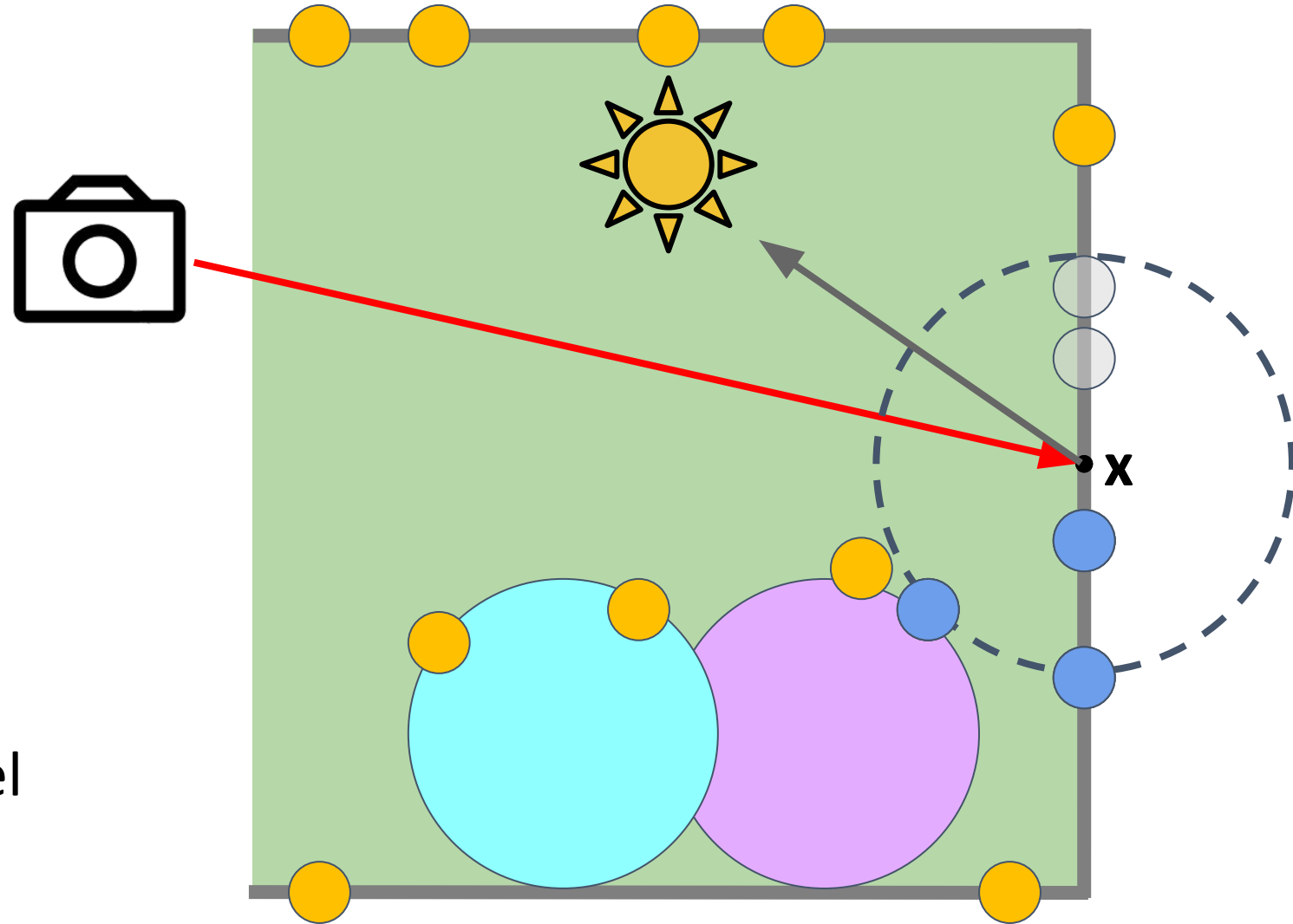
- When generating the photon map, do not store the first bounce (direct) photons



How to compute direct light

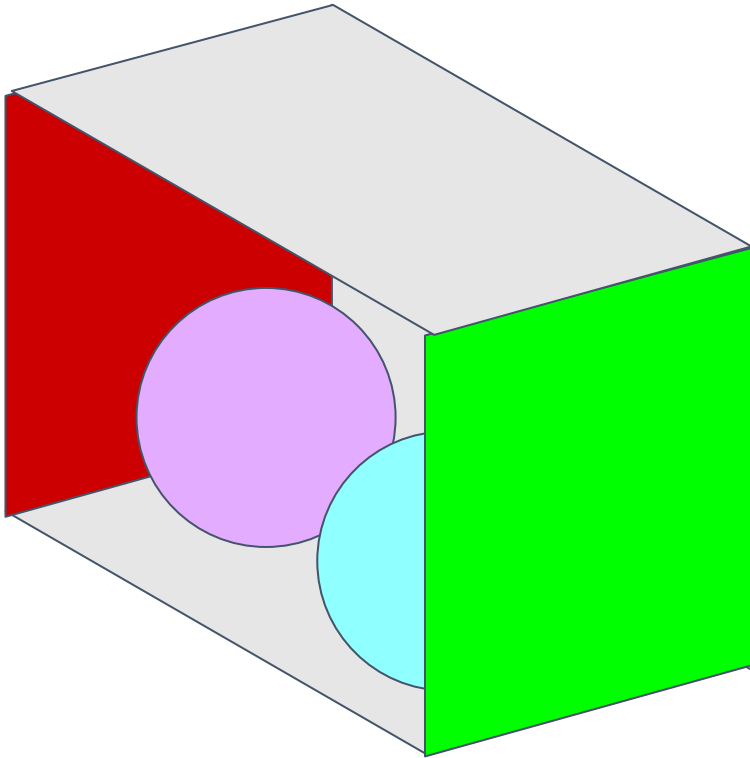
Method 2:

- When generating the photon map, do not store the first bounce (direct) photons
- Later, compute direct light using next-event estimation and indirect light with kernel density estimation



Example scene: Cornell Box

- **Geometry**



Planes defined by normal (n) and distance (d)

Left plane $n = (1, 0, 0)$, $d = 1$

Right plane $n = (-1, 0, 0)$, $d = 1$

Floor plane $n = (0, 1, 0)$, $d = 1$

Ceiling plane $n = (0, -1, 0)$, $d = 1$

Back plane $n = (0, 0, -1)$, $d = 1$

Spheres defined by center (c) and radius (r)

Left sphere $c = (-0.5, -0.7, 0.25)$, $r = 0.3$

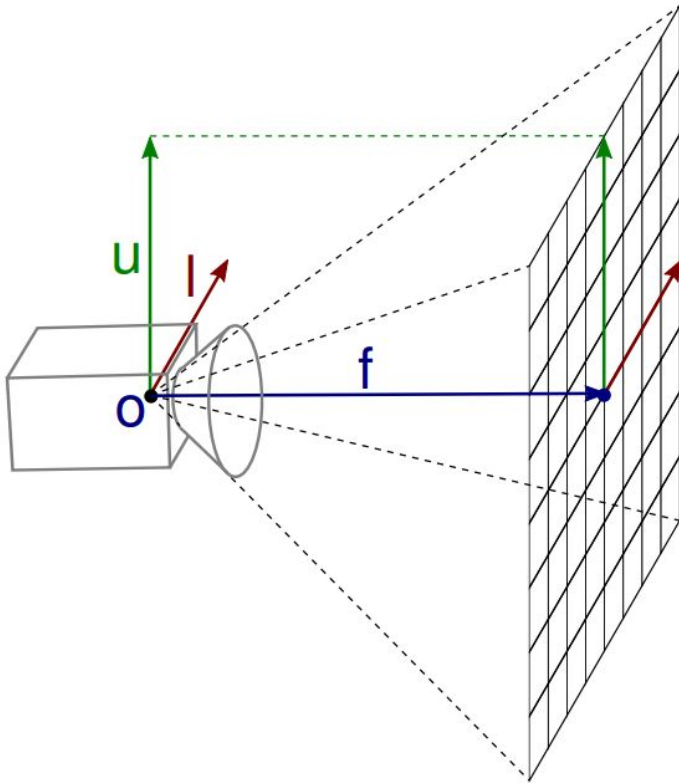
- Diffuse

Right sphere $c = (0.5, -0.7, -0.25)$, $r = 0.3$

- Diffuse

Example scene: Cornell Box

- Camera & light sources



Camera and image plane defined by

Origin $O = (0, 0, -3.5)$

Left $L = (-1, 0, 0)$

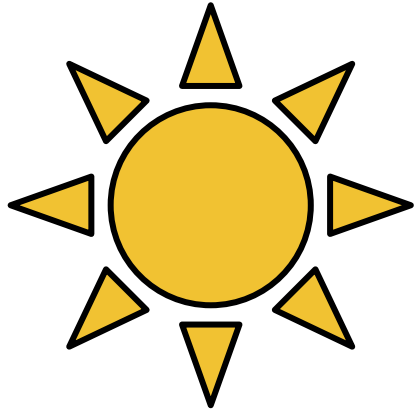
Up $U = (0, 1, 0)$

Forward $F = (0, 0, 3)$

Size 256x256 pixels

Example scene: Cornell Box

- Light sources



Center and power (emission)

Center $c = (0, 0.5, 0)$

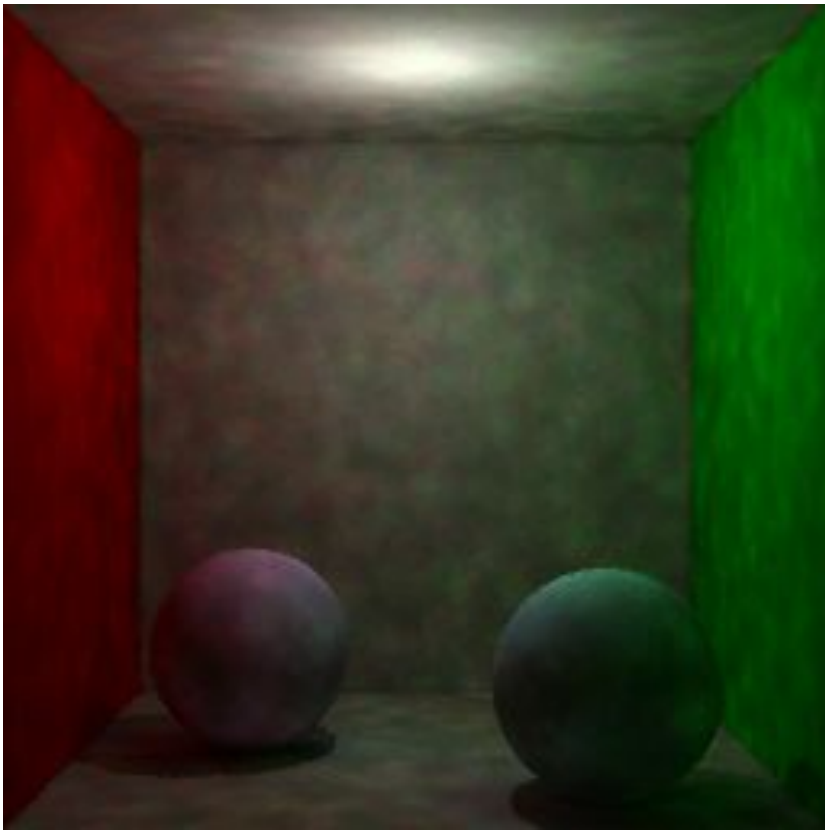
Power can be any number e.g. $p = (1, 1, 1)$

Just be careful with the #MAX

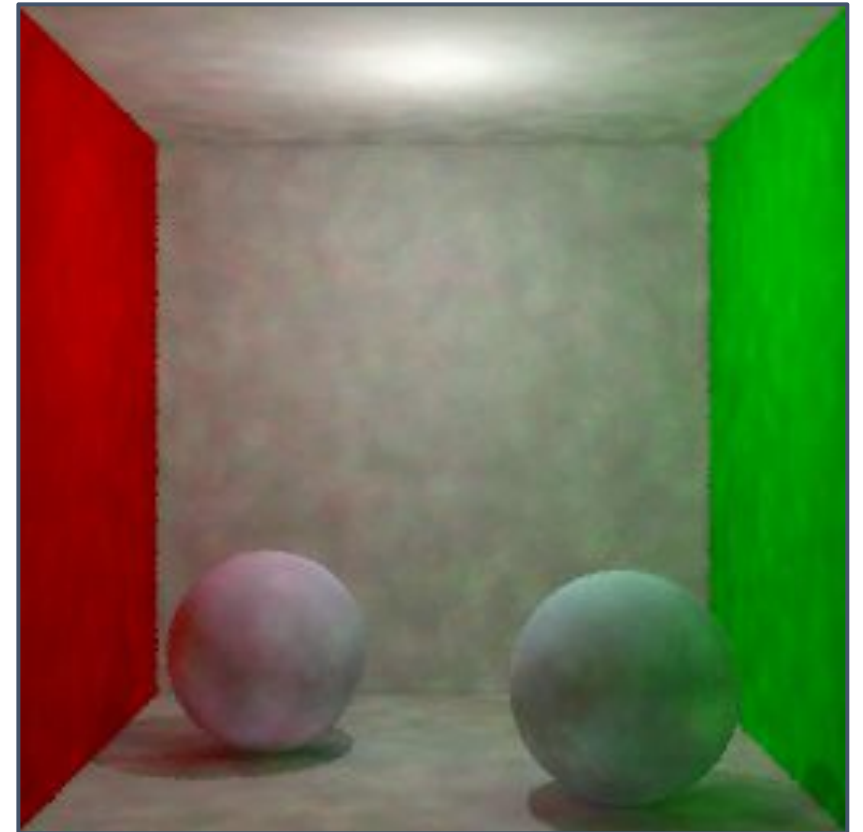
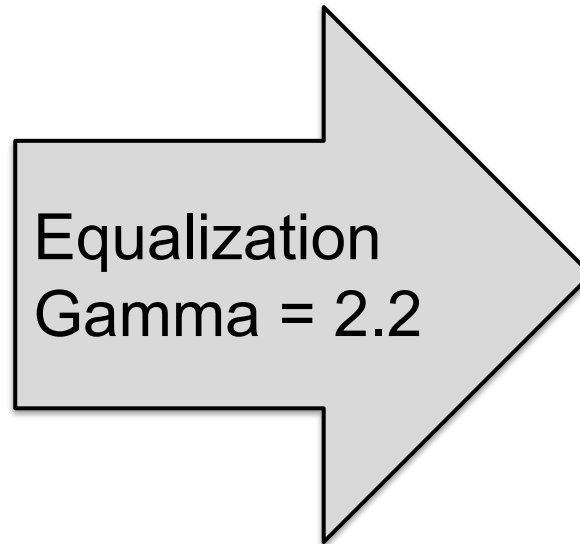
```
1  P3
2  # feep.ppm
3  #MAX=<maximum of your RGB memory values>
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

Example scene: Cornell Box

- Results (direct light is computed using next-event estimation)



Using a point light



With tone mapping

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice implement:

- Generate an image from your photon map:
 - Launch rays from the camera, intersect at point $\mathbf{x}$, find nearest photons
 - Use constant density estimation (box kernel) to estimate the Render Equation
 - Careful with direct light calculation!
- Recommended deadline: November 27th (moodle: January 11th)
 - Extensions (do not count towards recommended deadline):
 - **Recommended to finish base photon mapper before any optionals**
 - Try **different kernels** (cone, Gaussian, or more sophisticated ones)
 - Use an **adaptive kernel bandwidth** (radius)
 - Others: participating media, transient photon mapping, etc. (talk with us before)

Lab #5 – Photon mapping (part 3)

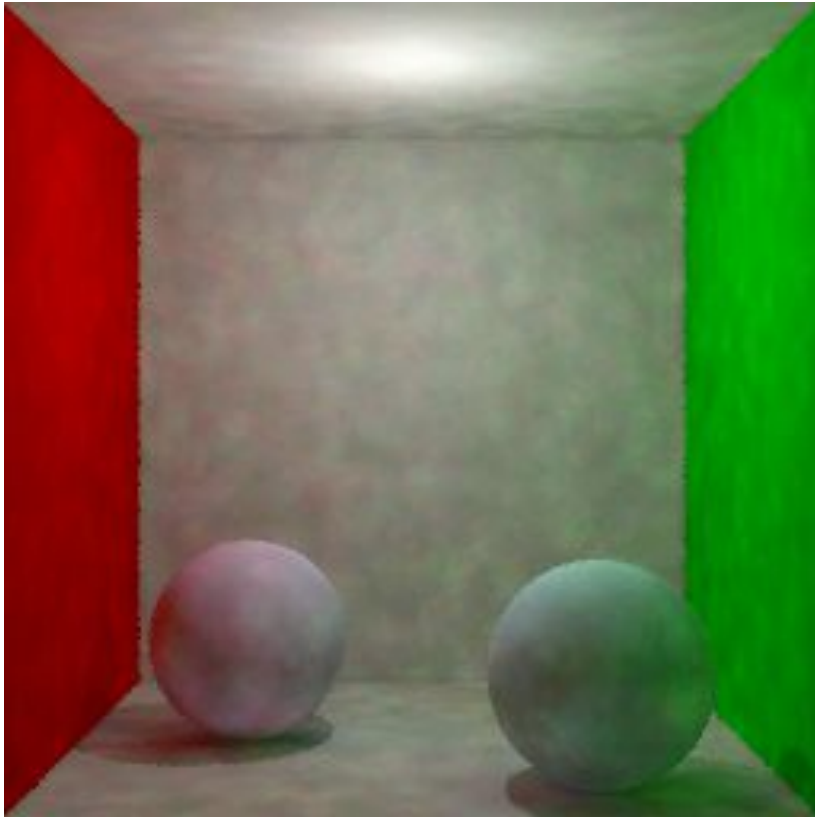
Informática Gráfica

Adolfo Muñoz - Julio Marco
Pablo Luesia - J. Daniel Subías – Óscar Pueyo

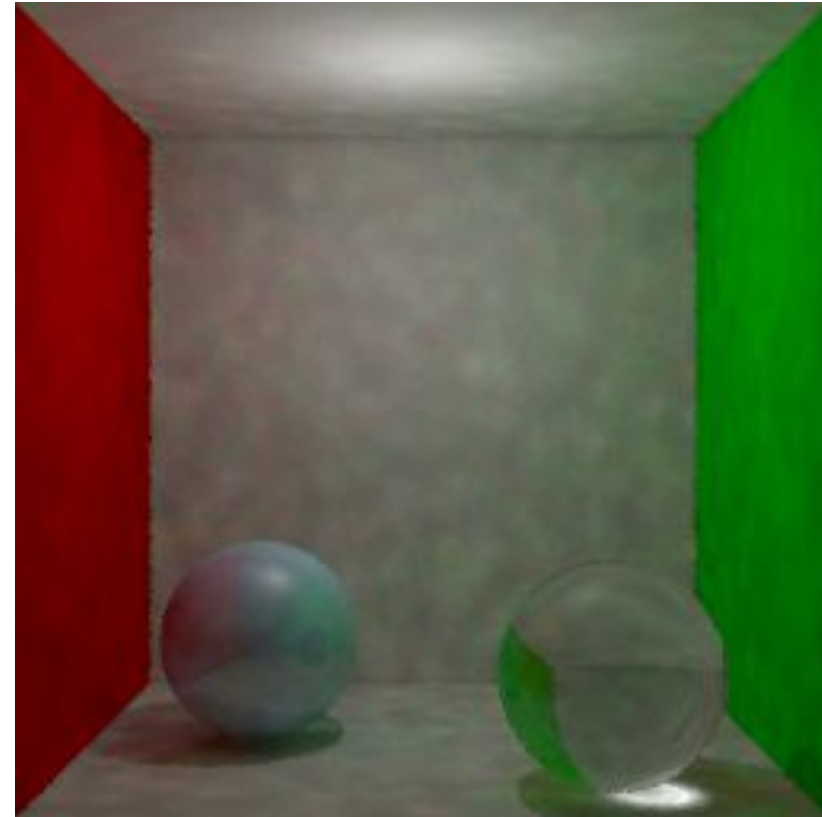


Before we begin...

- Today: add specular and refractive materials to your photon mapper



Only diffuse



+ specular and refraction

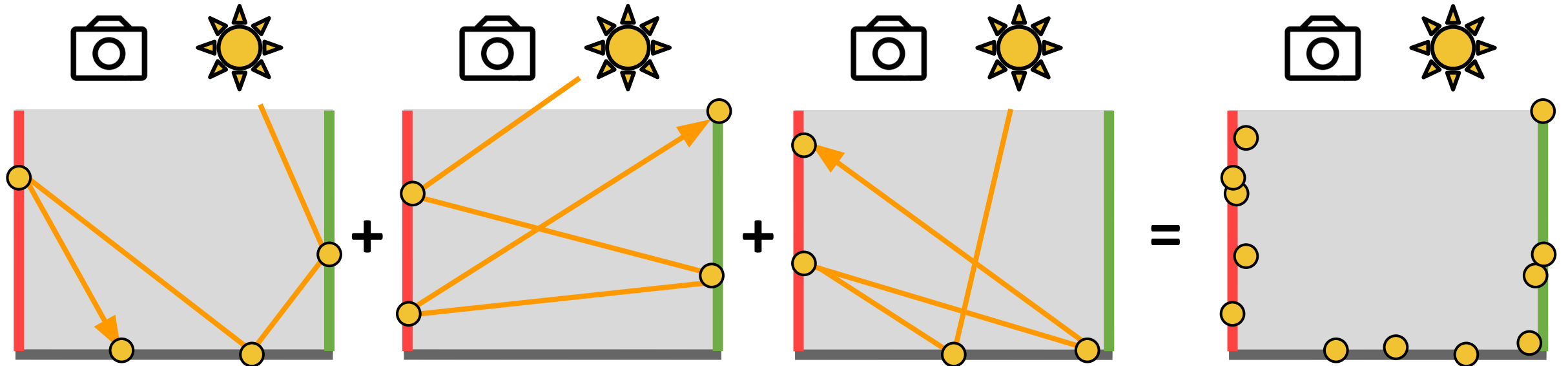
Before we begin...

- Lab 5 (photon mapping) **is the second submitted work**
 - Recommended deadline: December 4th
 - Moodle: January 11th
- You can probably **reuse most of your code** for this assignment
- Remember: Final work is 80% of the final grade

Recap: photon map construction

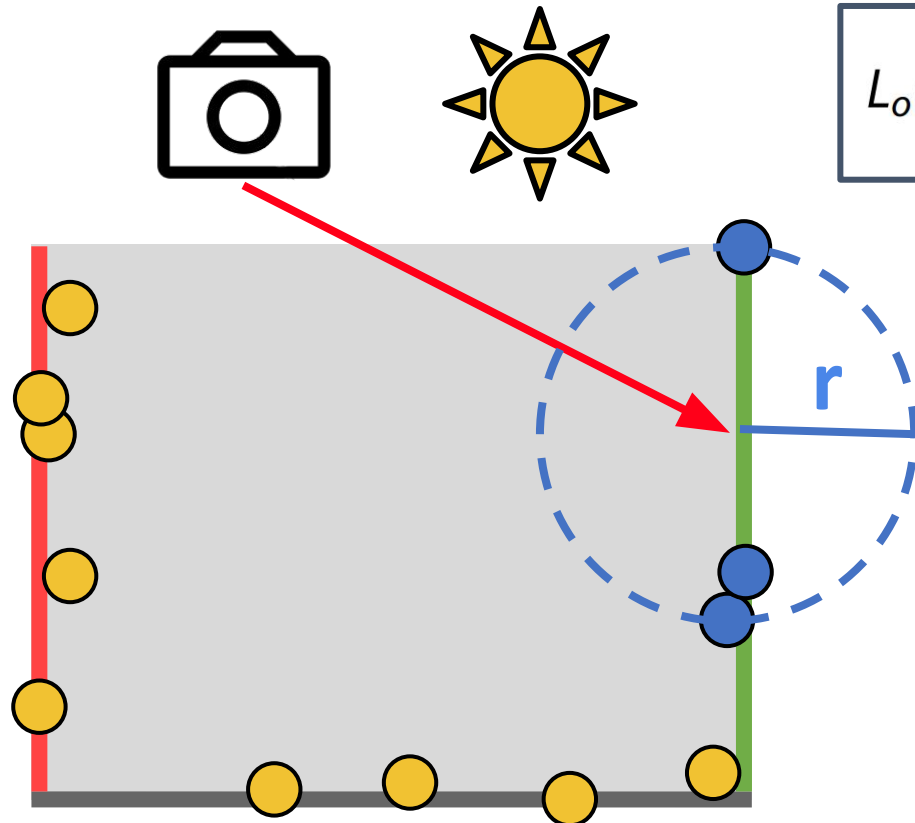
- Split a path into two:
 - (1) lights write into the photon map 
 - (2) camera reads from the photon map
- Photons are sent out from the light sources (**photon random walks**)

Final photon map



Recap: how the photon map is used

- Split a path into two:
 - (1) lights write into the photon map
 - (2) camera reads from the photon map 



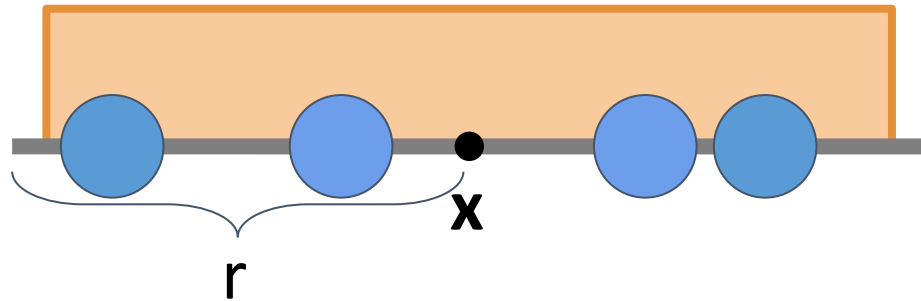
$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} L_i(\mathbf{x}, \omega_i) f_r(\mathbf{x}, \omega_i, \omega_o) |\mathbf{n} \cdot \omega_i| d\omega_i$$

- Idea: use **k nearby** photons (distance $< r$) to approximate the Render Equation
- The result is the **density estimation** of the three photons

Recap: photon density estimation

Constant density estimation

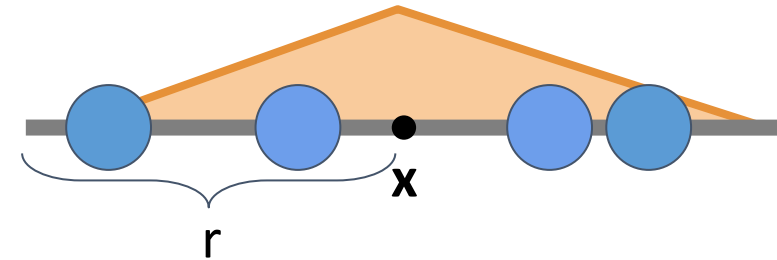
Box kernel



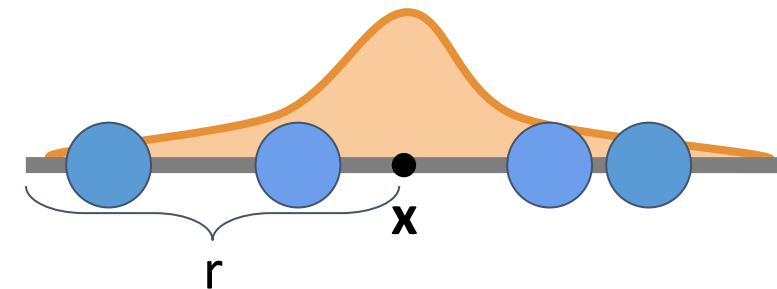
$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$

Non-constant density estimation

Cone kernel



Gaussian kernel

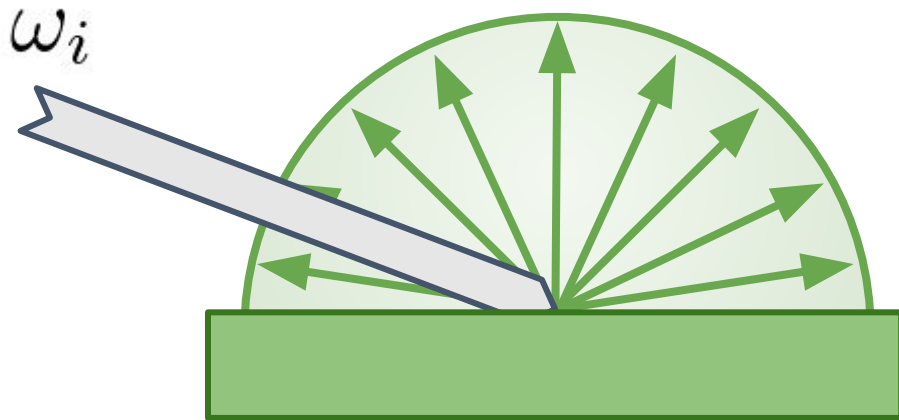


$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \Phi_p K_{2D}(|\mathbf{x} - \mathbf{x}_p|, r_k)$$

Recap: Diffuse and specular BRDFs

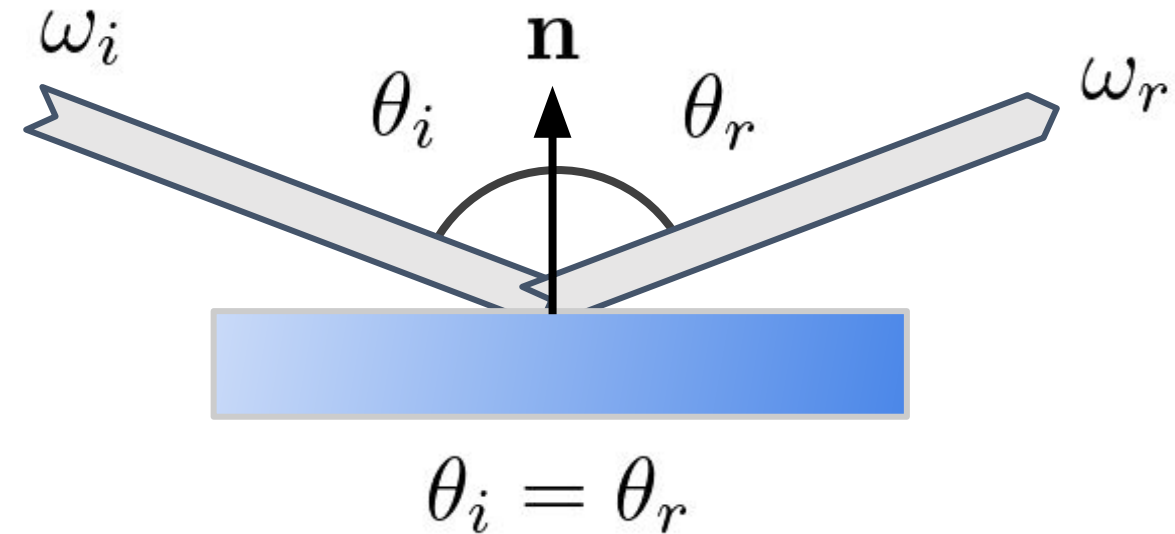
Diffuse material

Light is reflected in all directions equally

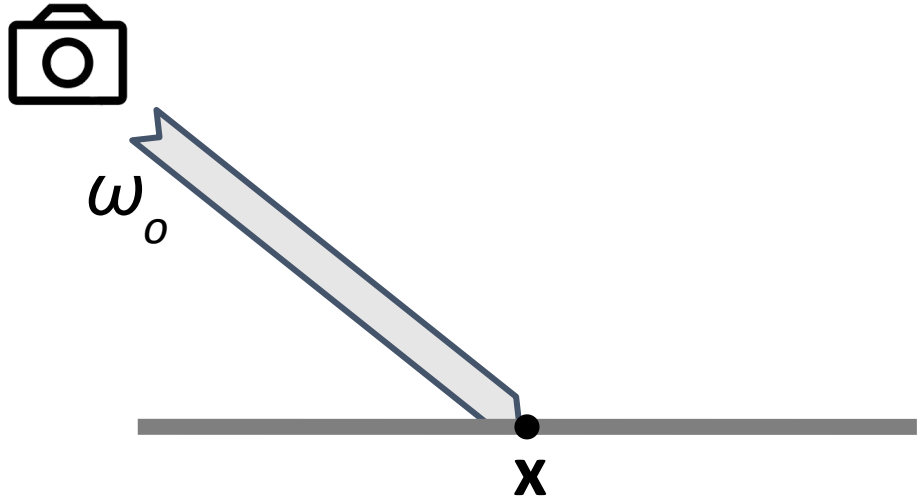


Perfect specular material

All light is (perfectly) reflected towards one direction ω_r

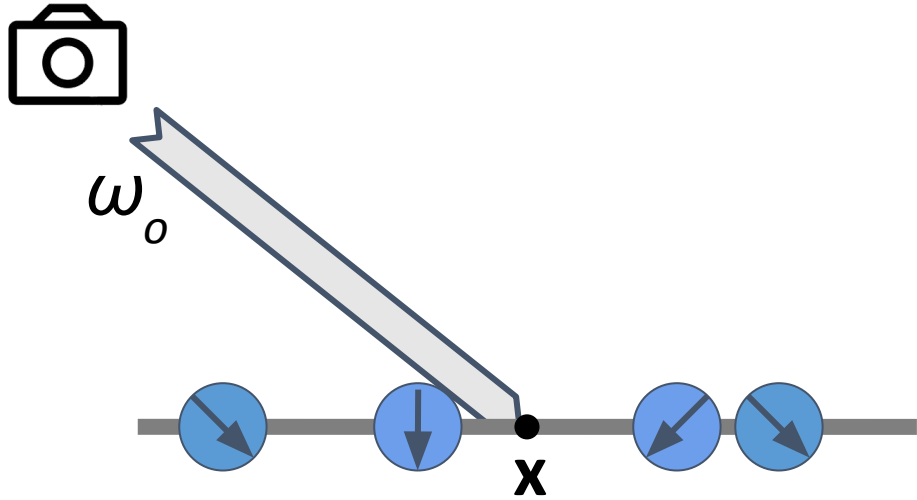


Only store photons in diffuse surfaces



A ray with direction ω_o intersects in $\mathbf{x}$

Only store photons in diffuse surfaces

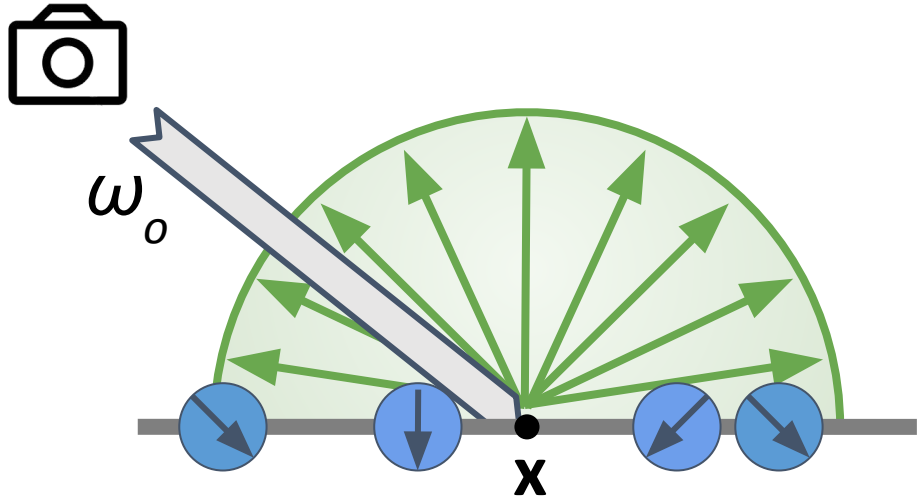


A ray with direction ω_o intersects in x

We obtain a set of near photons, each with:

- Φ_p flux
- ω_p incoming direction
- x_p position of the photon

Only store photons in diffuse surfaces



Diffuse material

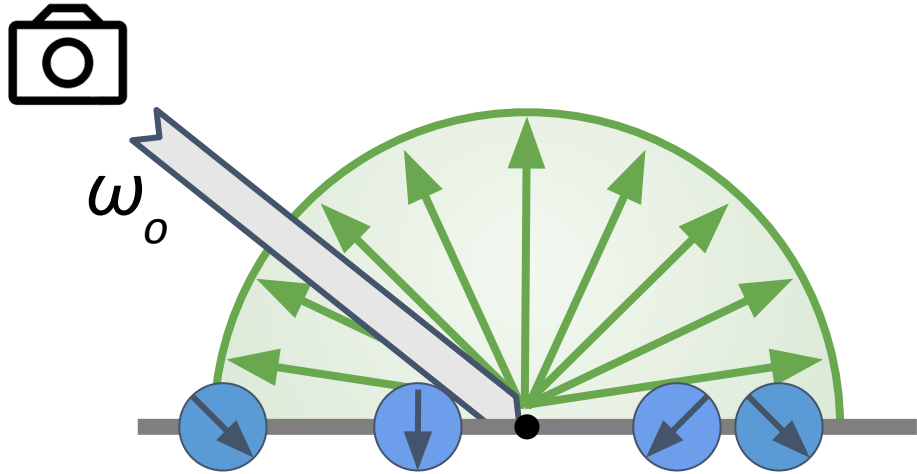
$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$

A ray with direction ω_o intersects in $\mathbf{x}$

We obtain a set of near photons, each with:

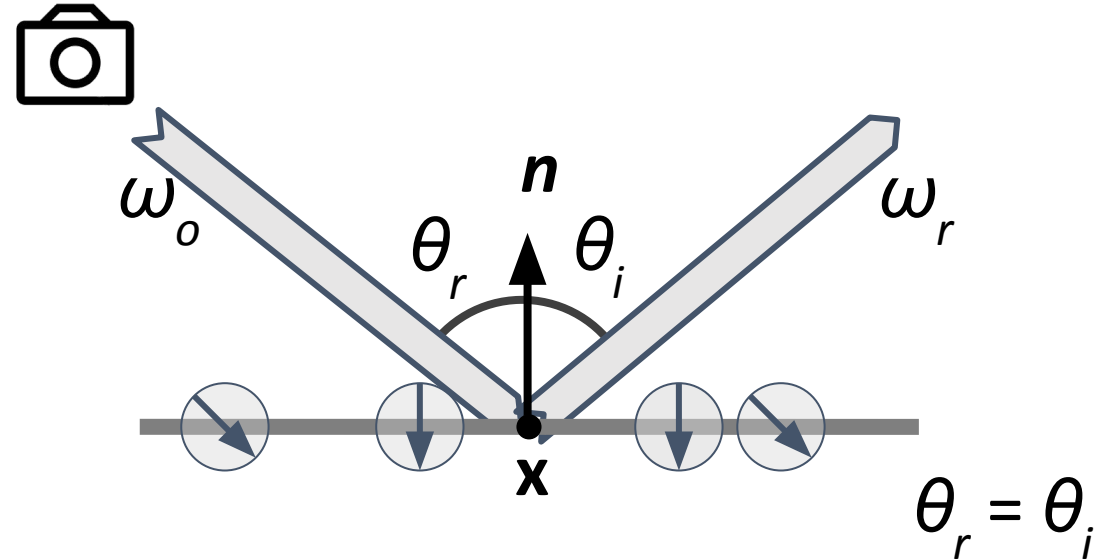
- Φ_p flux
- ω_p incoming direction
- $\mathbf{x}_p$ position of the photon

Only store photons in diffuse surfaces



Diffuse material

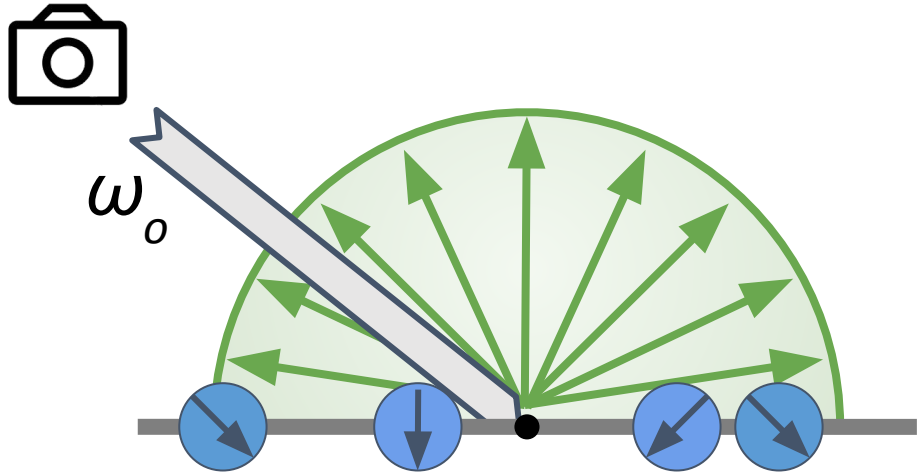
$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$



Specular material

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k f_r(\mathbf{x}, \omega_p, \omega_o) \frac{\Phi_p}{\pi r_k^2}$$

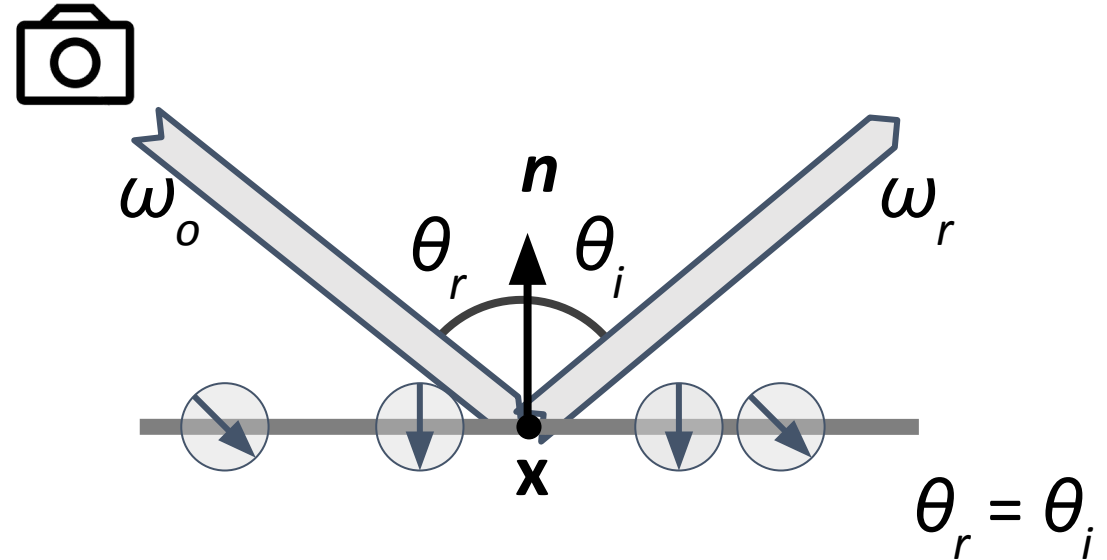
Only store photons in diffuse surfaces



Diffuse material

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k \boxed{f_r(\mathbf{x}, \omega_p, \omega_o)} \frac{\Phi_p}{\pi r_k^2}$$

$$f_r = Kd / \pi \rightarrow L_o > 0$$

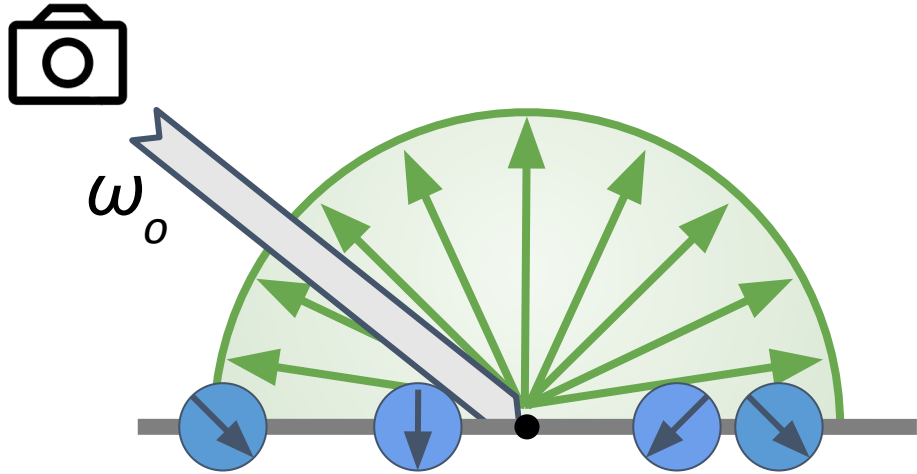


Specular material

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k \boxed{f_r(\mathbf{x}, \omega_p, \omega_o)} \frac{\Phi_p}{\pi r_k^2}$$

$$p(\omega_p = -\omega_r) = 0 \rightarrow f_r = 0 \rightarrow L_o = 0$$

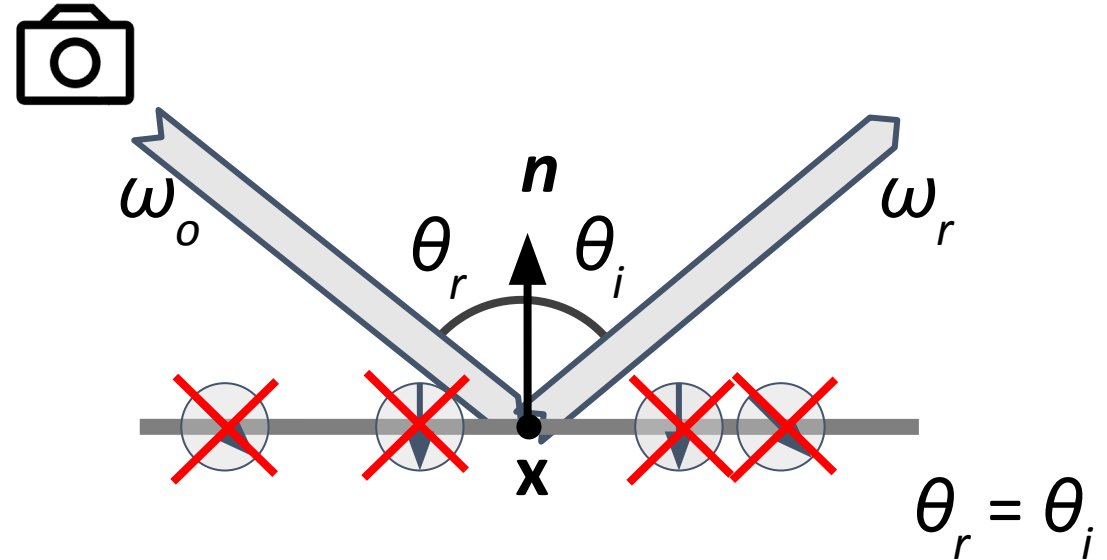
Only store photons in diffuse surfaces



Diffuse material

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k \boxed{f_r(\mathbf{x}, \omega_p, \omega_o)} \frac{\Phi_p}{\pi r_k^2}$$

Store photons when intersecting with diffuse surfaces

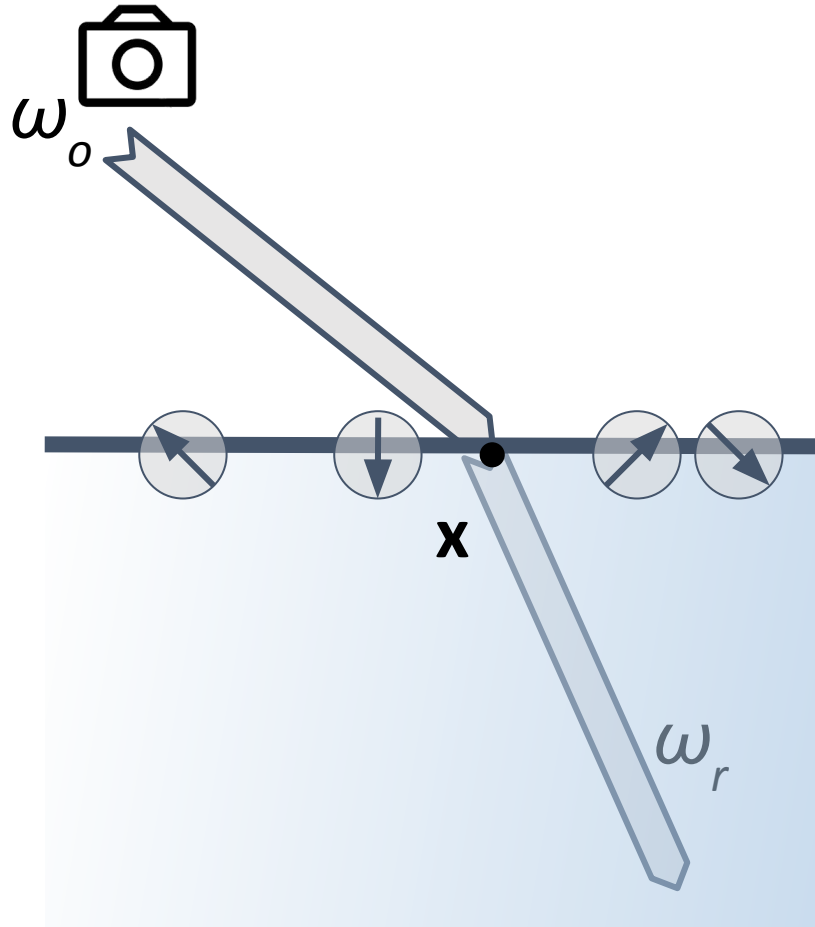


Specular material

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{p=1}^k \boxed{f_r(\mathbf{x}, \omega_p, \omega_o)} \frac{\Phi_p}{\pi r_k^2}$$

Do NOT store photons when intersecting with specular surfaces

Only store photons in diffuse surfaces

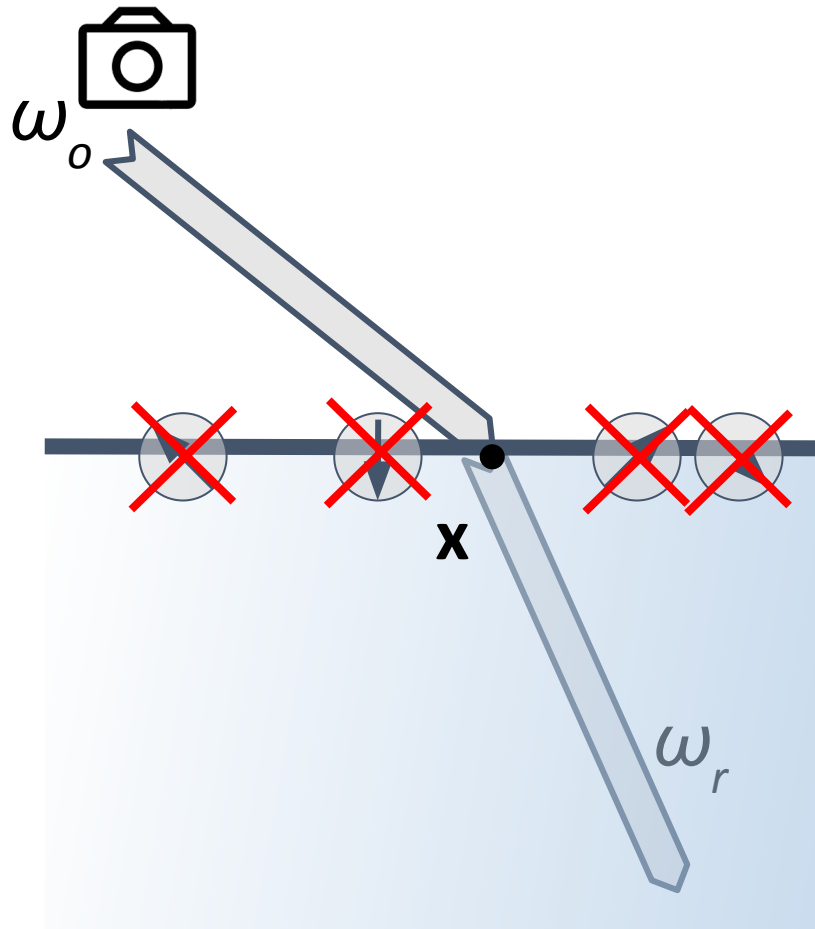


Refractive material

Same happens with refraction

$$p(\omega_p = -\omega_r) = 0 \rightarrow f_r = 0 \rightarrow L_o = 0$$

Only store photons in diffuse surfaces



Refractive material

Same happens with refraction

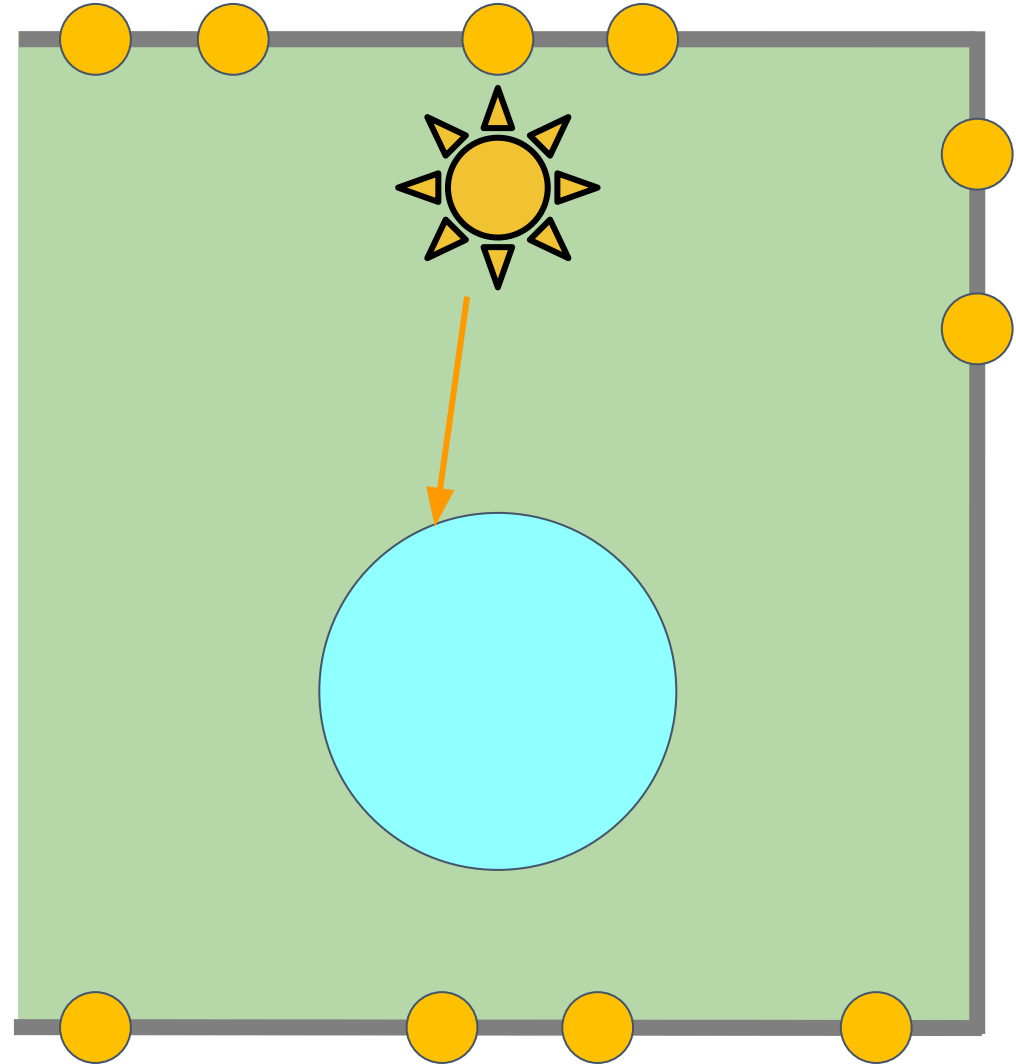
$$p(\omega_p = -\omega_r) = 0 \rightarrow f_r = 0 \rightarrow L_o = 0$$

**Do NOT store photons when
intersecting with refractive surfaces**

What to code in today's session

When a **delta BSDF** is hit:

1. Photon map generation:
continue the photon walk
without storing a photon



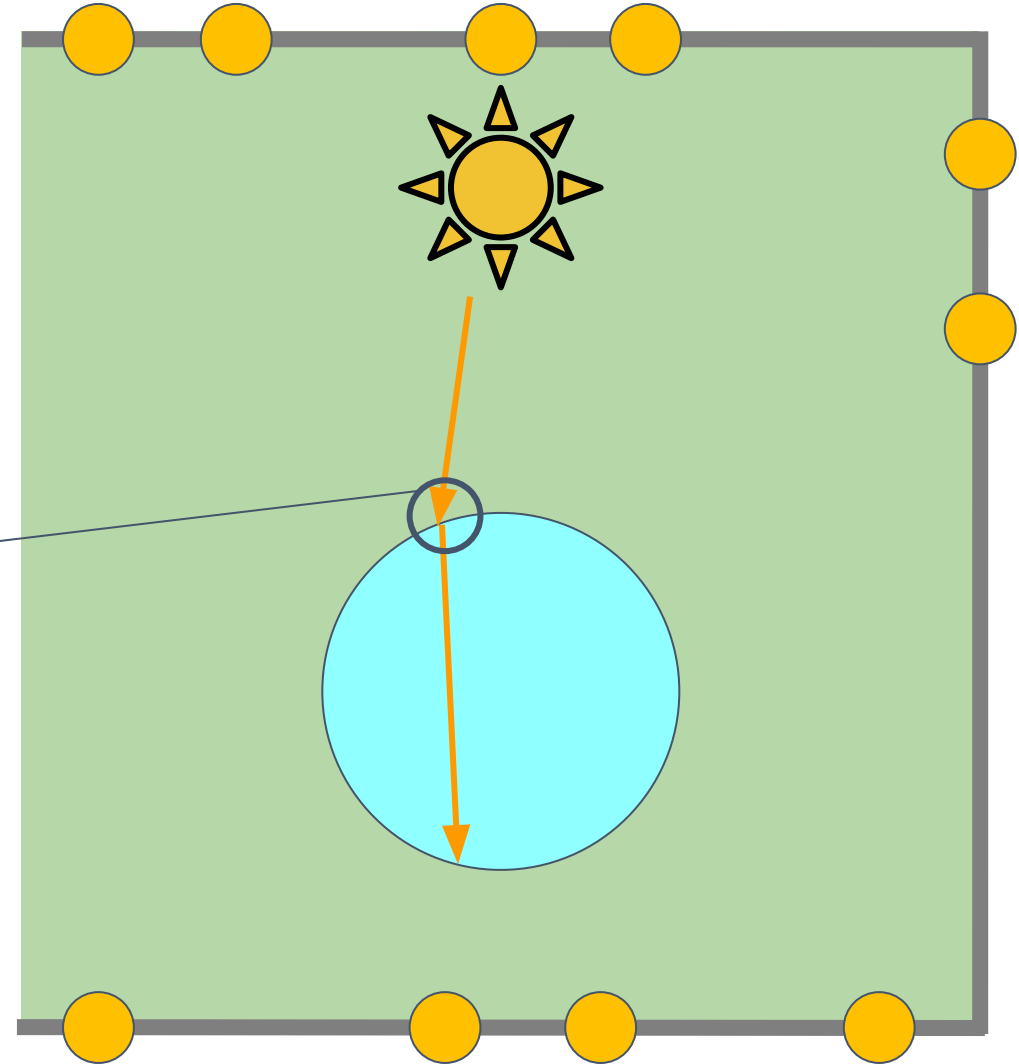
What to code in today's session

When a **delta BSDF** is hit:

1. Photon map generation:
continue the photon walk
without storing a photon



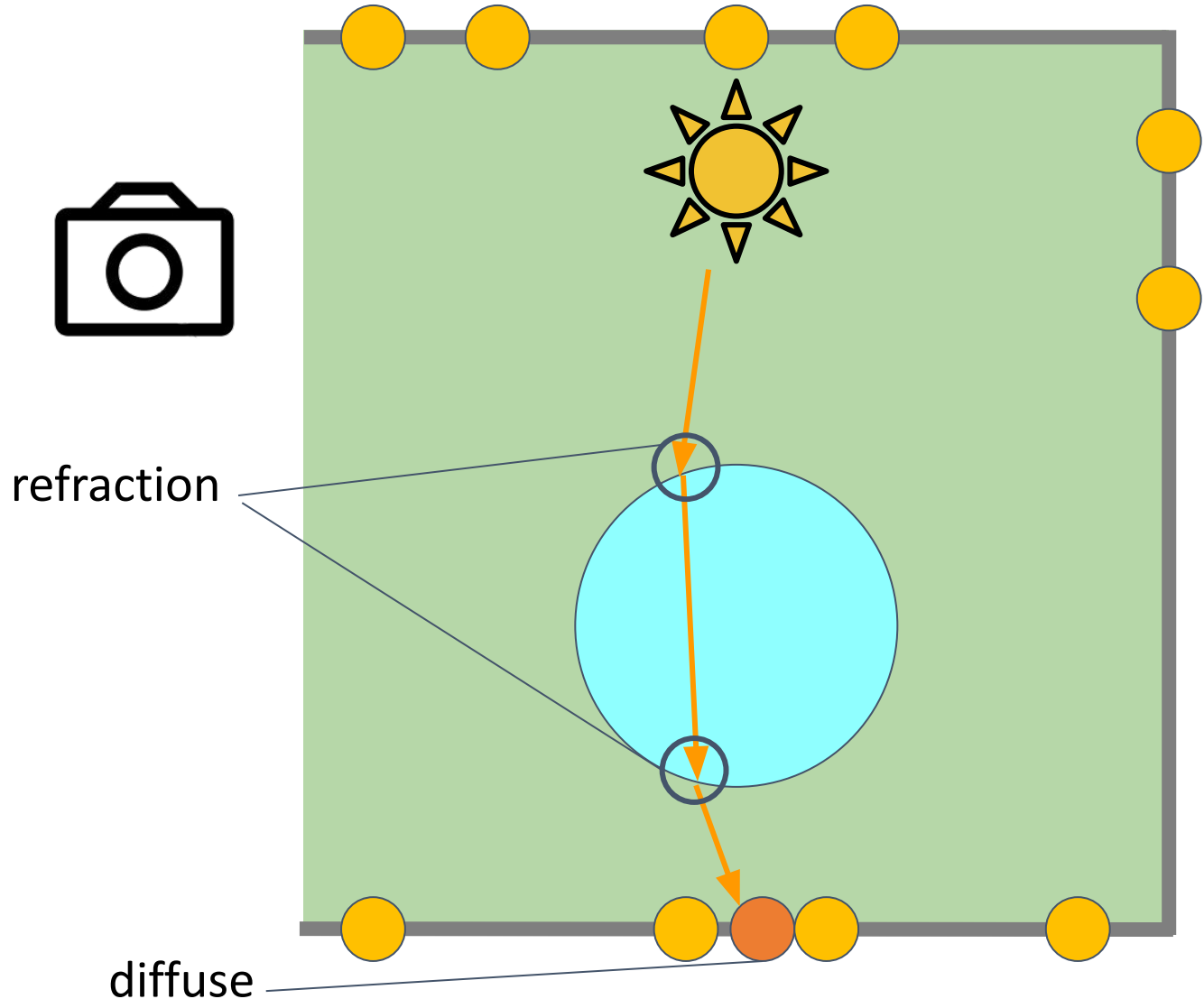
refraction



What to code in today's session

When a **delta BSDF** is hit:

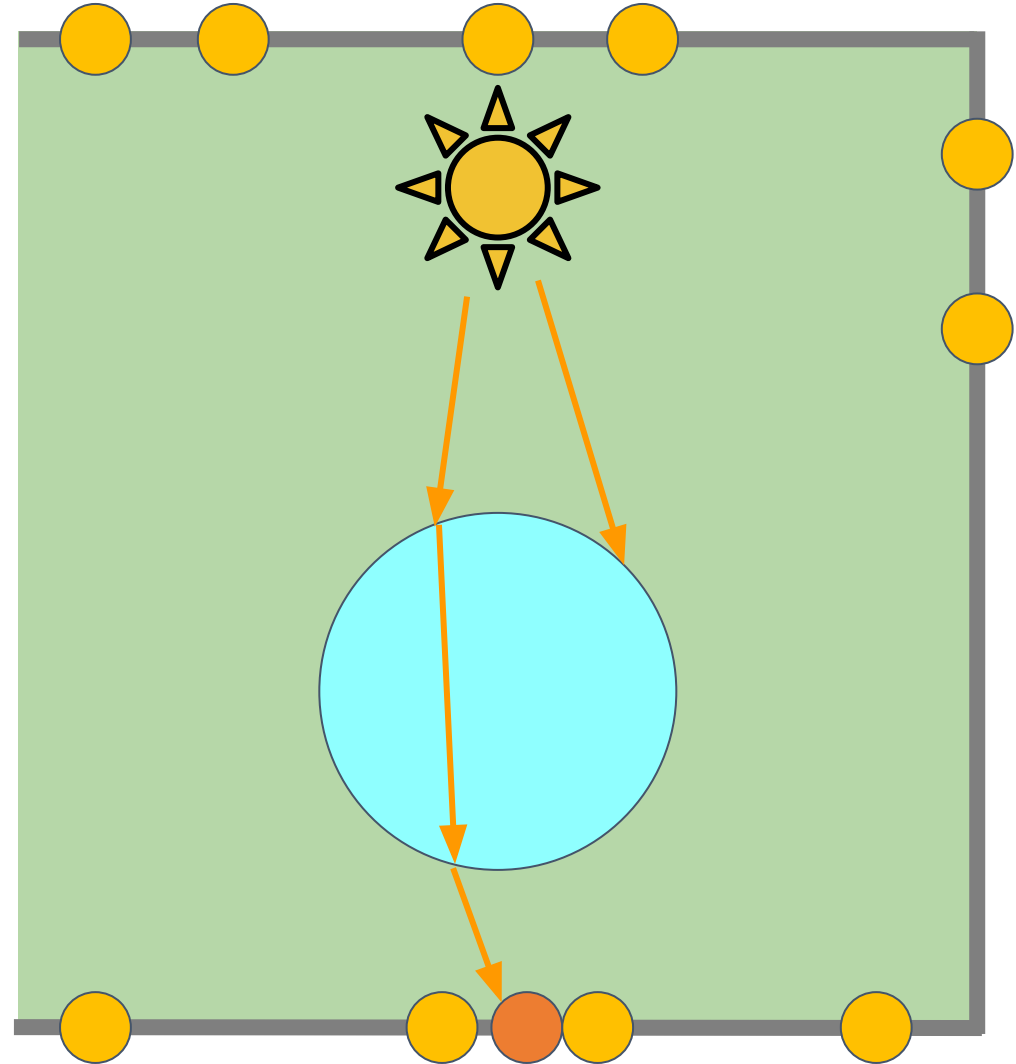
1. Photon map generation:
continue the photon walk
without storing a photon



What to code in today's session

When a **delta BSDF** is hit:

1. Photon map generation:
continue the photon walk
without storing a photon



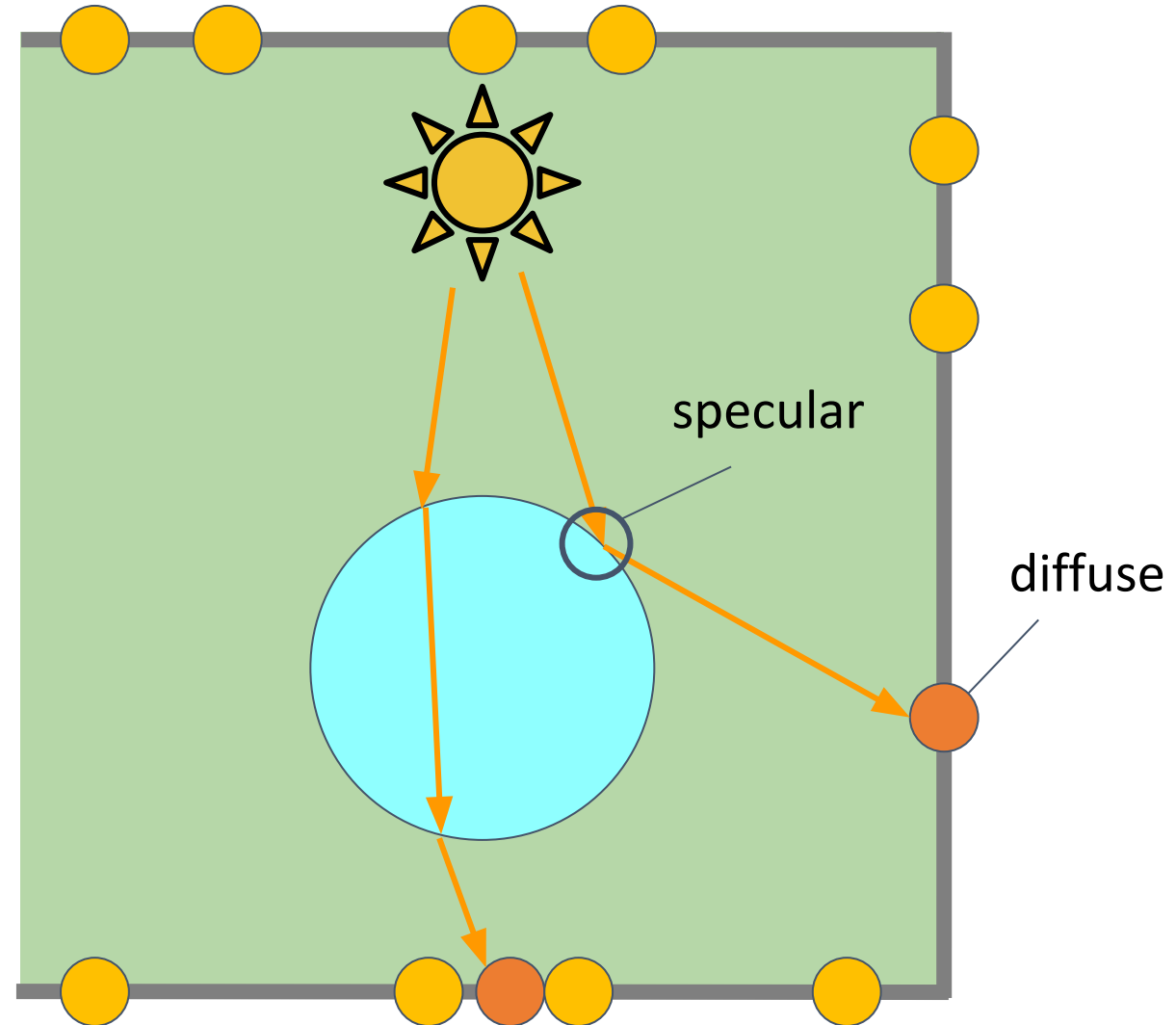
What to code in today's session

When a **delta BSDF** is hit:

1. Photon map generation:
continue the photon walk
without storing a photon



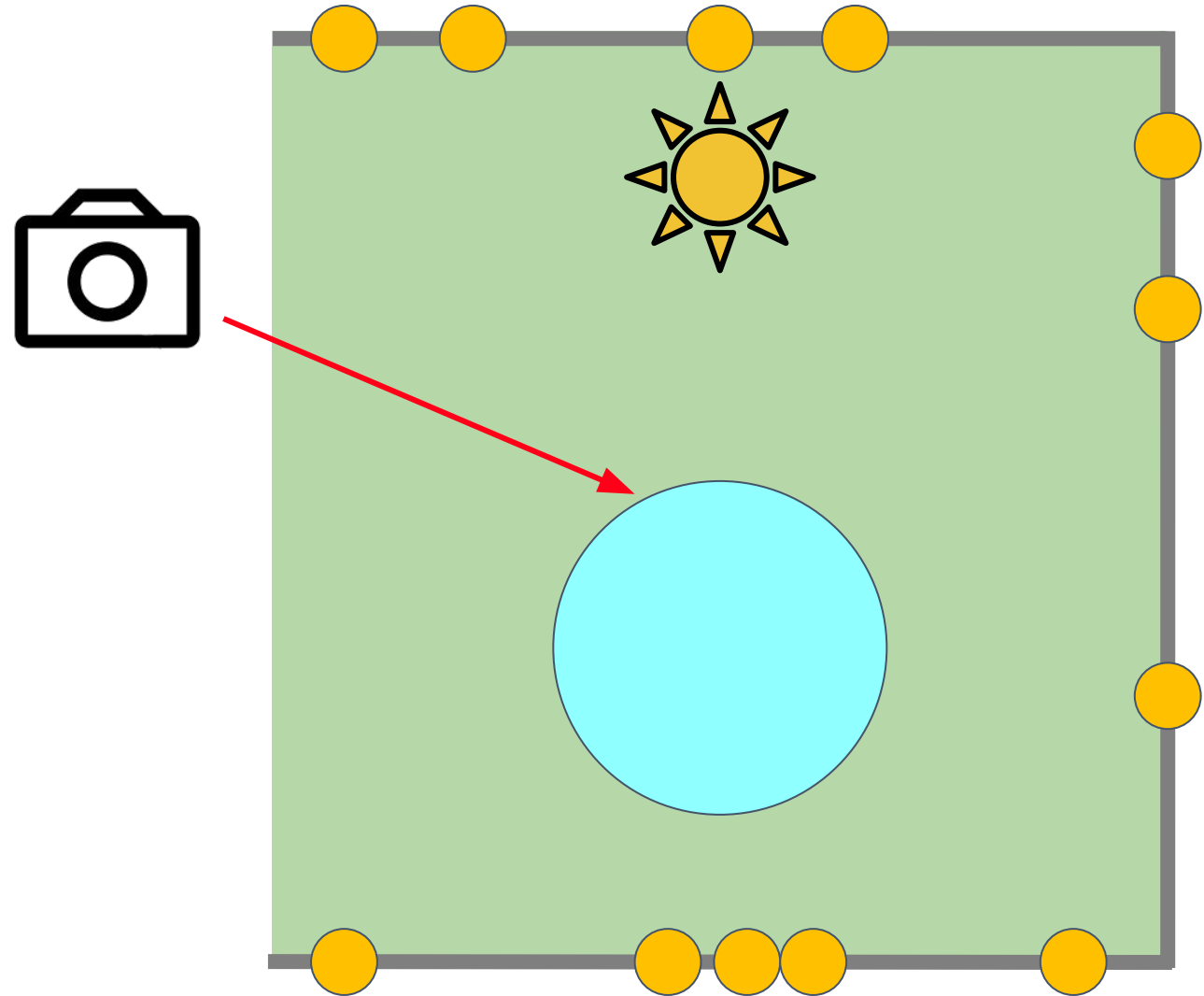
Diffuse, specular and refraction
sampling is already implemented
(see previous assignments)



What to code in today's session

When a **delta BSDF** is hit:

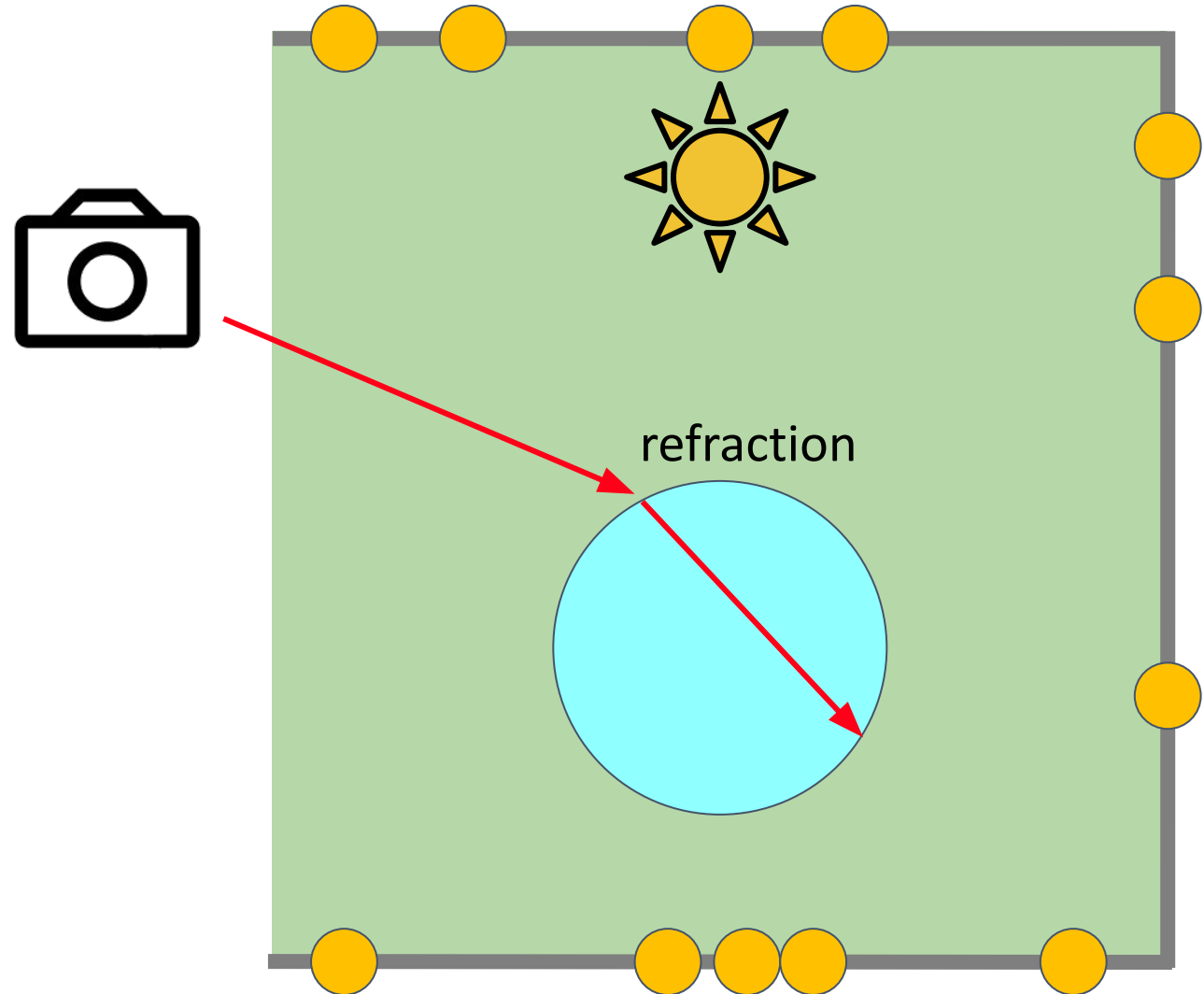
1. Photon map generation:
continue the photon walk
without storing a photon
2. Tracing rays from the
camera: follow the perfect
specular/refraction
directions **instead of
estimating radiance**



What to code in today's session

When a **delta BSDF** is hit:

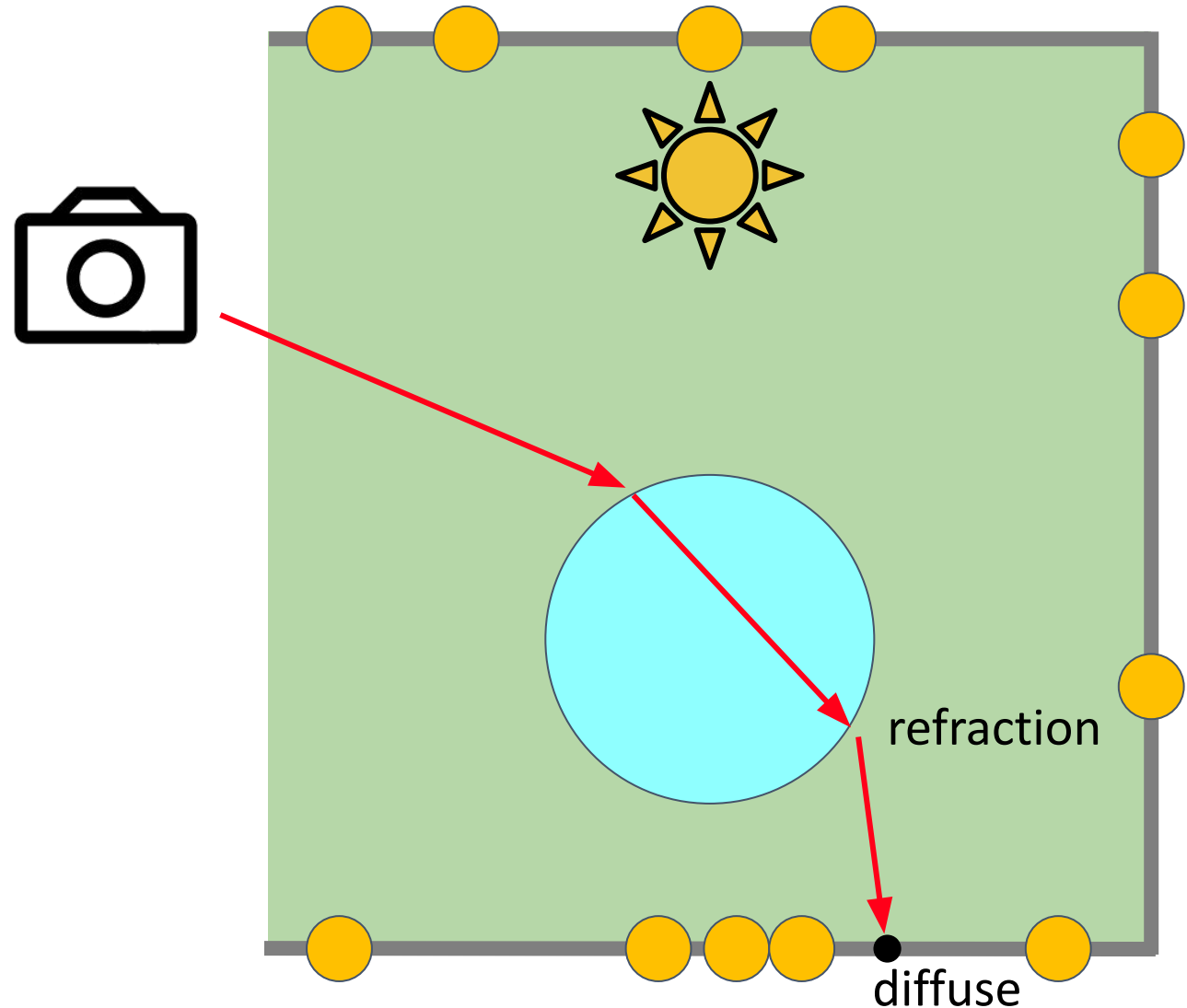
1. Photon map generation:
continue the photon walk
without storing a photon
2. Tracing rays from the camera:
follow the perfect
specular/refraction
directions **instead of
estimating radiance**



What to code in today's session

When a **delta BSDF** is hit:

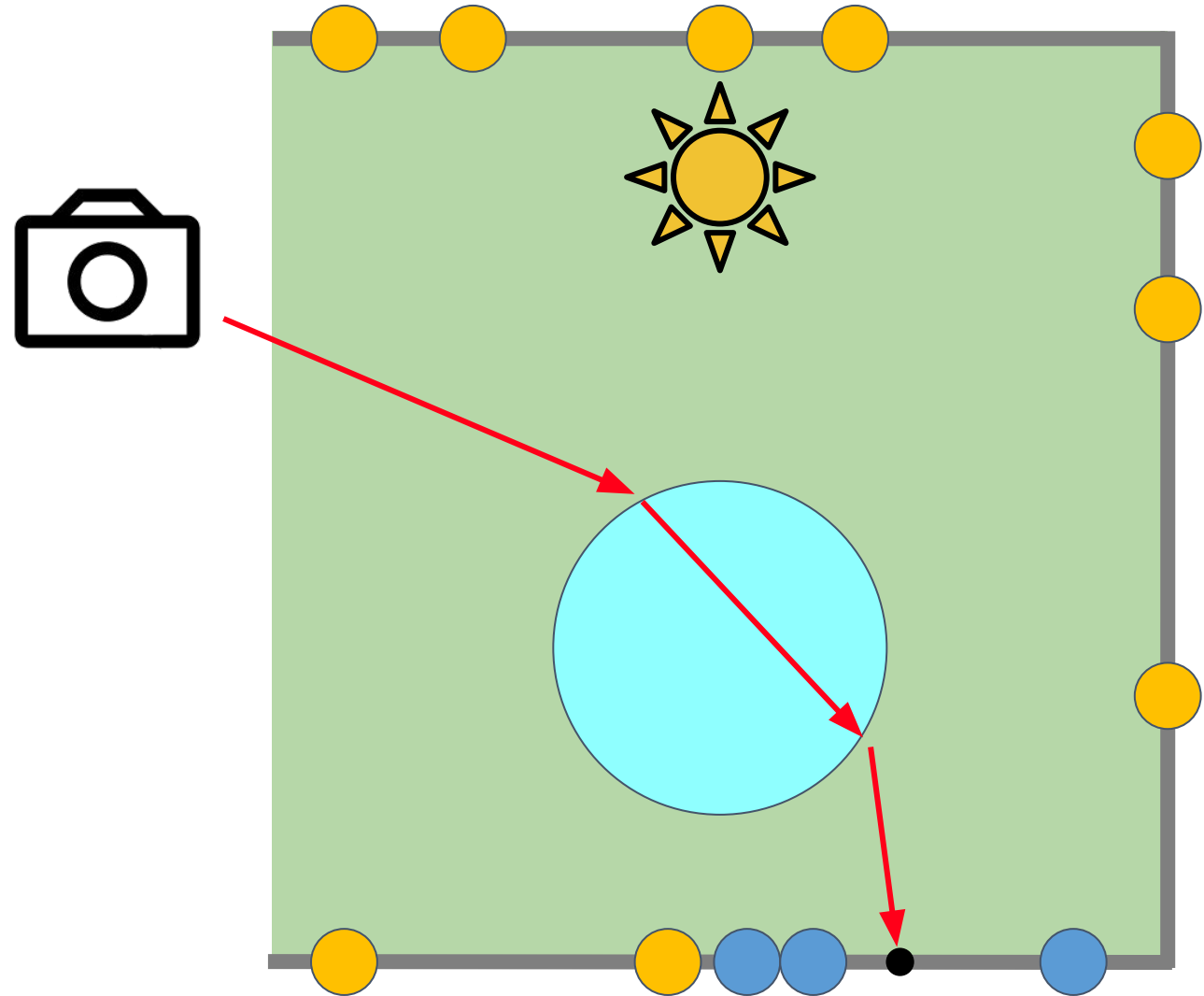
1. Photon map generation: continue the photon walk **without storing a photon**
2. Tracing rays from the camera: follow the perfect specular/refraction directions **instead of estimating radiance**



What to code in today's session

When a **delta BSDF** is hit:

1. Photon map generation: continue the photon walk **without storing a photon**
2. Tracing rays from the camera: follow the perfect specular/refraction directions **instead of estimating radiance**



Russian roulette and materials

- Recap: you can combine coefficients to get different materials

Diffuse



$$k_d > 0$$

Specular



$$k_s > 0$$

Plastic



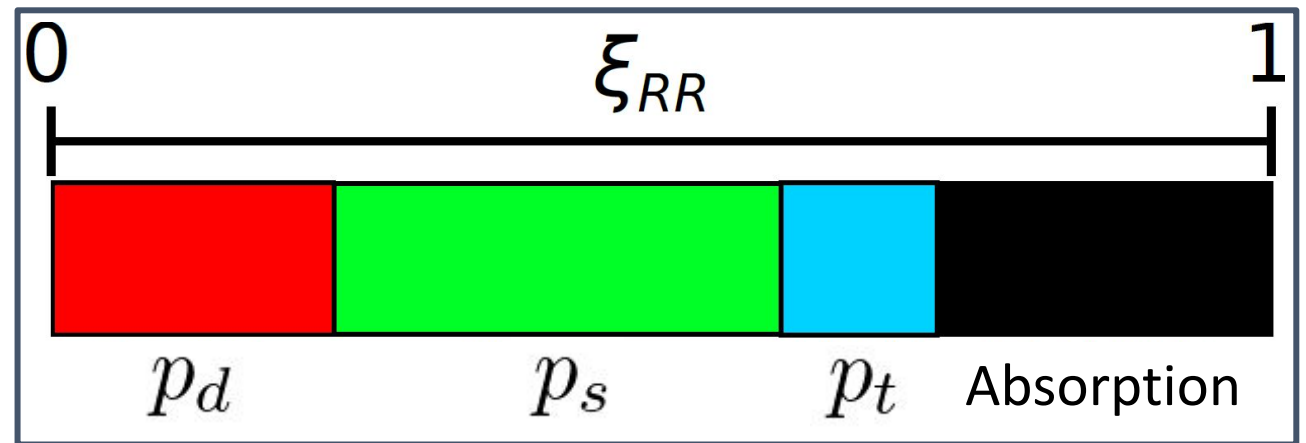
$$k_d, k_s > 0$$

Russian roulette and materials

Solution:

- **Russian Roulette:** sample one random event (diffuse, specular, refraction or absorption)

Random number $\xi_{RR} \in [0, 1]$:



$$f_r(\mathbf{x}, \omega_i, \omega_o) = k_d \frac{1}{\pi} + k_s \frac{\delta_{\omega_r}(\omega_i)}{\mathbf{n} \cdot \omega_i} + k_t \frac{\delta_{\omega_t}(\omega_i)}{\mathbf{n} \cdot \omega_i}$$

Russian Roulette and materials

Implement Russian Roulette in each part of the photon mapping algorithm:

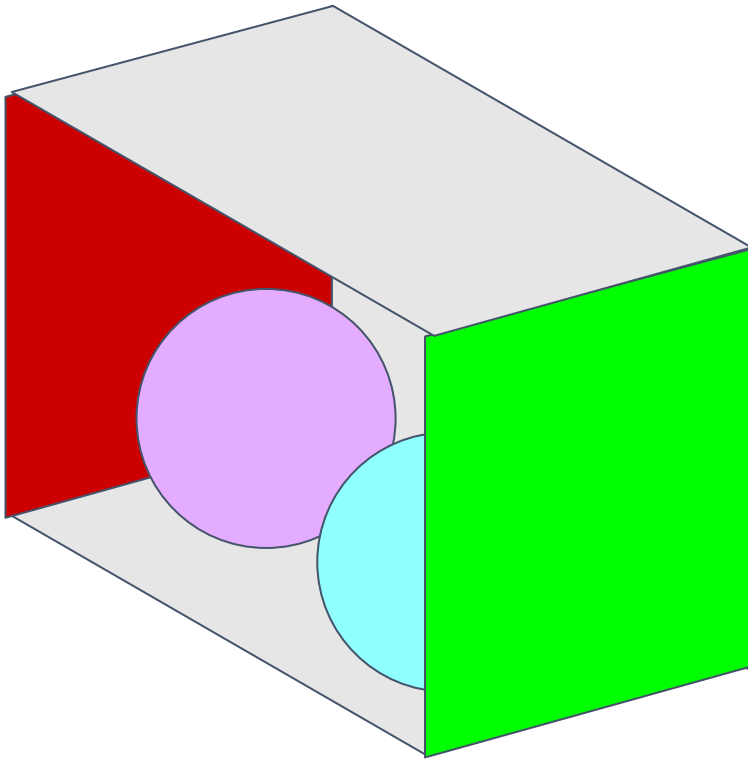
- Photon map generation
- Ray tracing evaluation

Both steps use the same Russian Roulette with four events:

- Diffuse:
 - Store the photon (photon map generation)
 - Use density estimation (tracing rays from the camera)
- Specular/refraction: the path continues in a delta direction instead of what it did before
- Absorption: the path ends (don't store photons/return no radiance)

Example scene: Cornell Box

- **Geometry**



Planes defined by normal (n) and distance (d)

Left plane $n = (1, 0, 0)$, $d = 1$

Right plane $n = (-1, 0, 0)$, $d = 1$

Floor plane $n = (0, 1, 0)$, $d = 1$

Ceiling plane $n = (0, -1, 0)$, $d = 1$

Back plane $n = (0, 0, -1)$, $d = 1$

Spheres defined by center (c) and radius (r)

Left sphere $c = (-0.5, -0.7, 0.25)$, $r = 0.3$

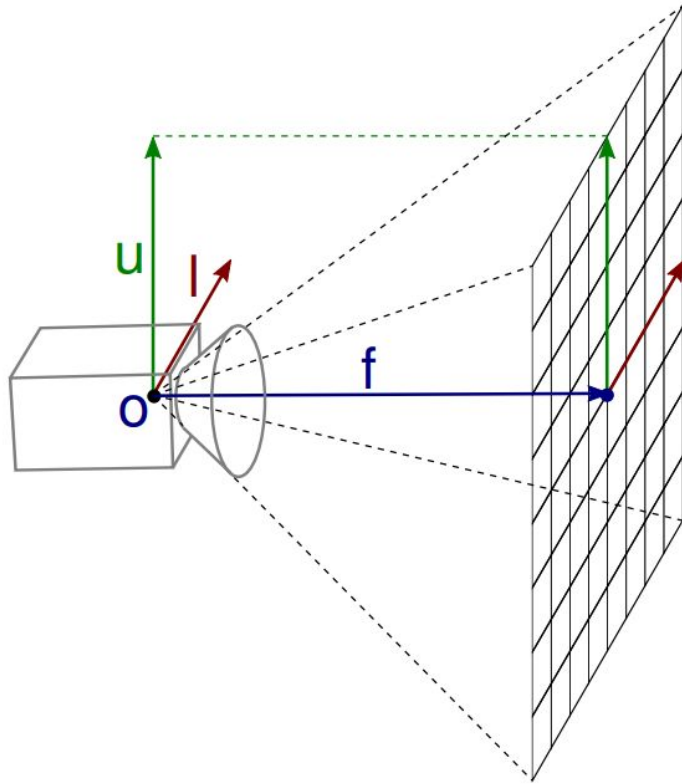
- Mix of blue diffuse + specular

Right sphere $c = (0.5, -0.7, -0.25)$, $r = 0.3$

- Mix of specular and refraction, $\eta = 1.5$

Example scene: Cornell Box

- Camera & light sources



Camera and image plane defined by

Origin $O = (0, 0, -3.5)$

Left $L = (-1, 0, 0)$

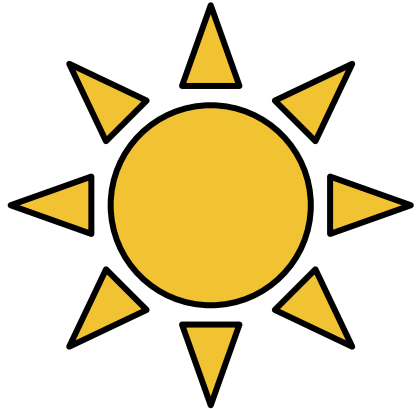
Up $U = (0, 1, 0)$

Forward $F = (0, 0, 3)$

Size 256x256 pixels

Example scene: Cornell Box

- Light sources



Center and power (emission)

Center $c = (0, 0.5, 0)$

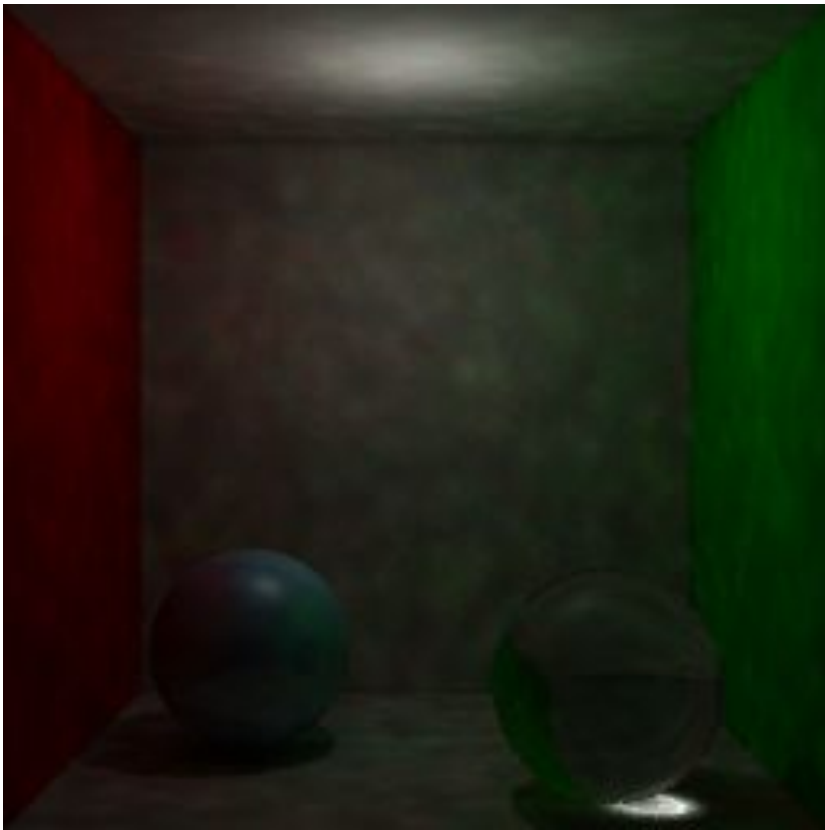
Power can be any number e.g. $p = (1, 1, 1)$

Just be careful with the #MAX

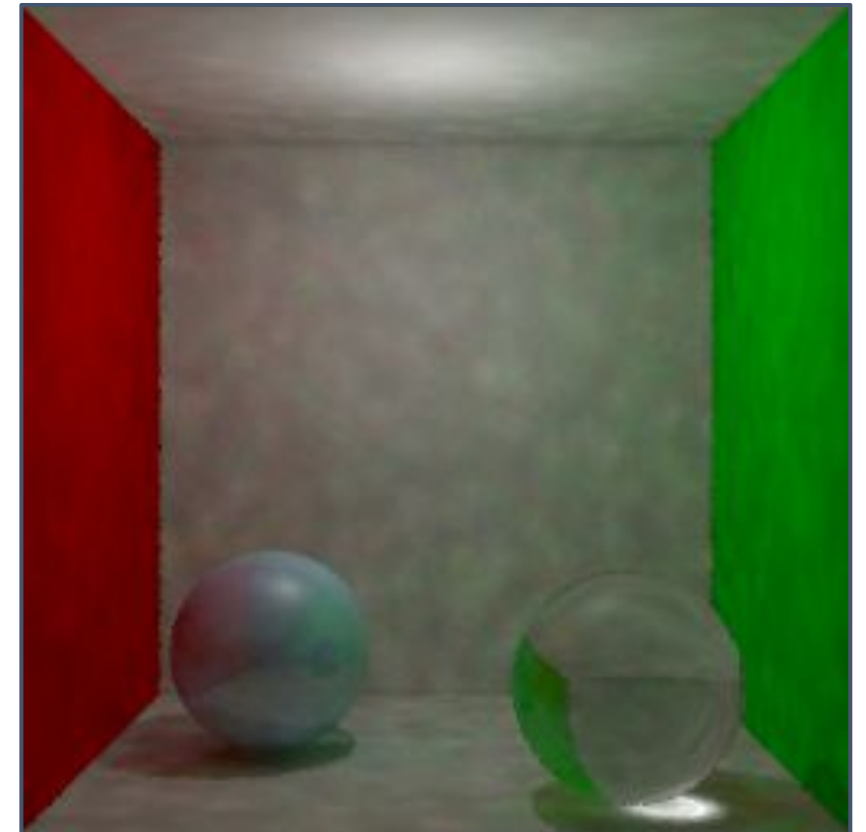
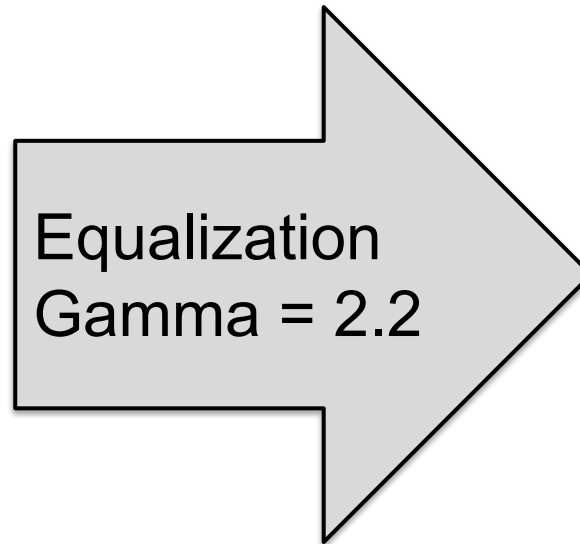
```
1  P3
2  # feep.ppm
3  #MAX=<maximum of your RGB memory values>
4  4 4
5  15
6  0 0 0 0 0 0 0 0 0 0 15 0 15
```

Example scene: Cornell Box

- Results (direct light is computed using next-event estimation)



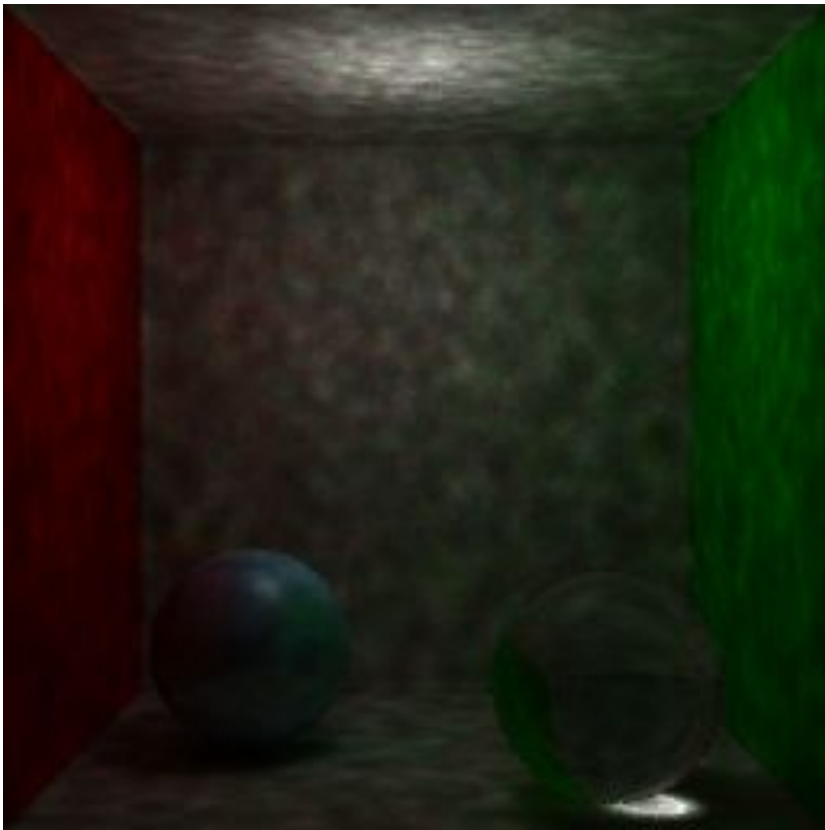
Using a point light



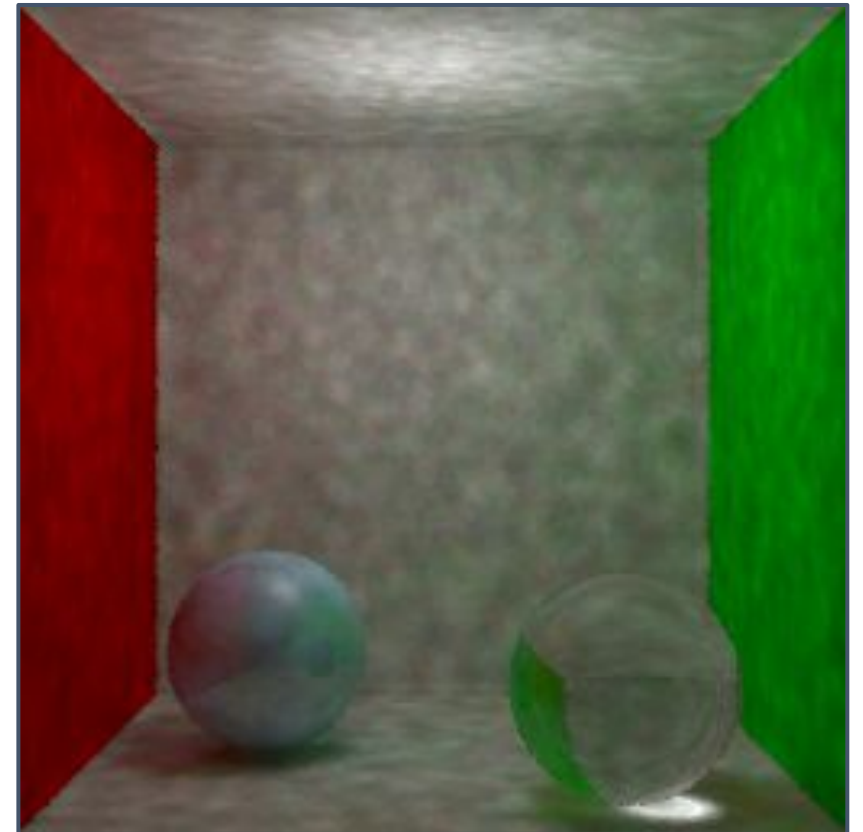
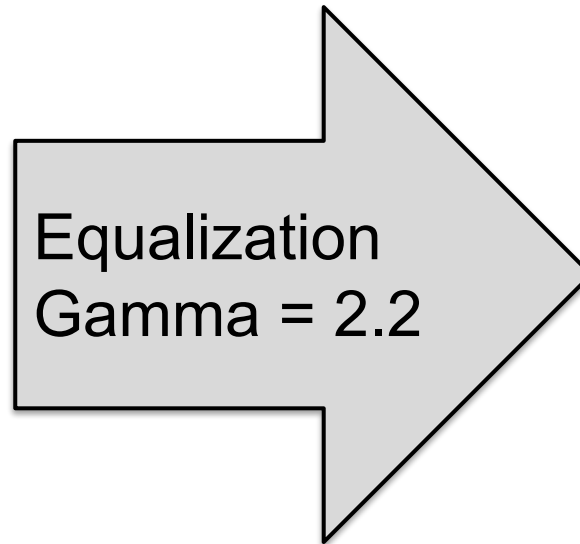
With tone mapping

Example scene: Cornell Box

- Results (direct light is computed using photon mapping)



Using a point light



With tone mapping

DO ASK questions, either now or after the lab

But be reasonable, please :)

pluesia@unizar.es | dsubias@unizar.es | o.pueyo@unizar.es

What to expect from this session

In the programming language of your choice implement:

- Perfect specular and refractive materials in photon mapping:
 - Use Russian Roulette to select an event (same as path tracing)
 - **Diffuse event:** same as previous session
 - **Specular/refraction events:** path continues in a delta direction instead of what it did before
 - **Absorption:** path ends (don't store photons, and return zero radiance)
- Recommended deadline: December 4th (moodle: January 11th)
 - Extensions (do not count towards recommended deadline):
 - **Recommended to finish base photon mapper before any optionals**
 - Try **different kernels** (cone, Gaussian, or more sophisticated ones)
 - Use an **adaptive kernel bandwidth** (radius)
 - Others: participating media, transient photon mapping, etc. (talk with us before)